

RISEDUAL AI

Complete Codebase Documentation

AI-Powered Trading Intelligence Platform

React + FastAPI + MongoDB + Stripe

Document Generated: April 07, 2026

Total Source Files: 60+

Architecture: Full-Stack PWA

Red Slate Holdings

managingdirector@redslateholdings.com

CONFIDENTIAL - PROPRIETARY SOURCE CODE

This document contains the complete source code for the RISEDUAL AI platform.

Table of Contents

1. Project Overview

RISEDUAL AI is a full-stack AI-powered trading intelligence platform built for Red Slate Holdings. It provides real-time market data, AI-driven predictions, dark pool analysis, options flow screening, crypto tracking, and a comprehensive trading workspace with subscription-based access control.

TECH STACK:

- Frontend: React 18, TailwindCSS, Shadcn/UI, Recharts, PWA (Service Worker + Web Push)
- Backend: Python FastAPI, MongoDB (Motor Async Driver), APScheduler
- AI: OpenAI GPT-5.2 via Emergent Integrations (Universal Key)
- Payments: Stripe (\$45/month subscription)
- Email: Resend (transactional + daily digest)
- Auth: JWT Bearer tokens (PyJWT + bcrypt)
- Market Data: Alpha Vantage API (stocks, crypto, options)

KEY FEATURES:

1. Real-time Stock & Crypto Tickers with price tracking
2. Multimodal AI Chat Assistant (text + image analysis)
3. AI Market Prediction Engine (scrapes news, macro data, real estate, foreign markets)
4. Dark Pool Data Monitoring (Pro feature, blur wall for free users)
5. Options Flow Radar & Screener
6. Perplexity-style Company Research with AI synthesis
7. Macro Intelligence Dashboard (World Events, Foreign Markets, Congressional Trades)
8. Trading Journal with P&L Analytics & AI Hypothesis linking
9. Stripe Subscription Gateway (\$45/month or \$486/year)
10. 7 Paywall/Pro Features (chat limits, watchlist caps, PDF export, etc.)
11. Gamified Referral Program with Leaderboard & Social Sharing
12. Admin-Controlled Promo Campaigns with sticky banners
13. Resend Email Notifications (Welcome, Referral, Reward)
14. Daily Digest Emails at 6 AM UTC (APScheduler)
15. PWA Web Push Notifications (5 trigger types, rate-limited)
16. Owner Admin Panel (user management, Pro grants, promo control)
17. Responsive Mobile Design with Bottom Navigation
18. Chart Pattern Library (SVG interactive patterns)
19. Portfolio Analyzer (Pro feature)
20. AI Market Signals (Pro feature)

USER ROLES:

- Owner: Full admin access, can activate/deactivate users, grant/revoke Pro
- Admin: Elevated access, can manage promos and broadcast notifications
- Pro User: Unlimited access to all features (\$45/month)
- Free User: Limited access (5 chats/day, 5 watchlist items, blur walls, 5 journal trades)
- Trial User: Pro access for 7 days (via referral program)

2. Architecture & Directory Structure

DIRECTORY STRUCTURE:

```
/app
|-- frontend/          # React PWA Application
|  |-- public/
|  |  |-- index.html    # HTML entry point with PWA meta tags
|  |  |-- manifest.json # PWA manifest (name, icons, theme)
|  |  |-- service-worker.js # Push notification handler + offline cache
|  |  |-- logo-*.png    # Brand logos (32, 192, 512, AI avatar)
|  |-- src/
|  |  |-- App.js        # Root component, routing, auth state
|  |  |-- App.css       # Global styles, glassmorphism, animations
|  |  |-- index.js      # React entry point
|  |  |-- index.css     # TailwindCSS + custom CSS variables
|  |  |-- mockData.js   # Fallback data for demo mode
|  |  |-- services/
|  |  |  |-- api.js     # Axios instance, auth interceptors
|  |  |  |-- contexts/
|  |  |  |  |-- AuthContext.jsx # JWT auth state management
|  |  |-- hooks/
|  |  |  |-- use-toast.js # Toast notification hook
|  |  |  |-- usePushNotifications.js # Web Push subscription hook
|  |  |-- components/
|  |  |  |-- Navbar.jsx  # Main navigation bar
|  |  |  |-- MobileBottomNav.jsx # Mobile bottom tab navigation
|  |  |  |-- AuthModal.jsx # Login/Register modal
|  |  |  |-- UserWorkspace.jsx # User dashboard workspace
|  |  |  |-- AdminPanel.jsx # Owner admin panel
|  |  |  |-- StockTicker.jsx # Real-time stock ticker strip
|  |  |  |-- CryptoTicker.jsx # Crypto price ticker
|  |  |  |-- CryptoSection.jsx # Crypto market section
|  |  |  |-- DataTable.jsx # Reusable data table component
|  |  |  |-- DarkPoolData.jsx # Dark pool activity (Pro blur wall)
|  |  |  |-- OptionsRadar.jsx # Options flow radar display
|  |  |  |-- OptionsFlowScreener.jsx # Advanced options screener
|  |  |  |-- FilterPanel.jsx # Market data filter panel
|  |  |  |-- TradeGPTChat.jsx # Multimodal AI chat interface
|  |  |  |-- AIHypothesis.jsx # AI hypothesis generation
|  |  |  |-- ChartPatternLibrary.jsx # SVG chart pattern reference
|  |  |  |-- CompanyResearch.jsx # Perplexity-style research
|  |  |  |-- MarketPrediction.jsx # AI market prediction display
|  |  |  |-- MacroDashboard.jsx # Macro intelligence dashboard
|  |  |  |-- MarketSignals.jsx # Pro market signals
|  |  |  |-- PortfolioAnalyzer.jsx # Portfolio analysis (Pro)
```

```
| | | |-- TradingJournal.jsx # Trade journal + analytics
| | | |-- Watchlist.jsx      # User watchlist
| | | |-- QuickTrade.jsx     # Quick trade panel
| | | |-- BrokerConnect.jsx  # Broker connection modal
| | | |-- SubscriptionPricing.jsx # Stripe pricing/checkout
| | | |-- PaymentStatus.jsx  # Payment confirmation page
| | | |-- ProBlurWall.jsx    # Blur overlay for free users
| | | |-- AlertsPanel.jsx    # Alerts & notifications
| | | |-- AdditionalSections.jsx # Extra dashboard sections
| | | |-- PromoBanner.jsx    # Admin promo banner display
| | | |-- ReferralLeaderboard.jsx # Referral leaderboard
| | | |-- SocialShareButtons.jsx # Social media share buttons
| | |-- components/ui/       # Shadcn/UI component library
| |-- package.json
| |-- tailwind.config.js
| |-- .env
|
|-- backend/                # FastAPI Backend
| |-- server.py             # App factory, DB init, router mounts, APScheduler
| |-- .env                  # Environment variables
| |-- requirements.txt       # Python dependencies
| |-- models/
| | |-- chat.py             # Chat & AI message schemas
| | |-- subscription.py     # Subscription plan models
| | |-- trading.py          # Trading & order models
| |-- routes/
| | |-- __init__.py
| | |-- auth.py             # Auth: login, register, admin, JWT
| | |-- ai.py               # AI: chat, hypothesis, predictions, research
| | |-- subscription.py     # Stripe checkout, webhooks, plan mgmt
| | |-- workspace.py        # Watchlist, history, notifications
| | |-- trading.py          # Broker, orders, positions
| | |-- market.py           # Stocks, crypto, dark pool, options
| | |-- referral.py         # Referral codes, tracking, rewards
| | |-- promo.py            # Admin promo campaign CRUD
| | |-- digest.py           # Daily digest trigger, opt-in/out
| | |-- push.py             # Push subscribe, broadcast, test
| | |-- journal.py          # Trading journal CRUD + analytics
| |-- services/
| | |-- ai_service.py       # GPT-5.2 chat, vision, analysis
| | |-- market_data_service.py # Alpha Vantage data fetching
| | |-- market_prediction_service.py # AI prediction engine
| | |-- company_research_service.py # Company research synthesis
| | |-- hypothesis_service.py # AI hypothesis generation
| | |-- financial_scraping_service.py # News & financial scraping
| | |-- real_estate_scraping_service.py # Real estate data
```

```
| | |-- crypto_scraping_service.py # Crypto market scraping
| | |-- world_events_service.py # Global events scraping
| | |-- foreign_markets_service.py # Foreign market data
| | |-- gov_filings_service.py # Congressional trade data
| | |-- broker_service.py # Mock broker integration
| | |-- payment_service.py # Stripe payment processing
| | |-- email_service.py # Resend email templates
| | |-- digest_service.py # Daily digest generation
| | |-- push_service.py # Web Push notification service
| |-- tests/ # Pytest test suite (17 test files)
```

3. Setup & Deployment Guide

PREREQUISITES:

- Node.js 18+ and Yarn
- Python 3.10+
- MongoDB 6.0+
- Stripe Account (for payment processing)
- Alpha Vantage API Key (free tier: 5 calls/min)
- Resend API Key (for email notifications)

BACKEND SETUP:

```
cd backend
python -m venv venv
source venv/bin/activate # Linux/Mac
pip install -r requirements.txt
# Copy .env.example to .env and fill in your values
uvicorn server:app --host 0.0.0.0 --port 8001 --reload
```

FRONTEND SETUP:

```
cd frontend
yarn install
# Copy .env.example to .env and set REACT_APP_BACKEND_URL
yarn start
```

ENVIRONMENT VARIABLES (Backend .env):

```
MONGO_URL=mongodb://localhost:27017
DB_NAME=tradealgo_db
JWT_SECRET=<generate-random-256-bit-hex>
ALPHA_VANTAGE_API_KEY=<your-key>
EMERGENT_LLM_KEY=<your-emergent-universal-key>
STRIPE_API_KEY=sk_live_...
STRIPE_PUBLISHABLE_KEY=pk_live_...
STRIPE_PRICE_ID=price_...
RESEND_API_KEY=re_...
SENDER_EMAIL=noreply@yourdomain.com
VAPID_PRIVATE_KEY=<generate-vapid-keys>
VAPID_PUBLIC_KEY=<generate-vapid-keys>
OWNER_EMAIL=<owner-email>
OWNER_PASSWORD=<owner-password>
ADMIN_EMAIL=<admin-email>
ADMIN_PASSWORD=<admin-password>
CORS_ORIGINS=*
```

ENVIRONMENT VARIABLES (Frontend .env):

```
REACT_APP_BACKEND_URL=https://yourdomain.com
```

REACT_APP_VAPID_PUBLIC_KEY=<same-as-backend-public-key>

DATABASE:

MongoDB collections are auto-created on first use:

- users, chat_sessions, payment_transactions, watchlists
- notifications, referral_codes, promos, push_subscriptions
- push_log, trades

DEPLOYMENT NOTES:

- All backend API routes must be prefixed with /api
- Frontend proxies /api/* requests to the backend
- VAPID keys must match between frontend and backend
- Owner and Admin accounts are auto-seeded on startup
- APScheduler runs daily digest at 6 AM UTC
- Service worker must be served from root (/) for PWA

4. Database Schemas

MONGODB COLLECTIONS:

users:

```
{
  email: String (unique),
  password: String (bcrypt hashed),
  name: String,
  role: "user" | "admin" | "owner",
  subscription: "free" | "pro" | "trial",
  trial_expires_at: DateTime | null,
  is_active: Boolean (default true),
  referred_by: String | null,
  created_at: DateTime
}
```

chat_sessions:

```
{
  session_id: String (UUID),
  user_id: String,
  messages: [
    { role: "user" | "assistant", content: String, timestamp: DateTime,
      image_data: String | null }
  ],
  title: String,
  created_at: DateTime,
  updated_at: DateTime
}
```

payment_transactions:

```
{
  session_id: String (Stripe session),
  user_id: String | null,
  amount: Number,
  currency: String,
  plan: String,
  status: "pending" | "completed" | "failed",
  created_at: DateTime
}
```

watchlists:

```
{
  user_id: String,
  symbols: [String],
}
```

```
    updated_at: DateTime
  }
```

notifications:

```
{
  user_id: String,
  type: String,
  title: String,
  message: String,
  read: Boolean,
  created_at: DateTime
}
```

referral_codes:

```
{
  user_id: String,
  code: String (unique, 8-char),
  rewards_earned: Number (default 0),
  rewards_remaining: Number (default 12),
  referrals: [
    { email: String, signed_up_at: DateTime, subscribed: Boolean }
  ],
  created_at: DateTime
}
```

promos:

```
{
  title: String,
  description: String,
  link: String | null,
  is_active: Boolean,
  created_by: String,
  created_at: DateTime
}
```

push_subscriptions:

```
{
  user_id: String,
  subscription_data: Object (Web Push subscription),
  is_active: Boolean,
  created_at: DateTime,
  updated_at: DateTime
}
```

push_log:

```
{
```

```
    user_id: String,  
    notification_type: String,  
    sent_at: DateTime  
}
```

trades:

```
{  
  user_id: String,  
  ticker: String,  
  type: "long" | "short",  
  entry_price: Number,  
  exit_price: Number | null,  
  quantity: Number,  
  status: "open" | "closed",  
  notes: String,  
  tags: [String],  
  ai_hypothesis_id: String | null,  
  opened_at: DateTime,  
  closed_at: DateTime | null,  
  created_at: DateTime  
}
```

5. API Reference

ALL ENDPOINTS (prefixed with /api):

AUTH (/api/auth):

- POST /api/auth/register - Register new user (email, password, name, referral_code?)
- POST /api/auth/login - Login (email, password) -> access_token, refresh_token
- POST /api/auth/refresh - Refresh access token
- GET /api/auth/me - Get current user profile [Auth Required]
- GET /api/auth/admin/users - List all users [Owner Only]
- PUT /api/auth/admin/users/{id}/toggle-active - Activate/deactivate user [Owner]
- PUT /api/auth/admin/users/{id}/toggle-pro - Grant/revoke Pro [Owner]

AI (/api/ai):

- POST /api/chat - Send chat message (multimodal) [Auth, Rate Limited]
- GET /api/chat/sessions - List chat sessions [Auth]
- GET /api/chat/sessions/{id} - Get session messages [Auth, Pro for history]
- POST /api/hypothesis/generate - Generate AI hypothesis [Auth]
- GET /api/predictions - Get AI market predictions [Auth]
- GET /api/research/{symbol} - Perplexity-style company research [Auth]
- POST /api/portfolio/analyze - AI portfolio analysis [Pro Only]
- GET /api/signals - AI market signals [Pro Only]

MARKET (/api/market):

- GET /api/stocks/top - Top stocks with real-time prices
- GET /api/crypto/prices - Crypto prices (BTC, ETH, etc.)
- GET /api/darkpool - Dark pool activity data [Pro Only]
- GET /api/options/flow - Options flow data
- GET /api/macro/dashboard - Macro intelligence dashboard

SUBSCRIPTION (/api/subscription):

- GET /api/subscription/plans - Available subscription plans
- POST /api/subscription/create-checkout-session - Stripe checkout [Auth]
- POST /api/subscription/webhook - Stripe webhook handler
- GET /api/payment/status/{session_id} - Payment status check

WORKSPACE (/api/workspace):

- GET /api/workspace/watchlist - Get user watchlist [Auth]
- POST /api/workspace/watchlist - Add to watchlist [Auth, Capped for Free]
- DELETE /api/workspace/watchlist/{s} - Remove from watchlist [Auth]
- GET /api/workspace/history - Chat history [Auth, Pro Only]
- GET /api/workspace/notifications - User notifications [Auth]
- PUT /api/workspace/notifications/{id}/read - Mark read [Auth]

TRADING (/api/trading):

POST /api/broker/connect - Connect broker account
POST /api/trading/order - Place trade order
GET /api/trading/positions - Get open positions
GET /api/trading/history - Trade execution history

REFERRAL (/api/referral):

POST /api/referral/register - Generate referral code [Auth]
GET /api/referral/status - Get referral stats [Auth]
GET /api/referral/leaderboard - Public referral leaderboard

PROMO (/api/promo):

POST /api/promo/create - Create promo campaign [Admin/Owner]
GET /api/promo/active - Get active promos
GET /api/promo/all - List all promos [Admin/Owner]
PUT /api/promo/{id}/toggle - Toggle promo active state [Admin/Owner]
DELETE /api/promo/{id} - Delete promo [Admin/Owner]

DIGEST (/api/digest):

POST /api/digest/trigger - Manually trigger digest [Admin]
POST /api/digest/opt-in - Opt into daily digest [Auth]
POST /api/digest/opt-out - Opt out of daily digest [Auth]
GET /api/digest/preview - Preview digest email [Auth]

PUSH (/api/push):

POST /api/push/subscribe - Subscribe to push notifications [Auth]
POST /api/push/unsubscribe - Unsubscribe from push [Auth]
GET /api/push/status - Get push subscription status [Auth]
POST /api/push/test - Send test notification [Auth]
POST /api/push/broadcast - Broadcast to all subscribers [Admin]

JOURNAL (/api/journal):

GET /api/journal/trades - List user trades [Auth]
POST /api/journal/trades - Create trade entry [Auth, Capped for Free]
PUT /api/journal/trades/{id} - Update trade [Auth]
DELETE /api/journal/trades/{id} - Delete trade [Auth]
PUT /api/journal/trades/{id}/close - Close trade with exit price [Auth]
POST /api/journal/trades/{id}/attach-hypothesis - Attach AI hypothesis [Auth]
GET /api/journal/analytics - Trade analytics (win rate, P&L) [Auth]

AUTHENTICATION:

All [Auth] endpoints require: Authorization: Bearer <access_token>
Tokens expire after 24 hours. Use /api/auth/refresh to renew.

RATE LIMITING:

Free users: 5 AI chats/day, 5 watchlist items, 5 journal trades
Pro users: Unlimited access to all features

Push notifications: Free users get 1/day, Pro users unlimited

6. Environment Configuration

ENV: backend/.env (sensitive values masked)

```
MONGO_URL=mongodb://localhost:27017
DB_NAME=tradealgo_db
ALPHA_VANTAGE_API_KEY=1FW6*****...
EMERGENT_LLM_KEY=sk-e*****...
CORS_ORIGINS=*

STRIPE_API_KEY=sk_t*****...
STRIPE_PUBLISHABLE_KEY=pk_t*****...
STRIPE_PRICE_ID=price_1TFQl54zEscw4uTH2Rx3VWq4

# PayPal Configuration (Add your keys before deploying)
PAYPAL_CLIENT_ID=your_paypal_client_id
PAYPAL_CLIENT_SECRET=your_paypal_secret
PAYPAL_MODE=sandbox
JWT_SECRET=2b01*****...
ADMIN_EMAIL=admin@risedual.ai
ADMIN_PASSWORD=Rise*****...
OWNER_EMAIL=managingdirector@redslateholdings.com
OWNER_PASSWORD=RedS*****...
RESEND_API_KEY=re_Y*****...
SENDER_EMAIL=onboarding@resend.dev
VAPID_PRIVATE_KEY=jpGu*****...
VAPID_PUBLIC_KEY=BF73*****...
```

ENV: frontend/.env (sensitive values masked)

```
REACT_APP_BACKEND_URL=https://risedual-trading.preview.emergentagent.com
WDS_SOCKET_PORT=443
ENABLE_HEALTH_CHECK=false
REACT_APP_VAPID_PUBLIC_KEY=BF73*****...
```

7. Backend - Core

FILE: backend/server.py

157 lines | 5237 characters

```
1 | """RISEDUAL AI ? FastAPI Application Entry Point.
2 |
3 | Thin orchestrator: connects MongoDB, registers route modules, handles startup/shutdown.
4 | """
5 | from fastapi import FastAPI, APIRouter
6 | from dotenv import load_dotenv
7 | from starlette.middleware.cors import CORSMiddleware
8 | from motor.motor_asyncio import AsyncIOMotorClient
9 | import os
10 | import logging
11 | from pathlib import Path
12 | from pydantic import BaseModel, Field, ConfigDict
13 | from typing import List
14 | import uuid
15 | from datetime import datetime, timezone
16 |
17 | ROOT_DIR = Path(__file__).parent
18 | load_dotenv(ROOT_DIR / '.env')
19 |
20 | # Route modules
21 | from routes.auth import auth_router, set_db as set_auth_db, seed_admin, create_indexes
22 | from routes.market import router as market_router
23 | from routes.trading import router as trading_router
24 | from routes.ai import router as ai_router, set_db as set_ai_db
25 | from routes.workspace import router as workspace_router, set_db as set_workspace_db
26 | from routes.subscription import router as subscription_router, set_db as set_subscription_db
27 | from routes.referral import router as referral_router, set_db as set_referral_db
28 | from routes.promo import router as promo_router, set_db as set_promo_db
29 | from routes.digest import router as digest_router, set_db as set_digest_db
30 | from routes.push import router as push_router, set_db as set_push_db
31 | from routes.journal import router as journal_router, set_db as set_journal_db
32 |
33 | # MongoDB connection
34 | mongo_url = os.environ['MONGO_URL']
35 | client = AsyncIOMotorClient(mongo_url)
36 | db = client[os.environ['DB_NAME']]
37 |
38 | # FastAPI app
39 | app = FastAPI()
40 |
41 | # Minimal status-check router (kept in server.py for health checks)
42 | api_router = APIRouter(prefix="/api")
43 |
44 |
45 | class StatusCheck(BaseModel):
46 |     model_config = ConfigDict(extra="ignore")
47 |     id: str = Field(default_factory=lambda: str(uuid.uuid4()))
48 |     client_name: str
49 |     timestamp: datetime = Field(default_factory=lambda: datetime.now(timezone.utc))
50 |
51 |
52 | class StatusCheckCreate(BaseModel):
53 |     client_name: str
54 |
55 |
56 | @api_router.get("/")
57 | async def root():
58 |     return {"message": "RISEDUAL AI API - Ready"}
```



```
59 |
60 |
61 | @api_router.post("/status", response_model=StatusCheck)
62 | async def create_status_check(input: StatusCheckCreate):
63 |     status_obj = StatusCheck(**input.model_dump())
64 |     doc = status_obj.model_dump()
65 |     doc['timestamp'] = doc['timestamp'].isoformat()
66 |     await db.status_checks.insert_one(doc)
67 |     return status_obj
68 |
69 |
70 | @api_router.get("/status", response_model=List[StatusCheck])
71 | async def get_status_checks():
72 |     status_checks = await db.status_checks.find({}, {"_id": 0}).to_list(1000)
73 |     for check in status_checks:
74 |         if isinstance(check['timestamp'], str):
75 |             check['timestamp'] = datetime.fromisoformat(check['timestamp'])
76 |     return status_checks
77 |
78 |
79 | # Register all routers
80 | app.include_router(api_router)
81 | app.include_router(auth_router)
82 | app.include_router(market_router)
83 | app.include_router(trading_router)
84 | app.include_router(ai_router)
85 | app.include_router(workspace_router)
86 | app.include_router(subscription_router)
87 | app.include_router(referral_router)
88 | app.include_router(promo_router)
89 | app.include_router(digest_router)
90 | app.include_router(push_router)
91 | app.include_router(journal_router)
92 |
93 | # CORS
94 | app.add_middleware(
95 |     CORSMiddleware,
96 |     allow_origins=["*"],
97 |     allow_methods=["*"],
98 |     allow_headers=["*"],
99 | )
100 |
101 | # Logging
102 | logging.basicConfig(
103 |     level=logging.INFO,
104 |     format='%(asctime)s - %(name)s - %(levelname)s - %(message)s'
105 | )
106 | logger = logging.getLogger(__name__)
107 |
108 |
109 | @app.on_event("startup")
110 | async def startup_event():
111 |     # Pass db reference to all route modules that need it
112 |     set_auth_db(db)
113 |     set_ai_db(db)
114 |     set_workspace_db(db)
115 |     set_subscription_db(db)
116 |     set_referral_db(db)
117 |     set_promo_db(db)
118 |     set_digest_db(db)
119 |     set_push_db(db)
120 |     set_journal_db(db)
121 |
122 |     # Start daily digest scheduler (6:00 AM UTC)
123 |     try:
124 |         from apscheduler.schedulers.asyncio import AsyncIOScheduler
125 |         from services.digest_service import send_daily_digest
```

```
126 |         scheduler = AsyncIOScheduler()
127 |         scheduler.add_job(send_daily_digest, 'cron', hour=6, minute=0, args=[db], id='daily_digest')
128 |         scheduler.start()
129 |         logger.info("Daily digest scheduler started (6:00 AM UTC)")
130 |     except Exception as e:
131 |         logger.warning(f"Digest scheduler setup failed: {e}")
132 |
133 |     await create_indexes()
134 |     await seed_admin()
135 |
136 |     # Write test credentials
137 |     creds_path = Path("/app/memory/test_credentials.md")
138 |     creds_path.parent.mkdir(parents=True, exist_ok=True)
139 |     creds_path.write_text(
140 |         "# Test Credentials\n\n"
141 |         "### Owner (REDSLATE)\n\n"
142 |         f"- Email: {os.environ.get('OWNER_EMAIL', 'managingdirector@redslateholdings.com')}\n\n"
143 |         f"- Password: {os.environ.get('OWNER_PASSWORD', '')}\n\n"
144 |         "- Role: owner\n- Subscription: pro\n\n"
145 |         "- Can activate/deactivate users and grant/revoke Pro\n\n"
146 |         f"### Admin\n- Email: {os.environ.get('ADMIN_EMAIL', 'admin@risedual.ai')}\n\n"
147 |         f"- Password: {os.environ.get('ADMIN_PASSWORD', '')}\n\n"
148 |         "- Role: admin\n- Subscription: pro\n\n"
149 |         "### Auth Method\n- Bearer token via localStorage\n\n"
150 |         "- POST /api/auth/login ? returns access_token + refresh_token\n\n"
151 |     )
152 |
153 |
154 | @app.on_event("shutdown")
155 | async def shutdown_db_client():
156 |     client.close()
157 |
```

FILE: backend/requirements.txt

134 lines | 2371 characters

```
1 | aiohappyeyeballs==2.6.1
2 | aiohttp==3.13.3
3 | aiosignal==1.4.0
4 | annotated-doc==0.0.4
5 | annotated-types==0.7.0
6 | anyio==4.12.1
7 | APScheduler==3.11.2
8 | attrs==25.4.0
9 | bcrypt==4.1.3
10 | beautifulsoup4==4.14.3
11 | black==26.1.0
12 | boto3==1.42.58
13 | botocore==1.42.58
14 | certifi==2026.2.25
15 | cffi==2.0.0
16 | charset-normalizer==3.4.4
17 | click==8.3.1
18 | cryptography==46.0.5
19 | distro==1.9.0
20 | dnspython==2.8.0
21 | ecdsa==0.19.1
22 | email-validator==2.3.0
23 | emergentintegrations==0.1.0
24 | fastapi==0.110.1
25 | fastuuid==0.14.0
26 | filelock==3.25.0
27 | flake8==7.3.0
28 | frozenlist==1.8.0
29 | fsspec==2026.2.0
30 | google-ai-generativelanguage==0.6.15
```

```
31 | google-api-core==2.30.0
32 | google-api-python-client==2.191.0
33 | google-auth==2.49.0.dev0
34 | google-auth-httplib2==0.3.0
35 | google-genai==1.65.0
36 | google-generativeai==0.8.6
37 | googleapis-common-protos==1.72.0
38 | grpcio==1.78.0
39 | grpcio-status==1.71.2
40 | h11==0.16.0
41 | hf-xet==1.3.2
42 | http_ece==1.2.1
43 | httpcore==1.0.9
44 | httplib2==0.31.2
45 | httpx==0.28.1
46 | huggingface_hub==1.5.0
47 | idna==3.11
48 | importlib_metadata==8.7.1
49 | iniconfig==2.3.0
50 | isort==8.0.0
51 | Jinja2==3.1.6
52 | jiter==0.13.0
53 | jmespath==1.1.0
54 | jq==1.11.0
55 | jsonschema==4.26.0
56 | jsonschema-specifications==2025.9.1
57 | librt==0.8.1
58 | litellm==1.80.0
59 | lxml==6.0.2
60 | markdown-it-py==4.0.0
61 | MarkupSafe==3.0.3
62 | mccabe==0.7.0
63 | mdurl==0.1.2
64 | motor==3.3.1
65 | multidict==6.7.1
66 | mypy==1.19.1
67 | mypy_extensions==1.1.0
68 | numpy==2.4.2
69 | oauthlib==3.3.1
70 | openai==1.99.9
71 | packaging==26.0
72 | pandas==3.0.1
73 | passlib==1.7.4
74 | pathspec==1.0.4
75 | pillow==12.1.1
76 | platformdirs==4.9.2
77 | pluggy==1.6.0
78 | propcache==0.4.1
79 | proto-plus==1.27.1
80 | protobuf==5.29.6
81 | py-vapid==1.9.4
82 | pyasn1==0.6.2
83 | pyasn1_modules==0.4.2
84 | pycodestyle==2.14.0
85 | pycparser==3.0
86 | pydantic==2.12.5
87 | pydantic_core==2.41.5
88 | pyflakes==3.4.0
89 | Pygments==2.19.2
90 | PyJWT==2.11.0
91 | pymongo==4.5.0
92 | pyparsing==3.3.2
93 | pytest==9.0.2
94 | pytest-asyncio==1.3.0
95 | python-dateutil==2.9.0.post0
96 | python-dotenv==1.2.1
97 | python-jose==3.5.0
```

```
98 | python-multipart==0.0.22
99 | pytokens==0.4.1
100 | pywebpush==2.3.0
101 | PyYAML==6.0.3
102 | referencing==0.37.0
103 | regex==2026.2.28
104 | requests==2.32.5
105 | requests-oauthlib==2.0.0
106 | resend==2.27.0
107 | rich==14.3.3
108 | rpds-py==0.30.0
109 | rsa==4.9.1
110 | s3transfer==0.16.0
111 | s5cmd==0.2.0
112 | shellingham==1.5.4
113 | six==1.17.0
114 | sniffio==1.3.1
115 | soupsieve==2.8.3
116 | starlette==0.37.2
117 | stripe==14.4.0
118 | tenacity==9.1.4
119 | tiktoken==0.12.0
120 | tokenizers==0.22.2
121 | tqdm==4.67.3
122 | typer==0.24.1
123 | typing-inspection==0.4.2
124 | typing_extensions==4.15.0
125 | tzdata==2025.3
126 | tzlocal==5.3.1
127 | uritemplate==4.2.0
128 | urllib3==2.6.3
129 | uvicorn==0.25.0
130 | watchfiles==1.1.1
131 | websockets==16.0
132 | yarl==1.23.0
133 | zipp==3.23.0
134 |
```

Backend - Models

FILE: [backend/models/chat.py](#)

26 lines | 829 characters

```
1 | from pydantic import BaseModel, Field
2 | from typing import List, Optional
3 | from datetime import datetime, timezone
4 | import uuid
5 |
6 | class ChatMessage(BaseModel):
7 |     role: str # 'user' or 'assistant'
8 |     content: str
9 |     image_base64: Optional[str] = None
10 |     timestamp: datetime = Field(default_factory=lambda: datetime.now(timezone.utc))
11 |
12 | class ChatSession(BaseModel):
13 |     id: str = Field(default_factory=lambda: str(uuid.uuid4()))
14 |     session_id: str
15 |     messages: List[ChatMessage] = []
16 |     created_at: datetime = Field(default_factory=lambda: datetime.now(timezone.utc))
17 |     updated_at: datetime = Field(default_factory=lambda: datetime.now(timezone.utc))
18 |
19 | class ChatRequest(BaseModel):
20 |     message: str
21 |     sessionId: str
```

```
22 |     image_base64: Optional[str] = None
23 |
24 | class ChatResponse(BaseModel):
25 |     response: str
26 |     sessionId: str
```

FILE: backend/models/subscription.py

31 lines | 1221 characters

```
1 | from pydantic import BaseModel, Field
2 | from typing import Optional
3 | from datetime import datetime
4 | import uuid
5 |
6 | class Subscription(BaseModel):
7 |     id: str = Field(default_factory=lambda: str(uuid.uuid4()))
8 |     user_id: str
9 |     user_email: str
10 |     plan_type: str = 'yearly' # 'yearly', 'monthly'
11 |     status: str = 'active' # 'active', 'cancelled', 'expired', 'past_due'
12 |     payment_provider: str # 'stripe', 'paypal'
13 |     stripe_customer_id: Optional[str] = None
14 |     stripe_subscription_id: Optional[str] = None
15 |     paypal_subscription_id: Optional[str] = None
16 |     current_period_start: datetime = Field(default_factory=datetime.utcnow)
17 |     current_period_end: Optional[datetime] = None
18 |     cancel_at_period_end: bool = False
19 |     created_at: datetime = Field(default_factory=datetime.utcnow)
20 |     updated_at: datetime = Field(default_factory=datetime.utcnow)
21 |
22 | class PaymentIntent(BaseModel):
23 |     id: str = Field(default_factory=lambda: str(uuid.uuid4()))
24 |     user_id: str
25 |     amount: float
26 |     currency: str = 'usd'
27 |     status: str # 'pending', 'succeeded', 'failed'
28 |     payment_provider: str
29 |     provider_payment_id: str
30 |     metadata: Optional[dict] = None
31 |     created_at: datetime = Field(default_factory=datetime.utcnow)
```

FILE: backend/models/trading.py

54 lines | 1874 characters

```
1 | from pydantic import BaseModel, Field
2 | from typing import Optional, List
3 | from datetime import datetime
4 | import uuid
5 |
6 | class BrokerConnection(BaseModel):
7 |     id: str = Field(default_factory=lambda: str(uuid.uuid4()))
8 |     user_id: str # In production, this would be from authenticated user
9 |     broker_id: str # 'td_ameritrade', 'alpaca', etc.
10 |     broker_name: str
11 |     access_token: str
12 |     refresh_token: Optional[str] = None
13 |     token_expires_at: Optional[datetime] = None
14 |     account_id: Optional[str] = None
15 |     is_active: bool = True
16 |     connected_at: datetime = Field(default_factory=datetime.utcnow)
17 |     last_used: datetime = Field(default_factory=datetime.utcnow)
18 |
19 | class TradeOrder(BaseModel):
20 |     id: str = Field(default_factory=lambda: str(uuid.uuid4()))
21 |     user_id: str
```

```
22 |     broker_connection_id: str
23 |     symbol: str
24 |     order_type: str # 'market', 'limit', 'stop', 'stop_limit'
25 |     side: str # 'buy', 'sell'
26 |     quantity: int
27 |     price: Optional[float] = None
28 |     stop_price: Optional[float] = None
29 |     time_in_force: str = 'day' # 'day', 'gtc', 'ioc', 'fok'
30 |     status: str = 'pending' # 'pending', 'submitted', 'filled', 'cancelled', 'rejected'
31 |     broker_order_id: Optional[str] = None
32 |     filled_quantity: int = 0
33 |     average_fill_price: Optional[float] = None
34 |     created_at: datetime = Field(default_factory=datetime.utcnow)
35 |     updated_at: datetime = Field(default_factory=datetime.utcnow)
36 |
37 | class Position(BaseModel):
38 |     symbol: str
39 |     quantity: int
40 |     average_entry_price: float
41 |     current_price: float
42 |     market_value: float
43 |     profit_loss: float
44 |     profit_loss_percent: float
45 |     side: str # 'long', 'short'
46 |
47 | class AccountBalance(BaseModel):
48 |     broker_name: str
49 |     account_id: str
50 |     cash: float
51 |     buying_power: float
52 |     portfolio_value: float
53 |     total_profit_loss: float
54 |     updated_at: datetime = Field(default_factory=datetime.utcnow)
```

Backend - Routes

FILE: backend/routes/__init__.py

2 lines | 17 characters

```
1 | # Routes package
2 |
```

FILE: backend/routes/ai.py

535 lines | 24160 characters

```
1 | """AI routes: chat, hypothesis, predictions, research, scraping endpoints."""
2 | from fastapi import APIRouter, HTTPException, Request
3 | import os
4 | import logging
5 | from datetime import datetime, timezone, timedelta
6 |
7 | from services.ai_service import AIService
8 | from models.chat import ChatRequest, ChatResponse, ChatSession, ChatMessage
9 | from routes.auth import get_current_user, get_optional_user
10 |
11 | router = APIRouter(prefix="/api")
12 | ai_service = AIService()
13 |
14 | FREE_CHAT_DAILY_LIMIT = 5
15 |
16 |
17 | def is_pro_user(user: dict) -> bool:
```

```

18 |     """Check if user has Pro access (includes 'pro' and 'trial' status)."""
19 |     if not user:
20 |         return False
21 |     status = user.get("subscription_status", "free")
22 |     return status in ("pro", "trial")
23 |
24 | # Module-level db reference, set by server.py on startup
25 | db = None
26 |
27 | def set_db(database):
28 |     global db
29 |     db = database
30 |
31 |
32 | # --- Chat Rate Limit ---
33 | @router.get("/chat/limit")
34 | async def get_chat_limit(request: Request):
35 |     """Return remaining chat messages for the user today."""
36 |     user = await get_optional_user(request)
37 |     if not user:
38 |         return {"limit": FREE_CHAT_DAILY_LIMIT, "used": 0, "remaining": FREE_CHAT_DAILY_LIMIT, "is_pro": False}
39 |     if is_pro_user(user):
40 |         return {"limit": -1, "used": 0, "remaining": -1, "is_pro": True}
41 |     today = datetime.now(timezone.utc).replace(hour=0, minute=0, second=0, microsecond=0).isoformat()
42 |     count = await db.chat_usage.count_documents({"user_id": user["_id"], "date": today})
43 |     return {"limit": FREE_CHAT_DAILY_LIMIT, "used": count, "remaining": max(0, FREE_CHAT_DAILY_LIMIT - count), "is_pro": False}
44 |
45 |
46 | # --- AI Chat ---
47 | @router.post("/chat", response_model=ChatResponse)
48 | async def chat(chat_request: ChatRequest, request: Request):
49 |     try:
50 |         # Rate limit for free users
51 |         user = await get_optional_user(request)
52 |         if user and not is_pro_user(user):
53 |             today = datetime.now(timezone.utc).replace(hour=0, minute=0, second=0, microsecond=0).isoformat()
54 |             count = await db.chat_usage.count_documents({"user_id": user["_id"], "date": today})
55 |             if count >= FREE_CHAT_DAILY_LIMIT:
56 |                 raise HTTPException(status_code=429, detail=f"Free accounts are limited to {FREE_CHAT_DAILY_LIMIT} AI messages per ...
57 |                 await db.chat_usage.insert_one({"user_id": user["_id"], "date": today, "timestamp": datetime.now(timezone.utc).isoform ...
58 |
59 |             session = await db.chat_sessions.find_one({"session_id": chat_request.sessionId})
60 |             if not session:
61 |                 new_session = ChatSession(session_id=chat_request.sessionId)
62 |                 session_doc = new_session.dict()
63 |                 if user:
64 |                     session_doc["user_id"] = user["_id"]
65 |                 await db.chat_sessions.insert_one(session_doc)
66 |
67 |             ai_response = await ai_service.chat(chat_request.message, chat_request.sessionId, chat_request.image_base64)
68 |
69 |             user_message = ChatMessage(
70 |                 role="user",
71 |                 content=chat_request.message,
72 |                 image_base64="[image_attached]" if chat_request.image_base64 else None
73 |             )
74 |             assistant_message = ChatMessage(role="assistant", content=ai_response)
75 |
76 |             await db.chat_sessions.update_one(
77 |                 {"session_id": chat_request.sessionId},
78 |                 {
79 |                     "$push": {"messages": {"$each": [user_message.dict(), assistant_message.dict()]}},
80 |                     "$set": {"updated_at": datetime.now(timezone.utc).isoformat()}
81 |                 }
82 |             )
83 |             return ChatResponse(response=ai_response, sessionId=chat_request.sessionId)
84 |     except HTTPException:

```

```

85 |         raise
86 |     except Exception as e:
87 |         logging.error(f"Error in chat endpoint: {e}")
88 |         raise HTTPException(status_code=500, detail="Error processing chat request")
89 |
90 |
91 | @router.get("/chat/history/{session_id}")
92 | async def get_chat_history(session_id: str, request: Request):
93 |     try:
94 |         session = await db.chat_sessions.find_one({"session_id": session_id})
95 |         if not session:
96 |             return {"messages": []}
97 |         messages = session.get("messages", [])
98 |
99 |         # Free users: only get messages from last 24 hours
100 |         user = await get_optional_user(request)
101 |         if user and not is_pro_user(user):
102 |             cutoff = (datetime.now(timezone.utc) - timedelta(hours=24)).isoformat()
103 |             messages = [m for m in messages if m.get("timestamp", "9999") >= cutoff]
104 |
105 |         return {"messages": messages, "history_limited": user is not None and not is_pro_user(user)}
106 |     except Exception as e:
107 |         logging.error(f"Error fetching chat history: {e}")
108 |         raise HTTPException(status_code=500, detail="Error fetching chat history")
109 |
110 |
111 | # --- PDF Export (Pro Only) ---
112 | @router.get("/export/hypothesis/{symbol}")
113 | async def export_hypothesis_pdf(symbol: str, request: Request):
114 |     """Generate a PDF report for a hypothesis. Pro only."""
115 |     user = await get_current_user(request)
116 |     if not is_pro_user(user):
117 |         raise HTTPException(status_code=403, detail="PDF export is a Pro feature. Upgrade to unlock.")
118 |
119 |     # Return JSON data that frontend will format into PDF
120 |     from services.hypothesis_service import HypothesisService
121 |     from services.financial_scraping_service import FinancialScrapingService
122 |     from services.world_events_service import WorldEventsService
123 |     from services.foreign_markets_service import ForeignMarketsService
124 |
125 |     financial_scraper = FinancialScrapingService()
126 |     news = await financial_scraper.scrape_financial_news()
127 |     social = await financial_scraper.scrape_reddit_sentiment()
128 |     world_events = await WorldEventsService().scrape_world_events()
129 |     foreign_markets = await ForeignMarketsService().get_foreign_markets()
130 |
131 |     data = {"news": news, "social": social, "world_events": world_events, "foreign_markets": foreign_markets}
132 |     hypothesis_svc = HypothesisService(os.environ.get("EMERGENT_LLM_KEY"))
133 |     hypothesis = await hypothesis_svc.generate_hypothesis(symbol, data)
134 |     hypothesis["export"] = True
135 |     hypothesis["exported_at"] = datetime.now(timezone.utc).isoformat()
136 |     hypothesis["exported_by"] = user.get("name", user.get("email"))
137 |     return hypothesis
138 |
139 | # --- Market Signals (Pro Only) ---
140 | @router.get("/signals")
141 | async def get_market_signals(request: Request):
142 |     """Get AI-detected market signals for the user's watchlist."""
143 |     user = await get_current_user(request)
144 |     if not is_pro_user(user):
145 |         return {"signals": [], "is_pro": False}
146 |     cursor = db.market_signals.find(
147 |         {"user_id": user["id"]}, {"_id": 0}
148 |     ).sort("detected_at", -1).limit(20)
149 |     signals = []
150 |     async for doc in cursor:
151 |         signals.append(doc)

```



```

152 |     return {"signals": signals, "is_pro": True}
153 |
154 |
155 | @router.post("/signals/scan")
156 | async def scan_for_signals(request: Request):
157 |     """Manually trigger a signal scan for the user's watchlist tickers."""
158 |     user = await get_current_user(request)
159 |     if not is_pro_user(user):
160 |         raise HTTPException(status_code=403, detail="Market signals is a Pro feature.")
161 |
162 |     # Get user's watchlist
163 |     wl = await db.watchlists.find_one({"user_id": user["_id"]})
164 |     tickers = wl.get("tickers", []) if wl else []
165 |     if not tickers:
166 |         return {"signals": [], "message": "No tickers in watchlist"}
167 |
168 |     from services.market_data_service import MarketDataService
169 |     market_svc = MarketDataService()
170 |
171 |     signals_found = []
172 |     for ticker in tickers[:10]: # Limit to 10 tickers
173 |         try:
174 |             dark_pool = market_svc.generate_dark_pool_data()
175 |             dp_match = [d for d in dark_pool if d.get("ticker") == ticker]
176 |             if dp_match and dp_match[0].get("volume", 0) > 500000:
177 |                 sig = {
178 |                     "user_id": user["_id"],
179 |                     "ticker": ticker,
180 |                     "type": "dark_pool_spike",
181 |                     "title": f"Dark Pool Spike: {ticker}",
182 |                     "detail": f"Unusual dark pool volume detected ({dp_match[0].get('volume', 0):,} shares)",
183 |                     "severity": "high",
184 |                     "detected_at": datetime.now(timezone.utc).isoformat(),
185 |                 }
186 |                 await db.market_signals.insert_one(sig)
187 |                 del sig["user_id"]
188 |                 signals_found.append(sig)
189 |
190 |             options = market_svc.generate_mock_options_data('flow')
191 |             opt_match = [o for o in options if o.get("symbol") == ticker or o.get("contract", "").startswith(ticker)]
192 |             if opt_match:
193 |                 for o in opt_match[:1]:
194 |                     sig = {
195 |                         "user_id": user["_id"],
196 |                         "ticker": ticker,
197 |                         "type": "options_flow",
198 |                         "title": f"Options Activity: {ticker}",
199 |                         "detail": f"Notable options flow detected ? {o.get('side', 'N/A')} {o.get('contract', ticker)}",
200 |                         "severity": "medium",
201 |                         "detected_at": datetime.now(timezone.utc).isoformat(),
202 |                     }
203 |                     await db.market_signals.insert_one(sig)
204 |                     del sig["user_id"]
205 |                     signals_found.append(sig)
206 |         except Exception as e:
207 |             logging.warning(f"Signal scan error for {ticker}: {e}")
208 |
209 |     return {"signals": signals_found, "tickers_scanned": len(tickers[:10])}
210 |
211 |
212 | # --- Portfolio Analyzer (Pro Only) ---
213 | @router.post("/portfolio/analyze")
214 | async def analyze_portfolio(request: Request):
215 |     """AI Portfolio Analyzer ? input holdings, get health score + suggestions."""
216 |     user = await get_current_user(request)
217 |     if not is_pro_user(user):
218 |         raise HTTPException(status_code=403, detail="Portfolio Analyzer is a Pro feature. Upgrade to unlock.")

```

```

219 |
220 |     body = await request.json()
221 |     holdings = body.get("holdings", [])
222 |     if not holdings:
223 |         raise HTTPException(status_code=400, detail="Please provide at least one holding.")
224 |
225 |     # Gather macro data for context
226 |     from services.world_events_service import WorldEventsService
227 |     from services.foreign_markets_service import ForeignMarketsService
228 |     world_events = await WorldEventsService().scrape_world_events()
229 |     foreign_markets = await ForeignMarketsService().get_foreign_markets()
230 |
231 |     holdings_text = "\n".join([f"- {h.get('ticker', 'UNKNOWN')}: {h.get('shares', 0)} shares @ ${h.get('avg_price', 0)}" for h in ...
232 |     total_value = sum(h.get("shares", 0) * h.get("avg_price", 0) for h in holdings)
233 |
234 |     prompt = f"""You are an expert portfolio analyst for RISEDUAL AI. Analyze this portfolio and provide a health score and rebala ...
235 |
236 | PORTFOLIO (Total Value: ${total_value:,.2f}):
237 | {holdings_text}
238 |
239 | CURRENT MACRO CONTEXT:
240 | - World Events: {world_events.get('total_events', 0)} tracked, {world_events.get('high_impact_count', 0)} high-impact
241 | - Top Affected Sectors: {', '.join([s['sector'] for s in world_events.get('affected_sectors', [])[:5]])}
242 | - Foreign Market Signals: {len(foreign_markets.get('correlation_signals', []))} correlation signals
243 |
244 | Respond in this EXACT JSON format:
245 | {{
246 |     "health_score": <0-100>,
247 |     "risk_level": "<low|medium|high|critical>",
248 |     "diversification_grade": "<A|B|C|D|F>",
249 |     "sector_exposure": "<brief description>",
250 |     "top_risk": "<biggest risk in portfolio>",
251 |     "suggestions": ["<suggestion 1>", "<suggestion 2>", "<suggestion 3>"],
252 |     "rebalance_actions": [
253 |         {{ "ticker": "<TICKER>", "action": "<buy|sell|hold>", "reason": "<why>" }}
254 |     ],
255 |     "summary": "<2-3 sentence overall assessment>"
256 | }}"""
257 |
258 |     try:
259 |         from emergentintegrations.llm.chat import LlmChat, UserMessage
260 |         import json
261 |         llm_key = os.environ.get("EMERGENT_LLM_KEY")
262 |         llm = LlmChat(
263 |             api_key=llm_key,
264 |             session_id=f"portfolio_{user['_id']}",
265 |             system_message="You are a professional portfolio analyst. Return ONLY valid JSON."
266 |         ).with_model("openai", "gpt-5.2")
267 |         response = await llm.send_message(UserMessage(text=prompt))
268 |         text = response.strip() if isinstance(response, str) else response
269 |         try:
270 |             result = json.loads(text.strip().strip("`json").strip("`"))
271 |         except json.JSONDecodeError:
272 |             result = {"health_score": 50, "summary": text, "suggestions": [], "rebalance_actions": [], "risk_level": "medium", "di ...
273 |
274 |         result["total_value"] = total_value
275 |         result["holdings_count"] = len(holdings)
276 |         result["analyzed_at"] = datetime.now(timezone.utc).isoformat()
277 |         return result
278 |     except Exception as e:
279 |         logging.error(f"Portfolio analysis error: {e}")
280 |         raise HTTPException(status_code=500, detail="Error analyzing portfolio")
281 |
282 |
283 | # --- Company Research ---
284 | @router.get("/research/{symbol}")
285 | async def research_company(symbol: str):

```

```
286 |     try:
287 |         from services.company_research_service import CompanyResearchService
288 |         service = CompanyResearchService()
289 |         session_id = f"research_{symbol}_{datetime.now(timezone.utc).isoformat()}"
290 |         return await service.research_company(symbol, session_id)
291 |     except Exception as e:
292 |         logging.error(f"Error researching {symbol}: {e}")
293 |         raise HTTPException(status_code=500, detail=str(e))
294 |
295 |
296 | # --- World Events & Macro ---
297 | @router.get("/world-events")
298 | async def get_world_events():
299 |     try:
300 |         from services.world_events_service import WorldEventsService
301 |         return await WorldEventsService().scrape_world_events()
302 |     except Exception as e:
303 |         logging.error(f"Error fetching world events: {e}")
304 |         raise HTTPException(status_code=500, detail=str(e))
305 |
306 |
307 | @router.get("/foreign-markets")
308 | async def get_foreign_markets():
309 |     try:
310 |         from services.foreign_markets_service import ForeignMarketsService
311 |         return await ForeignMarketsService().get_foreign_markets()
312 |     except Exception as e:
313 |         logging.error(f"Error fetching foreign markets: {e}")
314 |         raise HTTPException(status_code=500, detail=str(e))
315 |
316 |
317 | @router.get("/gov-filings")
318 | async def get_gov_filings():
319 |     try:
320 |         from services.gov_filings_service import GovFilingsService
321 |         return await GovFilingsService().get_all_gov_data()
322 |     except Exception as e:
323 |         logging.error(f"Error fetching gov filings: {e}")
324 |         raise HTTPException(status_code=500, detail=str(e))
325 |
326 |
327 | # --- Market Prediction ---
328 | @router.get("/market/prediction")
329 | async def get_market_prediction():
330 |     try:
331 |         from services.financial_scraping_service import FinancialScrapingService
332 |         from services.crypto_scraping_service import CryptoScrapingService
333 |         from services.real_estate_scraping_service import RealEstateScrapingService
334 |         from services.market_prediction_service import MarketPredictionService
335 |         from services.world_events_service import WorldEventsService
336 |         from services.foreign_markets_service import ForeignMarketsService
337 |         from services.gov_filings_service import GovFilingsService
338 |
339 |         financial_scraper = FinancialScrapingService()
340 |         crypto_scraper = CryptoScrapingService()
341 |         real_estate_scraper = RealEstateScrapingService()
342 |         world_events_svc = WorldEventsService()
343 |         foreign_markets_svc = ForeignMarketsService()
344 |         gov_filings_svc = GovFilingsService()
345 |
346 |         financial_news = await financial_scraper.scrape_financial_news()
347 |         reddit_sentiment = await financial_scraper.scrape_reddit_sentiment()
348 |         insider_trades = await financial_scraper.scrape_insider_trades()
349 |         crypto_data = await crypto_scraper.get_exchange_data()
350 |         whale_transactions = await crypto_scraper.get_whale_transactions()
351 |         crypto_sentiment = await crypto_scraper.get_crypto_sentiment()
352 |         real_estate_data = await real_estate_scraper.scrape_all_real_estate_data()
```

```

353 |         world_events = await world_events_svc.scrape_world_events()
354 |         foreign_markets = await foreign_markets_svc.get_foreign_markets()
355 |         gov_filings = await gov_filings_svc.get_all_gov_data()
356 |
357 |         all_crypto_data = crypto_data + [crypto_sentiment] + whale_transactions
358 |
359 |         prediction_service = MarketPredictionService(os.environ.get('EMERGENT_LLM_KEY'))
360 |         prediction = await prediction_service.analyze_market(
361 |             financial_news=financial_news,
362 |             crypto_data=all_crypto_data,
363 |             insider_trades=insider_trades,
364 |             social_sentiment=reddit_sentiment,
365 |             real_estate_data=real_estate_data,
366 |             world_events=world_events,
367 |             foreign_markets=foreign_markets,
368 |             gov_filings=gov_filings,
369 |         )
370 |
371 |         prediction['real_estate_summary'] = {
372 |             'housing_health': real_estate_data.get('housing', {}).get('market_health', 'unknown'),
373 |             'commercial_trend': 'mixed',
374 |             'data_sources': len(real_estate_data.get('housing', {}).get('sources', [])),
375 |             'implications': real_estate_data.get('trends', {}).get('market_implications', {})
376 |         }
377 |
378 |         prediction['macro_data'] = {
379 |             'world_events': {
380 |                 'total': world_events.get('total_events', 0),
381 |                 'high_impact': world_events.get('high_impact_count', 0),
382 |                 'top_sectors': [s['sector'] for s in world_events.get('affected_sectors', [])[:5]],
383 |             },
384 |             'foreign_markets': {
385 |                 'correlation_signals': foreign_markets.get('correlation_signals', [])[:5],
386 |                 'total_indices': len(foreign_markets.get('asia', [])) + foreign_markets.get('europe', []) + foreign_markets.get('am ...
387 |             },
388 |             'gov_filings': {
389 |                 'congressional_trades': gov_filings.get('congressional_count', 0),
390 |                 'fed_announcements': gov_filings.get('fed_count', 0),
391 |                 'insider_trades': gov_filings.get('insider_count', 0),
392 |             },
393 |         }
394 |
395 |         return prediction
396 |     except Exception as e:
397 |         logging.error(f"Error generating prediction: {e}")
398 |         raise HTTPException(status_code=500, detail="Error generating market prediction")
399 |
400 |
401 | # --- AI Hypothesis ---
402 | @router.get("/hypothesis/{symbol}")
403 | async def get_hypothesis(symbol: str, request: Request):
404 |     user = await get_optional_user(request)
405 |     is_pro = is_pro_user(user)
406 |
407 |     try:
408 |         from services.financial_scraping_service import FinancialScrapingService
409 |         from services.crypto_scraping_service import CryptoScrapingService
410 |         from services.world_events_service import WorldEventsService
411 |         from services.foreign_markets_service import ForeignMarketsService
412 |         from services.gov_filings_service import GovFilingsService
413 |         from services.hypothesis_service import HypothesisService
414 |
415 |         financial_scraper = FinancialScrapingService()
416 |         crypto_scraper = CryptoScrapingService()
417 |         world_events_svc = WorldEventsService()
418 |         foreign_markets_svc = ForeignMarketsService()
419 |         gov_filings_svc = GovFilingsService()

```

```

420 |
421 |     news = await financial_scraper.scrape_financial_news()
422 |     social = await financial_scraper.scrape_reddit_sentiment()
423 |     crypto_data = await crypto_scraper.get_exchange_data()
424 |     world_events = await world_events_svc.scrape_world_events()
425 |     foreign_markets = await foreign_markets_svc.get_foreign_markets()
426 |     gov_filings = await gov_filings_svc.get_all_gov_data()
427 |
428 |     data = {
429 |         "news": news, "social": social, "crypto": crypto_data,
430 |         "world_events": world_events, "foreign_markets": foreign_markets,
431 |         "gov_filings": gov_filings,
432 |     }
433 |
434 |     if not is_pro:
435 |         return {
436 |             "symbol": symbol.upper(),
437 |             "is_pro": False,
438 |             "teaser": {
439 |                 "data_sources_count": len(news) + len(social) + len(crypto_data),
440 |                 "world_events_count": world_events.get("total_events", 0),
441 |                 "congressional_trades_count": gov_filings.get("congressional_count", 0),
442 |                 "verdict": "LOCKED",
443 |                 "summary": f"Our AI has analyzed {len(news)} news articles, {world_events.get('total_events', 0)} world events ...
444 |             },
445 |         }
446 |
447 |     hypothesis_svc = HypothesisService(os.environ.get("EMERGENT_LLM_KEY"))
448 |     hypothesis = await hypothesis_svc.generate_hypothesis(symbol, data)
449 |     hypothesis["is_pro"] = True
450 |
451 |     # Check for verdict change ? generate notification
452 |     if user and hypothesis.get("verdict"):
453 |         try:
454 |             prev = await db.hypothesis_history.find_one(
455 |                 {"user_id": user["_id"], "symbol": symbol.upper()},
456 |                 sort=[("searched_at", -1)]
457 |             )
458 |             if prev and prev.get("verdict") and prev["verdict"] != hypothesis["verdict"]:
459 |                 wl = await db.watchlists.find_one({"user_id": user["_id"]})
460 |                 tickers = wl.get("tickers", []) if wl else []
461 |                 await db.notifications.insert_one({
462 |                     "user_id": user["_id"],
463 |                     "type": "verdict_change",
464 |                     "symbol": symbol.upper(),
465 |                     "old_verdict": prev["verdict"],
466 |                     "new_verdict": hypothesis["verdict"],
467 |                     "confidence": hypothesis.get("confidence", 0),
468 |                     "in_watchlist": symbol.upper() in tickers,
469 |                     "read": False,
470 |                     "created_at": datetime.now(timezone.utc).isoformat(),
471 |                 })
472 |             except Exception as notif_err:
473 |                 logging.warning(f"Notification creation error: {notif_err}")
474 |
475 |         return hypothesis
476 |     except Exception as e:
477 |         logging.error(f"Error generating hypothesis for {symbol}: {e}")
478 |         raise HTTPException(status_code=500, detail="Error generating hypothesis")
479 |
480 |
481 | # --- Scraping Data Endpoints ---
482 | @router.get("/market/news")
483 | async def get_financial_news():
484 |     try:
485 |         from services.financial_scraping_service import FinancialScrapingService
486 |         return await FinancialScrapingService().scrape_financial_news()

```

```
487 |     except Exception as e:
488 |         logging.error(f"Error fetching news: {e}")
489 |         raise HTTPException(status_code=500, detail="Error fetching news")
490 |
491 |
492 | @router.get("/market/social-sentiment")
493 | async def get_social_sentiment():
494 |     try:
495 |         from services.financial_scraping_service import FinancialScrapingService
496 |         return await FinancialScrapingService().scrape_reddit_sentiment()
497 |     except Exception as e:
498 |         logging.error(f"Error fetching social sentiment: {e}")
499 |         raise HTTPException(status_code=500, detail="Error fetching social sentiment")
500 |
501 |
502 | @router.get("/market/insider-trades")
503 | async def get_insider_trades():
504 |     try:
505 |         from services.financial_scraping_service import FinancialScrapingService
506 |         return await FinancialScrapingService().scrape_insider_trades()
507 |     except Exception as e:
508 |         logging.error(f"Error fetching insider trades: {e}")
509 |         raise HTTPException(status_code=500, detail="Error fetching insider trades")
510 |
511 |
512 | @router.get("/market/crypto-data")
513 | async def get_crypto_market_data():
514 |     try:
515 |         from services.crypto_scraping_service import CryptoScrapingService
516 |         scraper = CryptoScrapingService()
517 |         return {
518 |             'exchange_data': await scraper.get_exchange_data(),
519 |             'whale_transactions': await scraper.get_whale_transactions(),
520 |             'sentiment': await scraper.get_crypto_sentiment()
521 |         }
522 |     except Exception as e:
523 |         logging.error(f"Error fetching crypto data: {e}")
524 |         raise HTTPException(status_code=500, detail="Error fetching crypto data")
525 |
526 |
527 | @router.get("/market/real-estate")
528 | async def get_real_estate_data():
529 |     try:
530 |         from services.real_estate_scraping_service import RealEstateScrapingService
531 |         return await RealEstateScrapingService().scrape_all_real_estate_data()
532 |     except Exception as e:
533 |         logging.error(f"Error fetching real estate data: {e}")
534 |         raise HTTPException(status_code=500, detail="Error fetching real estate data")
535 |
```

FILE: backend/routes/auth.py

351 lines | 14879 characters

```
1 | import os
2 | import logging
3 | import bcrypt
4 | import jwt
5 | import secrets
6 | from datetime import datetime, timezone, timedelta
7 | from fastapi import APIRouter, HTTPException, Request, Response
8 | from pydantic import BaseModel, EmailStr
9 | from bson import ObjectId
10 |
11 | JWT_ALGORITHM = "HS256"
12 |
13 | auth_router = APIRouter(prefix="/api/auth", tags=["auth"])
```

```
14 |
15 | # Will be set by server.py
16 | db = None
17 |
18 | def set_db(database):
19 |     global db
20 |     db = database
21 |
22 | def hash_password(password: str) -> str:
23 |     return bcrypt.hashpw(password.encode("utf-8"), bcrypt.gensalt()).decode("utf-8")
24 |
25 | def verify_password(plain: str, hashed: str) -> bool:
26 |     return bcrypt.checkpw(plain.encode("utf-8"), hashed.encode("utf-8"))
27 |
28 | def get_jwt_secret():
29 |     return os.environ["JWT_SECRET"]
30 |
31 | def create_access_token(user_id: str, email: str) -> str:
32 |     payload = {"sub": user_id, "email": email, "exp": datetime.now(timezone.utc) + timedelta(minutes=15), "type": "access"}
33 |     return jwt.encode(payload, get_jwt_secret(), algorithm=JWT_ALGORITHM)
34 |
35 | def create_refresh_token(user_id: str) -> str:
36 |     payload = {"sub": user_id, "exp": datetime.now(timezone.utc) + timedelta(days=7), "type": "refresh"}
37 |     return jwt.encode(payload, get_jwt_secret(), algorithm=JWT_ALGORITHM)
38 |
39 | def set_auth_cookies(response: Response, access_token: str, refresh_token: str):
40 |     response.set_cookie(key="access_token", value=access_token, httponly=True, secure=False, samesite="lax", max_age=900, path="/" ...
41 |     response.set_cookie(key="refresh_token", value=refresh_token, httponly=True, secure=False, samesite="lax", max_age=604800, pat ...
42 |
43 | async def get_current_user(request: Request) -> dict:
44 |     token = request.cookies.get("access_token")
45 |     if not token:
46 |         auth_header = request.headers.get("Authorization", "")
47 |         if auth_header.startswith("Bearer "):
48 |             token = auth_header[7:]
49 |     if not token:
50 |         raise HTTPException(status_code=401, detail="Not authenticated")
51 |     try:
52 |         payload = jwt.decode(token, get_jwt_secret(), algorithms=[JWT_ALGORITHM])
53 |         if payload.get("type") != "access":
54 |             raise HTTPException(status_code=401, detail="Invalid token type")
55 |         user = await db.users.find_one({"_id": ObjectId(payload["sub"])})
56 |         if not user:
57 |             raise HTTPException(status_code=401, detail="User not found")
58 |         user["_id"] = str(user["_id"])
59 |         user.pop("password_hash", None)
60 |         return user
61 |     except jwt.ExpiredSignatureError:
62 |         raise HTTPException(status_code=401, detail="Token expired")
63 |     except jwt.InvalidTokenError:
64 |         raise HTTPException(status_code=401, detail="Invalid token")
65 |
66 | async def get_optional_user(request: Request):
67 |     """Returns user dict or None (no error if not logged in)"""
68 |     try:
69 |         return await get_current_user(request)
70 |     except HTTPException:
71 |         return None
72 |
73 | # Brute force check
74 | async def check_brute_force(identifier: str):
75 |     record = await db.login_attempts.find_one({"identifier": identifier})
76 |     if record and record.get("attempts", 0) >= 5:
77 |         locked_until = record.get("locked_until")
78 |         if locked_until and locked_until > datetime.now(timezone.utc):
79 |             raise HTTPException(status_code=429, detail="Too many failed attempts. Try again in 15 minutes.")
80 |         elif locked_until and locked_until <= datetime.now(timezone.utc):
```

```

81 |         await db.login_attempts.delete_one({"identifier": identifier})
82 |
83 | async def record_failed_attempt(identifier: str):
84 |     record = await db.login_attempts.find_one({"identifier": identifier})
85 |     attempts = (record.get("attempts", 0) if record else 0) + 1
86 |     update = {"$set": {"attempts": attempts, "last_attempt": datetime.now(timezone.utc)}}
87 |     if attempts >= 5:
88 |         update["$set"]["locked_until"] = datetime.now(timezone.utc) + timedelta(minutes=15)
89 |     await db.login_attempts.update_one({"identifier": identifier}, update, upsert=True)
90 |
91 | def user_response(user: dict) -> dict:
92 |     return {
93 |         "id": str(user["_id"]),
94 |         "email": user["email"],
95 |         "name": user.get("name", ""),
96 |         "role": user.get("role", "user"),
97 |         "subscription_status": user.get("subscription_status", "free"),
98 |         "is_active": user.get("is_active", True),
99 |         "trial_ends_at": user.get("trial_ends_at"),
100 |     }
101 |
102 | # --- Models ---
103 | class RegisterRequest(BaseModel):
104 |     email: str
105 |     password: str
106 |     name: str = ""
107 |     ref_code: str = ""
108 |
109 | class LoginRequest(BaseModel):
110 |     email: str
111 |     password: str
112 |
113 | class ForgotPasswordRequest(BaseModel):
114 |     email: str
115 |
116 | class ResetPasswordRequest(BaseModel):
117 |     token: str
118 |     new_password: str
119 |
120 | # --- Routes ---
121 | @auth_router.post("/register")
122 | async def register(req: RegisterRequest, response: Response):
123 |     email = req.email.strip().lower()
124 |     if len(req.password) < 6:
125 |         raise HTTPException(status_code=400, detail="Password must be at least 6 characters")
126 |     existing = await db.users.find_one({"email": email})
127 |     if existing:
128 |         raise HTTPException(status_code=400, detail="Email already registered")
129 |     user_doc = {
130 |         "email": email,
131 |         "password_hash": hash_password(req.password),
132 |         "name": req.name.strip() or email.split("@")[0],
133 |         "role": "user",
134 |         "subscription_status": "free",
135 |         "created_at": datetime.now(timezone.utc),
136 |     }
137 |     result = await db.users.insert_one(user_doc)
138 |     user_doc["_id"] = result.inserted_id
139 |
140 |     # Process referral code if provided
141 |     if req.ref_code and req.ref_code.strip():
142 |         try:
143 |             from routes.referral import process_referral_signup
144 |             await process_referral_signup(str(user_doc["_id"]), email, req.ref_code.strip())
145 |             # Reload user doc to reflect trial status
146 |             updated = await db.users.find_one({"_id": user_doc["_id"]})
147 |             if updated:

```



```
148 |         user_doc["subscription_status"] = updated.get("subscription_status", "free")
149 |     except Exception as e:
150 |         logging.warning(f"Referral processing error: {e}")
151 |
152 |     access = create_access_token(str(user_doc["_id"]), email)
153 |     refresh = create_refresh_token(str(user_doc["_id"]))
154 |     resp = user_response(user_doc)
155 |     resp["access_token"] = access
156 |     resp["refresh_token"] = refresh
157 |     return resp
158 |
159 | @auth_router.post("/login")
160 | async def login(req: LoginRequest, request: Request, response: Response):
161 |     email = req.email.strip().lower()
162 |     ip = request.client.host if request.client else "unknown"
163 |     identifier = f"{ip}:{email}"
164 |     await check_brute_force(identifier)
165 |     user = await db.users.find_one({"email": email})
166 |     if not user or not verify_password(req.password, user["password_hash"]):
167 |         await record_failed_attempt(identifier)
168 |         raise HTTPException(status_code=401, detail="Invalid email or password")
169 |     if user.get("is_active") is False:
170 |         raise HTTPException(status_code=403, detail="Your account has been deactivated. Contact support.")
171 |     await db.login_attempts.delete_one({"identifier": identifier})
172 |     access = create_access_token(str(user["_id"]), email)
173 |     refresh = create_refresh_token(str(user["_id"]))
174 |     resp = user_response(user)
175 |     resp["access_token"] = access
176 |     resp["refresh_token"] = refresh
177 |     return resp
178 |
179 | @auth_router.post("/logout")
180 | async def logout(response: Response):
181 |     response.delete_cookie("access_token", path="/")
182 |     response.delete_cookie("refresh_token", path="/")
183 |     return {"message": "Logged out"}
184 |
185 | @auth_router.get("/me")
186 | async def me(request: Request):
187 |     user = await get_current_user(request)
188 |     return user_response({"_id": user["_id"], **user})
189 |
190 | @auth_router.post("/refresh")
191 | async def refresh_token(request: Request, response: Response):
192 |     # Accept refresh token from body or Authorization header
193 |     body = {}
194 |     try:
195 |         body = await request.json()
196 |     except Exception:
197 |         pass
198 |     token = body.get("refresh_token") or ""
199 |     if not token:
200 |         auth_header = request.headers.get("Authorization", "")
201 |         if auth_header.startswith("Bearer "):
202 |             token = auth_header[7:]
203 |     if not token:
204 |         raise HTTPException(status_code=401, detail="No refresh token")
205 |     try:
206 |         payload = jwt.decode(token, get_jwt_secret(), algorithms=[JWT_ALGORITHM])
207 |         if payload.get("type") != "refresh":
208 |             raise HTTPException(status_code=401, detail="Invalid token type")
209 |         user = await db.users.find_one({"_id": ObjectId(payload["sub"])})
210 |         if not user:
211 |             raise HTTPException(status_code=401, detail="User not found")
212 |         access = create_access_token(str(user["_id"]), user["email"])
213 |         return {"access_token": access}
214 |     except jwt.ExpiredSignatureError:
```

```

215 |         raise HTTPException(status_code=401, detail="Refresh token expired")
216 |     except jwt.InvalidTokenError:
217 |         raise HTTPException(status_code=401, detail="Invalid refresh token")
218 |
219 | @auth_router.post("/forgot-password")
220 | async def forgot_password(req: ForgotPasswordRequest):
221 |     email = req.email.strip().lower()
222 |     user = await db.users.find_one({"email": email})
223 |     if not user:
224 |         return {"message": "If that email exists, a reset link has been sent."}
225 |     token = secrets.token_urlsafe(32)
226 |     await db.password_reset_tokens.insert_one({
227 |         "token": token,
228 |         "user_id": user["_id"],
229 |         "expires_at": datetime.now(timezone.utc) + timedelta(hours=1),
230 |         "used": False,
231 |     })
232 |     print(f"[PASSWORD RESET] Token for {email}: {token}")
233 |     return {"message": "If that email exists, a reset link has been sent."}
234 |
235 | @auth_router.post("/reset-password")
236 | async def reset_password(req: ResetPasswordRequest):
237 |     record = await db.password_reset_tokens.find_one({"token": req.token, "used": False})
238 |     if not record or record["expires_at"] < datetime.now(timezone.utc):
239 |         raise HTTPException(status_code=400, detail="Invalid or expired reset token")
240 |     if len(req.new_password) < 6:
241 |         raise HTTPException(status_code=400, detail="Password must be at least 6 characters")
242 |     await db.users.update_one({"_id": record["user_id"]}, {"$set": {"password_hash": hash_password(req.new_password)}})
243 |     await db.password_reset_tokens.update_one({"_id": record["_id"]}, {"$set": {"used": True}})
244 |     return {"message": "Password reset successful"}
245 |
246 | # --- Admin Seeding ---
247 | OWNER_EMAIL = os.environ.get("OWNER_EMAIL", "managingdirector@redslateholdings.com")
248 | OWNER_PASSWORD = os.environ.get("OWNER_PASSWORD")
249 |
250 | async def seed_admin():
251 |     # Seed original admin
252 |     admin_email = os.environ.get("ADMIN_EMAIL", "admin@risedual.ai")
253 |     admin_password = os.environ.get("ADMIN_PASSWORD")
254 |     if not admin_password:
255 |         logging.warning("ADMIN_PASSWORD not set in .env, skipping admin seed")
256 |         return
257 |     existing = await db.users.find_one({"email": admin_email})
258 |     if existing is None:
259 |         await db.users.insert_one({
260 |             "email": admin_email,
261 |             "password_hash": hash_password(admin_password),
262 |             "name": "Admin",
263 |             "role": "admin",
264 |             "subscription_status": "pro",
265 |             "created_at": datetime.now(timezone.utc),
266 |         })
267 |     elif not verify_password(admin_password, existing["password_hash"]):
268 |         await db.users.update_one({"email": admin_email}, {"$set": {"password_hash": hash_password(admin_password)}})
269 |
270 |     # Seed REDSLATE owner
271 |     if not OWNER_PASSWORD:
272 |         logging.warning("OWNER_PASSWORD not set in .env, skipping owner seed")
273 |         return
274 |     existing_owner = await db.users.find_one({"email": OWNER_EMAIL})
275 |     if existing_owner is None:
276 |         await db.users.insert_one({
277 |             "email": OWNER_EMAIL,
278 |             "password_hash": hash_password(OWNER_PASSWORD),
279 |             "name": "REDSLATE",
280 |             "role": "owner",
281 |             "subscription_status": "pro",

```

```
282 |         "is_active": True,
283 |         "created_at": datetime.now(timezone.utc),
284 |     })
285 |     else:
286 |         updates = {"role": "owner", "subscription_status": "pro", "name": "REDSLATE"}
287 |         if not verify_password(OWNER_PASSWORD, existing_owner["password_hash"]):
288 |             updates["password_hash"] = hash_password(OWNER_PASSWORD)
289 |         await db.users.update_one({"email": OWNER_EMAIL}, {"$set": updates})
290 |
291 | async def create_indexes():
292 |     await db.users.create_index("email", unique=True)
293 |     await db.password_reset_tokens.create_index("expires_at", expireAfterSeconds=0)
294 |     await db.login_attempts.create_index("identifier")
295 |
296 | # --- Owner-only guard ---
297 | async def require_owner(request: Request):
298 |     user = await get_current_user(request)
299 |     if user.get("role") != "owner":
300 |         raise HTTPException(status_code=403, detail="Owner access required")
301 |     return user
302 |
303 | # --- Admin Routes (Owner Only) ---
304 | @auth_router.get("/admin/users")
305 | async def list_users(request: Request):
306 |     await require_owner(request)
307 |     cursor = db.users.find({}, {"password_hash": 0}).limit(200)
308 |     users = []
309 |     async for u in cursor:
310 |         u["_id"] = str(u["_id"])
311 |         users.append(u)
312 |     return {"users": users, "total": len(users)}
313 |
314 | @auth_router.post("/admin/users/{user_id}/activate")
315 | async def activate_user(user_id: str, request: Request):
316 |     await require_owner(request)
317 |     result = await db.users.update_one({"_id": ObjectId(user_id)}, {"$set": {"is_active": True}})
318 |     if result.matched_count == 0:
319 |         raise HTTPException(status_code=404, detail="User not found")
320 |     return {"message": "User activated"}
321 |
322 | @auth_router.post("/admin/users/{user_id}/deactivate")
323 | async def deactivate_user(user_id: str, request: Request):
324 |     await require_owner(request)
325 |     target = await db.users.find_one({"_id": ObjectId(user_id)})
326 |     if not target:
327 |         raise HTTPException(status_code=404, detail="User not found")
328 |     if target.get("role") == "owner":
329 |         raise HTTPException(status_code=403, detail="Cannot deactivate owner account")
330 |     await db.users.update_one({"_id": ObjectId(user_id)}, {"$set": {"is_active": False}})
331 |     return {"message": "User deactivated"}
332 |
333 | @auth_router.post("/admin/users/{user_id}/grant-pro")
334 | async def grant_pro(user_id: str, request: Request):
335 |     await require_owner(request)
336 |     result = await db.users.update_one({"_id": ObjectId(user_id)}, {"$set": {"subscription_status": "pro"}})
337 |     if result.matched_count == 0:
338 |         raise HTTPException(status_code=404, detail="User not found")
339 |     return {"message": "Pro access granted"}
340 |
341 | @auth_router.post("/admin/users/{user_id}/revoke-pro")
342 | async def revoke_pro(user_id: str, request: Request):
343 |     await require_owner(request)
344 |     target = await db.users.find_one({"_id": ObjectId(user_id)})
345 |     if not target:
346 |         raise HTTPException(status_code=404, detail="User not found")
347 |     if target.get("role") == "owner":
348 |         raise HTTPException(status_code=403, detail="Cannot revoke owner's Pro access")
```

```
349 |     await db.users.update_one({"_id": ObjectId(user_id)}, {"$set": {"subscription_status": "free"}})
350 |     return {"message": "Pro access revoked"}
351 |
```

FILE: backend/routes/digest.py

74 lines | 2436 characters

```
1 | """Digest routes: admin trigger, user opt-in/out, preview."""
2 | import logging
3 | from fastapi import APIRouter, HTTPException, Request
4 | from routes.auth import get_current_user
5 | from bson import ObjectId
6 |
7 | router = APIRouter(prefix="/api/digest")
8 |
9 | db = None
10 |
11 | def set_db(database):
12 |     global db
13 |     db = database
14 |
15 |
16 | @router.post("/trigger")
17 | async def trigger_digest(request: Request):
18 |     """Admin/owner only: manually trigger the daily digest (for testing)."""
19 |     user = await get_current_user(request)
20 |     if user.get("role") not in ("owner", "admin"):
21 |         raise HTTPException(status_code=403, detail="Admin access required")
22 |
23 |     from services.digest_service import send_daily_digest
24 |     result = await send_daily_digest(db)
25 |     return result
26 |
27 |
28 | @router.post("/opt-out")
29 | async def opt_out_digest(request: Request):
30 |     """User opts out of daily digest emails."""
31 |     user = await get_current_user(request)
32 |     await db.users.update_one(
33 |         {"_id": ObjectId(user["_id"])},
34 |         {"$set": {"digest_opt_out": True}}
35 |     )
36 |     return {"message": "Unsubscribed from daily digest"}
37 |
38 |
39 | @router.post("/opt-in")
40 | async def opt_in_digest(request: Request):
41 |     """User opts back in to daily digest emails."""
42 |     user = await get_current_user(request)
43 |     await db.users.update_one(
44 |         {"_id": ObjectId(user["_id"])},
45 |         {"$set": {"digest_opt_out": False}}
46 |     )
47 |     return {"message": "Subscribed to daily digest"}
48 |
49 |
50 | @router.get("/status")
51 | async def get_digest_status(request: Request):
52 |     """Check if user is subscribed to digest."""
53 |     user = await get_current_user(request)
54 |     return {"subscribed": not user.get("digest_opt_out", False)}
55 |
56 |
57 | @router.get("/preview")
58 | async def preview_digest(request: Request):
59 |     """Admin/owner: preview digest HTML without sending."""
```

```

60 |     user = await get_current_user(request)
61 |     if user.get("role") not in ("owner", "admin"):
62 |         raise HTTPException(status_code=403, detail="Admin access required")
63 |
64 |     from services.digest_service import collect_digest_data, build_digest_html
65 |     data = await collect_digest_data(db)
66 |     is_pro = user.get("subscription_status") in ("pro", "trial")
67 |     name = user.get("name", user.get("email", "").split("@")[0])
68 |     html = build_digest_html(data, is_pro, name)
69 |     return {"html": html, "data_summary": {
70 |         "predictions": len(data["predictions"]),
71 |         "dark_pool": len(data["dark_pool"]),
72 |         "signals": len(data["signals"]),
73 |     }}
74 |

```

FILE: backend/routes/journal.py

254 lines | 8776 characters

```

1 | """Trading Journal routes: trade CRUD, analytics, hypothesis attachment."""
2 | import logging
3 | from datetime import datetime, timezone
4 | from bson import ObjectId
5 | from fastapi import APIRouter, HTTPException, Request
6 | from pydantic import BaseModel
7 | from typing import Optional
8 | from routes.auth import get_current_user
9 |
10 | router = APIRouter(prefix="/api/journal")
11 |
12 | db = None
13 | FREE_TRADE_LIMIT = 5
14 |
15 | def set_db(database):
16 |     global db
17 |     db = database
18 |
19 |
20 | def is_pro_user(user: dict) -> bool:
21 |     return user.get("subscription_status") in ("pro", "trial")
22 |
23 |
24 | class TradeCreate(BaseModel):
25 |     ticker: str
26 |     side: str # "buy" or "sell"
27 |     entry_price: float
28 |     quantity: float
29 |     entry_date: str
30 |     notes: str = ""
31 |     exit_price: Optional[float] = None
32 |     exit_date: Optional[str] = None
33 |
34 |
35 | class TradeUpdate(BaseModel):
36 |     exit_price: Optional[float] = None
37 |     exit_date: Optional[str] = None
38 |     notes: Optional[str] = None
39 |
40 |
41 | def trade_response(doc: dict) -> dict:
42 |     pnl = 0
43 |     pnl_pct = 0
44 |     status = "open"
45 |     if doc.get("exit_price") is not None and doc.get("entry_price"):
46 |         if doc["side"] == "buy":
47 |             pnl = (doc["exit_price"] - doc["entry_price"]) * doc["quantity"]

```

```

48 |         pnl_pct = ((doc["exit_price"] - doc["entry_price"]) / doc["entry_price"]) * 100
49 |     else:
50 |         pnl = (doc["entry_price"] - doc["exit_price"]) * doc["quantity"]
51 |         pnl_pct = ((doc["entry_price"] - doc["exit_price"]) / doc["entry_price"]) * 100
52 |         status = "closed"
53 |     return {
54 |         "id": str(doc["_id"]),
55 |         "ticker": doc["ticker"],
56 |         "side": doc["side"],
57 |         "entry_price": doc["entry_price"],
58 |         "exit_price": doc.get("exit_price"),
59 |         "quantity": doc["quantity"],
60 |         "entry_date": doc.get("entry_date", ""),
61 |         "exit_date": doc.get("exit_date"),
62 |         "notes": doc.get("notes", ""),
63 |         "status": status,
64 |         "pnl": round(pnl, 2),
65 |         "pnl_percent": round(pnl_pct, 2),
66 |         "hypothesis": doc.get("hypothesis"),
67 |         "created_at": doc.get("created_at", ""),
68 |     }
69 |
70 |
71 | @router.post("/trade")
72 | async def create_trade(req: TradeCreate, request: Request):
73 |     """Log a new trade."""
74 |     user = await get_current_user(request)
75 |     user_id = user["_id"]
76 |
77 |     # Free user trade limit
78 |     if not is_pro_user(user):
79 |         count = await db.trades.count_documents({"user_id": user_id})
80 |         if count >= FREE_TRADE_LIMIT:
81 |             raise HTTPException(status_code=403, detail=f"Free accounts limited to {FREE_TRADE_LIMIT} trades. Upgrade to Pro for u ...
82 |
83 |     doc = {
84 |         "user_id": user_id,
85 |         "ticker": req.ticker.upper().strip(),
86 |         "side": req.side.lower(),
87 |         "entry_price": req.entry_price,
88 |         "exit_price": req.exit_price,
89 |         "quantity": req.quantity,
90 |         "entry_date": req.entry_date,
91 |         "exit_date": req.exit_date,
92 |         "notes": req.notes,
93 |         "hypothesis": None,
94 |         "created_at": datetime.now(timezone.utc).isoformat(),
95 |     }
96 |     result = await db.trades.insert_one(doc)
97 |     doc["_id"] = result.inserted_id
98 |     return trade_response(doc)
99 |
100 |
101 | @router.get("/trades")
102 | async def get_trades(request: Request, skip: int = 0, limit: int = 50):
103 |     """Get user's trades, newest first."""
104 |     user = await get_current_user(request)
105 |     cursor = db.trades.find({"user_id": user["_id"]}).sort("created_at", -1).skip(skip).limit(limit)
106 |     trades = []
107 |     async for doc in cursor:
108 |         trades.append(trade_response(doc))
109 |     total = await db.trades.count_documents({"user_id": user["_id"]})
110 |     return {
111 |         "trades": trades,
112 |         "total": total,
113 |         "limit": FREE_TRADE_LIMIT if not is_pro_user(user) else -1,
114 |     }

```

```

115 |
116 |
117 | @router.put("/trade/{trade_id}")
118 | async def update_trade(trade_id: str, req: TradeUpdate, request: Request):
119 |     """Update a trade (close it, add notes)."""
120 |     user = await get_current_user(request)
121 |     trade = await db.trades.find_one({"_id": ObjectId(trade_id), "user_id": user["_id"]})
122 |     if not trade:
123 |         raise HTTPException(status_code=404, detail="Trade not found")
124 |
125 |     updates = {}
126 |     if req.exit_price is not None:
127 |         updates["exit_price"] = req.exit_price
128 |     if req.exit_date is not None:
129 |         updates["exit_date"] = req.exit_date
130 |     if req.notes is not None:
131 |         updates["notes"] = req.notes
132 |
133 |     if updates:
134 |         await db.trades.update_one({"_id": ObjectId(trade_id)}, {"$set": updates})
135 |
136 |     updated = await db.trades.find_one({"_id": ObjectId(trade_id)})
137 |     return trade_response(updated)
138 |
139 |
140 | @router.delete("/trade/{trade_id}")
141 | async def delete_trade(trade_id: str, request: Request):
142 |     """Delete a trade."""
143 |     user = await get_current_user(request)
144 |     result = await db.trades.delete_one({"_id": ObjectId(trade_id), "user_id": user["_id"]})
145 |     if result.deleted_count == 0:
146 |         raise HTTPException(status_code=404, detail="Trade not found")
147 |     return {"message": "Trade deleted"}
148 |
149 |
150 | @router.post("/trade/{trade_id}/attach-hypothesis")
151 | async def attach_hypothesis(trade_id: str, request: Request):
152 |     """Attach the latest AI hypothesis for the trade's ticker."""
153 |     user = await get_current_user(request)
154 |     trade = await db.trades.find_one({"_id": ObjectId(trade_id), "user_id": user["_id"]})
155 |     if not trade:
156 |         raise HTTPException(status_code=404, detail="Trade not found")
157 |
158 |     # Find latest hypothesis for this ticker
159 |     hypothesis = await db.hypothesis_history.find_one(
160 |         {"user_id": user["_id"], "ticker": trade["ticker"]},
161 |         sort=[("created_at", -1)]
162 |     )
163 |     if not hypothesis:
164 |         raise HTTPException(status_code=404, detail=f"No AI hypothesis found for {trade['ticker']}")
165 |
166 |     hyp_data = {
167 |         "verdict": hypothesis.get("verdict", "N/A"),
168 |         "confidence": hypothesis.get("confidence", 0),
169 |         "summary": hypothesis.get("summary", ""),
170 |         "attached_at": datetime.now(timezone.utc).isoformat(),
171 |     }
172 |     await db.trades.update_one({"_id": ObjectId(trade_id)}, {"$set": {"hypothesis": hyp_data}})
173 |
174 |     updated = await db.trades.find_one({"_id": ObjectId(trade_id)})
175 |     return trade_response(updated)
176 |
177 |
178 | @router.get("/analytics")
179 | async def get_analytics(request: Request):
180 |     """Get trading performance analytics."""
181 |     user = await get_current_user(request)

```

```
182 |     user_id = user["_id"]
183 |
184 |     cursor = db.trades.find({"user_id": user_id})
185 |     trades = []
186 |     async for doc in cursor:
187 |         trades.append(trade_response(doc))
188 |
189 |     closed = [t for t in trades if t["status"] == "closed"]
190 |     open_trades = [t for t in trades if t["status"] == "open"]
191 |
192 |     if not closed:
193 |         return {
194 |             "total_trades": len(trades),
195 |             "open_trades": len(open_trades),
196 |             "closed_trades": 0,
197 |             "total_pnl": 0,
198 |             "win_rate": 0,
199 |             "avg_gain": 0,
200 |             "avg_loss": 0,
201 |             "best_trade": None,
202 |             "worst_trade": None,
203 |             "by_ticker": {},
204 |             "pnl_timeline": [],
205 |         }
206 |
207 |     total_pnl = sum(t["pnl"] for t in closed)
208 |     wins = [t for t in closed if t["pnl"] > 0]
209 |     losses = [t for t in closed if t["pnl"] <= 0]
210 |     win_rate = (len(wins) / len(closed)) * 100 if closed else 0
211 |     avg_gain = sum(t["pnl"] for t in wins) / len(wins) if wins else 0
212 |     avg_loss = sum(t["pnl"] for t in losses) / len(losses) if losses else 0
213 |
214 |     best = max(closed, key=lambda t: t["pnl"])
215 |     worst = min(closed, key=lambda t: t["pnl"])
216 |
217 |     # Performance by ticker
218 |     by_ticker = {}
219 |     for t in closed:
220 |         tk = t["ticker"]
221 |         if tk not in by_ticker:
222 |             by_ticker[tk] = {"pnl": 0, "trades": 0, "wins": 0}
223 |             by_ticker[tk]["pnl"] = round(by_ticker[tk]["pnl"] + t["pnl"], 2)
224 |             by_ticker[tk]["trades"] += 1
225 |             if t["pnl"] > 0:
226 |                 by_ticker[tk]["wins"] += 1
227 |
228 |     # P&L timeline (cumulative by exit date)
229 |     sorted_closed = sorted(closed, key=lambda t: t.get("exit_date") or t.get("entry_date") or "")
230 |     cumulative = 0
231 |     timeline = []
232 |     for t in sorted_closed:
233 |         cumulative += t["pnl"]
234 |         timeline.append({
235 |             "date": t.get("exit_date") or t.get("entry_date", ""),
236 |             "pnl": round(t["pnl"], 2),
237 |             "cumulative": round(cumulative, 2),
238 |             "ticker": t["ticker"],
239 |         })
240 |
241 |     return {
242 |         "total_trades": len(trades),
243 |         "open_trades": len(open_trades),
244 |         "closed_trades": len(closed),
245 |         "total_pnl": round(total_pnl, 2),
246 |         "win_rate": round(win_rate, 1),
247 |         "avg_gain": round(avg_gain, 2),
248 |         "avg_loss": round(avg_loss, 2),
```



```
249 |         "best_trade": {"ticker": best["ticker"], "pnl": best["pnl"], "pnl_percent": best["pnl_percent"]},
250 |         "worst_trade": {"ticker": worst["ticker"], "pnl": worst["pnl"], "pnl_percent": worst["pnl_percent"]},
251 |         "by_ticker": by_ticker,
252 |         "pnl_timeline": timeline,
253 |     }
254 |
```

FILE: backend/routes/market.py*113 lines | 3742 characters*

```
1 | """Market data routes: stocks, crypto, dark pool, options."""
2 | from fastapi import APIRouter, HTTPException
3 | import logging
4 |
5 | from services.market_data_service import MarketDataService
6 |
7 | router = APIRouter(prefix="/api")
8 | market_service = MarketDataService()
9 |
10 |
11 | # --- Stock Market ---
12 | @router.get("/stocks/ticker")
13 | async def get_ticker():
14 |     try:
15 |         return await market_service.get_ticker_data()
16 |     except Exception as e:
17 |         logging.error(f"Error fetching ticker data: {e}")
18 |         raise HTTPException(status_code=500, detail="Error fetching ticker data")
19 |
20 |
21 | @router.get("/stocks/quote/{symbol}")
22 | async def get_quote(symbol: str):
23 |     try:
24 |         quote = await market_service.get_quote(symbol)
25 |         if not quote:
26 |             raise HTTPException(status_code=404, detail=f"Quote not found for {symbol}")
27 |         return quote
28 |     except HTTPException:
29 |         raise
30 |     except Exception as e:
31 |         logging.error(f"Error fetching quote for {symbol}: {e}")
32 |         raise HTTPException(status_code=500, detail="Error fetching quote")
33 |
34 |
35 | # --- Options ---
36 | @router.get("/options/radar")
37 | async def get_options_radar():
38 |     try:
39 |         return market_service.generate_mock_options_data('radar')
40 |     except Exception as e:
41 |         logging.error(f"Error fetching options radar: {e}")
42 |         raise HTTPException(status_code=500, detail="Error fetching options radar")
43 |
44 |
45 | @router.get("/options/flow")
46 | async def get_options_flow():
47 |     try:
48 |         return market_service.generate_mock_options_data('flow')
49 |     except Exception as e:
50 |         logging.error(f"Error fetching options flow: {e}")
51 |         raise HTTPException(status_code=500, detail="Error fetching options flow")
52 |
53 |
54 | @router.get("/options/momentum")
55 | async def get_momentum():
56 |     try:
```

```
57 |         return market_service.generate_mock_options_data('momentum')
58 |     except Exception as e:
59 |         logging.error(f"Error fetching momentum data: {e}")
60 |         raise HTTPException(status_code=500, detail="Error fetching momentum data")
61 |
62 |
63 | @router.get("/options/fast-movers")
64 | async def get_fast_movers():
65 |     try:
66 |         return market_service.generate_mock_options_data('fast_movers')
67 |     except Exception as e:
68 |         logging.error(f"Error fetching fast movers: {e}")
69 |         raise HTTPException(status_code=500, detail="Error fetching fast movers")
70 |
71 |
72 | @router.get("/options/unusual-volume")
73 | async def get_unusual_volume():
74 |     try:
75 |         return market_service.generate_mock_options_data('unusual_volume')
76 |     except Exception as e:
77 |         logging.error(f"Error fetching unusual volume: {e}")
78 |         raise HTTPException(status_code=500, detail="Error fetching unusual volume")
79 |
80 |
81 | # --- Crypto ---
82 | @router.get("/crypto/prices")
83 | async def get_crypto_prices():
84 |     try:
85 |         return await market_service.get_crypto_data()
86 |     except Exception as e:
87 |         logging.error(f"Error fetching crypto data: {e}")
88 |         raise HTTPException(status_code=500, detail="Error fetching crypto data")
89 |
90 |
91 | @router.get("/crypto/{symbol}")
92 | async def get_crypto_by_symbol(symbol: str):
93 |     try:
94 |         crypto = await market_service.get_crypto_quote(symbol)
95 |         if not crypto:
96 |             raise HTTPException(status_code=404, detail=f"Crypto not found for {symbol}")
97 |         return crypto
98 |     except HTTPException:
99 |         raise
100 |     except Exception as e:
101 |         logging.error(f"Error fetching crypto {symbol}: {e}")
102 |         raise HTTPException(status_code=500, detail="Error fetching crypto")
103 |
104 |
105 | # --- Dark Pool ---
106 | @router.get("/dark-pool")
107 | async def get_dark_pool():
108 |     try:
109 |         return market_service.generate_dark_pool_data()
110 |     except Exception as e:
111 |         logging.error(f"Error fetching dark pool data: {e}")
112 |         raise HTTPException(status_code=500, detail="Error fetching dark pool data")
113 |
```

FILE: backend/routes/promo.py

152 lines | 4932 characters

```
1 | """Promo campaign routes: admin CRUD + public active promo + user progress."""
2 | import logging
3 | from datetime import datetime, timezone
4 | from bson import ObjectId
5 | from fastapi import APIRouter, HTTPException, Request
```

```
6 | from pydantic import BaseModel
7 | from typing import Optional
8 | from routes.auth import get_current_user
9 |
10 | router = APIRouter(prefix="/api/promo")
11 |
12 | db = None
13 |
14 | def set_db(database):
15 |     global db
16 |     db = database
17 |
18 |
19 | class PromoCreate(BaseModel):
20 |     title: str
21 |     message: str
22 |     referral_target: int = 3
23 |     reward_months: int = 2
24 |     start_date: str
25 |     end_date: str
26 |
27 |
28 | def promo_response(doc: dict) -> dict:
29 |     return {
30 |         "id": str(doc["_id"]),
31 |         "title": doc["title"],
32 |         "message": doc["message"],
33 |         "referral_target": doc["referral_target"],
34 |         "reward_months": doc["reward_months"],
35 |         "start_date": doc["start_date"],
36 |         "end_date": doc["end_date"],
37 |         "is_active": doc.get("is_active", False),
38 |         "created_at": doc.get("created_at", ""),
39 |     }
40 |
41 |
42 | @router.get("/active")
43 | async def get_active_promo():
44 |     """Public: get the currently active promo (if any)."""
45 |     now = datetime.now(timezone.utc).isoformat()
46 |     promo = await db.promos.find_one({
47 |         "is_active": True,
48 |         "start_date": {"$lte": now},
49 |         "end_date": {"$gte": now},
50 |     })
51 |     if not promo:
52 |         return {"promo": None}
53 |     return {"promo": promo_response(promo)}
54 |
55 |
56 | @router.get("/progress")
57 | async def get_promo_progress(request: Request):
58 |     """Authenticated: user's progress toward the active promo."""
59 |     user = await get_current_user(request)
60 |     user_id = user["_id"]
61 |
62 |     now = datetime.now(timezone.utc).isoformat()
63 |     promo = await db.promos.find_one({
64 |         "is_active": True,
65 |         "start_date": {"$lte": now},
66 |         "end_date": {"$gte": now},
67 |     })
68 |     if not promo:
69 |         return {"promo": None, "progress": 0, "target": 0, "completed": False}
70 |
71 |     # Count referrals completed during promo period
72 |     referrals_in_period = await db.referrals.count_documents({
```

```
73 |         "referrer_id": user_id,
74 |         "status": "completed",
75 |         "completed_at": {"$gte": promo["start_date"], "$lte": promo["end_date"]},
76 |     })
77 |
78 |     target = promo["referral_target"]
79 |     return {
80 |         "promo": promo_response(promo),
81 |         "progress": referrals_in_period,
82 |         "target": target,
83 |         "completed": referrals_in_period >= target,
84 |     }
85 |
86 |
87 | @router.get("/all")
88 | async def get_all_promos(request: Request):
89 |     """Admin/owner only: list all promos."""
90 |     user = await get_current_user(request)
91 |     if user.get("role") not in ("owner", "admin"):
92 |         raise HTTPException(status_code=403, detail="Admin access required")
93 |
94 |     cursor = db.promos.find({}).sort("created_at", -1)
95 |     promos = []
96 |     async for doc in cursor:
97 |         promos.append(promo_response(doc))
98 |     return {"promos": promos}
99 |
100 |
101 | @router.post("/create")
102 | async def create_promo(req: PromoCreate, request: Request):
103 |     """Admin/owner only: create a new promo campaign."""
104 |     user = await get_current_user(request)
105 |     if user.get("role") not in ("owner", "admin"):
106 |         raise HTTPException(status_code=403, detail="Admin access required")
107 |
108 |     doc = {
109 |         "title": req.title.strip(),
110 |         "message": req.message.strip(),
111 |         "referral_target": req.referral_target,
112 |         "reward_months": req.reward_months,
113 |         "start_date": req.start_date,
114 |         "end_date": req.end_date,
115 |         "is_active": True,
116 |         "created_at": datetime.now(timezone.utc).isoformat(),
117 |         "created_by": user["_id"],
118 |     }
119 |     result = await db.promos.insert_one(doc)
120 |     doc["_id"] = result.inserted_id
121 |     logging.info(f"Promo created: {req.title} by {user.get('email')}")
122 |     return promo_response(doc)
123 |
124 |
125 | @router.put("/{promo_id}/toggle")
126 | async def toggle_promo(promo_id: str, request: Request):
127 |     """Admin/owner only: toggle promo active status."""
128 |     user = await get_current_user(request)
129 |     if user.get("role") not in ("owner", "admin"):
130 |         raise HTTPException(status_code=403, detail="Admin access required")
131 |
132 |     promo = await db.promos.find_one({"_id": ObjectId(promo_id)})
133 |     if not promo:
134 |         raise HTTPException(status_code=404, detail="Promo not found")
135 |
136 |     new_status = not promo.get("is_active", False)
137 |     await db.promos.update_one({"_id": ObjectId(promo_id)}, {"$set": {"is_active": new_status}})
138 |     return {"id": promo_id, "is_active": new_status}
139 |
```

```
140 |
141 | @router.delete("/{promo_id}")
142 | async def delete_promo(promo_id: str, request: Request):
143 |     """Admin/owner only: delete a promo."""
144 |     user = await get_current_user(request)
145 |     if user.get("role") not in ("owner", "admin"):
146 |         raise HTTPException(status_code=403, detail="Admin access required")
147 |
148 |     result = await db.promos.delete_one({"_id": ObjectId(promo_id)})
149 |     if result.deleted_count == 0:
150 |         raise HTTPException(status_code=404, detail="Promo not found")
151 |     return {"message": "Promo deleted"}
152 |
```

FILE: backend/routes/push.py*109 lines | 3470 characters*

```
1 | """Push notification routes: subscribe, unsubscribe, status, test."""
2 | import logging
3 | from fastapi import APIRouter, HTTPException, Request
4 | from pydantic import BaseModel
5 | from bson import ObjectId
6 | from datetime import datetime, timezone
7 | from routes.auth import get_current_user
8 |
9 | router = APIRouter(prefix="/api/push")
10 |
11 | db = None
12 |
13 | def set_db(database):
14 |     global db
15 |     db = database
16 |
17 |
18 | class PushSubscription(BaseModel):
19 |     subscription: dict
20 |
21 |
22 | @router.post("/subscribe")
23 | async def subscribe_push(req: PushSubscription, request: Request):
24 |     """Store user's push subscription for notifications."""
25 |     user = await get_current_user(request)
26 |     user_id = user["_id"]
27 |     is_pro = user.get("subscription_status") in ("pro", "trial")
28 |
29 |     # Upsert subscription
30 |     await db.push_subscriptions.update_one(
31 |         {"user_id": user_id},
32 |         {"$set": {
33 |             "user_id": user_id,
34 |             "subscription": req.subscription,
35 |             "is_pro": is_pro,
36 |             "subscribed_at": datetime.now(timezone.utc).isoformat(),
37 |         }},
38 |         upsert=True
39 |     )
40 |     return {"message": "Push notifications enabled"}
41 |
42 |
43 | @router.post("/unsubscribe")
44 | async def unsubscribe_push(request: Request):
45 |     """Remove user's push subscription."""
46 |     user = await get_current_user(request)
47 |     await db.push_subscriptions.delete_one({"user_id": user["_id"]})
48 |     return {"message": "Push notifications disabled"}
49 |
```

```
50 |
51 | @router.get("/status")
52 | async def get_push_status(request: Request):
53 |     """Check if user has active push subscription."""
54 |     user = await get_current_user(request)
55 |     sub = await db.push_subscriptions.find_one({"user_id": user["_id"]}, {"_id": 0, "subscription": 0})
56 |     return {
57 |         "subscribed": sub is not None,
58 |         "is_pro": user.get("subscription_status") in ("pro", "trial"),
59 |     }
60 |
61 |
62 | @router.get("/vapid-key")
63 | async def get_vapid_key():
64 |     """Public: return VAPID public key for frontend subscription."""
65 |     import os
66 |     key = os.environ.get('VAPID_PUBLIC_KEY', '')
67 |     return {"public_key": key}
68 |
69 |
70 | @router.post("/test")
71 | async def test_push(request: Request):
72 |     """Admin only: send a test push notification."""
73 |     user = await get_current_user(request)
74 |     if user.get("role") not in ("owner", "admin"):
75 |         raise HTTPException(status_code=403, detail="Admin access required")
76 |
77 |     from services.push_service import send_push
78 |     sub = await db.push_subscriptions.find_one({"user_id": user["_id"]})
79 |     if not sub:
80 |         raise HTTPException(status_code=400, detail="No push subscription found. Enable notifications first.")
81 |
82 |     success = await send_push(
83 |         subscription_info=sub["subscription"],
84 |         title="RISEDUAL AI Test",
85 |         body="Push notifications are working!",
86 |         url="/",
87 |         tag="test",
88 |         db=db,
89 |         user_id=user["_id"],
90 |         is_pro=True,
91 |     )
92 |     return {"sent": success}
93 |
94 |
95 | @router.post("/broadcast")
96 | async def broadcast_push(request: Request):
97 |     """Admin only: broadcast a notification to all subscribers."""
98 |     user = await get_current_user(request)
99 |     if user.get("role") not in ("owner", "admin"):
100 |         raise HTTPException(status_code=403, detail="Admin access required")
101 |
102 |     body = await request.json()
103 |     title = body.get("title", "RISEDUAL AI Update")
104 |     message = body.get("message", "Check out the latest market signals")
105 |
106 |     from services.push_service import broadcast_notification
107 |     result = await broadcast_notification(db, title, message)
108 |     return result
109 |
```

FILE: backend/routes/referral.py

253 lines | 9162 characters

```
1 | """Referral routes: referral code management, tracking, and rewards."""
2 | import asyncio
```

```
3 | import secrets
4 | import string
5 | import logging
6 | from datetime import datetime, timezone, timedelta
7 | from fastapi import APIRouter, HTTPException, Request
8 | from routes.auth import get_current_user
9 |
10 | router = APIRouter(prefix="/api/referral")
11 |
12 | db = None
13 | REWARD_CAP = 12 # Max rewards per 12-month rolling window
14 | TRIAL_DAYS = 7
15 | LEADERBOARD_SIZE = 10
16 |
17 | def set_db(database):
18 |     global db
19 |     db = database
20 |
21 |
22 | def _generate_code(length=8):
23 |     chars = string.ascii_uppercase + string.digits
24 |     return ''.join(secrets.choice(chars) for _ in range(length))
25 |
26 |
27 | def _mask_name(name: str) -> str:
28 |     """Partial name masking: 'John Doe' -> 'J*** D*****'"""
29 |     if not name:
30 |         return "Anonymous"
31 |     parts = name.strip().split()
32 |     masked = []
33 |     for part in parts:
34 |         if len(part) <= 1:
35 |             masked.append(part[0] + "****")
36 |         else:
37 |             masked.append(part[0] + "*" * (len(part) - 1))
38 |     return " ".join(masked)
39 |
40 |
41 | @router.get("/info")
42 | async def get_referral_info(request: Request):
43 |     """Get user's referral code, link, and reward stats."""
44 |     user = await get_current_user(request)
45 |     user_id = user["_id"]
46 |
47 |     # Get or create referral code
48 |     ref_doc = await db.referral_codes.find_one({"user_id": user_id, {"_id": 0}})
49 |     if not ref_doc:
50 |         code = _generate_code()
51 |         while await db.referral_codes.find_one({"code": code}):
52 |             code = _generate_code()
53 |         ref_doc = {
54 |             "user_id": user_id,
55 |             "code": code,
56 |             "created_at": datetime.now(timezone.utc).isoformat(),
57 |         }
58 |         await db.referral_codes.insert_one(ref_doc)
59 |         ref_doc.pop("_id", None)
60 |
61 |     code = ref_doc["code"]
62 |
63 |     # Count completed referrals
64 |     total_referrals = await db.referrals.count_documents({"referrer_id": user_id})
65 |     completed_referrals = await db.referrals.count_documents({"referrer_id": user_id, "status": "completed"})
66 |
67 |     # Rolling 12-month reward count
68 |     twelve_months_ago = (datetime.now(timezone.utc) - timedelta(days=365)).isoformat()
69 |     rewards_this_year = await db.referrals.count_documents({
```

```
70 |         "referrer_id": user_id,
71 |         "status": "completed",
72 |         "reward_granted": True,
73 |         "completed_at": {"$gte": twelve_months_ago},
74 |     })
75 |
76 |     # Get recent referrals
77 |     cursor = db.referrals.find(
78 |         {"referrer_id": user_id, {"_id": 0, "referrer_id": 0}
79 |     ).sort("created_at", -1).limit(20)
80 |     referrals = []
81 |     async for doc in cursor:
82 |         referrals.append(doc)
83 |
84 |     return {
85 |         "code": code,
86 |         "total_referrals": total_referrals,
87 |         "completed_referrals": completed_referrals,
88 |         "rewards_earned": rewards_this_year,
89 |         "rewards_remaining": max(0, REWARD_CAP - rewards_this_year),
90 |         "reward_cap": REWARD_CAP,
91 |         "referrals": referrals,
92 |     }
93 |
94 |
95 | @router.get("/validate/{code}")
96 | async def validate_referral_code(code: str):
97 |     """Check if a referral code is valid (used during registration)."""
98 |     ref = await db.referral_codes.find_one({"code": code.upper(), {"_id": 0})
99 |     if not ref:
100 |         return {"valid": False}
101 |     return {"valid": True, "code": ref["code"]}
102 |
103 |
104 | async def process_referral_signup(referred_user_id: str, referred_email: str, ref_code: str):
105 |     """Called after registration when a ref code is provided.
106 |     Sets the new user on a 7-day Pro trial and creates a pending referral."""
107 |     ref_doc = await db.referral_codes.find_one({"code": ref_code.upper()})
108 |     if not ref_doc:
109 |         return
110 |
111 |     referrer_id = ref_doc["user_id"]
112 |
113 |     # Don't allow self-referral
114 |     if referrer_id == referred_user_id:
115 |         return
116 |
117 |     # Check if already referred
118 |     existing = await db.referrals.find_one({"referred_id": referred_user_id})
119 |     if existing:
120 |         return
121 |
122 |     # Create pending referral
123 |     await db.referrals.insert_one({
124 |         "referrer_id": referrer_id,
125 |         "referred_id": referred_user_id,
126 |         "referred_email": referred_email,
127 |         "status": "pending",
128 |         "reward_granted": False,
129 |         "created_at": datetime.now(timezone.utc).isoformat(),
130 |         "completed_at": None,
131 |     })
132 |
133 |     # Grant 7-day Pro trial to referred user
134 |     trial_end = (datetime.now(timezone.utc) + timedelta(days=TRIAL_DAYS)).isoformat()
135 |     await db.users.update_one(
136 |         {"_id": referred_user_id} if not isinstance(referred_user_id, str) else {"email": referred_email},
```



```

137 |         {"$set": {
138 |             "subscription_status": "trial",
139 |             "trial_ends_at": trial_end,
140 |             "referred_by": referrer_id,
141 |         }}
142 |     )
143 |     logging.info(f"Referral: {referred_email} signed up via code {ref_code}, 7-day trial granted")
144 |
145 |     # Send email notifications (non-blocking, fire-and-forget)
146 |     try:
147 |         from services.email_service import send_referral_signup_email, send_welcome_referral_email
148 |         from bson import ObjectId
149 |         referrer = await db.users.find_one(
150 |             {"_id": ObjectId(referrer_id) if isinstance(referrer_id, str) else referrer_id},
151 |             {"name": 1, "email": 1}
152 |         )
153 |         if referrer:
154 |             r_name = referrer.get("name", referrer.get("email", "").split("@")[0])
155 |             r_email = referrer.get("email", "")
156 |             u_name = referred_email.split("@")[0]
157 |             asyncio.create_task(send_referral_signup_email(r_email, r_name, referred_email))
158 |             asyncio.create_task(send_welcome_referral_email(referred_email, u_name, r_name))
159 |     except Exception as e:
160 |         logging.warning(f"Email notification error (signup): {e}")
161 |
162 |
163 | async def complete_referral_reward(referred_user_id: str):
164 |     """Called when a referred user subscribes to Pro.
165 |     Grants the referrer a reward month if under the 12/12 cap."""
166 |     referral = await db.referrals.find_one({"referred_id": referred_user_id, "status": "pending"})
167 |     if not referral:
168 |         return
169 |
170 |     referrer_id = referral["referrer_id"]
171 |
172 |     # Check 12-month rolling cap
173 |     twelve_months_ago = (datetime.now(timezone.utc) - timedelta(days=365)).isoformat()
174 |     rewards_count = await db.referrals.count_documents({
175 |         "referrer_id": referrer_id,
176 |         "status": "completed",
177 |         "reward_granted": True,
178 |         "completed_at": {"$gte": twelve_months_ago},
179 |     })
180 |
181 |     reward_granted = rewards_count < REWARD_CAP
182 |
183 |     await db.referrals.update_one(
184 |         {"_id": referral["_id"]},
185 |         {"$set": {
186 |             "status": "completed",
187 |             "reward_granted": reward_granted,
188 |             "completed_at": datetime.now(timezone.utc).isoformat(),
189 |         }}
190 |     )
191 |
192 |     if reward_granted:
193 |         # Add reward month to referrer's account
194 |         await db.referral_rewards.insert_one({
195 |             "user_id": referrer_id,
196 |             "type": "free_month",
197 |             "from_referral": str(referral["_id"]),
198 |             "referred_email": referral["referred_email"],
199 |             "granted_at": datetime.now(timezone.utc).isoformat(),
200 |             "redeemed": False,
201 |         })
202 |         logging.info(f"Referral reward granted to {referrer_id} for {referral['referred_email']}")
203 |

```

```

204 |         # Send reward email to referrer (non-blocking)
205 |         try:
206 |             from services.email_service import send_reward_earned_email
207 |             from bson import ObjectId
208 |             referrer = await db.users.find_one({"_id": ObjectId(referrer_id) if isinstance(referrer_id, str) else referrer_id}, {"_id": 1, "email": 1})
209 |             if referrer:
210 |                 referrer_name = referrer.get("name", referrer.get("email", "").split("@")[0])
211 |                 asyncio.create_task(send_reward_earned_email(referrer["email"], referrer_name, referral["referred_email"]))
212 |             except Exception as e:
213 |                 logging.warning(f"Email notification error (reward): {e}")
214 |
215 |         return reward_granted
216 |
217 |
218 | @router.get("/leaderboard")
219 | async def get_referral_leaderboard():
220 |     """Public endpoint: top referrers with masked names."""
221 |     # Aggregate completed referrals per referrer
222 |     pipeline = [
223 |         {"$match": {"status": "completed"}},
224 |         {"$group": {"_id": "$referrer_id", "count": {"$sum": 1}}},
225 |         {"$sort": {"count": -1}},
226 |         {"$limit": LEADERBOARD_SIZE},
227 |     ]
228 |     results = []
229 |     async for doc in db.referrals.aggregate(pipeline):
230 |         results.append({"user_id": doc["_id"], "referral_count": doc["count"]})
231 |
232 |     # Fetch user names for masking
233 |     leaderboard = []
234 |     for i, entry in enumerate(results):
235 |         user = await db.users.find_one({"_id": entry["user_id"]}, {"name": 1, "email": 1})
236 |         if not user:
237 |             # Try string ID match
238 |             from bson import ObjectId
239 |             try:
240 |                 user = await db.users.find_one({"_id": ObjectId(entry["user_id"])}, {"name": 1, "email": 1})
241 |             except Exception:
242 |                 pass
243 |             name = user.get("name", "") if user else ""
244 |             if not name and user:
245 |                 name = user.get("email", "").split("@")[0]
246 |             leaderboard.append({
247 |                 "rank": i + 1,
248 |                 "name": _mask_name(name),
249 |                 "referrals": entry["referral_count"],
250 |             })
251 |
252 |     return {"leaderboard": leaderboard, "total_participants": await db.referral_codes.count_documents({})}
253 |

```

FILE: backend/routes/subscription.py

144 lines | 5202 characters

```

1 | """Subscription & payment routes: Stripe checkout, webhooks, status."""
2 | from fastapi import APIRouter, HTTPException, Request
3 | from pydantic import BaseModel
4 | import logging
5 | from datetime import datetime, timezone
6 |
7 | from routes.auth import get_current_user
8 |
9 | router = APIRouter(prefix="/api")
10 |
11 | # Module-level db reference, set by server.py on startup
12 | db = None

```

```
13 |
14 | def set_db(database):
15 |     global db
16 |     db = database
17 |
18 |
19 | class CheckoutRequest(BaseModel):
20 |     origin_url: str
21 |     plan: str = "monthly"
22 |
23 |
24 | @router.post("/subscription/create-checkout-session")
25 | async def create_checkout_session(request: CheckoutRequest, http_request: Request):
26 |     try:
27 |         from services.payment_service import StripePaymentService
28 |         payment_service = StripePaymentService()
29 |         host_url = str(http_request.base_url).rstrip('/')
30 |         webhook_url = f"{host_url}/api/webhook/stripe"
31 |
32 |         plan = request.plan if request.plan in ("monthly", "annual") else "monthly"
33 |         amount = 486.00 if plan == "annual" else 45.00
34 |
35 |         metadata = {"plan": f"risedualai_pro_{plan}", "source": "web_checkout"}
36 |
37 |         session = await payment_service.create_checkout_session(
38 |             origin_url=request.origin_url,
39 |             webhook_url=webhook_url,
40 |             metadata=metadata,
41 |             plan=plan
42 |         )
43 |
44 |         await db.payment_transactions.insert_one({
45 |             "session_id": session.session_id,
46 |             "amount": amount,
47 |             "currency": "usd",
48 |             "plan": f"risedualai_pro_{plan}",
49 |             "metadata": metadata,
50 |             "payment_status": "initiated",
51 |             "created_at": datetime.now(timezone.utc).isoformat()
52 |         })
53 |
54 |         return {"url": session.url, "session_id": session.session_id}
55 |     except Exception as e:
56 |         logging.error(f"Error creating checkout session: {e}")
57 |         raise HTTPException(status_code=500, detail=str(e))
58 |
59 |
60 | @router.get("/subscription/status/{session_id}")
61 | async def get_payment_status(session_id: str, http_request: Request):
62 |     try:
63 |         from services.payment_service import StripePaymentService
64 |         payment_service = StripePaymentService()
65 |         host_url = str(http_request.base_url).rstrip('/')
66 |         webhook_url = f"{host_url}/api/webhook/stripe"
67 |
68 |         status = await payment_service.get_checkout_status(session_id, webhook_url)
69 |
70 |         existing = await db.payment_transactions.find_one({"session_id": session_id})
71 |         if existing and existing.get("payment_status") != status.payment_status:
72 |             await db.payment_transactions.update_one(
73 |                 {"session_id": session_id},
74 |                 {"$set": {
75 |                     "payment_status": status.payment_status,
76 |                     "status": status.status,
77 |                     "updated_at": datetime.now(timezone.utc).isoformat()
78 |                 }}
79 |             )
```

```
80 |
81 |         return {
82 |             "status": status.status,
83 |             "payment_status": status.payment_status,
84 |             "amount_total": status.amount_total,
85 |             "currency": status.currency
86 |         }
87 |     except Exception as e:
88 |         logging.error(f"Error fetching payment status: {e}")
89 |         raise HTTPException(status_code=500, detail=str(e))
90 |
91 |
92 | @router.post("/webhook/stripe")
93 | async def stripe_webhook(request: Request):
94 |     try:
95 |         from services.payment_service import StripePaymentService
96 |         payment_service = StripePaymentService()
97 |         body = await request.body()
98 |         signature = request.headers.get("Stripe-Signature", "")
99 |         host_url = str(request.base_url).rstrip('/')
100 |         webhook_url = f"{host_url}/api/webhook/stripe"
101 |
102 |         webhook_response = await payment_service.handle_webhook(body, signature, webhook_url)
103 |
104 |         if webhook_response and webhook_response.session_id:
105 |             await db.payment_transactions.update_one(
106 |                 {"session_id": webhook_response.session_id},
107 |                 {"$set": {
108 |                     "payment_status": webhook_response.payment_status,
109 |                     "event_type": webhook_response.event_type,
110 |                     "updated_at": datetime.now(timezone.utc).isoformat()
111 |                 }}
112 |             )
113 |
114 |         return {"status": "ok"}
115 |     except Exception as e:
116 |         logging.error(f"Webhook error: {e}")
117 |         raise HTTPException(status_code=400, detail=str(e))
118 |
119 |
120 | @router.post("/user/subscription")
121 | async def update_subscription(request: Request):
122 |     user = await get_current_user(request)
123 |     body = await request.json()
124 |     status = body.get("status", "pro")
125 |     plan = body.get("plan", "monthly")
126 |     await db.users.update_one(
127 |         {"_id": __import__('bson').ObjectId(user["_id"])},
128 |         {"$set": {
129 |             "subscription_status": status,
130 |             "subscription_plan": plan,
131 |             "subscription_updated_at": datetime.now(timezone.utc).isoformat(),
132 |         }}
133 |     )
134 |
135 |     # If upgrading to pro, check for referral reward
136 |     if status == "pro":
137 |         try:
138 |             from routes.referral import complete_referral_reward
139 |             await complete_referral_reward(user["_id"])
140 |         except Exception as e:
141 |             logging.warning(f"Referral reward processing error: {e}")
142 |
143 |     return {"message": "Subscription updated", "status": status}
144 |
```

FILE: backend/routes/trading.py

114 lines | 4490 characters

```
1 | """Trading & broker integration routes."""
2 | from fastapi import APIRouter, HTTPException
3 | from typing import Dict
4 | import os
5 | import logging
6 |
7 | router = APIRouter(prefix="/api")
8 |
9 |
10 | def _get_alpaca_credentials():
11 |     return {
12 |         'api_key': os.environ.get('ALPACA_API_KEY'),
13 |         'api_secret': os.environ.get('ALPACA_API_SECRET'),
14 |         'paper': os.environ.get('ALPACA_PAPER_TRADING', 'true').lower() == 'true'
15 |     }
16 |
17 |
18 | @router.post("/broker/connect")
19 | async def connect_broker(broker_id: str, credentials: Dict):
20 |     try:
21 |         from services.broker_service import BrokerService
22 |         client = BrokerService.get_broker_client(broker_id, credentials)
23 |         account = client.get_account()
24 |         if not account:
25 |             raise HTTPException(status_code=400, detail="Failed to connect to broker")
26 |         return {
27 |             "status": "connected",
28 |             "broker_id": broker_id,
29 |             "account_id": account.get('account_number', 'N/A'),
30 |             "message": "Successfully connected to broker"
31 |         }
32 |     except Exception as e:
33 |         logging.error(f"Error connecting to broker: {e}")
34 |         raise HTTPException(status_code=500, detail=str(e))
35 |
36 |
37 | @router.get("/trading/account/{broker_id}")
38 | async def get_account_info(broker_id: str):
39 |     try:
40 |         from services.broker_service import BrokerService
41 |         client = BrokerService.get_broker_client(broker_id, _get_alpaca_credentials())
42 |         account = client.get_account()
43 |         if not account:
44 |             raise HTTPException(status_code=404, detail="Account not found")
45 |         return {
46 |             "broker": broker_id,
47 |             "account_id": account.get('account_number'),
48 |             "cash": float(account.get('cash', 0)),
49 |             "buying_power": float(account.get('buying_power', 0)),
50 |             "portfolio_value": float(account.get('portfolio_value', 0)),
51 |             "equity": float(account.get('equity', 0))
52 |         }
53 |     except Exception as e:
54 |         logging.error(f"Error fetching account: {e}")
55 |         raise HTTPException(status_code=500, detail="Error fetching account")
56 |
57 |
58 | @router.get("/trading/positions/{broker_id}")
59 | async def get_positions(broker_id: str):
60 |     try:
61 |         from services.broker_service import BrokerService
62 |         client = BrokerService.get_broker_client(broker_id, _get_alpaca_credentials())
63 |         return client.get_positions()
```

```

64 |     except Exception as e:
65 |         logging.error(f"Error fetching positions: {e}")
66 |         raise HTTPException(status_code=500, detail="Error fetching positions")
67 |
68 |
69 | @router.post("/trading/order/{broker_id}")
70 | async def place_order(broker_id: str, order: Dict):
71 |     try:
72 |         from services.broker_service import BrokerService
73 |         client = BrokerService.get_broker_client(broker_id, _get_alpaca_credentials())
74 |         result = client.place_order(
75 |             symbol=order.get('symbol'),
76 |             qty=order.get('quantity'),
77 |             side=order.get('side'),
78 |             order_type=order.get('type', 'market'),
79 |             time_in_force=order.get('time_in_force', 'day'),
80 |             limit_price=order.get('limit_price'),
81 |             stop_price=order.get('stop_price')
82 |         )
83 |         if not result:
84 |             raise HTTPException(status_code=400, detail="Failed to place order")
85 |         return result
86 |     except Exception as e:
87 |         logging.error(f"Error placing order: {e}")
88 |         raise HTTPException(status_code=500, detail=str(e))
89 |
90 |
91 | @router.get("/trading/orders/{broker_id}")
92 | async def get_orders(broker_id: str, status: str = 'all'):
93 |     try:
94 |         from services.broker_service import BrokerService
95 |         client = BrokerService.get_broker_client(broker_id, _get_alpaca_credentials())
96 |         return client.get_orders(status=status)
97 |     except Exception as e:
98 |         logging.error(f"Error fetching orders: {e}")
99 |         raise HTTPException(status_code=500, detail="Error fetching orders")
100 |
101 |
102 | @router.delete("/trading/order/{broker_id}/{order_id}")
103 | async def cancel_order(broker_id: str, order_id: str):
104 |     try:
105 |         from services.broker_service import BrokerService
106 |         client = BrokerService.get_broker_client(broker_id, _get_alpaca_credentials())
107 |         success = client.cancel_order(order_id)
108 |         if not success:
109 |             raise HTTPException(status_code=400, detail="Failed to cancel order")
110 |         return {"status": "cancelled", "order_id": order_id}
111 |     except Exception as e:
112 |         logging.error(f"Error cancelling order: {e}")
113 |         raise HTTPException(status_code=500, detail=str(e))
114 |

```

FILE: backend/routes/workspace.py

136 lines | 4693 characters

```

1 | """Workspace routes: watchlist, hypothesis history, notifications."""
2 | from fastapi import APIRouter, HTTPException, Request
3 | import logging
4 | from datetime import datetime, timezone
5 |
6 | from routes.auth import get_current_user
7 |
8 | router = APIRouter(prefix="/api")
9 |
10 | # Module-level db reference, set by server.py on startup
11 | db = None

```

```
12 |
13 | def set_db(database):
14 |     global db
15 |     db = database
16 |
17 |
18 | def is_pro_user(user: dict) -> bool:
19 |     """Check if user has Pro access (includes 'pro' and 'trial' status)."""
20 |     if not user:
21 |         return False
22 |     status = user.get("subscription_status", "free")
23 |     return status in ("pro", "trial")
24 |
25 |
26 | # --- Watchlist ---
27 | @router.get("/workspace/watchlist")
28 | async def get_watchlist(request: Request):
29 |     user = await get_current_user(request)
30 |     doc = await db.watchlists.find_one({"user_id": user["_id"]}, {"_id": 0})
31 |     return {"tickers": doc.get("tickers", []) if doc else []}
32 |
33 |
34 | FREE_WATCHLIST_LIMIT = 3
35 | FREE_CHAT_DAILY_LIMIT = 5
36 |
37 |
38 | @router.post("/workspace/watchlist/add")
39 | async def add_to_watchlist(request: Request):
40 |     user = await get_current_user(request)
41 |     body = await request.json()
42 |     ticker = body.get("ticker", "").upper().strip()
43 |     if not ticker:
44 |         raise HTTPException(status_code=400, detail="Ticker required")
45 |     # Enforce watchlist cap for free users
46 |     if not is_pro_user(user):
47 |         doc = await db.watchlists.find_one({"user_id": user["_id"]})
48 |         current = doc.get("tickers", []) if doc else []
49 |         if ticker not in current and len(current) >= FREE_WATCHLIST_LIMIT:
50 |             raise HTTPException(status_code=403, detail=f"Free accounts are limited to {FREE_WATCHLIST_LIMIT} watchlist tickers. U ...")
51 |         await db.watchlists.update_one(
52 |             {"user_id": user["_id"]},
53 |             {"$addToSet": {"tickers": ticker}, "$set": {"updated_at": datetime.now(timezone.utc).isoformat()}},
54 |             upsert=True
55 |         )
56 |     return {"message": f"{ticker} added to watchlist"}
57 |
58 |
59 | @router.post("/workspace/watchlist/remove")
60 | async def remove_from_watchlist(request: Request):
61 |     user = await get_current_user(request)
62 |     body = await request.json()
63 |     ticker = body.get("ticker", "").upper().strip()
64 |     await db.watchlists.update_one(
65 |         {"user_id": user["_id"]},
66 |         {"$pull": {"tickers": ticker}}
67 |     )
68 |     return {"message": f"{ticker} removed from watchlist"}
69 |
70 |
71 | # --- Hypothesis History ---
72 | @router.get("/workspace/history")
73 | async def get_hypothesis_history(request: Request):
74 |     user = await get_current_user(request)
75 |     cursor = db.hypothesis_history.find(
76 |         {"user_id": user["_id"]}, {"_id": 0}
77 |     ).sort("searched_at", -1).limit(20)
78 |     history = []
```

```
79 |     async for doc in cursor:
80 |         history.append(doc)
81 |     return {"history": history}
82 |
83 |
84 | @router.post("/workspace/history/save")
85 | async def save_hypothesis_history(request: Request):
86 |     user = await get_current_user(request)
87 |     body = await request.json()
88 |     symbol = body.get("symbol", "").upper().strip()
89 |     verdict = body.get("verdict", "")
90 |     confidence = body.get("confidence", 0)
91 |     if not symbol:
92 |         raise HTTPException(status_code=400, detail="Symbol required")
93 |     await db.hypothesis_history.insert_one({
94 |         "user_id": user["_id"],
95 |         "symbol": symbol,
96 |         "verdict": verdict,
97 |         "confidence": confidence,
98 |         "searched_at": datetime.now(timezone.utc).isoformat(),
99 |     })
100 |     return {"message": "Saved to history"}
101 |
102 |
103 | # --- Notifications (Pro Only) ---
104 | @router.get("/notifications")
105 | async def get_notifications(request: Request):
106 |     user = await get_current_user(request)
107 |     if not is_pro_user(user):
108 |         return {"notifications": [], "unread_count": 0, "is_pro": False}
109 |     cursor = db.notifications.find(
110 |         {"user_id": user["_id"]}, {"_id": 0}
111 |     ).sort("created_at", -1).limit(30)
112 |     notifications = []
113 |     async for doc in cursor:
114 |         notifications.append(doc)
115 |     unread = sum(1 for n in notifications if not n.get("read"))
116 |     return {"notifications": notifications, "unread_count": unread, "is_pro": True}
117 |
118 |
119 | @router.get("/notifications/unread-count")
120 | async def get_unread_count(request: Request):
121 |     user = await get_current_user(request)
122 |     if not is_pro_user(user):
123 |         return {"count": 0, "is_pro": False}
124 |     count = await db.notifications.count_documents({"user_id": user["_id"], "read": False})
125 |     return {"count": count, "is_pro": True}
126 |
127 |
128 | @router.post("/notifications/read-all")
129 | async def mark_all_read(request: Request):
130 |     user = await get_current_user(request)
131 |     await db.notifications.update_many(
132 |         {"user_id": user["_id"], "read": False},
133 |         {"$set": {"read": True}}
134 |     )
135 |     return {"message": "All notifications marked as read"}
136 |
```

Backend - Services

FILE: [backend/services/ai_service.py](#)

56 lines | 2530 characters


```
1 | import os
2 | import logging
3 | from typing import Optional
4 | from emergentintegrations.llm.chat import LlmChat, UserMessage, ImageContent
5 |
6 | logger = logging.getLogger(__name__)
7 |
8 | class AIService:
9 |     def __init__(self):
10 |         self.api_key = os.environ.get('EMERGENT_LLM_KEY')
11 |         self.system_message = """You are RISEDUALAI, an advanced AI-powered trading assistant.
12 |         You specialize in:
13 |         - Stock market analysis and insights
14 |         - Options trading strategies
15 |         - Technical and fundamental analysis
16 |         - Risk management
17 |         - Market trends and predictions
18 |         - Cryptocurrency market analysis
19 |         - Dark pool trading insights
20 |         - Analyzing stock charts, candlestick patterns, and technical indicators from uploaded images
21 |
22 |         When a user uploads an image of a chart or financial asset, analyze it thoroughly:
23 |         - Identify the asset/ticker if visible
24 |         - Describe chart patterns (head & shoulders, double top/bottom, triangles, etc.)
25 |         - Note support/resistance levels
26 |         - Identify trend direction and momentum
27 |         - Suggest potential trade setups based on what you see
28 |
29 |         Provide clear, actionable advice while always reminding users that trading involves risk.
30 |         Be professional, knowledgeable, and helpful. Use data-driven insights when possible.
31 |         Your responses should be informative yet concise."""
32 |
33 |     async def chat(self, message: str, session_id: str, image_base64: Optional[str] = None) -> str:
34 |         """Send a message to the AI and get a response, optionally with an image"""
35 |         try:
36 |             chat = LlmChat(
37 |                 api_key=self.api_key,
38 |                 session_id=session_id,
39 |                 system_message=self.system_message
40 |             ).with_model("openai", "gpt-5.2")
41 |
42 |             if image_base64:
43 |                 image_content = ImageContent(image_base64=image_base64)
44 |                 user_message = UserMessage(
45 |                     text=message or "Please analyze this chart/image and provide trading insights.",
46 |                     file_contents=[image_content]
47 |                 )
48 |             else:
49 |                 user_message = UserMessage(text=message)
50 |
51 |             response = await chat.send_message(user_message)
52 |             return response
53 |
54 |         except Exception as e:
55 |             logger.error(f"Error in AI chat: {str(e)}")
56 |             return "I apologize, but I'm experiencing technical difficulties. Please try again in a moment."
```

FILE: [backend/services/broker_service.py](#)

129 lines | 4713 characters

```
1 | import os
2 | import logging
3 | from typing import Optional, List, Dict
4 | import requests
5 | from datetime import datetime, timedelta
6 |
```

```
7 | logger = logging.getLogger(__name__)
8 |
9 | class AlpacaTradingService:
10 |     """Alpaca is the easiest broker to integrate - commission-free API-first trading"""
11 |
12 |     def __init__(self, api_key: str, api_secret: str, paper: bool = True):
13 |         self.api_key = api_key
14 |         self.api_secret = api_secret
15 |         self.base_url = 'https://paper-api.alpaca.markets' if paper else 'https://api.alpaca.markets'
16 |         self.headers = {
17 |             'APCA-API-KEY-ID': api_key,
18 |             'APCA-API-SECRET-KEY': api_secret
19 |         }
20 |
21 |     def get_account(self) -> Optional[Dict]:
22 |         """Get account information"""
23 |         try:
24 |             response = requests.get(f'{self.base_url}/v2/account', headers=self.headers)
25 |             response.raise_for_status()
26 |             return response.json()
27 |         except Exception as e:
28 |             logger.error(f"Error fetching account: {str(e)}")
29 |             return None
30 |
31 |     def get_positions(self) -> List[Dict]:
32 |         """Get all open positions"""
33 |         try:
34 |             response = requests.get(f'{self.base_url}/v2/positions', headers=self.headers)
35 |             response.raise_for_status()
36 |             return response.json()
37 |         except Exception as e:
38 |             logger.error(f"Error fetching positions: {str(e)}")
39 |             return []
40 |
41 |     def place_order(self, symbol: str, qty: int, side: str, order_type: str = 'market',
42 |                     time_in_force: str = 'day', limit_price: Optional[float] = None,
43 |                     stop_price: Optional[float] = None) -> Optional[Dict]:
44 |         """Place a trading order"""
45 |         try:
46 |             order_data = {
47 |                 'symbol': symbol.upper(),
48 |                 'qty': qty,
49 |                 'side': side.lower(),
50 |                 'type': order_type.lower(),
51 |                 'time_in_force': time_in_force.lower()
52 |             }
53 |
54 |             if limit_price:
55 |                 order_data['limit_price'] = limit_price
56 |             if stop_price:
57 |                 order_data['stop_price'] = stop_price
58 |
59 |             response = requests.post(
60 |                 f'{self.base_url}/v2/orders',
61 |                 headers=self.headers,
62 |                 json=order_data
63 |             )
64 |             response.raise_for_status()
65 |             return response.json()
66 |         except Exception as e:
67 |             logger.error(f"Error placing order: {str(e)}")
68 |             return None
69 |
70 |     def get_order(self, order_id: str) -> Optional[Dict]:
71 |         """Get order status"""
72 |         try:
73 |             response = requests.get(
```

```

74 |         f'{self.base_url}/v2/orders/{order_id}',
75 |         headers=self.headers
76 |     )
77 |     response.raise_for_status()
78 |     return response.json()
79 | except Exception as e:
80 |     logger.error(f"Error fetching order: {str(e)}")
81 |     return None
82 |
83 | def cancel_order(self, order_id: str) -> bool:
84 |     """Cancel an order"""
85 |     try:
86 |         response = requests.delete(
87 |             f'{self.base_url}/v2/orders/{order_id}',
88 |             headers=self.headers
89 |         )
90 |         response.raise_for_status()
91 |         return True
92 |     except Exception as e:
93 |         logger.error(f"Error cancelling order: {str(e)}")
94 |         return False
95 |
96 | def get_orders(self, status: str = 'all', limit: int = 50) -> List[Dict]:
97 |     """Get all orders"""
98 |     try:
99 |         params = {'status': status, 'limit': limit}
100 |         response = requests.get(
101 |             f'{self.base_url}/v2/orders',
102 |             headers=self.headers,
103 |             params=params
104 |         )
105 |         response.raise_for_status()
106 |         return response.json()
107 |     except Exception as e:
108 |         logger.error(f"Error fetching orders: {str(e)}")
109 |         return []
110 |
111 | class BrokerService:
112 |     """Main broker service that handles multiple brokers"""
113 |
114 |     @staticmethod
115 |     def get_broker_client(broker_id: str, credentials: Dict):
116 |         """Factory method to get appropriate broker client"""
117 |         if broker_id == 'alpaca':
118 |             return AlpacaTradingService(
119 |                 api_key=credentials.get('api_key'),
120 |                 api_secret=credentials.get('api_secret'),
121 |                 paper=credentials.get('paper', True)
122 |             )
123 |         # Add more brokers here:
124 |         # elif broker_id == 'td_ameritrade':
125 |         #     return TDAmeritradeTradingService(...)
126 |         # elif broker_id == 'interactive_brokers':
127 |         #     return InteractiveBrokersTradingService(...)
128 |         else:
129 |             raise ValueError(f"Unsupported broker: {broker_id}")

```

FILE: backend/services/company_research_service.py

153 lines | 6155 characters

```

1 | import os
2 | import logging
3 | import json
4 | from typing import Dict, Optional
5 | from services.market_data_service import MarketDataService
6 | from services.financial_scraping_service import FinancialScrapingService

```

```
7 | from services.ai_service import AIService
8 | import requests
9 |
10 | logger = logging.getLogger(__name__)
11 |
12 | class CompanyResearchService:
13 |     def __init__(self):
14 |         self.market_data = MarketDataService()
15 |         self.scrapers = FinancialScrapingService()
16 |         self.ai = AIService()
17 |         self.api_key = os.environ.get('ALPHA_VANTAGE_API_KEY')
18 |         self.base_url = 'https://www.alphavantage.co/query'
19 |
20 |     async def get_company_overview(self, symbol: str) -> Optional[Dict]:
21 |         """Fetch company fundamentals from Alpha Vantage OVERVIEW endpoint"""
22 |         try:
23 |             params = {
24 |                 'function': 'OVERVIEW',
25 |                 'symbol': symbol.upper(),
26 |                 'apikey': self.api_key
27 |             }
28 |             response = requests.get(self.base_url, params=params, timeout=10)
29 |             response.raise_for_status()
30 |             data = response.json()
31 |
32 |             if not data or 'Symbol' not in data:
33 |                 return None
34 |
35 |             return {
36 |                 'symbol': data.get('Symbol', symbol),
37 |                 'name': data.get('Name', ''),
38 |                 'description': data.get('Description', ''),
39 |                 'exchange': data.get('Exchange', ''),
40 |                 'sector': data.get('Sector', ''),
41 |                 'industry': data.get('Industry', ''),
42 |                 'market_cap': data.get('MarketCapitalization', ''),
43 |                 'pe_ratio': data.get('PERatio', ''),
44 |                 'forward_pe': data.get('ForwardPE', ''),
45 |                 'eps': data.get('EPS', ''),
46 |                 'dividend_yield': data.get('DividendYield', ''),
47 |                 'beta': data.get('Beta', ''),
48 |                 'week_52_high': data.get('52WeekHigh', ''),
49 |                 'week_52_low': data.get('52WeekLow', ''),
50 |                 'revenue_ttm': data.get('RevenueTTM', ''),
51 |                 'profit_margin': data.get('ProfitMargin', ''),
52 |                 'operating_margin': data.get('OperatingMarginTTM', ''),
53 |                 'return_on_equity': data.get('ReturnOnEquityTTM', ''),
54 |                 'price_to_book': data.get('PriceToBookRatio', ''),
55 |                 'price_to_sales': data.get('PriceToSalesRatioTTM', ''),
56 |                 'analyst_target': data.get('AnalystTargetPrice', ''),
57 |                 'analyst_rating': data.get('AnalystRatingStrongBuy', ''),
58 |                 'headquarters': data.get('Address', ''),
59 |                 'fiscal_year_end': data.get('FiscalYearEnd', ''),
60 |                 'ceo': data.get('CEO', data.get('OfficerTitle', '')),
61 |                 'employees': data.get('FullTimeEmployees', ''),
62 |             }
63 |         except Exception as e:
64 |             logger.error(f"Error fetching company overview for {symbol}: {str(e)}")
65 |             return None
66 |
67 |     async def research_company(self, symbol: str, session_id: str) -> Dict:
68 |         """Full Perplexity-style company research with AI synthesis"""
69 |         # Gather data in parallel-ish fashion
70 |         overview = await self.get_company_overview(symbol)
71 |         quote = await self.market_data.get_quote(symbol)
72 |         news_data = await self.scrapers.scrape_financial_news()
73 |
```

```

74 |         # Filter news relevant to this company
75 |         company_name = overview.get('name', symbol) if overview else symbol
76 |         relevant_news = []
77 |         all_articles = news_data if isinstance(news_data, list) else []
78 |         for article in all_articles:
79 |             title = article.get('title', '').lower()
80 |             if symbol.lower() in title or company_name.lower() in title:
81 |                 relevant_news.append(article)
82 |
83 |         # If no company-specific news, include top market news as context
84 |         if not relevant_news:
85 |             relevant_news = all_articles[:5]
86 |
87 |         # Build data package for AI synthesis
88 |         data_context = f"""
89 | COMPANY RESEARCH REQUEST: {symbol}
90 |
91 | === COMPANY FUNDAMENTALS (Alpha Vantage) ===
92 | {json.dumps(overview, indent=2) if overview else 'No fundamental data available.'}
93 |
94 | === REAL-TIME QUOTE ===
95 | {json.dumps(quote, indent=2) if quote else 'No quote data available.'}
96 |
97 | === RECENT NEWS ({len(relevant_news)} articles) ===
98 | """
99 |         sources = []
100 |        for i, article in enumerate(relevant_news[:8]):
101 |            src_num = i + 1
102 |            data_context += f"[{src_num}] {article.get('title', 'Untitled')} ? {article.get('source', 'Unknown')}\n"
103 |            sources.append({
104 |                'number': src_num,
105 |                'title': article.get('title', 'Untitled'),
106 |                'source': article.get('source', 'Unknown'),
107 |                'url': article.get('url', ''),
108 |            })
109 |
110 |        # AI synthesis prompt
111 |        synthesis_prompt = f"""You are a Perplexity-style financial research assistant. Given the data below, write a comprehensiv ...
112 |
113 | FORMAT YOUR RESPONSE EXACTLY LIKE THIS (use markdown):
114 |
115 | ## {company_name} ({symbol})
116 |
117 | **Company Overview**: [2-3 sentence summary of what the company does, sector, headquarters]
118 |
119 | **Key Executives**: CEO and leadership info if available.
120 |
121 | **Market Data**:
122 | - Market Cap: [value]
123 | - P/E Ratio: [value] | Forward P/E: [value]
124 | - EPS: [value]
125 | - Revenue (TTM): [value]
126 | - Dividend Yield: [value]
127 | - 52-Week Range: [low] - [high]
128 | - Beta: [value]
129 | - Analyst Target: [value]
130 |
131 | **Financial Health**: [2-3 sentences on margins, ROE, valuation metrics]
132 |
133 | **Recent Developments**: [Summarize relevant news, cite sources as [1], [2], etc.]
134 |
135 | **Outlook**: [2-3 sentence forward-looking analysis based on the data]
136 |
137 | Sources are numbered [1] through [{len(sources)}]. CITE them inline like Perplexity does.
138 |
139 | {data_context}"""
140 |

```

```

141 |         # Get AI synthesis
142 |         ai_response = await self.ai.chat(synthesis_prompt, f"research_{session_id}_{symbol}")
143 |
144 |         return {
145 |             'symbol': symbol.upper(),
146 |             'company_name': company_name,
147 |             'overview': overview,
148 |             'quote': quote,
149 |             'synthesis': ai_response,
150 |             'sources': sources,
151 |             'news_count': len(relevant_news),
152 |         }
153 |

```

FILE: backend/services/crypto_scraping_service.py

128 lines | 5113 characters

```

1 | import logging
2 | import requests
3 | from typing import Dict, List, Optional
4 | from datetime import datetime
5 | import asyncio
6 |
7 | logger = logging.getLogger(__name__)
8 |
9 | class CryptoScrapingService:
10 |     """Scrape crypto data from exchanges and blockchain"""
11 |
12 |     def __init__(self):
13 |         self.headers = {'User-Agent': 'Mozilla/5.0'}
14 |
15 |     async def get_exchange_data(self) -> List[Dict]:
16 |         """Get order book data from major exchanges"""
17 |         data = []
18 |
19 |         # Binance data
20 |         data.extend(await self._get_binance_orderbook())
21 |
22 |         # Coinbase data
23 |         data.extend(await self._get_coinbase_data())
24 |
25 |         return data
26 |
27 |     async def _get_binance_orderbook(self) -> List[Dict]:
28 |         """Get Binance order book depth"""
29 |         try:
30 |             symbols = ['BTCUSDT', 'ETHUSDT', 'BNBUSDT', 'SOLUSDT']
31 |             orderbooks = []
32 |
33 |             for symbol in symbols:
34 |                 url = f'https://api.binance.com/api/v3/depth?symbol={symbol}&limit=100'
35 |                 response = requests.get(url, timeout=10)
36 |                 data = response.json()
37 |
38 |                 # Calculate buy/sell pressure
39 |                 bids = sum(float(bid[1]) for bid in data.get('bids', [])[0:10])
40 |                 asks = sum(float(ask[1]) for ask in data.get('asks', [])[0:10])
41 |                 pressure = (bids - asks) / (bids + asks) if (bids + asks) > 0 else 0
42 |
43 |                 orderbooks.append({
44 |                     'exchange': 'Binance',
45 |                     'symbol': symbol.replace('USD', ''),
46 |                     'bid_volume': bids,
47 |                     'ask_volume': asks,
48 |                     'pressure': pressure,
49 |                     'sentiment': 'bullish' if pressure > 0.1 else 'bearish' if pressure < -0.1 else 'neutral',

```

```
50 |         'timestamp': datetime.utcnow().isoformat()
51 |     })
52 |
53 |     return orderbooks
54 | except Exception as e:
55 |     logger.error(f"Binance orderbook error: {str(e)}")
56 |     return []
57 |
58 | async def _get_coinbase_data(self) -> List[Dict]:
59 |     """Get Coinbase market data"""
60 |     try:
61 |         url = 'https://api.coinbase.com/v2/exchange-rates?currency=USD'
62 |         response = requests.get(url, timeout=10)
63 |         data = response.json()
64 |
65 |         rates = data.get('data', {}).get('rates', {})
66 |
67 |         cryptos = []
68 |         for crypto in ['BTC', 'ETH', 'SOL', 'XRP']:
69 |             if crypto in rates:
70 |                 cryptos.append({
71 |                     'exchange': 'Coinbase',
72 |                     'symbol': crypto,
73 |                     'rate': float(rates[crypto]),
74 |                     'timestamp': datetime.utcnow().isoformat()
75 |                 })
76 |
77 |         return cryptos
78 | except Exception as e:
79 |     logger.error(f"Coinbase data error: {str(e)}")
80 |     return []
81 |
82 | async def get_whale_transactions(self) -> List[Dict]:
83 |     """Get large crypto transactions (whale movements)"""
84 |     try:
85 |         # Using Whale Alert API (or similar service)
86 |         # Note: This typically requires an API key
87 |         url = 'https://api.whale-alert.io/v1/transactions?api_key=demo&min_value=5000000&limit=10'
88 |
89 |         # For demo purposes, return simulated data
90 |         # In production, use actual API with your key
91 |
92 |         return [
93 |             {
94 |                 'blockchain': 'bitcoin',
95 |                 'symbol': 'BTC',
96 |                 'amount': 1500,
97 |                 'amount_usd': 75000000,
98 |                 'from': 'unknown',
99 |                 'to': 'exchange',
100 |                 'timestamp': datetime.utcnow().isoformat(),
101 |                 'impact': 'high'
102 |             }
103 |         ]
104 | except Exception as e:
105 |     logger.error(f"Whale tracking error: {str(e)}")
106 |     return []
107 |
108 | async def get_crypto_sentiment(self) -> Dict:
109 |     """Aggregate crypto market sentiment"""
110 |     try:
111 |         # Using CoinGecko or similar for sentiment
112 |         url = 'https://api.coingecko.com/api/v3/global'
113 |         response = requests.get(url, timeout=10)
114 |         data = response.json()
115 |
116 |         global_data = data.get('data', {})
```

```

117 |
118 |         return {
119 |             'market_cap_usd': global_data.get('total_market_cap', {}).get('usd', 0),
120 |             'volume_24h': global_data.get('total_volume', {}).get('usd', 0),
121 |             'btc_dominance': global_data.get('market_cap_percentage', {}).get('btc', 0),
122 |             'eth_dominance': global_data.get('market_cap_percentage', {}).get('eth', 0),
123 |             'sentiment': 'bullish' if global_data.get('market_cap_change_percentage_24h_usd', 0) > 2 else 'bearish' if global_...
124 |             'timestamp': datetime.utcnow().isoformat()
125 |         }
126 |     except Exception as e:
127 |         logger.error(f"Crypto sentiment error: {str(e)}")
128 |     return {}

```

FILE: backend/services/digest_service.py

240 lines | 10471 characters

```

1 | """Daily digest email service - collects market data and sends morning briefing."""
2 | import logging
3 | import asyncio
4 | from datetime import datetime, timezone
5 | from typing import Dict, List, Optional
6 |
7 | logger = logging.getLogger(__name__)
8 |
9 | APP_NAME = "RISEDUAL AI"
10 | APP_URL = "https://risedual.ai"
11 |
12 |
13 | async def collect_digest_data(db) -> Dict:
14 |     """Collect all data needed for the daily digest."""
15 |     data = {
16 |         "predictions": [],
17 |         "dark_pool": [],
18 |         "signals": [],
19 |         "timestamp": datetime.now(timezone.utc).isoformat(),
20 |     }
21 |
22 |     # Get latest market predictions from cache/DB
23 |     try:
24 |         cursor = db.market_predictions.find({}).sort("created_at", -1).limit(5)
25 |         async for doc in cursor:
26 |             data["predictions"].append({
27 |                 "ticker": doc.get("ticker", "MARKET"),
28 |                 "verdict": doc.get("verdict", "N/A"),
29 |                 "confidence": doc.get("confidence", 0),
30 |                 "summary": doc.get("summary", ""),
31 |             })
32 |     except Exception as e:
33 |         logger.warning(f"Digest: error fetching predictions: {e}")
34 |
35 |     # Get latest dark pool data
36 |     try:
37 |         cursor = db.dark_pool_data.find({}).sort("created_at", -1).limit(5)
38 |         async for doc in cursor:
39 |             data["dark_pool"].append({
40 |                 "ticker": doc.get("ticker", ""),
41 |                 "volume": doc.get("volume", 0),
42 |                 "sentiment": doc.get("sentiment", "neutral"),
43 |             })
44 |     except Exception as e:
45 |         logger.warning(f"Digest: error fetching dark pool: {e}")
46 |
47 |     # Get latest market signals
48 |     try:
49 |         cursor = db.market_signals.find({}).sort("created_at", -1).limit(5)
50 |         async for doc in cursor:

```



```

51 |         data["signals"].append({
52 |             "ticker": doc.get("ticker", ""),
53 |             "signal": doc.get("signal_type", ""),
54 |             "strength": doc.get("strength", ""),
55 |         })
56 |     except Exception as e:
57 |         logger.warning(f"Digest: error fetching signals: {e}")
58 |
59 |     return data
60 |
61 |
62 | def _row(label: str, value: str, color: str = "#ffffff") -> str:
63 |     return f"""<tr>
64 | <td style="padding:8px 12px;border-bottom:1px solid #334155;color:#94A3B8;font-size:12px;">{label}</td>
65 | <td style="padding:8px 12px;border-bottom:1px solid #334155;color:{color};font-size:12px;font-weight:600;text-align:right;">{value} ...
66 | </tr>"""
67 |
68 |
69 | def _verdict_color(v: str) -> str:
70 |     v = (v or "").upper()
71 |     if "BULL" in v:
72 |         return "#10B981"
73 |     if "BEAR" in v:
74 |         return "#EF4444"
75 |     return "#F59E0B"
76 |
77 |
78 | def build_digest_html(data: Dict, is_pro: bool, user_name: str) -> str:
79 |     """Build the digest HTML email. Pro users get full data, free users get teaser."""
80 |     date_str = datetime.now(timezone.utc).strftime("%B %d, %Y")
81 |
82 |     # Predictions section
83 |     predictions_html = ""
84 |     for i, p in enumerate(data.get("predictions", [])[0:3]):
85 |         if not is_pro and i >= 1:
86 |             predictions_html += f"""<tr>
87 | <td style="padding:8px 12px;border-bottom:1px solid #334155;color:#475569;font-size:12px;filter:blur(4px);">??????</td>
88 | <td style="padding:8px 12px;border-bottom:1px solid #334155;color:#475569;font-size:12px;text-align:right;filter:blur(4px);">?????< ...
89 | </tr>"""
90 |         else:
91 |             vc = _verdict_color(p["verdict"])
92 |             predictions_html += _row(
93 |                 f"{p['ticker']}",
94 |                 f"{p['verdict']} ({p['confidence']}%)",
95 |                 vc
96 |             )
97 |
98 |     # Dark pool section
99 |     dark_pool_html = ""
100 |     for i, dp in enumerate(data.get("dark_pool", [])[0:5]):
101 |         if not is_pro and i >= 1:
102 |             dark_pool_html += f"""<tr>
103 | <td style="padding:8px 12px;border-bottom:1px solid #334155;color:#475569;font-size:12px;filter:blur(4px);">??????</td>
104 | <td style="padding:8px 12px;border-bottom:1px solid #334155;color:#475569;font-size:12px;text-align:right;filter:blur(4px);">?????< ...
105 | </tr>"""
106 |         else:
107 |             sc = "#10B981" if dp["sentiment"] == "bullish" else "#EF4444" if dp["sentiment"] == "bearish" else "#F59E0B"
108 |             dark_pool_html += _row(dp["ticker"], f"{dp['sentiment'].title()}", sc)
109 |
110 |     # Signals section
111 |     signals_html = ""
112 |     for i, s in enumerate(data.get("signals", [])[0:5]):
113 |         if not is_pro and i >= 1:
114 |             signals_html += f"""<tr>
115 | <td style="padding:8px 12px;border-bottom:1px solid #334155;color:#475569;font-size:12px;filter:blur(4px);">??????</td>
116 | <td style="padding:8px 12px;border-bottom:1px solid #334155;color:#475569;font-size:12px;text-align:right;filter:blur(4px);">?????< ...
117 | </tr>"""

```

```
118 |         else:
119 |             signals_html += _row(s["ticker"], f"{s['signal']}" ({s['strength']}))"
120 |
121 |     upgrade_cta = "" if is_pro else f""
122 | <table width="100%" cellpadding="0" cellspacing="0" style="background:linear-gradient(135deg,#0052FF20,#6366F120);border-radius:12 ...
123 | <tr><td style="padding:16px;text-align:center;">
124 | <p style="color:#0052FF;font-size:11px;margin:0 0 4px;text-transform:uppercase;letter-spacing:1px;font-weight:600;">Unlock Full Di ...
125 | <p style="color:#94A3B8;font-size:13px;margin:0 0 12px;">Upgrade to Pro to see all predictions, dark pool moves, and signals</p>
126 | <a href="{APP_URL}" style="display:inline-block;background-color:#0052FF;color:#ffffff;text-decoration:none;font-size:13px;font-we ...
127 | </td></tr>
128 | </table>""
129 |
130 |     no_data_msg = '<tr><td colspan="2" style="padding:12px;color:#64748B;font-size:12px;text-align:center;">No recent data availab ...
131 |
132 |     content = f""
133 | <h2 style="color:#ffffff;font-size:20px;margin:0 0 4px;font-weight:600;">Good Morning, {user_name}</h2>
134 | <p style="color:#64748B;font-size:12px;margin:0 0 20px;">{date_str} ? Daily Market Digest</p>
135 |
136 | <!-- AI Predictions -->
137 | <p style="color:#0052FF;font-size:11px;margin:0 0 8px;text-transform:uppercase;letter-spacing:1.5px;font-weight:600;">AI Market Pr ...
138 | <table width="100%" cellpadding="0" cellspacing="0" style="background-color:#0F172A;border-radius:10px;border:1px solid #334155;ma ...
139 | {predictions_html or no_data_msg}
140 | </table>
141 |
142 | <!-- Dark Pool Moves -->
143 | <p style="color:#0052FF;font-size:11px;margin:0 0 8px;text-transform:uppercase;letter-spacing:1.5px;font-weight:600;">Dark Pool Ac ...
144 | <table width="100%" cellpadding="0" cellspacing="0" style="background-color:#0F172A;border-radius:10px;border:1px solid #334155;ma ...
145 | {dark_pool_html or no_data_msg}
146 | </table>
147 |
148 | <!-- Market Signals -->
149 | <p style="color:#0052FF;font-size:11px;margin:0 0 8px;text-transform:uppercase;letter-spacing:1.5px;font-weight:600;">Market Signa ...
150 | <table width="100%" cellpadding="0" cellspacing="0" style="background-color:#0F172A;border-radius:10px;border:1px solid #334155;ma ...
151 | {signals_html or no_data_msg}
152 | </table>
153 |
154 | {upgrade_cta}
155 |
156 | <table cellpadding="0" cellspacing="0" style="margin:16px auto 0;">
157 | <tr><td style="background-color:#0052FF;border-radius:10px;padding:12px 28px;">
158 | <a href="{APP_URL}" style="color:#ffffff;text-decoration:none;font-size:14px;font-weight:600;">Open {APP_NAME}</a>
159 | </td></tr>
160 | </table>""
161 |
162 |     return _base_email_html(content)
163 |
164 |
165 | def _base_email_html(content: str) -> str:
166 |     return f""<!DOCTYPE html>
167 | <html>
168 | <head><meta charset="utf-8"><meta name="viewport" content="width=device-width, initial-scale=1.0"></head>
169 | <body style="margin:0;padding:0;background-color:#0F172A;font-family:-apple-system,BlinkMacSystemFont,'Segoe UI',sans-serif;">
170 | <table width="100%" cellpadding="0" cellspacing="0" style="background-color:#0F172A;padding:40px 20px;">
171 | <tr><td align="center">
172 | <table width="600" cellpadding="0" cellspacing="0" style="background-color:#1E293B;border-radius:16px;border:1px solid #334155;ove ...
173 | <tr><td style="background:linear-gradient(135deg,#0052FF,#6366F1);padding:24px 32px;">
174 | <table width="100%" cellpadding="0" cellspacing="0">
175 | <tr>
176 | <td><h1 style="color:#ffffff;font-size:20px;margin:0;font-weight:700;letter-spacing:-0.5px;">{APP_NAME}</h1>
177 | <p style="color:rgba(255,255,255,0.7);font-size:11px;margin:4px 0 0 0;">Daily Market Digest</p></td>
178 | <td style="text-align:right;"><p style="color:rgba(255,255,255,0.6);font-size:11px;margin:0;">6:00 AM UTC</p></td>
179 | </tr>
180 | </table>
181 | </td></tr>
182 | <tr><td style="padding:28px 32px;">{content}</td></tr>
183 | <tr><td style="padding:16px 32px;border-top:1px solid #334155;text-align:center;">
184 | <p style="color:#475569;font-size:10px;margin:0;">
```

```
185 | <a href="{APP_URL}" style="color:#64748B;text-decoration:none;">Unsubscribe</a> ? <a href="{APP_URL}" style="color:#64748B;text-de ...
186 | </p>
187 | </td></tr>
188 | </table>
189 | </td></tr>
190 | </table>
191 | </body>
192 | </html>""
193 |
194 |
195 | async def send_daily_digest(db):
196 |     """Main entry: collect data, loop through users, send digest emails."""
197 |     from services.email_service import _is_configured, SENDER_EMAIL, APP_NAME as EMAIL_APP_NAME
198 |     import resend
199 |
200 |     if not _is_configured():
201 |         logger.info("Daily digest skipped: Resend API key not configured")
202 |         return {"sent": 0, "skipped": True, "reason": "no_api_key"}
203 |
204 |     # Collect digest data
205 |     data = await collect_digest_data(db)
206 |     logger.info(f"Digest data collected: {len(data['predictions'])} predictions, {len(data['dark_pool'])} dark pool, {len(data['si ...
207 |
208 |     sent_count = 0
209 |     error_count = 0
210 |
211 |     # Get all users who haven't opted out
212 |     cursor = db.users.find(
213 |         {"digest_opt_out": {"$ne": True}, "is_active": {"$ne": False}},
214 |         {"_id": 0, "email": 1, "name": 1, "subscription_status": 1}
215 |     )
216 |
217 |     async for user in cursor:
218 |         email = user.get("email", "")
219 |         name = user.get("name", email.split("@")[0])
220 |         is_pro = user.get("subscription_status") in ("pro", "trial")
221 |
222 |         html = build_digest_html(data, is_pro, name)
223 |         subject = f"Your Morning Market Briefing ? {datetime.now(timezone.utc).strftime('%b %d')}"
224 |
225 |         try:
226 |             result = await asyncio.to_thread(resend.Emails.send, {
227 |                 "from": SENDER_EMAIL,
228 |                 "to": [email],
229 |                 "subject": subject,
230 |                 "html": html,
231 |             })
232 |             sent_count += 1
233 |             logger.info(f"Digest sent to {email} (pro={is_pro})")
234 |         except Exception as e:
235 |             error_count += 1
236 |             logger.error(f"Digest send failed for {email}: {e}")
237 |
238 |     logger.info(f"Daily digest complete: {sent_count} sent, {error_count} errors")
239 |     return {"sent": sent_count, "errors": error_count, "skipped": False}
240 |
```

FILE: backend/services/email_service.py

190 lines | 9038 characters

```
1 | """Email notification service using Resend for referral events."""
2 | import os
3 | import asyncio
4 | import logging
5 | import resend
6 | from dotenv import load_dotenv
```

```
7 | from pathlib import Path
8 |
9 | load_dotenv(Path(__file__).parent.parent / '.env')
10 |
11 | logger = logging.getLogger(__name__)
12 |
13 | RESEND_API_KEY = os.environ.get('RESEND_API_KEY', '')
14 | SENDER_EMAIL = os.environ.get('SENDER_EMAIL', 'onboarding@resend.dev')
15 | APP_NAME = "RISEDUAL AI"
16 | APP_URL = "https://risedual.ai"
17 |
18 | resend.api_key = RESEND_API_KEY
19 |
20 |
21 | def _is_configured():
22 |     return RESEND_API_KEY and not RESEND_API_KEY.startswith('re_YOUR')
23 |
24 |
25 | def _base_html(content: str) -> str:
26 |     return f"""<!DOCTYPE html>
27 | <html>
28 | <head><meta charset="utf-8"><meta name="viewport" content="width=device-width, initial-scale=1.0"></head>
29 | <body style="margin:0;padding:0;background-color:#0F172A;font-family:-apple-system,BlinkMacSystemFont,'Segoe UI',sans-serif;">
30 | <table width="100%" cellpadding="0" cellspacing="0" style="background-color:#0F172A;padding:40px 20px;">
31 | <tr><td align="center">
32 | <table width="600" cellpadding="0" cellspacing="0" style="background-color:#1E293B;border-radius:16px;border:1px solid #334155;ove ...
33 | <!-- Header -->
34 | <tr><td style="background:linear-gradient(135deg,#0052FF,#6366F1);padding:28px 32px;text-align:center;">
35 | <h1 style="color:#ffffff;font-size:22px;margin:0;font-weight:700;letter-spacing:-0.5px;">{APP_NAME}</h1>
36 | <p style="color:rgba(255,255,255,0.7);font-size:12px;margin:6px 0 0 0;">AI-Powered Trading Platform</p>
37 | </td></tr>
38 | <!-- Content -->
39 | <tr><td style="padding:32px;">{content}</td></tr>
40 | <!-- Footer -->
41 | <tr><td style="padding:20px 32px;border-top:1px solid #334155;text-align:center;">
42 | <p style="color:#64748B;font-size:11px;margin:0;">You're receiving this because you're a {APP_NAME} member.</p>
43 | <p style="color:#475569;font-size:11px;margin:8px 0 0 0;"><a href="{APP_URL}" style="color:#0052FF;text-decoration:none;">{APP_URL}< ...
44 | </td></tr>
45 | </table>
46 | </td></tr>
47 | </table>
48 | </body>
49 | </html>"""
50 |
51 |
52 | def _referral_signup_html(referrer_name: str, referred_email: str) -> str:
53 |     content = f"""
54 | <h2 style="color:#ffffff;font-size:20px;margin:0 0 8px;font-weight:600;">Your friend just signed up!</h2>
55 | <p style="color:#94A3B8;font-size:14px;line-height:1.6;margin:0 0 20px;">
56 | Hey {referrer_name}, great news &mdash; someone used your referral link to join {APP_NAME}!
57 | </p>
58 | <table width="100%" cellpadding="0" cellspacing="0" style="background-color:#0F172A;border-radius:12px;border:1px solid #334155;ma ...
59 | <tr><td style="padding:16px 20px;">
60 | <p style="color:#64748B;font-size:11px;margin:0 0 4px;text-transform:uppercase;letter-spacing:1px;">New Referral</p>
61 | <p style="color:#ffffff;font-size:16px;margin:0;font-weight:600;">{referred_email}</p>
62 | </td></tr>
63 | </table>
64 | <p style="color:#94A3B8;font-size:14px;line-height:1.6;margin:0 0 24px;">
65 | When they subscribe to Pro, you'll earn <strong style="color:#0052FF;">1 free month</strong> of {APP_NAME} Pro. Keep sharing your ...
66 | </p>
67 | <table cellpadding="0" cellspacing="0" style="margin:0 auto;">
68 | <tr><td style="background-color:#0052FF;border-radius:10px;padding:12px 28px;">
69 | <a href="{APP_URL}" style="color:#ffffff;text-decoration:none;font-size:14px;font-weight:600;">View Your Referrals</a>
70 | </td></tr>
71 | </table>"""
72 |     return _base_html(content)
73 |
```

```
74 |
75 | def _reward_earned_html(referrer_name: str, referred_email: str) -> str:
76 |     content = f"""
77 |     <h2 style="color:#ffffff;font-size:20px;margin:0 0 8px;font-weight:600;">You earned a free month!</h2>
78 |     <p style="color:#94A3B8;font-size:14px;line-height:1.6;margin:0 0 20px;">
79 |     Congrats {referrer_name}! Your referral just subscribed to Pro, and you've earned <strong style="color:#10B981;">1 free month</str ...
80 |     </p>
81 |     <table width="100%" cellpadding="0" cellspacing="0" style="background-color:#0F172A;border-radius:12px;border:1px solid #334155;ma ...
82 |     <tr><td style="padding:16px 20px;">
83 |     <table width="100%" cellpadding="0" cellspacing="0">
84 |     <tr>
85 |     <td style="width:50%;">
86 |     <p style="color:#64748B;font-size:11px;margin:0 0 4px;text-transform:uppercase;letter-spacing:1px;">Referral</p>
87 |     <p style="color:#ffffff;font-size:14px;margin:0;font-weight:500;">{referred_email}</p>
88 |     </td>
89 |     <td style="width:50%;text-align:right;">
90 |     <p style="color:#64748B;font-size:11px;margin:0 0 4px;text-transform:uppercase;letter-spacing:1px;">Reward</p>
91 |     <p style="color:#10B981;font-size:18px;margin:0;font-weight:700;">+1 Month Free</p>
92 |     </td>
93 |     </tr>
94 |     </table>
95 |     </td></tr>
96 |     </table>
97 |     <p style="color:#94A3B8;font-size:14px;line-height:1.6;margin:0 0 24px;">
98 |     Keep referring friends to earn more free months (up to 12 per year)!
99 |     </p>
100 |     <table cellpadding="0" cellspacing="0" style="margin:0 auto;">
101 |     <tr><td style="background-color:#0052FF;border-radius:10px;padding:12px 28px;">
102 |     <a href="{APP_URL}" style="color:#ffffff;text-decoration:none;font-size:14px;font-weight:600;">Share Your Link</a>
103 |     </td></tr>
104 |     </table>"""
105 |     return _base_html(content)
106 |
107 |
108 | def _welcome_referral_html(user_name: str, referrer_name: str) -> str:
109 |     content = f"""
110 |     <h2 style="color:#ffffff;font-size:20px;margin:0 0 8px;font-weight:600;">Welcome to {APP_NAME}!</h2>
111 |     <p style="color:#94A3B8;font-size:14px;line-height:1.6;margin:0 0 20px;">
112 |     Hey {user_name}, welcome aboard! You were referred by <strong style="color:#ffffff;">{referrer_name}</strong>, and we've activated ...
113 |     </p>
114 |     <table width="100%" cellpadding="0" cellspacing="0" style="background:linear-gradient(135deg,#0052FF20,#6366F120);border-radius:12 ...
115 |     <tr><td style="padding:20px;text-align:center;">
116 |     <p style="color:#0052FF;font-size:12px;margin:0 0 4px;text-transform:uppercase;letter-spacing:1.5px;font-weight:600;">Your Gift</p ...
117 |     <p style="color:#ffffff;font-size:24px;margin:0;font-weight:700;">7-Day Pro Trial</p>
118 |     <p style="color:#94A3B8;font-size:13px;margin:8px 0 0;">Full access to AI predictions, dark pool data, market signals & more</p>
119 |     </td></tr>
120 |     </table>
121 |     <p style="color:#94A3B8;font-size:14px;line-height:1.6;margin:0 0 24px;">
122 |     Dive in and explore everything {APP_NAME} has to offer. Your Pro trial starts now!
123 |     </p>
124 |     <table cellpadding="0" cellspacing="0" style="margin:0 auto;">
125 |     <tr><td style="background-color:#0052FF;border-radius:10px;padding:12px 28px;">
126 |     <a href="{APP_URL}" style="color:#ffffff;text-decoration:none;font-size:14px;font-weight:600;">Start Exploring</a>
127 |     </td></tr>
128 |     </table>"""
129 |     return _base_html(content)
130 |
131 |
132 | async def send_referral_signup_email(referrer_email: str, referrer_name: str, referred_email: str):
133 |     """Notify referrer when someone signs up via their link."""
134 |     if not _is_configured():
135 |         logger.info(f"Email skipped (no API key): referral signup notification to {referrer_email}")
136 |         return False
137 |     try:
138 |         params = {
139 |             "from": SENDER_EMAIL,
140 |             "to": [referrer_email],
```

```

141 |         "subject": f"Your friend just joined {APP_NAME}!",
142 |         "html": _referral_signup_html(referrer_name, referred_email),
143 |     }
144 |     result = await asyncio.to_thread(resend.Emails.send, params)
145 |     logger.info(f"Referral signup email sent to {referrer_email}, id: {result.get('id', 'unknown')}")
146 |     return True
147 | except Exception as e:
148 |     logger.error(f"Failed to send referral signup email to {referrer_email}: {e}")
149 |     return False
150 |
151 |
152 | async def send_reward_earned_email(referrer_email: str, referrer_name: str, referred_email: str):
153 |     """Notify referrer when they earn a free month."""
154 |     if not _is_configured():
155 |         logger.info(f"Email skipped (no API key): reward earned notification to {referrer_email}")
156 |         return False
157 |     try:
158 |         params = {
159 |             "from": SENDER_EMAIL,
160 |             "to": [referrer_email],
161 |             "subject": f"You earned a free month of {APP_NAME} Pro!",
162 |             "html": _reward_earned_html(referrer_name, referred_email),
163 |         }
164 |         result = await asyncio.to_thread(resend.Emails.send, params)
165 |         logger.info(f"Reward earned email sent to {referrer_email}, id: {result.get('id', 'unknown')}")
166 |         return True
167 |     except Exception as e:
168 |         logger.error(f"Failed to send reward email to {referrer_email}: {e}")
169 |         return False
170 |
171 |
172 | async def send_welcome_referral_email(user_email: str, user_name: str, referrer_name: str):
173 |     """Send welcome email to newly referred user."""
174 |     if not _is_configured():
175 |         logger.info(f"Email skipped (no API key): welcome email to {user_email}")
176 |         return False
177 |     try:
178 |         params = {
179 |             "from": SENDER_EMAIL,
180 |             "to": [user_email],
181 |             "subject": f>Welcome to {APP_NAME} ? Your 7-Day Pro Trial is Active!",
182 |             "html": _welcome_referral_html(user_name, referrer_name),
183 |         }
184 |         result = await asyncio.to_thread(resend.Emails.send, params)
185 |         logger.info(f>Welcome email sent to {user_email}, id: {result.get('id', 'unknown')}")
186 |         return True
187 |     except Exception as e:
188 |         logger.error(f"Failed to send welcome email to {user_email}: {e}")
189 |         return False
190 |

```

FILE: backend/services/financial_scraping_service.py

273 lines | 11851 characters

```

1 | import logging
2 | import requests
3 | from bs4 import BeautifulSoup
4 | from typing import Dict, List, Optional
5 | from datetime import datetime, timedelta
6 | import re
7 |
8 | logger = logging.getLogger(__name__)
9 |
10 | class FinancialScrapingService:
11 |     """Scrape financial news and data from multiple sources"""
12 |

```

```
13 |     def __init__(self):
14 |         self.headers = {
15 |             'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36'
16 |         }
17 |
18 |     async def scrape_financial_news(self) -> List[Dict]:
19 |         """Scrape latest financial news from multiple sources"""
20 |         news = []
21 |
22 |         # Scrape from various sources
23 |         news.extend(await self._scrape_cnbc())
24 |         news.extend(await self._scrape_reuters())
25 |         news.extend(await self._scrape_marketwatch())
26 |         news.extend(await self._scrape_fox_business())
27 |         news.extend(await self._scrape_wsj())
28 |         news.extend(await self._scrape_bloomberg())
29 |         news.extend(await self._scrape_oan())
30 |         news.extend(await self._scrape_epoch_times())
31 |
32 |         return news
33 |
34 |     async def _scrape_cnbc(self) -> List[Dict]:
35 |         """Scrape CNBC market news"""
36 |         try:
37 |             url = 'https://www.cnbc.com/markets/'
38 |             response = requests.get(url, headers=self.headers, timeout=10)
39 |             soup = BeautifulSoup(response.content, 'html.parser')
40 |
41 |             articles = []
42 |             for article in soup.find_all('div', class_='Card-titleContainer')[:10]:
43 |                 title = article.find('a')
44 |                 if title:
45 |                     articles.append({
46 |                         'source': 'CNBC',
47 |                         'title': title.get_text(strip=True),
48 |                         'url': title.get('href'),
49 |                         'timestamp': datetime.utcnow().isoformat(),
50 |                         'sentiment': None # Will be analyzed by AI
51 |                     })
52 |             return articles
53 |         except Exception as e:
54 |             logger.error(f"CNBC scraping error: {str(e)}")
55 |             return []
56 |
57 |     async def _scrape_reuters(self) -> List[Dict]:
58 |         """Scrape Reuters market news"""
59 |         try:
60 |             url = 'https://www.reuters.com/markets/'
61 |             response = requests.get(url, headers=self.headers, timeout=10)
62 |             soup = BeautifulSoup(response.content, 'html.parser')
63 |
64 |             articles = []
65 |             for article in soup.find_all('a', {'data-testid': 'Heading'})[:10]:
66 |                 articles.append({
67 |                     'source': 'Reuters',
68 |                     'title': article.get_text(strip=True),
69 |                     'url': 'https://www.reuters.com' + article.get('href', ''),
70 |                     'timestamp': datetime.utcnow().isoformat(),
71 |                     'sentiment': None
72 |                 })
73 |             return articles
74 |         except Exception as e:
75 |             logger.error(f"Reuters scraping error: {str(e)}")
76 |             return []
77 |
78 |     async def _scrape_marketwatch(self) -> List[Dict]:
79 |         """Scrape MarketWatch headlines"""
```

```

80 |         try:
81 |             url = 'https://www.marketwatch.com/latest-news'
82 |             response = requests.get(url, headers=self.headers, timeout=10)
83 |             soup = BeautifulSoup(response.content, 'html.parser')
84 |
85 |             articles = []
86 |             for article in soup.find_all('h3', class_='article__headline')[:10]:
87 |                 link = article.find('a')
88 |                 if link:
89 |                     articles.append({
90 |                         'source': 'MarketWatch',
91 |                         'title': link.get_text(strip=True),
92 |                         'url': link.get('href'),
93 |                         'timestamp': datetime.utcnow().isoformat(),
94 |                         'sentiment': None
95 |                     })
96 |             return articles
97 |         except Exception as e:
98 |             logger.error(f"MarketWatch scraping error: {str(e)}")
99 |             return []
100 |
101 |     async def _scrape_fox_business(self) -> List[Dict]:
102 |         """Scrape Fox Business news"""
103 |         try:
104 |             url = 'https://www.foxbusiness.com/markets'
105 |             response = requests.get(url, headers=self.headers, timeout=10)
106 |             soup = BeautifulSoup(response.content, 'html.parser')
107 |
108 |             articles = []
109 |             for article in soup.find_all('h2', class_='title')[:10]:
110 |                 link = article.find('a')
111 |                 if link:
112 |                     articles.append({
113 |                         'source': 'Fox Business',
114 |                         'title': link.get_text(strip=True),
115 |                         'url': 'https://www.foxbusiness.com' + link.get('href', ''),
116 |                         'timestamp': datetime.utcnow().isoformat(),
117 |                         'sentiment': None
118 |                     })
119 |             return articles
120 |         except Exception as e:
121 |             logger.error(f"Fox Business scraping error: {str(e)}")
122 |             return []
123 |
124 |     async def _scrape_wsjs(self) -> List[Dict]:
125 |         """Scrape Wall Street Journal market news"""
126 |         try:
127 |             url = 'https://www.wsj.com/news/markets'
128 |             response = requests.get(url, headers=self.headers, timeout=10)
129 |             soup = BeautifulSoup(response.content, 'html.parser')
130 |
131 |             articles = []
132 |             for article in soup.find_all('h3', class_='WSJTheme--headline')[:10]:
133 |                 link = article.find('a')
134 |                 if link:
135 |                     href = link.get('href', '')
136 |                     full_url = href if href.startswith('http') else 'https://www.wsj.com' + href
137 |                     articles.append({
138 |                         'source': 'Wall Street Journal',
139 |                         'title': link.get_text(strip=True),
140 |                         'url': full_url,
141 |                         'timestamp': datetime.utcnow().isoformat(),
142 |                         'sentiment': None
143 |                     })
144 |             return articles
145 |         except Exception as e:
146 |             logger.error(f"WSJ scraping error: {str(e)}")

```



```

147 |         return []
148 |
149 |     async def _scrape_bloomberg(self) -> List[Dict]:
150 |         """Scrape Bloomberg market news"""
151 |         try:
152 |             url = 'https://www.bloomberg.com/markets'
153 |             response = requests.get(url, headers=self.headers, timeout=10)
154 |             soup = BeautifulSoup(response.content, 'html.parser')
155 |
156 |             articles = []
157 |             for article in soup.find_all('article')[:10]:
158 |                 headline = article.find('a')
159 |                 if headline:
160 |                     href = headline.get('href', '')
161 |                     full_url = href if href.startswith('http') else 'https://www.bloomberg.com' + href
162 |                     articles.append({
163 |                         'source': 'Bloomberg',
164 |                         'title': headline.get_text(strip=True),
165 |                         'url': full_url,
166 |                         'timestamp': datetime.utcnow().isoformat(),
167 |                         'sentiment': None
168 |                     })
169 |             return articles
170 |         except Exception as e:
171 |             logger.error(f"Bloomberg scraping error: {str(e)}")
172 |         return []
173 |
174 |     async def _scrape_oan(self) -> List[Dict]:
175 |         """Scrape One America News (OAN) business news"""
176 |         try:
177 |             url = 'https://www.oann.com/category/business/'
178 |             response = requests.get(url, headers=self.headers, timeout=10)
179 |             soup = BeautifulSoup(response.content, 'html.parser')
180 |
181 |             articles = []
182 |             for article in soup.find_all('h2', class_='entry-title')[:10]:
183 |                 link = article.find('a')
184 |                 if link:
185 |                     articles.append({
186 |                         'source': 'OAN',
187 |                         'title': link.get_text(strip=True),
188 |                         'url': link.get('href'),
189 |                         'timestamp': datetime.utcnow().isoformat(),
190 |                         'sentiment': None
191 |                     })
192 |             return articles
193 |         except Exception as e:
194 |             logger.error(f"OAN scraping error: {str(e)}")
195 |         return []
196 |
197 |     async def _scrape_epoch_times(self) -> List[Dict]:
198 |         """Scrape Epoch Times business news"""
199 |         try:
200 |             url = 'https://www.theepochtimes.com/business'
201 |             response = requests.get(url, headers=self.headers, timeout=10)
202 |             soup = BeautifulSoup(response.content, 'html.parser')
203 |
204 |             articles = []
205 |             for article in soup.find_all('h3')[:10]:
206 |                 link = article.find('a')
207 |                 if link:
208 |                     href = link.get('href', '')
209 |                     full_url = href if href.startswith('http') else 'https://www.theepochtimes.com' + href
210 |                     articles.append({
211 |                         'source': 'Epoch Times',
212 |                         'title': link.get_text(strip=True),
213 |                         'url': full_url,

```

```
214 |         'timestamp': datetime.utcnow().isoformat(),
215 |         'sentiment': None
216 |     })
217 |     return articles
218 | except Exception as e:
219 |     logger.error(f"Epoch Times scraping error: {str(e)}")
220 |     return []
221 |
222 |
223 | async def scrape_reddit_sentiment(self) -> List[Dict]:
224 |     """Scrape Reddit WallStreetBets for sentiment"""
225 |     try:
226 |         # Using Reddit JSON API (no auth needed for public posts)
227 |         url = 'https://www.reddit.com/r/wallstreetbets/hot.json?limit=25'
228 |         response = requests.get(url, headers=self.headers, timeout=10)
229 |         data = response.json()
230 |
231 |         posts = []
232 |         for post in data['data']['children']:
233 |             post_data = post['data']
234 |             posts.append({
235 |                 'source': 'Reddit-WSB',
236 |                 'title': post_data['title'],
237 |                 'score': post_data['score'],
238 |                 'num_comments': post_data['num_comments'],
239 |                 'url': 'https://reddit.com' + post_data['permalink'],
240 |                 'timestamp': datetime.fromtimestamp(post_data['created_utc']).isoformat(),
241 |                 'sentiment': None
242 |             })
243 |         return posts
244 |     except Exception as e:
245 |         logger.error(f"Reddit scraping error: {str(e)}")
246 |         return []
247 |
248 | async def scrape_insider_trades(self) -> List[Dict]:
249 |     """Scrape recent insider trading activity"""
250 |     try:
251 |         # Using OpenInsider.com
252 |         url = 'http://openinsider.com/screener?s=&o=&pl=&ph=&ll=&lh=&fd=730&fdr=&td=0&tdr=&fdlyl=&fdlyh=&daysago=&xp=1&xs=1&vl ...
253 |         response = requests.get(url, headers=self.headers, timeout=10)
254 |         soup = BeautifulSoup(response.content, 'html.parser')
255 |
256 |         trades = []
257 |         table = soup.find('table', class_='tinytable')
258 |         if table:
259 |             rows = table.find_all('tr')[1:11] # Get first 10 trades
260 |             for row in rows:
261 |                 cols = row.find_all('td')
262 |                 if len(cols) >= 5:
263 |                     trades.append({
264 |                         'ticker': cols[3].get_text(strip=True) if len(cols) > 3 else '',
265 |                         'insider': cols[1].get_text(strip=True) if len(cols) > 1 else '',
266 |                         'trade_type': cols[5].get_text(strip=True) if len(cols) > 5 else '',
267 |                         'value': cols[7].get_text(strip=True) if len(cols) > 7 else '',
268 |                         'timestamp': datetime.utcnow().isoformat()
269 |                     })
270 |         return trades
271 |     except Exception as e:
272 |         logger.error(f"Insider trades scraping error: {str(e)}")
273 |         return []
```

FILE: backend/services/foreign_markets_service.py

113 lines | 4531 characters

```
1 | import logging
2 | import requests
```

```

3 | from typing import Dict, List
4 | from bs4 import BeautifulSoup
5 | from datetime import datetime, timezone
6 |
7 | logger = logging.getLogger(__name__)
8 |
9 |
10 | class ForeignMarketsService:
11 |     def __init__(self):
12 |         self.headers = {
13 |             'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36'
14 |         }
15 |
16 |     def _scrape_yahoo_quote(self, symbol: str, name: str, region: str) -> Dict:
17 |         try:
18 |             url = f'https://query1.finance.yahoo.com/v8/finance/chart/{symbol}?interval=1d&range=2d'
19 |             resp = requests.get(url, headers=self.headers, timeout=8)
20 |             data = resp.json()
21 |             meta = data.get('chart', {}).get('result', [{}])[0].get('meta', {})
22 |             price = meta.get('regularMarketPrice', 0)
23 |             prev_close = meta.get('previousClose', 0) or meta.get('chartPreviousClose', 0)
24 |             change = price - prev_close if prev_close else 0
25 |             change_pct = (change / prev_close * 100) if prev_close else 0
26 |             return {
27 |                 'symbol': symbol,
28 |                 'name': name,
29 |                 'region': region,
30 |                 'price': round(price, 2),
31 |                 'change': round(change, 2),
32 |                 'change_percent': round(change_pct, 2),
33 |                 'prev_close': round(prev_close, 2),
34 |                 'currency': meta.get('currency', 'USD'),
35 |                 'market_state': meta.get('marketState', 'CLOSED'),
36 |             }
37 |         except Exception as e:
38 |             logger.error(f"Error fetching {symbol}: {str(e)}")
39 |             return {
40 |                 'symbol': symbol, 'name': name, 'region': region,
41 |                 'price': 0, 'change': 0, 'change_percent': 0,
42 |                 'prev_close': 0, 'currency': 'N/A', 'market_state': 'ERROR',
43 |             }
44 |
45 |     async def get_foreign_markets(self) -> Dict:
46 |         indices = [
47 |             # Asia
48 |             ('^N225', 'Nikkei 225', 'Asia'),
49 |             ('^HSI', 'Hang Seng', 'Asia'),
50 |             ('000001.SS', 'Shanghai Composite', 'Asia'),
51 |             ('^KS11', 'KOSPI', 'Asia'),
52 |             ('^BSESEN', 'BSE Sensex', 'Asia'),
53 |             # Europe
54 |             ('^FTSE', 'FTSE 100', 'Europe'),
55 |             ('^GDAXI', 'DAX', 'Europe'),
56 |             ('^FCHI', 'CAC 40', 'Europe'),
57 |             ('^STOXX50E', 'Euro Stoxx 50', 'Europe'),
58 |             # Americas
59 |             ('^GSPC', 'S&P 500', 'Americas'),
60 |             ('^DJI', 'Dow Jones', 'Americas'),
61 |             ('^IXIC', 'NASDAQ', 'Americas'),
62 |             ('^BVSP', 'Bovespa', 'Americas'),
63 |         ]
64 |
65 |         commodities = [
66 |             ('CL=F', 'Crude Oil (WTI)', 'Commodity'),
67 |             ('BZ=F', 'Brent Crude', 'Commodity'),
68 |             ('GC=F', 'Gold', 'Commodity'),
69 |             ('SI=F', 'Silver', 'Commodity'),

```

```

70 |         ('NG=F', 'Natural Gas', 'Commodity'),
71 |         ('HG=F', 'Copper', 'Commodity'),
72 |     ]
73 |
74 |     currencies = [
75 |         ('EURUSD=X', 'EUR/USD', 'Currency'),
76 |         ('GBPUSD=X', 'GBP/USD', 'Currency'),
77 |         ('JPY=X', 'USD/JPY', 'Currency'),
78 |         ('CNY=X', 'USD/CNY', 'Currency'),
79 |     ]
80 |
81 |     all_data = []
82 |     for symbol, name, region in indices + commodities + currencies:
83 |         quote = self._scrape_yahoo_quote(symbol, name, region)
84 |         all_data.append(quote)
85 |
86 |     asia = [d for d in all_data if d['region'] == 'Asia']
87 |     europe = [d for d in all_data if d['region'] == 'Europe']
88 |     americas = [d for d in all_data if d['region'] == 'Americas']
89 |     comms = [d for d in all_data if d['region'] == 'Commodity']
90 |     currs = [d for d in all_data if d['region'] == 'Currency']
91 |
92 |     # Pre-market correlation signals
93 |     signals = []
94 |     for d in asia + europe:
95 |         if abs(d['change_percent']) >= 1.5:
96 |             direction = 'up' if d['change_percent'] > 0 else 'down'
97 |             signals.append({
98 |                 'market': d['name'],
99 |                 'change_percent': d['change_percent'],
100 |                 'signal': f"{d['name']} {direction} {abs(d['change_percent']):.1f}% - may impact US markets",
101 |                 'severity': 'high' if abs(d['change_percent']) >= 3 else 'medium',
102 |             })
103 |
104 |     return {
105 |         'asia': asia,
106 |         'europe': europe,
107 |         'americas': americas,
108 |         'commodities': comms,
109 |         'currencies': currs,
110 |         'correlation_signals': sorted(signals, key=lambda x: abs(x['change_percent']), reverse=True),
111 |         'timestamp': datetime.now(timezone.utc).isoformat(),
112 |     }
113 |

```

FILE: backend/services/gov_filings_service.py

205 lines | 9899 characters

```

1 | import logging
2 | import requests
3 | import re
4 | from typing import Dict, List
5 | from bs4 import BeautifulSoup
6 | from datetime import datetime, timezone, timedelta
7 |
8 | logger = logging.getLogger(__name__)
9 |
10 |
11 | class GovFilingsService:
12 |     def __init__(self):
13 |         self.headers = {
14 |             'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36',
15 |             'Accept': 'text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8',
16 |         }
17 |
18 |     async def get_sec_filings(self, ticker: str = None) -> List[Dict]:

```

```

19 |         """Fetch recent SEC EDGAR filings via full-text search API"""
20 |         filings = []
21 |         try:
22 |             # EDGAR EFTS search API for recent Form 4 filings
23 |             url = 'https://efts.sec.gov/LATEST/search-index?q=*%&forms=4'
24 |             resp = requests.get(url, headers={
25 |                 'User-Agent': 'RISEDUALAI admin@risedual.ai',
26 |                 'Accept': 'application/json',
27 |             }, timeout=10)
28 |             if resp.status_code == 200:
29 |                 data = resp.json()
30 |                 hits = data.get('hits', {}).get('hits', [])[:15]
31 |                 for hit in hits:
32 |                     src = hit.get('_source', {})
33 |                     filings.append({
34 |                         'form_type': src.get('forms', 'Form 4'),
35 |                         'company': src.get('display_names', ['Unknown'])[0] if src.get('display_names') else 'Unknown',
36 |                         'filed_date': src.get('file_date', ''),
37 |                         'description': 'Insider Transaction',
38 |                     })
39 |         except Exception as e:
40 |             logger.error(f"Error fetching SEC filings: {str(e)}")
41 |
42 |         # Fallback: scrape OpenInsider
43 |         if not filings:
44 |             filings = self._scrape_openinsider()
45 |
46 |         return filings[:15]
47 |
48 |     def _scrape_openinsider(self) -> List[Dict]:
49 |         """Scrape insider trading data from OpenInsider"""
50 |         trades = []
51 |         try:
52 |             url = 'http://openinsider.com/screener?s=&o=&pl=&ph=&ll=&lh=&fd=7&fdr=&td=0&tdr=&feession=&teession=&xp=1&vl=&vh=&ocl= ...
53 |             resp = requests.get(url, headers=self.headers, timeout=10)
54 |             soup = BeautifulSoup(resp.text, 'html.parser')
55 |             table = soup.find('table', class_='tinytable')
56 |             if table:
57 |                 rows = table.find_all('tr')[1:16]
58 |                 for row in rows:
59 |                     cols = row.find_all('td')
60 |                     if len(cols) >= 12:
61 |                         trades.append({
62 |                             'form_type': 'Form 4',
63 |                             'filed_date': cols[1].get_text(strip=True),
64 |                             'ticker': cols[3].get_text(strip=True),
65 |                             'company': cols[4].get_text(strip=True),
66 |                             'insider_name': cols[5].get_text(strip=True),
67 |                             'insider_title': cols[6].get_text(strip=True),
68 |                             'trade_type': cols[7].get_text(strip=True),
69 |                             'price': cols[8].get_text(strip=True),
70 |                             'qty': cols[9].get_text(strip=True),
71 |                             'value': cols[11].get_text(strip=True),
72 |                             'description': f"Insider {cols[7].get_text(strip=True)} by {cols[5].get_text(strip=True)}",
73 |                         })
74 |         except Exception as e:
75 |             logger.error(f"Error scraping OpenInsider: {str(e)}")
76 |         return trades
77 |
78 |     async def get_fed_announcements(self) -> List[Dict]:
79 |         """Scrape Federal Reserve announcements from RSS"""
80 |         announcements = []
81 |         try:
82 |             url = 'https://www.federalreserve.gov/feeds/press_all.xml'
83 |             resp = requests.get(url, headers=self.headers, timeout=10)
84 |             soup = BeautifulSoup(resp.content, 'xml')
85 |             items = soup.find_all('item')[:10]

```

```

86 |         for item in items:
87 |             title = item.find('title')
88 |             link = item.find('link')
89 |             pub_date = item.find('pubDate')
90 |             desc = item.find('description')
91 |             announcements.append({
92 |                 'title': title.get_text(strip=True) if title else '',
93 |                 'url': link.get_text(strip=True) if link else '',
94 |                 'date': pub_date.get_text(strip=True) if pub_date else '',
95 |                 'description': desc.get_text(strip=True)[:300] if desc else '',
96 |                 'source': 'Federal Reserve',
97 |                 'type': 'fed_announcement',
98 |             })
99 |     except Exception as e:
100 |         logger.error(f"Error fetching Fed announcements: {str(e)}")
101 |     return announcements
102 |
103 |     async def get_congressional_trades(self) -> List[Dict]:
104 |         """Scrape congressional stock trades from Capitol Trades"""
105 |         trades = []
106 |         try:
107 |             url = 'https://www.capitoltrades.com/trades'
108 |             resp = requests.get(url, headers=self.headers, timeout=15)
109 |             if resp.status_code == 200:
110 |                 soup = BeautifulSoup(resp.text, 'html.parser')
111 |                 table = soup.find('table')
112 |                 if table:
113 |                     rows = table.find_all('tr')[1:21] # Skip header, get up to 20
114 |                     for row in rows:
115 |                         cols = row.find_all('td')
116 |                         if len(cols) >= 8:
117 |                             politician_text = cols[0].get_text(strip=True)
118 |                             issuer_text = cols[1].get_text(strip=True)
119 |                             trade_type = cols[6].get_text(strip=True)
120 |                             size = cols[7].get_text(strip=True)
121 |
122 |                             # Parse politician info (name + party + chamber)
123 |                             name = politician_text
124 |                             party = ''
125 |                             chamber = ''
126 |                             if 'Republican' in politician_text:
127 |                                 party = 'R'
128 |                                 name = politician_text.split('Republican')[0].strip()
129 |                             elif 'Democrat' in politician_text:
130 |                                 party = 'D'
131 |                                 name = politician_text.split('Democrat')[0].strip()
132 |                             if 'House' in politician_text:
133 |                                 chamber = 'House'
134 |                             elif 'Senate' in politician_text:
135 |                                 chamber = 'Senate'
136 |
137 |                             # Extract ticker from issuer text
138 |                             ticker_match = re.search(r'([A-Z]{1,5}):[A-Z]{2}', issuer_text)
139 |                             ticker = ticker_match.group(1) if ticker_match else ''
140 |                             company = issuer_text.split(ticker)[0].strip() if ticker else issuer_text
141 |
142 |                             traded_date = cols[3].get_text(strip=True) if len(cols) > 3 else ''
143 |                             published = cols[2].get_text(strip=True) if len(cols) > 2 else ''
144 |
145 |                             trades.append({
146 |                                 'representative': name,
147 |                                 'ticker': ticker,
148 |                                 'company': company,
149 |                                 'transaction_date': traded_date,
150 |                                 'disclosure_date': published,
151 |                                 'type': trade_type,
152 |                                 'amount': size,

```

```

153 |                 'party': party,
154 |                 'chamber': chamber,
155 |                 'description': f"{trade_type.upper()} by {name} ({party}-{chamber}) - {ticker} {size}",
156 |             })
157 |     except Exception as e:
158 |         logger.error(f"Error scraping Capitol Trades: {str(e)}")
159 |
160 |     # Fallback: scrape QuiverQuant congress page
161 |     if not trades:
162 |         trades = self._scrape_quiverquant_congress()
163 |
164 |     return trades[:20]
165 |
166 |     def _scrape_quiverquant_congress(self) -> List[Dict]:
167 |         """Fallback: scrape QuiverQuant congressional trading page"""
168 |         trades = []
169 |         try:
170 |             url = 'https://www.quiverquant.com/congresstrading/'
171 |             resp = requests.get(url, headers=self.headers, timeout=10)
172 |             soup = BeautifulSoup(resp.text, 'html.parser')
173 |             table = soup.find('table')
174 |             if table:
175 |                 rows = table.find_all('tr')[1:16]
176 |                 for row in rows:
177 |                     cols = row.find_all('td')
178 |                     if len(cols) >= 5:
179 |                         trades.append({
180 |                             'representative': cols[0].get_text(strip=True),
181 |                             'ticker': cols[1].get_text(strip=True),
182 |                             'type': cols[2].get_text(strip=True) if len(cols) > 2 else '',
183 |                             'amount': cols[3].get_text(strip=True) if len(cols) > 3 else '',
184 |                             'transaction_date': cols[4].get_text(strip=True) if len(cols) > 4 else '',
185 |                             'description': f"Congressional trade: {cols[0].get_text(strip=True)} - {cols[1].get_text(strip=True)}" ...
186 |                         })
187 |         except Exception as e:
188 |             logger.error(f"Error scraping QuiverQuant: {str(e)}")
189 |         return trades
190 |
191 |     async def get_all_gov_data(self) -> Dict:
192 |         insider_trades = await self.get_sec_filings()
193 |         fed_announcements = await self.get_fed_announcements()
194 |         congressional_trades = await self.get_congressional_trades()
195 |
196 |         return {
197 |             'insider_trades': insider_trades,
198 |             'insider_count': len(insider_trades),
199 |             'fed_announcements': fed_announcements,
200 |             'fed_count': len(fed_announcements),
201 |             'congressional_trades': congressional_trades,
202 |             'congressional_count': len(congressional_trades),
203 |             'timestamp': datetime.now(timezone.utc).isoformat(),
204 |         }
205 |

```

FILE: backend/services/hypothesis_service.py

177 lines | 6479 characters

```

1 | import os
2 | import logging
3 | from typing import Dict
4 | from emergentintegrations.llm.chat import LlmChat, UserMessage
5 | import json
6 |
7 | logger = logging.getLogger(__name__)
8 |
9 |

```

```

10 | class HypothesisService:
11 |     """Per-ticker AI hypothesis engine using all scraped macro data"""
12 |
13 |     def __init__(self, api_key: str):
14 |         self.api_key = api_key
15 |         self.system_message = """You are an elite Wall Street quantitative analyst and AI research engine for RISEDUALAI.
16 | When given a ticker symbol (stock or crypto), produce a comprehensive investment hypothesis using ALL provided data.
17 |
18 | Your hypothesis MUST include:
19 | 1. VERDICT: BUY / SELL / HOLD
20 | 2. CONFIDENCE: 0-100%
21 | 3. PRICE TARGET: Near-term (1-2 weeks) and medium-term (1-3 months) targets
22 | 4. THESIS: 2-3 paragraph investment thesis explaining the reasoning
23 | 5. KEY CATALYSTS: Top 3-5 positive catalysts
24 | 6. KEY RISKS: Top 3-5 risk factors
25 | 7. CONGRESSIONAL ACTIVITY: Any relevant congressional trades in this ticker or sector
26 | 8. SECTOR IMPACT: How world events and macro conditions affect this specific ticker
27 | 9. TECHNICAL OUTLOOK: Brief technical assessment based on available data
28 |
29 | Be specific, cite actual data points from the provided information, and always acknowledge uncertainty.
30 | Output in JSON format:
31 | {
32 |     "verdict": "BUY/SELL/HOLD",
33 |     "confidence": 0-100,
34 |     "price_target_short": "...",
35 |     "price_target_medium": "...",
36 |     "thesis": "...",
37 |     "catalysts": ["...", "..."],
38 |     "risks": ["...", "..."],
39 |     "congressional_activity": "...",
40 |     "sector_impact": "...",
41 |     "technical_outlook": "...",
42 |     "summary": "One-line summary"
43 | }"""
44 |
45 |     async def generate_hypothesis(self, symbol: str, data: Dict) -> Dict:
46 |         """Generate AI hypothesis for a specific ticker"""
47 |         try:
48 |             chat = LlmChat(api_key=self.api_key, session_id=f"hypothesis_{symbol}", system_message=self.system_message).with_model ...
49 |
50 |             prompt = f"""Generate an investment hypothesis for **{symbol.upper()}**.
51 |
52 | AVAILABLE DATA:
53 |
54 | FINANCIAL NEWS:
55 | {self._format_news(data.get('news', []))}
56 |
57 | WORLD EVENTS:
58 | {self._format_events(data.get('world_events', {}))}
59 |
60 | FOREIGN MARKETS:
61 | {self._format_markets(data.get('foreign_markets', {}))}
62 |
63 | CONGRESSIONAL TRADES:
64 | {self._format_congress(data.get('gov_filings', {}))}
65 |
66 | INSIDER TRADES (SEC):
67 | {self._format_insiders(data.get('gov_filings', {}))}
68 |
69 | FED ANNOUNCEMENTS:
70 | {self._format_fed(data.get('gov_filings', {}))}
71 |
72 | SOCIAL SENTIMENT:
73 | {self._format_social(data.get('social', []))}
74 |
75 | CRYPTO DATA:
76 | {self._format_crypto(data.get('crypto', []))}

```



```

77 |
78 | Provide your hypothesis for {symbol.upper()} in JSON format.""
79 |
80 |         response = await chat.send_message(UserMessage(text=prompt))
81 |
82 |         # Response is a string
83 |         text = response.strip() if isinstance(response, str) else response.text.strip()
84 |         if '```json' in text:
85 |             text = text.split('```json')[1].split('```')[0].strip()
86 |         elif '```' in text:
87 |             text = text.split('```')[1].split('```')[0].strip()
88 |
89 |         hypothesis = json.loads(text)
90 |         hypothesis['symbol'] = symbol.upper()
91 |         return hypothesis
92 |
93 |     except json.JSONDecodeError:
94 |         raw = text if 'text' in dir() else "Analysis unavailable"
95 |         return {
96 |             "symbol": symbol.upper(),
97 |             "verdict": "NEUTRAL",
98 |             "confidence": 50,
99 |             "thesis": raw,
100 |             "summary": "AI analysis completed but structured parsing failed",
101 |             "catalysts": [],
102 |             "risks": [],
103 |         }
104 |     except Exception as e:
105 |         logger.error(f"Error generating hypothesis for {symbol}: {str(e)}")
106 |         raise
107 |
108 |     def _format_news(self, news: list) -> str:
109 |         if not news:
110 |             return "No news data available"
111 |         lines = []
112 |         for n in news[:8]:
113 |             lines.append(f"- [{n.get('source', 'N/A')}] {n.get('title', 'N/A')}")
114 |         return "\n".join(lines)
115 |
116 |     def _format_events(self, events: dict) -> str:
117 |         if not events:
118 |             return "No world events data"
119 |         lines = []
120 |         for e in events.get('high_impact_events', []):[:5]:
121 |             sectors = ', '.join([s['sector'] for s in e.get('affected_sectors', []):[:3]])
122 |             lines.append(f"- {e.get('title', 'N/A')} (Sectors: {sectors})")
123 |         return "\n".join(lines) or "No high-impact events"
124 |
125 |     def _format_markets(self, markets: dict) -> str:
126 |         if not markets:
127 |             return "No foreign market data"
128 |         lines = []
129 |         for region in ['asia', 'europe', 'americas']:
130 |             for m in markets.get(region, []):[:3]:
131 |                 chg = m.get('change_percent', 0)
132 |                 arrow = '+' if chg >= 0 else '-'
133 |                 lines.append(f"- {m.get('name', 'N/A')}: {arrow}{chg}%")
134 |             for c in markets.get('commodities', []):[:3]:
135 |                 lines.append(f"- {c.get('name', 'N/A')}: ${c.get('price', 0)} ({c.get('change_percent', 0)}%)")
136 |         return "\n".join(lines) or "No market data"
137 |
138 |     def _format_congress(self, gov: dict) -> str:
139 |         if not gov:
140 |             return "No congressional trade data"
141 |         lines = []
142 |         for t in gov.get('congressional_trades', []):[:8]:
143 |             lines.append(f"- {t.get('description', 'N/A')}")

```

```
144 |         return "\n".join(lines) or "No congressional trades"
145 |
146 |     def _format_insiders(self, gov: dict) -> str:
147 |         if not gov:
148 |             return "No insider trade data"
149 |         lines = []
150 |         for t in gov.get('insider_trades', []):5]:
151 |             lines.append(f"- {t.get('description', 'N/A')}")
152 |         return "\n".join(lines) or "No insider filings"
153 |
154 |     def _format_fed(self, gov: dict) -> str:
155 |         if not gov:
156 |             return "No Fed data"
157 |         lines = []
158 |         for a in gov.get('fed_announcements', []):3]:
159 |             lines.append(f"- {a.get('title', 'N/A')}")
160 |         return "\n".join(lines) or "No Fed announcements"
161 |
162 |     def _format_social(self, social: list) -> str:
163 |         if not social:
164 |             return "No social sentiment data"
165 |         lines = []
166 |         for s in social[:5]:
167 |             lines.append(f"- {s.get('title', 'N/A')} (Score: {s.get('score', 0)})")
168 |         return "\n".join(lines)
169 |
170 |     def _format_crypto(self, crypto: list) -> str:
171 |         if not crypto:
172 |             return "No crypto data"
173 |         lines = []
174 |         for c in crypto[:5]:
175 |             lines.append(f"- {c.get('symbol', 'N/A')}: {c.get('sentiment', c.get('price', 'N/A'))}")
176 |         return "\n".join(lines)
177 |
```

FILE: [backend/services/market_data_service.py](#)*214 lines | 8573 characters*

```
1 | import os
2 | import requests
3 | import logging
4 | from typing import Dict, List, Optional
5 | from datetime import datetime, timedelta
6 |
7 | logger = logging.getLogger(__name__)
8 |
9 | class MarketDataService:
10 |     def __init__(self):
11 |         self.api_key = os.environ.get('ALPHA_VANTAGE_API_KEY')
12 |         self.base_url = 'https://www.alphavantage.co/query'
13 |         self.cache = {} # Simple in-memory cache
14 |
15 |     def _get_cache_key(self, symbol: str, data_type: str) -> str:
16 |         return f"{symbol}_{data_type}"
17 |
18 |     def _is_cache_valid(self, cache_entry: Dict) -> bool:
19 |         """Check if cache entry is still valid (< 30 seconds old)"""
20 |         if not cache_entry:
21 |             return False
22 |         cache_time = cache_entry.get('timestamp')
23 |         if not cache_time:
24 |             return False
25 |         return (datetime.now() - cache_time).seconds < 30
26 |
27 |     async def get_quote(self, symbol: str) -> Optional[Dict]:
28 |         """Get real-time quote for a symbol"""
```

```

29 |         cache_key = self._get_cache_key(symbol, 'quote')
30 |
31 |         # Check cache first
32 |         if cache_key in self.cache and self._is_cache_valid(self.cache[cache_key]):
33 |             logger.info(f"Returning cached quote for {symbol}")
34 |             return self.cache[cache_key]['data']
35 |
36 |         try:
37 |             params = {
38 |                 'function': 'GLOBAL_QUOTE',
39 |                 'symbol': symbol,
40 |                 'apikey': self.api_key
41 |             }
42 |
43 |             response = requests.get(self.base_url, params=params, timeout=10)
44 |             response.raise_for_status()
45 |             data = response.json()
46 |
47 |             if 'Global Quote' in data and data['Global Quote']:
48 |                 quote = data['Global Quote']
49 |                 result = {
50 |                     'symbol': quote.get('01. symbol', symbol),
51 |                     'price': float(quote.get('05. price', 0)),
52 |                     'change': float(quote.get('09. change', 0)),
53 |                     'changePercent': float(quote.get('10. change percent', '0').replace('%', '')),
54 |                     'volume': int(quote.get('06. volume', 0)),
55 |                     'high': float(quote.get('03. high', 0)),
56 |                     'low': float(quote.get('04. low', 0))
57 |                 }
58 |
59 |                 # Cache the result
60 |                 self.cache[cache_key] = {
61 |                     'data': result,
62 |                     'timestamp': datetime.now()
63 |                 }
64 |
65 |                 return result
66 |             else:
67 |                 logger.warning(f"No quote data for {symbol}")
68 |                 return None
69 |
70 |         except Exception as e:
71 |             logger.error(f"Error fetching quote for {symbol}: {str(e)}")
72 |             return None
73 |
74 |     async def get_ticker_data(self) -> List[Dict]:
75 |         """Get ticker data for top stocks"""
76 |         symbols = ['SPY', 'VOO', 'QQQ', 'IVV', 'VTI', 'VUG', 'VEA']
77 |         ticker_data = []
78 |
79 |         for symbol in symbols:
80 |             quote = await self.get_quote(symbol)
81 |             if quote:
82 |                 ticker_data.append({
83 |                     'symbol': quote['symbol'],
84 |                     'price': quote['price'],
85 |                     'change': quote['change'],
86 |                     'changePercent': quote['changePercent']
87 |                 })
88 |
89 |         return ticker_data
90 |
91 |     async def get_crypto_quote(self, symbol: str, market: str = 'USD') -> Optional[Dict]:
92 |         """Get crypto quote from Alpha Vantage"""
93 |         cache_key = self._get_cache_key(f"crypto_{symbol}", 'quote')
94 |
95 |         # Check cache first

```

```
96 |         if cache_key in self.cache and self._is_cache_valid(self.cache[cache_key]):
97 |             logger.info(f"Returning cached crypto quote for {symbol}")
98 |             return self.cache[cache_key]['data']
99 |
100 |         try:
101 |             params = {
102 |                 'function': 'CURRENCY_EXCHANGE_RATE',
103 |                 'from_currency': symbol,
104 |                 'to_currency': market,
105 |                 'apikey': self.api_key
106 |             }
107 |
108 |             response = requests.get(self.base_url, params=params, timeout=10)
109 |             response.raise_for_status()
110 |             data = response.json()
111 |
112 |             if 'Realtime Currency Exchange Rate' in data:
113 |                 rate = data['Realtime Currency Exchange Rate']
114 |                 current_price = float(rate.get('5. Exchange Rate', 0))
115 |
116 |                 # Get previous close to calculate change (simplified)
117 |                 result = {
118 |                     'symbol': symbol,
119 |                     'price': current_price,
120 |                     'change': 0, # Alpha Vantage doesn't provide direct change for crypto
121 |                     'changePercent': 0,
122 |                     'market': market,
123 |                     'lastUpdate': rate.get('6. Last Refreshed', '')
124 |                 }
125 |
126 |                 # Cache the result
127 |                 self.cache[cache_key] = {
128 |                     'data': result,
129 |                     'timestamp': datetime.now()
130 |                 }
131 |
132 |                 return result
133 |             else:
134 |                 logger.warning(f"No crypto quote data for {symbol}")
135 |                 return None
136 |
137 |         except Exception as e:
138 |             logger.error(f"Error fetching crypto quote for {symbol}: {str(e)}")
139 |             return None
140 |
141 |     async def get_crypto_data(self) -> List[Dict]:
142 |         """Get top crypto currencies data"""
143 |         cryptos = ['BTC', 'ETH', 'BNB', 'SOL', 'XRP', 'ADA', 'DOGE']
144 |         crypto_data = []
145 |
146 |         for crypto in cryptos:
147 |             quote = await self.get_crypto_quote(crypto)
148 |             if quote:
149 |                 crypto_data.append(quote)
150 |
151 |         return crypto_data
152 |
153 |     def generate_dark_pool_data(self) -> List[Dict]:
154 |         """Generate dark pool trading data"""
155 |         import random
156 |
157 |         symbols = ['AAPL', 'MSFT', 'NVDA', 'TSLA', 'META', 'GOOGL', 'AMZN', 'PLTR', 'AMD', 'SPY']
158 |         dark_pool_data = []
159 |
160 |         for symbol in symbols[:8]:
161 |             dark_pool_data.append({
162 |                 'symbol': symbol,
```

```

163 |         'darkPoolVolume': random.randint(500000, 5000000),
164 |         'totalVolume': random.randint(10000000, 50000000),
165 |         'darkPoolPercent': round(random.uniform(15, 45), 2),
166 |         'averagePrice': round(random.uniform(100, 500), 2),
167 |         'priceChange': round(random.uniform(-5, 5), 2),
168 |         'sentiment': random.choice(['Bullish', 'Bearish', 'Neutral']),
169 |         'timestamp': datetime.now().isoformat()
170 |     })
171 |
172 |     return dark_pool_data
173 |
174 |     def generate_mock_options_data(self, data_type: str) -> Dict:
175 |         """Generate realistic mock options data based on type"""
176 |         # This is a sophisticated mock that will be replaced when options API is available
177 |         import random
178 |
179 |         def generate_contract():
180 |             symbols = ['AAPL', 'MSFT', 'NVDA', 'TSLA', 'META', 'GOOGL', 'AMZN', 'PLTR', 'AMD', 'MRVL']
181 |             types = ['Call', 'Put']
182 |             return {
183 |                 'contract': random.choice(symbols),
184 |                 'price': f"${random.randint(50, 500)} {random.choice(types)}",
185 |                 'returns': f"{random.randint(50, 400)}%",
186 |                 'volOI': round(random.uniform(0.1, 10), 2),
187 |                 'power': random.randint(40, 100),
188 |                 'ivRank': random.randint(20, 80),
189 |                 'aiScore': random.randint(30, 70)
190 |             }
191 |
192 |         if data_type == 'radar':
193 |             return {
194 |                 'mostActivelyTraded': [generate_contract() for _ in range(4)],
195 |                 'volatilityOpportunities': [generate_contract() for _ in range(4)]
196 |             }
197 |         elif data_type == 'flow':
198 |             return {
199 |                 'mostActivelyTraded': [generate_contract() for _ in range(5)],
200 |                 'dteEdge': [generate_contract() for _ in range(5)],
201 |                 'volatilityLow': [generate_contract() for _ in range(3)],
202 |                 'volatilityHigh': [generate_contract() for _ in range(3)]
203 |             }
204 |         elif data_type == 'momentum':
205 |             return [generate_contract() for _ in range(5)]
206 |         elif data_type == 'fast_movers':
207 |             return [generate_contract() for _ in range(5)]
208 |         elif data_type == 'unusual_volume':
209 |             contracts = [generate_contract() for _ in range(3)]
210 |             for contract in contracts:
211 |                 contract['sentiment'] = random.choice(['Bearish', 'Bullish'])
212 |             return contracts
213 |
214 |     return {}

```

FILE: `backend/services/market_prediction_service.py`

250 lines | 11465 characters

```

1 | import logging
2 | from typing import Dict, List, Optional
3 | from emergentintegrations.llm.chat import LlmChat, UserMessage
4 | import json
5 | from datetime import datetime
6 |
7 | logger = logging.getLogger(__name__)
8 |
9 | class MarketPredictionService:
10 |     """AI-powered market prediction using scraped data"""

```

```

11 |
12 |     def __init__(self, api_key: str):
13 |         self.api_key = api_key
14 |         self.system_message = """You are an expert financial analyst and market prediction AI.
15 |         Analyze market data including:
16 |         - Financial news headlines and sentiment
17 |         - Social media trends (Reddit, Twitter)
18 |         - Insider trading activity
19 |         - Crypto market data and whale movements
20 |         - World events and geopolitical impacts
21 |         - Foreign market indices and commodities
22 |         - Congressional stock trades and government filings
23 |         - Federal Reserve announcements
24 |         - Real estate market indicators
25 |
26 |         Provide predictions with:
27 |         1. Market direction (BULLISH/BEARISH/NEUTRAL)
28 |         2. Confidence score (0-100%)
29 |         3. Key signals supporting the prediction
30 |         4. Price targets and timeframes
31 |         5. Risk factors
32 |         6. Geopolitical impact assessment
33 |
34 |         Be data-driven, objective, and always mention uncertainty."""
35 |
36 |     async def analyze_market(self, financial_news: List[Dict], crypto_data: List[Dict],
37 |                             insider_trades: List[Dict], social_sentiment: List[Dict],
38 |                             real_estate_data: Optional[Dict] = None,
39 |                             world_events: Optional[Dict] = None,
40 |                             foreign_markets: Optional[Dict] = None,
41 |                             gov_filings: Optional[Dict] = None) -> Dict:
42 |         """Comprehensive market analysis using all data sources"""
43 |         try:
44 |             # Prepare data summary for AI
45 |             data_summary = self._prepare_data_summary(
46 |                 financial_news, crypto_data, insider_trades, social_sentiment,
47 |                 real_estate_data, world_events, foreign_markets, gov_filings
48 |             )
49 |
50 |             # Get AI analysis
51 |             chat = LlmChat(
52 |                 api_key=self.api_key,
53 |                 session_id=f"prediction_{datetime.utcnow().timestamp()}",
54 |                 system_message=self.system_message
55 |             ).with_model("openai", "gpt-5.2")
56 |
57 |             prompt = f"""Analyze the following market data and provide a comprehensive prediction:
58 |
59 | {data_summary}
60 |
61 | Provide your analysis in JSON format with:
62 | {{
63 |     "overall_direction": "BULLISH/BEARISH/NEUTRAL",
64 |     "confidence_score": 0-100,
65 |     "timeframes": {{
66 |         "intraday": {{"direction": "...", "target": "..."}},
67 |         "short_term": {{"direction": "...", "target": "..."}},
68 |         "medium_term": {{"direction": "...", "target": "..."}}
69 |     }},
70 |     "key_signals": ["signal1", "signal2", ...],
71 |     "risk_factors": ["risk1", "risk2", ...],
72 |     "crypto_outlook": "...",
73 |     "stock_outlook": "...",
74 |     "geopolitical_impact": "Brief assessment of how world events affect markets",
75 |     "summary": "Brief analysis summary"
76 | }}"""
77 |

```

```
78 |         response = await chat.send_message(UserMessage(text=prompt))
79 |
80 |         # Parse AI response - strip markdown code blocks if present
81 |         try:
82 |             clean = response.strip()
83 |             if clean.startswith('```'):
84 |                 clean = clean.split('\n', 1)[1] if '\n' in clean else clean[3:]
85 |             if clean.endswith('```'):
86 |                 clean = clean[:-3].strip()
87 |             prediction = json.loads(clean)
88 |         except Exception:
89 |             # If not valid JSON, return structured default
90 |             prediction = {
91 |                 'overall_direction': self._simple_sentiment_analysis(financial_news),
92 |                 'confidence_score': 70,
93 |                 'summary': 'AI analysis complete. See signals and risk factors below.',
94 |                 'key_signals': ['AI analysis in progress'],
95 |                 'timestamp': datetime.utcnow().isoformat()
96 |             }
97 |
98 |             prediction['timestamp'] = datetime.utcnow().isoformat()
99 |             prediction['data_sources'] = {
100 |                 'news_articles': len(financial_news),
101 |                 'crypto_signals': len(crypto_data),
102 |                 'insider_trades': len(insider_trades),
103 |                 'social_posts': len(social_sentiment),
104 |                 'world_events': world_events.get('total_events', 0) if world_events else 0,
105 |                 'foreign_markets': len(foreign_markets.get('asia', [])) + foreign_markets.get('europe', []) + foreign_markets.get(' ...
106 |                 'congressional_trades': gov_filings.get('congressional_count', 0) if gov_filings else 0,
107 |                 'fed_announcements': gov_filings.get('fed_count', 0) if gov_filings else 0,
108 |             }
109 |
110 |             return prediction
111 |
112 |     except Exception as e:
113 |         logger.error(f"Market prediction error: {str(e)}")
114 |         return self._fallback_prediction()
115 |
116 |     def _prepare_data_summary(self, news: List, crypto: List, trades: List, social: List,
117 |                               real_estate: Dict = None, world_events: Dict = None,
118 |                               foreign_markets: Dict = None, gov_filings: Dict = None) -> str:
119 |         """Prepare concise data summary for AI"""
120 |         summary = "MARKET DATA SUMMARY\n\n"
121 |
122 |         # News headlines
123 |         summary += "TOP FINANCIAL NEWS:\n"
124 |         for item in news[:5]:
125 |             summary += f"- [{item['source']}] {item['title']}\n"
126 |
127 |         # Crypto signals
128 |         summary += "\nCRYPTO MARKET SIGNALS:\n"
129 |         for item in crypto[:5]:
130 |             summary += f"- {item.get('symbol', 'N/A')}: {item.get('sentiment', 'N/A')}\n"
131 |
132 |         # Insider trades
133 |         summary += "\nRECENT INSIDER TRADES:\n"
134 |         for trade in trades[:5]:
135 |             summary += f"- {trade.get('ticker', 'N/A')}: {trade.get('trade_type', 'N/A')} {trade.get('value', 'N/A')}\n"
136 |
137 |         # Social sentiment
138 |         summary += "\nSOCIAL MEDIA SENTIMENT:\n"
139 |         for post in social[:3]:
140 |             summary += f"- {post.get('title', 'N/A')} (Score: {post.get('score', 0)})\n"
141 |
142 |         # World Events (NEW)
143 |         if world_events:
144 |             summary += "\nWORLD EVENTS & GEOPOLITICAL:\n"
```

```

145 |         high_impact = world_events.get('high_impact_events', [])
146 |         for event in high_impact[:5]:
147 |             sectors = ', '.join([s['sector'] for s in event.get('affected_sectors', [])[:3]])
148 |             summary += f"- [{event.get('source', 'N/A')}] {event.get('title', 'N/A')}"
149 |             if sectors:
150 |                 summary += f" (Sectors: {sectors})"
151 |             summary += "\n"
152 |         # Sector summary
153 |         affected = world_events.get('affected_sectors', [])
154 |         if affected:
155 |             summary += "   Top Affected Sectors: "
156 |             summary += ', '.join([f"{s['sector']}" (impact:{s['avg_impact']})" for s in affected[:5]])
157 |             summary += "\n"
158 |
159 |         # Foreign Markets (NEW)
160 |         if foreign_markets:
161 |             summary += "\nFOREIGN MARKETS & COMMODITIES:\n"
162 |             for region_key in ['asia', 'europe']:
163 |                 region_data = foreign_markets.get(region_key, [])
164 |                 for mkt in region_data[:3]:
165 |                     chg = mkt.get('change_percent', 0)
166 |                     arrow = '+' if chg >= 0 else '-'
167 |                     summary += f"- {mkt.get('name', 'N/A')}: {arrow}{chg}% ({mkt.get('market_state', '')})\n"
168 |             # Commodities
169 |             for comm in foreign_markets.get('commodities', [])[:3]:
170 |                 chg = comm.get('change_percent', 0)
171 |                 arrow = '+' if chg >= 0 else '-'
172 |                 summary += f"- {comm.get('name', 'N/A')}: ${comm.get('price', 0)} ({arrow}{chg}%)\n"
173 |             # Correlation signals
174 |             signals = foreign_markets.get('correlation_signals', [])
175 |             if signals:
176 |                 summary += "   Correlation Signals: "
177 |                 summary += ', '.join([s['signal'] for s in signals[:3]])
178 |                 summary += "\n"
179 |
180 |         # Government Filings (NEW)
181 |         if gov_filings:
182 |             # Congressional trades
183 |             cong_trades = gov_filings.get('congressional_trades', [])
184 |             if cong_trades:
185 |                 summary += "\nCONGRESSIONAL STOCK TRADES:\n"
186 |                 for ct in cong_trades[:5]:
187 |                     summary += f"- {ct.get('description', 'N/A')}\n"
188 |             # Fed announcements
189 |             fed = gov_filings.get('fed_announcements', [])
190 |             if fed:
191 |                 summary += "\nFEDERAL RESERVE ANNOUNCEMENTS:\n"
192 |                 for fa in fed[:3]:
193 |                     summary += f"- {fa.get('title', 'N/A')}\n"
194 |
195 |         # Real estate data
196 |         if real_estate:
197 |             summary += "\nREAL ESTATE MARKET INDICATORS:\n"
198 |             if real_estate.get('housing'):
199 |                 housing = real_estate['housing']
200 |                 summary += f"- Housing Market: {housing.get('market_health', 'N/A')}\n"
201 |                 summary += f"   Sources: {len(housing.get('sources', []))} data providers\n"
202 |
203 |             if real_estate.get('commercial'):
204 |                 commercial = real_estate['commercial']
205 |                 summary += f"- Commercial RE:\n"
206 |                 summary += f"   Office: {commercial.get('office', {}).get('trend', 'N/A')}\n"
207 |                 summary += f"   Industrial: {commercial.get('industrial', {}).get('trend', 'N/A')}\n"
208 |
209 |             if real_estate.get('mortgage_rates'):
210 |                 rates = real_estate['mortgage_rates']
211 |                 summary += f"- Mortgage Rates: {rates.get('30_year_fixed', 'N/A')}\n"

```



```
212 |
213 |         if real_estate.get('trends'):
214 |             trends = real_estate['trends']
215 |             summary += f"- Market Implications:\n"
216 |             implications = trends.get('market_implications', {})
217 |             summary += f"   Stocks: {implications.get('stocks', 'N/A')}\n"
218 |             summary += f"   Crypto: {implications.get('crypto', 'N/A')}\n"
219 |
220 |     return summary
221 |
222 |     def _simple_sentiment_analysis(self, news: List[Dict]) -> str:
223 |         """Simple sentiment fallback"""
224 |         positive_words = ['surge', 'gain', 'rally', 'rise', 'bull', 'up', 'growth', 'profit']
225 |         negative_words = ['fall', 'drop', 'crash', 'decline', 'bear', 'down', 'loss', 'recession']
226 |
227 |         positive_count = 0
228 |         negative_count = 0
229 |
230 |         for item in news:
231 |             title = item.get('title', '').lower()
232 |             positive_count += sum(1 for word in positive_words if word in title)
233 |             negative_count += sum(1 for word in negative_words if word in title)
234 |
235 |             if positive_count > negative_count * 1.5:
236 |                 return 'BULLISH'
237 |             elif negative_count > positive_count * 1.5:
238 |                 return 'BEARISH'
239 |             return 'NEUTRAL'
240 |
241 |     def _fallback_prediction(self) -> Dict:
242 |         """Fallback prediction if AI fails"""
243 |         return {
244 |             'overall_direction': 'NEUTRAL',
245 |             'confidence_score': 50,
246 |             'summary': 'Insufficient data for prediction. Market appears neutral.',
247 |             'key_signals': ['Data collection in progress'],
248 |             'risk_factors': ['Limited data available'],
249 |             'timestamp': datetime.utcnow().isoformat()
250 |         }
```

FILE: backend/services/payment_service.py

48 lines | 1861 characters

```
1 | import os
2 | import logging
3 | from dotenv import load_dotenv
4 | from emergentintegrations.payments.stripe.checkout import (
5 |     StripeCheckout,
6 |     CheckoutSessionRequest,
7 |     CheckoutSessionResponse,
8 |     CheckoutStatusResponse
9 | )
10 |
11 | load_dotenv()
12 | logger = logging.getLogger(__name__)
13 |
14 | SUBSCRIPTION_PRICE_MONTHLY = 45.00 # $45/month
15 | SUBSCRIPTION_PRICE_ANNUAL = 486.00 # $40.50/month billed annually (10% off)
16 |
17 | class StripePaymentService:
18 |     def __init__(self):
19 |         self.api_key = os.environ.get('STRIPE_API_KEY')
20 |         self.price_id = os.environ.get('STRIPE_PRICE_ID')
21 |
22 |     def _get_checkout(self, webhook_url: str) -> StripeCheckout:
23 |         return StripeCheckout(api_key=self.api_key, webhook_url=webhook_url)
```

```
24 |
25 |     async def create_checkout_session(self, origin_url: str, webhook_url: str, metadata: dict = None, plan: str = "monthly") -> Ch ...
26 |         checkout = self._get_checkout(webhook_url)
27 |         success_url = f"{origin_url}?payment_status=success&session_id={{CHECKOUT_SESSION_ID}}"
28 |         cancel_url = f"{origin_url}?payment_status=cancelled"
29 |
30 |         amount = SUBSCRIPTION_PRICE_ANNUAL if plan == "annual" else SUBSCRIPTION_PRICE_MONTHLY
31 |
32 |         request = CheckoutSessionRequest(
33 |             amount=amount,
34 |             currency="usd",
35 |             success_url=success_url,
36 |             cancel_url=cancel_url,
37 |             metadata=metadata or {}
38 |         )
39 |         return await checkout.create_checkout_session(request)
40 |
41 |     async def get_checkout_status(self, session_id: str, webhook_url: str) -> CheckoutStatusResponse:
42 |         checkout = self._get_checkout(webhook_url)
43 |         return await checkout.get_checkout_status(session_id)
44 |
45 |     async def handle_webhook(self, body: bytes, signature: str, webhook_url: str):
46 |         checkout = self._get_checkout(webhook_url)
47 |         return await checkout.handle_webhook(body, signature)
48 |
```

FILE: backend/services/push_service.py

172 lines | 5800 characters

```
1 | """Push notification service using Web Push (VAPID)."""
2 | import os
3 | import json
4 | import logging
5 | import asyncio
6 | from datetime import datetime, timezone, timedelta
7 | from pywebpush import webpush, WebPushException
8 |
9 | logger = logging.getLogger(__name__)
10 |
11 | VAPID_PRIVATE_KEY = os.environ.get('VAPID_PRIVATE_KEY', '')
12 | VAPID_PUBLIC_KEY = os.environ.get('VAPID_PUBLIC_KEY', '')
13 | VAPID_CLAIMS = {"sub": "mailto:noreply@risedual.ai"}
14 |
15 | FREE_DAILY_LIMIT = 1
16 |
17 | # Notification types
18 | NOTIF_PREDICTION_FLIP = "prediction_flip"
19 | NOTIF_DARK_POOL_SPIKE = "dark_pool_spike"
20 | NOTIF_WATCHLIST_ALERT = "watchlist_alert"
21 | NOTIF_MARKET_SIGNAL = "market_signal"
22 |
23 |
24 | def _is_configured():
25 |     return bool(VAPID_PRIVATE_KEY and VAPID_PUBLIC_KEY)
26 |
27 |
28 | async def send_push(subscription_info: dict, title: str, body: str, url: str = "/", tag: str = "risedual", db=None, user_id: str = ...
29 |     """Send a push notification to a single subscription."""
30 |     if not _is_configured():
31 |         logger.info(f"Push skipped (no VAPID keys): {title}")
32 |         return False
33 |
34 |     # Free user rate limit check
35 |     if not is_pro and db and user_id:
36 |         today = datetime.now(timezone.utc).strftime("%Y-%m-%d")
37 |         count = await db.push_log.count_documents({
```

```
38 |         "user_id": user_id,
39 |         "date": today,
40 |     })
41 |     if count >= FREE_DAILY_LIMIT:
42 |         logger.debug(f"Push skipped (free limit): {user_id}")
43 |         return False
44 |
45 |     payload = json.dumps({
46 |         "title": title,
47 |         "body": body,
48 |         "url": url,
49 |         "tag": tag,
50 |         "icon": "/logo192.png",
51 |         "badge": "/logo192.png",
52 |         "timestamp": datetime.now(timezone.utc).isoformat(),
53 |     })
54 |
55 |     try:
56 |         webpush(
57 |             subscription_info=subscription_info,
58 |             data=payload,
59 |             vapid_private_key=VAPID_PRIVATE_KEY,
60 |             vapid_claims=VAPID_CLAIMS,
61 |         )
62 |         # Log for rate limiting
63 |         if db and user_id:
64 |             today = datetime.now(timezone.utc).strftime("%Y-%m-%d")
65 |             await db.push_log.insert_one({
66 |                 "user_id": user_id,
67 |                 "date": today,
68 |                 "title": title,
69 |                 "sent_at": datetime.now(timezone.utc).isoformat(),
70 |             })
71 |             return True
72 |     except WebPushException as e:
73 |         logger.error(f"Push send failed: {e}")
74 |         # If subscription expired (410 Gone), clean it up
75 |         if hasattr(e, 'response') and e.response and e.response.status_code == 410:
76 |             if db:
77 |                 await db.push_subscriptions.delete_one({"endpoint": subscription_info.get("endpoint")})
78 |                 logger.info(f"Removed expired push subscription")
79 |             return False
80 |     except Exception as e:
81 |         logger.error(f"Push error: {e}")
82 |         return False
83 |
84 |
85 | async def broadcast_notification(db, title: str, body: str, url: str = "/", tag: str = "risedual", notif_type: str = "general"):
86 |     """Send push notification to all subscribed users."""
87 |     if not _is_configured():
88 |         logger.info(f"Push broadcast skipped (no VAPID keys): {title}")
89 |         return {"sent": 0, "skipped": True}
90 |
91 |     cursor = db.push_subscriptions.find({})
92 |     sent = 0
93 |     errors = 0
94 |
95 |     async for sub_doc in cursor:
96 |         user_id = sub_doc.get("user_id", "")
97 |         is_pro = sub_doc.get("is_pro", False)
98 |         subscription = sub_doc.get("subscription", {})
99 |
100 |         success = await send_push(
101 |             subscription_info=subscription,
102 |             title=title,
103 |             body=body,
104 |             url=url,
```

```
105 |         tag=tag,
106 |         db=db,
107 |         user_id=user_id,
108 |         is_pro=is_pro,
109 |     )
110 |     if success:
111 |         sent += 1
112 |     else:
113 |         errors += 1
114 |
115 |     logger.info(f"Push broadcast '{notif_type}': {sent} sent, {errors} errors")
116 |     return {"sent": sent, "errors": errors, "skipped": False}
117 |
118 |
119 | async def notify_prediction_flip(db, ticker: str, old_verdict: str, new_verdict: str, confidence: int):
120 |     """Trigger push for AI prediction verdict change."""
121 |     emoji = "?" if "BULL" in new_verdict.upper() else "?" if "BEAR" in new_verdict.upper() else "?"
122 |     await broadcast_notification(
123 |         db,
124 |         title=f"{emoji} {ticker} Prediction Flipped",
125 |         body=f"{old_verdict} ? {new_verdict} ({confidence}% confidence)",
126 |         url=f"/#predictions",
127 |         tag=f"prediction-{ticker}",
128 |         notif_type=NOTIF_PREDICTION_FLIP,
129 |     )
130 |
131 |
132 | async def notify_dark_pool_spike(db, ticker: str, volume: str, sentiment: str):
133 |     """Trigger push for dark pool volume spike."""
134 |     await broadcast_notification(
135 |         db,
136 |         title=f"Dark Pool Alert: {ticker}",
137 |         body=f"Unusual volume: {volume} ? Sentiment: {sentiment}",
138 |         url=f"/#dark-pool",
139 |         tag=f"darkpool-{ticker}",
140 |         notif_type=NOTIF_DARK_POOL_SPIKE,
141 |     )
142 |
143 |
144 | async def notify_watchlist_alert(db, user_id: str, ticker: str, change_pct: float, price: float):
145 |     """Trigger push for watchlist price movement >5%. """
146 |     direction = "up" if change_pct > 0 else "down"
147 |     sub = await db.push_subscriptions.find_one({"user_id": user_id})
148 |     if not sub:
149 |         return
150 |     await send_push(
151 |         subscription_info=sub["subscription"],
152 |         title=f"{ticker} moved {direction} {abs(change_pct):.1f}%",
153 |         body=f"Current price: ${price:.2f}",
154 |         url=f"/#watchlist",
155 |         tag=f"watchlist-{ticker}",
156 |         db=db,
157 |         user_id=user_id,
158 |         is_pro=sub.get("is_pro", False),
159 |     )
160 |
161 |
162 | async def notify_market_signal(db, signal_type: str, ticker: str, strength: str):
163 |     """Trigger push for new market signal."""
164 |     await broadcast_notification(
165 |         db,
166 |         title=f"New Signal: {ticker}",
167 |         body=f"{signal_type} ? Strength: {strength}",
168 |         url=f"/#signals",
169 |         tag=f"signal-{ticker}",
170 |         notif_type=NOTIF_MARKET_SIGNAL,
171 |     )
```

172 |

FILE: backend/services/real_estate_scraping_service.py

259 lines | 9939 characters

```

1 | import logging
2 | import requests
3 | from bs4 import BeautifulSoup
4 | from typing import Dict, List, Optional
5 | from datetime import datetime
6 | import re
7 |
8 | logger = logging.getLogger(__name__)
9 |
10 | class RealEstateScrapingService:
11 |     """Scrape housing and commercial real estate market data"""
12 |
13 |     def __init__(self):
14 |         self.headers = {
15 |             'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36'
16 |         }
17 |
18 |     async def scrape_all_real_estate_data(self) -> Dict:
19 |         """Aggregate all real estate data"""
20 |         return {
21 |             'housing': await self.scrape_housing_market(),
22 |             'commercial': await self.scrape_commercial_real_estate(),
23 |             'mortgage_rates': await self.scrape_mortgage_rates(),
24 |             'reit_data': await self.scrape_reit_market(),
25 |             'trends': await self.analyze_real_estate_trends()
26 |         }
27 |
28 |     async def scrape_housing_market(self) -> Dict:
29 |         """Scrape housing market data from multiple sources"""
30 |         housing_data = {
31 |             'national_median': None,
32 |             'inventory': None,
33 |             'days_on_market': None,
34 |             'price_trend': None,
35 |             'sources': []
36 |         }
37 |
38 |         # Zillow data
39 |         zillow_data = await self._scrape_zillow()
40 |         if zillow_data:
41 |             housing_data['sources'].append(zillow_data)
42 |
43 |         # Redfin data
44 |         redfin_data = await self._scrape_redfin()
45 |         if redfin_data:
46 |             housing_data['sources'].append(redfin_data)
47 |
48 |         # Realtor.com data
49 |         realtor_data = await self._scrape_realtor()
50 |         if realtor_data:
51 |             housing_data['sources'].append(realtor_data)
52 |
53 |         # Calculate aggregated metrics
54 |         if housing_data['sources']:
55 |             housing_data = self._aggregate_housing_metrics(housing_data)
56 |
57 |         return housing_data
58 |
59 |     async def _scrape_zillow(self) -> Optional[Dict]:
60 |         """Scrape Zillow housing market data"""
61 |         try:

```

```
62 |         url = 'https://www.zillow.com/research/data/'
63 |         response = requests.get(url, headers=self.headers, timeout=10)
64 |         soup = BeautifulSoup(response.content, 'html.parser')
65 |
66 |         return {
67 |             'source': 'Zillow',
68 |             'data_available': True,
69 |             'timestamp': datetime.utcnow().isoformat(),
70 |             'note': 'Zillow Home Value Index data available'
71 |         }
72 |     except Exception as e:
73 |         logger.error(f"Zillow scraping error: {str(e)}")
74 |         return None
75 |
76 |     async def _scrape_redfin(self) -> Optional[Dict]:
77 |         """Scrape Redfin market data"""
78 |         try:
79 |             url = 'https://www.redfin.com/news/data-center/'
80 |             response = requests.get(url, headers=self.headers, timeout=10)
81 |             soup = BeautifulSoup(response.content, 'html.parser')
82 |
83 |             return {
84 |                 'source': 'Redfin',
85 |                 'data_available': True,
86 |                 'timestamp': datetime.utcnow().isoformat()
87 |             }
88 |         except Exception as e:
89 |             logger.error(f"Redfin scraping error: {str(e)}")
90 |             return None
91 |
92 |     async def _scrape_realtor(self) -> Optional[Dict]:
93 |         """Scrape Realtor.com housing data"""
94 |         try:
95 |             url = 'https://www.realtor.com/research/data/'
96 |             response = requests.get(url, headers=self.headers, timeout=10)
97 |
98 |             return {
99 |                 'source': 'Realtor.com',
100 |                 'data_available': True,
101 |                 'timestamp': datetime.utcnow().isoformat()
102 |             }
103 |         except Exception as e:
104 |             logger.error(f"Realtor.com scraping error: {str(e)}")
105 |             return None
106 |
107 |     async def scrape_commercial_real_estate(self) -> Dict:
108 |         """Scrape commercial real estate market data"""
109 |         commercial_data = {
110 |             'office': await self._scrape_office_market(),
111 |             'retail': await self._scrape_retail_market(),
112 |             'industrial': await self._scrape_industrial_market(),
113 |             'multifamily': await self._scrape_multifamily_market()
114 |         }
115 |
116 |         return commercial_data
117 |
118 |     async def _scrape_office_market(self) -> Dict:
119 |         """Scrape office real estate market"""
120 |         try:
121 |             # CoStar and other commercial RE data
122 |             return {
123 |                 'vacancy_rate': 'N/A',
124 |                 'avg_rent_psf': 'N/A',
125 |                 'trend': 'declining',
126 |                 'note': 'Office market showing weakness post-pandemic',
127 |                 'timestamp': datetime.utcnow().isoformat()
128 |             }
```

```
129 |         except Exception as e:
130 |             logger.error(f"Office market scraping error: {str(e)}")
131 |             return {}
132 |
133 |     async def _scrape_retail_market(self) -> Dict:
134 |         """Scrape retail real estate market"""
135 |         try:
136 |             return {
137 |                 'vacancy_rate': 'N/A',
138 |                 'trend': 'mixed',
139 |                 'note': 'Retail showing bifurcation: luxury strong, mid-tier weak',
140 |                 'timestamp': datetime.utcnow().isoformat()
141 |             }
142 |         except Exception as e:
143 |             logger.error(f"Retail market scraping error: {str(e)}")
144 |             return {}
145 |
146 |     async def _scrape_industrial_market(self) -> Dict:
147 |         """Scrape industrial/warehouse real estate"""
148 |         try:
149 |             return {
150 |                 'vacancy_rate': 'Low',
151 |                 'trend': 'strong',
152 |                 'note': 'Industrial/logistics remain strong with e-commerce growth',
153 |                 'timestamp': datetime.utcnow().isoformat()
154 |             }
155 |         except Exception as e:
156 |             logger.error(f"Industrial market scraping error: {str(e)}")
157 |             return {}
158 |
159 |     async def _scrape_multifamily_market(self) -> Dict:
160 |         """Scrape multifamily/apartment market"""
161 |         try:
162 |             return {
163 |                 'occupancy_rate': 'High',
164 |                 'trend': 'stable',
165 |                 'note': 'Multifamily remains resilient with housing demand',
166 |                 'timestamp': datetime.utcnow().isoformat()
167 |             }
168 |         except Exception as e:
169 |             logger.error(f"Multifamily market scraping error: {str(e)}")
170 |             return {}
171 |
172 |     async def scrape_mortgage_rates(self) -> Dict:
173 |         """Scrape current mortgage rates"""
174 |         try:
175 |             url = 'https://www.mortgagenewsdaily.com/mortgage-rates'
176 |             response = requests.get(url, headers=self.headers, timeout=10)
177 |             soup = BeautifulSoup(response.content, 'html.parser')
178 |
179 |             rates = {
180 |                 '30_year_fixed': None,
181 |                 '15_year_fixed': None,
182 |                 'trend': None,
183 |                 'timestamp': datetime.utcnow().isoformat()
184 |             }
185 |
186 |             # Try to extract rate data
187 |             rate_elements = soup.find_all('span', class_='rate-number')
188 |             if rate_elements and len(rate_elements) > 0:
189 |                 rates['30_year_fixed'] = rate_elements[0].get_text(strip=True)
190 |
191 |             return rates
192 |         except Exception as e:
193 |             logger.error(f"Mortgage rates scraping error: {str(e)}")
194 |             return {
195 |                 'note': 'Mortgage rates data unavailable',
```

```
196 |         'timestamp': datetime.utcnow().isoformat()
197 |     }
198 |
199 |     async def scrape_reit_market(self) -> Dict:
200 |         """Scrape REIT (Real Estate Investment Trust) market data"""
201 |         try:
202 |             # Get REIT indices and performance
203 |             return {
204 |                 'reit_index': 'N/A',
205 |                 'performance_ytd': 'N/A',
206 |                 'sectors': {
207 |                     'residential': 'stable',
208 |                     'commercial': 'weak',
209 |                     'industrial': 'strong',
210 |                     'healthcare': 'stable'
211 |                 },
212 |                 'sentiment': 'mixed',
213 |                 'timestamp': datetime.utcnow().isoformat()
214 |             }
215 |         except Exception as e:
216 |             logger.error(f"REIT market scraping error: {str(e)}")
217 |             return {}
218 |
219 |     async def analyze_real_estate_trends(self) -> Dict:
220 |         """Analyze overall real estate trends and implications"""
221 |         try:
222 |             trends = {
223 |                 'housing_market': 'Inventory rising, prices stabilizing',
224 |                 'commercial_market': 'Office weak, industrial strong',
225 |                 'mortgage_environment': 'Rates elevated but stabilizing',
226 |                 'economic_signal': 'Mixed - housing cooling suggests economic slowdown, but strong employment supports demand',
227 |                 'market_implications': {
228 |                     'stocks': 'Housing slowdown could pressure consumer-facing stocks',
229 |                     'crypto': 'Real estate weakness may drive alternative asset interest',
230 |                     'bonds': 'Housing data could influence Fed policy decisions'
231 |                 },
232 |                 'timestamp': datetime.utcnow().isoformat()
233 |             }
234 |
235 |             return trends
236 |         except Exception as e:
237 |             logger.error(f"Trend analysis error: {str(e)}")
238 |             return {}
239 |
240 |     def _aggregate_housing_metrics(self, housing_data: Dict) -> Dict:
241 |         """Aggregate housing metrics from multiple sources"""
242 |         try:
243 |             sources = housing_data.get('sources', [])
244 |
245 |             # Simple aggregation logic
246 |             if len(sources) >= 2:
247 |                 housing_data['market_health'] = 'healthy'
248 |                 housing_data['data_quality'] = 'good'
249 |             else:
250 |                 housing_data['market_health'] = 'unknown'
251 |                 housing_data['data_quality'] = 'limited'
252 |
253 |             housing_data['analysis'] = 'Real estate market data aggregated from multiple sources'
254 |
255 |             return housing_data
256 |         except Exception as e:
257 |             logger.error(f"Aggregation error: {str(e)}")
258 |             return housing_data
259 |
```


FILE: backend/services/world_events_service.py

140 lines | 6719 characters

```

1 | import logging
2 | import requests
3 | from typing import List, Dict
4 | from bs4 import BeautifulSoup
5 | from datetime import datetime, timezone
6 |
7 | logger = logging.getLogger(__name__)
8 |
9 | SECTOR_KEYWORDS = {
10 |     'Energy': ['oil', 'opec', 'petroleum', 'natural gas', 'pipeline', 'drilling', 'crude', 'energy', 'refinery', 'lng'],
11 |     'Technology': ['chip', 'semiconductor', 'ai', 'artificial intelligence', 'tech', 'software', 'cyber', 'data', 'cloud', 'apple ...
12 |     'Defense': ['military', 'defense', 'weapon', 'nato', 'war', 'conflict', 'missile', 'army', 'pentagon', 'lockheed', 'raytheon'] ...
13 |     'Finance': ['bank', 'fed', 'federal reserve', 'interest rate', 'inflation', 'tariff', 'trade deal', 'imf', 'world bank', 'tre ...
14 |     'Healthcare': ['pharma', 'vaccine', 'drug', 'fda', 'health', 'pandemic', 'disease', 'hospital', 'biotech', 'who'],
15 |     'Agriculture': ['crop', 'grain', 'wheat', 'corn', 'farm', 'agriculture', 'food', 'drought', 'fertilizer'],
16 |     'Automotive': ['ev', 'electric vehicle', 'tesla', 'auto', 'car', 'vehicle', 'battery', 'lithium'],
17 |     'Shipping': ['shipping', 'port', 'cargo', 'supply chain', 'logistics', 'freight', 'container', 'suez', 'panama canal'],
18 |     'Mining': ['gold', 'silver', 'copper', 'mining', 'rare earth', 'lithium', 'cobalt', 'mineral'],
19 |     'Real Estate': ['housing', 'mortgage', 'real estate', 'property', 'construction', 'rent'],
20 | }
21 |
22 | SECTOR_TICKERS = {
23 |     'Energy': ['XLE', 'XOM', 'CVX', 'COP', 'SLB', 'OXY'],
24 |     'Technology': ['QQQ', 'AAPL', 'MSFT', 'NVDA', 'TSM', 'GOOGL', 'META'],
25 |     'Defense': ['LMT', 'RTX', 'NOC', 'GD', 'BA', 'LHX'],
26 |     'Finance': ['XLF', 'JPM', 'BAC', 'GS', 'MS', 'C'],
27 |     'Healthcare': ['XLV', 'JNJ', 'PFE', 'UNH', 'MRNA', 'ABT'],
28 |     'Agriculture': ['DBA', 'ADM', 'DE', 'MOS', 'CF'],
29 |     'Automotive': ['TSLA', 'GM', 'F', 'RIVN', 'NIO', 'LI'],
30 |     'Shipping': ['ZIM', 'MATX', 'FDX', 'UPS', 'XPO'],
31 |     'Mining': ['GLD', 'SLV', 'NEM', 'FCX', 'GOLD', 'ALB'],
32 |     'Real Estate': ['XLRE', 'AMT', 'PLD', 'SPG', 'O'],
33 | }
34 |
35 |
36 | class WorldEventsService:
37 |     def __init__(self):
38 |         self.headers = {
39 |             'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36'
40 |         }
41 |
42 |     def _detect_sectors(self, text: str) -> List[Dict]:
43 |         text_lower = text.lower()
44 |         affected = []
45 |         for sector, keywords in SECTOR_KEYWORDS.items():
46 |             matches = [kw for kw in keywords if kw in text_lower]
47 |             if matches:
48 |                 impact_score = min(len(matches) * 25, 100)
49 |                 affected.append({
50 |                     'sector': sector,
51 |                     'impact_score': impact_score,
52 |                     'keywords_matched': matches,
53 |                     'tickers': SECTOR_TICKERS.get(sector, []),
54 |                 })
55 |         return sorted(affected, key=lambda x: x['impact_score'], reverse=True)
56 |
57 |     def _scrape_rss(self, url: str, source_name: str) -> List[Dict]:
58 |         articles = []
59 |         try:
60 |             resp = requests.get(url, headers=self.headers, timeout=10)
61 |             soup = BeautifulSoup(resp.content, 'xml')
62 |             items = soup.find_all('item')[0:10]
63 |             for item in items:

```

```

64 |         title = item.find('title')
65 |         desc = item.find('description')
66 |         link = item.find('link')
67 |         pub_date = item.find('pubDate')
68 |         title_text = title.get_text(strip=True) if title else ''
69 |         desc_text = desc.get_text(strip=True) if desc else ''
70 |         combined = f"{title_text} {desc_text}"
71 |         sectors = self._detect_sectors(combined)
72 |         if title_text:
73 |             articles.append({
74 |                 'title': title_text,
75 |                 'description': desc_text[:300],
76 |                 'source': source_name,
77 |                 'url': link.get_text(strip=True) if link else '',
78 |                 'published': pub_date.get_text(strip=True) if pub_date else '',
79 |                 'affected_sectors': sectors,
80 |                 'is_high_impact': len(sectors) >= 2 or any(s['impact_score'] >= 50 for s in sectors),
81 |             })
82 |     except Exception as e:
83 |         logger.error(f"Error scraping {source_name} RSS: {str(e)}")
84 |     return articles
85 |
86 |     async def scrape_world_events(self) -> Dict:
87 |         rss_sources = [
88 |             ('https://feeds.reuters.com/reuters/worldNews', 'Reuters World'),
89 |             ('https://feeds.reuters.com/reuters/businessNews', 'Reuters Business'),
90 |             ('https://feeds.bbc.co.uk/news/world/rss.xml', 'BBC World'),
91 |             ('https://rss.nytimes.com/services/xml/rss/nyt/World.xml', 'NY Times World'),
92 |             ('https://feeds.a.pnnews.com/apnews/topnews', 'AP News'),
93 |             ('https://www.cnn.com/id/100727362/device/rss/rss.html', 'CNN World'),
94 |         ]
95 |
96 |         all_events = []
97 |         for url, name in rss_sources:
98 |             events = self._scrape_rss(url, name)
99 |             all_events.extend(events)
100 |
101 |         # Deduplicate by title similarity
102 |         seen_titles = set()
103 |         unique_events = []
104 |         for event in all_events:
105 |             title_key = event['title'][:50].lower()
106 |             if title_key not in seen_titles:
107 |                 seen_titles.add(title_key)
108 |                 unique_events.append(event)
109 |
110 |         # Separate high-impact events
111 |         high_impact = [e for e in unique_events if e.get('is_high_impact')]
112 |         all_affected_sectors = {}
113 |         for event in unique_events:
114 |             for sector in event.get('affected_sectors', []):
115 |                 name = sector['sector']
116 |                 if name not in all_affected_sectors:
117 |                     all_affected_sectors[name] = {
118 |                         'sector': name,
119 |                         'total_mentions': 0,
120 |                         'avg_impact': 0,
121 |                         'tickers': sector['tickers'],
122 |                         'events': [],
123 |                     }
124 |                     all_affected_sectors[name]['total_mentions'] += 1
125 |                     all_affected_sectors[name]['events'].append(event['title'][:80])
126 |                     all_affected_sectors[name]['avg_impact'] = max(
127 |                         all_affected_sectors[name]['avg_impact'], sector['impact_score']
128 |                     )
129 |
130 |         sector_summary = sorted(all_affected_sectors.values(), key=lambda x: x['avg_impact'], reverse=True)

```

```
131 |  
132 |     return {  
133 |         'total_events': len(unique_events),  
134 |         'high_impact_count': len(high_impact),  
135 |         'high_impact_events': high_impact[:5],  
136 |         'all_events': unique_events[:20],  
137 |         'affected_sectors': sector_summary[:8],  
138 |         'timestamp': datetime.now(timezone.utc).isoformat(),  
139 |     }  
140 |
```

8. Frontend - Core

FILE: frontend/src/App.js

128 lines | 6780 characters

```
1 | import React, { useState, useEffect } from 'react';
2 | import './App.css';
3 | import { AuthProvider, useAuth } from './contexts/AuthContext';
4 | import Navbar from './components/Navbar';
5 | import StockTicker from './components/StockTicker';
6 | import CryptoTicker from './components/CryptoTicker';
7 | import Watchlist from './components/Watchlist';
8 | import AlertsPanel from './components/AlertsPanel';
9 | import MarketPrediction from './components/MarketPrediction';
10 | import OptionsRadar from './components/OptionsRadar';
11 | import OptionsFlowScreener from './components/OptionsFlowScreener';
12 | import AdditionalSections from './components/AdditionalSections';
13 | import DarkPoolData from './components/DarkPoolData';
14 | import CryptoSection from './components/CryptoSection';
15 | import TradeGPTChat from './components/TradeGPTChat';
16 | import PaymentStatus from './components/PaymentStatus';
17 | import CompanyResearch from './components/CompanyResearch';
18 | import MacroDashboard from './components/MacroDashboard';
19 | import MobileBottomNav from './components/MobileBottomNav';
20 | import AIHypothesis from './components/AIHypothesis';
21 | import PromoBanner from './components/PromoBanner';
22 | import AuthModal from './components/AuthModal';
23 | import SubscriptionPricing from './components/SubscriptionPricing';
24 | import AdminPanel from './components/AdminPanel';
25 | import UserWorkspace from './components/UserWorkspace';
26 | import PortfolioAnalyzer from './components/PortfolioAnalyzer';
27 | import MarketSignals from './components/MarketSignals';
28 | import ReferralLeaderboard from './components/ReferralLeaderboard';
29 | import TradingJournal from './components/TradingJournal';
30 |
31 | // Register service worker
32 | if ('serviceWorker' in navigator) {
33 |   window.addEventListener('load', () => {
34 |     navigator.serviceWorker.register('/service-worker.js').catch(() => {});
35 |   });
36 | }
37 |
38 | function AppContent() {
39 |   const [paymentInfo, setPaymentInfo] = useState(null);
40 |   const [showAuth, setShowAuth] = useState(false);
41 |   const [authTab, setAuthTab] = useState('login');
42 |   const [showSubscription, setShowSubscription] = useState(false);
43 |   const [showAdmin, setShowAdmin] = useState(false);
44 |   const [showWorkspace, setShowWorkspace] = useState(false);
45 |   const [showPortfolio, setShowPortfolio] = useState(false);
46 |   const [showSignals, setShowSignals] = useState(false);
47 |   const [showJournal, setShowJournal] = useState(false);
48 |   const { user } = useAuth();
49 |
50 |   useEffect(() => {
51 |     const params = new URLSearchParams(window.location.search);
52 |     const paymentStatus = params.get('payment_status');
53 |     const sessionId = params.get('session_id');
54 |     if (paymentStatus && sessionId) {
55 |       setPaymentInfo({ status: paymentStatus, sessionId });
56 |       window.history.replaceState({}, '', window.location.pathname);
57 |     } else if (paymentStatus === 'cancelled') {
58 |       setPaymentInfo({ status: 'cancelled', sessionId: null });
```

```
59 |     window.history.replaceState({}, '', window.location.pathname);
60 |   }
61 | }, []);
62 |
63 | const openChat = () => window.dispatchEvent(new CustomEvent('risedualai-open-chat'));
64 | const openLogin = () => { setAuthTab('login'); setShowAuth(true); };
65 | const openRegister = () => { setAuthTab('register'); setShowAuth(true); };
66 |
67 | return (
68 |   <div className="min-h-screen bg-[#0F172A] pb-16 lg:pb-0">
69 |     <PromoBanner onSubscribe={() => setShowSubscription(true)} />
70 |     <div id="stock-ticker"><StockTicker /></div>
71 |     <Navbar onLogin={openLogin} onRegister={openRegister} onSubscribe={() => setShowSubscription(true)} onOpenAdmin={() => setSh ...
72 |     <CryptoTicker />
73 |     <AlertsPanel onSubscribe={() => setShowSubscription(true)} />
74 |
75 |     <main className="max-w-[1600px] mx-auto px-3 sm:px-6 py-4 sm:py-8">
76 |       <div className="mb-6 sm:mb-8"><Watchlist onSubscribe={() => setShowSubscription(true)} /></div>
77 |
78 |       {/* Referral Leaderboard */}
79 |       <div className="mb-6 sm:mb-8"><ReferralLeaderboard /></div>
80 |
81 |       {/* AI Investment Hypothesis (Paywalled) */}
82 |       <div id="ai-hypothesis" className="mb-6 sm:mb-8">
83 |         <AIHypothesis onSubscribe={() => setShowSubscription(true)} onLogin={openLogin} />
84 |       </div>
85 |
86 |       <div id="market-prediction" className="mb-6 sm:mb-8"><MarketPrediction /></div>
87 |       <div id="company-research" className="mb-6 sm:mb-8"><CompanyResearch /></div>
88 |       <div id="macro-dashboard" className="mb-6 sm:mb-8"><MacroDashboard onSubscribe={() => setShowSubscription(true)} /></div>
89 |       <div id="options-radar"><OptionsRadar /></div>
90 |       <div id="options-flow"><OptionsFlowScreener /></div>
91 |       <AdditionalSections />
92 |       <div id="dark-pool"><DarkPoolData onSubscribe={() => setShowSubscription(true)} /></div>
93 |       <div id="crypto"><CryptoSection /></div>
94 |     </main>
95 |
96 |     <TradeGPTChat onSubscribe={() => setShowSubscription(true)} />
97 |     <MobileBottomNav onOpenChat={openChat} />
98 |
99 |     {paymentInfo && <PaymentStatus sessionId={paymentInfo.sessionId} initialStatus={paymentInfo.status} onClose={() => setPaymen ...
100 |     {showAuth && <AuthModal onClose={() => setShowAuth(false)} initialTab={authTab} />}
101 |     {showSubscription && <SubscriptionPricing onClose={() => setShowSubscription(false)} />}
102 |     {showAdmin && user?.role === 'owner' && <AdminPanel onClose={() => setShowAdmin(false)} />}
103 |     {showWorkspace && user && <UserWorkspace onClose={() => setShowWorkspace(false)} onSubscribe={() => { setShowWorkspace(false) ...
104 |     {showPortfolio && user && <PortfolioAnalyzer onClose={() => setShowPortfolio(false)} onSubscribe={() => { setShowPortfolio(f ...
105 |     {showSignals && user && <MarketSignals onClose={() => setShowSignals(false)} onSubscribe={() => { setShowSignals(false); set ...
106 |     {showJournal && user && <TradingJournal onClose={() => setShowJournal(false)} onSubscribe={() => { setShowJournal(false); se ...
107 |
108 |     <div className="text-center py-6 sm:py-8 text-gray-500 text-xs sm:text-sm">
109 |       <div className="flex items-center justify-center gap-2 mb-1">
110 |         
111 |         <p>RISEDUAL AI - Advanced AI-Powered Trading Platform</p>
112 |       </div>
113 |       <p>Last updated on {new Date().toLocaleString('en-US', { weekday: 'long', month: 'short', day: 'numeric', year: 'numeric', ...
114 |     </div>
115 |   </div>
116 | );
117 | }
118 |
119 | function App() {
120 |   return (
121 |     <AuthProvider>
122 |       <AppContent />
123 |     </AuthProvider>
124 |   );
125 | }
```

```
126 |  
127 | export default App;  
128 |
```

FILE: frontend/src/App.css*30 lines | 464 characters*

```
1 | .App {  
2 |   min-height: 100vh;  
3 | }  
4 |  
5 | /* Scrollbar Styles */  
6 | ::-webkit-scrollbar {  
7 |   width: 8px;  
8 |   height: 8px;  
9 | }  
10 |  
11 | ::-webkit-scrollbar-track {  
12 |   background: #1E293B;  
13 | }  
14 |  
15 | ::-webkit-scrollbar-thumb {  
16 |   background: #475569;  
17 |   border-radius: 4px;  
18 | }  
19 |  
20 | ::-webkit-scrollbar-thumb:hover {  
21 |   background: #64748B;  
22 | }  
23 |  
24 | /* Smooth transitions */  
25 | * {  
26 |   transition-property: color, background-color, border-color;  
27 |   transition-duration: 0.15s;  
28 |   transition-timing-function: ease-in-out;  
29 | }  
30 |
```

FILE: frontend/src/index.js*12 lines | 255 characters*

```
1 | import React from "react";  
2 | import ReactDOM from "react-dom/client";  
3 | import "@index.css";  
4 | import App from "@App";  
5 |  
6 | const root = ReactDOM.createRoot(document.getElementById("root"));  
7 | root.render(  
8 |   <React.StrictMode>  
9 |     <App />  
10 |   </React.StrictMode>,  
11 | );  
12 |
```

FILE: frontend/src/index.css*183 lines | 4705 characters*

```
1 | @tailwind base;  
2 | @tailwind components;  
3 | @tailwind utilities;  
4 |  
5 | html,  
6 | body,
```

```
7 | #root {
8 |   margin: 0;
9 |   padding: 0;
10 |   background-color: #0F172A !important;
11 |   min-height: 100vh;
12 |   -webkit-overflow-scrolling: touch;
13 | }
14 |
15 | body {
16 |   margin: 0;
17 |   font-family: 'IBM Plex Sans', -apple-system, BlinkMacSystemFont, "Segoe UI", sans-serif;
18 |   -webkit-font-smoothing: antialiased;
19 |   -moz-osx-font-smoothing: grayscale;
20 |   background-color: #0F172A !important;
21 | }
22 |
23 | h1, h2, h3, h4, h5, h6 {
24 |   font-family: 'Manrope', -apple-system, BlinkMacSystemFont, sans-serif;
25 | }
26 |
27 | code, .font-mono {
28 |   font-family: 'JetBrains Mono', source-code-pro, Menlo, Monaco, Consolas, monospace;
29 | }
30 |
31 | /* Safe area padding for notched devices */
32 | .safe-area-bottom {
33 |   padding-bottom: env(safe-area-inset-bottom, 0);
34 | }
35 |
36 | /* Custom scrollbar for all platforms */
37 | .custom-scrollbar::-webkit-scrollbar {
38 |   width: 4px;
39 |   height: 4px;
40 | }
41 | .custom-scrollbar::-webkit-scrollbar-track {
42 |   background: transparent;
43 | }
44 | .custom-scrollbar::-webkit-scrollbar-thumb {
45 |   background: rgba(100, 116, 139, 0.3);
46 |   border-radius: 4px;
47 | }
48 |
49 | /* Responsive table wrapper */
50 | .table-responsive {
51 |   overflow-x: auto;
52 |   -webkit-overflow-scrolling: touch;
53 | }
54 |
55 | /* Hide desktop chat FAB on mobile (bottom nav handles it) */
56 | @media (max-width: 767px) {
57 |   /* Ensure modals are above bottom nav */
58 |   [role="dialog"] {
59 |     z-index: 60 !important;
60 |   }
61 |   /* Prevent horizontal overflow */
62 |   body, #root {
63 |     overflow-x: hidden;
64 |   }
65 | }
66 |
67 | /* Touch-friendly tap targets */
68 | @media (pointer: coarse) {
69 |   button, a, [role="button"] {
70 |     min-height: 44px;
71 |     min-width: 44px;
72 |   }
73 |   input, select, textarea {
```

```
74 |         font-size: 16px !important; /* Prevent iOS zoom */
75 |     }
76 | }
77 |
78 | @layer base {
79 |     :root {
80 |         --background: 222 47% 11%;
81 |         --foreground: 210 40% 98%;
82 |         --card: 217 33% 17%;
83 |         --card-foreground: 210 40% 98%;
84 |         --popover: 217 33% 17%;
85 |         --popover-foreground: 210 40% 98%;
86 |         --primary: 221 100% 50%;
87 |         --primary-foreground: 0 0% 100%;
88 |         --secondary: 217 19% 27%;
89 |         --secondary-foreground: 210 40% 98%;
90 |         --muted: 217 19% 27%;
91 |         --muted-foreground: 215 16% 62%;
92 |         --accent: 217 19% 27%;
93 |         --accent-foreground: 210 40% 98%;
94 |         --destructive: 0 84% 60%;
95 |         --destructive-foreground: 0 0% 100%;
96 |         --border: 217 19% 27%;
97 |         --input: 217 19% 27%;
98 |         --ring: 221 100% 50%;
99 |         --chart-1: 221 100% 50%;
100 |         --chart-2: 142 72% 29%;
101 |         --chart-3: 197 37% 24%;
102 |         --chart-4: 43 74% 66%;
103 |         --chart-5: 27 87% 67%;
104 |         --radius: 0.75rem;
105 |     }
106 |     .dark {
107 |         --background: 222 47% 11%;
108 |         --foreground: 210 40% 98%;
109 |         --card: 217 33% 17%;
110 |         --card-foreground: 210 40% 98%;
111 |         --popover: 217 33% 17%;
112 |         --popover-foreground: 210 40% 98%;
113 |         --primary: 221 100% 50%;
114 |         --primary-foreground: 0 0% 100%;
115 |         --secondary: 217 19% 27%;
116 |         --secondary-foreground: 210 40% 98%;
117 |         --muted: 217 19% 27%;
118 |         --muted-foreground: 215 16% 62%;
119 |         --accent: 217 19% 27%;
120 |         --accent-foreground: 210 40% 98%;
121 |         --destructive: 0 63% 31%;
122 |         --destructive-foreground: 0 0% 100%;
123 |         --border: 217 19% 27%;
124 |         --input: 217 19% 27%;
125 |         --ring: 221 100% 50%;
126 |         --chart-1: 221 100% 50%;
127 |         --chart-2: 142 72% 29%;
128 |         --chart-3: 30 80% 55%;
129 |         --chart-4: 280 65% 60%;
130 |         --chart-5: 340 75% 55%;
131 |     }
132 | }
133 |
134 | @layer base {
135 |     * {
136 |         @apply border-border;
137 |     }
138 |     body {
139 |         @apply bg-background text-foreground;
140 |     }
```



```
141 | }
142 |
143 | @layer base {
144 |   [data-debug-wrapper="true"] {
145 |     display: contents !important;
146 |   }
147 |
148 |   [data-debug-wrapper="true"] > * {
149 |     margin-left: inherit;
150 |     margin-right: inherit;
151 |     margin-top: inherit;
152 |     margin-bottom: inherit;
153 |     padding-left: inherit;
154 |     padding-right: inherit;
155 |     padding-top: inherit;
156 |     padding-bottom: inherit;
157 |     column-gap: inherit;
158 |     row-gap: inherit;
159 |     gap: inherit;
160 |     border-left-width: inherit;
161 |     border-right-width: inherit;
162 |     border-top-width: inherit;
163 |     border-bottom-width: inherit;
164 |     border-left-style: inherit;
165 |     border-right-style: inherit;
166 |     border-top-style: inherit;
167 |     border-bottom-style: inherit;
168 |     border-left-color: inherit;
169 |     border-right-color: inherit;
170 |     border-top-color: inherit;
171 |     border-bottom-color: inherit;
172 |   }
173 | }
174 |
175 |
176 | @keyframes shimmer {
177 |   0% { transform: translateX(-100%); }
178 |   100% { transform: translateX(200%); }
179 | }
180 | .animate-shimmer {
181 |   animation: shimmer 3s ease-in-out infinite;
182 | }
183 |
```

FILE: frontend/src/mockData.js

75 lines | 5057 characters

```
1 | // Mock data for TradealgoGPT clone - will be replaced with real API data
2 |
3 | export const stockTickerData = [
4 |   { symbol: 'SPY', price: 572.38, change: -8.93, changePercent: -1.31 },
5 |   { symbol: 'VOO', price: 618.43, change: -8.38, changePercent: -1.34 },
6 |   { symbol: 'QQQ', price: 599.75, change: -9.16, changePercent: -1.50 },
7 |   { symbol: 'IVV', price: 675.40, change: -9.06, changePercent: -1.32 },
8 |   { symbol: 'VTI', price: 331.41, change: -4.59, changePercent: -1.37 },
9 |   { symbol: 'VUG', price: 458.08, change: -6.23, changePercent: -1.34 },
10 |  { symbol: 'VEA', price: 65.28, change: -0.51, changePercent: -0.78 },
11 | ];
12 |
13 | export const optionsRadarData = {
14 |   mostActivelyTraded: [
15 |     { contract: 'CORZ', price: '$16 Put', returns: '167%', volOI: 0.14, power: 91, ivRank: 30, aiScore: 33 },
16 |     { contract: 'BBAI', price: '$4 Call', returns: '164%', volOI: 1.09, power: 47, ivRank: 25, aiScore: 36 },
17 |     { contract: 'BX', price: '$115 Put', returns: '136%', volOI: 0.22, power: 66, ivRank: 48, aiScore: 27 },
18 |     { contract: 'PLTR', price: '$150 Call', returns: '74%', volOI: 0.77, power: 83, ivRank: 21, aiScore: 38 },
19 |   ],
```

```
20 | volatilityOpportunities: [
21 |   { contract: 'T', price: '$29 Call', ivRank: 27, returns: '59%', power: 98, volOI: 2.61, aiScore: 52 },
22 |   { contract: 'QQQ', price: '$634 Put', ivRank: 50, returns: '24%', power: 85, volOI: 1.95, aiScore: 37 },
23 |   { contract: 'IEP', price: '$75 Call', ivRank: 29, returns: '20%', power: 97, volOI: 0.05, aiScore: 37 },
24 |   { contract: 'VZ', price: '$50 Call', ivRank: 46, returns: '18%', power: 95, volOI: 0.06, aiScore: 50 },
25 | ]
26 | };
27 |
28 | export const optionsFlowData = {
29 |   mostActivelyTraded: [
30 |     { contract: 'MRVL', price: '$95 Call', returns: '309%', volOI: 8.37, power: 60, ivRank: 69, aiScore: 34 },
31 |     { contract: 'AMZN', price: '$2175 Put', returns: '282%', volOI: 0.94, power: 90, ivRank: 28, aiScore: 40 },
32 |     { contract: 'META', price: '$655 Put', returns: '257%', volOI: 0.46, power: 58, ivRank: 21, aiScore: 40 },
33 |     { contract: 'PLTR', price: '$170 Call', returns: '217%', volOI: 3.11, power: 67, ivRank: 21, aiScore: 38 },
34 |     { contract: 'NVDA', price: '$175 Put', returns: '189%', volOI: 7.62, power: 80, ivRank: 38, aiScore: 35 },
35 |   ],
36 |   dteEdge: [
37 |     { contract: 'USO', price: '$100 Call', power: 89, returns: '1134%', volOI: 0.83, aiScore: 47, ivRank: 50 },
38 |     { contract: 'MRVL', price: '$85 Call', power: 61, returns: '384%', volOI: 1.01, aiScore: 34, ivRank: 69 },
39 |     { contract: 'LITE', price: '$6175 Put', power: 59, returns: '341%', volOI: 2.61, aiScore: 47, ivRank: 84 },
40 |     { contract: 'PLTR', price: '$155 Call', power: 57, returns: '310%', volOI: 5.36, aiScore: 38, ivRank: 21 },
41 |     { contract: 'NVDA', price: '$180 Put', power: 73, returns: '247%', volOI: 7.44, aiScore: 35, ivRank: 38 },
42 |   ],
43 |   volatilityLow: [
44 |     { contract: 'BAC', price: '$48 Put', ivRank: 21, returns: '151%', power: 93, volOI: 0.04, aiScore: 37 },
45 |     { contract: 'DGX', price: '$200 Put', ivRank: 41, returns: '71%', power: 100, volOI: 0.25, aiScore: 51 },
46 |     { contract: 'RTX', price: '$220 Call', ivRank: 63, returns: '64%', power: 98, volOI: 0.35, aiScore: 66 },
47 |   ],
48 |   volatilityHigh: [
49 |     { contract: 'SWCE', price: '$20 Call', ivRank: 21, returns: '544%', power: 98, volOI: 2.60, aiScore: 39 },
50 |     { contract: 'MU', price: '$290 Put', ivRank: 97, returns: '82%', power: 100, volOI: 98.08, aiScore: 46 },
51 |     { contract: 'APLD', price: '$22 Put', ivRank: 45, returns: '93%', power: 94, volOI: 0.35, aiScore: 39 },
52 |   ]
53 | };
54 |
55 | export const momentumData = [
56 |   { contract: 'MRVL', price: '$95 Call', returns: '300%', power: 66, volOI: 1.03, ivRank: 69, aiScore: 34 },
57 |   { contract: 'CF', price: '$125 Call', returns: '146%', power: 73, volOI: 32.73, ivRank: 86, aiScore: 60 },
58 |   { contract: 'PLTR', price: '$170 Call', returns: '85%', power: 84, volOI: 3.45, ivRank: 21, aiScore: 38 },
59 |   { contract: 'AMPX', price: '$20 Call', returns: '85%', power: 70, volOI: 2.36, ivRank: 35, aiScore: 66 },
60 |   { contract: 'ASPI', price: '$55 Call', returns: '61%', power: 85, volOI: 51.66, ivRank: 39, aiScore: 34 },
61 | ];
62 |
63 | export const fastMoverCallsData = [
64 |   { contract: 'BA', price: '$240 Call', returns: '364%', volOI: 5.64, ivRank: 25, power: 24, aiScore: 39 },
65 |   { contract: 'BLK', price: '$960 Put', returns: '296%', volOI: 9.35, ivRank: 50, power: 76, aiScore: 36 },
66 |   { contract: 'SNDK', price: '$400 Put', returns: '280%', volOI: 2.27, ivRank: 62, power: 52, aiScore: 48 },
67 |   { contract: 'MRVL', price: '$80 Call', returns: '266%', volOI: 0.31, ivRank: 69, power: 77, aiScore: 34 },
68 |   { contract: 'MU', price: '$280 Put', returns: '223%', volOI: 1.64, ivRank: 97, power: 86, aiScore: 46 },
69 | ];
70 |
71 | export const unusualVolumeData = [
72 |   { contract: 'GLW', price: '$118 Put', volOI: 39.62, returns: '142%', power: 94, sentiment: 'Bearish', aiScore: 58 },
73 |   { contract: 'META', price: '$6.475 Put', volOI: 9.54, returns: '99%', power: 54, sentiment: 'Bearish', aiScore: 40 },
74 |   { contract: 'SNDK', price: '$5.375 Put', volOI: 14.64, returns: '78%', power: 98, sentiment: 'Bearish', aiScore: 48 },
75 | ];
```

FILE: frontend/src/services/api.js

146 lines | 3631 characters

```
1 | import axios from 'axios';
2 |
3 | const BACKEND_URL = process.env.REACT_APP_BACKEND_URL;
4 | const API = `${BACKEND_URL}/api`;
5 |
6 | // Stock Market APIs
```

```
7 | export const getTickerData = async () => {
8 |   try {
9 |     const response = await axios.get(`${API}/stocks/ticker`);
10 |     return response.data;
11 |   } catch (error) {
12 |     console.error('Error fetching ticker data:', error);
13 |     throw error;
14 |   }
15 | };
16 |
17 | export const getStockQuote = async (symbol) => {
18 |   try {
19 |     const response = await axios.get(`${API}/stocks/quote/${symbol}`);
20 |     return response.data;
21 |   } catch (error) {
22 |     console.error('Error fetching quote for ${symbol}:', error);
23 |     throw error;
24 |   }
25 | };
26 |
27 | // Options APIs
28 | export const getOptionsRadar = async () => {
29 |   try {
30 |     const response = await axios.get(`${API}/options/radar`);
31 |     return response.data;
32 |   } catch (error) {
33 |     console.error('Error fetching options radar:', error);
34 |     throw error;
35 |   }
36 | };
37 |
38 | export const getOptionsFlow = async () => {
39 |   try {
40 |     const response = await axios.get(`${API}/options/flow`);
41 |     return response.data;
42 |   } catch (error) {
43 |     console.error('Error fetching options flow:', error);
44 |     throw error;
45 |   }
46 | };
47 |
48 | export const getMomentumData = async () => {
49 |   try {
50 |     const response = await axios.get(`${API}/options/momentum`);
51 |     return response.data;
52 |   } catch (error) {
53 |     console.error('Error fetching momentum data:', error);
54 |     throw error;
55 |   }
56 | };
57 |
58 | export const getFastMovers = async () => {
59 |   try {
60 |     const response = await axios.get(`${API}/options/fast-movers`);
61 |     return response.data;
62 |   } catch (error) {
63 |     console.error('Error fetching fast movers:', error);
64 |     throw error;
65 |   }
66 | };
67 |
68 | export const getUnusualVolume = async () => {
69 |   try {
70 |     const response = await axios.get(`${API}/options/unusual-volume`);
71 |     return response.data;
72 |   } catch (error) {
73 |     console.error('Error fetching unusual volume:', error);
```

```
74 |     throw error;
75 |   }
76 | };
77 |
78 | // Crypto APIs
79 | export const getCryptoPrices = async () => {
80 |   try {
81 |     const response = await axios.get(`${API}/crypto/prices`);
82 |     return response.data;
83 |   } catch (error) {
84 |     console.error('Error fetching crypto prices:', error);
85 |     throw error;
86 |   }
87 | };
88 |
89 | export const getCryptoBySymbol = async (symbol) => {
90 |   try {
91 |     const response = await axios.get(`${API}/crypto/${symbol}`);
92 |     return response.data;
93 |   } catch (error) {
94 |     console.error('Error fetching crypto ${symbol}:', error);
95 |     throw error;
96 |   }
97 | };
98 |
99 | // Dark Pool APIs
100 | export const getDarkPoolData = async () => {
101 |   try {
102 |     const response = await axios.get(`${API}/dark-pool`);
103 |     return response.data;
104 |   } catch (error) {
105 |     console.error('Error fetching dark pool data:', error);
106 |     throw error;
107 |   }
108 | };
109 |
110 | // AI Chat APIs
111 | export const sendChatMessage = async (message, sessionId, imageBase64 = null) => {
112 |   try {
113 |     const payload = { message, sessionId };
114 |     if (imageBase64) {
115 |       payload.image_base64 = imageBase64;
116 |     }
117 |     const response = await axios.post(`${API}/chat`, payload);
118 |     return response.data;
119 |   } catch (error) {
120 |     console.error('Error sending chat message:', error);
121 |     throw error;
122 |   }
123 | };
124 |
125 | export const getChatHistory = async (sessionId) => {
126 |   try {
127 |     const response = await axios.get(`${API}/chat/history/${sessionId}`);
128 |     return response.data;
129 |   } catch (error) {
130 |     console.error('Error fetching chat history:', error);
131 |     throw error;
132 |   }
133 | };
134 |
135 | // Company Research API
136 | export const researchCompany = async (symbol) => {
137 |   try {
138 |     const response = await axios.get(`${API}/research/${symbol}`);
139 |     return response.data;
140 |   } catch (error) {
```

```
141 |     console.error('Error researching company:', error);
142 |     throw error;
143 |   }
144 | };
145 |
146 |
```

FILE: frontend/src/contexts/AuthContext.jsx*175 lines | 6146 characters*

```
1 | import React, { createContext, useContext, useState, useEffect, useCallback } from 'react';
2 |
3 | const API = `${process.env.REACT_APP_BACKEND_URL}/api`;
4 |
5 | const AuthContext = createContext(null);
6 |
7 | export const useAuth = () => useContext(AuthContext);
8 |
9 | // Auth tokens stored in localStorage by architectural requirement:
10 | // The Kubernetes ingress enforces wildcard CORS (*), which blocks credentials:include.
11 | // Bearer token auth via localStorage is the only viable approach in this environment.
12 | // In a production deployment with a custom domain, migrate to httpOnly cookies.
13 | export const authFetch = async (url, options = {}, retries = 3) => {
14 |   const token = localStorage.getItem('access_token');
15 |   const headers = { ...options.headers };
16 |   if (token) headers['Authorization'] = `Bearer ${token}`;
17 |   if (options.body && !headers['Content-Type']) headers['Content-Type'] = 'application/json';
18 |
19 |   for (let i = 0; i <= retries; i++) {
20 |     const controller = new AbortController();
21 |     const timeoutId = setTimeout(() => controller.abort(), 30000);
22 |     try {
23 |       const res = await fetch(url, { ...options, headers, signal: controller.signal });
24 |       clearTimeout(timeoutId);
25 |       if (res.status === 502 && i < retries) {
26 |         await new Promise(r => setTimeout(r, 1500 * (i + 1)));
27 |         continue;
28 |       }
29 |       return res;
30 |     } catch (e) {
31 |       clearTimeout(timeoutId);
32 |       if (i === retries) throw e;
33 |       await new Promise(r => setTimeout(r, 1500 * (i + 1)));
34 |     }
35 |   }
36 | };
37 |
38 | export const AuthProvider = ({ children }) => {
39 |   const [user, setUser] = useState(null);
40 |   const [loading, setLoading] = useState(true);
41 |
42 |   // Intentionally empty deps: checkAuth runs once on mount. All referenced
43 |   // functions (authFetch, tryRefresh, clearTokens) are stable module/component-level refs.
44 |   // eslint-disable-next-line react-hooks/exhaustive-deps
45 |   const checkAuth = useCallback(async () => {
46 |     const token = localStorage.getItem('access_token');
47 |     if (!token) { setUser(false); setLoading(false); return; }
48 |     try {
49 |       const res = await authFetch(`${API}/auth/me`);
50 |       if (res.ok) {
51 |         setUser(await res.json());
52 |       } else {
53 |         // Try refresh
54 |         const refreshed = await tryRefresh();
55 |         if (!refreshed) { clearTokens(); setUser(false); }
56 |       }

```

```
57 |     } catch {
58 |       setUser(false);
59 |     } finally {
60 |       setLoading(false);
61 |     }
62 |   }, []);
63 |
64 |   useEffect(() => { checkAuth(); }, [checkAuth]);
65 |
66 |   const tryRefresh = async () => {
67 |     const refreshToken = localStorage.getItem('refresh_token');
68 |     if (!refreshToken) return false;
69 |     try {
70 |       const res = await fetch(`${API}/auth/refresh`, {
71 |         method: 'POST',
72 |         headers: { 'Content-Type': 'application/json' },
73 |         body: JSON.stringify({ refresh_token: refreshToken }),
74 |       });
75 |       if (res.ok) {
76 |         const data = await res.json();
77 |         localStorage.setItem('access_token', data.access_token);
78 |         const meRes = await authFetch(`${API}/auth/me`);
79 |         if (meRes.ok) { setUser(await meRes.json()); return true; }
80 |       }
81 |     } catch (e) {
82 |       console.error('Token refresh failed:', e);
83 |     }
84 |     return false;
85 |   };
86 |
87 |   const storeTokens = (data) => {
88 |     if (data.access_token) localStorage.setItem('access_token', data.access_token);
89 |     if (data.refresh_token) localStorage.setItem('refresh_token', data.refresh_token);
90 |   };
91 |
92 |   const clearTokens = () => {
93 |     localStorage.removeItem('access_token');
94 |     localStorage.removeItem('refresh_token');
95 |   };
96 |
97 |   const fetchWithRetry = async (url, opts, retries = 3) => {
98 |     for (let i = 0; i <= retries; i++) {
99 |       const controller = new AbortController();
100 |      const timeoutId = setTimeout(() => controller.abort(), 30000);
101 |      try {
102 |        const res = await fetch(url, { ...opts, signal: controller.signal });
103 |        clearTimeout(timeoutId);
104 |        if (res.status === 502 && i < retries) {
105 |          await new Promise(r => setTimeout(r, 1500 * (i + 1)));
106 |          continue;
107 |        }
108 |        return res;
109 |      } catch (e) {
110 |        clearTimeout(timeoutId);
111 |        if (i === retries) throw new Error('Network error. Please check your connection and try again.');
```

```
124 |     let detail;
125 |     try { detail = (await res.json()).detail; } catch { detail = `Server error (${res.status}). Please try again.`; }
126 |     throw new Error(formatDetail(detail));
127 |   }
128 |   const data = await res.json();
129 |   storeTokens(data);
130 |   setUser(data);
131 |   return data;
132 | };
133 |
134 | const register = async (email, password, name, refCode) => {
135 |   const body = { email, password, name };
136 |   if (refCode) body.ref_code = refCode;
137 |   const res = await fetchWithRetry(`${API}/auth/register`, {
138 |     method: 'POST',
139 |     headers: { 'Content-Type': 'application/json' },
140 |     body: JSON.stringify(body),
141 |   });
142 |   if (!res.ok) {
143 |     let detail;
144 |     try { detail = (await res.json()).detail; } catch { detail = `Server error (${res.status}). Please try again.`; }
145 |     throw new Error(formatDetail(detail));
146 |   }
147 |   const data = await res.json();
148 |   storeTokens(data);
149 |   setUser(data);
150 |   return data;
151 | };
152 |
153 | const logout = async () => {
154 |   await fetch(`${API}/auth/logout`, { method: 'POST' }).catch(() => {});
155 |   clearTokens();
156 |   setUser(false);
157 | };
158 |
159 | const isPro = user && (user.subscription_status === 'pro' || user.subscription_status === 'trial');
160 |
161 | return (
162 |   <AuthContext.Provider value={{ user, loading, login, register, logout, isPro, checkAuth, authFetch }}>
163 |     {children}
164 |   </AuthContext.Provider>
165 | );
166 | };
167 |
168 | function formatDetail(detail) {
169 |   if (detail == null) return 'Something went wrong. Please try again.';
170 |   if (typeof detail === 'string') return detail;
171 |   if (Array.isArray(detail)) return detail.map(e => e?.msg || JSON.stringify(e)).join(' ');
172 |   if (detail?.msg) return detail.msg;
173 |   return String(detail);
174 | }
175 |
```

FILE: frontend/src/hooks/use-toast.js

156 lines | 3085 characters

```
1 | "use client";
2 | // Inspired by react-hot-toast library
3 | import * as React from "react"
4 |
5 | const TOAST_LIMIT = 1
6 | const TOAST_REMOVE_DELAY = 1000000
7 |
8 | const actionTypes = {
9 |   ADD_TOAST: "ADD_TOAST",
10 |   UPDATE_TOAST: "UPDATE_TOAST",
```

```
11 |   DISMISS_TOAST: "DISMISS_TOAST",
12 |   REMOVE_TOAST: "REMOVE_TOAST"
13 | }
14 |
15 | let count = 0
16 |
17 | function genId() {
18 |   count = (count + 1) % Number.MAX_SAFE_INTEGER
19 |   return count.toString();
20 | }
21 |
22 | const toastTimeouts = new Map()
23 |
24 | const addToRemoveQueue = (toastId) => {
25 |   if (toastTimeouts.has(toastId)) {
26 |     return
27 |   }
28 |
29 |   const timeout = setTimeout(() => {
30 |     toastTimeouts.delete(toastId)
31 |     dispatch({
32 |       type: "REMOVE_TOAST",
33 |       toastId: toastId,
34 |     })
35 |   }, TOAST_REMOVE_DELAY)
36 |
37 |   toastTimeouts.set(toastId, timeout)
38 | }
39 |
40 | export const reducer = (state, action) => {
41 |   switch (action.type) {
42 |     case "ADD_TOAST":
43 |       return {
44 |         ...state,
45 |         toasts: [action.toast, ...state.toasts].slice(0, TOAST_LIMIT),
46 |       };
47 |
48 |     case "UPDATE_TOAST":
49 |       return {
50 |         ...state,
51 |         toasts: state.toasts.map((t) =>
52 |           t.id === action.toast.id ? { ...t, ...action.toast } : t),
53 |       };
54 |
55 |     case "DISMISS_TOAST": {
56 |       const { toastId } = action
57 |
58 |       // ! Side effects ! - This could be extracted into a dismissToast() action,
59 |       // but I'll keep it here for simplicity
60 |       if (toastId) {
61 |         addToRemoveQueue(toastId)
62 |       } else {
63 |         state.toasts.forEach((toast) => {
64 |           addToRemoveQueue(toast.id)
65 |         })
66 |       }
67 |
68 |       return {
69 |         ...state,
70 |         toasts: state.toasts.map((t) =>
71 |           t.id === toastId || toastId === undefined
72 |             ? {
73 |               ...t,
74 |               open: false,
75 |             }
76 |             : t),
77 |       };
```



```
78 |     }
79 |     case "REMOVE_TOAST":
80 |       if (action.toastId === undefined) {
81 |         return {
82 |           ...state,
83 |           toasts: [],
84 |         }
85 |       }
86 |       return {
87 |         ...state,
88 |         toasts: state.toasts.filter((t) => t.id !== action.toastId),
89 |       };
90 |     }
91 |   }
92 |
93 |   const listeners = []
94 |
95 |   let memoryState = { toasts: [] }
96 |
97 |   function dispatch(action) {
98 |     memoryState = reducer(memoryState, action)
99 |     listeners.forEach((listener) => {
100 |       listener(memoryState)
101 |     })
102 |   }
103 |
104 |   function toast({
105 |     ...props
106 |   }) {
107 |     const id = genId()
108 |
109 |     const update = (props) =>
110 |       dispatch({
111 |         type: "UPDATE_TOAST",
112 |         toast: { ...props, id },
113 |       })
114 |     const dismiss = () => dispatch({ type: "DISMISS_TOAST", toastId: id })
115 |
116 |     dispatch({
117 |       type: "ADD_TOAST",
118 |       toast: {
119 |         ...props,
120 |         id,
121 |         open: true,
122 |         onOpenChange: (open) => {
123 |           if (!open) dismiss()
124 |         },
125 |       },
126 |     })
127 |
128 |     return {
129 |       id: id,
130 |       dismiss,
131 |       update,
132 |     }
133 |   }
134 |
135 |   function useToast() {
136 |     const [state, setState] = React.useState(memoryState)
137 |
138 |     React.useEffect(() => {
139 |       listeners.push(setState)
140 |       return () => {
141 |         const index = listeners.indexOf(setState)
142 |         if (index > -1) {
143 |           listeners.splice(index, 1)
144 |         }
```

```
145 |     };
146 |   }, [state])
147 |
148 |   return {
149 |     ...state,
150 |     toast,
151 |     dismiss: (toastId) => dispatch({ type: "DISMISS_TOAST", toastId }),
152 |   };
153 | }
154 |
155 | export { useToast, toast }
156 |
```

FILE: frontend/src/hooks/usePushNotifications.js*94 lines | 2970 characters*

```
1 | import { useState, useEffect, useCallback } from 'react';
2 | import { authFetch } from '../contexts/AuthContext';
3 |
4 | const API = process.env.REACT_APP_BACKEND_URL;
5 | const VAPID_KEY = process.env.REACT_APP_VAPID_PUBLIC_KEY;
6 |
7 | function urlBase64ToUint8Array(base64String) {
8 |   const padding = '='.repeat((4 - base64String.length % 4) % 4);
9 |   const base64 = (base64String + padding).replace(/-/g, '+').replace(/_/g, '/');
10 |   const rawData = window.atob(base64);
11 |   const outputArray = new Uint8Array(rawData.length);
12 |   for (let i = 0; i < rawData.length; ++i) {
13 |     outputArray[i] = rawData.charCodeAtAt(i);
14 |   }
15 |   return outputArray;
16 | }
17 |
18 | export function usePushNotifications() {
19 |   const [permission, setPermission] = useState('default');
20 |   const [subscribed, setSubscribed] = useState(false);
21 |   const [loading, setLoading] = useState(true);
22 |   const [supported, setSupported] = useState(false);
23 |
24 |   useEffect(() => {
25 |     const isSupported = 'serviceWorker' in navigator && 'PushManager' in window && 'Notification' in window;
26 |     setSupported(isSupported);
27 |     if (isSupported) {
28 |       setPermission(Notification.permission);
29 |     }
30 |     setLoading(false);
31 |   }, []);
32 |
33 |   // Check backend subscription status
34 |   useEffect(() => {
35 |     const check = async () => {
36 |       try {
37 |         const res = await authFetch(`${API}/api/push/status`);
38 |         if (res.ok) {
39 |           const data = await res.json();
40 |           setSubscribed(data.subscribed);
41 |         }
42 |       } catch (e) { /* not logged in */ }
43 |     };
44 |     check();
45 |   }, []);
46 |
47 |   const subscribe = useCallback(async () => {
48 |     if (!supported) return false;
49 |     try {
50 |       const perm = await Notification.requestPermission();
```

```
51 |     setPermission(perm);
52 |     if (perm !== 'granted') return false;
53 |
54 |     const registration = await navigator.serviceWorker.ready;
55 |     const subscription = await registration.pushManager.subscribe({
56 |       userVisibleOnly: true,
57 |       applicationServerKey: urlBase64ToUint8Array(VAPID_KEY),
58 |     });
59 |
60 |     const res = await authFetch(`${API}/api/push/subscribe`, {
61 |       method: 'POST',
62 |       headers: { 'Content-Type': 'application/json' },
63 |       body: JSON.stringify({ subscription: subscription.toJSON() }),
64 |     });
65 |
66 |     if (res.ok) {
67 |       setSubscribed(true);
68 |       return true;
69 |     }
70 |     return false;
71 |   } catch (e) {
72 |     console.error('Push subscribe error:', e);
73 |     return false;
74 |   }
75 | }, [supported]);
76 |
77 | const unsubscribe = useCallback(async () => {
78 |   try {
79 |     const registration = await navigator.serviceWorker.ready;
80 |     const subscription = await registration.pushManager.getSubscription();
81 |     if (subscription) await subscription.unsubscribe();
82 |
83 |     await authFetch(`${API}/api/push/unsubscribe`, { method: 'POST' });
84 |     setSubscribed(false);
85 |     return true;
86 |   } catch (e) {
87 |     console.error('Push unsubscribe error:', e);
88 |     return false;
89 |   }
90 | }, []);
91 |
92 | return { permission, subscribed, loading, supported, subscribe, unsubscribe };
93 | }
94 |
```

FILE: frontend/src/lib/utils.js*7 lines | 137 characters*

```
1 | import { clsx } from "clsx";
2 | import { twMerge } from "tailwind-merge"
3 |
4 | export function cn(...inputs) {
5 |   return twMerge(clsx(inputs));
6 | }
7 |
```

Frontend - Components

FILE: frontend/src/components/AIHypothesis.jsx*322 lines | 15305 characters*

```
1 | import React, { useState } from 'react';
```

```
2 | import { Search, TrendingUp, TrendingDown, Minus, Lock, Shield, BarChart3, Globe, Landmark, Zap, Sparkles, Download } from 'lucide ...
3 | import { Card } from './ui/card';
4 | import { Badge } from './ui/badge';
5 | import { Button } from './ui/button';
6 | import { Input } from './ui/input';
7 | import { useAuth, authFetch } from '../contexts/AuthContext';
8 |
9 | const API = `${process.env.REACT_APP_BACKEND_URL}/api`;
10 |
11 | const AIHypothesis = ({ onSubscribe, onLogin }) => {
12 |   const { user, isPro } = useAuth();
13 |   const [symbol, setSymbol] = useState('');
14 |   const [hypothesis, setHypothesis] = useState(null);
15 |   const [loading, setLoading] = useState(false);
16 |   const [error, setError] = useState('');
17 |   const [exporting, setExporting] = useState(false);
18 |
19 |   const exportReport = () => {
20 |     if (!hypothesis || !hypothesis.is_pro) return;
21 |     setExporting(true);
22 |     try {
23 |       const lines = [
24 |         `RISEDUAL AI ? HYPOTHESIS REPORT`,
25 |         `????????????????????????????????????????`,
26 |         `Symbol: ${hypothesis.symbol}`,
27 |         `Generated: ${new Date().toLocaleString()}`,
28 |         ``,
29 |         `VERDICT: ${hypothesis.verdict}`,
30 |         `Confidence: ${hypothesis.confidence}%`,
31 |         ``,
32 |         `SUMMARY`,
33 |         `??????`,
34 |         hypothesis.summary || 'N/A',
35 |         ``,
36 |       ];
37 |       if (hypothesis.catalysts?.length) {
38 |         lines.push('CATALYSTS', '????????');
39 |         hypothesis.catalysts.forEach((c, i) => lines.push(`${i + 1}. ${c}`));
40 |         lines.push('');
41 |       }
42 |       if (hypothesis.risks?.length) {
43 |         lines.push('RISKS', '?????');
44 |         hypothesis.risks.forEach((r, i) => lines.push(`${i + 1}. ${r}`));
45 |         lines.push('');
46 |       }
47 |       lines.push('', '? RISEDUAL AI ? risedual.ai');
48 |       const blob = new Blob([lines.join('\n')], { type: 'text/plain' });
49 |       const url = URL.createObjectURL(blob);
50 |       const a = document.createElement('a');
51 |       a.href = url;
52 |       a.download = `RISEDUAL_AI_${hypothesis.symbol}_Report.txt`;
53 |       a.click();
54 |       URL.revokeObjectURL(url);
55 |     } finally {
56 |       setExporting(false);
57 |     }
58 |   };
59 |
60 |   const search = async (e) => {
61 |     e?.preventDefault();
62 |     if (!symbol.trim()) return;
63 |     setLoading(true);
64 |     setError('');
65 |     setHypothesis(null);
66 |     try {
67 |       const res = await authFetch(`${API}/hypothesis/${symbol.trim().toUpperCase()}`);
68 |       if (!res.ok) throw new Error('Failed to generate hypothesis');
```

```
69 |     const data = await res.json();
70 |     setHypothesis(data);
71 |     // Auto-save to history for Pro users
72 |     if (data.is_pro && data.verdict) {
73 |         authFetch(`${API}/workspace/history/save`, {
74 |             method: 'POST',
75 |             body: JSON.stringify({ symbol: data.symbol || symbol.trim().toUpperCase(), verdict: data.verdict, confidence: data.confidence ...
76 |         }).catch(() => {});
77 |     }
78 | } catch (err) {
79 |     setError(err.message);
80 | } finally {
81 |     setLoading(false);
82 | }
83 | };
84 |
85 | const VerdictIcon = ({ verdict }) => {
86 |     if (verdict === 'BUY') return <TrendingUp className="w-6 h-6 text-emerald-400" />;
87 |     if (verdict === 'SELL') return <TrendingDown className="w-6 h-6 text-red-400" />;
88 |     return <Minus className="w-6 h-6 text-amber-400" />;
89 | };
90 |
91 | const verdictColor = (v) => {
92 |     if (v === 'BUY') return 'text-emerald-400 bg-emerald-900/30 border-emerald-700/50';
93 |     if (v === 'SELL') return 'text-red-400 bg-red-900/30 border-red-700/50';
94 |     return 'text-amber-400 bg-amber-900/30 border-amber-700/50';
95 | };
96 |
97 | return (
98 |     <div className="space-y-6" data-testid="ai-hypothesis">
99 |         <div className="flex flex-col sm:flex-row items-start sm:items-center justify-between gap-3">
100 |             <div className="flex items-center gap-3">
101 |                 <div className="w-10 h-10 bg-gradient-to-br from-[#0052FF] to-cyan-500 rounded-xl flex items-center justify-center">
102 |                     <Sparkles className="w-6 h-6 text-white" />
103 |                 </div>
104 |                 <div>
105 |                     <h2 className="text-white text-xl sm:text-2xl font-bold" style={{fontFamily: 'Manrope, sans-serif'}}>AI Investment Hyp ...
106 |                     <p className="text-slate-400 text-xs sm:text-sm">Per-ticker analysis using all scraped macro data</p>
107 |                 </div>
108 |             </div>
109 |             <div>
110 |                 {isPro && <Badge className="bg-gradient-to-r from-[#0052FF] to-cyan-500 text-white border-0">PRO</Badge>}
111 |             </div>
112 |
113 |             <div>
114 |                 <div className="flex gap-3">
115 |                     <div className="relative flex-1">
116 |                         <Search className="absolute left-3 top-1/2 -translate-y-1/2 text-slate-400 w-4 h-4" />
117 |                         <input
118 |                             placeholder="Enter ticker (AAPL, BTC, TSLA...)"
119 |                             value={symbol}
120 |                             onChange={e => setSymbol(e.target.value.toUpperCase())}
121 |                             className="pl-10 bg-slate-800 border-slate-600 text-white rounded-xl"
122 |                             data-testid="hypothesis-search"
123 |                         />
124 |                     </div>
125 |                     <button type="submit" disabled={loading || !symbol.trim()} className="bg-[#0052FF] hover:bg-[#2563EB] text-white rounded-x ...
126 |                         {loading ? 'Analyzing...' : 'Analyze'}
127 |                     </button>
128 |                 </div>
129 |             </div>
130 |
131 |             <div>
132 |                 <div className="bg-slate-800/50 border-slate-700/40 rounded-xl p-8 text-center">
133 |                     <div className="animate-pulse space-y-3">
134 |                         <Sparkles className="w-8 h-8 text-[#0052FF] mx-auto animate-spin" />
135 |                         <p className="text-white font-medium">Generating AI Hypothesis for {symbol.toUpperCase()}...</p>
```

```
136 |         <p className="text-slate-400 text-sm">Analyzing news, world events, congressional trades, and market data</p>
137 |     </div>
138 | </Card>
139 | }}
140 |
141 | {error && <div className="bg-red-900/30 border border-red-800/50 text-red-400 text-sm p-3 rounded-lg">{error}</div>}
142 |
143 | {/* Locked State (Free User) */}
144 | {hypothesis && !hypothesis.is_pro && (
145 |     <Card className="relative bg-slate-800/50 border-slate-700/40 rounded-xl overflow-hidden" data-testid="hypothesis-locked">
146 |         {/* Teaser Stats */}
147 |         <div className="p-6 space-y-4">
148 |             <div className="flex items-center gap-2 text-white font-semibold text-lg">
149 |                 <Sparkles className="w-5 h-5 text-[#0052FF]" />
150 |                 AI Hypothesis Ready for {hypothesis.symbol}
151 |             </div>
152 |             <div className="grid grid-cols-3 gap-3">
153 |                 <div className="bg-slate-900/60 rounded-lg p-3 text-center">
154 |                     <BarChart3 className="w-4 h-4 text-blue-400 mx-auto mb-1" />
155 |                     <p className="text-white text-lg font-bold">{hypothesis.teaser.data_sources_count}</p>
156 |                     <p className="text-slate-500 text-[10px]">Data Points</p>
157 |                 </div>
158 |                 <div className="bg-slate-900/60 rounded-lg p-3 text-center">
159 |                     <Globe className="w-4 h-4 text-emerald-400 mx-auto mb-1" />
160 |                     <p className="text-white text-lg font-bold">{hypothesis.teaser.world_events_count}</p>
161 |                     <p className="text-slate-500 text-[10px]">World Events</p>
162 |                 </div>
163 |                 <div className="bg-slate-900/60 rounded-lg p-3 text-center">
164 |                     <Landmark className="w-4 h-4 text-violet-400 mx-auto mb-1" />
165 |                     <p className="text-white text-lg font-bold">{hypothesis.teaser.congressional_trades_count}</p>
166 |                     <p className="text-slate-500 text-[10px]">Congress Trades</p>
167 |                 </div>
168 |             </div>
169 |         </div>
170 |
171 |         {/* Blurred Preview */}
172 |         <div className="relative px-6 pb-6">
173 |             <div className="blur-md select-none pointer-events-none aria-hidden="true">
174 |                 <div className="space-y-3">
175 |                     <div className="flex items-center gap-3">
176 |                         <span className="text-3xl font-bold text-emerald-400">BUY</span>
177 |                         <span className="text-slate-400">|</span>
178 |                         <span className="text-white text-xl font-semibold">Confidence: 78%</span>
179 |                     </div>
180 |                     <p className="text-slate-300 text-sm">Based on analysis of 19 news articles, 30 world events, 12 congressional tra ...
181 |                     <div className="flex gap-2">
182 |                         <Badge className="bg-emerald-900/40 text-emerald-400">Price Target: $XXX</Badge>
183 |                         <Badge className="bg-blue-900/40 text-blue-400">Upside: XX%</Badge>
184 |                     </div>
185 |                 </div>
186 |             </div>
187 |
188 |         {/* Overlay */}
189 |         <div className="absolute inset-0 flex flex-col items-center justify-center bg-slate-900/50 backdrop-blur-sm rounded-b- ...
190 |             <Lock className="w-8 h-8 text-[#0052FF] mb-3" />
191 |             <p className="text-white font-semibold text-lg mb-1">Unlock Full AI Hypothesis</p>
192 |             <p className="text-slate-400 text-sm text-center max-w-xs mb-4">{hypothesis.teaser.summary}</p>
193 |             <div className="flex gap-3">
194 |                 {!user ? (
195 |                     <Button onClick={onLogin} className="bg-slate-700 hover:bg-slate-600 text-white rounded-xl" data-testid="hypothe ...
196 |                         Log In
197 |                     </Button>
198 |                 ) : null}
199 |                 <Button onClick={onSubscribe} className="bg-[#0052FF] hover:bg-[#2563EB] text-white rounded-xl" data-testid="hypot ...
200 |                     <Zap className="w-4 h-4 mr-2" /> Subscribe to Pro
201 |                 </Button>
202 |             </div>
```

```
203 |         </div>
204 |     </div>
205 | </Card>
206 | }}
207 |
208 | {/* Full Hypothesis (Pro User) */}
209 | {hypothesis && hypothesis.is_pro && (
210 |     <div className="space-y-5" data-testid="hypothesis-full">
211 |         {/* Verdict Card */}
212 |         <Card className={`border-2 rounded-xl p-6 ${verdictColor(hypothesis.verdict)}`}>
213 |             <div className="flex items-center justify-between">
214 |                 <div className="flex items-center gap-3">
215 |                     <VerdictIcon verdict={hypothesis.verdict} />
216 |                     <div>
217 |                         <div className="text-3xl font-black">{hypothesis.verdict}</div>
218 |                         <div className="text-sm opacity-70">{hypothesis.symbol}</div>
219 |                     </div>
220 |                 </div>
221 |                 <div className="text-right flex flex-col items-end gap-1">
222 |                     <div className="text-2xl font-bold">{hypothesis.confidence}%</div>
223 |                     <div className="text-sm opacity-70">Confidence</div>
224 |                     <Button size="sm" variant="outline" className="mt-1 border-white/20 text-white/80 hover:bg-white/10 rounded-lg tex ...
225 |                         <Download className="w-3 h-3 mr-1" /> Export
226 |                     </Button>
227 |                 </div>
228 |             </div>
229 |             {hypothesis.summary && (
230 |                 <p className="mt-4 text-sm opacity-90">{hypothesis.summary}</p>
231 |             )}
232 |         </Card>
233 |
234 |         {/* Price Targets */}
235 |         <div className="grid grid-cols-1 sm:grid-cols-2 gap-4">
236 |             <Card className="bg-slate-800/50 border-slate-700/40 rounded-xl p-5">
237 |                 <h3 className="text-slate-400 text-xs font-medium uppercase mb-2">Short-Term Target (1-2 weeks)</h3>
238 |                 <p className="text-white text-lg font-bold">{hypothesis.price_target_short || 'N/A'}</p>
239 |             </Card>
240 |             <Card className="bg-slate-800/50 border-slate-700/40 rounded-xl p-5">
241 |                 <h3 className="text-slate-400 text-xs font-medium uppercase mb-2">Medium-Term Target (1-3 months)</h3>
242 |                 <p className="text-white text-lg font-bold">{hypothesis.price_target_medium || 'N/A'}</p>
243 |             </Card>
244 |         </div>
245 |
246 |         {/* Thesis */}
247 |         {hypothesis.thesis && (
248 |             <Card className="bg-slate-800/50 border-slate-700/40 rounded-xl p-5">
249 |                 <h3 className="text-white font-semibold mb-3">Investment Thesis</h3>
250 |                 <p className="text-slate-300 text-sm leading-relaxed whitespace-pre-line">{hypothesis.thesis}</p>
251 |             </Card>
252 |         )}
253 |
254 |         {/* Catalysts & Risks */}
255 |         <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
256 |             {hypothesis.catalysts?.length > 0 && (
257 |                 <Card className="bg-emerald-950/20 border-emerald-800/30 rounded-xl p-5">
258 |                     <h3 className="text-emerald-400 font-semibold mb-3 flex items-center gap-2">
259 |                         <TrendingUp className="w-4 h-4" /> Key Catalysts
260 |                     </h3>
261 |                     <ul className="space-y-2">
262 |                         {hypothesis.catalysts.map((c, i) => (
263 |                             <li key={`catalyst-${i}-${c.slice(0,20)}`} className="text-slate-300 text-sm flex items-start gap-2">
264 |                                 <span className="text-emerald-500 mt-1">+</span> {c}
265 |                             </li>
266 |                         )})}
267 |                     </ul>
268 |                 </Card>
269 |             )}
```

```
270 |         {hypothesis.risks?.length > 0 && (
271 |             <Card className="bg-red-950/20 border-red-800/30 rounded-xl p-5">
272 |                 <h3 className="text-red-400 font-semibold mb-3 flex items-center gap-2">
273 |                     <TrendingDown className="w-4 h-4" /> Key Risks
274 |                 </h3>
275 |                 <ul className="space-y-2">
276 |                     {hypothesis.risks.map((r, i) => (
277 |                         <li key={i} className="text-slate-300 text-sm flex items-start gap-2">
278 |                             <span className="text-red-500 mt-1">-</span> {r}
279 |                         </li>
280 |                     ))}
281 |                 </ul>
282 |             </Card>
283 |         )}
284 |     </div>
285 |
286 |     {/* Congressional Activity & Sector Impact */}
287 |     <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
288 |         {hypothesis.congressional_activity && (
289 |             <Card className="bg-violet-950/20 border-violet-800/30 rounded-xl p-5">
290 |                 <h3 className="text-violet-400 font-semibold mb-3 flex items-center gap-2">
291 |                     <Landmark className="w-4 h-4" /> Congressional Activity
292 |                 </h3>
293 |                 <p className="text-slate-300 text-sm">{hypothesis.congressional_activity}</p>
294 |             </Card>
295 |         )}
296 |         {hypothesis.sector_impact && (
297 |             <Card className="bg-blue-950/20 border-blue-800/30 rounded-xl p-5">
298 |                 <h3 className="text-blue-400 font-semibold mb-3 flex items-center gap-2">
299 |                     <Globe className="w-4 h-4" /> Sector & Macro Impact
300 |                 </h3>
301 |                 <p className="text-slate-300 text-sm">{hypothesis.sector_impact}</p>
302 |             </Card>
303 |         )}
304 |     </div>
305 |
306 |     {/* Technical Outlook */}
307 |     {hypothesis.technical_outlook && (
308 |         <Card className="bg-slate-800/50 border-slate-700/40 rounded-xl p-5">
309 |             <h3 className="text-white font-semibold mb-3 flex items-center gap-2">
310 |                 <BarChart3 className="w-4 h-4 text-[#0052FF]" /> Technical Outlook
311 |             </h3>
312 |             <p className="text-slate-300 text-sm">{hypothesis.technical_outlook}</p>
313 |         </Card>
314 |     )}
315 | </div>
316 | )}
317 | </div>
318 | );
319 | };
320 |
321 | export default AIHypothesis;
322 |
```

FILE: frontend/src/components/AdditionalSections.jsx

58 lines | 1909 characters

```
1 | import React from 'react';
2 | import DataTable from '../DataTable';
3 | import { momentumData, fastMoverCallsData, unusualVolumeData } from '../mockData';
4 |
5 | const AdditionalSections = () => {
6 |     const momentumColumns = [
7 |         { key: 'contract', label: 'Contract', sortable: true },
8 |         { key: 'price', label: 'Price', sortable: true },
9 |         { key: 'returns', label: '% Returns', sortable: true },
```



```
10 |   { key: 'power', label: 'Power', sortable: true },
11 |   { key: 'volOI', label: 'Vol/OI', sortable: true },
12 |   { key: 'ivRank', label: 'IV Rank', sortable: true },
13 |   { key: 'aiScore', label: 'AI Score', sortable: true },
14 | ];
15 |
16 | const unusualVolumeColumns = [
17 |   { key: 'contract', label: 'Contract', sortable: true },
18 |   { key: 'price', label: 'Price', sortable: true },
19 |   { key: 'volOI', label: 'Vol/OI', sortable: true },
20 |   { key: 'returns', label: '% Returns', sortable: true },
21 |   { key: 'power', label: 'Power', sortable: true },
22 |   { key: 'sentiment', label: 'Sentiment', sortable: true },
23 |   { key: 'aiScore', label: 'AI Score', sortable: true },
24 | ];
25 |
26 | return (
27 |   <div className="space-y-6 mt-8">
28 |     {/* Momentum Close Strength and Fast Mover Calls */}
29 |     <div className="grid grid-cols-1 lg:grid-cols-2 gap-6">
30 |       <div id="momentum">
31 |         <DataTable
32 |           title="Momentum Close Strength"
33 |           columns={momentumColumns}
34 |           data={momentumData}
35 |         />
36 |       </div>
37 |       <div id="fast-movers">
38 |         <DataTable
39 |           title="Fast Mover Calls"
40 |           columns={momentumColumns}
41 |           data={fastMoverCallsData}
42 |         />
43 |       </div>
44 |     </div>
45 |
46 |     {/* Unusual Options Volume */}
47 |     <div id="unusual-volume">
48 |       <DataTable
49 |         title="Unusual Options Volume"
50 |         columns={unusualVolumeColumns}
51 |         data={unusualVolumeData}
52 |       />
53 |     </div>
54 |   </div>
55 | );
56 | };
57 |
58 | export default AdditionalSections;
```

FILE: frontend/src/components/AdminPanel.jsx

382 lines | 18210 characters

```
1 | import React, { useState, useEffect, useCallback } from 'react';
2 | import { Users, Shield, ShieldOff, Crown, UserCheck, UserX, RefreshCw, Search, Gift, Plus, Trash2, ToggleLeft, ToggleRight, Calend ...
3 | import { Card } from './ui/card';
4 | import { Badge } from './ui/badge';
5 | import { Button } from './ui/button';
6 | import { Input } from './ui/input';
7 | import { authFetch } from '../contexts/AuthContext';
8 |
9 | const API = `${process.env.REACT_APP_BACKEND_URL}/api`;
10 |
11 | const AdminPanel = ({ onClose }) => {
12 |   const [tab, setTab] = useState('users');
13 |   const [users, setUsers] = useState([]);
```

```
14 |   const [loading, setLoading] = useState(true);
15 |   const [filter, setFilter] = useState('');
16 |   const [actionLoading, setActionLoading] = useState(null);
17 |
18 |   const fetchUsers = useCallback(async () => {
19 |     setLoading(true);
20 |     try {
21 |       const res = await authFetch(`${API}/auth/admin/users`);
22 |       if (res.ok) {
23 |         const data = await res.json();
24 |         setUsers(data.users);
25 |       }
26 |     } catch (e) {
27 |       console.error('Error fetching users:', e);
28 |     } finally {
29 |       setLoading(false);
30 |     }
31 |   }, []);
32 |
33 |   useEffect(() => { fetchUsers(); }, [fetchUsers]);
34 |
35 |   const doAction = async (userId, action) => {
36 |     setActionLoading(`${userId}-${action}`);
37 |     try {
38 |       const res = await authFetch(`${API}/auth/admin/users/${userId}/${action}`, { method: 'POST' });
39 |       if (res.ok) fetchUsers();
40 |       else {
41 |         const d = await res.json();
42 |         alert(d.detail || 'Action failed');
43 |       }
44 |     } catch (e) {
45 |       alert('Action failed');
46 |     } finally {
47 |       setActionLoading(null);
48 |     }
49 |   };
50 |
51 |   const filtered = users.filter(u =>
52 |     u.name?.toLowerCase().includes(filter.toLowerCase()) ||
53 |     u.email?.toLowerCase().includes(filter.toLowerCase())
54 |   );
55 |
56 |   const tabs = [
57 |     { id: 'users', label: 'Users', icon: Users },
58 |     { id: 'promos', label: 'Promos', icon: Gift },
59 |   ];
60 |
61 |   return (
62 |     <div className="fixed inset-0 bg-black/70 backdrop-blur-sm z-50 flex items-start justify-center overflow-y-auto p-4" data-test ...
63 |       <div className="bg-slate-900 rounded-2xl max-w-4xl w-full my-4 border border-slate-700/50">
64 |         <div className="p-6 border-b border-slate-700 flex items-center justify-between">
65 |           <div className="flex items-center gap-3">
66 |             <div className="w-10 h-10 bg-red-600 rounded-xl flex items-center justify-center">
67 |               <Crown className="w-6 h-6 text-white" />
68 |             </div>
69 |             <div>
70 |               <h2 className="text-white text-xl font-bold">Admin Panel</h2>
71 |               <p className="text-slate-400 text-sm">{users.length} users total</p>
72 |             </div>
73 |           </div>
74 |           <div className="flex items-center gap-2">
75 |             <Button variant="outline" size="sm" onClick={fetchUsers} className="bg-slate-800 border-slate-700 text-white">
76 |               <RefreshCw className={`w-4 h-4 ${loading ? 'animate-spin' : ''}`} />
77 |             </Button>
78 |             <button onClick={onClose} className="text-slate-400 hover:text-white text-xl px-2">x</button>
79 |           </div>
80 |         </div>
```

```
81 |
82 |     {/* Tabs */}
83 |     <div className="flex border-b border-slate-700/50 px-4">
84 |         {tabs.map(t => (
85 |             <button
86 |                 key={t.id}
87 |                 onClick={() => setTab(t.id)}
88 |                 data-testid={`admin-tab-${t.id}`}
89 |                 className={`flex items-center gap-2 px-4 py-3 text-sm font-medium border-b-2 transition-all ${
90 |                     tab === t.id ? 'text-[#0052FF] border-[#0052FF]' : 'text-slate-400 border-transparent hover:text-slate-300'
91 |                 }`}
92 |             >
93 |                 <t.icon className="w-4 h-4" />
94 |                 {t.label}
95 |             </button>
96 |         ))}
97 |     </div>
98 |
99 |     {tab === 'users' ? (
100 |         <>
101 |             <div className="p-4 border-b border-slate-700/50">
102 |                 <div className="relative">
103 |                     <Search className="absolute left-3 top-1/2 -translate-y-1/2 text-slate-400 w-4 h-4" />
104 |                     <Input placeholder="Search users..." value={filter} onChange={e => setFilter(e.target.value)}
105 |                         className="pl-10 bg-slate-800 border-slate-600 text-white rounded-xl" />
106 |                 </div>
107 |             </div>
108 |
109 |             <div className="overflow-x-auto">
110 |                 <table className="w-full text-sm" data-testid="admin-users-table">
111 |                     <thead>
112 |                         <tr className="border-b border-slate-700/40">
113 |                             <th className="text-left text-slate-400 text-xs font-medium px-4 py-3">User</th>
114 |                             <th className="text-left text-slate-400 text-xs font-medium px-4 py-3">Role</th>
115 |                             <th className="text-left text-slate-400 text-xs font-medium px-4 py-3">Plan</th>
116 |                             <th className="text-left text-slate-400 text-xs font-medium px-4 py-3">Status</th>
117 |                             <th className="text-right text-slate-400 text-xs font-medium px-4 py-3">Actions</th>
118 |                         </tr>
119 |                     </thead>
120 |                     <tbody>
121 |                         {filtered.map(u => (
122 |                             <tr key={u._id} className="border-b border-slate-800/40 hover:bg-slate-800/30">
123 |                                 <td className="px-4 py-3">
124 |                                     <p className="text-white font-medium">{u.name}</p>
125 |                                     <p className="text-slate-500 text-xs">{u.email}</p>
126 |                                 </td>
127 |                                 <td className="px-4 py-3">
128 |                                     <Badge className={`text-[10px] ${
129 |                                         u.role === 'owner' ? 'bg-red-900/40 text-red-400 border-red-800' :
130 |                                         u.role === 'admin' ? 'bg-blue-900/40 text-blue-400 border-blue-800' :
131 |                                         'bg-slate-700 text-slate-400'
132 |                                     }`}>{u.role}</Badge>
133 |                                 </td>
134 |                                 <td className="px-4 py-3">
135 |                                     <span className={`text-xs font-semibold ${u.subscription_status === 'pro' ? 'text-[#0052FF]' : u.subscription ...
136 |                                         {u.subscription_status === 'pro' ? 'PRO' : u.subscription_status === 'trial' ? 'TRIAL' : 'FREE'}}
137 |                                     </span>
138 |                                 </td>
139 |                                 <td className="px-4 py-3">
140 |                                     <span className={`text-xs ${u.is_active !== false ? 'text-emerald-400' : 'text-red-400'}`}>
141 |                                         {u.is_active !== false ? 'Active' : 'Disabled'}
142 |                                     </span>
143 |                                 </td>
144 |                                 <td className="px-4 py-3 text-right">
145 |                                     {u.role !== 'owner' && (
146 |                                         <div className="flex items-center justify-end gap-1">
147 |                                             {u.subscription_status !== 'pro' ? (
```

```
148 |         <Button size="sm" variant="outline" className="text-[10px] px-2 py-1 h-7 bg-blue-900/30 text-blue-40 ...
149 |             disabled={actionLoading === `${u._id}-grant-pro`}
150 |             onClick={() => doAction(u._id, 'grant-pro')}>
151 |             <Shield className="w-3 h-3 mr-1" /> Grant Pro
152 |         </Button>
153 |     ) : (
154 |         <Button size="sm" variant="outline" className="text-[10px] px-2 py-1 h-7 bg-slate-700 text-slate-400 ...
155 |             disabled={actionLoading === `${u._id}-revoke-pro`}
156 |             onClick={() => doAction(u._id, 'revoke-pro')}>
157 |             <ShieldOff className="w-3 h-3 mr-1" /> Revoke Pro
158 |         </Button>
159 |     )}
160 |     {u.is_active !== false ? (
161 |         <Button size="sm" variant="outline" className="text-[10px] px-2 py-1 h-7 bg-red-900/30 text-red-400 ...
162 |             disabled={actionLoading === `${u._id}-deactivate`}
163 |             onClick={() => doAction(u._id, 'deactivate')}>
164 |             <UserX className="w-3 h-3 mr-1" /> Disable
165 |         </Button>
166 |     ) : (
167 |         <Button size="sm" variant="outline" className="text-[10px] px-2 py-1 h-7 bg-emerald-900/30 text-emer ...
168 |             disabled={actionLoading === `${u._id}-activate`}
169 |             onClick={() => doAction(u._id, 'activate')}>
170 |             <UserCheck className="w-3 h-3 mr-1" /> Enable
171 |         </Button>
172 |     )}
173 | </div>
174 | )}
175 | </td>
176 | </tr>
177 | )}}
178 | </tbody>
179 | </table>
180 | </div>
181 | </>
182 | ) : (
183 |     <PromoManager />
184 | )}
185 | </div>
186 | </div>
187 | );
188 | };
189 |
190 | const PromoManager = () => {
191 |     const [promos, setPromos] = useState([]);
192 |     const [loading, setLoading] = useState(true);
193 |     const [showCreate, setShowCreate] = useState(false);
194 |     const [form, setForm] = useState({
195 |         title: '',
196 |         message: '',
197 |         referral_target: 3,
198 |         reward_months: 2,
199 |         start_date: new Date().toISOString().split('T')[0],
200 |         end_date: '',
201 |     });
202 |     const [creating, setCreating] = useState(false);
203 |
204 |     const fetchPromos = useCallback(async () => {
205 |         setLoading(true);
206 |         try {
207 |             const res = await authFetch(`${API}/promo/all`);
208 |             if (res.ok) {
209 |                 const data = await res.json();
210 |                 setPromos(data.promos);
211 |             }
212 |         } catch (e) {
213 |             console.error('Error fetching promos:', e);
214 |         } finally {
```

```
215 |     setLoading(false);
216 |   }
217 | }, []);
218 |
219 | useEffect(() => { fetchPromos(); }, [fetchPromos]);
220 |
221 | const createPromo = async () => {
222 |   if (!form.title || !form.end_date) return;
223 |   setCreating(true);
224 |   try {
225 |     const body = {
226 |       ...form,
227 |       referral_target: parseInt(form.referral_target),
228 |       reward_months: parseInt(form.reward_months),
229 |       start_date: new Date(form.start_date).toISOString(),
230 |       end_date: new Date(form.end_date + 'T23:59:59').toISOString(),
231 |     };
232 |     const res = await authFetch(`${API}/promo/create`, {
233 |       method: 'POST',
234 |       headers: { 'Content-Type': 'application/json' },
235 |       body: JSON.stringify(body),
236 |     });
237 |     if (res.ok) {
238 |       setShowCreate(false);
239 |       setForm({ title: '', message: '', referral_target: 3, reward_months: 2, start_date: new Date().toISOString().split('T')[0] ...
240 |       fetchPromos();
241 |     }
242 |   } catch (e) {
243 |     console.error('Create promo error:', e);
244 |   } finally {
245 |     setCreating(false);
246 |   }
247 | };
248 |
249 | const togglePromo = async (id) => {
250 |   try {
251 |     await authFetch(`${API}/promo/${id}/toggle`, { method: 'PUT' });
252 |     fetchPromos();
253 |   } catch (e) {
254 |     console.error('Toggle promo error:', e);
255 |   }
256 | };
257 |
258 | const deletePromo = async (id) => {
259 |   try {
260 |     await authFetch(`${API}/promo/${id}`, { method: 'DELETE' });
261 |     fetchPromos();
262 |   } catch (e) {
263 |     console.error('Delete promo error:', e);
264 |   }
265 | };
266 |
267 | const formatDate = (d) => d ? new Date(d).toLocaleDateString('en-US', { month: 'short', day: 'numeric', year: 'numeric' }) : '';
268 |
269 | return (
270 |   <div className="p-4 space-y-4" data-testid="promo-manager">
271 |     <div className="flex items-center justify-between">
272 |       <h3 className="text-white text-sm font-semibold">Promo Campaigns</h3>
273 |       <Button size="sm" className="bg-[#0052FF] hover:bg-[#2563EB] text-white text-xs h-8" onClick={() => setShowCreate(!showCre ...
274 |         <Plus className="w-3.5 h-3.5 mr-1" /> New Campaign
275 |       </Button>
276 |     </div>
277 |
278 |     {/* Create form */}
279 |     {showCreate && (
280 |       <Card className="bg-slate-800/60 border-slate-700/40 rounded-xl p-4 space-y-3" data-testid="promo-create-form">
281 |         <div className="grid grid-cols-1 sm:grid-cols-2 gap-3">
```

```
282 |         <div>
283 |             <label className="text-slate-400 text-[10px] block mb-1">Title</label>
284 |             <Input
285 |                 placeholder="e.g., Launch Week Special"
286 |                 value={form.title}
287 |                 onChange={e => setForm({ ...form, title: e.target.value })}
288 |                 className="bg-slate-900 border-slate-600 text-white text-sm rounded-xl h-9"
289 |                 data-testid="promo-title-input"
290 |             />
291 |         </div>
292 |         <div>
293 |             <label className="text-slate-400 text-[10px] block mb-1">Message</label>
294 |             <Input
295 |                 placeholder="e.g., Refer 3 friends, get 2 months free!"
296 |                 value={form.message}
297 |                 onChange={e => setForm({ ...form, message: e.target.value })}
298 |                 className="bg-slate-900 border-slate-600 text-white text-sm rounded-xl h-9"
299 |                 data-testid="promo-message-input"
300 |             />
301 |         </div>
302 |     </div>
303 |     <div className="grid grid-cols-2 sm:grid-cols-4 gap-3">
304 |         <div>
305 |             <label className="text-slate-400 text-[10px] block mb-1">Referral Target</label>
306 |             <Input type="number" min="1" value={form.referral_target} onChange={e => setForm({ ...form, referral_target: e.targe ...
307 |                 className="bg-slate-900 border-slate-600 text-white text-sm rounded-xl h-9" data-testid="promo-target-input" />
308 |         </div>
309 |         <div>
310 |             <label className="text-slate-400 text-[10px] block mb-1">Reward (Months)</label>
311 |             <Input type="number" min="1" value={form.reward_months} onChange={e => setForm({ ...form, reward_months: e.target.va ...
312 |                 className="bg-slate-900 border-slate-600 text-white text-sm rounded-xl h-9" data-testid="promo-reward-input" />
313 |         </div>
314 |         <div>
315 |             <label className="text-slate-400 text-[10px] block mb-1">Start Date</label>
316 |             <Input type="date" value={form.start_date} onChange={e => setForm({ ...form, start_date: e.target.value })}
317 |                 className="bg-slate-900 border-slate-600 text-white text-sm rounded-xl h-9" data-testid="promo-start-input" />
318 |         </div>
319 |         <div>
320 |             <label className="text-slate-400 text-[10px] block mb-1">End Date</label>
321 |             <Input type="date" value={form.end_date} onChange={e => setForm({ ...form, end_date: e.target.value })}
322 |                 className="bg-slate-900 border-slate-600 text-white text-sm rounded-xl h-9" data-testid="promo-end-input" />
323 |         </div>
324 |     </div>
325 |     <div className="flex justify-end gap-2">
326 |         <Button size="sm" variant="outline" className="text-xs h-8 bg-slate-700 text-slate-300 border-slate-600" onClick={() = ...
327 |         <Button size="sm" className="text-xs h-8 bg-[#0052FF] text-white" onClick={createPromo} disabled={creating || !form.ti ...
328 |             {creating ? 'Creating...' : 'Create Campaign'}
329 |         </Button>
330 |     </div>
331 | </Card>
332 | )}
333 |
334 | {/* Promo list */}
335 | {loading ? (
336 |     <div className="flex justify-center py-8"><RefreshCw className="w-5 h-5 text-[#0052FF] animate-spin" /></div>
337 | ) : promos.length === 0 ? (
338 |     <div className="text-center py-8">
339 |         <Gift className="w-10 h-10 text-slate-600 mx-auto mb-3" />
340 |         <p className="text-slate-400 text-sm">No promo campaigns yet</p>
341 |         <p className="text-slate-500 text-xs mt-1">Create your first campaign to drive referrals</p>
342 |     </div>
343 | ) : (
344 |     <div className="space-y-2">
345 |         {promos.map(p => {
346 |             const now = new Date();
347 |             const isLive = p.is_active && new Date(p.start_date) <= now && new Date(p.end_date) >= now;
348 |             const isExpired = new Date(p.end_date) < now;
```

```
349 |         return (
350 |             <div key={p.id} className="flex items-center justify-between bg-slate-800/60 border border-slate-700/40 rounded-xl p ...
351 |                 <div className="flex-1 min-w-0">
352 |                     <div className="flex items-center gap-2 mb-1">
353 |                         <p className="text-white text-sm font-medium truncate">{p.title}</p>
354 |                         {isLive && <Badge className="text-[9px] bg-emerald-900/30 text-emerald-400 border-emerald-700/50 animate-pulse ...
355 |                         {isExpired && <Badge className="text-[9px] bg-slate-700 text-slate-400">EXPIRED</Badge>}
356 |                         {!p.is_active && !isExpired && <Badge className="text-[9px] bg-amber-900/30 text-amber-400 border-amber-700/50 ...
357 |                     </div>
358 |                     <p className="text-slate-400 text-xs truncate">{p.message}</p>
359 |                     <div className="flex items-center gap-3 mt-1">
360 |                         <span className="text-slate-500 text-[10px] flex items-center gap-1"><Calendar className="w-3 h-3" /> {formatD ...
361 |                         <span className="text-[#0052FF] text-[10px] font-medium">{p.referral_target} refs = {p.reward_months} months</ ...
362 |                     </div>
363 |                 </div>
364 |                 <div className="flex items-center gap-2 ml-3">
365 |                     <button onClick={() => togglePromo(p.id)} className="text-slate-400 hover:text-white transition-colors" title={p ...
366 |                         {p.is_active ? <ToggleRight className="w-6 h-6 text-emerald-400" /> : <ToggleLeft className="w-6 h-6" />}
367 |                     </button>
368 |                     <button onClick={() => deletePromo(p.id)} className="text-slate-400 hover:text-red-400 transition-colors" title= ...
369 |                     <Trash2 className="w-4 h-4" />
370 |                 </div>
371 |             </div>
372 |         </div>
373 |     );
374 |   }}}
375 | </div>
376 |   )}
377 | </div>
378 | );
379 | };
380 |
381 | export default AdminPanel;
382 |
```

FILE: frontend/src/components/AlertsPanel.jsx

172 lines | 7840 characters

```
1 | import React, { useState, useEffect, useCallback } from 'react';
2 | import { Bell, X, TrendingUp, TrendingDown, Minus, Lock, Star } from 'lucide-react';
3 | import { Card } from './ui/card';
4 | import { Button } from './ui/button';
5 | import { Badge } from './ui/badge';
6 | import { useAuth, authFetch } from '../contexts/AuthContext';
7 |
8 | const API = `${process.env.REACT_APP_BACKEND_URL}/api`;
9 |
10 | const AlertsPanel = ({ onSubscribe }) => {
11 |   const { user, isPro } = useAuth();
12 |   const [notifications, setNotifications] = useState([]);
13 |   const [unreadCount, setUnreadCount] = useState(0);
14 |   const [isOpen, setIsOpen] = useState(false);
15 |   const [loading, setLoading] = useState(false);
16 |
17 |   const fetchNotifications = useCallback(async () => {
18 |     if (!user || !isPro) return;
19 |     try {
20 |       const res = await authFetch(`${API}/notifications`);
21 |       if (res.ok) {
22 |         const data = await res.json();
23 |         setNotifications(data.notifications || []);
24 |         setUnreadCount(data.unread_count || 0);
25 |       }
26 |     } catch (e) {
27 |       console.error('Notification fetch error:', e);
28 |     }
```

```
29 |   }, [user, isPro]);
30 |
31 |   useEffect(() => {
32 |     fetchNotifications();
33 |     if (!isPro) return;
34 |     const interval = setInterval(fetchNotifications, 60000);
35 |     return () => clearInterval(interval);
36 |   }, [fetchNotifications, isPro]);
37 |
38 |   const markAllRead = async () => {
39 |     setLoading(true);
40 |     try {
41 |       await authFetch(`${API}/notifications/read-all`, { method: 'POST' });
42 |       setNotifications(prev => prev.map(n => ({ ...n, read: true })));
43 |       setUnreadCount(0);
44 |     } catch (e) {
45 |       console.error('Mark read error:', e);
46 |     } finally {
47 |       setLoading(false);
48 |     }
49 |   };
50 |
51 |   const verdictIcon = (verdict) => {
52 |     if (verdict === 'BUY') return <TrendingUp className="w-4 h-4 text-emerald-400" />;
53 |     if (verdict === 'SELL') return <TrendingDown className="w-4 h-4 text-red-400" />;
54 |     return <Minus className="w-4 h-4 text-amber-400" />;
55 |   };
56 |
57 |   const verdictColor = (verdict) => {
58 |     if (verdict === 'BUY') return 'text-emerald-400';
59 |     if (verdict === 'SELL') return 'text-red-400';
60 |     return 'text-amber-400';
61 |   };
62 |
63 |   if (!user) return null;
64 |
65 |   return (
66 |     <div className="fixed top-20 right-6 z-40" data-testid="alerts-panel">
67 |       <div>
68 |         <button
69 |           onClick={() => setIsOpen(!isOpen)}
70 |           className="relative bg-[#0F172A] border border-slate-700 rounded-full p-3 hover:bg-slate-800 transition-colors"
71 |           data-testid="alerts-bell">
72 |           >
73 |             <Bell className="w-5 h-5 text-white" />
74 |             {isPro && unreadCount > 0 && (
75 |               <span className="absolute -top-1 -right-1 bg-[#0052FF] text-white text-xs rounded-full w-5 h-5 flex items-center justify-between" />
76 |               {unreadCount > 9 ? '9+' : unreadCount}
77 |             )}
78 |           </div>
79 |           {!isPro && (
80 |             <span className="absolute -top-1 -right-1 bg-slate-600 text-slate-300 text-[8px] rounded-full w-4 h-4 flex items-center justify-between" />
81 |             <Lock className="w-2.5 h-2.5" />
82 |           )}
83 |         </button>
84 |       </div>
85 |
86 |       <div>
87 |         {isOpen && (
88 |           <Card className="absolute top-14 right-0 w-[340px] sm:w-96 bg-slate-900 border-slate-700/50 shadow-2xl rounded-xl max-h-[400px] p-4" />
89 |           <div className="border-b border-slate-700 p-4 flex items-center justify-between">
90 |             <div className="flex items-center gap-2">
91 |               <h3 className="text-white font-semibold text-sm">AI Alerts</h3>
92 |               {isPro && unreadCount > 0 && (
93 |                 <Badge className="bg-[#0052FF] text-white text-[10px] px-1.5 py-0">{unreadCount} new</Badge>
94 |               )}
95 |               {isPro && <Badge className="bg-[#0052FF]/20 text-[#0052FF] border-0 text-[10px]">PRO</Badge>}
```



```
96 |         </div>
97 |         <div className="flex items-center gap-1">
98 |             {isPro && notifications.length > 0 && (
99 |                 <Button variant="ghost" size="sm" onClick={markAllRead} disabled={loading || unreadCount === 0}
100 |                     className="text-slate-400 hover:text-white text-[10px] h-7 px-2">
101 |                         Mark all read
102 |                     </Button>
103 |                 )}
104 |                 <button onClick={() => setIsOpen(false)} className="text-slate-400 hover:text-white p-1">
105 |                     <X className="w-4 h-4" />
106 |                 </button>
107 |             </div>
108 |         </div>
109 |
110 |         <div className="overflow-y-auto flex-1">
111 |             {!isPro ? (
112 |                 /* Free user ? paywall teaser */
113 |                 <div className="text-center py-8 px-6" data-testid="alerts-paywall">
114 |                     <div className="w-14 h-14 bg-slate-800 rounded-2xl flex items-center justify-center mx-auto mb-4">
115 |                         <Lock className="w-7 h-7 text-slate-500" />
116 |                     </div>
117 |                     <h4 className="text-white font-semibold mb-1">Pro AI Alerts</h4>
118 |                     <p className="text-slate-400 text-xs leading-relaxed mb-4">
119 |                         Get notified when our AI detects a verdict change on your watchlist tickers. Never miss a BUY?SELL flip.
120 |                     </p>
121 |                     <Button className="bg-[#0052FF] hover:bg-[#2563EB] text-white rounded-xl text-sm w-full" onClick={() => { onSubscr ...
122 |                         Upgrade to Pro
123 |                     </Button>
124 |                 </div>
125 |             ) : notifications.length === 0 ? (
126 |                 /* Pro but no notifications */
127 |                 <div className="text-center py-8 px-4">
128 |                     <Bell className="w-10 h-10 text-slate-600 mx-auto mb-3" />
129 |                     <p className="text-slate-400 text-sm">No alerts yet</p>
130 |                     <p className="text-slate-500 text-xs mt-1">You'll be notified when AI detects a verdict change on your watchlist t ...
131 |                 </div>
132 |             ) : (
133 |                 /* Pro with notifications */
134 |                 <div className="divide-y divide-slate-800/60">
135 |                     {notifications.map((n, i) => (
136 |                         <div key={i} className={`px-4 py-3 transition-colors ${!n.read ? 'bg-[#0052FF]/5' : 'hover:bg-slate-800/40'}`} d ...
137 |                             <div className="flex items-start gap-3">
138 |                                 <div className="mt-0.5 flex-shrink-0">
139 |                                     {verdictIcon(n.new_verdict)}
140 |                                 </div>
141 |                                 <div className="flex-1 min-w-0">
142 |                                     <div className="flex items-center gap-1.5 mb-0.5">
143 |                                         <span className="text-white text-sm font-semibold">{n.symbol}</span>
144 |                                         {n.in_watchlist && <Star className="w-3 h-3 text-amber-400 fill-amber-400" />}
145 |                                         {!n.read && <div className="w-1.5 h-1.5 bg-[#0052FF] rounded-full" />}
146 |                                     </div>
147 |                                     <p className="text-slate-300 text-xs">
148 |                                         Verdict changed: <span className={verdictColor(n.old_verdict)}>{n.old_verdict}</span>
149 |                                         {' ? '}
150 |                                         <span className={verdictColor(n.new_verdict)}>{n.new_verdict}</span>
151 |                                     </p>
152 |                                     {n.confidence > 0 && (
153 |                                         <p className="text-slate-500 text-[10px] mt-0.5">Confidence: {n.confidence}%</p>
154 |                                     )}
155 |                                     <p className="text-slate-600 text-[10px] mt-0.5">
156 |                                         {n.created_at ? new Date(n.created_at).toLocaleDateString('en-US', { month: 'short', day: 'numeric', hou ...
157 |                                     </p>
158 |                                 </div>
159 |                             </div>
160 |                         </div>
161 |                     )))}
162 |                 </div>
```

```
163 |         )}
164 |     </div>
165 | </Card>
166 |     )}
167 | </div>
168 | );
169 | };
170 |
171 | export default AlertsPanel;
172 |
```

FILE: frontend/src/components/AuthModal.jsx*140 lines | 5478 characters*

```
1 | import React, { useState, useEffect } from 'react';
2 | import { X, Mail, Lock, User, Eye, EyeOff, Gift } from 'lucide-react';
3 | import { Button } from './ui/button';
4 | import { Input } from './ui/input';
5 | import { useAuth } from '../contexts/AuthContext';
6 |
7 | const AuthModal = ({ onClose, initialTab = 'login' }) => {
8 |   const [tab, setTab] = useState(initialTab);
9 |   const [email, setEmail] = useState('');
10 |   const [password, setPassword] = useState('');
11 |   const [name, setName] = useState('');
12 |   const [showPw, setShowPw] = useState(false);
13 |   const [error, setError] = useState('');
14 |   const [loading, setLoading] = useState(false);
15 |   const { login, register } = useAuth();
16 |
17 |   // Detect referral code from URL
18 |   const refCode = new URLSearchParams(window.location.search).get('ref') || '';
19 |
20 |   useEffect(() => {
21 |     if (refCode) setTab('register');
22 |   }, [refCode]);
23 |
24 |   const handleSubmit = async (e) => {
25 |     e.preventDefault();
26 |     setError('');
27 |     setLoading(true);
28 |     try {
29 |       if (tab === 'login') {
30 |         await login(email, password);
31 |       } else {
32 |         await register(email, password, name, refCode);
33 |       }
34 |       onClose();
35 |     } catch (err) {
36 |       setError(err.message);
37 |     } finally {
38 |       setLoading(false);
39 |     }
40 |   };
41 |
42 |   return (
43 |     <div className="fixed inset-0 bg-black/70 backdrop-blur-sm z-50 flex items-center justify-center p-4" data-testid="auth-modal" ...
44 |       <div className="bg-slate-900 rounded-2xl border border-slate-700/50 w-full max-w-md p-6 relative">
45 |         <button onClick={onClose} className="absolute top-4 right-4 text-slate-400 hover:text-white" data-testid="auth-close-btn">
46 |           <X className="w-5 h-5" />
47 |         </button>
48 |
49 |         {/* Tab Toggle */}
50 |         <div className="flex gap-1 bg-slate-800 rounded-xl p-1 mb-6">
51 |           {[ 'login', 'register' ].map(t => (
52 |             <button
```

```

53 |         key={t}
54 |         onClick={() => { setTab(t); setError(''); }}
55 |         data-testid={`auth-tab-${t}`}
56 |         className={`flex-1 py-2 text-sm font-medium rounded-lg transition-all ${
57 |             tab === t ? 'bg-[#0052FF] text-white' : 'text-slate-400 hover:text-white'
58 |         }}
59 |     >
60 |         {t === 'login' ? 'Log In' : 'Sign Up'}
61 |     </button>
62 |   )}
63 | </div>
64 |
65 |   <h2 className="text-white text-xl font-bold mb-1">
66 |     {tab === 'login' ? 'Welcome back' : 'Create your account'}
67 |   </h2>
68 |   <p className="text-slate-400 text-sm mb-6">
69 |     {tab === 'login' ? 'Log in to access your workspace' : 'Start your AI trading journey'}
70 |   </p>
71 |
72 |   {error && (
73 |     <div className="bg-red-900/30 border border-red-800/50 text-red-400 text-sm p-3 rounded-lg mb-4" data-testid="auth-error ...
74 |       {error}
75 |     </div>
76 |   )}
77 |
78 |   <form onSubmit={handleSubmit} className="space-y-4">
79 |     {tab === 'register' && refCode && (
80 |       <div className="bg-emerald-900/20 border border-emerald-700/40 rounded-xl px-3 py-2 flex items-center gap-2" data-test ...
81 |         <Gift className="w-4 h-4 text-emerald-400 flex-shrink-0" />
82 |         <p className="text-emerald-300 text-xs">You've been referred! Sign up to get a <strong>7-day free Pro trial</strong> ...
83 |       </div>
84 |     )}
85 |     {tab === 'register' && (
86 |       <div className="relative">
87 |         <User className="absolute left-3 top-1/2 -translate-y-1/2 text-slate-400 w-4 h-4" />
88 |         <Input
89 |           placeholder="Full name"
90 |           value={name}
91 |           onChange={e => setName(e.target.value)}
92 |           className="pl-10 bg-slate-800 border-slate-600 text-white rounded-xl"
93 |           data-testid="auth-name-input"
94 |         />
95 |       </div>
96 |     )}
97 |     <div className="relative">
98 |       <Mail className="absolute left-3 top-1/2 -translate-y-1/2 text-slate-400 w-4 h-4" />
99 |       <Input
100 |         type="email"
101 |         placeholder="Email address"
102 |         value={email}
103 |         onChange={e => setEmail(e.target.value)}
104 |         required
105 |         className="pl-10 bg-slate-800 border-slate-600 text-white rounded-xl"
106 |         data-testid="auth-email-input"
107 |       />
108 |     </div>
109 |     <div className="relative">
110 |       <Lock className="absolute left-3 top-1/2 -translate-y-1/2 text-slate-400 w-4 h-4" />
111 |       <Input
112 |         type={showPw ? 'text' : 'password'}
113 |         placeholder="Password"
114 |         value={password}
115 |         onChange={e => setPassword(e.target.value)}
116 |         required
117 |         minLength={6}
118 |         className="pl-10 pr-10 bg-slate-800 border-slate-600 text-white rounded-xl"
119 |         data-testid="auth-password-input"

```

```
120 |         />
121 |         <button type="button" onClick={() => setShowPw(!showPw)} className="absolute right-3 top-1/2 -translate-y-1/2 text-sla ...
122 |         {showPw ? <EyeOff className="w-4 h-4" /> : <Eye className="w-4 h-4" />}
123 |     </button>
124 | </div>
125 |     <Button
126 |         type="submit"
127 |         disabled={loading}
128 |         className="w-full bg-[#0052FF] hover:bg-[#2563EB] text-white font-semibold py-5 rounded-xl"
129 |         data-testid="auth-submit-btn"
130 |     >
131 |         {loading ? 'Please wait...' : tab === 'login' ? 'Log In' : 'Create Account'}
132 |     </Button>
133 | </form>
134 | </div>
135 | </div>
136 | );
137 | };
138 |
139 | export default AuthModal;
140 |
```

FILE: frontend/src/components/BrokerConnect.jsx

256 lines | 9575 characters

```
1 | import React, { useState } from 'react';
2 | import ReactDOM from 'react-dom';
3 | import { Building2, CheckCircle, ExternalLink, ArrowRight, Shield } from 'lucide-react';
4 | import { Button } from './ui/button';
5 | import { Card } from './ui/card';
6 | import { Badge } from './ui/badge';
7 |
8 | const BrokerConnect = () => {
9 |     const [connectedBrokers, setConnectedBrokers] = useState([]);
10 |     const [isModalOpen, setIsModalOpen] = useState(false);
11 |
12 |     const brokers = [
13 |         {
14 |             id: 'td_ameritrade',
15 |             name: 'TD Ameritrade',
16 |             logo: '?',
17 |             description: 'Commission-free trades with advanced tools',
18 |             features: ['Real-time data', 'Options trading', 'Paper trading'],
19 |             apiDocs: 'https://developer.tdameritrade.com',
20 |             connectUrl: 'https://www.tdameritrade.com',
21 |             popular: true
22 |         },
23 |         {
24 |             id: 'interactive_brokers',
25 |             name: 'Interactive Brokers',
26 |             logo: '?',
27 |             description: 'Professional trading platform with global access',
28 |             features: ['Low commissions', 'Global markets', 'API access'],
29 |             apiDocs: 'https://www.interactivebrokers.com/api',
30 |             connectUrl: 'https://www.interactivebrokers.com',
31 |             popular: true
32 |         },
33 |         {
34 |             id: 'robinhood',
35 |             name: 'Robinhood',
36 |             logo: '?',
37 |             description: 'Simple commission-free investing',
38 |             features: ['Zero commissions', 'Crypto trading', 'Easy to use'],
39 |             apiDocs: 'https://robinhood.com/us/en/support/articles/api/',
40 |             connectUrl: 'https://robinhood.com',
41 |             popular: true
```

```
42 |     },
43 |     {
44 |       id: 'etrade',
45 |       name: 'E*TRADE',
46 |       logo: '?',
47 |       description: 'Comprehensive trading and investing platform',
48 |       features: ['Research tools', 'Retirement accounts', 'Mobile app'],
49 |       apiDocs: 'https://developer.etrade.com',
50 |       connectUrl: 'https://www.etrade.com',
51 |       popular: false
52 |     },
53 |     {
54 |       id: 'schwab',
55 |       name: 'Charles Schwab',
56 |       logo: '??',
57 |       description: 'Full-service brokerage with excellent research',
58 |       features: ['No commissions', 'Banking services', 'Robo-advisor'],
59 |       apiDocs: 'https://developer.schwab.com',
60 |       connectUrl: 'https://www.schwab.com',
61 |       popular: false
62 |     },
63 |     {
64 |       id: 'fidelity',
65 |       name: 'Fidelity',
66 |       logo: '?',
67 |       description: 'Trusted broker with extensive resources',
68 |       features: ['Research', 'Retirement planning', 'No fees'],
69 |       apiDocs: 'https://www.fidelity.com/bin-public/060_www_fidelity_com/documents/landing-zones/Fidelity-WealthscapeIntegration-G ...',
70 |       connectUrl: 'https://www.fidelity.com',
71 |       popular: false
72 |     },
73 |     {
74 |       id: 'webull',
75 |       name: 'Webull',
76 |       logo: '?',
77 |       description: 'Mobile-first platform with advanced charts',
78 |       features: ['Paper trading', 'Extended hours', 'Free tools'],
79 |       apiDocs: 'https://www.webull.com',
80 |       connectUrl: 'https://www.webull.com',
81 |       popular: false
82 |     },
83 |     {
84 |       id: 'alpaca',
85 |       name: 'Alpaca',
86 |       logo: '?',
87 |       description: 'Commission-free API-first broker for algo trading',
88 |       features: ['API trading', 'Paper trading', 'No minimums'],
89 |       apiDocs: 'https://alpaca.markets/docs',
90 |       connectUrl: 'https://alpaca.markets',
91 |       popular: true
92 |     }
93 |   ];
94 |
95 |   const handleConnect = (broker) => {
96 |     // In production, this would initiate OAuth flow
97 |     const isConnected = connectedBrokers.includes(broker.id);
98 |
99 |     if (isConnected) {
100 |       setConnectedBrokers(connectedBrokers.filter(id => id !== broker.id));
101 |       alert(`Disconnected from ${broker.name}`);
102 |     } else {
103 |       // Open broker's website in new tab
104 |       window.open(broker.connectUrl, '_blank');
105 |       // Simulate connection (in production, this would happen after OAuth)
106 |       setConnectedBrokers([...connectedBrokers, broker.id]);
107 |     }
108 |   };

```

```
109 |  
110 |   const BrokerCard = ({ broker }) => {  
111 |     const isConnected = connectedBrokers.includes(broker.id);  
112 |  
113 |     return (  
114 |       <Card className="bg-slate-800/50 border-slate-700/50 rounded-xl p-5 hover:border-slate-600 transition-all">  
115 |         <div className="flex items-start justify-between mb-3">  
116 |           <div className="flex items-center gap-3">  
117 |             <div className="text-4xl">{broker.logo}</div>  
118 |             <div>  
119 |               <div className="flex items-center gap-2">  
120 |                 <h4 className="text-white font-semibold">{broker.name}</h4>  
121 |                 {broker.popular && (  
122 |                   <Badge className="bg-[#0052FF] text-xs">Popular</Badge>  
123 |                 )}  
124 |                 {isConnected && (  
125 |                   <Badge className="bg-green-600 text-xs flex items-center gap-1">  
126 |                     <CheckCircle className="w-3 h-3" />  
127 |                     Connected  
128 |                   </Badge>  
129 |                 )}  
130 |               </div>  
131 |               <p className="text-slate-400 text-sm mt-1">{broker.description}</p>  
132 |             </div>  
133 |           </div>  
134 |         </div>  
135 |  
136 |         {/* Features */}  
137 |         <div className="flex flex-wrap gap-2 mb-4">  
138 |           {broker.features.map((feature, index) => (  
139 |             <span  
140 |               key={index}  
141 |               className="bg-[#1E293B] text-slate-300 text-xs px-2 py-1 rounded"  
142 |             >  
143 |               {feature}  
144 |             </span>  
145 |           ))}  
146 |         </div>  
147 |  
148 |         {/* Actions */}  
149 |         <div className="flex gap-2">  
150 |           <Button  
151 |             onClick={() => handleConnect(broker)}  
152 |             className={`flex-1 ${  
153 |               isConnected  
154 |               ? 'bg-slate-600 hover:bg-gray-600'  
155 |               : 'bg-[#0052FF] hover:bg-[#2563EB]'  
156 |             }`} >  
157 |             {isConnected ? 'Disconnect' : 'Connect Account'}  
158 |             {!isConnected && <ArrowRight className="w-4 h-4 ml-2" />}  
159 |           </Button>  
160 |           <Button  
161 |             variant="outline"  
162 |             size="sm"  
163 |             onClick={() => window.open(broker.apiDocs, '_blank')}  
164 |             className="bg-[#1E293B] border-slate-600 text-white hover:bg-slate-700"  
165 |           >  
166 |             API Docs  
167 |             <ExternalLink className="w-3 h-3 ml-1" />  
168 |           </Button>  
169 |         </div>  
170 |       </Card>  
171 |     );  
172 |   };  
173 | };  
174 |  
175 | return (
```

```
176 |     <>
177 |     {/* Trigger Button */}
178 |     <Button
179 |         onClick={() => setIsModalOpen(true)}
180 |         className="bg-[#0052FF] hover:bg-[#2563EB]"
181 |     >
182 |         <Building2 className="w-4 h-4 mr-2" />
183 |         Connect Broker
184 |         {connectedBrokers.length > 0 && (
185 |             <Badge className="ml-2 bg-green-600">{connectedBrokers.length}</Badge>
186 |         )}
187 |     </Button>
188 |
189 |     {/* Modal via Portal to escape navbar stacking context */}
190 |     {isModalOpen && ReactDOM.createPortal(
191 |         <div className="fixed inset-0 bg-black/70 backdrop-blur-sm z-50 flex items-center justify-center p-4">
192 |             <div className="bg-slate-900 rounded-2xl max-w-6xl w-full max-h-[90vh] overflow-hidden flex flex-col border border-slate ...
193 |                 {/* Header */}
194 |                 <div className="border-b border-slate-700 p-6">
195 |                     <div className="flex items-start justify-between">
196 |                         <div>
197 |                             <h2 className="text-white text-2xl font-bold mb-2">Connect Your Broker</h2>
198 |                             <p className="text-slate-400">
199 |                                 Link your brokerage account to execute trades directly from RISEDUAL AI
200 |                             </p>
201 |                             <div className="flex items-center gap-2 mt-3 text-sm text-slate-500">
202 |                                 <Shield className="w-4 h-4" />
203 |                                 <span>Bank-level security ? OAuth 2.0 ? Data encrypted</span>
204 |                             </div>
205 |                         </div>
206 |                         <button
207 |                             onClick={() => setIsModalOpen(false)}
208 |                             className="text-slate-400 hover:text-slate-50 text-2xl"
209 |                         >
210 |                             ?
211 |                         </button>
212 |                     </div>
213 |                 </div>
214 |
215 |                 {/* Connected Brokers Summary */}
216 |                 {connectedBrokers.length > 0 && (
217 |                     <div className="bg-emerald-900 bg-opacity-20 border-b border-green-800 p-4">
218 |                         <div className="flex items-center gap-2 text-emerald-400">
219 |                             <CheckCircle className="w-5 h-5" />
220 |                             <span className="font-medium">
221 |                                 {connectedBrokers.length} broker{connectedBrokers.length > 1 ? 's' : ''} connected
222 |                             </span>
223 |                         </div>
224 |                     </div>
225 |                 )}
226 |
227 |                 {/* Brokers Grid */}
228 |                 <div className="flex-1 overflow-y-auto p-6">
229 |                     <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
230 |                         {brokers.map((broker) => (
231 |                             <BrokerCard key={broker.id} broker={broker} />
232 |                         ))}
233 |                     </div>
234 |
235 |                     {/* Info Section */}
236 |                     <div className="mt-6 p-4 bg-slate-800/50 border border-slate-700/50 rounded-xl">
237 |                         <h3 className="text-white font-semibold mb-2">How it works:</h3>
238 |                         <ol className="text-slate-400 text-sm space-y-1 list-decimal list-inside">
239 |                             <li>Click "Connect Account" on your preferred broker</li>
240 |                             <li>You'll be redirected to securely login to your broker account</li>
241 |                             <li>Authorize RISEDUAL AI to access trading capabilities</li>
242 |                             <li>Start executing trades directly from our platform</li>
```

```
243 |         </ol>
244 |         <p className="text-slate-500 text-xs mt-3">
245 |             Note: This feature requires broker API access. Some brokers may require additional approval.
246 |         </p>
247 |     </div>
248 | </div>
249 | </div>
250 | </div>
251 |     , document.body))
252 | </>
253 | );
254 | };
255 |
256 | export default BrokerConnect;
```

FILE: frontend/src/components/ChartPatternLibrary.jsx

153 lines | 5388 characters

```
1 | import React from 'react';
2 |
3 | const patternData = [
4 |   {
5 |     id: 'head-shoulders',
6 |     name: 'Head & Shoulders',
7 |     type: 'Bearish Reversal',
8 |     description: 'Three peaks where the middle peak (head) is highest. Signals trend reversal from bullish to bearish when necklin ...
9 |     color: '#ef4444',
10 |     points: '20,70 40,40 55,60 75,20 95,60 110,40 130,70',
11 |     neckline: { x1: 20, y1: 70, x2: 130, y2: 70 },
12 |   },
13 |   {
14 |     id: 'double-top',
15 |     name: 'Double Top',
16 |     type: 'Bearish Reversal',
17 |     description: 'Two peaks at roughly the same price level. Bearish signal when price breaks below the support between the peaks. ...
18 |     color: '#ef4444',
19 |     points: '20,70 45,25 75,60 105,25 130,70',
20 |     neckline: { x1: 20, y1: 60, x2: 130, y2: 60 },
21 |   },
22 |   {
23 |     id: 'double-bottom',
24 |     name: 'Double Bottom',
25 |     type: 'Bullish Reversal',
26 |     description: 'Two troughs at roughly the same price level. Bullish signal when price breaks above resistance between the troug ...
27 |     color: '#22c55e',
28 |     points: '20,25 45,75 75,35 105,75 130,25',
29 |     neckline: { x1: 20, y1: 35, x2: 130, y2: 35 },
30 |   },
31 |   {
32 |     id: 'ascending-triangle',
33 |     name: 'Ascending Triangle',
34 |     type: 'Bullish Continuation',
35 |     description: 'Flat resistance with rising support. Indicates buying pressure building. Bullish breakout expected.',
36 |     color: '#22c55e',
37 |     points: '20,75 40,30 55,65 75,30 90,55 110,30 130,15',
38 |     neckline: { x1: 20, y1: 30, x2: 130, y2: 30 },
39 |   },
40 |   {
41 |     id: 'descending-triangle',
42 |     name: 'Descending Triangle',
43 |     type: 'Bearish Continuation',
44 |     description: 'Flat support with falling resistance. Selling pressure is building. Bearish breakdown expected.',
45 |     color: '#ef4444',
46 |     points: '20,20 40,70 55,30 75,70 90,40 110,70 130,80',
47 |     neckline: { x1: 20, y1: 70, x2: 130, y2: 70 },
48 |   },
```



```
49 | {
50 |   id: 'cup-handle',
51 |   name: 'Cup & Handle',
52 |   type: 'Bullish Continuation',
53 |   description: 'U-shaped cup followed by a small handle pullback. Strong bullish pattern with breakout above cup rim.',
54 |   color: '#22c55e',
55 |   points: '15,25 25,40 35,55 50,70 65,75 80,70 95,55 105,40 115,25 122,32 130,20',
56 |   neckline: { x1: 15, y1: 25, x2: 130, y2: 25 },
57 | },
58 | {
59 |   id: 'bull-flag',
60 |   name: 'Bull Flag',
61 |   type: 'Bullish Continuation',
62 |   description: 'Sharp rise (flagpole) followed by a slight downward consolidation (flag). Expect continuation upward.',
63 |   color: '#22c55e',
64 |   points: '15,80 35,25 50,35 65,30 80,40 95,35 110,25 130,10',
65 |   neckline: null,
66 | },
67 | {
68 |   id: 'bear-flag',
69 |   name: 'Bear Flag',
70 |   type: 'Bearish Continuation',
71 |   description: 'Sharp decline (flagpole) followed by slight upward consolidation (flag). Expect continuation downward.',
72 |   color: '#ef4444',
73 |   points: '15,15 35,70 50,60 65,65 80,55 95,60 110,70 130,85',
74 |   neckline: null,
75 | },
76 | ];
77 |
78 | const PatternSVG = ({ pattern }) => (
79 |   <svg viewBox="0 0 150 90" className="w-full h-full">
80 |     <rect x="0" y="0" width="150" height="90" fill="#1E293B" rx="4" />
81 |     { /* Grid lines */ }
82 |     [[20, 40, 60, 80].map((y) => (
83 |       <line key={y} x1="10" y1={y} x2="140" y2={y} stroke="#334155" strokeWidth="0.5" />
84 |     ))]
85 |     { /* Neckline */ }
86 |     {pattern.neckline && (
87 |       <line
88 |         x1={pattern.neckline.x1}
89 |         y1={pattern.neckline.y1}
90 |         x2={pattern.neckline.x2}
91 |         y2={pattern.neckline.y2}
92 |         stroke="#fbbf24"
93 |         strokeWidth="1"
94 |         strokeDasharray="4,3"
95 |         opacity="0.6"
96 |       />
97 |     )}
98 |     { /* Main pattern line */ }
99 |     <polyline
100 |       points={pattern.points}
101 |       fill="none"
102 |       stroke={pattern.color}
103 |       strokeWidth="2.5"
104 |       strokeLinejoin="round"
105 |       strokeLinecap="round"
106 |     />
107 |     { /* Glow effect */ }
108 |     <polyline
109 |       points={pattern.points}
110 |       fill="none"
111 |       stroke={pattern.color}
112 |       strokeWidth="4"
113 |       strokeLinejoin="round"
114 |       strokeLinecap="round"
115 |       opacity="0.2"
```

```
116 |     />
117 |   </svg>
118 | );
119 |
120 | const PatternCard = ({ pattern, onClick }) => (
121 |   <button
122 |     onClick={() => onClick(pattern)}
123 |     className="bg-[#1E293B] border border-slate-700 rounded-xl p-2 hover:border-[#0052FF] transition-all duration-200 hover:shadow ...
124 |     data-testid={`pattern-card-${pattern.id}`}
125 |   >
126 |     <div className="aspect-[5/3] mb-2 rounded overflow-hidden">
127 |       <PatternSVG pattern={pattern} />
128 |     </div>
129 |     <p className="text-xs font-semibold text-white truncate group-hover:text-blue-400 transition-colors">{pattern.name}</p>
130 |     <p className={`text-[10px] font-medium ${pattern.color} === '#22c55e' ? 'text-emerald-400' : 'text-red-400'}`>
131 |       {pattern.type}
132 |     </p>
133 |   </button>
134 | );
135 |
136 | const ChartPatternLibrary = ({ onSelectPattern }) => (
137 |   <div className="bg-[#0F172A] rounded-xl p-3 border border-slate-700" data-testid="pattern-library">
138 |     <div className="flex items-center gap-2 mb-3">
139 |       <div className="w-2 h-2 rounded-full bg-blue-500 animate-pulse" />
140 |       <p className="text-xs font-bold text-blue-400 uppercase tracking-wider">Chart Pattern Library</p>
141 |     </div>
142 |     <p className="text-[11px] text-slate-400 mb-3">Tap any pattern to get AI analysis and trading strategies.</p>
143 |     <div className="grid grid-cols-2 gap-2">
144 |       {patternData.map((pattern) => (
145 |         <PatternCard key={pattern.id} pattern={pattern} onClick={onSelectPattern} />
146 |       ))}
147 |     </div>
148 |   </div>
149 | );
150 |
151 | export { patternData };
152 | export default ChartPatternLibrary;
153 |
```

FILE: frontend/src/components/CompanyResearch.jsx

281 lines | 11222 characters

```
1 | import React, { useState, useEffect } from 'react';
2 | import { Search, Building2, TrendingUp, BarChart3, Globe, Users, Loader2, ExternalLink, ChevronDown, ChevronUp, Star } from 'lucid ...
3 | import { Card } from './ui/card';
4 | import { Badge } from './ui/badge';
5 | import { Button } from './ui/button';
6 | import { Input } from './ui/input';
7 | import { researchCompany } from '../services/api';
8 |
9 | const formatMarketCap = (value) => {
10 |   if (!value) return 'N/A';
11 |   const num = parseFloat(value);
12 |   if (isNaN(num)) return value;
13 |   if (num >= 1e12) return `$$${(num / 1e12).toFixed(2)}T`;
14 |   if (num >= 1e9) return `$$${(num / 1e9).toFixed(2)}B`;
15 |   if (num >= 1e6) return `$$${(num / 1e6).toFixed(1)}M`;
16 |   return `$$${num.toLocaleString()}`;
17 | };
18 |
19 | const formatRevenue = (value) => formatMarketCap(value);
20 |
21 | const MetricCard = ({ label, value, prefix = '' }) => (
22 |   <div className="bg-slate-800/50 rounded-lg p-3">
23 |     <p className="text-slate-500 text-xs mb-1">{label}</p>
24 |     <p className="text-white font-semibold text-sm">{prefix}{value || 'N/A'}</p>
```

```
25 |   </div>
26 | );
27 |
28 | const SourceBadge = ({ source }) => (
29 |   <a
30 |     href={source.url || '#'}
31 |     target="_blank"
32 |     rel="noopener noreferrer"
33 |     className="inline-flex items-center gap-1 bg-slate-800 hover:bg-slate-700 text-slate-300 hover:text-white text-xs px-2 py-1 ro ...
34 |     data-testid={`source-${source.number}`}
35 |   >
36 |     <span className="text-[#0052FF] font-bold">[source.number]</span>
37 |     <span className="truncate max-w-[150px]">{source.source}</span>
38 |     <ExternalLink className="w-3 h-3 shrink-0 opacity-50" />
39 |   </a>
40 | );
41 |
42 | const ResearchCard = ({ data, compact = false }) => {
43 |   const [expanded, setExpanded] = useState(!compact);
44 |   const [inWatchlist, setInWatchlist] = useState(false);
45 |   const overview = data.overview || {};
46 |
47 |   useEffect(() => {
48 |     const saved = localStorage.getItem('risedualai_watchlist');
49 |     if (saved) {
50 |       const list = JSON.parse(saved);
51 |       setInWatchlist(list.some(item => item.symbol === data.symbol));
52 |     }
53 |   }, [data.symbol]);
54 |
55 |   const addToWatchlist = () => {
56 |     window.dispatchEvent(new CustomEvent('risedualai-add-watchlist', { detail: data.symbol }));
57 |     setInWatchlist(true);
58 |   };
59 |
60 |   return (
61 |     <Card className="bg-slate-800/30 border-slate-700/50 rounded-xl overflow-hidden" data-testid="research-card">
62 |       <div className="px-5 py-4 border-b border-slate-700/50 flex items-center justify-between">
63 |         <div className="flex items-center gap-3">
64 |           <div className="w-10 h-10 bg-[#0052FF] rounded-xl flex items-center justify-center">
65 |             <Building2 className="w-5 h-5 text-white" />
66 |           </div>
67 |           <div>
68 |             <div className="flex items-center gap-2">
69 |               <h3 className="text-white font-bold text-lg" style={{fontFamily: 'Manrope, sans-serif'}}>
70 |                 {data.company_name || data.symbol}
71 |               </h3>
72 |               <Badge className="bg-[#0052FF]/20 text-[#0052FF] border-0 text-xs">{data.symbol}</Badge>
73 |             </div>
74 |             <p className="text-slate-500 text-xs">
75 |               {overview.sector || 'Technology'} &middot; {overview.exchange || 'NYSE'}
76 |               {overview.employees && ` &middot; ${parseInt(overview.employees).toLocaleString()} employees`
77 |             </p>
78 |           </div>
79 |         </div>
80 |         <div className="flex items-center gap-2">
81 |           <Button
82 |             variant="ghost"
83 |             size="sm"
84 |             onClick={addToWatchlist}
85 |             disabled={inWatchlist}
86 |             className={inWatchlist ? "text-yellow-500" : "text-slate-400 hover:text-yellow-500"}
87 |             data-testid="add-to-watchlist-btn"
88 |           >
89 |             <Star className={`w-4 h-4 ${inWatchlist ? 'fill-yellow-500' : ''}`} />
90 |           </Button>
91 |         </div>

```

```
92 |         {compact && (
93 |             <Button
94 |                 variant="ghost"
95 |                 size="sm"
96 |                 onClick={() => setExpanded(!expanded)}
97 |                 className="text-slate-400 hover:text-white"
98 |                 data-testid="expand-research"
99 |             >
100 |                 {expanded ? <ChevronUp className="w-4 h-4" /> : <ChevronDown className="w-4 h-4" />}
101 |             </Button>
102 |         )}
103 |     </div>
104 | </div>
105 |
106 |     {expanded && (
107 |         <>
108 |             {/* Key Metrics Grid */}
109 |             {overview.market_cap && (
110 |                 <div className="px-5 py-3 border-b border-slate-700/50">
111 |                     <div className="grid grid-cols-4 gap-2">
112 |                         <MetricCard label="Market Cap" value={formatMarketCap(overview.market_cap)} />
113 |                         <MetricCard label="P/E Ratio" value={overview.pe_ratio} />
114 |                         <MetricCard label="EPS" value={overview.eps} prefix="$" />
115 |                         <MetricCard label="Revenue (TTM)" value={formatRevenue(overview.revenue_ttm)} />
116 |                     </div>
117 |                     <div className="grid grid-cols-4 gap-2 mt-2">
118 |                         <MetricCard label="52W High" value={overview.week_52_high} prefix="$" />
119 |                         <MetricCard label="52W Low" value={overview.week_52_low} prefix="$" />
120 |                         <MetricCard label="Div Yield" value={overview.dividend_yield ? `${parseFloat(overview.dividend_yield) * 100}.toFi ...
121 |                         <MetricCard label="Beta" value={overview.beta} />
122 |                     </div>
123 |                 </div>
124 |             )}
125 |
126 |             {/* AI Synthesis */}
127 |             <div className="px-5 py-4">
128 |                 <div className="flex items-center gap-2 mb-3">
129 |                     <div className="w-2 h-2 rounded-full bg-[#0052FF] animate-pulse" />
130 |                     <p className="text-xs font-bold text-[#0052FF] uppercase tracking-wider" style={{fontFamily: 'Manrope, sans-serif'}}>
131 |                         AI Research Summary
132 |                     </p>
133 |                 </div>
134 |                 <div className="prose prose-sm prose-invert max-w-none text-slate-300 text-sm leading-relaxed research-content">
135 |                     {data.synthesis?.split('\n').map((line, i) => {
136 |                         if (line.startsWith('## ')) return <h3 key={i} className="text-white font-bold text-base mt-3 mb-2" style={{fontFa ...
137 |                         if (line.startsWith('***') && line.includes('***:')) {
138 |                             const parts = line.split('***:');
139 |                             const label = parts[0].replace(/\*\*/g, '');
140 |                             const value = parts.slice(1).join('***:');
141 |                             return <p key={i} className="mb-2"><strong className="text-white">{label}</strong>{value}</p>;
142 |                         }
143 |                         if (line.startsWith('- ')) return <p key={i} className="ml-4 mb-1 text-slate-400">{line}</p>;
144 |                         if (line.trim() === '') return <br key={i} />;
145 |                         return <p key={i} className="mb-2">{line}</p>;
146 |                     })}
147 |                 </div>
148 |             </div>
149 |
150 |             {/* Sources */}
151 |             {data.sources && data.sources.length > 0 && (
152 |                 <div className="px-5 py-3 border-t border-slate-700/50 bg-slate-900/30">
153 |                     <p className="text-xs text-slate-500 mb-2 font-medium">Sources ({data.sources.length})</p>
154 |                     <div className="flex flex-wrap gap-2">
155 |                         {data.sources.map((source) => (
156 |                             <SourceBadge key={source.number} source={source} />
157 |                         ))}
158 |                     </div>
```

```
159 |         </div>
160 |     )}
161 | </>
162 | )}
163 | </Card>
164 | );
165 | };
166 |
167 | // Standalone Research Section for the main page
168 | const CompanyResearch = () => {
169 |     const [symbol, setSymbol] = useState('');
170 |     const [research, setResearch] = useState(null);
171 |     const [isLoading, setIsLoading] = useState(false);
172 |     const [error, setError] = useState(null);
173 |
174 |     const handleSearch = async (e) => {
175 |         e?.preventDefault();
176 |         if (!symbol.trim()) return;
177 |         doSearch(symbol.trim().toUpperCase());
178 |     };
179 |
180 |     const doSearch = async (ticker) => {
181 |         setIsLoading(true);
182 |         setError(null);
183 |         setResearch(null);
184 |         setSymbol(ticker);
185 |
186 |         try {
187 |             const data = await researchCompany(ticker);
188 |             setResearch(data);
189 |         } catch (err) {
190 |             setError('Could not find data for this symbol. Please try another ticker.');
```

191 | } finally {

192 | setIsLoading(false);

193 | }

194 | };
195 |
196 | useEffect(() => {
197 | const handleNavSearch = (e) => {
198 | if (e.detail) doSearch(e.detail);
199 | };
200 | window.addEventListener('risedualai-research', handleNavSearch);
201 | return () => window.removeEventListener('risedualai-research', handleNavSearch);
202 | }, []);
203 |
204 | const popularTickers = ['AAPL', 'MSFT', 'GOOGL', 'AMZN', 'TSLA', 'NVDA'];
205 |
206 | return (
207 | <div className="space-y-6" data-testid="research-section">
208 | <div className="flex items-center justify-between">
209 | <div className="flex items-center gap-3">
210 | <div className="w-10 h-10 bg-[#0052FF] rounded-xl flex items-center justify-center">
211 | <Globe className="w-5 h-5 text-white" />
212 | </div>
213 | <div>
214 | <h2 className="text-white text-xl font-bold" style={{fontFamily:'Manrope,sans-serif'}}>Company Research</h2>
215 | <p className="text-slate-500 text-xs">Perplexity-style AI research with cited sources</p>
216 | </div>
217 | </div>
218 | </div>
219 | </div>
220 |
221 | <div>
222 | <div>
223 | <div className="flex gap-3">
224 | <div className="relative flex-1">
225 | <Search className="absolute left-3 top-1/2 -translate-y-1/2 text-slate-500 w-4 h-4" />

```
226 |         type="text"
227 |         value={symbol}
228 |         onChange={(e) => setSymbol(e.target.value.toUpperCase())}
229 |         placeholder="Enter ticker symbol (e.g. AAPL, TSLA, MSFT)..."
230 |         className="pl-10 bg-slate-800/50 border-slate-700/50 text-white placeholder-slate-500 rounded-xl"
231 |         data-testid="research-input"
232 |     />
233 | </div>
234 | <Button
235 |     type="submit"
236 |     disabled={isLoading || !symbol.trim()}
237 |     className="bg-[#0052FF] hover:bg-[#2563EB] text-white rounded-xl px-6"
238 |     data-testid="research-search-btn"
239 |     >
240 |     {isLoading ? <Loader2 className="w-4 h-4 animate-spin" /> : 'Research'}
241 | </Button>
242 | </form>
243 |
244 | {/* Quick Tickers */}
245 | <div className="flex gap-2 flex-wrap">
246 |     {popularTickers.map((ticker) => (
247 |         <button
248 |             key={ticker}
249 |             onClick={() => { setSymbol(ticker); }}
250 |             className="text-xs px-3 py-1.5 rounded-full border border-slate-700/50 text-slate-400 hover:text-[#0052FF] hover:borde ...
251 |             data-testid={`quick-ticker-${ticker}`}
252 |             >
253 |             {ticker}
254 |         </button>
255 |     ))}
256 | </div>
257 |
258 | {/* Loading State */}
259 | {isLoading && (
260 |     <div className="flex flex-col items-center justify-center py-12 gap-3">
261 |         <Loader2 className="w-8 h-8 text-[#0052FF] animate-spin" />
262 |         <p className="text-slate-400 text-sm">Researching {symbol}... Gathering data from multiple sources</p>
263 |     </div>
264 | )}
265 |
266 | {/* Error */}
267 | {error && (
268 |     <div className="bg-red-500/10 border border-red-500/30 rounded-xl p-4 text-red-400 text-sm">
269 |         {error}
270 |     </div>
271 | )}
272 |
273 | {/* Results */}
274 | {research && <ResearchCard data={research} />}
275 | </div>
276 | );
277 | };
278 |
279 | export { ResearchCard };
280 | export default CompanyResearch;
281 |
```

FILE: frontend/src/components/CryptoSection.jsx

129 lines | 5169 characters

```
1 | import React, { useState, useEffect } from 'react';
2 | import { Bitcoin, TrendingUp, TrendingDown, DollarSign } from 'lucide-react';
3 | import { Card } from './ui/card';
4 |
5 | const BACKEND_URL = process.env.REACT_APP_BACKEND_URL;
6 |
```

```
7 | const CryptoSection = () => {
8 |   const [cryptos, setCryptos] = useState([]);
9 |   const [loading, setLoading] = useState(true);
10 |
11 |   useEffect(() => {
12 |     fetchCryptoData();
13 |     const interval = setInterval(fetchCryptoData, 60000);
14 |     return () => clearInterval(interval);
15 |   }, []);
16 |
17 |   const fetchCryptoData = async () => {
18 |     try {
19 |       const response = await fetch(`${BACKEND_URL}/api/crypto/prices`);
20 |       if (response.ok) {
21 |         const data = await response.json();
22 |         setCryptos(data);
23 |       }
24 |     } catch (error) {
25 |       console.error('Error fetching crypto data:', error);
26 |     } finally {
27 |       setLoading(false);
28 |     }
29 |   };
30 |
31 |   if (loading) {
32 |     return (
33 |       <div className="bg-[#0F172A] rounded-xl border border-slate-700/50 p-6">
34 |         <div className="text-slate-400">Loading crypto data...</div>
35 |       </div>
36 |     );
37 |   }
38 |
39 |   return (
40 |     <div className="space-y-6 mt-8">
41 |       <div className="flex items-center gap-3">
42 |         <div className="w-10 h-10 bg-orange-500 rounded-lg flex items-center justify-center">
43 |           <Bitcoin className="w-6 h-6 text-white" />
44 |         </div>
45 |         <div>
46 |           <h2 className="text-white text-2xl font-bold">Cryptocurrency Market</h2>
47 |           <p className="text-slate-400 text-sm">Real-time cryptocurrency prices and market data</p>
48 |         </div>
49 |       </div>
50 |
51 |       <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 xl:grid-cols-4 gap-4">
52 |         {cryptos.map((crypto) => (
53 |           <Card key={crypto.symbol} className="bg-[#0F172A] border-slate-700 p-4 hover:border-slate-600 transition-colors">
54 |             <div className="flex items-start justify-between mb-3">
55 |               <div className="flex items-center gap-2">
56 |                 <div className="w-10 h-10 bg-gradient-to-br from-orange-500 to-orange-600 rounded-full flex items-center justify-c ...
57 |                 <span className="text-white font-bold text-sm">{crypto.symbol.substring(0, 2)}</span>
58 |               </div>
59 |               <div>
60 |                 <h3 className="text-white font-semibold">{crypto.symbol}</h3>
61 |                 <p className="text-slate-500 text-xs">{crypto.market}</p>
62 |               </div>
63 |             </div>
64 |             <div>
65 |               {crypto.changePercent >= 0 ? (
66 |                 <TrendingUp className="w-5 h-5 text-emerald-400" />
67 |               ) : (
68 |                 <TrendingDown className="w-5 h-5 text-red-400" />
69 |               )}
70 |             </div>
71 |           </div>
72 |         )}
73 |       </div>
```

```
74 |         <div>
75 |             <p className="text-slate-500 text-xs mb-1">Price</p>
76 |             <p className="text-white text-2xl font-bold">${crypto.price.toLocaleString()}</p>
77 |         </div>
78 |
79 |         <div className="flex items-center justify-between pt-2 border-t border-slate-700">
80 |             <span className="text-slate-500 text-xs">24h Change</span>
81 |             <span className={`text-sm font-medium ${
82 |                 crypto.changePercent >= 0 ? 'text-emerald-400' : 'text-red-400'
83 |             }`}>
84 |                 {crypto.changePercent >= 0 ? '+' : ''}{crypto.changePercent.toFixed(2)}%
85 |             </span>
86 |         </div>
87 |
88 |         {crypto.change !== 0 && (
89 |             <div className="flex items-center justify-between">
90 |                 <span className="text-slate-500 text-xs">24h Value</span>
91 |                 <span className={`text-sm font-medium ${
92 |                     crypto.change >= 0 ? 'text-emerald-400' : 'text-red-400'
93 |                 }`}>
94 |                     {crypto.change >= 0 ? '+' : ''}${Math.abs(crypto.change).toFixed(2)}
95 |                 </span>
96 |             </div>
97 |         )}
98 |     </div>
99 | </Card>
100 | )}}
101 | </div>
102 |
103 | { /* Market Info */ }
104 | <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
105 |     <div className="bg-slate-800/50 border border-slate-700/50 rounded-xl p-4">
106 |         <div className="flex items-center gap-2 mb-2">
107 |             <DollarSign className="w-5 h-5 text-emerald-400" />
108 |             <h3 className="text-white font-semibold">Market Overview</h3>
109 |         </div>
110 |         <p className="text-slate-400 text-sm">
111 |             Track the top cryptocurrencies with real-time price updates. Data refreshes every minute to keep you informed.
112 |         </p>
113 |     </div>
114 |
115 |     <div className="bg-slate-800/50 border border-slate-700/50 rounded-xl p-4">
116 |         <div className="flex items-center gap-2 mb-2">
117 |             <Bitcoin className="w-5 h-5 text-orange-400" />
118 |             <h3 className="text-white font-semibold">Trading Insights</h3>
119 |         </div>
120 |         <p className="text-slate-400 text-sm">
121 |             Monitor price movements and percentage changes to identify trading opportunities in the volatile crypto market.
122 |         </p>
123 |     </div>
124 | </div>
125 | </div>
126 | );
127 | };
128 |
129 | export default CryptoSection;
```

FILE: frontend/src/components/CryptoTicker.jsx

77 lines | 2752 characters

```
1 | import React, { useState, useEffect } from 'react';
2 | import { TrendingUp, TrendingDown, Activity } from 'lucide-react';
3 | import { Card } from './ui/card';
4 |
5 | const BACKEND_URL = process.env.REACT_APP_BACKEND_URL;
6 |
```



```
7 | const CryptoTicker = () => {
8 |   const [cryptos, setCryptos] = useState([]);
9 |   const [loading, setLoading] = useState(true);
10 |
11 |   useEffect(() => {
12 |     fetchCryptoData();
13 |     const interval = setInterval(fetchCryptoData, 60000); // Update every minute
14 |     return () => clearInterval(interval);
15 |   }, []);
16 |
17 |   const fetchCryptoData = async () => {
18 |     try {
19 |       const response = await fetch(`${BACKEND_URL}/api/crypto/prices`);
20 |       if (response.ok) {
21 |         const data = await response.json();
22 |         setCryptos(data);
23 |       }
24 |     } catch (error) {
25 |       console.error('Error fetching crypto data:', error);
26 |     } finally {
27 |       setLoading(false);
28 |     }
29 |   };
30 |
31 |   if (loading) {
32 |     return (
33 |       <div className="bg-[#0F172A] border-b border-slate-700 py-4 px-6">
34 |         <div className="text-slate-400 text-sm">Loading crypto data...</div>
35 |       </div>
36 |     );
37 |   }
38 |
39 |   return (
40 |     <div className="bg-[#0F172A] border-b border-slate-700">
41 |       <div className="overflow-x-auto">
42 |         <div className="flex gap-6 px-6 py-3 min-w-max">
43 |           {cryptos.map((crypto) => (
44 |             <div key={crypto.symbol} className="flex items-center gap-3">
45 |               <div className="flex items-center gap-2">
46 |                 <div className="w-8 h-8 bg-orange-500 rounded-full flex items-center justify-center">
47 |                   <span className="text-white text-xs font-bold">{crypto.symbol.substring(0, 2)}</span>
48 |                 </div>
49 |                 <div>
50 |                   <div className="flex items-center gap-2">
51 |                     <span className="text-white font-medium text-sm">{crypto.symbol}</span>
52 |                     <span className="text-slate-400 text-xs">/USD</span>
53 |                   </div>
54 |                   <div className="flex items-center gap-2">
55 |                     <span className="text-white text-sm">${crypto.price.toLocaleString()}</span>
56 |                     <span className={`text-xs flex items-center gap-1 ${
57 |                       crypto.changePercent >= 0 ? 'text-emerald-400' : 'text-red-400'
58 |                     }`}>
59 |                       {crypto.changePercent >= 0 ? (
60 |                         <TrendingUp className="w-3 h-3" />
61 |                       ) : (
62 |                         <TrendingDown className="w-3 h-3" />
63 |                       )}
64 |                       {Math.abs(crypto.changePercent).toFixed(2)}%
65 |                     </span>
66 |                   </div>
67 |                 </div>
68 |               </div>
69 |             </div>
70 |           )})
71 |         </div>
72 |       </div>
73 |     </div>
```

```
74 |   });
75 | };
76 |
77 | export default CryptoTicker;
```

FILE: frontend/src/components/DarkPoolData.jsx

120 lines | 4552 characters

```
1 | import React, { useState, useEffect } from 'react';
2 | import { Eye, TrendingUp, Activity } from 'lucide-react';
3 | import DataTable from '../DataTable';
4 | import ProBlurWall from '../ProBlurWall';
5 |
6 | const BACKEND_URL = process.env.REACT_APP_BACKEND_URL;
7 |
8 | const DarkPoolData = ({ onSubscribe }) => {
9 |   const [darkPoolData, setDarkPoolData] = useState([]);
10 |   const [loading, setLoading] = useState(true);
11 |
12 |   useEffect(() => {
13 |     fetchDarkPoolData();
14 |     const interval = setInterval(fetchDarkPoolData, 120000); // Update every 2 minutes
15 |     return () => clearInterval(interval);
16 |   }, []);
17 |
18 |   const fetchDarkPoolData = async () => {
19 |     try {
20 |       const response = await fetch(`${BACKEND_URL}/api/dark-pool`);
21 |       if (response.ok) {
22 |         const data = await response.json();
23 |         setDarkPoolData(data);
24 |       }
25 |     } catch (error) {
26 |       console.error('Error fetching dark pool data:', error);
27 |     } finally {
28 |       setLoading(false);
29 |     }
30 |   };
31 |
32 |   const columns = [
33 |     { key: 'symbol', label: 'Symbol', sortable: true },
34 |     { key: 'darkPoolVolume', label: 'Dark Pool Vol', sortable: true },
35 |     { key: 'totalVolume', label: 'Total Volume', sortable: true },
36 |     { key: 'darkPoolPercent', label: 'Dark Pool %', sortable: true },
37 |     { key: 'averagePrice', label: 'Avg Price', sortable: true },
38 |     { key: 'priceChange', label: 'Price Change', sortable: true },
39 |     { key: 'sentiment', label: 'Sentiment', sortable: true },
40 |   ];
41 |
42 |   // Format data for DataTable
43 |   const formattedData = darkPoolData.map(item => ({
44 |     symbol: item.symbol,
45 |     darkPoolVolume: item.darkPoolVolume.toLocaleString(),
46 |     totalVolume: item.totalVolume.toLocaleString(),
47 |     darkPoolPercent: `${item.darkPoolPercent}%`,
48 |     averagePrice: `$$${item.averagePrice}`,
49 |     priceChange: item.priceChange >= 0 ? `+${item.priceChange}%` : `-${item.priceChange}%`,
50 |     sentiment: item.sentiment,
51 |   }));
52 |
53 |   if (loading) {
54 |     return (
55 |       <div className="bg-[#0F172A] rounded-xl border border-slate-700/50 p-6">
56 |         <div className="text-slate-400">Loading dark pool data...</div>
57 |       </div>
58 |     );
```

```
59 |   }
60 |
61 |   return (
62 |     <div className="space-y-6 mt-8">
63 |       {/* Header */}
64 |       <div className="flex items-center gap-3">
65 |         <div className="w-10 h-10 bg-purple-600 rounded-lg flex items-center justify-center">
66 |           <Eye className="w-6 h-6 text-white" />
67 |         </div>
68 |         <div>
69 |           <h2 className="text-white text-2xl font-bold">Dark Pool Trading</h2>
70 |           <p className="text-slate-400 text-sm">Off-exchange institutional trading activity</p>
71 |         </div>
72 |       </div>
73 |
74 |       {/* Dark Pool Table */}
75 |       <ProBlurWall freeRowCount={3} onSubscribe={onSubscribe} label="Dark Pool Data">
76 |         <DataTable
77 |           title="Dark Pool Activity"
78 |           subtitle="Real-time dark pool trading data showing institutional buying and selling patterns"
79 |           columns={columns}
80 |           data={formattedData}
81 |         />
82 |       </ProBlurWall>
83 |
84 |       {/* Info Cards */}
85 |       <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
86 |         <div className="bg-slate-800/50 border border-slate-700/50 rounded-xl p-4">
87 |           <div className="flex items-center gap-2 mb-2">
88 |             <Activity className="w-5 h-5 text-purple-400" />
89 |             <h3 className="text-white font-semibold">What is Dark Pool?</h3>
90 |           </div>
91 |           <p className="text-slate-400 text-sm">
92 |             Dark pools are private exchanges where institutional investors trade large blocks of securities anonymously.
93 |           </p>
94 |         </div>
95 |
96 |         <div className="bg-slate-800/50 border border-slate-700/50 rounded-xl p-4">
97 |           <div className="flex items-center gap-2 mb-2">
98 |             <TrendingUp className="w-5 h-5 text-emerald-400" />
99 |             <h3 className="text-white font-semibold">Why It Matters</h3>
100 |           </div>
101 |           <p className="text-slate-400 text-sm">
102 |             High dark pool activity can indicate institutional interest and potential price movements in securities.
103 |           </p>
104 |         </div>
105 |
106 |         <div className="bg-slate-800/50 border border-slate-700/50 rounded-xl p-4">
107 |           <div className="flex items-center gap-2 mb-2">
108 |             <Eye className="w-5 h-5 text-blue-400" />
109 |             <h3 className="text-white font-semibold">How to Use</h3>
110 |           </div>
111 |           <p className="text-slate-400 text-sm">
112 |             Monitor dark pool percentage - higher values suggest significant institutional positioning.
113 |           </p>
114 |         </div>
115 |       </div>
116 |     </div>
117 |   );
118 | };
119 |
120 | export default DarkPoolData;
```

FILE: frontend/src/components/DataTable.jsx

188 lines | 7081 characters

```
1 | import React, { useState } from 'react';
2 | import { ThumbsUp, ThumbsDown, HelpCircle, Download } from 'lucide-react';
3 | import { Button } from './ui/button';
4 | import FilterPanel from './FilterPanel';
5 | import QuickTrade from './QuickTrade';
6 |
7 | const DataTable = ({ title, subtitle, columns, data, showLikes = true, showFilters = false, showTrading = true }) => {
8 |   const [filteredData, setFilteredData] = useState(data);
9 |   const [filters, setFilters] = useState(null);
10 |
11 |   const getPowerColor = (power) => {
12 |     if (power >= 80) return 'bg-emerald-500';
13 |     if (power >= 60) return 'bg-yellow-500';
14 |     return 'bg-orange-500';
15 |   };
16 |
17 |   const getPowerBarWidth = (power) => {
18 |     return `${power}%`;
19 |   };
20 |
21 |   const exportToCSV = () => {
22 |     const csvData = filteredData.map(row => {
23 |       return columns.map(col => row[col.key]).join(',');
24 |     });
25 |     const headers = columns.map(col => col.label).join(',');
26 |     const csv = [headers, ...csvData].join('\n');
27 |
28 |     const blob = new Blob([csv], { type: 'text/csv' });
29 |     const url = window.URL.createObjectURL(blob);
30 |     const a = document.createElement('a');
31 |     a.href = url;
32 |     a.download = `${title.replace(/\s+/g, '_').toLowerCase()}_${new Date().toISOString().split('T')[0]}.csv`;
33 |     document.body.appendChild(a);
34 |     a.click();
35 |     document.body.removeChild(a);
36 |     window.URL.revokeObjectURL(url);
37 |   };
38 |
39 |   const handleFilterChange = (newFilters) => {
40 |     setFilters(newFilters);
41 |     // Apply filters to data
42 |     let filtered = [...data];
43 |
44 |     if (newFilters.ivRankMin || newFilters.ivRankMax !== 100) {
45 |       filtered = filtered.filter(row =>
46 |         row.ivRank >= newFilters.ivRankMin && row.ivRank <= newFilters.ivRankMax
47 |       );
48 |     }
49 |
50 |     if (newFilters.powerMin > 0) {
51 |       filtered = filtered.filter(row => row.power >= newFilters.powerMin);
52 |     }
53 |
54 |     setFilteredData(filtered);
55 |   };
56 |
57 |   return (
58 |     <div className="bg-slate-800/50 rounded-xl border border-slate-700/50 p-6">
59 |       {/* Header */}
60 |       <div className="flex items-center justify-between mb-4">
61 |         <div>
62 |           <div className="flex items-center gap-2">
63 |             <h3 className="text-white text-lg font-semibold">{title}</h3>
64 |             <HelpCircle className="w-4 h-4 text-slate-500" />
65 |           </div>
66 |           {subtitle && (
67 |             <p className="text-slate-500 text-sm mt-1">{subtitle}</p>
```

```
68 |     )}
69 |   </div>
70 |   <div className="flex items-center gap-2">
71 |     {showFilters && <FilterPanel onFilterChange={handleFilterChange} />}
72 |     <Button
73 |       onClick={exportToCSV}
74 |       variant="outline"
75 |       size="sm"
76 |       className="bg-[#1E293B] border-slate-600 text-white hover:bg-slate-700"
77 |     >
78 |       <Download className="w-4 h-4 mr-2" />
79 |       Export
80 |     </Button>
81 |     {showLikes && (
82 |       <>
83 |         <button
84 |           onClick={() => alert('Thanks for your feedback!')}
85 |           className="text-slate-500 hover:text-emerald-400 transition-colors"
86 |         >
87 |           <ThumbsUp className="w-5 h-5" />
88 |         </button>
89 |         <button
90 |           onClick={() => alert('Thanks for your feedback! We\'ll improve this.')}
91 |           className="text-slate-500 hover:text-red-400 transition-colors"
92 |         >
93 |           <ThumbsDown className="w-5 h-5" />
94 |         </button>
95 |       </>
96 |     )}
97 |   </div>
98 | </div>
99 |
100 | {/* Table */}
101 | <div className="overflow-x-auto">
102 |   <table className="w-full">
103 |     <thead>
104 |       <tr className="border-b border-slate-700">
105 |         {columns.map((col, index) => (
106 |           <th
107 |             key={index}
108 |             className="text-left py-3 px-3 text-slate-400 text-xs font-medium uppercase tracking-wider"
109 |           >
110 |             <div className="flex items-center gap-1">
111 |               {col.label}
112 |               {col.sortable && <HelpCircle className="w-3 h-3" />}
113 |             </div>
114 |           </th>
115 |         ))}
116 |         {showTrading && (
117 |           <th className="text-left py-3 px-3 text-slate-400 text-xs font-medium uppercase tracking-wider">
118 |             QUICK TRADE
119 |           </th>
120 |         )}
121 |       </tr>
122 |     </thead>
123 |     <tbody>
124 |       {filteredData.map((row, rowIndex) => (
125 |         <tr
126 |           key={rowIndex}
127 |           className="border-b border-slate-700 hover:bg-slate-700 transition-colors"
128 |         >
129 |           {columns.map((col, colIndex) => (
130 |             <td key={colIndex} className="py-3 px-3 text-sm">
131 |               {col.key === 'contract' ? (
132 |                 <span
133 |                   onClick={() => alert(`Viewing details for ${row[col.key]}\n\nThis will show:\n- Detailed contract info\n- ...
134 |                 className="text-blue-400 font-medium hover:underline cursor-pointer"
```

```
135 |         >
136 |         {row[col.key]}
137 |     </span>
138 |     ) : col.key === 'power' ? (
139 |         <div className="flex items-center gap-2">
140 |             <div className="w-12 h-2 bg-slate-700 rounded-full overflow-hidden">
141 |                 <div
142 |                     className={`h-full ${getPowerColor(row[col.key])}` }
143 |                     style={{ width: getPowerBarWidth(row[col.key]) }}
144 |                 />
145 |             </div>
146 |             <span className="text-slate-300 text-xs">{row[col.key]}%</span>
147 |         </div>
148 |     ) : col.key === 'returns' ? (
149 |         <span className="text-emerald-400">{row[col.key]}</span>
150 |     ) : col.key === 'sentiment' ? (
151 |         <span className="text-red-400">{row[col.key]}</span>
152 |     ) : col.key === 'aiScore' ? (
153 |         <span className={row[col.key] >= 50 ? 'text-emerald-400' : 'text-orange-400'}>
154 |             {row[col.key]}
155 |         </span>
156 |     ) : (
157 |         <span className="text-slate-300">{row[col.key]}</span>
158 |     )
159 | </td>
160 | )}
161 | {showTrading && (
162 |     <td className="py-3 px-3">
163 |         <QuickTrade symbol={row.contract} />
164 |     </td>
165 | )}
166 | </tr>
167 | )}
168 | </tbody>
169 | </table>
170 | </div>
171 |
172 | { /* Results Info */ }
173 | <div className="mt-4 flex items-center justify-between">
174 |     <p className="text-slate-500 text-sm">
175 |         Showing {filteredData.length} of {data.length} results
176 |     </p>
177 |     <button
178 |         onClick={() => alert('Full data view coming soon!')}
179 |         className="text-blue-400 hover:text-blue-300 text-sm font-medium transition-colors"
180 |     >
181 |         See more ?
182 |     </button>
183 | </div>
184 | </div>
185 | );
186 | };
187 |
188 | export default DataTable;
```

FILE: frontend/src/components/FilterPanel.jsx

164 lines | 5818 characters

```
1 | import React, { useState } from 'react';
2 | import { Filter, X, RotateCcw } from 'lucide-react';
3 | import { Button } from './ui/button';
4 | import { Card } from './ui/card';
5 | import { Label } from './ui/label';
6 | import { Slider } from './ui/slider';
7 | import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from './ui/select';
8 |
```

```
9 | const FilterPanel = ({ onFilterChange }) => {
10 |   const [isOpen, setIsOpen] = useState(false);
11 |   const [filters, setFilters] = useState({
12 |     ivRankMin: 0,
13 |     ivRankMax: 100,
14 |     powerMin: 0,
15 |     volumeMin: 0,
16 |     sentiment: 'all',
17 |     timeframe: 'today'
18 |   });
19 |
20 |   const handleFilterChange = (key, value) => {
21 |     const updated = { ...filters, [key]: value };
22 |     setFilters(updated);
23 |     onFilterChange(updated);
24 |   };
25 |
26 |   const resetFilters = () => {
27 |     const defaultFilters = {
28 |       ivRankMin: 0,
29 |       ivRankMax: 100,
30 |       powerMin: 0,
31 |       volumeMin: 0,
32 |       sentiment: 'all',
33 |       timeframe: 'today'
34 |     };
35 |     setFilters(defaultFilters);
36 |     onFilterChange(defaultFilters);
37 |   };
38 |
39 |   return (
40 |     <>
41 |       <Button
42 |         onClick={() => setIsOpen(!isOpen)}
43 |         variant="outline"
44 |         className="bg-[#1E293B] border-slate-600 text-white hover:bg-slate-700"
45 |       >
46 |         <Filter className="w-4 h-4 mr-2" />
47 |         Filters
48 |       </Button>
49 |
50 |       {isOpen && (
51 |         <div className="fixed inset-0 bg-black bg-opacity-50 z-50 flex items-center justify-center">
52 |           <Card className="bg-slate-800/50 border-slate-700/50 rounded-xl p-6 w-full max-w-md">
53 |             {/* Header */}
54 |             <div className="flex items-center justify-between mb-6">
55 |               <h3 className="text-white text-lg font-semibold">Filter Options</h3>
56 |               <button
57 |                 onClick={() => setIsOpen(false)}
58 |                 className="text-slate-400 hover:text-slate-50"
59 |               >
60 |                 <X className="w-5 h-5" />
61 |               </button>
62 |             </div>
63 |
64 |             {/* Filter Controls */}
65 |             <div className="space-y-6">
66 |               {/* IV Rank Range */}
67 |               <div>
68 |                 <Label className="text-white mb-2 block">
69 |                   IV Rank: {filters.ivRankMin}% - {filters.ivRankMax}%
70 |                 </Label>
71 |                 <div className="space-y-2">
72 |                   <Slider
73 |                     value={[filters.ivRankMin]}
74 |                     onChange={([value]) => handleFilterChange('ivRankMin', value)}
75 |                     max={100}
```

```
76 |         step={5}
77 |         className="w-full"
78 |     />
79 |     <Slider
80 |         value={{filters.ivRankMax}}
81 |         onChange={{(value) => handleFilterChange('ivRankMax', value)}}
82 |         max={100}
83 |         step={5}
84 |         className="w-full"
85 |     />
86 | </div>
87 | </div>
88 |
89 | {/* Power Minimum */}
90 | <div>
91 |     <Label className="text-white mb-2 block">
92 |         Minimum Power: {filters.powerMin}%
93 |     </Label>
94 |     <Slider
95 |         value={{filters.powerMin}}
96 |         onChange={{(value) => handleFilterChange('powerMin', value)}}
97 |         max={100}
98 |         step={10}
99 |         className="w-full"
100 |     />
101 | </div>
102 |
103 | {/* Sentiment */}
104 | <div>
105 |     <Label className="text-white mb-2 block">Sentiment</Label>
106 |     <Select
107 |         value={filters.sentiment}
108 |         onChange={{(value) => handleFilterChange('sentiment', value)}}
109 |     >
110 |         <SelectTrigger className="bg-[#1E293B] border-slate-600 text-white">
111 |             <SelectValue />
112 |         </SelectTrigger>
113 |         <SelectContent className="bg-slate-800 border-slate-700">
114 |             <SelectItem value="all" className="text-white">All</SelectItem>
115 |             <SelectItem value="bullish" className="text-white">Bullish</SelectItem>
116 |             <SelectItem value="bearish" className="text-white">Bearish</SelectItem>
117 |         </SelectContent>
118 |     </Select>
119 | </div>
120 |
121 | {/* Timeframe */}
122 | <div>
123 |     <Label className="text-white mb-2 block">Timeframe</Label>
124 |     <Select
125 |         value={filters.timeframe}
126 |         onChange={{(value) => handleFilterChange('timeframe', value)}}
127 |     >
128 |         <SelectTrigger className="bg-[#1E293B] border-slate-600 text-white">
129 |             <SelectValue />
130 |         </SelectTrigger>
131 |         <SelectContent className="bg-slate-800 border-slate-700">
132 |             <SelectItem value="today" className="text-white">Today</SelectItem>
133 |             <SelectItem value="week" className="text-white">This Week</SelectItem>
134 |             <SelectItem value="month" className="text-white">This Month</SelectItem>
135 |         </SelectContent>
136 |     </Select>
137 | </div>
138 | </div>
139 |
140 | {/* Actions */}
141 | <div className="flex gap-3 mt-6">
142 |     <Button
```



```
143 |         onClick={resetFilters}
144 |         variant="outline"
145 |         className="flex-1 bg-[#1E293B] border-slate-600 text-white hover:bg-slate-700"
146 |     >
147 |         <RotateCcw className="w-4 h-4 mr-2" />
148 |         Reset
149 |     </Button>
150 |     <Button
151 |         onClick={() => setIsOpen(false)}
152 |         className="flex-1 bg-[#0052FF] hover:bg-[#2563EB]"
153 |     >
154 |         Apply Filters
155 |     </Button>
156 | </div>
157 | </Card>
158 | </div>
159 | })
160 | </>
161 | );
162 | };
163 |
164 | export default FilterPanel;
```

FILE: frontend/src/components/MacroDashboard.jsx

539 lines | 25085 characters

```
1 | import React, { useState, useEffect, useCallback, useRef } from 'react';
2 | import { Globe, BarChart3, Landmark, RefreshCw, AlertTriangle, TrendingUp, TrendingDown, ChevronRight, Clock, Zap, Shield, Radio } ...
3 | import { Card } from './ui/card';
4 | import { Badge } from './ui/badge';
5 | import { Button } from './ui/button';
6 | import ProBlurWall from './ProBlurWall';
7 | import { useAuth } from '../contexts/AuthContext';
8 |
9 | const BACKEND_URL = process.env.REACT_APP_BACKEND_URL;
10 |
11 | const AUTO_REFRESH_MS = 30000; // 30 seconds
12 |
13 | const TABS = [
14 |   { id: 'world', label: 'World Events', icon: Globe },
15 |   { id: 'markets', label: 'Foreign Markets', icon: BarChart3 },
16 |   { id: 'congress', label: 'Congress Trades', icon: Landmark },
17 | ];
18 |
19 | const MacroDashboard = ({ onSubscribe }) => {
20 |   const { isPro } = useAuth();
21 |   const [activeTab, setActiveTab] = useState('world');
22 |   const [worldEvents, setWorldEvents] = useState(null);
23 |   const [foreignMarkets, setForeignMarkets] = useState(null);
24 |   const [govFilings, setGovFilings] = useState(null);
25 |   const [loading, setLoading] = useState({});
26 |   const [autoRefresh, setAutoRefresh] = useState(true);
27 |   const [lastRefresh, setLastRefresh] = useState(null);
28 |   const [changedSymbols, setChangedSymbols] = useState(new Set());
29 |   const prevMarketsRef = useRef(null);
30 |
31 |   const fetchData = useCallback(async (endpoint, setter, key) => {
32 |     setLoading(prev => ({ ...prev, [key]: true }));
33 |     try {
34 |       const res = await fetch(`${BACKEND_URL}/api/${endpoint}`);
35 |       if (res.ok) setter(await res.json());
36 |     } catch (e) {
37 |       console.error(`Error fetching ${endpoint}:`, e);
38 |     } finally {
39 |       setLoading(prev => ({ ...prev, [key]: false }));
40 |     }
```

```
41 |   }, []);
42 |
43 |   // Detect price changes for pulse animation
44 |   const fetchMarketsWithDiff = useCallback(async () => {
45 |     setLoading(prev => ({ ...prev, markets: true }));
46 |     try {
47 |       const res = await fetch(`${BACKEND_URL}/api/foreign-markets`);
48 |       if (res.ok) {
49 |         const newData = await res.json();
50 |         // Compare with previous data to find changed symbols
51 |         if (prevMarketsRef.current) {
52 |           const changes = new Set();
53 |           const allPrev = [
54 |             ...(prevMarketsRef.current.asia || []),
55 |             ...(prevMarketsRef.current.europe || []),
56 |             ...(prevMarketsRef.current.americas || []),
57 |             ...(prevMarketsRef.current.commodities || []),
58 |             ...(prevMarketsRef.current.currencies || []),
59 |           ];
60 |           const allNew = [
61 |             ...(newData.asia || []),
62 |             ...(newData.europe || []),
63 |             ...(newData.americas || []),
64 |             ...(newData.commodities || []),
65 |             ...(newData.currencies || []),
66 |           ];
67 |           const prevMap = {};
68 |           allPrev.forEach(m => { prevMap[m.symbol] = m.price; });
69 |           allNew.forEach(m => {
70 |             if (prevMap[m.symbol] !== undefined && prevMap[m.symbol] !== m.price) {
71 |               changes.add(m.symbol);
72 |             }
73 |           });
74 |           if (changes.size > 0) {
75 |             setChangedSymbols(changes);
76 |             setTimeout(() => setChangedSymbols(new Set()), 2000);
77 |           }
78 |         }
79 |         prevMarketsRef.current = newData;
80 |         setForeignMarkets(newData);
81 |         setLastRefresh(new Date());
82 |       }
83 |     } catch (e) {
84 |       console.error('Error fetching foreign-markets:', e);
85 |     } finally {
86 |       setLoading(prev => ({ ...prev, markets: false }));
87 |     }
88 |   }, []);
89 |
90 |   // Initial load
91 |   useEffect(() => {
92 |     fetchData('world-events', setWorldEvents, 'world');
93 |     fetchMarketsWithDiff();
94 |     fetchData('gov-filings', setGovFilings, 'congress');
95 |   }, [fetchData, fetchMarketsWithDiff]);
96 |
97 |   // Auto-refresh timer
98 |   useEffect(() => {
99 |     if (!autoRefresh) return;
100 |     const interval = setInterval(() => {
101 |       if (activeTab === 'markets') {
102 |         fetchMarketsWithDiff();
103 |       } else if (activeTab === 'world') {
104 |         fetchData('world-events', setWorldEvents, 'world');
105 |         setLastRefresh(new Date());
106 |       } else {
107 |         fetchData('gov-filings', setGovFilings, 'congress');
```

```
108 |     setLastRefresh(new Date());
109 |   }
110 | }, AUTO_REFRESH_MS);
111 |   return () => clearInterval(interval);
112 | }, [autoRefresh, activeTab, fetchData, fetchMarketsWithDiff]);
113 |
114 | const refresh = () => {
115 |   if (activeTab === 'world') { fetchData('world-events', setWorldEvents, 'world'); setLastRefresh(new Date()); }
116 |   if (activeTab === 'markets') fetchMarketsWithDiff();
117 |   if (activeTab === 'congress') { fetchData('gov-filings', setGovFilings, 'congress'); setLastRefresh(new Date()); }
118 | };
119 |
120 | return (
121 |   <div className="space-y-6" data-testid="macro-dashboard">
122 |     {/* Header */}
123 |     <div className="flex flex-col sm:flex-row items-start sm:items-center justify-between gap-3">
124 |       <div className="flex items-center gap-3">
125 |         <div className="w-10 h-10 bg-gradient-to-br from-[#0052FF] to-violet-600 rounded-xl flex items-center justify-center">
126 |           <Globe className="w-6 h-6 text-white" />
127 |         </div>
128 |         <div>
129 |           <h2 className="text-white text-xl sm:text-2xl font-bold" style={{fontFamily: 'Manrope, sans-serif'}}>Macro Intelligenc ...
130 |           <p className="text-slate-400 text-xs sm:text-sm">Real-time world events, foreign markets & government activity</p>
131 |         </div>
132 |       </div>
133 |       <div className="flex items-center gap-3">
134 |         <Button onClick={refresh} disabled={loading[activeTab]} variant="outline" className="bg-[#1E293B] border-slate-600 text- ...
135 |           <RefreshCw className={`w-4 h-4 mr-2 ${loading[activeTab] ? 'animate-spin' : ''}`} />
136 |           Refresh
137 |         </Button>
138 |         <button
139 |           onClick={() => setAutoRefresh(prev => !prev)}
140 |           className={`flex items-center gap-1.5 px-3 py-2 rounded-lg text-xs font-medium transition-all border ${
141 |             autoRefresh
142 |               ? 'bg-emerald-900/30 border-emerald-700/50 text-emerald-400'
143 |               : 'bg-slate-800 border-slate-700 text-slate-500'
144 |             }`}
145 |           data-testid="auto-refresh-toggle"
146 |         >
147 |           <Radio className={`w-3 h-3 ${autoRefresh ? 'animate-pulse' : ''}`} />
148 |           {autoRefresh ? 'LIVE' : 'PAUSED'}
149 |         </button>
150 |       </div>
151 |     </div>
152 |
153 |     {/* Tab Bar */}
154 |     <div className="flex gap-1 bg-slate-800/60 p-1 rounded-xl border border-slate-700/50" data-testid="macro-tabs">
155 |       {TABS.map(tab => {
156 |         const Icon = tab.icon;
157 |         const isActive = activeTab === tab.id;
158 |         return (
159 |           <button
160 |             key={tab.id}
161 |             onClick={() => setActiveTab(tab.id)}
162 |             data-testid={`macro-tab-${tab.id}`}
163 |             className={`flex-1 flex items-center justify-center gap-2 py-2.5 px-4 rounded-lg text-sm font-medium transition-all ...
164 |               isActive
165 |                 ? 'bg-[#0052FF] text-white shadow-lg shadow-blue-500/20'
166 |                 : 'text-slate-400 hover:text-slate-200 hover:bg-slate-700/50'
167 |             }`}
168 |           >
169 |             <Icon className="w-4 h-4" />
170 |             {tab.label}
171 |           </button>
172 |         );
173 |       })}
174 |     </div>
```

```
175 |
176 |     {/* Tab Content */}
177 |     {activeTab === 'world' && <WorldEventsTab data={worldEvents} loading={loading.world} />}
178 |     {activeTab === 'markets' && <ForeignMarketsTab data={foreignMarkets} loading={loading.markets} changedSymbols={changedSymbol ...
179 |     {activeTab === 'congress' && <CongressTab data={govFilings} loading={loading.congress} isPro={isPro} onSubscribe={onSubscrib ...
180 |
181 |     {/* Last refresh indicator */}
182 |     {lastRefresh && (
183 |         <div className="flex items-center justify-center gap-2 text-slate-600 text-[10px]">
184 |             <Clock className="w-3 h-3" />
185 |             Last refreshed: {lastRefresh.toLocaleTimeString()}
186 |             {autoRefresh && <span className="text-emerald-600">? Auto-refreshing every 30s</span>}
187 |         </div>
188 |     )}
189 | </div>
190 | );
191 | };
192 |
193 | /* ?? World Events Tab ?? */
194 | const WorldEventsTab = ({ data, loading }) => {
195 |     if (loading || !data) return <LoadingState text="Scanning world events..." />;
196 |
197 |     const { high_impact_events = [], all_events = [], affected_sectors = [] } = data;
198 |
199 |     return (
200 |         <div className="space-y-5" data-testid="world-events-tab">
201 |             {/* Stats Bar */}
202 |             <div className="grid grid-cols-2 sm:grid-cols-3 gap-3 sm:gap-4">
203 |                 <StatCard icon=<Zap className="w-4 h-4 text-amber-400" /> label="Total Events" value={data.total_events || 0} />
204 |                 <StatCard icon=<AlertTriangle className="w-4 h-4 text-red-400" /> label="High Impact" value={data.high_impact_count || 0} ...
205 |                 <StatCard icon=<Shield className="w-4 h-4 text-blue-400" /> label="Sectors Affected" value={affected_sectors.length} acc ...
206 |             </div>
207 |
208 |             {/* Affected Sectors Heat Strip */}
209 |             {affected_sectors.length > 0 && (
210 |                 <Card className="bg-slate-800/50 border-slate-700/40 p-4 rounded-xl">
211 |                     <h3 className="text-white text-sm font-semibold mb-3">Sector Impact Map</h3>
212 |                     <div className="flex flex-wrap gap-2">
213 |                         {affected_sectors.map((s) => (
214 |                             <div key={s.sector} className="flex items-center gap-2 bg-slate-900/60 border border-slate-700/50 rounded-lg px-3 py ...
215 |                                 <div className={`w-2 h-2 rounded-full ${s.avg_impact >= 75 ? 'bg-red-500' : s.avg_impact >= 50 ? 'bg-amber-500' : ...
216 |                                 <span className="text-white text-xs font-medium">{s.sector}</span>
217 |                                 <span className={`text-xs font-bold ${s.avg_impact >= 75 ? 'text-red-400' : s.avg_impact >= 50 ? 'text-amber-400' ...
218 |                                 <div className="flex gap-1 ml-1">
219 |                                     {(s.tickers || []).slice(0, 3).map(t => (
220 |                                         <span key={t} className="text-[10px] text-slate-400 bg-slate-800 px-1.5 py-0.5 rounded">{t}</span>
221 |                                     ))}
222 |                                 </div>
223 |                             </div>
224 |                         ))}
225 |                     </div>
226 |                 </Card>
227 |             )}
228 |
229 |             {/* High Impact Events */}
230 |             {high_impact_events.length > 0 && (
231 |                 <div>
232 |                     <h3 className="text-red-400 text-sm font-semibold mb-3 flex items-center gap-2">
233 |                         <AlertTriangle className="w-4 h-4" /> High Impact Events
234 |                     </h3>
235 |                     <div className="space-y-2">
236 |                         {high_impact_events.map((e, i) => (
237 |                             <EventCard key={e.title || i} event={e} isHighImpact />
238 |                         ))}
239 |                     </div>
240 |                 </div>
241 |             )}
```

```
242 |
243 |     {/* All Events */}
244 |     <div>
245 |         <h3 className="text-slate-300 text-sm font-semibold mb-3">All Events ({all_events.length})</h3>
246 |         <div className="space-y-2 max-h-[420px] overflow-y-auto pr-1 custom-scrollbar">
247 |             {all_events.map((e, i) => (
248 |                 <EventCard key={e.title || i} event={e} />
249 |             ))}
250 |         </div>
251 |     </div>
252 | </div>
253 | );
254 | };
255 |
256 | const EventCard = ({ event, isHighImpact }) => (
257 |     <Card className={`p-3 rounded-xl border transition-all hover:border-slate-600 ${
258 |         isHighImpact ? 'bg-red-950/20 border-red-900/40' : 'bg-slate-800/40 border-slate-700/30'
259 |     }`} >
260 |         <div className="flex items-start justify-between gap-3">
261 |             <div className="flex-1 min-w-0">
262 |                 <p className="text-white text-sm font-medium leading-snug truncate">{event.title}</p>
263 |                 <div className="flex items-center gap-2 mt-1.5">
264 |                     <span className="text-[10px] text-slate-500 bg-slate-800 px-2 py-0.5 rounded-full">{event.source}</span>
265 |                     {event.published && <span className="text-[10px] text-slate-500">{event.published}</span>}
266 |                 </div>
267 |             </div>
268 |             {event.affected_sectors?.length > 0 && (
269 |                 <div className="flex gap-1 flex-shrink-0">
270 |                     {event.affected_sectors.slice(0, 2).map((s) => (
271 |                         <Badge key={s.sector} variant="outline" className={`text-[10px] border-slate-700 ${
272 |                             s.impact_score >= 50 ? 'text-amber-400 border-amber-800/50' : 'text-slate-400'
273 |                         }`} >{s.sector}</Badge>
274 |                     ))}
275 |                 </div>
276 |             )}
277 |         </div>
278 |     </Card>
279 | );
280 |
281 | /* ?? Foreign Markets Tab ?? */
282 | const ForeignMarketsTab = ({ data, loading, changedSymbols = new Set() }) => {
283 |     if (loading || !data) return <LoadingState text="Fetching global markets..." />;
284 |
285 |     const regions = [
286 |         { key: 'asia', label: 'Asia-Pacific', emoji: '?' },
287 |         { key: 'europe', label: 'Europe', emoji: '?' },
288 |         { key: 'americas', label: 'Americas', emoji: '?' },
289 |     ];
290 |
291 |     return (
292 |         <div className="space-y-5" data-testid="foreign-markets-tab">
293 |             {/* Correlation Signals Banner */}
294 |             {data.correlation_signals?.length > 0 && (
295 |                 <Card className="bg-gradient-to-r from-amber-950/30 to-orange-950/20 border-amber-800/40 p-4 rounded-xl">
296 |                     <h3 className="text-amber-400 text-xs font-bold uppercase tracking-wider mb-2 flex items-center gap-2">
297 |                         <Zap className="w-3.5 h-3.5" /> Pre-Market Correlation Signals
298 |                     </h3>
299 |                     <div className="space-y-1">
300 |                         {data.correlation_signals.slice(0, 4).map((sig, i) => (
301 |                             <div key={sig.signal || i} className="flex items-center gap-2 text-sm">
302 |                                 {sig.change_percent > 0
303 |                                     ? <TrendingUp className="w-3.5 h-3.5 text-emerald-400 flex-shrink-0" />
304 |                                     : <TrendingDown className="w-3.5 h-3.5 text-red-400 flex-shrink-0" />}
305 |                                 <span className="text-slate-300">{sig.signal}</span>
306 |                                 <Badge className={`ml-auto text-[10px] ${sig.severity === 'high' ? 'bg-red-900/40 text-red-400 border-red-800' : ' ...
307 |                                     {sig.severity}
308 |                                 </Badge>
```

```
309 |         </div>
310 |     )}}
311 | </div>
312 | </Card>
313 | })
314 |
315 | {/* Regional Indices */}
316 | {regions.map(region => {
317 |     const items = data[region.key] || [];
318 |     if (!items.length) return null;
319 |     return (
320 |         <div key={region.key}>
321 |             <h3 className="text-slate-300 text-sm font-semibold mb-3 flex items-center gap-2">
322 |                 <span>{region.emoji}</span> {region.label}
323 |             </h3>
324 |             <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 xl:grid-cols-4 gap-3">
325 |                 {items.map((mkt) => (
326 |                     <MarketCard key={mkt.symbol || mkt.name} market={mkt} isPulsing={changedSymbols.has(mkt.symbol)} />
327 |                 ))}
328 |             </div>
329 |         </div>
330 |     );
331 | })}
332 |
333 | {/* Commodities & Currencies Row */}
334 | <div className="grid grid-cols-1 lg:grid-cols-2 gap-5">
335 |     {data.commodities?.length > 0 && (
336 |         <div>
337 |             <h3 className="text-slate-300 text-sm font-semibold mb-3">Commodities</h3>
338 |             <div className="grid grid-cols-2 gap-3">
339 |                 {data.commodities.map((c) => <MarketCard key={c.symbol || c.name} market={c} compact isPulsing={changedSymbols.has(c ...
340 |             </div>
341 |         </div>
342 |     )}
343 |     {data.currencies?.length > 0 && (
344 |         <div>
345 |             <h3 className="text-slate-300 text-sm font-semibold mb-3">Currencies</h3>
346 |             <div className="grid grid-cols-2 gap-3">
347 |                 {data.currencies.map((c) => <MarketCard key={c.symbol || c.name} market={c} compact isPulsing={changedSymbols.has(c. ...
348 |             </div>
349 |         </div>
350 |     )}
351 | </div>
352 | </div>
353 | );
354 | }];
355 |
356 | const MarketCard = ({ market, compact, isPulsing }) => {
357 |     const isUp = market.change_percent >= 0;
358 |     const absPct = Math.abs(market.change_percent || 0).toFixed(2);
359 |     const isHot = Math.abs(market.change_percent || 0) >= 2;
360 |
361 |     return (
362 |         <Card className={`p-3 rounded-xl border transition-all hover:border-slate-600 ${
363 |             isPulsing ? 'ring-2 ring-[#0052FF]/50 animate-pulse' :
364 |             isHot
365 |             ? isUp ? 'bg-emerald-950/15 border-emerald-800/30' : 'bg-red-950/15 border-red-800/30'
366 |             : 'bg-slate-800/40 border-slate-700/30'
367 |         }`} data-testid={`market-card-${market.symbol}`}>
368 |             <div className="flex items-center justify-between mb-1">
369 |                 <span className="text-white text-xs font-semibold truncate">{market.name}</span>
370 |                 <span className={`text-[10px] px-1.5 py-0.5 rounded ${
371 |                     market.market_state === 'REGULAR' ? 'bg-emerald-900/40 text-emerald-400' :
372 |                     market.market_state === 'PRE' ? 'bg-amber-900/40 text-amber-400' :
373 |                     'bg-slate-700 text-slate-400'
374 |                 }`}>{market.market_state === 'REGULAR' ? 'OPEN' : market.market_state || 'CLOSED'}</span>
375 |             </div>
```

```
376 |     <div className="flex items-end justify-between">
377 |         <span className="text-white text-lg font-bold tabular-nums">
378 |             {market.price ? (market.region === 'Currency' ? market.price.toFixed(4) : market.price.toLocaleString(undefined, {minimu ...
379 |         </span>
380 |         <div className={`flex items-center gap-1 text-xs font-semibold ${isUp ? 'text-emerald-400' : 'text-red-400'}`}>
381 |             {isUp ? <TrendingUp className="w-3 h-3" /> : <TrendingDown className="w-3 h-3" />}
382 |             {isUp ? '+' : '-'}{absPct}%
383 |         </div>
384 |     </div>
385 |     {!compact && (
386 |         <div className="mt-2 w-full bg-slate-700/40 rounded-full h-1">
387 |             <div className={`h-1 rounded-full ${isUp ? 'bg-emerald-500' : 'bg-red-500'}`}>
388 |                 style={{ width: `${Math.min(Math.abs(market.change_percent || 0) * 10, 100)}%` }} />
389 |             </div>
390 |         )}
391 |     </Card>
392 | );
393 | }];
394 |
395 | /* ?? Congressional Trades Tab ?? */
396 | const CongressTab = ({ data, loading, isPro, onSubscribe }) => {
397 |     if (loading || !data) return <LoadingState text="Fetching government filings..." />;
398 |
399 |     const { congressional_trades = [], fed_announcements = [], insider_trades = [] } = data;
400 |
401 |     return (
402 |         <div className="space-y-5" data-testid="congress-tab">
403 |             </* Stats */>
404 |             <div className="grid grid-cols-2 sm:grid-cols-3 gap-3 sm:gap-4">
405 |                 <StatCard icon={<Landmark className="w-4 h-4 text-violet-400" />} label="Congressional Trades" value={data.congressional_c ...
406 |                 <StatCard icon={<Shield className="w-4 h-4 text-blue-400" />} label="Fed Announcements" value={data.fed_count || 0} accent ...
407 |                 <StatCard icon={<BarChart3 className="w-4 h-4 text-emerald-400" />} label="SEC Insider Filings" value={data.insider_count ...
408 |             </div>
409 |
410 |             </* Congressional Trades Table */>
411 |             {congressional_trades.length > 0 && (
412 |                 <Card className="bg-slate-800/50 border-slate-700/40 rounded-xl overflow-hidden">
413 |                     <div className="p-4 border-b border-slate-700/40">
414 |                         <h3 className="text-white text-sm font-semibold flex items-center gap-2">
415 |                             <Landmark className="w-4 h-4 text-violet-400" /> Recent Congressional Stock Trades
416 |                         </h3>
417 |                     </div>
418 |                     <ProBlurWall freeRowCount={3} onSubscribe={onSubscribe} label="Congressional Trades">
419 |                         <div className="overflow-x-auto -mx-4 px-4 sm:mx-0 sm:px-0 overflow-hidden" style={{maxHeight: isPro ? 'none' : '320px'}} ...
420 |                         <table className="w-full text-sm" data-testid="congress-trades-table">
421 |                             <thead>
422 |                                 <tr className="border-b border-slate-700/40">
423 |                                     <th className="text-left text-slate-400 text-xs font-medium px-4 py-2.5">Representative</th>
424 |                                     <th className="text-left text-slate-400 text-xs font-medium px-4 py-2.5">Party</th>
425 |                                     <th className="text-left text-slate-400 text-xs font-medium px-4 py-2.5">Ticker</th>
426 |                                     <th className="text-left text-slate-400 text-xs font-medium px-4 py-2.5">Type</th>
427 |                                     <th className="text-left text-slate-400 text-xs font-medium px-4 py-2.5">Amount</th>
428 |                                     <th className="text-left text-slate-400 text-xs font-medium px-4 py-2.5">Date</th>
429 |                                 </tr>
430 |                             </thead>
431 |                             <tbody>
432 |                                 {congressional_trades.map((trade, i) => (
433 |                                     <tr key={trade.ticker ? `${trade.representative}-${trade.ticker}-${i}` : i} className="border-b border-slate-800 ...
434 |                                     <td className="px-4 py-2.5 text-white font-medium">{trade.representative || 'N/A'}</td>
435 |                                     <td className="px-4 py-2.5">
436 |                                         <span className={`inline-flex items-center gap-1 text-xs font-semibold px-2 py-0.5 rounded-full ${
437 |                                             trade.party === 'R' ? 'bg-red-900/30 text-red-400' :
438 |                                             trade.party === 'D' ? 'bg-blue-900/30 text-blue-400' :
439 |                                             'bg-slate-700 text-slate-400'
440 |                                         }`}>
441 |                                             {trade.party === 'R' ? 'R' : trade.party === 'D' ? 'D' : trade.party || '?'}
442 |                                             {trade.chamber ? ` ${trade.chamber}` : ''}
```

```
443 |         </span>
444 |     </td>
445 |     <td className="px-4 py-2.5">
446 |         <span className="text-[#0052FF] font-bold">{trade.ticker || '?'}</span>
447 |     </td>
448 |     <td className="px-4 py-2.5">
449 |         <span className={`text-xs font-semibold uppercase ${
450 |             trade.type?.toLowerCase() === 'buy' || trade.type?.toLowerCase() === 'purchase'
451 |             ? 'text-emerald-400' : 'text-red-400'
452 |         }`}>{trade.type || '?'}</span>
453 |     </td>
454 |     <td className="px-4 py-2.5 text-slate-300 text-xs">{trade.amount || '?'}</td>
455 |     <td className="px-4 py-2.5 text-slate-500 text-xs">{trade.transaction_date || '?'}</td>
456 | </tr>
457 |   )})
458 | </tbody>
459 | </table>
460 | </div>
461 | </ProBlurWall>
462 | </Card>
463 | )}
464 |
465 | {/* Fed Announcements */}
466 | {fed_announcements.length > 0 && (
467 |   <div>
468 |     <h3 className="text-slate-300 text-sm font-semibold mb-3 flex items-center gap-2">
469 |       <Shield className="w-4 h-4 text-blue-400" /> Federal Reserve Announcements
470 |     </h3>
471 |     <div className="space-y-2">
472 |       {fed_announcements.map((ann) => (
473 |         <Card key={ann.title || ann.date} className="bg-slate-800/40 border-slate-700/30 rounded-xl p-3 hover:border-slate-6 ...
474 |         <p className="text-white text-sm font-medium leading-snug">{ann.title}</p>
475 |         <div className="flex items-center gap-2 mt-2">
476 |           <Clock className="w-3 h-3 text-slate-500" />
477 |           <span className="text-[10px] text-slate-500">{ann.date}</span>
478 |           {ann.url && (
479 |             <a href={ann.url} target="_blank" rel="noopener noreferrer" className="ml-auto text-[10px] text-[#0052FF] hove ...
480 |               View <ChevronRight className="w-3 h-3" />
481 |             </a>
482 |           )}
483 |         </div>
484 |       </Card>
485 |     )})
486 |   </div>
487 | </div>
488 | )}
489 |
490 | {/* SEC Insider Trades */}
491 | {insider_trades.length > 0 && (
492 |   <div>
493 |     <h3 className="text-slate-300 text-sm font-semibold mb-3 flex items-center gap-2">
494 |       <BarChart3 className="w-4 h-4 text-emerald-400" /> SEC Insider Filings
495 |     </h3>
496 |     <div className="space-y-2 max-h-[300px] overflow-y-auto pr-1 custom-scrollbar">
497 |       {insider_trades.map((t, i) => (
498 |         <Card key={i} className="bg-slate-800/40 border-slate-700/30 rounded-xl p-3">
499 |           <div className="flex items-center justify-between">
500 |             <div>
501 |               <p className="text-white text-sm font-medium">{t.company || t.ticker || 'Unknown'}</p>
502 |               <p className="text-slate-500 text-xs mt-0.5">{t.description}</p>
503 |             </div>
504 |             <span className="text-slate-500 text-xs">{t.filed_date}</span>
505 |           </div>
506 |         </Card>
507 |       )})
508 |     </div>
509 |   </div>
```



```
510 |     )}
511 |   </div>
512 | );
513 | };
514 |
515 | /* ?? Shared helpers ?? */
516 | const StatCard = ({ icon, label, value, accent }) => {
517 |   const colors = {
518 |     red: 'from-red-950/30 to-red-900/10 border-red-800/30',
519 |     blue: 'from-blue-950/30 to-blue-900/10 border-blue-800/30',
520 |     violet: 'from-violet-950/30 to-violet-900/10 border-violet-800/30',
521 |     emerald: 'from-emerald-950/30 to-emerald-900/10 border-emerald-800/30',
522 |   };
523 |   return (
524 |     <Card className={`bg-gradient-to-br ${colors[accent]} || 'from-slate-800/50 to-slate-800/30 border-slate-700/40'} p-4 rounded-x ...
525 |       <div className="flex items-center gap-2 mb-1">{icon}<span className="text-slate-400 text-xs">{label}</span></div>
526 |       <p className="text-white text-2xl font-bold tabular-nums">{value}</p>
527 |     </Card>
528 |   );
529 | };
530 |
531 | const LoadingState = ({ text }) => (
532 |   <div className="bg-slate-800/30 border border-slate-700/40 rounded-xl p-12 flex items-center justify-center">
533 |     <RefreshCw className="w-5 h-5 text-blue-400 animate-spin mr-3" />
534 |     <span className="text-slate-400 text-sm">{text}</span>
535 |   </div>
536 | );
537 |
538 | export default MacroDashboard;
539 |
```

FILE: frontend/src/components/MarketPrediction.jsx

412 lines | 19383 characters

```
1 | import React, { useState, useEffect } from 'react';
2 | import { TrendingUp, TrendingDown, Minus, Brain, AlertTriangle, Target, Clock, RefreshCw } from 'lucide-react';
3 | import { Card } from './ui/card';
4 | import { Badge } from './ui/badge';
5 | import { Button } from './ui/button';
6 |
7 | const BACKEND_URL = process.env.REACT_APP_BACKEND_URL;
8 |
9 | const MarketPrediction = () => {
10 |   const [prediction, setPrediction] = useState(null);
11 |   const [loading, setLoading] = useState(true);
12 |   const [lastUpdated, setLastUpdated] = useState(null);
13 |
14 |   useEffect(() => {
15 |     fetchPrediction();
16 |     // Auto-refresh every 5 minutes
17 |     const interval = setInterval(fetchPrediction, 300000);
18 |     return () => clearInterval(interval);
19 |     // eslint-disable-next-line react-hooks/exhaustive-deps
20 |   }, []);
21 |
22 |   const fetchPrediction = async () => {
23 |     setLoading(true);
24 |     try {
25 |       const response = await fetch(`${BACKEND_URL}/api/market/prediction`);
26 |       const data = await response.json();
27 |       setPrediction(data);
28 |       setLastUpdated(new Date());
29 |     } catch (error) {
30 |       console.error('Error fetching prediction:', error);
31 |     } finally {
32 |       setLoading(false);
```

```

33 |   }
34 | };
35 |
36 | const getDirectionIcon = (direction) => {
37 |   if (direction === 'BULLISH') return <TrendingUp className="w-6 h-6 text-emerald-400" />;
38 |   if (direction === 'BEARISH') return <TrendingDown className="w-6 h-6 text-red-400" />;
39 |   return <Minus className="w-6 h-6 text-yellow-400" />;
40 | };
41 |
42 | const getDirectionColor = (direction) => {
43 |   if (direction === 'BULLISH') return 'text-emerald-400 bg-emerald-900 bg-opacity-20 border-green-800';
44 |   if (direction === 'BEARISH') return 'text-red-400 bg-red-900 bg-opacity-20 border-red-800';
45 |   return 'text-yellow-400 bg-yellow-900 bg-opacity-20 border-yellow-800';
46 | };
47 |
48 | const getConfidenceColor = (score) => {
49 |   if (score >= 75) return 'text-emerald-400';
50 |   if (score >= 50) return 'text-yellow-400';
51 |   return 'text-orange-400';
52 | };
53 |
54 | if (loading && !prediction) {
55 |   return (
56 |     <div className="bg-[#0F172A] rounded-xl border border-slate-700/50 p-8">
57 |       <div className="flex items-center justify-center gap-3">
58 |         <RefreshCw className="w-6 h-6 text-blue-400 animate-spin" />
59 |         <p className="text-white">Analyzing market data...</p>
60 |       </div>
61 |     </div>
62 |   );
63 | }
64 |
65 | return (
66 |   <div className="space-y-6">
67 |     <div className="flex flex-col sm:flex-row items-start sm:items-center justify-between gap-3">
68 |       <div className="flex items-center gap-3">
69 |         <div className="w-10 h-10 bg-[#0052FF] rounded-xl flex items-center justify-center">
70 |           <Brain className="w-6 h-6 text-white" />
71 |         </div>
72 |         <div>
73 |           <h2 className="text-white text-xl sm:text-2xl font-bold">AI Market Prediction</h2>
74 |           <p className="text-slate-400 text-xs sm:text-sm">
75 |             Powered by real-time scraping & GPT-5.2 analysis
76 |           </p>
77 |         </div>
78 |       </div>
79 |       <div>
80 |         <Button
81 |           onClick={fetchPrediction}
82 |           disabled={loading}
83 |           variant="outline"
84 |           className="bg-[#1E293B] border-slate-600 text-white hover:bg-slate-700"
85 |         >
86 |           <RefreshCw className={`w-4 h-4 mr-2 ${loading ? 'animate-spin' : ''}`} />
87 |           Refresh
88 |         </Button>
89 |       </div>
90 |
91 |       {prediction && (
92 |         <div className="/* Main Prediction Card */"
93 |         <Card className={`bg-gradient-to-br from-purple-900 to-blue-900 border-2 ${
94 |           prediction.overall_direction === 'BULLISH' ? 'border-green-500' :
95 |           prediction.overall_direction === 'BEARISH' ? 'border-red-500' :
96 |           'border-yellow-500'
97 |         } p-6`}
98 |         <div className="grid grid-cols-1 md:grid-cols-3 gap-6">

```

```
100 |         {/* Overall Direction */}
101 |         <div className="text-center">
102 |             <div className="flex items-center justify-center mb-2">
103 |                 {getDirectionIcon(prediction.overall_direction)}
104 |             </div>
105 |             <h3 className="text-white text-2xl font-bold mb-1">
106 |                 {prediction.overall_direction}
107 |             </h3>
108 |             <p className="text-slate-300 text-sm">Overall Direction</p>
109 |         </div>
110 |
111 |         {/* Confidence Score */}
112 |         <div className="text-center">
113 |             <div className={`text-4xl font-bold mb-2 ${getConfidenceColor(prediction.confidence_score)} `}>
114 |                 {prediction.confidence_score}%
115 |             </div>
116 |             <p className="text-slate-300 text-sm">Confidence Score</p>
117 |             <div className="mt-2 w-full bg-slate-700 rounded-full h-2">
118 |                 <div
119 |                     className={`h-2 rounded-full ${
120 |                         prediction.confidence_score >= 75 ? 'bg-emerald-500' :
121 |                         prediction.confidence_score >= 50 ? 'bg-yellow-500' :
122 |                         'bg-orange-500'
123 |                     } `}>
124 |                     style={{ width: `${prediction.confidence_score}%` }}
125 |                 </div>
126 |             </div>
127 |         </div>
128 |
129 |         {/* Data Sources */}
130 |         <div className="text-center">
131 |             <div className="text-white text-sm space-y-1">
132 |                 <div className="font-semibold mb-2">Data Sources:</div>
133 |                 <div>? {prediction.data_sources?.news_articles || 0} Articles</div>
134 |                 <div className="text-xs text-slate-400">
135 |                     CNBC ? Reuters ? MarketWatch ? Fox Business<br/>
136 |                     WSJ ? Bloomberg ? OAN ? Epoch Times
137 |                 </div>
138 |                 <div>? {prediction.data_sources?.social_posts || 0} Social Posts</div>
139 |                 <div>? {prediction.data_sources?.crypto_signals || 0} Crypto Signals</div>
140 |                 <div>? {prediction.data_sources?.insider_trades || 0} Insider Trades</div>
141 |                 {prediction.data_sources?.world_events > 0 && (
142 |                     <div>? {prediction.data_sources.world_events} World Events</div>
143 |                 )}
144 |                 {prediction.data_sources?.foreign_markets > 0 && (
145 |                     <div>? {prediction.data_sources.foreign_markets} Foreign Indices</div>
146 |                 )}
147 |                 {prediction.data_sources?.congressional_trades > 0 && (
148 |                     <div>? {prediction.data_sources.congressional_trades} Congressional Trades</div>
149 |                 )}
150 |                 {prediction.data_sources?.fed_announcements > 0 && (
151 |                     <div>? {prediction.data_sources.fed_announcements} Fed Announcements</div>
152 |                 )}
153 |                 {prediction.real_estate_summary && (
154 |                     <div>? {prediction.real_estate_summary?.data_sources || 0} Real Estate Sources</div>
155 |                 )}
156 |             </div>
157 |         </div>
158 |     </div>
159 |
160 |     {/* Summary */}
161 |     {prediction.summary && (
162 |         <div className="mt-6 pt-6 border-t border-slate-600">
163 |             <p className="text-white text-center">{prediction.summary}</p>
164 |         </div>
165 |     )}
166 | </Card>
```

```

167 |
168 |     {/* Timeframe Predictions */}
169 |     {prediction.timeframes && (
170 |         <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
171 |             {[ 'intraday', 'short_term', 'medium_term' ].map((timeframe) => {
172 |                 const data = prediction.timeframes[timeframe];
173 |                 if (!data) return null;
174 |
175 |                 return (
176 |                     <Card key={timeframe} className="bg-[#0F172A] border-slate-700 p-4">
177 |                         <div className="flex items-center justify-between mb-3">
178 |                             <div className="flex items-center gap-2">
179 |                                 <Clock className="w-4 h-4 text-blue-400" />
180 |                                 <h4 className="text-white font-semibold capitalize">
181 |                                     {timeframe.replace('_', ' ')}
182 |                                 </h4>
183 |                             </div>
184 |                             <Badge className={getDirectionColor(data.direction)}>
185 |                                 {data.direction}
186 |                             </Badge>
187 |                         </div>
188 |                         {data.target && (
189 |                             <div className="flex items-center gap-2 text-slate-300 text-sm">
190 |                                 <Target className="w-4 h-4" />
191 |                                 <span>Target: {data.target}</span>
192 |                             </div>
193 |                         )}
194 |                     </Card>
195 |                 );
196 |             })}
197 |         </div>
198 |     )}
199 |
200 |     {/* Key Signals */}
201 |     <div className="grid grid-cols-1 md:grid-cols-2 gap-6">
202 |         {/* Bullish Signals */}
203 |         {prediction.key_signals && prediction.key_signals.length > 0 && (
204 |             <Card className="bg-slate-800/50 border-slate-700/50 rounded-xl p-5">
205 |                 <h3 className="text-white font-semibold mb-3 flex items-center gap-2">
206 |                     <TrendingUp className="w-5 h-5 text-emerald-400" />
207 |                     Key Signals
208 |                 </h3>
209 |                 <ul className="space-y-2">
210 |                     {prediction.key_signals.map((signal, index) => (
211 |                         <li key={`signal-${index}-${signal.slice(0,20)}`} className="text-slate-300 text-sm flex items-start gap-2">
212 |                             <span className="text-emerald-400">?</span>
213 |                             {signal}
214 |                         </li>
215 |                     ))}
216 |                 </ul>
217 |             </Card>
218 |         )}
219 |
220 |         {/* Risk Factors */}
221 |         {prediction.risk_factors && prediction.risk_factors.length > 0 && (
222 |             <Card className="bg-slate-800/50 border-slate-700/50 rounded-xl p-5">
223 |                 <h3 className="text-white font-semibold mb-3 flex items-center gap-2">
224 |                     <AlertTriangle className="w-5 h-5 text-orange-400" />
225 |                     Risk Factors
226 |                 </h3>
227 |                 <ul className="space-y-2">
228 |                     {prediction.risk_factors.map((risk, index) => (
229 |                         <li key={`risk-${index}-${risk.slice(0,20)}`} className="text-slate-300 text-sm flex items-start gap-2">
230 |                             <span className="text-orange-400">?</span>
231 |                             {risk}
232 |                         </li>
233 |                     ))}

```

```
234 |         </ul>
235 |     </Card>
236 | })
237 | </div>
238 |
239 | { /* Market Outlooks */}
240 | <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
241 |     {prediction.stock_outlook && (
242 |         <Card className="bg-slate-800/50 border-slate-700/50 rounded-xl p-5">
243 |             <h3 className="text-white font-semibold mb-2">? Stock Market Outlook</h3>
244 |             <p className="text-slate-300 text-sm">{prediction.stock_outlook}</p>
245 |         </Card>
246 |     )}
247 |     {prediction.crypto_outlook && (
248 |         <Card className="bg-slate-800/50 border-slate-700/50 rounded-xl p-5">
249 |             <h3 className="text-white font-semibold mb-2">? Crypto Market Outlook</h3>
250 |             <p className="text-slate-300 text-sm">{prediction.crypto_outlook}</p>
251 |         </Card>
252 |     )}
253 | </div>
254 |
255 | { /* Geopolitical Impact */}
256 | {prediction.geopolitical_impact && (
257 |     <Card className="bg-gradient-to-r from-blue-900/40 to-indigo-900/40 border-blue-700/50 rounded-xl p-5">
258 |         <h3 className="text-white font-semibold mb-3 flex items-center gap-2">
259 |             ? Geopolitical Impact Assessment
260 |         </h3>
261 |         <p className="text-slate-300 text-sm">{prediction.geopolitical_impact}</p>
262 |     </Card>
263 | )}
264 |
265 | { /* Macro Data Intelligence */}
266 | {prediction.macro_data && (
267 |     <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
268 |         { /* World Events */}
269 |         {prediction.macro_data.world_events && prediction.macro_data.world_events.total > 0 && (
270 |             <Card className="bg-slate-800/50 border-slate-700/50 rounded-xl p-5" data-testid="macro-world-events">
271 |                 <h3 className="text-white font-semibold mb-3 flex items-center gap-2">
272 |                     ? World Events
273 |                 </h3>
274 |                 <div className="space-y-2 text-sm">
275 |                     <div className="flex justify-between">
276 |                         <span className="text-slate-400">Events Tracked</span>
277 |                         <span className="text-white font-medium">{prediction.macro_data.world_events.total}</span>
278 |                     </div>
279 |                     <div className="flex justify-between">
280 |                         <span className="text-slate-400">High Impact</span>
281 |                         <span className="text-red-400 font-medium">{prediction.macro_data.world_events.high_impact}</span>
282 |                     </div>
283 |                     {prediction.macro_data.world_events.top_sectors?.length > 0 && (
284 |                         <div>
285 |                             <span className="text-slate-400 text-xs">Affected Sectors:</span>
286 |                             <div className="flex flex-wrap gap-1 mt-1">
287 |                                 {prediction.macro_data.world_events.top_sectors.map((s) => (
288 |                                     <Badge key={s} variant="outline" className="text-xs border-slate-600 text-slate-300">{s}</Badge>
289 |                                 ))}
290 |                             </div>
291 |                         </div>
292 |                     )}
293 |                 </div>
294 |             </Card>
295 |         )}
296 |
297 |         { /* Foreign Markets */}
298 |         {prediction.macro_data.foreign_markets && prediction.macro_data.foreign_markets.total_indices > 0 && (
299 |             <Card className="bg-slate-800/50 border-slate-700/50 rounded-xl p-5" data-testid="macro-foreign-markets">
300 |                 <h3 className="text-white font-semibold mb-3 flex items-center gap-2">
```

```
301 |         ? Foreign Markets
302 |     </h3>
303 |     <div className="space-y-2 text-sm">
304 |         <div className="flex justify-between">
305 |             <span className="text-slate-400">Indices Tracked</span>
306 |             <span className="text-white font-medium">{prediction.macro_data.foreign_markets.total_indices}</span>
307 |         </div>
308 |         {prediction.macro_data.foreign_markets.correlation_signals?.length > 0 && (
309 |             <div>
310 |                 <span className="text-slate-400 text-xs">Correlation Signals:</span>
311 |                 <ul className="mt-1 space-y-1">
312 |                     {prediction.macro_data.foreign_markets.correlation_signals.slice(0, 3).map((sig, i) => (
313 |                         <li key={sig.signal || i} className="text-xs text-slate-300">
314 |                             <span className={sig.change_percent > 0 ? 'text-emerald-400' : 'text-red-400'}>
315 |                                 {sig.change_percent > 0 ? '?' : ''}
316 |                             </span> {sig.signal}
317 |                         </li>
318 |                     ))}
319 |                 </ul>
320 |             </div>
321 |         )}
322 |     </div>
323 | </Card>
324 | )}
325 |
326 | {/* Government Data */}
327 | {prediction.macro_data.gov_filings && (
328 |     <Card className="bg-slate-800/50 border-slate-700/50 rounded-xl p-5" data-testid="macro-gov-filings">
329 |         <h3 className="text-white font-semibold mb-3 flex items-center gap-2">
330 |             ? Government Data
331 |         </h3>
332 |         <div className="space-y-2 text-sm">
333 |             <div className="flex justify-between">
334 |                 <span className="text-slate-400">Congressional Trades</span>
335 |                 <span className="text-white font-medium">{prediction.macro_data.gov_filings.congressional_trades}</span>
336 |             </div>
337 |             <div className="flex justify-between">
338 |                 <span className="text-slate-400">Fed Announcements</span>
339 |                 <span className="text-white font-medium">{prediction.macro_data.gov_filings.fed_announcements}</span>
340 |             </div>
341 |             <div className="flex justify-between">
342 |                 <span className="text-slate-400">SEC Insider Filings</span>
343 |                 <span className="text-white font-medium">{prediction.macro_data.gov_filings.insider_trades}</span>
344 |             </div>
345 |         </div>
346 |     </Card>
347 | )}
348 | </div>
349 | )}
350 |
351 | {/* Real Estate Impact */}
352 | {prediction.real_estate_summary && (
353 |     <Card className="bg-gradient-to-r from-orange-900 to-red-900 bg-opacity-20 border-orange-800 p-5">
354 |         <h3 className="text-white font-semibold mb-3 flex items-center gap-2">
355 |             ? Real Estate Market Impact
356 |         </h3>
357 |         <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
358 |             <div>
359 |                 <p className="text-slate-400 text-sm mb-1">Housing Market Health:</p>
360 |                 <p className="text-white font-medium capitalize">
361 |                     {prediction.real_estate_summary.housing_health || 'Analyzing...'}
362 |                 </p>
363 |             </div>
364 |             <div>
365 |                 <p className="text-slate-400 text-sm mb-1">Commercial Trend:</p>
366 |                 <p className="text-white font-medium capitalize">
367 |                     {prediction.real_estate_summary.commercial_trend || 'Mixed'}
```

```
368 |         </p>
369 |     </div>
370 | </div>
371 |     {prediction.real_estate_summary.implications && (
372 |         <div className="mt-4 pt-4 border-t border-orange-800">
373 |             <p className="text-slate-400 text-xs mb-2">Market Implications:</p>
374 |             <div className="space-y-1 text-sm">
375 |                 {prediction.real_estate_summary.implications.stocks && (
376 |                     <p className="text-slate-300">? Stocks: {prediction.real_estate_summary.implications.stocks}</p>
377 |                 )}
378 |                 {prediction.real_estate_summary.implications.crypto && (
379 |                     <p className="text-slate-300">? Crypto: {prediction.real_estate_summary.implications.crypto}</p>
380 |                 )}
381 |             </div>
382 |         </div>
383 |     )}
384 | </Card>
385 | )}
386 |
387 |     {/* Disclaimer */}
388 |     <div className="bg-yellow-900 bg-opacity-20 border border-yellow-800 rounded-lg p-4">
389 |         <div className="flex items-start gap-3">
390 |             <AlertTriangle className="w-5 h-5 text-yellow-400 flex-shrink-0 mt-0.5" />
391 |             <div className="text-sm text-yellow-200">
392 |                 <strong>Disclaimer:</strong> This prediction is generated by AI analyzing multiple data sources.
393 |                 It should not be considered financial advice. Markets are unpredictable and past performance
394 |                 doesn't guarantee future results. Always do your own research and consult with financial advisors.
395 |             </div>
396 |         </div>
397 |     </div>
398 |
399 |     {/* Last Updated */}
400 |     {lastUpdated && (
401 |         <p className="text-center text-slate-500 text-xs">
402 |             Last updated: {lastUpdated.toLocaleString()}
403 |         </p>
404 |     )}
405 | </>
406 | )}
407 | </div>
408 | );
409 | };
410 |
411 | export default MarketPrediction;
412 |
```

FILE: frontend/src/components/MarketSignals.jsx

134 lines | 6376 characters

```
1 | import React, { useState, useEffect, useCallback } from 'react';
2 | import { Radio, Zap, RefreshCw, Lock, AlertTriangle, TrendingUp, Star } from 'lucide-react';
3 | import { Card } from '../ui/card';
4 | import { Badge } from '../ui/badge';
5 | import { Button } from '../ui/button';
6 | import { useAuth, authFetch } from '../contexts/AuthContext';
7 |
8 | const API = `${process.env.REACT_APP_BACKEND_URL}/api`;
9 |
10 | const severityStyle = (s) => ({
11 |   high: { icon: 'text-red-400', bg: 'border-red-700/40 bg-red-900/10' },
12 |   medium: { icon: 'text-amber-400', bg: 'border-amber-700/40 bg-amber-900/10' },
13 |   low: { icon: 'text-blue-400', bg: 'border-blue-700/40 bg-blue-900/10' },
14 | })[s] || { icon: 'text-slate-400', bg: 'border-slate-700/40' });
15 |
16 | const MarketSignals = ({ onClose, onSubscribe }) => {
17 |   const { isPro } = useAuth();
```

```
18 |   const [signals, setSignals] = useState([]);
19 |   const [loading, setLoading] = useState(false);
20 |   const [scanning, setScanning] = useState(false);
21 |
22 |   const fetchSignals = useCallback(async () => {
23 |     if (!isPro) return;
24 |     setLoading(true);
25 |     try {
26 |       const res = await authFetch(`${API}/signals`);
27 |       if (res.ok) {
28 |         const data = await res.json();
29 |         setSignals(data.signals || []);
30 |       }
31 |     } catch (e) {
32 |       console.error('Signals fetch error:', e);
33 |     } finally {
34 |       setLoading(false);
35 |     }
36 |   }, [isPro]);
37 |
38 |   useEffect(() => { fetchSignals(); }, [fetchSignals]);
39 |
40 |   const scan = async () => {
41 |     setScanning(true);
42 |     try {
43 |       const res = await authFetch(`${API}/signals/scan`, { method: 'POST' });
44 |       if (res.ok) {
45 |         const data = await res.json();
46 |         if (data.signals?.length) {
47 |           setSignals(prev => [...data.signals, ...prev]);
48 |         }
49 |       }
50 |     } catch (e) {
51 |       console.error('Scan error:', e);
52 |     } finally {
53 |       setScanning(false);
54 |     }
55 |   };
56 |
57 |   return (
58 |     <div className="fixed inset-0 bg-black/70 backdrop-blur-sm z-50 flex items-start justify-center overflow-y-auto p-4" data-test ...
59 |       <div className="bg-slate-900 rounded-2xl max-w-lg w-full my-4 border border-slate-700/50">
60 |         <div className="p-5 border-b border-slate-700 flex items-center justify-between">
61 |           <div className="flex items-center gap-3">
62 |             <div className="w-10 h-10 bg-gradient-to-br from-amber-500 to-red-500 rounded-xl flex items-center justify-center">
63 |               <Radio className="w-5 h-5 text-white" />
64 |             </div>
65 |             <div>
66 |               <h2 className="text-white text-lg font-bold" style={{ fontFamily: 'Manrope, sans-serif' }}>Market Signals</h2>
67 |               <p className="text-slate-400 text-xs">AI-detected unusual activity</p>
68 |             </div>
69 |           </div>
70 |           <div className="flex items-center gap-2">
71 |             <Badge className="bg-gradient-to-r from-amber-500 to-red-500 text-white border-0 text-xs">PRO</Badge>
72 |             <button onClick={onClose} className="text-slate-400 hover:text-white text-xl px-2">?</button>
73 |           </div>
74 |         </div>
75 |
76 |         <div className="p-5">
77 |           {!isPro ? (
78 |             <div className="text-center py-8">
79 |               <Lock className="w-10 h-10 text-slate-500 mx-auto mb-3" />
80 |               <p className="text-white font-semibold mb-1">Pro Feature</p>
81 |               <p className="text-slate-400 text-xs mb-4">AI monitors your watchlist for dark pool spikes, whale movements, and unu ...
82 |               <Button className="bg-[#0052FF] text-white rounded-xl" onClick={onSubscribe}>Upgrade to Pro</Button>
83 |             </div>
84 |           ) : (
```



```
85 |         <div className="space-y-4">
86 |             <Button onClick={scan} disabled={scanning} className="bg-[#0052FF] text-white rounded-xl w-full">
87 |                 {scanning ? <><RefreshCw className="w-4 h-4 mr-2 animate-spin" /> Scanning Watchlist...</> : <><Zap className="w-4 ...
88 |             </Button>
89 |
90 |             {loading ? (
91 |                 <div className="flex items-center justify-center py-8"><RefreshCw className="w-6 h-6 text-[#0052FF] animate-spin" ...
92 |             ) : signals.length === 0 ? (
93 |                 <div className="text-center py-8">
94 |                     <Radio className="w-10 h-10 text-slate-600 mx-auto mb-3" />
95 |                     <p className="text-slate-400 text-sm">No signals detected yet</p>
96 |                     <p className="text-slate-500 text-xs mt-1">Add tickers to your watchlist and scan for signals</p>
97 |                 </div>
98 |             ) : (
99 |                 <div className="space-y-2 max-h-[400px] overflow-y-auto">
100 |                     {signals.map((sig, i) => {
101 |                         const style = severityStyle(sig.severity);
102 |                         return (
103 |                             <Card key={`sig-${i}-${sig.ticker}`} className={`border rounded-xl p-3 ${style.bg}`} data-testid={`signal-${ ...
104 |                                 <div className="flex items-start gap-3">
105 |                                     <div className="mt-0.5">
106 |                                         {sig.type === 'dark_pool_spike' ? <AlertTriangle className={`w-4 h-4 ${style.icon}`} /> : <TrendingUp ...
107 |                                     </div>
108 |                                     <div className="flex-1 min-w-0">
109 |                                         <div className="flex items-center gap-2 mb-0.5">
110 |                                             <span className="text-white text-sm font-semibold">{sig.ticker}</span>
111 |                                             <Badge className={`text-[10px] border-0 ${sig.severity === 'high' ? 'bg-red-900/40 text-red-400' : ' ...
112 |                                         </div>
113 |                                         <p className="text-white text-xs font-medium">{sig.title}</p>
114 |                                         <p className="text-slate-400 text-[11px] mt-0.5">{sig.detail}</p>
115 |                                         <p className="text-slate-600 text-[10px] mt-1">
116 |                                             {sig.detected_at ? new Date(sig.detected_at).toLocaleDateString('en-US', { month: 'short', day: 'num ...
117 |                                         </p>
118 |                                     </div>
119 |                                 </div>
120 |                             </Card>
121 |                         );
122 |                     })}
123 |                 </div>
124 |             )}
125 |         </div>
126 |     )}
127 | </div>
128 | </div>
129 | </div>
130 | );
131 | }];
132 |
133 | export default MarketSignals;
134 |
```

FILE: frontend/src/components/MobileBottomNav.jsx

44 lines | 1750 characters

```
1 | import React from 'react';
2 | import { BarChart3, Globe, TrendingUp, Search, Sparkles } from 'lucide-react';
3 |
4 | const MobileBottomNav = ({ onOpenChat }) => {
5 |     const scrollTo = (id) => {
6 |         document.getElementById(id)?.scrollIntoView({ behavior: 'smooth' });
7 |     };
8 |
9 |     const items = [
10 |         { icon: TrendingUp, label: 'Markets', action: () => scrollTo('stock-ticker') },
11 |         { icon: BarChart3, label: 'Options', action: () => scrollTo('options-radar') },
12 |         { icon: Globe, label: 'Macro', action: () => scrollTo('macro-dashboard') },
```

```
13 |     { icon: Search, label: 'Research', action: () => scrollTo('company-research') },
14 |     { icon: Sparkles, label: 'AI Chat', action: onOpenChat, accent: true },
15 | ];
16 |
17 | return (
18 |   <nav className="lg:hidden fixed bottom-0 left-0 right-0 z-50 bg-[#0F172A]/95 backdrop-blur-xl border-t border-slate-700/50 saf ...
19 |     <div className="flex items-center justify-around px-1 py-1.5">
20 |       {items.map((item, i) => {
21 |         const Icon = item.icon;
22 |         return (
23 |           <button
24 |             key={i}
25 |             onClick={item.action}
26 |             className={`flex flex-col items-center gap-0.5 px-3 py-1.5 rounded-xl transition-colors ${
27 |               item.accent
28 |                 ? 'bg-[#0052FF] text-white shadow-lg shadow-blue-500/30'
29 |                 : 'text-slate-400 active:text-white active:bg-slate-800'
30 |             }`}
31 |             data-testid={`mobile-nav-${item.label.toLowerCase().replace(' ', '-')}`}
32 |           >
33 |             <Icon className="w-5 h-5" />
34 |             <span className="text-[10px] font-medium">{item.label}</span>
35 |           </button>
36 |         );
37 |       })}
38 |     </div>
39 |   </nav>
40 | );
41 | };
42 |
43 | export default MobileBottomNav;
44 |
```

FILE: frontend/src/components/Navbar.jsx

261 lines | 17235 characters

```
1 | import React, { useState } from 'react';
2 | import { Search, ChevronDown, Menu, X, User, LogOut, Briefcase, Crown, PieChart, Radio, BookOpen } from 'lucide-react';
3 | import { Button } from './ui/button';
4 | import { Input } from './ui/input';
5 | import BrokerConnect from './BrokerConnect';
6 | import { useAuth } from '../contexts/AuthContext';
7 | import {
8 |   DropdownMenu,
9 |   DropdownMenuContent,
10 |   DropdownMenuItem,
11 |   DropdownMenuTrigger,
12 | } from './ui/dropdown-menu';
13 |
14 | const scrollTo = (id) => document.getElementById(id)?.scrollIntoView({ behavior: 'smooth' });
15 | const openChat = () => window.dispatchEvent(new CustomEvent('risedualai-open-chat'));
16 |
17 | const Navbar = ({ onLogin, onRegister, onSubscribe, onOpenAdmin, onOpenWorkspace, onOpenPortfolio, onOpenSignals, onOpenJournal }) ...
18 |   const [searchValue, setSearchValue] = useState('');
19 |   const [mobileMenuOpen, setMobileMenuOpen] = useState(false);
20 |   const { user, logout, isPro } = useAuth();
21 |
22 |   const handleSearch = (e) => {
23 |     e.preventDefault();
24 |     if (searchValue.trim()) {
25 |       window.dispatchEvent(new CustomEvent('risedualai-research', { detail: searchValue.trim().toUpperCase() }));
26 |       scrollTo('company-research');
27 |       setMobileMenuOpen(false);
28 |     }
29 |   };
30 |
```

```
31 |   const mobileNav = (id) => { scrollTo(id); setMobileMenuOpen(false); };
32 |
33 |   return (
34 |     <nav className="bg-slate-900/80 backdrop-blur-xl border-b border-slate-700/50 px-3 sm:px-6 py-3 sticky top-0 z-40">
35 |       <div className="flex items-center justify-between">
36 |         <div className="flex items-center gap-3 sm:gap-8">
37 |           <div className="flex items-center gap-2.5 cursor-pointer" onClick={() => window.scrollTo({ top: 0, behavior: 'smooth' })} ...
38 |             <div className="w-11 h-11 rounded-lg bg-slate-800/80 border border-slate-600/40 flex items-center justify-center p-1" ...
39 |               
40 |             </div>
41 |             <span className="text-white font-bold text-xl tracking-tight" style={{fontFamily: 'Manrope, sans-serif'}}>RISEDUAL AI< ...
42 |           </div>
43 |
44 |           <form onSubmit={handleSearch} className="relative hidden lg:block">
45 |             <Search className="absolute left-3 top-1/2 transform -translate-y-1/2 text-slate-400 w-4 h-4" />
46 |             <Input type="text" placeholder="Search Symbol" value={searchValue} onChange={(e) => setSearchValue(e.target.value)}
47 |               className="pl-10 bg-slate-800 border-slate-600 text-white placeholder-slate-500 focus:border-[#0052FF] rounded-xl" d ...
48 |           </form>
49 |         </div>
50 |
51 |         {/* Desktop Nav */}
52 |         <div className="hidden lg:flex items-center gap-4">
53 |           <DropdownMenu modal={false}>
54 |             <DropdownMenuTrigger className="text-slate-300 hover:text-slate-50 flex items-center gap-1 text-sm transition-colors o ...
55 |               Platform <ChevronDown className="w-4 h-4" />
56 |             </DropdownMenuTrigger>
57 |             <DropdownMenuContent className="bg-slate-800 border-slate-700 z-[60]">
58 |               <DropdownMenuItem className="text-slate-300 hover:text-slate-50 hover:bg-slate-700 cursor-pointer" onSelect={() => s ...
59 |               <DropdownMenuItem className="text-slate-300 hover:text-slate-50 hover:bg-slate-700 cursor-pointer" onSelect={() => s ...
60 |               <DropdownMenuItem className="text-slate-300 hover:text-slate-50 hover:bg-slate-700 cursor-pointer" onSelect={() => s ...
61 |               <DropdownMenuItem className="text-slate-300 hover:text-slate-50 hover:bg-slate-700 cursor-pointer" onSelect={() => s ...
62 |               <DropdownMenuItem className="text-slate-300 hover:text-slate-50 hover:bg-slate-700 cursor-pointer" onSelect={() => s ...
63 |               <DropdownMenuItem className="text-slate-300 hover:text-slate-50 hover:bg-slate-700 cursor-pointer" onSelect={() => s ...
64 |             </DropdownMenuContent>
65 |           </DropdownMenu>
66 |
67 |           <DropdownMenu modal={false}>
68 |             <DropdownMenuTrigger className="text-slate-300 hover:text-slate-50 flex items-center gap-1 text-sm transition-colors o ...
69 |               Strategies <ChevronDown className="w-4 h-4" />
70 |             </DropdownMenuTrigger>
71 |             <DropdownMenuContent className="bg-slate-800 border-slate-700 z-[60]">
72 |               <DropdownMenuItem className="text-slate-300 hover:text-slate-50 hover:bg-slate-700 cursor-pointer" onSelect={() => s ...
73 |               <DropdownMenuItem className="text-slate-300 hover:text-slate-50 hover:bg-slate-700 cursor-pointer" onSelect={() => s ...
74 |               <DropdownMenuItem className="text-slate-300 hover:text-slate-50 hover:bg-slate-700 cursor-pointer" onSelect={() => s ...
75 |             </DropdownMenuContent>
76 |           </DropdownMenu>
77 |
78 |           <DropdownMenu modal={false}>
79 |             <DropdownMenuTrigger className="text-slate-300 hover:text-slate-50 flex items-center gap-1 text-sm transition-colors o ...
80 |               Education <ChevronDown className="w-4 h-4" />
81 |             </DropdownMenuTrigger>
82 |             <DropdownMenuContent className="bg-slate-800 border-slate-700 z-[60]">
83 |               <DropdownMenuItem className="text-slate-300 hover:text-slate-50 hover:bg-slate-700 cursor-pointer" onSelect={openCha ...
84 |               <DropdownMenuItem className="text-slate-300 hover:text-slate-50 hover:bg-slate-700 cursor-pointer" onSelect={() => s ...
85 |               <DropdownMenuItem className="text-slate-300 hover:text-slate-50 hover:bg-slate-700 cursor-pointer" onSelect={() => s ...
86 |             </DropdownMenuContent>
87 |           </DropdownMenu>
88 |
89 |           <DropdownMenu modal={false}>
90 |             <DropdownMenuTrigger className="text-slate-300 hover:text-slate-50 flex items-center gap-1 text-sm transition-colors o ...
91 |               Resources <ChevronDown className="w-4 h-4" />
92 |             </DropdownMenuTrigger>
93 |             <DropdownMenuContent className="bg-slate-800 border-slate-700 z-[60]">
94 |               <DropdownMenuItem className="text-slate-300 hover:text-slate-50 hover:bg-slate-700 cursor-pointer" onSelect={() => s ...
95 |               <DropdownMenuItem className="text-slate-300 hover:text-slate-50 hover:bg-slate-700 cursor-pointer" onSelect={() => s ...
96 |               <DropdownMenuItem className="text-slate-300 hover:text-slate-50 hover:bg-slate-700 cursor-pointer" onSelect={() => s ...
97 |               <DropdownMenuItem className="text-slate-300 hover:text-slate-50 hover:bg-slate-700 cursor-pointer" onSelect={() => s ...
```

```
98 |         <DropdownMenuItem className="text-slate-300 hover:text-slate-50 hover:bg-slate-700 cursor-pointer" onSelect={() => w ...
99 |         </DropdownMenuContent>
100 |     </DropdownMenu>
101 |
102 |     <BrokerConnect />
103 |
104 |     {!isPro && (
105 |         <Button variant="outline" className="bg-[#0052FF] text-white hover:bg-[#2563EB] border-0 font-semibold rounded-xl"
106 |             onClick={onSubscribe} data-testid="upgrade-btn">
107 |             Upgrade Pro
108 |         </Button>
109 |     )}
110 |
111 |     {user ? (
112 |         <DropdownMenu modal={false}>
113 |             <DropdownMenuTrigger className="flex items-center gap-2 text-slate-300 hover:text-white outline-none" data-testid="u ...
114 |                 <div className="w-8 h-8 bg-[#0052FF] rounded-full flex items-center justify-center text-white text-sm font-bold">
115 |                     {(user.name || user.email || '?')[0].toUpperCase()}
116 |                 </div>
117 |             </DropdownMenuTrigger>
118 |             <DropdownMenuContent className="bg-slate-800 border-slate-700 z-[60] min-w-[200px]">
119 |                 <div className="px-3 py-2 border-b border-slate-700">
120 |                     <p className="text-white text-sm font-medium">{user.name || user.email}</p>
121 |                     <p className="text-slate-400 text-xs">{user.email}</p>
122 |                     <span className={`text-[10px] font-bold uppercase px-1.5 py-0.5 rounded mt-1 inline-block ${isPro ? 'bg-[#0052FF] ...
123 |                         {isPro ? 'PRO' : 'FREE'}`>
124 |                     </span>
125 |                 </div>
126 |                 {!isPro && <DropdownMenuItem className="text-[#0052FF] hover:bg-slate-700 cursor-pointer" onSelect={onSubscribe}>U ...
127 |                 <DropdownMenuItem className="text-slate-300 hover:bg-slate-700 cursor-pointer" onSelect={onOpenWorkspace} data-tes ...
128 |                 <Briefcase className="w-4 h-4 mr-2" /> My Workspace
129 |                 </DropdownMenuItem>
130 |                 <DropdownMenuItem className="text-slate-300 hover:bg-slate-700 cursor-pointer" onSelect={onOpenPortfolio} data-tes ...
131 |                 <PieChart className="w-4 h-4 mr-2" /> Portfolio Analyzer
132 |                 </DropdownMenuItem>
133 |                 <DropdownMenuItem className="text-slate-300 hover:bg-slate-700 cursor-pointer" onSelect={onOpenSignals} data-testi ...
134 |                 <Radio className="w-4 h-4 mr-2" /> Market Signals
135 |                 </DropdownMenuItem>
136 |                 <DropdownMenuItem className="text-slate-300 hover:bg-slate-700 cursor-pointer" onSelect={onOpenJournal} data-testi ...
137 |                 <BookOpen className="w-4 h-4 mr-2" /> Trading Journal
138 |                 </DropdownMenuItem>
139 |                 {user.role === 'owner' && (
140 |                     <DropdownMenuItem className="text-red-400 hover:bg-slate-700 cursor-pointer" onSelect={onOpenAdmin} data-testid= ...
141 |                     <Crown className="w-4 h-4 mr-2" /> Admin Panel
142 |                     </DropdownMenuItem>
143 |                 )}
144 |                 <DropdownMenuItem className="text-red-400 hover:bg-slate-700 cursor-pointer" onSelect={logout}>
145 |                     <LogOut className="w-4 h-4 mr-2" /> Log Out
146 |                 </DropdownMenuItem>
147 |             </DropdownMenuContent>
148 |         </DropdownMenu>
149 |     ) : (
150 |         <Button variant="outline" className="bg-white text-slate-900 hover:bg-slate-100 border-0 rounded-xl font-medium"
151 |             onClick={onLogin} data-testid="login-btn">
152 |             Login
153 |         </Button>
154 |     )}
155 | </div>
156 |
157 |     { /* Mobile */ }
158 |     <div className="flex lg:hidden items-center gap-2">
159 |         {!isPro && (
160 |             <Button variant="outline" size="sm" className="bg-[#0052FF] text-white hover:bg-[#2563EB] border-0 rounded-xl text-xs ...
161 |                 onClick={onSubscribe} data-testid="mobile-upgrade-btn">
162 |                 Pro
163 |             </Button>
164 |         )}
```

```
165 |         {user ? (  
166 |             <div className="w-7 h-7 bg-[#0052FF] rounded-full flex items-center justify-center text-white text-xs font-bold" onCli ...  
167 |                 {(user.name || user.email || '?')[0].toUpperCase()}  
168 |             </div>  
169 |         ) : (  
170 |             <Button variant="outline" size="sm" className="bg-white text-slate-900 hover:bg-slate-100 border-0 rounded-xl text-xs ...  
171 |                 onClick={onLogin} data-testid="mobile-login-btn">  
172 |                     Login  
173 |                 </Button>  
174 |             )}  
175 |             <button onClick={() => setMobileMenuOpen(!mobileMenuOpen)} className="text-slate-300 hover:text-white p-2" data-testid=" ...  
176 |                 {mobileMenuOpen ? <X className="w-6 h-6" /> : <Menu className="w-6 h-6" />}  
177 |             </button>  
178 |         </div>  
179 |     </div>  
180 |  
181 |     {/* Mobile Menu */}  
182 |     {mobileMenuOpen && (  
183 |         <div className="lg:hidden mt-3 pb-3 border-t border-slate-700/50 pt-3 space-y-3" data-testid="mobile-menu">  
184 |             <form onSubmit={handleSearch} className="relative">  
185 |                 <Search className="absolute left-3 top-1/2 transform -translate-y-1/2 text-slate-400 w-4 h-4" />  
186 |                 <Input type="text" placeholder="Search Symbol" value={searchValue} onChange={(e) => setSearchValue(e.target.value)}  
187 |                     className="pl-10 bg-slate-800 border-slate-600 text-white placeholder-slate-500 focus:border-[#0052FF] rounded-xl w- ...  
188 |                 </form>  
189 |                 <div className="grid grid-cols-2 gap-2">  
190 |                     {[  
191 |                         { label: 'AI Hypothesis', id: 'ai-hypothesis' },  
192 |                         { label: 'Market Prediction', id: 'market-prediction' },  
193 |                         { label: 'Company Research', id: 'company-research' },  
194 |                         { label: 'Macro Intelligence', id: 'macro-dashboard' },  
195 |                         { label: 'Options Radar', id: 'options-radar' },  
196 |                         { label: 'Dark Pool', id: 'dark-pool' },  
197 |                         { label: 'Crypto Market', id: 'crypto' },  
198 |                         { label: 'Momentum', id: 'momentum' },  
199 |                     ].map(item => (  
200 |                         <button key={item.id} onClick={() => mobileNav(item.id)}  
201 |                             className="text-left text-slate-300 text-sm py-2.5 px-3 rounded-lg bg-slate-800/50 border border-slate-700/50 hove ...  
202 |                             {item.label}  
203 |                         </button>  
204 |                     ))}  
205 |                 </div>  
206 |                 <div className="flex gap-2 pt-1">  
207 |                     {user ? (  
208 |                         <>  
209 |                             <div className="flex-1 text-sm text-slate-300 flex items-center gap-2 px-3">  
210 |                                 <User className="w-4 h-4" /> {user.name || user.email}  
211 |                                 <span className={`text-[10px] font-bold uppercase px-1.5 py-0.5 rounded ${isPro ? 'bg-[#0052FF]/20 text-[#0052FF] ...  
212 |                                     {isPro ? 'PRO' : 'FREE'}  
213 |                                 </span>  
214 |                             </div>  
215 |                             <Button variant="outline" size="sm" className="bg-[#0052FF]/20 text-[#0052FF] border-[#0052FF]/30 rounded-xl text- ...  
216 |                                 onClick={() => { onOpenWorkspace(); setMobileMenuOpen(false); }} data-testid="mobile-workspace-btn">  
217 |                                     <Briefcase className="w-3 h-3 mr-1" /> Workspace  
218 |                             </Button>  
219 |                             <Button variant="outline" size="sm" className="bg-[#0052FF]/20 text-[#0052FF] border-[#0052FF]/30 rounded-xl text- ...  
220 |                                 onClick={() => { onOpenPortfolio(); setMobileMenuOpen(false); }} data-testid="mobile-portfolio-btn">  
221 |                                     <PieChart className="w-3 h-3 mr-1" /> Portfolio  
222 |                             </Button>  
223 |                             <Button variant="outline" size="sm" className="bg-amber-900/30 text-amber-400 border-amber-800/50 rounded-xl text- ...  
224 |                                 onClick={() => { onOpenSignals(); setMobileMenuOpen(false); }} data-testid="mobile-signals-btn">  
225 |                                     <Radio className="w-3 h-3 mr-1" /> Signals  
226 |                             </Button>  
227 |                             <Button variant="outline" size="sm" className="bg-indigo-900/30 text-indigo-400 border-indigo-800/50 rounded-xl te ...  
228 |                                 onClick={() => { onOpenJournal(); setMobileMenuOpen(false); }} data-testid="mobile-journal-btn">  
229 |                                     <BookOpen className="w-3 h-3 mr-1" /> Journal  
230 |                             </Button>  
231 |                         {user.role === 'owner' && (  

```

```
232 |         <Button variant="outline" size="sm" className="bg-red-900/30 text-red-400 border-red-800/50 rounded-xl text-xs"
233 |             onClick={() => { onOpenAdmin(); setMobileMenuOpen(false); }} data-testid="mobile-admin-btn">
234 |             <Crown className="w-3 h-3 mr-1" /> Admin
235 |         </Button>
236 |     )}
237 |     <Button variant="outline" size="sm" className="bg-red-900/30 text-red-400 border-red-800/50 rounded-xl text-xs"
238 |         onClick={() => { logout(); setMobileMenuOpen(false); }}>
239 |         Log Out
240 |     </Button>
241 | </>
242 | ) : (
243 | <>
244 |     <Button className="flex-1 bg-[#0052FF] hover:bg-[#2563EB] text-white rounded-xl text-sm" onClick={() => { onRegist ...
245 |         Sign Up
246 |     </Button>
247 |     <Button variant="outline" className="flex-1 bg-white text-slate-900 hover:bg-slate-100 border-0 rounded-xl text-sm ...
248 |         onClick={() => { onLogin(); setMobileMenuOpen(false); }}>
249 |         Login
250 |     </Button>
251 | </>
252 | )}
253 | </div>
254 | </div>
255 | )}
256 | </nav>
257 | );
258 | };
259 |
260 | export default Navbar;
261 |
```

FILE: frontend/src/components/OptionsFlowScreener.jsx

67 lines | 2246 characters

```
1 | import React from 'react';
2 | import DataTable from './DataTable';
3 | import { optionsFlowData } from '../mockData';
4 | import { Tabs, TabsContent, TabsList, TabsTrigger } from './ui/tabs';
5 |
6 | const OptionsFlowScreener = () => {
7 |     const columns = [
8 |         { key: 'contract', label: 'Contract', sortable: true },
9 |         { key: 'price', label: 'Price', sortable: true },
10 |         { key: 'returns', label: '% Returns', sortable: true },
11 |         { key: 'volOI', label: 'Vol/OI', sortable: true },
12 |         { key: 'power', label: 'Power', sortable: true },
13 |         { key: 'ivRank', label: 'IV Rank', sortable: true },
14 |         { key: 'aiScore', label: 'AI Score', sortable: true },
15 |     ];
16 |
17 |     return (
18 |         <div className="space-y-6 mt-8">
19 |             </* Header */>
20 |             <div className="flex items-center gap-3">
21 |                 <div className="w-10 h-10 bg-[#0052FF] rounded-xl flex items-center justify-center">
22 |                     <svg viewBox="0 0 24 24" fill="none" className="w-6 h-6">
23 |                         <circle cx="12" cy="12" r="10" stroke="white" strokeWidth="2" />
24 |                         <path d="M12 6v6l4 2" stroke="white" strokeWidth="2" />
25 |                     </svg>
26 |                 </div>
27 |                 <div>
28 |                     <h2 className="text-white text-2xl font-bold">Options Flow Screener</h2>
29 |                     <p className="text-slate-400 text-sm">OPRA data is delayed by 15 minutes</p>
30 |                 </div>
31 |             </div>
32 |
```

```
33 |     {/* Content */}
34 |     <div className="grid grid-cols-1 lg:grid-cols-2 gap-6">
35 |       {/* Most Actively Traded */}
36 |       <DataTable
37 |         title="Most Actively Traded"
38 |         columns={columns}
39 |         data={optionsFlowData.mostActivelyTraded}
40 |       />
41 |
42 |       {/* 0-DTE Edge */}
43 |       <DataTable
44 |         title="0-DTE Edge"
45 |         columns={columns}
46 |         data={optionsFlowData.dteEdge}
47 |       />
48 |     </div>
49 |
50 |     {/* Volatility Opportunities */}
51 |     <div className="grid grid-cols-1 lg:grid-cols-2 gap-6">
52 |       <DataTable
53 |         title="Volatility Opportunities (Low IV Rank)"
54 |         columns={columns}
55 |         data={optionsFlowData.volatilityLow}
56 |       />
57 |       <DataTable
58 |         title="Volatility Opportunities (High IV Rank)"
59 |         columns={columns}
60 |         data={optionsFlowData.volatilityHigh}
61 |       />
62 |     </div>
63 |   </div>
64 | );
65 | };
66 |
67 | export default OptionsFlowScreener;
```

FILE: frontend/src/components/OptionsRadar.jsx

53 lines | 1766 characters

```
1 | import React from 'react';
2 | import DataTable from './DataTable';
3 | import { optionsRadarData } from '../mockData';
4 |
5 | const OptionsRadar = () => {
6 |   const columns = [
7 |     { key: 'contract', label: 'Contract', sortable: true },
8 |     { key: 'price', label: 'Price', sortable: true },
9 |     { key: 'returns', label: '% Returns', sortable: true },
10 |    { key: 'volOI', label: 'Vol/OI', sortable: true },
11 |    { key: 'power', label: 'Power', sortable: true },
12 |    { key: 'ivRank', label: 'IV Rank', sortable: true },
13 |    { key: 'aiScore', label: 'AI Score', sortable: true },
14 |  ];
15 |
16 |   return (
17 |     <div className="space-y-6">
18 |       {/* Header */}
19 |       <div className="flex items-center gap-3">
20 |         <div className="w-10 h-10 bg-[#0052FF] rounded-xl flex items-center justify-center">
21 |           <svg viewBox="0 0 24 24" fill="none" className="w-6 h-6">
22 |             <circle cx="12" cy="12" r="8" stroke="white" strokeWidth="2" />
23 |             <circle cx="12" cy="12" r="3" fill="white" />
24 |           </svg>
25 |         </div>
26 |         <div>
27 |           <h2 className="text-white text-2xl font-bold">AI Options Radar</h2>
```

```

28 |         <p className="text-slate-400 text-sm">OPRA data is delayed by 15 minutes</p>
29 |     </div>
30 | </div>
31 |
32 |     { /* Most Actively Traded */ }
33 |     <DataTable
34 |         title="Most Actively Traded"
35 |         columns={columns}
36 |         data={optionsRadarData.mostActivelyTraded}
37 |         showFilters={true}
38 |     />
39 |
40 |     { /* Volatility Opportunities */ }
41 |     <div className="grid grid-cols-1 gap-6">
42 |         <DataTable
43 |             title="Volatility Opportunities (Low IV Rank)"
44 |             columns={columns}
45 |             data={optionsRadarData.volatilityOpportunities}
46 |             showFilters={true}
47 |         />
48 |     </div>
49 | </div>
50 | );
51 | };
52 |
53 | export default OptionsRadar;

```

FILE: frontend/src/components/PaymentStatus.jsx

117 lines | 4150 characters

```

1 | import React, { useState, useEffect } from 'react';
2 | import { CheckCircle, XCircle, Loader2 } from 'lucide-react';
3 | import { Button } from './ui/button';
4 |
5 | const API = `${process.env.REACT_APP_BACKEND_URL}/api`;
6 |
7 | const PaymentStatus = ({ sessionId, initialStatus, onClose }) => {
8 |   const [status, setStatus] = useState(initialStatus === 'cancelled' ? 'cancelled' : 'checking');
9 |   const [attempts, setAttempts] = useState(0);
10 |   const maxAttempts = 6;
11 |
12 |   useEffect(() => {
13 |     if (!sessionId || initialStatus === 'cancelled') return;
14 |
15 |     const pollStatus = async () => {
16 |       try {
17 |         const res = await fetch(`${API}/subscription/status/${sessionId}`);
18 |         if (!res.ok) throw new Error('Failed to check status');
19 |         const data = await res.json();
20 |
21 |         if (data.payment_status === 'paid') {
22 |           setStatus('success');
23 |           return;
24 |         }
25 |         if (data.status === 'expired') {
26 |           setStatus('expired');
27 |           return;
28 |         }
29 |
30 |         setAttempts((prev) => {
31 |           if (prev + 1 >= maxAttempts) {
32 |             setStatus('timeout');
33 |             return prev + 1;
34 |           }
35 |           return prev + 1;
36 |         });

```



```
37 |     } catch {
38 |       setAttempts((prev) => {
39 |         if (prev + 1 >= maxAttempts) {
40 |           setStatus('error');
41 |           return prev + 1;
42 |         }
43 |         return prev + 1;
44 |       });
45 |     }
46 |   };
47 |
48 |   if (status === 'checking' && attempts < maxAttempts) {
49 |     const timer = setTimeout(pollStatus, 2000);
50 |     return () => clearTimeout(timer);
51 |   }
52 | }, [sessionId, initialStatus, status, attempts]);
53 |
54 | const renderContent = () => {
55 |   switch (status) {
56 |     case 'checking':
57 |       return (
58 |         <div className="text-center" data-testid="payment-checking">
59 |           <Loader2 className="w-16 h-16 text-blue-500 animate-spin mx-auto mb-4" />
60 |           <h3 className="text-xl font-bold text-white mb-2">Verifying Payment...</h3>
61 |           <p className="text-slate-400">Please wait while we confirm your subscription.</p>
62 |         </div>
63 |       );
64 |     case 'success':
65 |       return (
66 |         <div className="text-center" data-testid="payment-success">
67 |           <CheckCircle className="w-16 h-16 text-emerald-400 mx-auto mb-4" />
68 |           <h3 className="text-xl font-bold text-white mb-2">Payment Successful!</h3>
69 |           <p className="text-slate-400 mb-2">Welcome to RISEDUAL AI Premium.</p>
70 |           <p className="text-emerald-400 text-sm">Your annual subscription is now active.</p>
71 |         </div>
72 |       );
73 |     case 'cancelled':
74 |       return (
75 |         <div className="text-center" data-testid="payment-cancelled">
76 |           <XCircle className="w-16 h-16 text-yellow-500 mx-auto mb-4" />
77 |           <h3 className="text-xl font-bold text-white mb-2">Payment Cancelled</h3>
78 |           <p className="text-slate-400">No charges were made. You can try again anytime.</p>
79 |         </div>
80 |       );
81 |     case 'expired':
82 |     case 'timeout':
83 |     case 'error':
84 |       return (
85 |         <div className="text-center" data-testid="payment-error">
86 |           <XCircle className="w-16 h-16 text-red-400 mx-auto mb-4" />
87 |           <h3 className="text-xl font-bold text-white mb-2">
88 |             {status === 'expired' ? 'Session Expired' : 'Status Check Failed'}
89 |           </h3>
90 |           <p className="text-slate-400">Please check your email for confirmation or try again.</p>
91 |         </div>
92 |       );
93 |     default:
94 |       return null;
95 |   }
96 | };
97 |
98 | return (
99 |   <div className="fixed inset-0 bg-black/70 backdrop-blur-sm z-50 flex items-center justify-center p-4">
100 |     <div className="bg-slate-900 border border-slate-700/50 rounded-2xl max-w-md w-full p-8" data-testid="payment-status-modal">
101 |       {renderContent()}
102 |     <div className="mt-6 text-center">
103 |       <Button
```

```
104 |         onClick={onClose}
105 |         className="bg-[#0052FF] hover:bg-[#2563EB] text-white px-8"
106 |         data-testid="payment-status-close-btn"
107 |     >
108 |         {status === 'checking' ? 'Close' : 'Continue to Dashboard'}
109 |     </Button>
110 | </div>
111 | </div>
112 | </div>
113 | );
114 | };
115 |
116 | export default PaymentStatus;
117 |
```

FILE: frontend/src/components/PortfolioAnalyzer.jsx*156 lines | 9113 characters*

```
1 | import React, { useState } from 'react';
2 | import { PieChart, Plus, Trash2, Zap, TrendingUp, TrendingDown, Minus, Shield, AlertTriangle, Lock } from 'lucide-react';
3 | import { Card } from './ui/card';
4 | import { Badge } from './ui/badge';
5 | import { Button } from './ui/button';
6 | import { Input } from './ui/input';
7 | import { useAuth, authFetch } from '../contexts/AuthContext';
8 |
9 | const API = `${process.env.REACT_APP_BACKEND_URL}/api`;
10 |
11 | const gradeColor = (g) => ({ A: 'text-emerald-400 bg-emerald-900/30', B: 'text-blue-400 bg-blue-900/30', C: 'text-amber-400 bg-amb ...
12 | const riskColor = (r) => ({ low: 'text-emerald-400', medium: 'text-amber-400', high: 'text-orange-400', critical: 'text-red-400' } ...
13 |
14 | const PortfolioAnalyzer = ({ onClose, onSubscribe }) => {
15 |   const { isPro } = useAuth();
16 |   const [holdings, setHoldings] = useState([{}]);
17 |   const [result, setResult] = useState(null);
18 |   const [loading, setLoading] = useState(false);
19 |   const [error, setError] = useState('');
20 |
21 |   const updateHolding = (i, field, value) => {
22 |     setHoldings(prev => prev.map((h, idx) => idx === i ? { ...h, [field]: value } : h));
23 |   };
24 |
25 |   const addRow = () => setHoldings(prev => [...prev, {}]);
26 |   const removeRow = (i) => setHoldings(prev => prev.filter((_, idx) => idx !== i));
27 |
28 |   const analyze = async () => {
29 |     if (!isPro) { onSubscribe?(); return; }
30 |     const valid = holdings.filter(h => h.ticker && h.shares && h.avg_price);
31 |     if (!valid.length) { setError('Add at least one holding'); return; }
32 |     setLoading(true);
33 |     setError('');
34 |     try {
35 |       const res = await authFetch(`${API}/portfolio/analyze`, {
36 |         method: 'POST',
37 |         body: JSON.stringify({ holdings: valid.map(h => ({ ticker: h.ticker.toUpperCase(), shares: parseFloat(h.shares), avg_price ...
38 |       });
39 |       if (res.status === 403) { onSubscribe?(); return; }
40 |       if (!res.ok) throw new Error('Analysis failed');
41 |       setResult(await res.json());
42 |     } catch (e) {
43 |       setError(e.message);
44 |     } finally {
45 |       setLoading(false);
46 |     }
47 |   };
48 | }
```

```
49 |   return (
50 |     <div className="fixed inset-0 bg-black/70 backdrop-blur-sm z-50 flex items-start justify-center overflow-y-auto p-4" data-test ...
51 |       <div className="bg-slate-900 rounded-2xl max-w-2xl w-full my-4 border border-slate-700/50">
52 |         <div className="p-5 border-b border-slate-700 flex items-center justify-between">
53 |           <div className="flex items-center gap-3">
54 |             <div className="w-10 h-10 bg-gradient-to-br from-[#0052FF] to-cyan-500 rounded-xl flex items-center justify-center">
55 |               <PieChart className="w-5 h-5 text-white" />
56 |             </div>
57 |             <div>
58 |               <h2 className="text-white text-lg font-bold" style={{ fontFamily: 'Manrope, sans-serif' }}>Portfolio Analyzer</h2>
59 |               <p className="text-slate-400 text-xs">AI-powered health score & rebalancing</p>
60 |             </div>
61 |           </div>
62 |           <div className="flex items-center gap-2">
63 |             <Badge className="bg-gradient-to-r from-[#0052FF] to-cyan-500 text-white border-0 text-xs">PRO</Badge>
64 |             <button onClick={onClose} className="text-slate-400 hover:text-white text-xl px-2">?</button>
65 |           </div>
66 |         </div>
67 |
68 |         <div className="p-5 space-y-4">
69 |           {!isPro && (
70 |             <div className="bg-slate-800/60 border border-slate-700/50 rounded-xl p-4 text-center">
71 |               <Lock className="w-8 h-8 text-slate-500 mx-auto mb-2" />
72 |               <p className="text-white font-semibold text-sm mb-1">Pro Feature</p>
73 |               <p className="text-slate-400 text-xs mb-3">Input your holdings and get an AI health score, risk analysis, and rebala ...
74 |               <Button className="bg-[#0052FF] text-white rounded-xl" onClick={onSubscribe}>Upgrade to Pro</Button>
75 |             </div>
76 |           )}
77 |
78 |           {isPro && !result && (
79 |             <>
80 |               <div className="space-y-2">
81 |                 {holdings.map((h, i) => (
82 |                   <div key={i} className="flex gap-2 items-center">
83 |                     <Input placeholder="Ticker" value={h.ticker} onChange={e => updateHolding(i, 'ticker', e.target.value.toUpperC ...
84 |                     <Input placeholder="Shares" type="number" value={h.shares} onChange={e => updateHolding(i, 'shares', e.target. ...
85 |                     <Input placeholder="Avg Price" type="number" value={h.avg_price} onChange={e => updateHolding(i, 'avg_price', ...
86 |                     {holdings.length > 1 && (
87 |                       <button onClick={() => removeRow(i)} className="text-slate-500 hover:text-red-400"><Trash2 className="w-4 h- ...
88 |                     </div>
89 |                   </div>
90 |                 )}
91 |               </div>
92 |               <div className="flex gap-2">
93 |                 <Button variant="outline" size="sm" onClick={addRow} className="border-slate-600 text-slate-300 rounded-xl"><Plus ...
94 |                 <Button onClick={analyze} disabled={loading} className="bg-[#0052FF] text-white rounded-xl flex-1">
95 |                   {loading ? 'Analyzing...' : 'Analyze Portfolio'}
96 |                 </Button>
97 |               </div>
98 |               {error && <p className="text-red-400 text-xs">{error}</p>}
99 |             </>
100 |           )}
101 |
102 |           {result && (
103 |             <div className="space-y-4">
104 |               <div className="grid grid-cols-3 gap-3">
105 |                 <Card className="bg-slate-800/60 border-slate-700/40 rounded-xl p-3 text-center">
106 |                   <p className="text-3xl font-bold text-white">{result.health_score}</p>
107 |                   <p className="text-slate-400 text-xs">Health Score</p>
108 |                 </Card>
109 |                 <Card className="bg-slate-800/60 border-slate-700/40 rounded-xl p-3 text-center">
110 |                   <p className={`text-lg font-bold ${riskColor(result.risk_level)}`}>{result.risk_level?.toUpperCase()}</p>
111 |                   <p className="text-slate-400 text-xs">Risk Level</p>
112 |                 </Card>
113 |                 <Card className="bg-slate-800/60 border-slate-700/40 rounded-xl p-3 text-center">
114 |                   <span className={`text-2xl font-bold px-3 py-1 rounded-lg ${gradeColor(result.diversification_grade)}`}>{result. ...
115 |                   <p className="text-slate-400 text-xs mt-1">Diversification</p>
```

```
116 |         </Card>
117 |     </div>
118 |
119 |     {result.summary && <p className="text-slate-300 text-sm bg-slate-800/40 rounded-xl p-3 border border-slate-700/30">{ ...
120 |
121 |     {result.suggestions?.length > 0 && (
122 |         <div>
123 |             <h4 className="text-white text-sm font-semibold mb-2 flex items-center gap-1"><Zap className="w-4 h-4 text-amber ...
124 |             <ul className="space-y-1">
125 |                 {result.suggestions.map((s, i) => <li key={`sug-${i}`} className="text-slate-300 text-xs flex items-start gap- ...
126 |             </ul>
127 |         </div>
128 |     )}
129 |
130 |     {result.rebalance_actions?.length > 0 && (
131 |         <div>
132 |             <h4 className="text-white text-sm font-semibold mb-2 flex items-center gap-1"><Shield className="w-4 h-4 text-bl ...
133 |             <div className="space-y-1">
134 |                 {result.rebalance_actions.map((a, i) => (
135 |                     <div key={`reb-${i}`} className="flex items-center gap-2 bg-slate-800/40 rounded-lg px-3 py-2 text-xs">
136 |                         {a.action === 'buy' ? <TrendingUp className="w-3.5 h-3.5 text-emerald-400" /> : a.action === 'sell' ? <Tre ...
137 |                         <span className="text-white font-semibold">{a.ticker}</span>
138 |                         <Badge className={`text-[10px] border ${a.action === 'buy' ? 'text-emerald-400 bg-emerald-900/30 border-em ...
139 |                         <span className="text-slate-400 flex-1">{a.reason}</span>
140 |                     </div>
141 |                 )})
142 |             </div>
143 |         </div>
144 |     )}
145 |
146 |     <Button variant="outline" onClick={() => setResult(null)} className="border-slate-600 text-slate-300 rounded-xl w-fu ...
147 |     </div>
148 | )}
149 | </div>
150 | </div>
151 | </div>
152 | );
153 | };
154 |
155 | export default PortfolioAnalyzer;
156 |
```

FILE: frontend/src/components/ProBlurWall.jsx

32 lines | 1351 characters

```
1 | import React from 'react';
2 | import { Lock } from 'lucide-react';
3 | import { Button } from './ui/button';
4 | import { useAuth } from '../contexts/AuthContext';
5 |
6 | const ProBlurWall = ({ children, freeRowCount = 3, onSubscribe, label = "Full Data" }) => {
7 |     const { isPro } = useAuth();
8 |
9 |     if (isPro) return children;
10 |
11 |     return (
12 |         <div className="relative" data-testid="pro-blur-wall">
13 |             {children}
14 |             <div className="absolute inset-0 flex flex-col items-center justify-end pb-6" style={{ top: `${freeRowCount * 48 + 60}px` }} ...
15 |                 <div className="absolute inset-0 backdrop-blur-md bg-slate-900/60" />
16 |                 <div className="relative z-10 text-center px-4">
17 |                     <div className="w-12 h-12 bg-slate-800 rounded-2xl flex items-center justify-center mx-auto mb-3 border border-slate-600 ...
18 |                         <Lock className="w-6 h-6 text-slate-400" />
19 |                     </div>
20 |                     <p className="text-white font-semibold text-sm mb-1">Upgrade to See {label}</p>
21 |                     <p className="text-slate-400 text-xs mb-3">First {freeRowCount} rows are free. Get full access with Pro.</p>
```

```
22 |         <Button className="bg-[#0052FF] hover:bg-[#2563EB] text-white rounded-xl text-sm" onClick={onSubscribe} data-testid="blu ...
23 |             Upgrade to Pro
24 |         </Button>
25 |     </div>
26 | </div>
27 | </div>
28 | );
29 | };
30 |
31 | export default ProBlurWall;
32 |
```

FILE: frontend/src/components/PromoBanner.jsx

141 lines | 5368 characters

```
1 | import React, { useState, useEffect } from 'react';
2 | import { X, Gift, Clock, ChevronRight, Zap } from 'lucide-react';
3 | import { useAuth, authFetch } from '../contexts/AuthContext';
4 |
5 | const API = process.env.REACT_APP_BACKEND_URL;
6 |
7 | const PromoBanner = ({ onSubscribe }) => {
8 |   const [promo, setPromo] = useState(null);
9 |   const [progress, setProgress] = useState(null);
10 |   const [dismissed, setDismissed] = useState(false);
11 |   const [timeLeft, setTimeLeft] = useState('');
12 |   const { user } = useAuth();
13 |
14 |   useEffect(() => {
15 |     const load = async () => {
16 |       try {
17 |         const res = await fetch(`${API}/api/promo/active`);
18 |         if (res.ok) {
19 |           const data = await res.json();
20 |           if (data.promo) setPromo(data.promo);
21 |         }
22 |       } catch (e) { /* ignore */ }
23 |     };
24 |     load();
25 |   }, []);
26 |
27 |   useEffect(() => {
28 |     if (!user || !promo) return;
29 |     const loadProgress = async () => {
30 |       try {
31 |         const res = await authFetch(`${API}/api/promo/progress`);
32 |         if (res.ok) setProgress(await res.json());
33 |       } catch (e) { /* ignore */ }
34 |     };
35 |     loadProgress();
36 |   }, [user, promo]);
37 |
38 |   // Countdown timer
39 |   useEffect(() => {
40 |     if (!promo) return;
41 |     const update = () => {
42 |       const end = new Date(promo.end_date);
43 |       const now = new Date();
44 |       const diff = end - now;
45 |       if (diff <= 0) { setTimeLeft('Expired'); return; }
46 |       const d = Math.floor(diff / 86400000);
47 |       const h = Math.floor((diff % 86400000) / 3600000);
48 |       const m = Math.floor((diff % 3600000) / 60000);
49 |       if (d > 0) setTimeLeft(`${d}d ${h}h left`);
50 |       else if (h > 0) setTimeLeft(`${h}h ${m}m left`);
51 |       else setTimeLeft(`${m}m left`);
```

```
52 |     };
53 |     update();
54 |     const interval = setInterval(update, 60000);
55 |     return () => clearInterval(interval);
56 | }, [promo]);
57 |
58 | // Check localStorage for dismissal
59 | useEffect(() => {
60 |     if (promo) {
61 |         const key = `promo_dismissed_${promo.id}`;
62 |         if (localStorage.getItem(key) === 'true') setDismissed(true);
63 |     }
64 | }, [promo]);
65 |
66 | if (!promo || dismissed || timeLeft === 'Expired') return null;
67 |
68 | const dismiss = () => {
69 |     localStorage.setItem(`promo_dismissed_${promo.id}`, 'true');
70 |     setDismissed(true);
71 | };
72 |
73 | const progressCount = progress?.progress || 0;
74 | const target = progress?.target || promo.referral_target;
75 | const completed = progress?.completed || false;
76 | const pct = Math.min(100, Math.round((progressCount / target) * 100));
77 |
78 | return (
79 |     <div className="sticky top-0 z-40 w-full" data-testid="promo-banner">
80 |         <div className="bg-gradient-to-r from-[#0052FF] via-indigo-600 to-purple-600 px-4 py-2.5 relative overflow-hidden">
81 |             {/* Animated background shimmer */}
82 |             <div className="absolute inset-0 bg-gradient-to-r from-transparent via-white/5 to-transparent animate-shimmer" />
83 |
84 |             <div className="max-w-[1600px] mx-auto flex items-center justify-between gap-3 relative">
85 |                 <div className="flex items-center gap-3 flex-1 min-w-0">
86 |                     <div className="w-8 h-8 rounded-lg bg-white/15 flex items-center justify-center flex-shrink-0">
87 |                         <Gift className="w-4 h-4 text-white" />
88 |                     </div>
89 |
90 |                     <div className="flex items-center gap-3 flex-1 min-w-0 flex-wrap">
91 |                         <div className="flex items-center gap-2">
92 |                             <Zap className="w-3.5 h-3.5 text-yellow-300 flex-shrink-0" />
93 |                             <p className="text-white text-sm font-semibold truncate" data-testid="promo-title">{promo.title}</p>
94 |                         </div>
95 |                         <p className="text-white/80 text-xs hidden sm:block">{promo.message}</p>
96 |                     </div>
97 |                 </div>
98 |
99 |                 <div className="flex items-center gap-3 flex-shrink-0">
100 |                    {/* Progress bar (logged-in users) */}
101 |                    {user && (
102 |                        <div className="hidden md:flex items-center gap-2">
103 |                            <div className="w-24 h-1.5 bg-white/20 rounded-full overflow-hidden">
104 |                                <div className="h-full bg-yellow-300 rounded-full transition-all duration-500" style={{ width: `${pct}%` }} />
105 |                            </div>
106 |                            <span className="text-white/80 text-[10px] font-medium whitespace-nowrap" data-testid="promo-progress">
107 |                                {completed ? 'Claimed!' : `${progressCount}/${target}`}
108 |                            </span>
109 |                        </div>
110 |                    )}
111 |
112 |                    {/* Timer */}
113 |                    <div className="flex items-center gap-1 bg-white/10 rounded-lg px-2 py-1" data-testid="promo-timer">
114 |                        <Clock className="w-3 h-3 text-white/70" />
115 |                        <span className="text-white text-[10px] font-medium whitespace-nowrap">{timeLeft}</span>
116 |                    </div>
117 |
118 |                    {/* CTA */}
```

```
119 |         {!user && (  
120 |             <button  
121 |                 onClick={onSubscribe}  
122 |                 className="flex items-center gap-1 bg-white text-[#0052FF] text-xs font-semibold px-3 py-1.5 rounded-lg hover:bg-w ...  
123 |                 data-testid="promo-cta"  
124 |             >  
125 |                 Get Started <ChevronRight className="w-3 h-3" />  
126 |             </button>  
127 |         )}  
128 |  
129 |         { /* Dismiss */ }  
130 |         <button onClick={dismiss} className="text-white/50 hover:text-white p-0.5" data-testid="promo-dismiss">  
131 |             <X className="w-3.5 h-3.5" />  
132 |         </button>  
133 |     </div>  
134 | </div>  
135 | </div>  
136 | </div>  
137 | );  
138 | };  
139 |  
140 | export default PromoBanner;  
141 |
```

FILE: frontend/src/components/QuickTrade.jsx

259 lines | 9572 characters

```
1 | import React, { useState } from 'react';  
2 | import { TrendingUp, TrendingDown, DollarSign, AlertCircle } from 'lucide-react';  
3 | import { Button } from './ui/button';  
4 | import { Input } from './ui/input';  
5 | import { Card } from './ui/card';  
6 | import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from './ui/select';  
7 | import { Label } from './ui/label';  
8 | import axios from 'axios';  
9 |  
10 | const BACKEND_URL = process.env.REACT_APP_BACKEND_URL;  
11 |  
12 | const QuickTrade = ({ symbol, currentPrice }) => {  
13 |     const [isOpen, setIsOpen] = useState(false);  
14 |     const [orderData, setOrderData] = useState({  
15 |         symbol: symbol || '',  
16 |         side: 'buy',  
17 |         quantity: 1,  
18 |         type: 'market',  
19 |         limitPrice: '',  
20 |         broker: 'alpaca'  
21 |     });  
22 |     const [isLoading, setIsLoading] = useState(false);  
23 |     const [result, setResult] = useState(null);  
24 |  
25 |     const handleTrade = async () => {  
26 |         setIsLoading(true);  
27 |         setResult(null);  
28 |  
29 |         try {  
30 |             const order = {  
31 |                 symbol: orderData.symbol.toUpperCase(),  
32 |                 quantity: parseInt(orderData.quantity),  
33 |                 side: orderData.side,  
34 |                 type: orderData.type,  
35 |                 time_in_force: 'day'  
36 |             };  
37 |  
38 |             if (orderData.type === 'limit' && orderData.limitPrice) {  
39 |                 order.limit_price = parseFloat(orderData.limitPrice);
```

```
40 |     }
41 |
42 |     const response = await axios.post(
43 |       `${BACKEND_URL}/api/trading/order/${orderData.broker}`,
44 |       order
45 |     );
46 |
47 |     setResult({
48 |       success: true,
49 |       message: 'Order placed successfully!',
50 |       orderId: response.data.id
51 |     });
52 |
53 |     // Reset form after successful trade
54 |     setTimeout(() => {
55 |       setIsOpen(false);
56 |       setResult(null);
57 |     }, 3000);
58 |   } catch (error) {
59 |     setResult({
60 |       success: false,
61 |       message: error.response?.data?.detail || 'Failed to place order'
62 |     });
63 |   } finally {
64 |     setIsLoading(false);
65 |   }
66 | };
67 |
68 | const estimatedCost = orderData.quantity * (orderData.type === 'limit' ? parseFloat(orderData.limitPrice || 0) : (currentPrice | ...
69 |
70 | return (
71 |   <>
72 |     { /* Quick Trade Button */ }
73 |     <div className="flex gap-2">
74 |       <Button
75 |         onClick={() => {
76 |           setOrderData({ ...orderData, side: 'buy', symbol: symbol || '' });
77 |           setIsOpen(true);
78 |         }}
79 |         className="bg-green-600 hover:bg-green-700 flex items-center gap-2"
80 |         size="sm"
81 |       >
82 |         <TrendingUp className="w-4 h-4" />
83 |         Buy
84 |       </Button>
85 |       <Button
86 |         onClick={() => {
87 |           setOrderData({ ...orderData, side: 'sell', symbol: symbol || '' });
88 |           setIsOpen(true);
89 |         }}
90 |         className="bg-red-600 hover:bg-red-700 flex items-center gap-2"
91 |         size="sm"
92 |       >
93 |         <TrendingDown className="w-4 h-4" />
94 |         Sell
95 |       </Button>
96 |     </div>
97 |
98 |     { /* Trading Modal */ }
99 |     {isOpen && (
100 |       <div className="fixed inset-0 bg-black bg-opacity-70 z-50 flex items-center justify-center p-4">
101 |         <Card className="bg-slate-800/50 border-slate-700/50 rounded-xl p-6 w-full max-w-md">
102 |           { /* Header */ }
103 |           <div className="flex items-center justify-between mb-6">
104 |             <h3 className="text-white text-xl font-bold">
105 |               {orderData.side === 'buy' ? 'Buy' : 'Sell'} {orderData.symbol}
106 |             </h3>
```



```
107 |         <button
108 |             onClick={() => setIsOpen(false)}
109 |             className="text-slate-400 hover:text-slate-50 text-2xl"
110 |         >
111 |             ?
112 |         </button>
113 |     </div>
114 |
115 |     {/* Current Price */}
116 |     {currentPrice && (
117 |         <div className="bg-[#1E293B] rounded-lg p-3 mb-4">
118 |             <div className="flex items-center justify-between">
119 |                 <span className="text-slate-400 text-sm">Current Price</span>
120 |                 <span className="text-white font-semibold">${currentPrice.toFixed(2)}</span>
121 |             </div>
122 |         </div>
123 |     )}
124 |
125 |     {/* Order Form */}
126 |     <div className="space-y-4">
127 |         {/* Symbol */}
128 |         <div>
129 |             <Label className="text-white mb-2 block">Symbol</Label>
130 |             <Input
131 |                 value={orderData.symbol}
132 |                 onChange={(e) => setOrderData({ ...orderData, symbol: e.target.value })}
133 |                 placeholder="AAPL"
134 |                 className="bg-[#1E293B] border-slate-600 text-white"
135 |             />
136 |         </div>
137 |
138 |         {/* Order Type */}
139 |         <div>
140 |             <Label className="text-white mb-2 block">Order Type</Label>
141 |             <Select
142 |                 value={orderData.type}
143 |                 onChange={(value) => setOrderData({ ...orderData, type: value })}
144 |             >
145 |                 <SelectTrigger className="bg-[#1E293B] border-slate-600 text-white">
146 |                     <SelectValue />
147 |                 </SelectTrigger>
148 |                 <SelectContent className="bg-slate-800 border-slate-700">
149 |                     <SelectItem value="market" className="text-white">Market Order</SelectItem>
150 |                     <SelectItem value="limit" className="text-white">Limit Order</SelectItem>
151 |                 </SelectContent>
152 |             </Select>
153 |         </div>
154 |
155 |         {/* Limit Price (if limit order) */}
156 |         {orderData.type === 'limit' && (
157 |             <div>
158 |                 <Label className="text-white mb-2 block">Limit Price</Label>
159 |                 <Input
160 |                     type="number"
161 |                     step="0.01"
162 |                     value={orderData.limitPrice}
163 |                     onChange={(e) => setOrderData({ ...orderData, limitPrice: e.target.value })}
164 |                     placeholder="0.00"
165 |                     className="bg-[#1E293B] border-slate-600 text-white"
166 |                 />
167 |             </div>
168 |         )}
169 |
170 |         {/* Quantity */}
171 |         <div>
172 |             <Label className="text-white mb-2 block">Quantity</Label>
173 |             <Input
```

```
174 |         type="number"
175 |         min="1"
176 |         value={orderData.quantity}
177 |         onChange={(e) => setOrderData({ ...orderData, quantity: e.target.value })}
178 |         className="bg-[#1E293B] border-slate-600 text-white"
179 |     />
180 | </div>
181 |
182 |     { /* Broker */ }
183 |     <div>
184 |         <Label className="text-white mb-2 block">Broker</Label>
185 |         <Select
186 |             value={orderData.broker}
187 |             onChange={(value) => setOrderData({ ...orderData, broker: value })}
188 |         >
189 |             <SelectTrigger className="bg-[#1E293B] border-slate-600 text-white">
190 |                 <SelectValue />
191 |             </SelectTrigger>
192 |             <SelectContent className="bg-slate-800 border-slate-700">
193 |                 <SelectItem value="alpaca" className="text-white">Alpaca</SelectItem>
194 |             </SelectContent>
195 |         </Select>
196 |     </div>
197 |
198 |     { /* Estimated Cost */ }
199 |     <div className="bg-[#1E293B] rounded-lg p-3">
200 |         <div className="flex items-center justify-between">
201 |             <span className="text-slate-400">Estimated {orderData.side === 'buy' ? 'Cost' : 'Value'}</span>
202 |             <div className="flex items-center gap-1">
203 |                 <DollarSign className="w-4 h-4 text-emerald-400" />
204 |                 <span className="text-white font-semibold">{estimatedCost.toFixed(2)}</span>
205 |             </div>
206 |         </div>
207 |     </div>
208 | </div>
209 |
210 |     { /* Result Message */ }
211 |     {result && (
212 |         <div className={`mt-4 p-3 rounded-lg flex items-start gap-2 ${
213 |             result.success ? 'bg-emerald-900 bg-opacity-20 border border-green-800' : 'bg-red-900 bg-opacity-20 border border- ...
214 |         }`}>
215 |             <AlertCircle className={`w-5 h-5 flex-shrink-0 ${result.success ? 'text-emerald-400' : 'text-red-400'}`} />
216 |             <div>
217 |                 <p className={result.success ? 'text-emerald-400' : 'text-red-400'}>{result.message}</p>
218 |                 {result.orderId && (
219 |                     <p className="text-slate-400 text-xs mt-1">Order ID: {result.orderId}</p>
220 |                 )}
221 |             </div>
222 |         </div>
223 |     )}
224 |
225 |     { /* Action Buttons */ }
226 |     <div className="flex gap-3 mt-6">
227 |         <Button
228 |             onClick={() => setIsOpen(false)}
229 |             variant="outline"
230 |             className="flex-1 bg-[#1E293B] border-slate-600 text-white hover:bg-slate-700"
231 |         >
232 |             Cancel
233 |         </Button>
234 |         <Button
235 |             onClick={handleTrade}
236 |             disabled={isLoading || !orderData.symbol || !orderData.quantity}
237 |             className={`flex-1 ${
238 |                 orderData.side === 'buy'
239 |                 ? 'bg-green-600 hover:bg-green-700'
240 |                 : 'bg-red-600 hover:bg-red-700'

```

```
241 |         `}``  
242 |     >  
243 |     {isLoading ? 'Placing Order...' : `${orderData.side === 'buy' ? 'Buy' : 'Sell'} ${orderData.quantity} ${orderData. ...  
244 |     </Button>  
245 | </div>  
246 |  
247 |     {/* Warning */}  
248 |     <p className="text-slate-500 text-xs mt-4 text-center">  
249 |         ?? Trading involves risk. This is connected to your real brokerage account.  
250 |     </p>  
251 | </Card>  
252 | </div>  
253 |     )}  
254 | </>  
255 | );  
256 | };  
257 |  
258 | export default QuickTrade;  
259 |
```

FILE: frontend/src/components/ReferralLeaderboard.jsx

141 lines | 5666 characters

```
1 | import React, { useState, useEffect } from 'react';  
2 | import { Trophy, Users, Medal, ChevronUp, Flame, Share2 } from 'lucide-react';  
3 | import { Card } from '../ui/card';  
4 | import { useAuth, authFetch } from '../contexts/AuthContext';  
5 | import SocialShareButtons from '../SocialShareButtons';  
6 |  
7 | const API = process.env.REACT_APP_BACKEND_URL;  
8 |  
9 | const RANK_STYLES = [  
10 |   { bg: 'from-amber-500/20 to-yellow-600/10', border: 'border-amber-500/40', icon: 'text-amber-400', text: 'text-amber-300' },  
11 |   { bg: 'from-slate-300/15 to-gray-400/10', border: 'border-slate-400/40', icon: 'text-slate-300', text: 'text-slate-300' },  
12 |   { bg: 'from-orange-600/15 to-amber-700/10', border: 'border-orange-600/40', icon: 'text-orange-400', text: 'text-orange-300' },  
13 | ];  
14 |  
15 | const ReferralLeaderboard = () => {  
16 |   const [data, setData] = useState(null);  
17 |   const [loading, setLoading] = useState(true);  
18 |   const [refCode, setRefCode] = useState(null);  
19 |   const { user } = useAuth();  
20 |  
21 |   useEffect(() => {  
22 |     const load = async () => {  
23 |       try {  
24 |         const res = await fetch(`${API}/api/referral/leaderboard`);  
25 |         if (res.ok) setData(await res.json());  
26 |       } catch (e) {  
27 |         console.error('Leaderboard load error:', e);  
28 |       } finally {  
29 |         setLoading(false);  
30 |       }  
31 |     };  
32 |     load();  
33 |   }, []);  
34 |  
35 |   useEffect(() => {  
36 |     if (!user) { setRefCode(null); return; }  
37 |     const loadCode = async () => {  
38 |       try {  
39 |         const res = await authFetch(`${API}/api/referral/info`);  
40 |         if (res.ok) {  
41 |           const info = await res.json();  
42 |           setRefCode(info.code);  
43 |         }  
44 |       }  
45 |     }  
46 |   }, [user]);  
47 |
```

```
44 |     } catch (e) { /* ignore */ }
45 |   };
46 |   loadCode();
47 | }, [user]);
48 |
49 | if (loading) {
50 |   return (
51 |     <Card className="bg-[#0F172A]/80 border-slate-700/40 rounded-2xl p-6" data-testid="leaderboard-loading">
52 |       <div className="flex items-center gap-3 mb-4">
53 |         <Trophy className="w-5 h-5 text-amber-400" />
54 |         <h3 className="text-white font-semibold text-sm">Top Referrers</h3>
55 |       </div>
56 |       <div className="space-y-3">
57 |         {[...Array(3)].map((_, i) => (
58 |           <div key={`skel-${i}`} className="h-12 bg-slate-800/50 rounded-xl animate-pulse" />
59 |         ))}
60 |       </div>
61 |     </Card>
62 |   );
63 | }
64 |
65 | if (!data || data.leaderboard.length === 0) {
66 |   return (
67 |     <Card className="bg-[#0F172A]/80 border-slate-700/40 rounded-2xl p-6" data-testid="leaderboard-empty">
68 |       <div className="flex items-center gap-3 mb-4">
69 |         <Trophy className="w-5 h-5 text-amber-400" />
70 |         <h3 className="text-white font-semibold text-sm">Top Referrers</h3>
71 |       </div>
72 |       <div className="text-center py-6">
73 |         <Users className="w-8 h-8 text-slate-600 mx-auto mb-2" />
74 |         <p className="text-slate-400 text-xs">Be the first to refer a friend!</p>
75 |       </div>
76 |     </Card>
77 |   );
78 | }
79 |
80 | return (
81 |   <Card className="bg-[#0F172A]/80 border-slate-700/40 rounded-2xl p-5" data-testid="referral-leaderboard">
82 |     <div className="flex items-center justify-between mb-4">
83 |       <div className="flex items-center gap-2.5">
84 |         <div className="w-8 h-8 rounded-lg bg-gradient-to-br from-amber-500/20 to-amber-600/10 flex items-center justify-center" ...
85 |         <Trophy className="w-4.5 h-4.5 text-amber-400" />
86 |       </div>
87 |       <div>
88 |         <h3 className="text-white font-semibold text-sm">Top Referrers</h3>
89 |         <p className="text-slate-500 text-[10px]">{data.total_participants} participants</p>
90 |       </div>
91 |     </div>
92 |     <Flame className="w-4 h-4 text-orange-500 animate-pulse" />
93 |   </div>
94 |
95 |   <div className="space-y-2">
96 |     {data.leaderboard.map((entry, i) => {
97 |       const style = RANK_STYLES[i] || { bg: 'from-slate-800/50 to-slate-800/30', border: 'border-slate-700/30', icon: 'text-sl ...
98 |       return (
99 |         <div
100 |           key={`lb-${entry.rank}`}
101 |           className={`flex items-center justify-between bg-gradient-to-r ${style.bg} border ${style.border} rounded-xl px-4 py ...
102 |           data-testid={`leaderboard-rank-${entry.rank}`}
103 |         >
104 |           <div className="flex items-center gap-3">
105 |             <div className="flex items-center justify-center w-7 h-7 rounded-lg bg-slate-900/60">
106 |               {i < 3 ? (
107 |                 <Medal className={`w-4 h-4 ${style.icon}`} />
108 |               ) : (
109 |                 <span className="text-slate-400 text-xs font-bold">#{entry.rank}</span>
110 |               )}
```

```
111 |         </div>
112 |         <span className={`text-sm font-medium ${style.text}`}>{entry.name}</span>
113 |         </div>
114 |         <div className="flex items-center gap-1.5">
115 |             <ChevronUp className="w-3.5 h-3.5 text-emerald-400" />
116 |             <span className="text-white text-sm font-semibold">{entry.referrals}</span>
117 |             <span className="text-slate-500 text-[10px]">referrals</span>
118 |         </div>
119 |     </div>
120 | );
121 | }}}
122 | </div>
123 |
124 | { /* Share section for logged-in users */ }
125 | {refCode && (
126 |     <div className="mt-4 pt-3 border-t border-slate-700/30">
127 |         <div className="flex items-center justify-between">
128 |             <div className="flex items-center gap-1.5">
129 |                 <Share2 className="w-3.5 h-3.5 text-slate-400" />
130 |                 <p className="text-slate-400 text-[10px]">Invite friends & climb the ranks</p>
131 |             </div>
132 |             <SocialShareButtons referralLink={` ${window.location.origin}?ref=${refCode}`} compact />
133 |         </div>
134 |     </div>
135 | )}
136 | </Card>
137 | );
138 | };
139 |
140 | export default ReferralLeaderboard;
141 |
```

FILE: frontend/src/components/SocialShareButtons.jsx

118 lines | 6294 characters

```
1 | import React from 'react';
2 | import { Mail } from 'lucide-react';
3 |
4 | const SHARE_MESSAGE = "I use RISEDUAL AI for AI-powered market predictions, dark pool data & real-time signals. Try it free with m ...
5 |
6 | const platforms = [
7 |   {
8 |     id: 'twitter',
9 |     label: 'X',
10 |    color: 'hover:bg-white/10',
11 |    icon: (
12 |      <svg viewBox="0 0 24 24" className="w-4 h-4 fill-current">
13 |        <path d="M18.244 2.25h3.308l-7.227 8.26 8.502 11.24H16.17l-5.214-6.817L4.99 21.75H1.68l7.73-8.835L1.254 2.25H8.08l4.713 6. ...
14 |      </svg>
15 |    ),
16 |    getUrl: (link, msg) => `https://twitter.com/intent/tweet?text=${encodeURIComponent(msg)}&url=${encodeURIComponent(link)}`,
17 |  },
18 |  {
19 |    id: 'linkedin',
20 |    label: 'LinkedIn',
21 |    color: 'hover:bg-[#0A66C2]/20',
22 |    icon: (
23 |      <svg viewBox="0 0 24 24" className="w-4 h-4 fill-current">
24 |        <path d="M20.447 20.452h-3.554v-5.569c0-1.328-.027-3.037-1.852-3.037-1.853 0-2.136 1.445-2.136 2.939v5.667H9.351V9h3.414v1 ...
25 |      </svg>
26 |    ),
27 |    getUrl: (link) => `https://www.linkedin.com/sharing/share-offsite?url=${encodeURIComponent(link)}`,
28 |  },
29 |  {
30 |    id: 'facebook',
31 |    label: 'Facebook',
```

```
32 |     color: 'hover:bg-[#1877F2]/20',
33 |     icon: (
34 |       <svg viewBox="0 0 24 24" className="w-4 h-4 fill-current">
35 |         <path d="M24 12.073c0-6.627-5.373-12-12-12s-12 5.373-12 12c0 5.99 4.388 10.954 10.125 11.854v-8.385H7.078v-3.47h3.047V9.43 ...
36 |       </svg>
37 |     ),
38 |     getUrl: (link) => `https://www.facebook.com/sharer/sharer.php?u=${encodeURIComponent(link)}`,
39 |   },
40 |   {
41 |     id: 'whatsapp',
42 |     label: 'WhatsApp',
43 |     color: 'hover:bg-[#25D366]/20',
44 |     icon: (
45 |       <svg viewBox="0 0 24 24" className="w-4 h-4 fill-current">
46 |         <path d="M17.472 14.382c-.297-.149-1.758-.867-2.03-.967-.273-.099-.471-.148-.67.15-.197.297-.767.966-.94 1.164-.173.199-.3 ...
47 |       </svg>
48 |     ),
49 |     getUrl: (link, msg) => `https://wa.me/?text=${encodeURIComponent(msg + ' ' + link)}`,
50 |   },
51 |   {
52 |     id: 'telegram',
53 |     label: 'Telegram',
54 |     color: 'hover:bg-[#0088CC]/20',
55 |     icon: (
56 |       <svg viewBox="0 0 24 24" className="w-4 h-4 fill-current">
57 |         <path d="M11.944 0A12 12 0 00 12a12 12 0 0012 12 0 0012-12A12 12 0 0012 0a12 12 0 00-.056 0zm4.962 7.224c.1-.002.32 ...
58 |       </svg>
59 |     ),
60 |     getUrl: (link, msg) => `https://t.me/share/url?url=${encodeURIComponent(link)}&text=${encodeURIComponent(msg)}`,
61 |   },
62 |   {
63 |     id: 'email',
64 |     label: 'Email',
65 |     color: 'hover:bg-slate-500/20',
66 |     icon: <Mail className="w-4 h-4" />,
67 |     getUrl: (link, msg) => `mailto:?subject=${encodeURIComponent('Check out RISEDUAL AI')}&body=${encodeURIComponent(msg + '\n\n' ...
68 |   },
69 | ];
70 |
71 | const SocialShareButtons = ({ referrallink, compact = false }) => {
72 |   if (!referrallink) return null;
73 |
74 |   const handleShare = (platform) => {
75 |     const url = platform.getUrl(referrallink, SHARE_MESSAGE);
76 |     window.open(url, '_blank', 'noopener,noreferrer,width=600,height=400');
77 |   };
78 |
79 |   if (compact) {
80 |     return (
81 |       <div className="flex items-center gap-1.5" data-testid="social-share-compact">
82 |         {platforms.map((p) => (
83 |           <button
84 |             key={p.id}
85 |             onClick={() => handleShare(p)}
86 |             className={`w-8 h-8 rounded-lg bg-slate-800/60 border border-slate-700/40 flex items-center justify-center text-slate- ...
87 |             title={`Share on ${p.label}`}
88 |             data-testid={`share-${p.id}`}
89 |           >
90 |             {p.icon}
91 |           </button>
92 |         ))}
93 |       </div>
94 |     );
95 |   }
96 |
97 |   return (
98 |     <div className="space-y-2" data-testid="social-share-buttons">
```

```
99 |         <p className="text-slate-500 text-[10px] font-medium uppercase tracking-wider">Share your link</p>
100 |     <div className="flex flex-wrap gap-2">
101 |         {platforms.map((p) => (
102 |             <button
103 |                 key={p.id}
104 |                 onClick={() => handleShare(p)}
105 |                 className={`flex items-center gap-1.5 px-3 py-1.5 rounded-xl bg-slate-800/60 border border-slate-700/40 text-slate-400 ...
106 |                 data-testid={`share-${p.id}`}
107 |             >
108 |                 {p.icon}
109 |                 <span>{p.label}</span>
110 |             </button>
111 |         ))}
112 |     </div>
113 | </div>
114 | );
115 | };
116 |
117 | export default SocialShareButtons;
118 |
```

FILE: frontend/src/components/StockTicker.jsx

90 lines | 2508 characters

```
1 | import React, { useEffect, useRef, useState } from 'react';
2 | import { getTickerData } from '../services/api';
3 |
4 | const StockTicker = () => {
5 |     const tickerRef = useRef(null);
6 |     const [stockData, setStockData] = useState([]);
7 |     const [loading, setLoading] = useState(true);
8 |
9 |     useEffect(() => {
10 |         fetchTickerData();
11 |         // Update every 30 seconds
12 |         const interval = setInterval(fetchTickerData, 30000);
13 |         return () => clearInterval(interval);
14 |     }, []);
15 |
16 |     const fetchTickerData = async () => {
17 |         try {
18 |             const data = await getTickerData();
19 |             if (data && data.length > 0) {
20 |                 setStockData(data);
21 |             }
22 |         } catch (error) {
23 |             console.error('Failed to fetch ticker data:', error);
24 |         } finally {
25 |             setLoading(false);
26 |         }
27 |     };
28 |
29 |     useEffect(() => {
30 |         if (stockData.length === 0) return;
31 |
32 |         const ticker = tickerRef.current;
33 |         if (!ticker) return;
34 |
35 |         let animationId;
36 |         let position = 0;
37 |
38 |         const animate = () => {
39 |             position -= 1;
40 |             if (position <= -ticker.scrollWidth / 2) {
41 |                 position = 0;
42 |             }

```

```
43 |     ticker.style.transform = `translateX(${position}px)`;
44 |     animationId = requestAnimationFrame(animate);
45 |   };
46 |
47 |   animationId = requestAnimationFrame(animate);
48 |
49 |   return () => cancelAnimationFrame(animationId);
50 | }, [stockData]);
51 |
52 | const TickerItem = ({ symbol, price, change, changePercent }) => {
53 |   const isNegative = change < 0;
54 |   return (
55 |     <div className="inline-flex items-center gap-2 px-4 whitespace-nowrap">
56 |       <span className="text-white font-medium">{symbol}</span>
57 |       <span className="text-slate-300 text-sm">{price.toFixed(2)}</span>
58 |       <span className={`text-sm ${isNegative ? 'text-red-400' : 'text-emerald-400'}`}>
59 |         {change.toFixed(2)} ({changePercent.toFixed(2)}%)
60 |       </span>
61 |     </div>
62 |   );
63 | };
64 |
65 | if (loading) {
66 |   return (
67 |     <div className="bg-[#0F172A] border-b border-slate-700 overflow-hidden">
68 |       <div className="flex py-2 px-6">
69 |         <span className="text-slate-400 text-sm">Loading market data...</span>
70 |       </div>
71 |     </div>
72 |   );
73 | }
74 |
75 | // Duplicate data for seamless loop
76 | const duplicatedData = [...stockData, ...stockData, ...stockData];
77 |
78 | return (
79 |   <div className="bg-[#0F172A] border-b border-slate-700 overflow-hidden">
80 |     <div ref={tickerRef} className="flex py-2">
81 |       {duplicatedData.map((stock, index) => (
82 |         <TickerItem key={index} {...stock} />
83 |       ))}
84 |     </div>
85 |   </div>
86 | );
87 | };
88 |
89 | export default StockTicker;
90 |
```

FILE: frontend/src/components/SubscriptionPricing.jsx

194 lines | 8878 characters

```
1 | import React, { useState } from 'react';
2 | import { Check, Crown, Zap, TrendingUp, Shield, Clock, Star } from 'lucide-react';
3 | import { Button } from './ui/button';
4 | import { Card } from './ui/card';
5 | import { Badge } from './ui/badge';
6 |
7 | const API = `${process.env.REACT_APP_BACKEND_URL}/api`;
8 |
9 | const SubscriptionPricing = ({ onClose }) => {
10 |   const [selectedPlan, setSelectedPlan] = useState('annual');
11 |   const [isProcessing, setIsProcessing] = useState(false);
12 |
13 |   const features = [
14 |     { icon: TrendingUp, text: 'Real-time market data & alerts' },
```



```
15 |     { icon: Zap, text: 'AI-powered trading insights' },
16 |     { icon: Shield, text: 'Advanced options flow analysis' },
17 |     { icon: Clock, text: 'Dark pool trading intelligence' },
18 |     { icon: Crown, text: 'Direct broker integration' },
19 |     { icon: Check, text: 'Macro Intelligence dashboard' },
20 |     { icon: Check, text: 'Export data to CSV' },
21 |     { icon: Check, text: 'Priority customer support' },
22 | ];
23 |
24 | const handleStripeCheckout = async () => {
25 |     setIsProcessing(true);
26 |     try {
27 |         const originUrl = window.location.origin;
28 |         const response = await fetch(`${API}/subscription/create-checkout-session`, {
29 |             method: 'POST',
30 |             headers: { 'Content-Type': 'application/json' },
31 |             body: JSON.stringify({ origin_url: originUrl, plan: selectedPlan })
32 |         });
33 |
34 |         if (!response.ok) throw new Error('Failed to create checkout session');
35 |         const data = await response.json();
36 |
37 |         if (data.url) {
38 |             window.location.href = data.url;
39 |         } else {
40 |             throw new Error('No checkout URL received');
41 |         }
42 |     } catch (error) {
43 |         console.error('Stripe checkout error:', error);
44 |         alert('Payment processing error. Please try again.');
```

```
45 |         setIsProcessing(false);
46 |     }
47 | };
48 |
49 | return (
50 |     <div className="fixed inset-0 bg-black/70 backdrop-blur-sm z-50 flex items-start justify-center overflow-y-auto p-4">
51 |         <div className="bg-slate-900 rounded-2xl max-w-4xl w-full my-4 border border-slate-700/50 relative" data-testid="subscription ...
52 |             {/* Close button */}
53 |             <button
54 |                 onClick={onClose}
55 |                 className="sticky top-2 float-right mr-4 mt-2 z-10 bg-slate-800 hover:bg-slate-700 text-slate-400 hover:text-slate-50 ro ...
56 |                 data-testid="subscription-close-btn"
57 |             >
58 |                 x
59 |             </button>
60 |
61 |             {/* Header */}
62 |             <div className="p-8 pt-2 text-center border-b border-slate-700">
63 |                 <div className="flex items-center justify-center gap-2 mb-2">
64 |                     <Crown className="w-8 h-8 text-yellow-500" />
65 |                     <h2 className="text-3xl font-bold text-white">Upgrade to Premium</h2>
66 |                 </div>
67 |                 <p className="text-slate-400 mt-2">Unlock the full power of RISEDUAL AI</p>
68 |             </div>
69 |
70 |             {/* Pricing Cards */}
71 |             <div className="p-8">
72 |                 <div className="grid grid-cols-1 md:grid-cols-2 gap-5 max-w-2xl mx-auto">
73 |                     {/* Monthly */}
74 |                     <Card
75 |                         className={`relative rounded-2xl p-6 cursor-pointer transition-all border-2 ${
76 |                             selectedPlan === 'monthly'
77 |                                 ? 'bg-slate-800/80 border-[#0052FF] shadow-lg shadow-blue-500/10'
78 |                                 : 'bg-slate-800/40 border-slate-700/50 hover:border-slate-600'
79 |                         }`}
80 |                         onClick={() => setSelectedPlan('monthly')}
81 |                         data-testid="plan-monthly"
```

```
82 |         >
83 |         <div className="flex items-center gap-3 mb-4">
84 |             <div className={`w-5 h-5 rounded-full border-2 flex items-center justify-center ${
85 |                 selectedPlan === 'monthly' ? 'border-[#0052FF]' : 'border-slate-600'
86 |             }`}>
87 |                 {selectedPlan === 'monthly' && <div className="w-2.5 h-2.5 rounded-full bg-[#0052FF]" />}
88 |             </div>
89 |             <span className="text-white font-semibold text-lg">Monthly</span>
90 |         </div>
91 |         <div className="mb-1">
92 |             <span className="text-4xl font-bold text-white">$45</span>
93 |             <span className="text-slate-400 text-sm">/month</span>
94 |         </div>
95 |         <p className="text-slate-500 text-xs">Billed monthly. Cancel anytime.</p>
96 |     </Card>
97 |
98 |     {/* Annual */}
99 |     <Card
100 |         className={`relative rounded-2xl p-6 cursor-pointer transition-all border-2 ${
101 |             selectedPlan === 'annual'
102 |             ? 'bg-gradient-to-br from-[#0052FF]/15 to-slate-800/80 border-[#0052FF] shadow-lg shadow-blue-500/10'
103 |             : 'bg-slate-800/40 border-slate-700/50 hover:border-slate-600'
104 |         }`}
105 |         onClick={() => setSelectedPlan('annual')}
106 |         data-testid="plan-annual"
107 |     >
108 |         <Badge className="absolute -top-2.5 right-4 bg-emerald-600 text-white font-bold text-[10px] px-2.5 py-0.5">
109 |             SAVE 10%
110 |         </Badge>
111 |         <div className="flex items-center gap-3 mb-4">
112 |             <div className={`w-5 h-5 rounded-full border-2 flex items-center justify-center ${
113 |                 selectedPlan === 'annual' ? 'border-[#0052FF]' : 'border-slate-600'
114 |             }`}>
115 |                 {selectedPlan === 'annual' && <div className="w-2.5 h-2.5 rounded-full bg-[#0052FF]" />}
116 |             </div>
117 |             <span className="text-white font-semibold text-lg">Annual</span>
118 |             <Star className="w-4 h-4 text-yellow-500" />
119 |         </div>
120 |         <div className="mb-1">
121 |             <span className="text-4xl font-bold text-white">$40.50</span>
122 |             <span className="text-slate-400 text-sm">/month</span>
123 |         </div>
124 |         <p className="text-slate-500 text-xs">
125 |             $486/year <span className="line-through text-slate-600">$540</span>
126 |         </p>
127 |     </Card>
128 | </div>
129 |
130 | {/* CTA */}
131 | <div className="max-w-2xl mx-auto mt-6 space-y-3">
132 |     <Button
133 |         onClick={handleStripeCheckout}
134 |         disabled={isProcessing}
135 |         className="w-full bg-[#0052FF] hover:bg-[#2563EB] text-white font-semibold py-6 text-lg rounded-xl"
136 |         data-testid="stripe-checkout-btn"
137 |     >
138 |         {isProcessing
139 |             ? 'Redirecting to Stripe...'
140 |             : selectedPlan === 'annual'
141 |             ? 'Subscribe Now ? $486/year'
142 |             : 'Subscribe Now ? $45/month'
143 |         }
144 |     </Button>
145 |     <div className="flex items-center justify-center gap-2 text-slate-400 text-xs">
146 |         <Shield className="w-4 h-4" />
147 |         <span>Secure payment via Stripe -- Cancel anytime -- 30-day money-back guarantee</span>
148 |     </div>
```

```
149 |         </div>
150 |
151 |         {/* Features List */}
152 |         <div className="mt-8">
153 |             <h3 className="text-white text-xl font-semibold text-center mb-6">Everything You Get:</h3>
154 |             <div className="grid grid-cols-1 md:grid-cols-2 gap-4 max-w-3xl mx-auto">
155 |                 {features.map((feature, index) => {
156 |                     const Icon = feature.icon;
157 |                     return (
158 |                         <div key={index} className="flex items-center gap-3 text-slate-300">
159 |                             <div className="bg-emerald-900 bg-opacity-30 p-2 rounded-lg">
160 |                                 <Icon className="w-5 h-5 text-emerald-400" />
161 |                             </div>
162 |                             <span>{feature.text}</span>
163 |                         </div>
164 |                     );
165 |                 })}
166 |             </div>
167 |         </div>
168 |
169 |         {/* FAQ */}
170 |         <div className="mt-8 border-t border-slate-700 pt-8">
171 |             <h3 className="text-white text-lg font-semibold text-center mb-4">Frequently Asked Questions</h3>
172 |             <div className="space-y-4 max-w-2xl mx-auto">
173 |                 <div>
174 |                     <p className="text-white font-medium">Can I cancel anytime?</p>
175 |                     <p className="text-slate-400 text-sm mt-1">Yes! Cancel your subscription anytime with no penalties. You'll retain ...
176 |                 </div>
177 |                 <div>
178 |                     <p className="text-white font-medium">Can I switch between monthly and annual?</p>
179 |                     <p className="text-slate-400 text-sm mt-1">Yes, you can switch plans at any time. If upgrading to annual, you'll r ...
180 |                 </div>
181 |                 <div>
182 |                     <p className="text-white font-medium">Is my payment information secure?</p>
183 |                     <p className="text-slate-400 text-sm mt-1">Absolutely! All transactions are processed securely through Stripe. We ...
184 |                 </div>
185 |             </div>
186 |         </div>
187 |     </div>
188 | </div>
189 | </div>
190 | );
191 | });
192 |
193 | export default SubscriptionPricing;
194 |
```

FILE: frontend/src/components/TradeGPTChat.jsx

357 lines | 14240 characters

```
1 | import React, { useState, useRef, useEffect, useCallback } from 'react';
2 | import { Send, Sparkles, Image, X, BarChart3, Building2, ChevronDown, Lock } from 'lucide-react';
3 | import { Button } from './ui/button';
4 | import { Input } from './ui/input';
5 | import { Card } from './ui/card';
6 | import { Badge } from './ui/badge';
7 | import { useAuth, authFetch } from '../contexts/AuthContext';
8 | import { sendChatMessage, researchCompany } from '../services/api';
9 | import ChartPatternLibrary from './ChartPatternLibrary';
10 | import { ResearchCard } from './CompanyResearch';
11 |
12 | const API = `${process.env.REACT_APP_BACKEND_URL}/api`;
13 |
14 | const TradeGPTChat = ({ onSubscribe }) => {
15 |     const { user, isPro } = useAuth();
16 |     const [chatLimit, setChatLimit] = useState({ remaining: 5, used: 0, is_pro: false });
```

```
17 |   const [messages, setMessages] = useState([
18 |     {
19 |       role: 'assistant',
20 |       content: 'Hello! I\'m RISEDUAL AI, your AI-powered trading assistant. Ask me anything about stocks, options, market analysis ...
21 |     },
22 |   ]);
23 |   const [input, setInput] = useState('');
24 |   const [isLoading, setIsLoading] = useState(false);
25 |   const [sessionId] = useState(() => `session_${Date.now()}_${Math.random().toString(36).substring(7)}`);
26 |   const [isOpen, setIsOpen] = useState(false);
27 |
28 |   const fetchChatLimit = useCallback(async () => {
29 |     if (!user) return;
30 |     try {
31 |       const res = await authFetch(`${API}/chat/limit`);
32 |       if (res.ok) setChatLimit(await res.json());
33 |     } catch {}
34 |   }, [user]);
35 |
36 |   useEffect(() => { if (isOpen) fetchChatLimit(); }, [isOpen, fetchChatLimit]);
37 |   const [imagePreview, setImagePreview] = useState(null);
38 |   const [imageBase64, setImageBase64] = useState(null);
39 |   const fileInputRef = useRef(null);
40 |   const messagesEndRef = useRef(null);
41 |
42 |   useEffect(() => {
43 |     messagesEndRef.current?.scrollIntoView({ behavior: 'smooth' });
44 |   }, [messages]);
45 |
46 |   useEffect(() => {
47 |     const handleOpenChat = () => setIsOpen(true);
48 |     window.addEventListener('risedualai-open-chat', handleOpenChat);
49 |     return () => window.removeEventListener('risedualai-open-chat', handleOpenChat);
50 |   }, []);
51 |
52 |   const handleImageUpload = (e) => {
53 |     const file = e.target.files[0];
54 |     if (!file) return;
55 |
56 |     const validTypes = ['image/jpeg', 'image/png', 'image/webp'];
57 |     if (!validTypes.includes(file.type)) {
58 |       alert('Please upload a JPEG, PNG, or WEBP image.');
```

```
84 |     const prompt = `Explain the "${pattern.name}" chart pattern in detail. Include: 1) How to identify it, 2) What it signals (${p ...
85 |     setInput('');
86 |
87 |     const userMessage = {
88 |       role: 'user',
89 |       content: `Tell me about the ${pattern.name} pattern`,
90 |     };
91 |     setMessages((prev) => [...prev, userMessage]);
92 |     setIsLoading(true);
93 |
94 |     sendChatMessage(prompt, sessionId)
95 |       .then((response) => {
96 |         setMessages((prev) => [...prev, { role: 'assistant', content: response.response }]);
97 |       })
98 |       .catch(() => {
99 |         setMessages((prev) => [...prev, { role: 'assistant', content: 'Sorry, I encountered an error analyzing that pattern. Pleas ...
100 |       })
101 |       .finally(() => setIsLoading(false));
102 |   };
103 |
104 |   const handleSend = async () => {
105 |     if (!input.trim() && !imageBase64) return;
106 |
107 |     // Rate limit check for free users
108 |     if (user && !isPro && chatLimit.remaining <= 0) {
109 |       setMessages((prev) => [...prev, { role: 'assistant', content: `You've used all ${chatLimit.limit} free messages for today. * ...
110 |       return;
111 |     }
112 |
113 |     // Check for /patterns command
114 |     if (input.trim().toLowerCase() === '/patterns') {
115 |       setMessages((prev) => [...prev,
116 |         { role: 'user', content: '/patterns' },
117 |         { role: 'assistant', content: '__PATTERN_LIBRARY__' }
118 |       ]);
119 |       setInput('');
120 |       return;
121 |     }
122 |
123 |     // Check for /research TICKER command
124 |     const researchMatch = input.trim().match(/^\/research\s+(\w+)\$/i);
125 |     if (researchMatch) {
126 |       const ticker = researchMatch[1].toUpperCase();
127 |       setMessages((prev) => [...prev, { role: 'user', content: `\/research ${ticker}` }]);
128 |       setInput('');
129 |       setIsLoading(true);
130 |       try {
131 |         const data = await researchCompany(ticker);
132 |         setMessages((prev) => [...prev, { role: 'assistant', content: '__RESEARCH__', researchData: data }]);
133 |       } catch {
134 |         setMessages((prev) => [...prev, { role: 'assistant', content: `Could not find research data for ${ticker}.` }]);
135 |       } finally {
136 |         setIsLoading(false);
137 |       }
138 |       return;
139 |     }
140 |
141 |     const userMessage = {
142 |       role: 'user',
143 |       content: input || (imageBase64 ? 'Analyze this chart' : ''),
144 |       image: imagePreview || null,
145 |     };
146 |     setMessages((prev) => [...prev, userMessage]);
147 |
148 |     const currentMessage = input || 'Please analyze this chart/image and provide trading insights.';
149 |     const currentImage = imageBase64;
150 |
```

```
151 |     setInput('');
152 |     clearImage();
153 |     setIsLoading(true);
154 |
155 |     try {
156 |         const response = await sendChatMessage(currentMessage, sessionId, currentImage);
157 |         setMessages((prev) => [...prev, { role: 'assistant', content: response.response }]);
158 |         if (user && !isPro) setChatLimit(prev => ({ ...prev, used: prev.used + 1, remaining: Math.max(0, prev.remaining - 1) }));
159 |     } catch (error) {
160 |         if (error?.message?.includes('429') || error?.message?.includes('limited')) {
161 |             setMessages((prev) => [...prev, { role: 'assistant', content: `You've reached your daily free message limit. **Upgrade to ...`
162 |         } else {
163 |             console.error('Error sending message:', error);
164 |             setMessages((prev) => [...prev, { role: 'assistant', content: 'I apologize, but I encountered an error. Please try again.' ...
165 |         }
166 |     } finally {
167 |         setIsLoading(false);
168 |     }
169 | };
170 |
171 | const renderMessageContent = (message, index) => {
172 |     if (message.content === '__PATTERN_LIBRARY__') {
173 |         return <ChartPatternLibrary onSelectPattern={handlePatternSelect} />;
174 |     }
175 |
176 |     if (message.content === '__RESEARCH__' && message.researchData) {
177 |         return <ResearchCard data={message.researchData} compact={true} />;
178 |     }
179 |
180 |     return (
181 |         <>
182 |             {message.image && (
183 |                 <img
184 |                     src={message.image}
185 |                     alt="Uploaded chart"
186 |                     className="rounded mb-2 max-h-40 w-auto"
187 |                     data-testid={`chat-image-${index}`}
188 |                 />
189 |             )}
190 |             <p className="text-sm whitespace-pre-wrap">{message.content}</p>
191 |         </>
192 |     );
193 | };
194 |
195 | return (
196 |     <div className="fixed bottom-6 right-6 z-40 max-lg:bottom-20 max-lg:right-3">
197 |         <div className="mb-4 flex justify-end">
198 |             <Button
199 |                 data-testid="chat-toggle-btn"
200 |                 className="bg-[#0052FF] hover:bg-[#2563EB] text-white rounded-full w-14 h-14 shadow-lg hidden lg:flex"
201 |                 onClick={() => setIsOpen(!isOpen)}
202 |             >
203 |                 <Sparkles className="w-6 h-6" />
204 |             </Button>
205 |         </div>
206 |
207 |         {isOpen && <Card
208 |             data-testid="chat-window"
209 |             className="w-96 max-lg:w-[calc(100vw-1.5rem)] max-lg:max-w-none h-[500px] max-lg:h-[70vh] bg-slate-900 border-slate-700/50 ...
210 |         >
211 |             <div className="bg-[#0052FF] text-white px-4 py-3 rounded-t-xl">
212 |                 <div className="flex items-center justify-between">
213 |                     <div className="flex items-center gap-2">
214 |                         <div className="w-8 h-8 rounded-md bg-white/15 flex items-center justify-center p-0.5">
215 |                             
216 |                         </div>
217 |                         <h3 className="font-semibold text-lg">RISEDUAL AI</h3>
```

```
218 |         </div>
219 |         <button onClick={() => setIsOpen(false)} className="text-white/70 hover:text-white p-1 rounded-lg hover:bg-white/10 tr ...
220 |         <ChevronDown className="w-5 h-5" />
221 |     </button>
222 | </div>
223 | <p className="text-xs text-blue-200 mt-1 flex items-center gap-2">
224 |     AI Trading Assistant
225 |     {user && !isPro && (
226 |         <Badge className="bg-white/20 text-white/80 border-0 text-[10px]">
227 |             {chatLimit.remaining}/{chatLimit.limit} msgs left
228 |         </Badge>
229 |     )}
230 |     {isPro && <Badge className="bg-white/20 text-white/80 border-0 text-[10px]">Unlimited</Badge>}
231 | </p>
232 | </div>
233 |
234 | <div className="flex-1 overflow-y-auto p-4 space-y-4" data-testid="chat-messages">
235 |     {messages.map((message, index) => (
236 |         <div
237 |             key={`msg-${index}-${message.role}`}
238 |             className={`flex ${message.role === 'user' ? 'justify-end' : 'justify-start'}`}
239 |         >
240 |             <div
241 |                 className={`rounded-lg px-4 py-2 ${
242 |                     message.content === '__PATTERN_LIBRARY__' || message.content === '__RESEARCH__'
243 |                         ? 'max-w-[95%]'
244 |                         : 'max-w-[80%]'
245 |                     } ${
246 |                         message.role === 'user'
247 |                             ? 'bg-[#0052FF] text-white'
248 |                             : 'bg-[#1E293B] text-slate-200'
249 |                     }`}
250 |             >
251 |                 {renderMessageContent(message, index)}
252 |             </div>
253 |         </div>
254 |     ))}
255 |     {isLoading && (
256 |         <div className="flex justify-start">
257 |             <div className="bg-[#1E293B] text-slate-200 rounded-lg px-4 py-2">
258 |                 <div className="flex gap-1">
259 |                     <div className="w-2 h-2 bg-gray-400 rounded-full animate-bounce" />
260 |                     <div className="w-2 h-2 bg-gray-400 rounded-full animate-bounce" style={{ animationDelay: '0.2s' }} />
261 |                     <div className="w-2 h-2 bg-gray-400 rounded-full animate-bounce" style={{ animationDelay: '0.4s' }} />
262 |                 </div>
263 |             </div>
264 |         </div>
265 |     )}
266 |     <div ref={messagesEndRef} />
267 | </div>
268 |
269 | {imagePreview && (
270 |     <div className="px-4 py-2 border-t border-slate-700" data-testid="image-preview-area">
271 |         <div className="relative inline-block">
272 |             <img
273 |                 src={imagePreview}
274 |                 alt="Upload preview"
275 |                 className="h-16 w-auto rounded border border-slate-600"
276 |                 data-testid="image-preview-thumbnail"
277 |             />
278 |             <button
279 |                 onClick={clearImage}
280 |                 className="absolute -top-2 -right-2 bg-red-500 hover:bg-red-600 text-white rounded-full w-5 h-5 flex items-center ...
281 |                 data-testid="clear-image-btn"
282 |             >
283 |                 <X className="w-3 h-3" />
284 |             </button>
```

```
285 |         </div>
286 |     </div>
287 | })
288 |
289 |     <div className="border-t border-slate-700 p-3">
290 |         <div className="flex gap-2 mb-2">
291 |             <button
292 |                 onClick={() => {
293 |                     setMessages((prev) => [...prev,
294 |                         { role: 'user', content: '/patterns' },
295 |                         { role: 'assistant', content: '___PATTERN_LIBRARY___' }
296 |                     ]);
297 |                 }}
298 |                 className="text-[10px] px-2 py-1 rounded-full border border-slate-600 text-slate-400 hover:text-blue-400 hover:borde ...
299 |                 data-testid="patterns-shortcut-btn"
300 |             >
301 |                 <BarChart3 className="w-3 h-3" />
302 |                 /patterns
303 |             </button>
304 |             <button
305 |                 onClick={() => setInput('/research ')}
306 |                 className="text-[10px] px-2 py-1 rounded-full border border-slate-600 text-slate-400 hover:text-[#0052FF] hover:bord ...
307 |                 data-testid="research-shortcut-btn"
308 |             >
309 |                 <Building2 className="w-3 h-3" />
310 |                 /research
311 |             </button>
312 |         </div>
313 |         <div className="flex gap-2">
314 |             <input
315 |                 type="file"
316 |                 ref={fileInputRef}
317 |                 onChange={handleImageUpload}
318 |                 accept="image/jpeg,image/png,image/webp"
319 |                 className="hidden"
320 |                 data-testid="image-file-input"
321 |             />
322 |             <Button
323 |                 onClick={() => fileInputRef.current?.click()}
324 |                 variant="outline"
325 |                 size="icon"
326 |                 className="border-slate-600 bg-[#1E293B] hover:bg-[#334155] text-slate-300 shrink-0"
327 |                 title="Upload chart screenshot"
328 |                 data-testid="upload-image-btn"
329 |             >
330 |                 <Image className="w-4 h-4" />
331 |             </Button>
332 |             <Input
333 |                 type="text"
334 |                 placeholder="Ask about stocks, or type /patterns"
335 |                 value={input}
336 |                 onChange={(e) => setInput(e.target.value)}
337 |                 onKeyDown={(e) => e.key === 'Enter' && handleSend()}
338 |                 className="flex-1 bg-[#1E293B] border-slate-600 text-white placeholder-slate-500"
339 |                 data-testid="chat-input"
340 |             />
341 |             <Button
342 |                 onClick={handleSend}
343 |                 disabled={isLoading || (!input.trim() && !imageBase64)}
344 |                 className="bg-[#0052FF] hover:bg-[#2563EB] text-white"
345 |                 data-testid="chat-send-btn"
346 |             >
347 |                 <Send className="w-4 h-4" />
348 |             </Button>
349 |         </div>
350 |     </div>
351 | </Card>
```



```
352 |     </div>
353 |   );
354 | }];
355 |
356 | export default TradeGPTChat;
357 |
```

FILE: frontend/src/components/TradingJournal.jsx

399 lines | 21286 characters

```
1 | import React, { useState, useEffect, useCallback } from 'react';
2 | import { X, Plus, TrendingUp, TrendingDown, Target, BarChart3, BookOpen, Trash2, Paperclip, Lock, RefreshCw, ChevronDown, ChevronU ...
3 | import { Card } from './ui/card';
4 | import { Badge } from './ui/badge';
5 | import { Button } from './ui/button';
6 | import { Input } from './ui/input';
7 | import { useAuth, authFetch } from '../contexts/AuthContext';
8 | import { LineChart, Line, XAxis, YAxis, Tooltip, ResponsiveContainer, Area, AreaChart } from 'recharts';
9 |
10 | const API = `${process.env.REACT_APP_BACKEND_URL}/api`;
11 |
12 | const TradingJournal = ({ onClose, onSubscribe }) => {
13 |   const { user, isPro } = useAuth();
14 |   const [tab, setTab] = useState('trades');
15 |   const [trades, setTrades] = useState([]);
16 |   const [analytics, setAnalytics] = useState(null);
17 |   const [loading, setLoading] = useState(true);
18 |   const [showForm, setShowForm] = useState(false);
19 |   const [tradeLimit, setTradeLimit] = useState(-1);
20 |   const [form, setForm] = useState({
21 |     ticker: '', side: 'buy', entry_price: '', quantity: '', entry_date: new Date().toISOString().split('T')[0],
22 |     exit_price: '', exit_date: '', notes: '',
23 |   });
24 |   const [saving, setSaving] = useState(false);
25 |   const [closeForm, setCloseForm] = useState(null);
26 |
27 |   const fetchTrades = useCallback(async () => {
28 |     setLoading(true);
29 |     try {
30 |       const res = await authFetch(`${API}/journal/trades`);
31 |       if (res.ok) {
32 |         const data = await res.json();
33 |         setTrades(data.trades);
34 |         setTradeLimit(data.limit);
35 |       }
36 |     } catch (e) { console.error(e); }
37 |     finally { setLoading(false); }
38 |   }, []);
39 |
40 |   const fetchAnalytics = useCallback(async () => {
41 |     try {
42 |       const res = await authFetch(`${API}/journal/analytics`);
43 |       if (res.ok) setAnalytics(await res.json());
44 |     } catch (e) { console.error(e); }
45 |   }, []);
46 |
47 |   useEffect(() => { fetchTrades(); fetchAnalytics(); }, [fetchTrades, fetchAnalytics]);
48 |
49 |   const createTrade = async () => {
50 |     if (!form.ticker || !form.entry_price || !form.quantity) return;
51 |     setSaving(true);
52 |     try {
53 |       const body = {
54 |         ...form,
55 |         entry_price: parseFloat(form.entry_price),
56 |         quantity: parseFloat(form.quantity),
```

```

57 |         exit_price: form.exit_price ? parseFloat(form.exit_price) : null,
58 |         exit_date: form.exit_date || null,
59 |     };
60 |     const res = await authFetch(`${API}/journal/trade`, {
61 |         method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify(body),
62 |     });
63 |     if (res.ok) {
64 |         setShowForm(false);
65 |         setForm({ ticker: '', side: 'buy', entry_price: '', quantity: '', entry_date: new Date().toISOString().split('T')[0], exit ...
66 |         fetchTrades();
67 |         fetchAnalytics();
68 |     } else {
69 |         const d = await res.json();
70 |         alert(d.detail || 'Failed to log trade');
71 |     }
72 | } catch (e) { alert('Error logging trade'); }
73 | finally { setSaving(false); }
74 | };
75 |
76 | const closeTrade = async (tradeId) => {
77 |     if (!closeForm?.exit_price) return;
78 |     try {
79 |         const res = await authFetch(`${API}/journal/trade/${tradeId}`, {
80 |             method: 'PUT', headers: { 'Content-Type': 'application/json' },
81 |             body: JSON.stringify({ exit_price: parseFloat(closeForm.exit_price), exit_date: closeForm.exit_date || new Date().toISOStr ...
82 |         });
83 |         if (res.ok) { setCloseForm(null); fetchTrades(); fetchAnalytics(); }
84 |     } catch (e) { console.error(e); }
85 | };
86 |
87 | const deleteTrade = async (id) => {
88 |     try {
89 |         await authFetch(`${API}/journal/trade/${id}`, { method: 'DELETE' });
90 |         fetchTrades();
91 |         fetchAnalytics();
92 |     } catch (e) { console.error(e); }
93 | };
94 |
95 | const attachHypothesis = async (tradeId) => {
96 |     try {
97 |         const res = await authFetch(`${API}/journal/trade/${tradeId}/attach-hypothesis`, { method: 'POST' });
98 |         if (res.ok) fetchTrades();
99 |         else { const d = await res.json(); alert(d.detail || 'No hypothesis found'); }
100 |     } catch (e) { console.error(e); }
101 | };
102 |
103 | const tabs = [
104 |     { id: 'trades', label: 'My Trades', icon: BookOpen },
105 |     { id: 'analytics', label: 'Analytics', icon: BarChart3 },
106 | ];
107 |
108 | return (
109 |     <div className="fixed inset-0 bg-black/70 backdrop-blur-sm z-50 flex items-start justify-center overflow-y-auto p-4" data-test ...
110 |     <div className="bg-slate-900 rounded-2xl max-w-4xl w-full my-4 border border-slate-700/50">
111 |         { /* Header */ }
112 |         <div className="p-5 border-b border-slate-700 flex items-center justify-between">
113 |             <div className="flex items-center gap-3">
114 |                 <div className="w-10 h-10 bg-gradient-to-br from-[#0052FF] to-indigo-600 rounded-xl flex items-center justify-center">
115 |                     <BookOpen className="w-5 h-5 text-white" />
116 |                 </div>
117 |                 <div>
118 |                     <h2 className="text-white text-lg font-bold">Trading Journal</h2>
119 |                     <p className="text-slate-400 text-xs">{trades.length} trade{trades.length !== 1 ? 's' : ''} logged{tradeLimit > 0 ? ...
120 |                 </div>
121 |             </div>
122 |             <div className="flex items-center gap-2">
123 |                 <Button size="sm" className="bg-[#0052FF] text-white text-xs h-8" onClick={() => setShowForm(!showForm)} data-testid=" ...

```

```
124 |         disabled={tradeLimit > 0 && trades.length >= tradeLimit}>
125 |         <Plus className="w-3.5 h-3.5 mr-1" /> Log Trade
126 |     </Button>
127 |     <button onClick={onClose} className="text-slate-400 hover:text-white px-2 text-xl">x</button>
128 |   </div>
129 | </div>
130 |
131 |   {/* Tabs */}
132 |   <div className="flex border-b border-slate-700/50 px-4">
133 |     {tabs.map(t => (
134 |       <button key={t.id} onClick={() => setTab(t.id)} data-testid={`journal-tab-${t.id}`}
135 |         className={`flex items-center gap-2 px-4 py-3 text-sm font-medium border-b-2 transition-all ${
136 |           tab === t.id ? 'text-[#0052FF] border-[#0052FF]' : 'text-slate-400 border-transparent hover:text-slate-300'
137 |         }`}>
138 |         <t.icon className="w-4 h-4" /> {t.label}
139 |       </button>
140 |     ))}
141 |   </div>
142 |
143 |   <div className="p-4">
144 |     {/* Trade limit warning */}
145 |     {tradeLimit > 0 && trades.length >= tradeLimit && (
146 |       <div className="flex items-center gap-2 bg-amber-900/20 border border-amber-800/40 rounded-xl px-4 py-2.5 mb-4" data-t ...
147 |         <Lock className="w-4 h-4 text-amber-400" />
148 |         <p className="text-amber-300 text-xs flex-1">Free limit reached ({tradeLimit} trades). Upgrade to Pro for unlimited. ...
149 |         <Button size="sm" className="bg-[#0052FF] text-white text-xs h-7 px-3" onClick={onSubscribe}>Upgrade</Button>
150 |       </div>
151 |     )}
152 |
153 |     {/* New Trade Form */}
154 |     {showForm && (
155 |       <Card className="bg-slate-800/60 border-slate-700/40 rounded-xl p-4 mb-4 space-y-3" data-testid="trade-form">
156 |         <div className="grid grid-cols-2 sm:grid-cols-4 gap-3">
157 |           <div>
158 |             <label className="text-slate-400 text-[10px] block mb-1">Ticker</label>
159 |             <Input placeholder="AAPL" value={form.ticker} onChange={e => setForm({...form, ticker: e.target.value})}
160 |               className="bg-slate-900 border-slate-600 text-white text-sm h-9 rounded-xl" data-testid="trade-ticker" />
161 |           </div>
162 |           <div>
163 |             <label className="text-slate-400 text-[10px] block mb-1">Side</label>
164 |             <select value={form.side} onChange={e => setForm({...form, side: e.target.value})}
165 |               className="w-full bg-slate-900 border border-slate-600 text-white text-sm h-9 rounded-xl px-2" data-testid="tr ...
166 |             <option value="buy">Buy (Long)</option>
167 |             <option value="sell">Sell (Short)</option>
168 |           </select>
169 |         </div>
170 |         <div>
171 |           <label className="text-slate-400 text-[10px] block mb-1">Entry Price</label>
172 |           <Input type="number" step="0.01" placeholder="150.00" value={form.entry_price} onChange={e => setForm({...form, ...
173 |             className="bg-slate-900 border-slate-600 text-white text-sm h-9 rounded-xl" data-testid="trade-entry" />
174 |         </div>
175 |         <div>
176 |           <label className="text-slate-400 text-[10px] block mb-1">Quantity</label>
177 |           <Input type="number" step="0.01" placeholder="10" value={form.quantity} onChange={e => setForm({...form, quantit ...
178 |             className="bg-slate-900 border-slate-600 text-white text-sm h-9 rounded-xl" data-testid="trade-qty" />
179 |         </div>
180 |       </div>
181 |       <div className="grid grid-cols-2 sm:grid-cols-4 gap-3">
182 |         <div>
183 |           <label className="text-slate-400 text-[10px] block mb-1">Entry Date</label>
184 |           <Input type="date" value={form.entry_date} onChange={e => setForm({...form, entry_date: e.target.value})}
185 |             className="bg-slate-900 border-slate-600 text-white text-sm h-9 rounded-xl" />
186 |         </div>
187 |         <div>
188 |           <label className="text-slate-400 text-[10px] block mb-1">Exit Price (opt.)</label>
189 |           <Input type="number" step="0.01" placeholder="" value={form.exit_price} onChange={e => setForm({...form, exit_p ...
190 |             className="bg-slate-900 border-slate-600 text-white text-sm h-9 rounded-xl" />
```

```
191 |         </div>
192 |         <div>
193 |           <label className="text-slate-400 text-[10px] block mb-1">Exit Date (opt.)</label>
194 |           <Input type="date" value={form.exit_date} onChange={e => setForm({...form, exit_date: e.target.value}})
195 |             className="bg-slate-900 border-slate-600 text-white text-sm h-9 rounded-xl" />
196 |         </div>
197 |         <div>
198 |           <label className="text-slate-400 text-[10px] block mb-1">Notes</label>
199 |           <Input placeholder="Why I took this trade..." value={form.notes} onChange={e => setForm({...form, notes: e.targe ...
200 |             className="bg-slate-900 border-slate-600 text-white text-sm h-9 rounded-xl" />
201 |         </div>
202 |       </div>
203 |       <div className="flex justify-end gap-2">
204 |         <Button size="sm" variant="outline" className="text-xs h-8 bg-slate-700 text-slate-300 border-slate-600" onClick={ ...
205 |         <Button size="sm" className="text-xs h-8 bg-[#0052FF] text-white" onClick={createTrade} disabled={saving || !form. ...
206 |           {saving ? 'Saving...' : 'Log Trade'}
207 |         </Button>
208 |       </div>
209 |     </Card>
210 |   )}
211 |
212 |   {tab === 'trades' ? (
213 |     <TradesTab trades={trades} loading={loading} onDelete={deleteTrade} onClose={closeTrade} onAttach={attachHypothesis}
214 |     closeForm={closeForm} setCloseForm={setCloseForm} />
215 |   ) : (
216 |     <AnalyticsTab analytics={analytics} />
217 |   )}
218 | </div>
219 | </div>
220 | </div>
221 | );
222 | };
223 |
224 | const TradesTab = ({ trades, loading, onDelete, onClose, onAttach, closeForm, setCloseForm }) => {
225 |   if (loading) return <div className="flex justify-center py-12"><RefreshCw className="w-6 h-6 text-[#0052FF] animate-spin" /></di ...
226 |   if (trades.length === 0) return (
227 |     <div className="text-center py-12">
228 |       <BookOpen className="w-12 h-12 text-slate-600 mx-auto mb-3" />
229 |       <p className="text-slate-400 text-sm">No trades logged yet</p>
230 |       <p className="text-slate-500 text-xs mt-1">Click "Log Trade" to start tracking your performance</p>
231 |     </div>
232 |   );
233 |
234 |   return (
235 |     <div className="space-y-2 max-h-[400px] overflow-y-auto">
236 |       {trades.map(t => (
237 |         <div key={t.id} className="bg-slate-800/60 border border-slate-700/40 rounded-xl px-4 py-3" data-testid={`trade-${t.id}`}>
238 |           <div className="flex items-center justify-between mb-1">
239 |             <div className="flex items-center gap-2">
240 |               {t.side === 'buy' ? <TrendingUp className="w-4 h-4 text-emerald-400" /> : <TrendingDown className="w-4 h-4 text-red- ...
241 |               <span className="text-white font-semibold text-sm">{t.ticker}</span>
242 |             <Badge className={`text-[9px]`} ${t.side === 'buy' ? 'bg-emerald-900/30 text-emerald-400 border-emerald-700/50' : 'bg- ...
243 |               {t.side.toUpperCase()}
244 |             </Badge>
245 |             <Badge className={`text-[9px]`} ${t.status === 'closed' ? 'bg-slate-700 text-slate-400' : 'bg-[#0052FF]/20 text-[#0052 ...
246 |               {t.status.toUpperCase()}
247 |             </Badge>
248 |           </div>
249 |           <div className="flex items-center gap-1.5">
250 |             {t.status === 'closed' && (
251 |               <span className={`text-sm font-bold`} ${t.pnl >= 0 ? 'text-emerald-400' : 'text-red-400'}>{t.pnl}>
252 |               {t.pnl >= 0 ? '+' : ''}{t.pnl.toFixed(2)} {t.pnl_percent >= 0 ? '+' : ''}{t.pnl_percent.toFixed(1)}%>
253 |             </span>
254 |           )}
255 |         </div>
256 |       </div>
257 |     <div className="flex items-center gap-4 text-[10px] text-slate-500">
```

```
258 |         <span>Entry: ${t.entry_price?.toFixed(2)}</span>
259 |         {t.exit_price && <span>Exit: ${t.exit_price.toFixed(2)}</span>}
260 |         <span>Qty: {t.quantity}</span>
261 |         <span>{t.entry_date}</span>
262 |     </div>
263 |     {t.notes && <p className="text-slate-400 text-xs mt-1 italic">{t.notes}</p>}
264 |     {t.hypothesis && (
265 |         <div className="mt-2 bg-slate-900/50 rounded-lg px-3 py-2 border border-slate-700/30">
266 |             <p className="text-[10px] text-slate-500 mb-0.5">AI Hypothesis</p>
267 |             <p className="text-xs text-white font-medium">{t.hypothesis.verdict} ({t.hypothesis.confidence}%)</p>
268 |             {t.hypothesis.summary && <p className="text-[10px] text-slate-400 mt-0.5 line-clamp-2">{t.hypothesis.summary}</p>}
269 |         </div>
270 |     )}
271 |     <div className="flex items-center gap-2 mt-2">
272 |         {t.status === 'open' && (
273 |             closeForm?.id === t.id ? (
274 |                 <div className="flex items-center gap-2">
275 |                     <Input type="number" step="0.01" placeholder="Exit price" value={closeForm.exit_price || ''} onChange={e => setC ...
276 |                         className="bg-slate-900 border-slate-600 text-white text-xs h-7 w-28 rounded-lg" />
277 |                     <Input type="date" value={closeForm.exit_date || ''} onChange={e => setCloseForm({...closeForm, exit_date: e.tar ...
278 |                         className="bg-slate-900 border-slate-600 text-white text-xs h-7 w-32 rounded-lg" />
279 |                     <Button size="sm" className="text-[10px] h-7 px-2 bg-emerald-600 text-white" onClick={() => onClose(t.id)}>Close ...
280 |                     <Button size="sm" variant="ghost" className="text-[10px] h-7 px-2 text-slate-400" onClick={() => setCloseForm(nu ...
281 |                 </div>
282 |             ) : (
283 |                 <Button size="sm" variant="outline" className="text-[10px] h-6 px-2 bg-slate-700 text-slate-400 border-slate-600" ...
284 |                 <Target className="w-3 h-3 mr-1" /> Close Trade
285 |             </Button>
286 |         )
287 |     )}
288 |     {!t.hypothesis && (
289 |         <Button size="sm" variant="outline" className="text-[10px] h-6 px-2 bg-indigo-900/30 text-indigo-400 border-indigo-7 ...
290 |         <Paperclip className="w-3 h-3 mr-1" /> Attach AI Hypothesis
291 |     </Button>
292 |     )}
293 |     <Button size="sm" variant="ghost" className="text-[10px] h-6 px-2 text-red-400 hover:text-red-300 ml-auto" onClick={() ...
294 |     <Trash2 className="w-3 h-3" />
295 |     </Button>
296 | </div>
297 | </div>
298 | )}
299 | </div>
300 | );
301 | };
302 |
303 | const AnalyticsTab = ({ analytics }) => {
304 |   if (!analytics || analytics.closed_trades === 0) return (
305 |     <div className="text-center py-12">
306 |       <BarChart3 className="w-12 h-12 text-slate-600 mx-auto mb-3" />
307 |       <p className="text-slate-400 text-sm">No closed trades yet</p>
308 |       <p className="text-slate-500 text-xs mt-1">Close some trades to see your performance analytics</p>
309 |     </div>
310 |   );
311 |
312 |   const a = analytics;
313 |   const stats = [
314 |     { label: 'Total P&L', value: `${a.total_pnl.toFixed(2)}`, color: a.total_pnl >= 0 ? 'text-emerald-400' : 'text-red-400' },
315 |     { label: 'Win Rate', value: `${a.win_rate}%`, color: a.win_rate >= 50 ? 'text-emerald-400' : 'text-red-400' },
316 |     { label: 'Avg Gain', value: `${a.avg_gain.toFixed(2)}`, color: 'text-emerald-400' },
317 |     { label: 'Avg Loss', value: `${a.avg_loss.toFixed(2)}`, color: 'text-red-400' },
318 |     { label: 'Open', value: a.open_trades, color: 'text-[#0052FF]' },
319 |     { label: 'Closed', value: a.closed_trades, color: 'text-white' },
320 |   ];
321 |
322 |   return (
323 |     <div className="space-y-4">
324 |       {/* Stats Grid */}
```

```
325 |         <div className="grid grid-cols-3 sm:grid-cols-6 gap-2">
326 |             {stats.map(s => (
327 |                 <Card key={s.label} className="bg-slate-800/60 border-slate-700/40 rounded-xl p-3 text-center">
328 |                     <p className={`text-lg font-bold ${s.color}`}>{s.value}</p>
329 |                     <p className="text-slate-500 text-[10px]">{s.label}</p>
330 |                 </Card>
331 |             )]}
332 |         </div>
333 |
334 |         {/* P&L Timeline Chart */}
335 |         {a.pnl_timeline.length > 0 && (
336 |             <Card className="bg-slate-800/60 border-slate-700/40 rounded-xl p-4">
337 |                 <h4 className="text-white text-sm font-semibold mb-3">Cumulative P&L</h4>
338 |                 <ResponsiveContainer width="100%" height={200}>
339 |                     <AreaChart data={a.pnl_timeline}>
340 |                         <defs>
341 |                             <linearGradient id="pnlGrad" x1="0" y1="0" x2="0" y2="1">
342 |                                 <stop offset="5%" stopColor="#0052FF" stopOpacity={0.3} />
343 |                                 <stop offset="95%" stopColor="#0052FF" stopOpacity={0} />
344 |                             </linearGradient>
345 |                         </defs>
346 |                         <XAxis dataKey="date" tick={{ fill: '#64748B', fontSize: 10 }} tickLine={false} axisLine={false}
347 |                             tickFormatter={v => v ? new Date(v).toLocaleDateString('en-US', { month: 'short', day: 'numeric' }) : ''} />
348 |                         <YAxis tick={{ fill: '#64748B', fontSize: 10 }} tickLine={false} axisLine={false} tickFormatter={v => `$$${v}`} />
349 |                         <Tooltip contentStyle={{ backgroundColor: '#1E293B', border: '1px solid #334155', borderRadius: '8px', fontSize: '12 ...
350 |                             labelStyle={{ color: '#94A3B8' }} formatter={(v) => [`${v.toFixed(2)}`, 'Cumulative P&L']} />
351 |                         <Area type="monotone" dataKey="cumulative" stroke="#0052FF" fill="url(#pnlGrad)" strokeWidth={2} />
352 |                     </AreaChart>
353 |                 </ResponsiveContainer>
354 |             </Card>
355 |         )}
356 |
357 |         {/* Best/Worst */}
358 |         <div className="grid grid-cols-2 gap-3">
359 |             {a.best_trade && (
360 |                 <Card className="bg-emerald-900/10 border-emerald-700/30 rounded-xl p-3">
361 |                     <p className="text-[10px] text-slate-500 mb-1">Best Trade</p>
362 |                     <p className="text-emerald-400 text-lg font-bold">{a.best_trade.ticker}</p>
363 |                     <p className="text-emerald-300 text-xs">+${a.best_trade.pnl.toFixed(2)} ({a.best_trade.pnl_percent > 0 ? '+' : ''}{a.b ...
364 |                 </Card>
365 |             )}
366 |             {a.worst_trade && (
367 |                 <Card className="bg-red-900/10 border-red-700/30 rounded-xl p-3">
368 |                     <p className="text-[10px] text-slate-500 mb-1">Worst Trade</p>
369 |                     <p className="text-red-400 text-lg font-bold">{a.worst_trade.ticker}</p>
370 |                     <p className="text-red-300 text-xs">${a.worst_trade.pnl.toFixed(2)} ({a.worst_trade.pnl_percent.toFixed(1)}%)</p>
371 |                 </Card>
372 |             )}
373 |         </div>
374 |
375 |         {/* By Ticker */}
376 |         {Object.keys(a.by_ticker).length > 0 && (
377 |             <Card className="bg-slate-800/60 border-slate-700/40 rounded-xl p-4">
378 |                 <h4 className="text-white text-sm font-semibold mb-2">Performance by Ticker</h4>
379 |                 <div className="space-y-1.5">
380 |                     {Object.entries(a.by_ticker).sort((a, b) => b[1].pnl - a[1].pnl).map(([ticker, data]) => (
381 |                         <div key={ticker} className="flex items-center justify-between bg-slate-900/50 rounded-lg px-3 py-2">
382 |                             <div className="flex items-center gap-2">
383 |                                 <span className="text-white text-sm font-medium">{ticker}</span>
384 |                                 <span className="text-slate-500 text-[10px]">{data.trades} trades ({data.wins}W)</span>
385 |                             </div>
386 |                             <span className={`text-sm font-bold ${data.pnl >= 0 ? 'text-emerald-400' : 'text-red-400'}`}>
387 |                                 {data.pnl >= 0 ? '+' : ''}${data.pnl.toFixed(2)}
388 |                             </span>
389 |                         </div>
390 |                     )]}
391 |                 </div>
```

```
392 |         </Card>
393 |     )}
394 | </div>
395 | );
396 | };
397 |
398 | export default TradingJournal;
399 |
```

FILE: frontend/src/components/UserWorkspace.jsx

523 lines | 21510 characters

```
1 | import React, { useState, useEffect, useCallback } from 'react';
2 | import { Briefcase, Star, Clock, TrendingUp, TrendingDown, Minus, Trash2, RefreshCw, X, Search, Plus, Lock, Gift, Copy, Check, Use ...
3 | import { Card } from './ui/card';
4 | import { Badge } from './ui/badge';
5 | import { Button } from './ui/button';
6 | import { Input } from './ui/input';
7 | import { useAuth, authFetch } from '../contexts/AuthContext';
8 | import ReferralLeaderboard from './ReferralLeaderboard';
9 | import SocialShareButtons from './SocialShareButtons';
10 | import { usePushNotifications } from '../hooks/usePushNotifications';
11 |
12 | const API = `${process.env.REACT_APP_BACKEND_URL}/api`;
13 | const FREE_WATCHLIST_LIMIT = 3;
14 |
15 | const UserWorkspace = ({ onClose, onSubscribe }) => {
16 |   const { user, isPro } = useAuth();
17 |   const [tab, setTab] = useState('watchlist');
18 |   const [watchlist, setWatchlist] = useState([]);
19 |   const [history, setHistory] = useState([]);
20 |   const [loading, setLoading] = useState(true);
21 |   const [addTicker, setAddTicker] = useState('');
22 |   const [addLoading, setAddLoading] = useState(false);
23 |   const [addError, setAddError] = useState('');
24 |
25 |   // Stable deps: API and authFetch are module-level constants
26 |   // eslint-disable-next-line react-hooks/exhaustive-deps
27 |   const fetchWatchlist = useCallback(async () => {
28 |     try {
29 |       const res = await authFetch(`${API}/workspace/watchlist`);
30 |       if (res.ok) {
31 |         const data = await res.json();
32 |         setWatchlist(data.tickers || []);
33 |       }
34 |     } catch (e) {
35 |       console.error('Watchlist fetch error:', e);
36 |     }
37 |   }, []);
38 |
39 |   const fetchHistory = useCallback(async () => {
40 |     try {
41 |       const res = await authFetch(`${API}/workspace/history`);
42 |       if (res.ok) {
43 |         const data = await res.json();
44 |         setHistory(data.history || []);
45 |       }
46 |     } catch (e) {
47 |       console.error('History fetch error:', e);
48 |     }
49 |   }, []);
50 |
51 |   useEffect(() => {
52 |     const load = async () => {
53 |       setLoading(true);
54 |       await Promise.all([fetchWatchlist(), fetchHistory()]);
```

```
55 |     setLoading(false);
56 |   };
57 |   load();
58 | }, [fetchWatchlist, fetchHistory]);
59 |
60 | const handleAddTicker = async (e) => {
61 |   e.preventDefault();
62 |   if (!addTicker.trim()) return;
63 |   setAddLoading(true);
64 |   setAddError('');
65 |   try {
66 |     const res = await authFetch(`${API}/workspace/watchlist/add`, {
67 |       method: 'POST',
68 |       body: JSON.stringify({ ticker: addTicker.trim().toUpperCase() }),
69 |     });
70 |     if (res.ok) {
71 |       setAddTicker('');
72 |       await fetchWatchlist();
73 |     } else if (res.status === 403) {
74 |       setAddError(`Free accounts are limited to ${FREE_WATCHLIST_LIMIT} tickers. Upgrade to Pro for unlimited.`);
75 |     }
76 |   } catch (e) {
77 |     console.error('Add ticker error:', e);
78 |   } finally {
79 |     setAddLoading(false);
80 |   }
81 | };
82 |
83 | const removeTicker = async (ticker) => {
84 |   try {
85 |     await authFetch(`${API}/workspace/watchlist/remove`, {
86 |       method: 'POST',
87 |       body: JSON.stringify({ ticker }),
88 |     });
89 |     await fetchWatchlist();
90 |   } catch (e) {
91 |     console.error('Remove ticker error:', e);
92 |   }
93 | };
94 |
95 | const verdictIcon = (v) => {
96 |   if (v === 'BUY') return <TrendingUp className="w-4 h-4 text-emerald-400" />;
97 |   if (v === 'SELL') return <TrendingDown className="w-4 h-4 text-red-400" />;
98 |   return <Minus className="w-4 h-4 text-amber-400" />;
99 | };
100 |
101 | const verdictStyle = (v) => {
102 |   if (v === 'BUY') return 'text-emerald-400 bg-emerald-900/30 border-emerald-700/50';
103 |   if (v === 'SELL') return 'text-red-400 bg-red-900/30 border-red-700/50';
104 |   return 'text-amber-400 bg-amber-900/30 border-amber-700/50';
105 | };
106 |
107 | const tabs = [
108 |   { id: 'watchlist', label: 'Watchlist', icon: Star },
109 |   { id: 'history', label: 'Hypothesis History', icon: Clock },
110 |   { id: 'referrals', label: 'Referrals', icon: Gift },
111 | ];
112 |
113 | return (
114 |   <div className="fixed inset-0 bg-black/70 backdrop-blur-sm z-50 flex items-start justify-center overflow-y-auto p-4" data-test ...
115 |     <div className="bg-slate-900 rounded-2xl max-w-3xl w-full my-4 border border-slate-700/50">
116 |       </* Header */>
117 |       <div className="p-6 border-b border-slate-700 flex items-center justify-between">
118 |         <div className="flex items-center gap-3">
119 |           <div className="w-10 h-10 bg-[#0052FF] rounded-xl flex items-center justify-center">
120 |             <Briefcase className="w-6 h-6 text-white" />
121 |           </div>
```



```
122 |         <div>
123 |             <h2 className="text-white text-xl font-bold" style={{ fontFamily: 'Manrope, sans-serif' }}>My Workspace</h2>
124 |             <p className="text-slate-400 text-sm">{user?.name || user?.email}</p>
125 |         </div>
126 |     </div>
127 |     <div className="flex items-center gap-2">
128 |         {isPro && <Badge className="bg-gradient-to-r from-[#0052FF] to-cyan-500 text-white border-0 text-xs">PRO</Badge>}
129 |         <button onClick={onClose} className="text-slate-400 hover:text-white text-xl px-2" data-testid="workspace-close">
130 |             <X className="w-5 h-5" />
131 |         </button>
132 |     </div>
133 | </div>
134 |
135 |     {/* Tabs */}
136 |     <div className="flex border-b border-slate-700/50">
137 |         {tabs.map(t => {
138 |             const Icon = t.icon;
139 |             return (
140 |                 <button
141 |                     key={t.id}
142 |                     onClick={() => setTab(t.id)}
143 |                     className={`flex-1 flex items-center justify-center gap-2 py-3 text-sm font-medium transition-colors ${
144 |                         tab === t.id
145 |                             ? 'text-[#0052FF] border-b-2 border-[#0052FF] bg-[#0052FF]/5'
146 |                             : 'text-slate-400 hover:text-slate-200'
147 |                     }`}
148 |                     data-testid={`workspace-tab-${t.id}`}
149 |                 >
150 |                     <Icon className="w-4 h-4" />
151 |                     {t.label}
152 |                 </button>
153 |             );
154 |         })}
155 |     </div>
156 |
157 |     {/* Content */}
158 |     <div className="p-4 sm:p-6 min-h-[300px]">
159 |         {loading ? (
160 |             <div className="flex items-center justify-center py-12">
161 |                 <RefreshCw className="w-6 h-6 text-[#0052FF] animate-spin" />
162 |             </div>
163 |         ) : tab === 'watchlist' ? (
164 |             <div className="space-y-4">
165 |                 {/* Add Ticker Form */}
166 |                 <form onSubmit={handleAddTicker} className="flex gap-2" data-testid="add-ticker-form">
167 |                     <div className="relative flex-1">
168 |                         <Search className="absolute left-3 top-1/2 -translate-y-1/2 text-slate-400 w-4 h-4" />
169 |                         <Input
170 |                             placeholder="Add ticker (e.g., AAPL, TSLA)"
171 |                             value={addTicker}
172 |                             onChange={e => setAddTicker(e.target.value.toUpperCase())}
173 |                             className="pl-10 bg-slate-800 border-slate-600 text-white rounded-xl"
174 |                             data-testid="add-ticker-input"
175 |                         />
176 |                     </div>
177 |                     <Button type="submit" disabled={addLoading || !addTicker.trim()} className="bg-[#0052FF] hover:bg-[#2563EB] text-w ...
178 |                         <Plus className="w-4 h-4 mr-1" /> Add
179 |                     </Button>
180 |                 </form>
181 |
182 |                 {/* Watchlist cap indicator for free users */}
183 |                 {!isPro && (
184 |                     <div className="flex items-center justify-between text-xs">
185 |                         <span className="text-slate-500">{watchlist.length}/{FREE_WATCHLIST_LIMIT} free tickers used</span>
186 |                         {watchlist.length >= FREE_WATCHLIST_LIMIT && (
187 |                             <button onClick={onSubscribe} className="text-[#0052FF] hover:underline flex items-center gap-1">
188 |                                 <Lock className="w-3 h-3" /> Upgrade for unlimited
```

```
189 |         </button>
190 |     )}
191 | </div>
192 | )}
193 |
194 |     {addError && (
195 |         <div className="bg-amber-900/20 border border-amber-700/40 rounded-xl px-3 py-2 flex items-center justify-between" ...
196 |         <span className="text-amber-400 text-xs">{addError}</span>
197 |         <Button size="sm" className="bg-[#0052FF] hover:bg-[#2563EB] text-white rounded-lg text-xs h-7 px-3" onClick={on ...
198 |         Upgrade
199 |         </Button>
200 |     </div>
201 |     )}
202 |
203 |     {/* Watchlist */}
204 |     {watchlist.length === 0 ? (
205 |         <div className="text-center py-10">
206 |             <Star className="w-10 h-10 text-slate-600 mx-auto mb-3" />
207 |             <p className="text-slate-400 text-sm">No tickers in your watchlist yet</p>
208 |             <p className="text-slate-500 text-xs mt-1">Add tickers above to start tracking</p>
209 |         </div>
210 |     ) : (
211 |         <div className="grid grid-cols-2 sm:grid-cols-3 gap-2">
212 |             {watchlist.map(ticker => (
213 |                 <div key={ticker} className="flex items-center justify-between bg-slate-800/60 border border-slate-700/40 round ...
214 |                 <span className="text-white font-semibold text-sm">{ticker}</span>
215 |                 <button
216 |                     onClick={() => removeTicker(ticker)}
217 |                     className="text-slate-500 hover:text-red-400 transition-colors opacity-0 group-hover:opacity-100"
218 |                     data-testid={`remove-ticker-${ticker}`}
219 |                 >
220 |                     <Trash2 className="w-3.5 h-3.5" />
221 |                 </button>
222 |             </div>
223 |             )}}
224 |         </div>
225 |     )}
226 | </div>
227 | ) : tab === 'history' ? (
228 |     <div className="space-y-3">
229 |         {history.length === 0 ? (
230 |             <div className="text-center py-10">
231 |                 <Clock className="w-10 h-10 text-slate-600 mx-auto mb-3" />
232 |                 <p className="text-slate-400 text-sm">No hypothesis history yet</p>
233 |                 <p className="text-slate-500 text-xs mt-1">Generate an AI Hypothesis to see it here</p>
234 |             </div>
235 |         ) : (
236 |             history.map((h, i) => (
237 |                 <div key={` ${h.symbol}-${h.searched_at} || i`} className="flex items-center justify-between bg-slate-800/60 bord ...
238 |                 <div className="flex items-center gap-3">
239 |                     {verdictIcon(h.verdict)}
240 |                     <div>
241 |                         <span className="text-white font-semibold text-sm">{h.symbol}</span>
242 |                         <p className="text-slate-500 text-xs">
243 |                             {h.searched_at ? new Date(h.searched_at).toLocaleDateString('en-US', { month: 'short', day: 'numeric', h ...
244 |                         </p>
245 |                     </div>
246 |                 </div>
247 |                 <div className="flex items-center gap-2">
248 |                     {h.verdict && (
249 |                         <Badge className={`text-[10px] border ${verdictStyle(h.verdict)} `}>
250 |                             {h.verdict}
251 |                         </Badge>
252 |                     )}
253 |                     {h.confidence > 0 && (
254 |                         <span className="text-slate-400 text-xs font-mono">{h.confidence}%</span>
255 |                     )}
```

```
256 |         </div>
257 |     </div>
258 |     ))
259 |   })
260 | </div>
261 |   ) : tab === 'referrals' ? (
262 |     <ReferralsTab />
263 |   ) : null}
264 | </div>
265 | </div>
266 | </div>
267 | );
268 | };
269 |
270 | const ReferralsTab = () => {
271 |   const [info, setInfo] = useState(null);
272 |   const [loading, setLoading] = useState(true);
273 |   const [copied, setCopied] = useState(false);
274 |
275 |   useEffect(() => {
276 |     const load = async () => {
277 |       try {
278 |         const res = await authFetch(`${API}/referral/info`);
279 |         if (res.ok) setInfo(await res.json());
280 |       } catch (e) {
281 |         console.error('Referral info error:', e);
282 |       } finally {
283 |         setLoading(false);
284 |       }
285 |     };
286 |     load();
287 |   }, []);
288 |
289 |   const referralLink = info ? `${window.location.origin}?ref=${info.code}` : '';
290 |
291 |   const copyLink = () => {
292 |     navigator.clipboard.writeText(referralLink);
293 |     setCopied(true);
294 |     setTimeout(() => setCopied(false), 2000);
295 |   };
296 |
297 |   if (loading) {
298 |     return (
299 |       <div className="flex items-center justify-center py-12">
300 |         <RefreshCw className="w-6 h-6 text-[#0052FF] animate-spin" />
301 |       </div>
302 |     );
303 |   }
304 |
305 |   if (!info) {
306 |     return <div className="text-center py-10 text-slate-400 text-sm">Unable to load referral info</div>;
307 |   }
308 |
309 |   return (
310 |     <div className="space-y-5" data-testid="referrals-tab">
311 |       </* Share Link */>
312 |       <Card className="bg-gradient-to-br from-[#0052FF]/10 to-cyan-900/10 border-[#0052FF]/30 rounded-xl p-4">
313 |         <div className="flex items-center gap-2 mb-3">
314 |           <Gift className="w-5 h-5 text-[#0052FF]" />
315 |           <h3 className="text-white text-sm font-semibold">Share & Earn</h3>
316 |         </div>
317 |         <p className="text-slate-400 text-xs mb-3">
318 |           Invite friends to RISEDUAL AI. When they subscribe to Pro, you earn <strong className="text-white">1 free month</strong> ...
319 |         </p>
320 |         <div className="flex gap-2">
321 |           <Input
322 |             readOnly
```

```
323 |         value={referralLink}
324 |         className="bg-slate-800 border-slate-600 text-white text-xs rounded-xl flex-1"
325 |         data-testid="referral-link-input"
326 |     />
327 |     <Button
328 |       onClick={copyLink}
329 |       className={`rounded-xl px-4 text-sm ${copied ? 'bg-emerald-600' : 'bg-[#0052FF] hover:bg-[#2563EB]'} text-white`}
330 |       data-testid="copy-referral-btn"
331 |     >
332 |       {copied ? <><Check className="w-4 h-4 mr-1" /> Copied</> : <><Copy className="w-4 h-4 mr-1" /> Copy</>}
333 |     </Button>
334 |   </div>
335 |   <p className="text-slate-500 text-[10px] mt-2">Your code: <span className="text-white font-mono">{info.code}</span></p>
336 |   <div className="mt-3">
337 |     <SocialShareButtons referralLink={referralLink} />
338 |   </div>
339 | </Card>
340 |
341 | {/* Stats */}
342 | <div className="grid grid-cols-3 gap-3">
343 |   <Card className="bg-slate-800/60 border-slate-700/40 rounded-xl p-3 text-center">
344 |     <p className="text-2xl font-bold text-white">{info.total_referrals}</p>
345 |     <p className="text-slate-400 text-[10px]">Total Referrals</p>
346 |   </Card>
347 |   <Card className="bg-slate-800/60 border-slate-700/40 rounded-xl p-3 text-center">
348 |     <p className="text-2xl font-bold text-emerald-400">{info.rewards_earned}</p>
349 |     <p className="text-slate-400 text-[10px]">Months Earned</p>
350 |   </Card>
351 |   <Card className="bg-slate-800/60 border-slate-700/40 rounded-xl p-3 text-center">
352 |     <p className="text-2xl font-bold text-[#0052FF]">{info.rewards_remaining}</p>
353 |     <p className="text-slate-400 text-[10px]">Remaining ({info.reward_cap}/yr)</p>
354 |   </Card>
355 | </div>
356 |
357 | {/* Referral History */}
358 | <div>
359 |   <h4 className="text-white text-sm font-semibold mb-2 flex items-center gap-2">
360 |     <Users className="w-4 h-4 text-slate-400" /> Referral History
361 |   </h4>
362 |   {info.referrals.length === 0 ? (
363 |     <div className="text-center py-8">
364 |       <Users className="w-10 h-10 text-slate-600 mx-auto mb-3" />
365 |       <p className="text-slate-400 text-sm">No referrals yet</p>
366 |       <p className="text-slate-500 text-xs mt-1">Share your link to start earning free months</p>
367 |     </div>
368 |   ) : (
369 |     <div className="space-y-2 max-h-[250px] overflow-y-auto">
370 |       {info.referrals.map((ref, i) => (
371 |         <div key={`ref-${i}`} className="flex items-center justify-between bg-slate-800/60 border border-slate-700/40 rounde ...
372 |         <div>
373 |           <p className="text-white text-sm">{ref.referred_email}</p>
374 |           <p className="text-slate-500 text-[10px]">
375 |             {ref.created_at ? new Date(ref.created_at).toLocaleDateString('en-US', { month: 'short', day: 'numeric', year: ...
376 |           </p>
377 |         </div>
378 |         <div className="flex items-center gap-2">
379 |           <Badge className={`text-[10px] border ${
380 |             ref.status === 'completed' ? 'text-emerald-400 bg-emerald-900/30 border-emerald-700/50' : 'text-amber-400 bg-a ...
381 |           }`}>
382 |             {ref.status === 'completed' ? 'Subscribed' : 'Pending'}
383 |           </Badge>
384 |           {ref.reward_granted && (
385 |             <Badge className="text-[10px] bg-[#0052FF]/20 text-[#0052FF] border-[#0052FF]/30">+1 Month</Badge>
386 |           )}
387 |         </div>
388 |       </div>
389 |     )})}
```

```
390 |         </div>
391 |     }}
392 | </div>
393 |     { /* Leaderboard */ }
394 |     <ReferralLeaderboard />
395 |
396 |     { /* Digest Email Preferences */ }
397 |     <DigestToggle />
398 |
399 |     { /* Push Notifications */ }
400 |     <PushToggle />
401 | </div>
402 | );
403 | };
404 |
405 | const DigestToggle = () => {
406 |   const [subscribed, setSubscribed] = useState(true);
407 |   const [loading, setLoading] = useState(true);
408 |   const [togglng, setTogglng] = useState(false);
409 |
410 |   useEffect(() => {
411 |     const load = async () => {
412 |       try {
413 |         const res = await authFetch(`${API}/digest/status`);
414 |         if (res.ok) {
415 |           const data = await res.json();
416 |           setSubscribed(data.subscribed);
417 |         }
418 |       } catch (e) { /* ignore */ }
419 |       finally { setLoading(false); }
420 |     };
421 |     load();
422 |   }, []);
423 |
424 |   const toggle = async () => {
425 |     setTogglng(true);
426 |     try {
427 |       const endpoint = subscribed ? 'opt-out' : 'opt-in';
428 |       const res = await authFetch(`${API}/digest/${endpoint}`, { method: 'POST' });
429 |       if (res.ok) setSubscribed(!subscribed);
430 |     } catch (e) { /* ignore */ }
431 |     finally { setTogglng(false); }
432 |   };
433 |
434 |   if (loading) return null;
435 |
436 |   return (
437 |     <Card className="bg-slate-800/40 border-slate-700/30 rounded-xl p-4" data-testid="digest-toggle">
438 |       <div className="flex items-center justify-between">
439 |         <div className="flex items-center gap-3">
440 |           <div className="w-8 h-8 rounded-lg bg-[#0052FF]/10 flex items-center justify-center">
441 |             <Mail className="w-4 h-4 text-[#0052FF]" />
442 |           </div>
443 |           <div>
444 |             <p className="text-white text-sm font-medium">Daily Market Digest</p>
445 |             <p className="text-slate-500 text-[10px]">Morning briefing at 6:00 AM UTC</p>
446 |           </div>
447 |         </div>
448 |         <button
449 |           onClick={toggle}
450 |           disabled={togglng}
451 |           className={`flex items-center gap-1.5 px-3 py-1.5 rounded-lg text-xs font-medium transition-all ${
452 |             subscribed
453 |               ? 'bg-emerald-900/30 text-emerald-400 border border-emerald-700/50 hover:bg-emerald-800/40'
454 |               : 'bg-slate-700 text-slate-400 border border-slate-600 hover:bg-slate-600'
455 |             }`}
456 |           data-testid="digest-toggle-btn"
```

```
457 |         >
458 |         {subscribed ? <Bell className="w-3.5 h-3.5" /> : <Belloff className="w-3.5 h-3.5" />}
459 |         {subscribed ? 'Subscribed' : 'Unsubscribed'}
460 |     </button>
461 | </div>
462 | </Card>
463 | );
464 | };
465 |
466 | const PushToggle = () => {
467 |     const { permission, subscribed, supported, subscribe, unsubscribe } = usePushNotifications();
468 |     const [togglng, setTogglng] = useState(false);
469 |     const { isPro } = useAuth();
470 |
471 |     if (!supported) return null;
472 |
473 |     const handleToggle = async () => {
474 |         setTogglng(true);
475 |         try {
476 |             if (subscribed) await unsubscribe();
477 |             else await subscribe();
478 |         } finally {
479 |             setTogglng(false);
480 |         }
481 |     };
482 |
483 |     return (
484 |         <Card className="bg-slate-800/40 border-slate-700/30 rounded-xl p-4" data-testid="push-toggle">
485 |             <div className="flex items-center justify-between">
486 |                 <div className="flex items-center gap-3">
487 |                     <div className="w-8 h-8 rounded-lg bg-purple-500/10 flex items-center justify-center">
488 |                         <Bell className="w-4 h-4 text-purple-400" />
489 |                     </div>
490 |                     <div>
491 |                         <p className="text-white text-sm font-medium">Push Notifications</p>
492 |                         <p className="text-slate-500 text-[10px]">
493 |                             {isPro ? 'All alerts: predictions, dark pool, watchlist, signals' : '1 alert/day (Pro: unlimited)'}
494 |                         </p>
495 |                     </div>
496 |                 </div>
497 |                 <button
498 |                     onClick={handleToggle}
499 |                     disabled={togglng || permission === 'denied'}
500 |                     className={`flex items-center gap-1.5 px-3 py-1.5 rounded-lg text-xs font-medium transition-all ${
501 |                         permission === 'denied'
502 |                         ? 'bg-red-900/20 text-red-400 border border-red-800/40 cursor-not-allowed'
503 |                         : subscribed
504 |                         ? 'bg-emerald-900/30 text-emerald-400 border border-emerald-700/50 hover:bg-emerald-800/40'
505 |                         : 'bg-slate-700 text-slate-400 border border-slate-600 hover:bg-slate-600'
506 |                     }`}
507 |                     data-testid="push-toggle-btn"
508 |                 >
509 |                     {permission === 'denied' ? (
510 |                         <<Belloff className="w-3.5 h-3.5" /> Blocked</>
511 |                     ) : subscribed ? (
512 |                         <<Bell className="w-3.5 h-3.5" /> Enabled</>
513 |                     ) : (
514 |                         <<Belloff className="w-3.5 h-3.5" /> Enable</>
515 |                     )}
516 |                 </button>
517 |             </div>
518 |         </Card>
519 |     );
520 | };
521 |
522 | export default UserWorkspace;
523 |
```

FILE: frontend/src/components/Watchlist.jsx

156 lines | 6441 characters

```
1 | import React, { useState, useEffect } from 'react';
2 | import { Star, X, Plus, TrendingUp, TrendingDown, Lock } from 'lucide-react';
3 | import { Button } from './ui/button';
4 | import { Input } from './ui/input';
5 | import { Card } from './ui/card';
6 | import { useAuth } from '../contexts/AuthContext';
7 |
8 | const FREE_WATCHLIST_LIMIT = 3;
9 |
10 | const Watchlist = ({ onSubscribe }) => {
11 |   const { user, isPro } = useAuth();
12 |   const [watchlist, setWatchlist] = useState([]);
13 |   const [newSymbol, setNewSymbol] = useState('');
14 |   const [isExpanded, setIsExpanded] = useState(false);
15 |   const [capWarning, setCapWarning] = useState('');
16 |
17 |   useEffect(() => {
18 |     const saved = localStorage.getItem('risedualai_watchlist');
19 |     if (saved) {
20 |       setWatchlist(JSON.parse(saved));
21 |     }
22 |
23 |     const handleAdd = (e) => {
24 |       if (!e.detail) return;
25 |       const symbol = e.detail.toUpperCase().trim();
26 |       setWatchlist((prev) => {
27 |         if (prev.find(item => item.symbol === symbol)) return prev;
28 |         const updated = [...prev, { symbol, addedAt: new Date().toISOString(), price: 0, change: 0, changePercent: 0 }];
29 |         localStorage.setItem('risedualai_watchlist', JSON.stringify(updated));
30 |         return updated;
31 |       });
32 |       setIsExpanded(true);
33 |     };
34 |     window.addEventListener('risedualai-add-watchlist', handleAdd);
35 |     return () => window.removeEventListener('risedualai-add-watchlist', handleAdd);
36 |   }, []);
37 |
38 |   const addSymbol = () => {
39 |     if (!newSymbol.trim()) return;
40 |     const symbol = newSymbol.toUpperCase().trim();
41 |     if (watchlist.find(item => item.symbol === symbol)) {
42 |       setCapWarning('Symbol already in watchlist');
43 |       setTimeout(() => setCapWarning(''), 3000);
44 |       return;
45 |     }
46 |     if (user && !isPro && watchlist.length >= FREE_WATCHLIST_LIMIT) {
47 |       setCapWarning(`Free accounts are limited to ${FREE_WATCHLIST_LIMIT} tickers. Upgrade to Pro for unlimited.`);
48 |       return;
49 |     }
50 |     const newItem = { symbol, addedAt: new Date().toISOString(), price: 0, change: 0, changePercent: 0 };
51 |     const updated = [...watchlist, newItem];
52 |     setWatchlist(updated);
53 |     localStorage.setItem('risedualai_watchlist', JSON.stringify(updated));
54 |     setNewSymbol('');
55 |     setCapWarning('');
56 |   };
57 |
58 |   const removeSymbol = (symbol) => {
59 |     const updated = watchlist.filter(item => item.symbol !== symbol);
60 |     setWatchlist(updated);
61 |     localStorage.setItem('risedualai_watchlist', JSON.stringify(updated));
62 |   };
63 |
```

```
64 |   return (
65 |     <Card className="bg-slate-800/50 border-slate-700/50 rounded-xl p-4">
66 |       <div className="flex items-center justify-between mb-4">
67 |         <div className="flex items-center gap-2">
68 |           <Star className="w-5 h-5 text-yellow-500 fill-yellow-500" />
69 |           <h3 className="text-white font-semibold">My Watchlist</h3>
70 |           <span className="text-slate-500 text-sm">({watchlist.length}{user && !isPro ? `/${FREE_WATCHLIST_LIMIT}` : ''})</span>
71 |         </div>
72 |         <Button
73 |           variant="ghost"
74 |           size="sm"
75 |           onClick={() => setIsExpanded(!isExpanded)}
76 |           className="text-slate-400 hover:text-slate-50"
77 |         >
78 |           {isExpanded ? 'Collapse' : 'Expand'}
79 |         </Button>
80 |       </div>
81 |
82 |       {isExpanded && (
83 |         <>
84 |           {/* Add Symbol Input */}
85 |           <div className="flex gap-2 mb-2">
86 |             <Input
87 |               placeholder="Add symbol (e.g., AAPL)"
88 |               value={newSymbol}
89 |               onChange={(e) => setNewSymbol(e.target.value)}
90 |               onPress={(e) => e.key === 'Enter' && addSymbol()}
91 |               className="bg-[#1E293B] border-slate-600 text-white"
92 |               data-testid="watchlist-input"
93 |             />
94 |             <Button onClick={addSymbol} className="bg-[#0052FF] hover:bg-[#2563EB]" data-testid="watchlist-add-btn">
95 |               <Plus className="w-4 h-4" />
96 |             </Button>
97 |           </div>
98 |           {capWarning && (
99 |             <div className="flex items-center gap-2 mb-3 bg-amber-900/20 border border-amber-800/40 rounded-lg px-3 py-2" data-tes ...
100 |               <Lock className="w-3.5 h-3.5 text-amber-400 flex-shrink-0" />
101 |               <p className="text-amber-300 text-xs flex-1">{capWarning}</p>
102 |               {onSubscribe && <Button size="sm" className="bg-[#0052FF] text-white text-xs h-6 px-2 rounded-lg" onClick={onSubscri ...
103 |             </div>
104 |           )}
105 |
106 |           {/* Watchlist Items */}
107 |           <div className="space-y-2 max-h-96 overflow-y-auto">
108 |             {watchlist.length === 0 ? (
109 |               <div className="text-center py-8 text-slate-500">
110 |                 <Star className="w-12 h-12 mx-auto mb-2 opacity-20" />
111 |                 <p>Your watchlist is empty</p>
112 |                 <p className="text-sm">Add symbols to track them</p>
113 |               </div>
114 |             ) : (
115 |               watchlist.map((item) => (
116 |                 <div
117 |                   key={item.symbol}
118 |                   className="flex items-center justify-between p-3 bg-[#1E293B] rounded-lg hover:bg-slate-700 transition-colors"
119 |                 >
120 |                   <div className="flex items-center gap-3">
121 |                     <span className="text-white font-medium">{item.symbol}</span>
122 |                     {item.changePercent !== 0 && (
123 |                       <div className={`flex items-center gap-1 text-sm ${
124 |                         item.changePercent >= 0 ? 'text-emerald-400' : 'text-red-400'
125 |                       }`}>
126 |                         {item.changePercent >= 0 ? (
127 |                           <TrendingUp className="w-3 h-3" />
128 |                         ) : (
129 |                           <TrendingDown className="w-3 h-3" />
130 |                         )}
131 |                       </div>
132 |                     )}
133 |                   </div>
134 |                 </div>
135 |               )
136 |             )}
137 |           </div>
138 |         </>
139 |       )
140 |     )
141 |   )
142 | }
```



```
131 |         {Math.abs(item.changePercent).toFixed(2)}%
132 |     </div>
133 |   })
134 | </div>
135 |   <div className="flex items-center gap-2">
136 |     {item.price > 0 && (
137 |       <span className="text-slate-300 text-sm">${item.price.toFixed(2)}</span>
138 |     )}
139 |     <button
140 |       onClick={() => removeSymbol(item.symbol)}
141 |       className="text-slate-500 hover:text-red-400 transition-colors"
142 |     >
143 |       <X className="w-4 h-4" />
144 |     </button>
145 |   </div>
146 | </div>
147 |   ))
148 | })
149 | </div>
150 | </>
151 |   )}
152 | </Card>
153 | );
154 | };
155 |
156 | export default Watchlist;
```

9. Frontend - Configuration & PWA

FILE: frontend/package.json

92 lines | 3014 characters

```
1 | {
2 |   "name": "frontend",
3 |   "version": "0.1.0",
4 |   "private": true,
5 |   "dependencies": {
6 |     "@hookform/resolvers": "^5.0.1",
7 |     "@radix-ui/react-accordion": "^1.2.8",
8 |     "@radix-ui/react-alert-dialog": "^1.1.11",
9 |     "@radix-ui/react-aspect-ratio": "^1.1.4",
10 |    "@radix-ui/react-avatar": "^1.1.7",
11 |    "@radix-ui/react-checkbox": "^1.2.3",
12 |    "@radix-ui/react-collapsible": "^1.1.8",
13 |    "@radix-ui/react-context-menu": "^2.2.12",
14 |    "@radix-ui/react-dialog": "^1.1.11",
15 |    "@radix-ui/react-dropdown-menu": "^2.1.12",
16 |    "@radix-ui/react-hover-card": "^1.1.11",
17 |    "@radix-ui/react-label": "^2.1.4",
18 |    "@radix-ui/react-menubar": "^1.1.12",
19 |    "@radix-ui/react-navigation-menu": "^1.2.10",
20 |    "@radix-ui/react-popover": "^1.1.11",
21 |    "@radix-ui/react-progress": "^1.1.4",
22 |    "@radix-ui/react-radio-group": "^1.3.4",
23 |    "@radix-ui/react-scroll-area": "^1.2.6",
24 |    "@radix-ui/react-select": "^2.2.2",
25 |    "@radix-ui/react-separator": "^1.1.4",
26 |    "@radix-ui/react-slider": "^1.3.2",
27 |    "@radix-ui/react-slot": "^1.2.0",
28 |    "@radix-ui/react-switch": "^1.2.2",
29 |    "@radix-ui/react-tabs": "^1.1.9",
30 |    "@radix-ui/react-toast": "^1.2.11",
31 |    "@radix-ui/react-toggle": "^1.1.6",
32 |    "@radix-ui/react-toggle-group": "^1.1.7",
33 |    "@radix-ui/react-tooltip": "^1.2.4",
34 |    "axios": "^1.8.4",
35 |    "class-variance-authority": "^0.7.1",
36 |    "clsx": "^2.1.1",
37 |    "cmdk": "^1.1.1",
38 |    "cra-template": "1.2.0",
39 |    "date-fns": "^4.1.0",
40 |    "embla-carousel-react": "^8.6.0",
41 |    "input-otp": "^1.4.2",
42 |    "lucide-react": "^0.507.0",
43 |    "next-themes": "^0.4.6",
44 |    "react": "^19.0.0",
45 |    "react-day-picker": "8.10.1",
46 |    "react-dom": "^19.0.0",
47 |    "react-hook-form": "^7.56.2",
48 |    "react-resizable-panels": "^3.0.1",
49 |    "react-router-dom": "^7.5.1",
50 |    "react-scripts": "5.0.1",
51 |    "recharts": "^3.6.0",
52 |    "sonner": "^2.0.3",
53 |    "tailwind-merge": "^3.2.0",
54 |    "tailwindcss-animate": "^1.0.7",
55 |    "vaul": "^1.1.2",
56 |    "zod": "^3.24.4"
57 |   },
58 |   "scripts": {
```

```
59 |     "start": "craco start",
60 |     "build": "craco build",
61 |     "test": "craco test"
62 |   },
63 |   "browserslist": {
64 |     "production": [
65 |       ">0.2%",
66 |       "not dead",
67 |       "not op_mini all"
68 |     ],
69 |     "development": [
70 |       "last 1 chrome version",
71 |       "last 1 firefox version",
72 |       "last 1 safari version"
73 |     ]
74 |   },
75 |   "devDependencies": {
76 |     "@babel/plugin-proposal-private-property-in-object": "^7.21.11",
77 |     "@craco/craco": "^7.1.0",
78 |     "@emergentbase/visual-edits": "https://assets.emergent.sh/npm/emergentbase-visual-edits-1.0.8.tgz",
79 |     "@eslint/js": "9.23.0",
80 |     "autoprefixer": "^10.4.20",
81 |     "eslint": "9.23.0",
82 |     "eslint-plugin-import": "2.31.0",
83 |     "eslint-plugin-jsx-a11y": "6.10.2",
84 |     "eslint-plugin-react": "7.37.4",
85 |     "eslint-plugin-react-hooks": "5.2.0",
86 |     "globals": "15.15.0",
87 |     "postcss": "^8.4.49",
88 |     "tailwindcss": "^3.4.17"
89 |   },
90 |   "packageManager": "yarn@1.22.22+sha512.a6b2f7906b721bba3d67d4aff083df04dad64c399707841b7acf00f6b133b7ac24255f2652fa22ae3534329dc ..."
91 | }
92 |
```

FILE: frontend/tailwind.config.js

82 lines | 2013 characters

```
1 | /** @type {import('tailwindcss').Config} */
2 | module.exports = {
3 |   darkMode: ["class"],
4 |   content: [
5 |     "./src/**/*.{js,jsx,ts,tsx}",
6 |     "./public/index.html"
7 |   ],
8 |   theme: {
9 |     extend: {
10 |       borderRadius: {
11 |         lg: 'var(--radius)',
12 |         md: 'calc(var(--radius) - 2px)',
13 |         sm: 'calc(var(--radius) - 4px)'
14 |       },
15 |       colors: {
16 |         background: 'hsl(var(--background))',
17 |         foreground: 'hsl(var(--foreground))',
18 |         card: {
19 |           DEFAULT: 'hsl(var(--card))',
20 |           foreground: 'hsl(var(--card-foreground))'
21 |         },
22 |         popover: {
23 |           DEFAULT: 'hsl(var(--popover))',
24 |           foreground: 'hsl(var(--popover-foreground))'
25 |         },
26 |         primary: {
27 |           DEFAULT: 'hsl(var(--primary))',
28 |           foreground: 'hsl(var(--primary-foreground))'
```

```
29 |         },
30 |         secondary: {
31 |             DEFAULT: 'hsl(var(--secondary))',
32 |             foreground: 'hsl(var(--secondary-foreground))'
33 |         },
34 |         muted: {
35 |             DEFAULT: 'hsl(var(--muted))',
36 |             foreground: 'hsl(var(--muted-foreground))'
37 |         },
38 |         accent: {
39 |             DEFAULT: 'hsl(var(--accent))',
40 |             foreground: 'hsl(var(--accent-foreground))'
41 |         },
42 |         destructive: {
43 |             DEFAULT: 'hsl(var(--destructive))',
44 |             foreground: 'hsl(var(--destructive-foreground))'
45 |         },
46 |         border: 'hsl(var(--border))',
47 |         input: 'hsl(var(--input))',
48 |         ring: 'hsl(var(--ring))',
49 |         chart: {
50 |             '1': 'hsl(var(--chart-1))',
51 |             '2': 'hsl(var(--chart-2))',
52 |             '3': 'hsl(var(--chart-3))',
53 |             '4': 'hsl(var(--chart-4))',
54 |             '5': 'hsl(var(--chart-5))'
55 |         }
56 |     },
57 |     keyframes: {
58 |         'accordion-down': {
59 |             from: {
60 |                 height: '0'
61 |             },
62 |             to: {
63 |                 height: 'var(--radix-accordion-content-height)'
64 |             }
65 |         },
66 |         'accordion-up': {
67 |             from: {
68 |                 height: 'var(--radix-accordion-content-height)'
69 |             },
70 |             to: {
71 |                 height: '0'
72 |             }
73 |         }
74 |     },
75 |     animation: {
76 |         'accordion-down': 'accordion-down 0.2s ease-out',
77 |         'accordion-up': 'accordion-up 0.2s ease-out'
78 |     }
79 | },
80 | },
81 | plugins: [require("tailwindcss-animate")],
82 | };
```

FILE: frontend/postcss.config.js

7 lines | 82 characters

```
1 | module.exports = {
2 |   plugins: {
3 |     tailwindcss: {},
4 |     autoprefixer: {},
5 |   },
6 | }
7 |
```

FILE: frontend/craco.config.js

101 lines | 2916 characters

```
1 | // craco.config.js
2 | const path = require("path");
3 | require("dotenv").config();
4 |
5 | // Check if we're in development/preview mode (not production build)
6 | // Craco sets NODE_ENV=development for start, NODE_ENV=production for build
7 | const isDevServer = process.env.NODE_ENV !== "production";
8 |
9 | // Environment variable overrides
10 | const config = {
11 |   enableHealthCheck: process.env.ENABLE_HEALTH_CHECK === "true",
12 | };
13 |
14 | // Conditionally load health check modules only if enabled
15 | let WebpackHealthPlugin;
16 | let setupHealthEndpoints;
17 | let healthPluginInstance;
18 |
19 | if (config.enableHealthCheck) {
20 |   WebpackHealthPlugin = require("./plugins/health-check/webpack-health-plugin");
21 |   setupHealthEndpoints = require("./plugins/health-check/health-endpoints");
22 |   healthPluginInstance = new WebpackHealthPlugin();
23 | }
24 |
25 | let webpackConfig = {
26 |   eslint: {
27 |     configure: {
28 |       extends: ["plugin:react-hooks/recommended"],
29 |       rules: {
30 |         "react-hooks/rules-of-hooks": "error",
31 |         "react-hooks/exhaustive-deps": "warn",
32 |       },
33 |     },
34 |   },
35 |   webpack: {
36 |     alias: {
37 |       '@': path.resolve(__dirname, 'src'),
38 |     },
39 |     configure: (webpackConfig) => {
40 |
41 |       // Add ignored patterns to reduce watched directories
42 |       webpackConfig.watchOptions = {
43 |         ...webpackConfig.watchOptions,
44 |         ignored: [
45 |           '**/node_modules/**',
46 |           '**/.git/**',
47 |           '**/build/**',
48 |           '**/dist/**',
49 |           '**/coverage/**',
50 |           '**/public/**',
51 |         ],
52 |       };
53 |
54 |       // Add health check plugin to webpack if enabled
55 |       if (config.enableHealthCheck && healthPluginInstance) {
56 |         webpackConfig.plugins.push(healthPluginInstance);
57 |       }
58 |       return webpackConfig;
59 |     },
60 |   },
61 | };
62 |
63 | webpackConfig.devServer = (devServerConfig) => {
```

```
64 | // Add health check endpoints if enabled
65 | if (config.enableHealthCheck && setupHealthEndpoints && healthPluginInstance) {
66 |   const originalSetupMiddlewares = devServerConfig.setupMiddlewares;
67 |
68 |   devServerConfig.setupMiddlewares = (middlewares, devServer) => {
69 |     // Call original setup if exists
70 |     if (originalSetupMiddlewares) {
71 |       middlewares = originalSetupMiddlewares(middlewares, devServer);
72 |     }
73 |
74 |     // Setup health endpoints
75 |     setupHealthEndpoints(devServer, healthPluginInstance);
76 |
77 |     return middlewares;
78 |   };
79 | }
80 |
81 | return devServerConfig;
82 | };
83 |
84 | // Wrap with visual edits (automatically adds babel plugin, dev server, and overlay in dev mode)
85 | if (isDevServer) {
86 |   try {
87 |     const { withVisualEdits } = require("@emergentbase/visual-edits/craco");
88 |     webpackConfig = withVisualEdits(webpackConfig);
89 |   } catch (err) {
90 |     if (err.code === 'MODULE_NOT_FOUND' && err.message.includes('@emergentbase/visual-edits/craco')) {
91 |       console.warn(
92 |         "[visual-edits] @emergentbase/visual-edits not installed ? visual editing disabled."
93 |       );
94 |     } else {
95 |       throw err;
96 |     }
97 |   }
98 | }
99 |
100 | module.exports = webpackConfig;
101 |
```

FILE: frontend/jsconfig.json

9 lines | 116 characters

```
1 | {
2 |   "compilerOptions": {
3 |     "baseUrl": ".",
4 |     "paths": {
5 |       "@/*": ["src/*"]
6 |     }
7 |   },
8 |   "include": ["src"]
9 | }
```

FILE: frontend/components.json

21 lines | 444 characters

```
1 | {
2 |   "$schema": "https://ui.shadcn.com/schema.json",
3 |   "style": "new-york",
4 |   "rsc": false,
5 |   "tsx": false,
6 |   "tailwind": {
7 |     "config": "tailwind.config.js",
8 |     "css": "src/index.css",
9 |     "baseColor": "neutral",
10 |     "cssVariables": true,
```

```
11 |     "prefix": ""
12 |   },
13 |   "aliases": {
14 |     "components": "@/components",
15 |     "utils": "@/lib/utils",
16 |     "ui": "@/components/ui",
17 |     "lib": "@/lib",
18 |     "hooks": "@/hooks"
19 |   },
20 |   "iconLibrary": "lucide"
21 | }
```

FILE: frontend/public/manifest.json

27 lines | 708 characters

```
1 | {
2 |   "short_name": "RISEDUAL AI",
3 |   "name": "RISEDUAL AI - AI Trading Platform",
4 |   "icons": [
5 |     {
6 |       "src": "logo-192.png",
7 |       "sizes": "192x192",
8 |       "type": "image/png",
9 |       "purpose": "any maskable"
10 |    },
11 |    {
12 |       "src": "logo-512.png",
13 |       "sizes": "512x512",
14 |       "type": "image/png",
15 |       "purpose": "any maskable"
16 |    }
17 |  ],
18 |   "start_url": ".",
19 |   "display": "standalone",
20 |   "theme_color": "#0F172A",
21 |   "background_color": "#0F172A",
22 |   "orientation": "portrait-primary",
23 |   "scope": "/",
24 |   "description": "Advanced AI-powered trading platform with real-time market data, options analysis, crypto tracking, and intellig ...
25 |   "categories": ["finance", "business"]
26 | }
27 |
```

FILE: frontend/public/service-worker.js

69 lines | 1875 characters

```
1 | const CACHE_NAME = 'risedualai-v1';
2 | const STATIC_ASSETS = [
3 |   '/',
4 |   '/index.html',
5 |   '/manifest.json',
6 |   '/icon-192.png',
7 |   '/icon-512.png',
8 | ];
9 |
10 | self.addEventListener('install', (event) => {
11 |   event.waitUntil(
12 |     caches.open(CACHE_NAME).then((cache) => cache.addAll(STATIC_ASSETS))
13 |   );
14 |   self.skipWaiting();
15 | });
16 |
17 | self.addEventListener('activate', (event) => {
18 |   event.waitUntil(
19 |     caches.keys().then((keys) =>
```

```
20 |     Promise.all(keys.filter((k) => k !== CACHE_NAME).map((k) => caches.delete(k)))
21 |   )
22 |   );
23 |   self.clients.claim();
24 | });
25 |
26 | self.addEventListener('fetch', (event) => {
27 |   if (event.request.url.includes('/api/')) return;
28 |   event.respondWith(
29 |     fetch(event.request).catch(() => caches.match(event.request))
30 |   );
31 | });
32 |
33 | // Push notification handler
34 | self.addEventListener('push', (event) => {
35 |   if (!event.data) return;
36 |   try {
37 |     const data = event.data.json();
38 |     const options = {
39 |       body: data.body || '',
40 |       icon: data.icon || '/logo192.png',
41 |       badge: data.badge || '/logo192.png',
42 |       tag: data.tag || 'risedual',
43 |       data: { url: data.url || '/' },
44 |       vibrate: [100, 50, 100],
45 |       actions: [{ action: 'open', title: 'View' }],
46 |     };
47 |     event.waitUntil(self.registration.showNotification(data.title || 'RISEDUAL AI', options));
48 |   } catch (e) {
49 |     console.error('Push event error:', e);
50 |   }
51 | });
52 |
53 | // Notification click handler
54 | self.addEventListener('notificationclick', (event) => {
55 |   event.notification.close();
56 |   const url = event.notification.data?.url || '/';
57 |   event.waitUntil(
58 |     clients.matchAll({ type: 'window', includeUncontrolled: true }).then((windowClients) => {
59 |       for (const client of windowClients) {
60 |         if (client.url.includes(self.location.origin) && 'focus' in client) {
61 |           client.navigate(url);
62 |           return client.focus();
63 |         }
64 |       }
65 |       return clients.openWindow(url);
66 |     })
67 |   );
68 | });
69 |
```

FILE: frontend/public/index.html

165 lines | 9993 characters

```
1 | <!doctype html>
2 | <html lang="en">
3 |   <head>
4 |     <meta charset="utf-8" />
5 |     <meta name="viewport" content="width=device-width, initial-scale=1, maximum-scale=1, user-scalable=no, viewport-fit=cover" ...
6 |     <meta name="theme-color" content="#0F172A" />
7 |     <meta name="apple-mobile-web-app-capable" content="yes" />
8 |     <meta name="apple-mobile-web-app-status-bar-style" content="black-translucent" />
9 |     <meta name="apple-mobile-web-app-title" content="RISEDUAL AI" />
10 |     <link rel="apple-touch-icon" href="%PUBLIC_URL%/logo-192.png" />
11 |     <link rel="icon" type="image/png" sizes="32x32" href="%PUBLIC_URL%/logo-32.png" />
12 |     <link rel="manifest" href="%PUBLIC_URL%/manifest.json" />
```



```

13 |     <meta name="description" content="RISEDUAL AI - Advanced AI-powered trading platform with real-time market data, options a ...
14 |     <link rel="preconnect" href="https://fonts.googleapis.com" />
15 |     <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />
16 |     <link href="https://fonts.googleapis.com/css2?family=Inter:wght@600&family=Manrope:wght@400;500;600;700;800&family=IBM+Ple ...
17 |     <!--
18 |     manifest.json provides metadata used when your web app is installed on a
19 |     user's mobile device or desktop. See https://developers.google.com/web/fundamentals/web-app-manifest/
20 |     -->
21 |     <!--
22 |     Notice the use of %PUBLIC_URL% in the tags above.
23 |     It will be replaced with the URL of the `public` folder during the build.
24 |     Only files inside the `public` folder can be referenced from the HTML.
25 |
26 |     Unlike "/favicon.ico" or "favicon.ico", "%PUBLIC_URL%/favicon.ico" will
27 |     work correctly both with client-side routing and a non-root public URL.
28 |     Learn how to configure a non-root public URL by running `npm run build`.
29 |     -->
30 |     <title>RISEDUAL AI - AI-Powered Trading Platform</title>
31 |     <script>window.addEventListener("error",function(e){if(e.error instanceof DOMException&&e.error.name==="DataCloneError"&&e ...
32 |     <script src="https://assets.emergent.sh/scripts/emergent-main.js"></script>
33 | </head>
34 | <body>
35 |     <noscript>You need to enable JavaScript to run this app.</noscript>
36 |     <div id="root"></div>
37 |     <!--
38 |     This HTML file is a template.
39 |     If you open it directly in the browser, you will see an empty page.
40 |
41 |     You can add webfonts, meta tags, or analytics to this file.
42 |     The build step will place the bundled scripts into the <body> tag.
43 |
44 |     To begin the development, run `npm start` or `yarn start`.
45 |     To create a production bundle, use `npm run build` or `yarn build`.
46 |     -->
47 |     <a
48 |         id="emergent-badge"
49 |         target="_blank"
50 |         href="https://app.emergent.sh/?utm_source=emergent-badge"
51 |         style="
52 |             display: inline-flex !important;
53 |             box-sizing: border-box;
54 |             width: 178px;
55 |             height: 40px;
56 |             padding: 8px 12px 8px 12px;
57 |             align-items: center !important;
58 |             gap: 8px;
59 |             border-radius: 50px !important;
60 |             background: #000 !important;
61 |             position: fixed !important;
62 |             bottom: 16px;
63 |             right: 16px;
64 |             text-decoration: none;
65 |             font-family: -apple-system, BlinkMacSystemFont,
66 |                 &quot;Segoe UI&quot;, Roboto, Oxygen, Ubuntu, Cantarell,
67 |                 &quot;Open Sans&quot;, &quot;Helvetica Neue&quot;;
68 |             sans-serif !important;
69 |             font-size: 12px !important;
70 |             z-index: 9999 !important;
71 |         "
72 |     >
73 |     <svg xmlns="http://www.w3.org/2000/svg" width="16" height="16" viewBox="0 0 16 16" fill="none">
74 |         <path d="M15.5702 8.13142C15.7729 8.0412 16.0007 8.18878 15.9892 8.4103C15.8374 11.3192 14.0965 14.0405 11.2531 15 ...
75 |         <path fill-rule="evenodd" clip-rule="evenodd" d="M15.3066 4.74698L15.5067 5.19653C15.5759 5.35178 15.5061 5.53366 ...
76 |     </svg>
77 |     <p
78 |         style="
79 |             color: #FFF !important;

```

```
80 |         font-family: 'Inter', sans-serif !important;
81 |         font-size: 13px !important;
82 |         font-style: normal !important;
83 |         font-weight: 600 !important;
84 |         line-height: 20px !important;
85 |         margin: 0 !important;
86 |         white-space: nowrap !important;
87 |     "
88 |     >
89 |         Made with Emergent
90 |     </p>
91 | </a>
92 | <script>
93 |     !(function (t, e) {
94 |         var o, n, p, r;
95 |         e.__SV ||
96 |             ((window.posthog = e),
97 |              (e._i = []),
98 |              (e.init = function (i, s, a) {
99 |                  function g(t, e) {
100 |                      var o = e.split(".");
101 |                      2 == o.length && ((t = t[o[0]]), (e = o[1])),
102 |                      (t[e] = function () {
103 |                          t.push(
104 |                              [e].concat(
105 |                                  Array.prototype.slice.call(
106 |                                      arguments,
107 |                                      0,
108 |                                      ),
109 |                                  ),
110 |                              );
111 |                      });
112 |                  }
113 |                  ((p = t.createElement("script")).type =
114 |                   "text/javascript",
115 |                   (p.crossOrigin = "anonymous"),
116 |                   (p.async = !0),
117 |                   (p.src =
118 |                    s.api_host.replace(
119 |                      ".i.posthog.com",
120 |                      "-assets.i.posthog.com",
121 |                      ) + "/static/array.js"),
122 |                   (r =
123 |                    t.getElementsByTagName(
124 |                      "script",
125 |                      )[0]).parentNode.insertBefore(p, r);
126 |                   var u = e;
127 |                   for (
128 |                       void 0 !== a ? (u = e[a] = []) : (a = "posthog"),
129 |                       u.people = u.people || [],
130 |                       u.toString = function (t) {
131 |                           var e = "posthog";
132 |                           return (
133 |                               "posthog" !== a && (e += "." + a),
134 |                               t || (e += " (stub)"),
135 |                               e
136 |                           );
137 |                       },
138 |                       u.people.toString = function () {
139 |                           return u.toString(1) + ".people (stub)";
140 |                       },
141 |                       o =
142 |                           "init me ws ys ps bs capture je Di ks register register_once register_for_session unregister u ...
143 |                           " ",
144 |                       ),
145 |                       n = 0;
146 |                       n < o.length;
```

```
147 |         n++
148 |     )
149 |     g(u, o[n]);
150 |     e._i.push([i, s, a]);
151 | },
152 |     (e.__SV = 1));
153 | })(document, window.posthog || []);
154 | posthog.init("phc_xAvL2Iq4tFmANRE7kzbKwaSq1HJjN7x48s3vr0CMjs", {
155 |     api_host: "https://us.i.posthog.com",
156 |     person_profiles: "identified_only", // or 'always' to create profiles for anonymous users as well,
157 |     session_recording: {
158 |         recordCrossOriginIframes: true,
159 |         capturePerformance: false,
160 |     },
161 | });
162 | </script>
163 | </body>
164 | </html>
165 |
```

10. Test Suite (Backend)

FILE: backend/tests/test_admin_workspace.py

304 lines | 11820 characters

```
1 | """
2 | Test Admin Panel and User Workspace APIs
3 | - Admin endpoints (owner-only): list users, activate, deactivate, grant-pro, revoke-pro
4 | - Workspace endpoints (authenticated): watchlist CRUD, hypothesis history
5 | """
6 | import pytest
7 | import requests
8 | import os
9 |
10 | BASE_URL = os.environ.get('REACT_APP_BACKEND_URL', '').rstrip('/')
11 |
12 | # Test credentials from environment
13 | OWNER_EMAIL = os.environ.get("OWNER_EMAIL", "managingdirector@redslateholdings.com")
14 | OWNER_PASSWORD = os.environ.get("OWNER_PASSWORD", "")
15 | ADMIN_EMAIL = os.environ.get("ADMIN_EMAIL", "admin@risedual.ai")
16 | ADMIN_PASSWORD = os.environ.get("ADMIN_PASSWORD", "")
17 |
18 |
19 | @pytest.fixture(scope="module")
20 | def owner_token():
21 |     """Get owner access token"""
22 |     res = requests.post(f"{BASE_URL}/api/auth/login", json={
23 |         "email": OWNER_EMAIL,
24 |         "password": OWNER_PASSWORD
25 |     })
26 |     if res.status_code != 200:
27 |         pytest.skip(f"Owner login failed: {res.status_code} - {res.text}")
28 |     return res.json().get("access_token")
29 |
30 |
31 | @pytest.fixture(scope="module")
32 | def admin_token():
33 |     """Get admin access token"""
34 |     res = requests.post(f"{BASE_URL}/api/auth/login", json={
35 |         "email": ADMIN_EMAIL,
36 |         "password": ADMIN_PASSWORD
37 |     })
38 |     if res.status_code != 200:
39 |         pytest.skip(f"Admin login failed: {res.status_code} - {res.text}")
40 |     return res.json().get("access_token")
41 |
42 |
43 | @pytest.fixture(scope="module")
44 | def test_user():
45 |     """Create a test user for admin operations"""
46 |     import uuid
47 |     email = f"TEST_user_{uuid.uuid4().hex[:8]}@test.com"
48 |     res = requests.post(f"{BASE_URL}/api/auth/register", json={
49 |         "email": email,
50 |         "password": "TestPass123!",
51 |         "name": "Test User"
52 |     })
53 |     if res.status_code not in [200, 201]:
54 |         pytest.skip(f"Test user creation failed: {res.status_code}")
55 |     data = res.json()
56 |     return {
57 |         "id": data.get("id"),
58 |         "email": email,
```

```
59 |         "token": data.get("access_token")
60 |     }
61 |
62 |
63 | class TestOwnerLogin:
64 |     """Test owner login functionality"""
65 |
66 |     def test_owner_login_success(self):
67 |         """Owner can login with correct credentials"""
68 |         res = requests.post(f"{BASE_URL}/api/auth/login", json={
69 |             "email": OWNER_EMAIL,
70 |             "password": OWNER_PASSWORD
71 |         })
72 |         assert res.status_code == 200, f"Expected 200, got {res.status_code}: {res.text}"
73 |         data = res.json()
74 |         assert "access_token" in data
75 |         assert data.get("role") == "owner"
76 |         assert data.get("subscription_status") == "pro"
77 |         print(f"Owner login successful: role={data.get('role')}, subscription={data.get('subscription_status')}")
78 |
79 |     def test_admin_login_success(self):
80 |         """Admin can login with correct credentials"""
81 |         res = requests.post(f"{BASE_URL}/api/auth/login", json={
82 |             "email": ADMIN_EMAIL,
83 |             "password": ADMIN_PASSWORD
84 |         })
85 |         assert res.status_code == 200, f"Expected 200, got {res.status_code}: {res.text}"
86 |         data = res.json()
87 |         assert "access_token" in data
88 |         assert data.get("role") == "admin"
89 |         print(f"Admin login successful: role={data.get('role')}")
90 |
91 |
92 | class TestAdminPanel:
93 |     """Test Admin Panel endpoints (owner-only)"""
94 |
95 |     def test_list_users_as_owner(self, owner_token):
96 |         """Owner can list all users"""
97 |         res = requests.get(
98 |             f"{BASE_URL}/api/auth/admin/users",
99 |             headers={"Authorization": f"Bearer {owner_token}"})
100 |
101 |         assert res.status_code == 200, f"Expected 200, got {res.status_code}: {res.text}"
102 |         data = res.json()
103 |         assert "users" in data
104 |         assert "total" in data
105 |         assert isinstance(data["users"], list)
106 |         print(f"Listed {data['total']} users")
107 |
108 |     def test_list_users_as_admin_forbidden(self, admin_token):
109 |         """Admin (non-owner) cannot list users"""
110 |         res = requests.get(
111 |             f"{BASE_URL}/api/auth/admin/users",
112 |             headers={"Authorization": f"Bearer {admin_token}"})
113 |
114 |         assert res.status_code == 403, f"Expected 403, got {res.status_code}: {res.text}"
115 |         print("Admin correctly denied access to user list")
116 |
117 |     def test_list_users_unauthenticated(self):
118 |         """Unauthenticated request is rejected"""
119 |         res = requests.get(f"{BASE_URL}/api/auth/admin/users")
120 |         assert res.status_code == 401, f"Expected 401, got {res.status_code}"
121 |
122 |     def test_deactivate_user(self, owner_token, test_user):
123 |         """Owner can deactivate a user"""
124 |         res = requests.post(
125 |             f"{BASE_URL}/api/auth/admin/users/{test_user['id']}/deactivate",
```

```
126 |         headers={"Authorization": f"Bearer {owner_token}"})
127 |     )
128 |     assert res.status_code == 200, f"Expected 200, got {res.status_code}: {res.text}"
129 |     data = res.json()
130 |     assert "message" in data
131 |     print(f"User deactivated: {data.get('message')}")
132 |
133 |     def test_activate_user(self, owner_token, test_user):
134 |         """Owner can activate a user"""
135 |         res = requests.post(
136 |             f"{BASE_URL}/api/auth/admin/users/{test_user['id']}/activate",
137 |             headers={"Authorization": f"Bearer {owner_token}"})
138 |         )
139 |         assert res.status_code == 200, f"Expected 200, got {res.status_code}: {res.text}"
140 |         data = res.json()
141 |         assert "message" in data
142 |         print(f"User activated: {data.get('message')}")
143 |
144 |     def test_grant_pro(self, owner_token, test_user):
145 |         """Owner can grant Pro to a user"""
146 |         res = requests.post(
147 |             f"{BASE_URL}/api/auth/admin/users/{test_user['id']}/grant-pro",
148 |             headers={"Authorization": f"Bearer {owner_token}"})
149 |         )
150 |         assert res.status_code == 200, f"Expected 200, got {res.status_code}: {res.text}"
151 |         data = res.json()
152 |         assert "message" in data
153 |         print(f"Pro granted: {data.get('message')}")
154 |
155 |     def test_revoke_pro(self, owner_token, test_user):
156 |         """Owner can revoke Pro from a user"""
157 |         res = requests.post(
158 |             f"{BASE_URL}/api/auth/admin/users/{test_user['id']}/revoke-pro",
159 |             headers={"Authorization": f"Bearer {owner_token}"})
160 |         )
161 |         assert res.status_code == 200, f"Expected 200, got {res.status_code}: {res.text}"
162 |         data = res.json()
163 |         assert "message" in data
164 |         print(f"Pro revoked: {data.get('message')}")
165 |
166 |     def test_admin_cannot_deactivate(self, admin_token, test_user):
167 |         """Admin (non-owner) cannot deactivate users"""
168 |         res = requests.post(
169 |             f"{BASE_URL}/api/auth/admin/users/{test_user['id']}/deactivate",
170 |             headers={"Authorization": f"Bearer {admin_token}"})
171 |         )
172 |         assert res.status_code == 403, f"Expected 403, got {res.status_code}"
173 |         print("Admin correctly denied deactivate access")
174 |
175 |
176 | class TestWorkspaceWatchlist:
177 |     """Test Workspace Watchlist endpoints"""
178 |
179 |     def test_get_watchlist_authenticated(self, admin_token):
180 |         """Authenticated user can get their watchlist"""
181 |         res = requests.get(
182 |             f"{BASE_URL}/api/workspace/watchlist",
183 |             headers={"Authorization": f"Bearer {admin_token}"})
184 |         )
185 |         assert res.status_code == 200, f"Expected 200, got {res.status_code}: {res.text}"
186 |         data = res.json()
187 |         assert "tickers" in data
188 |         print(f"Watchlist retrieved: {data.get('tickers')}")
189 |
190 |     def test_get_watchlist_unauthenticated(self):
191 |         """Unauthenticated request is rejected"""
192 |         res = requests.get(f"{BASE_URL}/api/workspace/watchlist")
```

```
193 |         assert res.status_code == 401, f"Expected 401, got {res.status_code}"
194 |
195 |     def test_add_ticker_to_watchlist(self, admin_token):
196 |         """User can add a ticker to watchlist"""
197 |         res = requests.post(
198 |             f"{BASE_URL}/api/workspace/watchlist/add",
199 |             headers={
200 |                 "Authorization": f"Bearer {admin_token}",
201 |                 "Content-Type": "application/json"
202 |             },
203 |             json={"ticker": "MSFT"}
204 |         )
205 |         assert res.status_code == 200, f"Expected 200, got {res.status_code}: {res.text}"
206 |         data = res.json()
207 |         assert "message" in data
208 |         print(f"Ticker added: {data.get('message')}")
209 |
210 |         # Verify ticker is in watchlist
211 |         res2 = requests.get(
212 |             f"{BASE_URL}/api/workspace/watchlist",
213 |             headers={"Authorization": f"Bearer {admin_token}"})
214 |
215 |         assert res2.status_code == 200
216 |         tickers = res2.json().get("tickers", [])
217 |         assert "MSFT" in tickers, f"MSFT not found in watchlist: {tickers}"
218 |         print(f"Verified MSFT in watchlist: {tickers}")
219 |
220 |     def test_remove_ticker_from_watchlist(self, admin_token):
221 |         """User can remove a ticker from watchlist"""
222 |         # First add a ticker
223 |         requests.post(
224 |             f"{BASE_URL}/api/workspace/watchlist/add",
225 |             headers={
226 |                 "Authorization": f"Bearer {admin_token}",
227 |                 "Content-Type": "application/json"
228 |             },
229 |             json={"ticker": "GOOG"}
230 |         )
231 |
232 |         # Remove it
233 |         res = requests.post(
234 |             f"{BASE_URL}/api/workspace/watchlist/remove",
235 |             headers={
236 |                 "Authorization": f"Bearer {admin_token}",
237 |                 "Content-Type": "application/json"
238 |             },
239 |             json={"ticker": "GOOG"}
240 |         )
241 |         assert res.status_code == 200, f"Expected 200, got {res.status_code}: {res.text}"
242 |
243 |         # Verify removal
244 |         res2 = requests.get(
245 |             f"{BASE_URL}/api/workspace/watchlist",
246 |             headers={"Authorization": f"Bearer {admin_token}"})
247 |
248 |         tickers = res2.json().get("tickers", [])
249 |         assert "GOOG" not in tickers, f"GOOG still in watchlist: {tickers}"
250 |         print(f"Verified GOOG removed from watchlist")
251 |
252 |
253 | class TestWorkspaceHistory:
254 |     """Test Workspace Hypothesis History endpoints"""
255 |
256 |     def test_get_history_authenticated(self, admin_token):
257 |         """Authenticated user can get their hypothesis history"""
258 |         res = requests.get(
259 |             f"{BASE_URL}/api/workspace/history",
```

```
260 |         headers={"Authorization": f"Bearer {admin_token}"}
261 |     )
262 |     assert res.status_code == 200, f"Expected 200, got {res.status_code}: {res.text}"
263 |     data = res.json()
264 |     assert "history" in data
265 |     print(f"History retrieved: {len(data.get('history', []))} entries")
266 |
267 |     def test_get_history_unauthenticated(self):
268 |         """Unauthenticated request is rejected"""
269 |         res = requests.get(f"{BASE_URL}/api/workspace/history")
270 |         assert res.status_code == 401, f"Expected 401, got {res.status_code}"
271 |
272 |     def test_save_history_entry(self, admin_token):
273 |         """User can save a hypothesis to history"""
274 |         res = requests.post(
275 |             f"{BASE_URL}/api/workspace/history/save",
276 |             headers={
277 |                 "Authorization": f"Bearer {admin_token}",
278 |                 "Content-Type": "application/json"
279 |             },
280 |             json={
281 |                 "symbol": "AAPL",
282 |                 "verdict": "BUY",
283 |                 "confidence": 85
284 |             }
285 |         )
286 |         assert res.status_code == 200, f"Expected 200, got {res.status_code}: {res.text}"
287 |         data = res.json()
288 |         assert "message" in data
289 |         print(f"History saved: {data.get('message')}")
290 |
291 |         # Verify entry is in history
292 |         res2 = requests.get(
293 |             f"{BASE_URL}/api/workspace/history",
294 |             headers={"Authorization": f"Bearer {admin_token}"}
295 |         )
296 |         history = res2.json().get("history", [])
297 |         aapl_entries = [h for h in history if h.get("symbol") == "AAPL"]
298 |         assert len(aapl_entries) > 0, f"AAPL not found in history"
299 |         print(f"Verified AAPL in history: {aapl_entries[0]}")
300 |
301 |
302 | if __name__ == "__main__":
303 |     pytest.main([__file__, "-v", "--tb=short"])
304 |
```

FILE: backend/tests/test_auth_hypothesis.py

282 lines | 12747 characters

```
1 | """
2 | Test suite for RISEDUALAI Auth System and AI Hypothesis Engine
3 | Tests: JWT auth (register, login, logout, me, refresh), brute force protection, hypothesis payroll
4 | """
5 | import pytest
6 | import requests
7 | import os
8 | import time
9 |
10 | BASE_URL = os.environ.get('REACT_APP_BACKEND_URL', '').rstrip('/')
11 |
12 | # Admin credentials from environment
13 | ADMIN_EMAIL = os.environ.get("ADMIN_EMAIL", "admin@risedual.ai")
14 | ADMIN_PASSWORD = os.environ.get("ADMIN_PASSWORD", "")
15 |
16 | # Test user for registration
17 | TEST_USER_EMAIL = f"test_user_{int(time.time())}@test.com"
```



```
18 | TEST_USER_PASSWORD = os.environ.get("TEST_USER_PASSWORD", "TestPass123!")
19 | TEST_USER_NAME = "Test User"
20 |
21 |
22 | class TestAuthRegister:
23 |     """Test POST /api/auth/register"""
24 |
25 |     def test_register_success(self):
26 |         """Register a new user successfully"""
27 |         response = requests.post(f"{BASE_URL}/api/auth/register", json={
28 |             "email": TEST_USER_EMAIL,
29 |             "password": TEST_USER_PASSWORD,
30 |             "name": TEST_USER_NAME
31 |         })
32 |         assert response.status_code == 200, f"Expected 200, got {response.status_code}: {response.text}"
33 |
34 |         data = response.json()
35 |         assert "id" in data, "Response should contain user id"
36 |         assert data["email"] == TEST_USER_EMAIL.lower(), "Email should match"
37 |         assert data["name"] == TEST_USER_NAME, "Name should match"
38 |         assert data["role"] == "user", "New user should have role=user"
39 |         assert data["subscription_status"] == "free", "New user should have subscription_status=free"
40 |
41 |         # Check cookies are set
42 |         assert "access_token" in response.cookies, "access_token cookie should be set"
43 |         assert "refresh_token" in response.cookies, "refresh_token cookie should be set"
44 |         print(f"? Register success: {data['email']} with role={data['role']}, subscription={data['subscription_status']}")
45 |
46 |     def test_register_duplicate_email(self):
47 |         """Registering with existing email should fail"""
48 |         response = requests.post(f"{BASE_URL}/api/auth/register", json={
49 |             "email": ADMIN_EMAIL,
50 |             "password": "SomePassword123",
51 |             "name": "Duplicate"
52 |         })
53 |         assert response.status_code == 400, f"Expected 400 for duplicate email, got {response.status_code}"
54 |         assert "already registered" in response.json().get("detail", "").lower()
55 |         print("? Duplicate email registration correctly rejected")
56 |
57 |     def test_register_short_password(self):
58 |         """Password less than 6 chars should fail"""
59 |         response = requests.post(f"{BASE_URL}/api/auth/register", json={
60 |             "email": f"short_pw_{int(time.time())}@test.com",
61 |             "password": "12345",
62 |             "name": "Short PW"
63 |         })
64 |         assert response.status_code == 400, f"Expected 400 for short password, got {response.status_code}"
65 |         print("? Short password correctly rejected")
66 |
67 |
68 | class TestAuthLogin:
69 |     """Test POST /api/auth/login"""
70 |
71 |     def test_login_admin_success(self):
72 |         """Login with admin credentials"""
73 |         session = requests.Session()
74 |         response = session.post(f"{BASE_URL}/api/auth/login", json={
75 |             "email": ADMIN_EMAIL,
76 |             "password": ADMIN_PASSWORD
77 |         })
78 |         assert response.status_code == 200, f"Expected 200, got {response.status_code}: {response.text}"
79 |
80 |         data = response.json()
81 |         assert data["email"] == ADMIN_EMAIL, "Email should match"
82 |         assert data["role"] == "admin", "Admin should have role=admin"
83 |         assert data["subscription_status"] == "pro", "Admin should have subscription_status=pro"
84 |
```

```
85 |         # Check cookies
86 |         assert "access_token" in session.cookies, "access_token cookie should be set"
87 |         assert "refresh_token" in session.cookies, "refresh_token cookie should be set"
88 |         print(f"? Admin login success: role={data['role']}, subscription={data['subscription_status']}")
89 |
90 |     def test_login_wrong_password(self):
91 |         """Login with wrong password should return 401"""
92 |         response = requests.post(f"{BASE_URL}/api/auth/login", json={
93 |             "email": ADMIN_EMAIL,
94 |             "password": "WrongPassword123"
95 |         })
96 |         assert response.status_code == 401, f"Expected 401, got {response.status_code}"
97 |         assert "invalid" in response.json().get("detail", "").lower()
98 |         print(f"? Wrong password correctly rejected with 401")
99 |
100 |    def test_login_nonexistent_user(self):
101 |        """Login with non-existent email should return 401"""
102 |        response = requests.post(f"{BASE_URL}/api/auth/login", json={
103 |            "email": "nonexistent@test.com",
104 |            "password": "SomePassword123"
105 |        })
106 |        assert response.status_code == 401, f"Expected 401, got {response.status_code}"
107 |        print(f"? Non-existent user login correctly rejected")
108 |
109 |
110 |    class TestAuthMe:
111 |        """Test GET /api/auth/me"""
112 |
113 |        def test_me_with_valid_session(self):
114 |            """Get current user with valid cookies"""
115 |            session = requests.Session()
116 |            # Login first
117 |            login_resp = session.post(f"{BASE_URL}/api/auth/login", json={
118 |                "email": ADMIN_EMAIL,
119 |                "password": ADMIN_PASSWORD
120 |            })
121 |            assert login_resp.status_code == 200, "Login should succeed"
122 |
123 |            # Get /me
124 |            me_resp = session.get(f"{BASE_URL}/api/auth/me")
125 |            assert me_resp.status_code == 200, f"Expected 200, got {me_resp.status_code}: {me_resp.text}"
126 |
127 |            data = me_resp.json()
128 |            assert data["email"] == ADMIN_EMAIL
129 |            assert data["role"] == "admin"
130 |            assert data["subscription_status"] == "pro"
131 |            print(f"? /me returns correct user: {data['email']}")
132 |
133 |        def test_me_without_auth(self):
134 |            """Get /me without cookies should return 401"""
135 |            response = requests.get(f"{BASE_URL}/api/auth/me")
136 |            assert response.status_code == 401, f"Expected 401, got {response.status_code}"
137 |            print(f"? /me without auth correctly returns 401")
138 |
139 |
140 |    class TestAuthLogout:
141 |        """Test POST /api/auth/logout"""
142 |
143 |        def test_logout_clears_cookies(self):
144 |            """Logout should clear auth cookies"""
145 |            session = requests.Session()
146 |            # Login first
147 |            login_resp = session.post(f"{BASE_URL}/api/auth/login", json={
148 |                "email": ADMIN_EMAIL,
149 |                "password": ADMIN_PASSWORD
150 |            })
151 |            assert login_resp.status_code == 200
```

```
152 |
153 |     # Logout
154 |     logout_resp = session.post(f"{BASE_URL}/api/auth/logout")
155 |     assert logout_resp.status_code == 200, f"Expected 200, got {logout_resp.status_code}"
156 |     assert logout_resp.json().get("message") == "Logged out"
157 |
158 |     # Verify /me now fails (cookies should be cleared)
159 |     me_resp = session.get(f"{BASE_URL}/api/auth/me")
160 |     assert me_resp.status_code == 401, "After logout, /me should return 401"
161 |     print("? Logout clears cookies and /me returns 401")
162 |
163 |
164 | class TestAuthRefresh:
165 |     """Test POST /api/auth/refresh"""
166 |
167 |     def test_refresh_token(self):
168 |         """Refresh token should work with valid refresh_token cookie"""
169 |         session = requests.Session()
170 |         # Login first
171 |         login_resp = session.post(f"{BASE_URL}/api/auth/login", json={
172 |             "email": ADMIN_EMAIL,
173 |             "password": ADMIN_PASSWORD
174 |         })
175 |         assert login_resp.status_code == 200
176 |
177 |         # Refresh
178 |         refresh_resp = session.post(f"{BASE_URL}/api/auth/refresh")
179 |         assert refresh_resp.status_code == 200, f"Expected 200, got {refresh_resp.status_code}: {refresh_resp.text}"
180 |         assert refresh_resp.json().get("message") == "Token refreshed"
181 |         print("? Token refresh works")
182 |
183 |     def test_refresh_without_token(self):
184 |         """Refresh without refresh_token should fail"""
185 |         response = requests.post(f"{BASE_URL}/api/auth/refresh")
186 |         assert response.status_code == 401, f"Expected 401, got {response.status_code}"
187 |         print("? Refresh without token correctly returns 401")
188 |
189 |
190 | class TestHypothesisFreeUser:
191 |     """Test GET /api/hypothesis/{symbol} as free user (no auth or new user)"""
192 |
193 |     def test_hypothesis_free_user_returns_locked(self):
194 |         """Free user should get locked/teaser hypothesis"""
195 |         # No cookies = free user
196 |         response = requests.get(f"{BASE_URL}/api/hypothesis/AAPL", timeout=60)
197 |         assert response.status_code == 200, f"Expected 200, got {response.status_code}: {response.text}"
198 |
199 |         data = response.json()
200 |         assert data["symbol"] == "AAPL", "Symbol should be AAPL"
201 |         assert data["is_pro"] == False, "is_pro should be False for free user"
202 |         assert "teaser" in data, "Response should contain teaser"
203 |         assert data["teaser"]["verdict"] == "LOCKED", "Verdict should be LOCKED"
204 |         assert "data_sources_count" in data["teaser"], "Teaser should have data_sources_count"
205 |         assert "world_events_count" in data["teaser"], "Teaser should have world_events_count"
206 |         assert "congressional_trades_count" in data["teaser"], "Teaser should have congressional_trades_count"
207 |         assert "summary" in data["teaser"], "Teaser should have summary"
208 |         print(f"? Free user hypothesis: is_pro={data['is_pro']}, verdict={data['teaser']['verdict']}")
209 |         print(f" Teaser stats: {data['teaser']['data_sources_count']} data points, {data['teaser']['world_events_count']} world e ...")
210 |
211 |
212 | class TestHypothesisProUser:
213 |     """Test GET /api/hypothesis/{symbol} as Pro user (admin)"""
214 |
215 |     def test_hypothesis_pro_user_returns_full(self):
216 |         """Pro user should get full hypothesis with verdict, confidence, thesis, etc."""
217 |         session = requests.Session()
218 |         # Login as admin (pro user)
```

```

219 |         login_resp = session.post(f"{BASE_URL}/api/auth/login", json={
220 |             "email": ADMIN_EMAIL,
221 |             "password": ADMIN_PASSWORD
222 |         })
223 |         assert login_resp.status_code == 200, "Admin login should succeed"
224 |
225 |         # Get hypothesis - this takes 60-90 seconds for AI analysis
226 |         print(" Fetching hypothesis for AAPL as Pro user (may take 60-90 seconds)...")
227 |         response = session.get(f"{BASE_URL}/api/hypothesis/AAPL", timeout=150)
228 |         assert response.status_code == 200, f"Expected 200, got {response.status_code}: {response.text}"
229 |
230 |         data = response.json()
231 |         assert data["symbol"] == "AAPL", "Symbol should be AAPL"
232 |         assert data["is_pro"] == True, "is_pro should be True for pro user"
233 |         assert "verdict" in data, "Response should contain verdict"
234 |         assert data["verdict"] in ["BUY", "SELL", "HOLD", "NEUTRAL"], f"Verdict should be BUY/SELL/HOLD/NEUTRAL, got {data.get('ve ...
235 |         assert "confidence" in data, "Response should contain confidence"
236 |         assert isinstance(data["confidence"], (int, float)), "Confidence should be a number"
237 |         print(f"? Pro user hypothesis: is_pro={data['is_pro']}, verdict={data['verdict']}, confidence={data['confidence']}%")
238 |
239 |         # Check optional fields
240 |         if "thesis" in data:
241 |             print(f" Thesis: {data['thesis'][:100]}...")
242 |         if "catalysts" in data:
243 |             print(f" Catalysts: {len(data['catalysts'])} items")
244 |         if "risks" in data:
245 |             print(f" Risks: {len(data['risks'])} items")
246 |
247 |
248 | class TestBcryptHashFormat:
249 |     """Verify bcrypt hash format starts with $2b$"""
250 |
251 |     def test_bcrypt_hash_format(self):
252 |         """Register user and verify password is hashed with bcrypt"""
253 |         # This is an indirect test - we verify login works which means bcrypt is working
254 |         unique_email = f"bcrypt_test_{int(time.time())}@test.com"
255 |
256 |         # Register
257 |         reg_resp = requests.post(f"{BASE_URL}/api/auth/register", json={
258 |             "email": unique_email,
259 |             "password": "BcryptTest123!",
260 |             "name": "Bcrypt Test"
261 |         })
262 |         assert reg_resp.status_code == 200, f"Registration failed: {reg_resp.text}"
263 |
264 |         # Login with same password
265 |         login_resp = requests.post(f"{BASE_URL}/api/auth/login", json={
266 |             "email": unique_email,
267 |             "password": "BcryptTest123!"
268 |         })
269 |         assert login_resp.status_code == 200, "Login should work with correct password"
270 |
271 |         # Login with wrong password should fail
272 |         wrong_resp = requests.post(f"{BASE_URL}/api/auth/login", json={
273 |             "email": unique_email,
274 |             "password": "WrongPassword"
275 |         })
276 |         assert wrong_resp.status_code == 401, "Login should fail with wrong password"
277 |         print("? Bcrypt password hashing working correctly (login/verify works)")
278 |
279 |
280 | if __name__ == "__main__":
281 |     pytest.main([__file__, "-v", "--tb=short"])
282 |

```

FILE: backend/tests/test_bearer_auth.py

264 lines | 11300 characters

```
1 | """
2 | Backend tests for RISEQUALAI Bearer Token Auth and AI Hypothesis
3 | Tests: Login, Register, /me, Refresh, Hypothesis (free vs pro)
4 | """
5 | import pytest
6 | import requests
7 | import os
8 | import time
9 |
10 | BASE_URL = os.environ.get('REACT_APP_BACKEND_URL', '').rstrip('/')
11 |
12 | # Admin credentials from environment
13 | ADMIN_EMAIL = os.environ.get("ADMIN_EMAIL", "admin@risedual.ai")
14 | ADMIN_PASSWORD = os.environ.get("ADMIN_PASSWORD", "")
15 |
16 |
17 | class TestAuthLogin:
18 |     """Test login endpoint with Bearer token response"""
19 |
20 |     def test_login_admin_success(self):
21 |         """POST /api/auth/login with admin credentials returns tokens"""
22 |         response = requests.post(f"{BASE_URL}/api/auth/login", json={
23 |             "email": ADMIN_EMAIL,
24 |             "password": ADMIN_PASSWORD
25 |         })
26 |         assert response.status_code == 200, f"Expected 200, got {response.status_code}: {response.text}"
27 |
28 |         data = response.json()
29 |         # Verify tokens in response body
30 |         assert "access_token" in data, "Missing access_token in response"
31 |         assert "refresh_token" in data, "Missing refresh_token in response"
32 |         assert len(data["access_token"]) > 50, "access_token too short"
33 |         assert len(data["refresh_token"]) > 50, "refresh_token too short"
34 |
35 |         # Verify user info
36 |         assert data["email"] == ADMIN_EMAIL
37 |         assert data["role"] == "admin"
38 |         assert data["subscription_status"] == "pro"
39 |         print(f"PASSED: Admin login returns tokens and user info (role={data['role']}, subscription={data['subscription_status']}) ...")
40 |
41 |     def test_login_wrong_password(self):
42 |         """POST /api/auth/login with wrong password returns 401"""
43 |         response = requests.post(f"{BASE_URL}/api/auth/login", json={
44 |             "email": ADMIN_EMAIL,
45 |             "password": "WrongPassword123"
46 |         })
47 |         assert response.status_code == 401, f"Expected 401, got {response.status_code}"
48 |         print("PASSED: Wrong password returns 401")
49 |
50 |     def test_login_nonexistent_user(self):
51 |         """POST /api/auth/login with nonexistent email returns 401"""
52 |         response = requests.post(f"{BASE_URL}/api/auth/login", json={
53 |             "email": "nonexistent@test.com",
54 |             "password": "AnyPassword123"
55 |         })
56 |         assert response.status_code == 401, f"Expected 401, got {response.status_code}"
57 |         print("PASSED: Nonexistent user returns 401")
58 |
59 |
60 | class TestAuthRegister:
61 |     """Test register endpoint"""
62 |
63 |     def test_register_new_user(self):
```

```
64 | """POST /api/auth/register creates user with free subscription"""
65 | unique_email = f"test_user_{int(time.time())}@test.com"
66 | response = requests.post(f"{BASE_URL}/api/auth/register", json={
67 |     "email": unique_email,
68 |     "password": "TestPass123!",
69 |     "name": "Test User"
70 | })
71 | assert response.status_code == 200, f"Expected 200, got {response.status_code}: {response.text}"
72 |
73 | data = response.json()
74 | # Verify tokens returned
75 | assert "access_token" in data, "Missing access_token"
76 | assert "refresh_token" in data, "Missing refresh_token"
77 |
78 | # Verify user info
79 | assert data["email"] == unique_email
80 | assert data["subscription_status"] == "free", f"Expected free, got {data['subscription_status']}"
81 | assert data["role"] == "user"
82 | print(f"PASSED: Register returns tokens + user with subscription_status=free")
83 |
84 | def test_register_duplicate_email(self):
85 |     """POST /api/auth/register with existing email returns 400"""
86 |     response = requests.post(f"{BASE_URL}/api/auth/register", json={
87 |         "email": ADMIN_EMAIL,
88 |         "password": "TestPass123!",
89 |         "name": "Duplicate"
90 |     })
91 |     assert response.status_code == 400, f"Expected 400, got {response.status_code}"
92 |     print("PASSED: Duplicate email returns 400")
93 |
94 | def test_register_short_password(self):
95 |     """POST /api/auth/register with short password returns 400"""
96 |     response = requests.post(f"{BASE_URL}/api/auth/register", json={
97 |         "email": f"short_pw_{int(time.time())}@test.com",
98 |         "password": "12345",
99 |         "name": "Short PW"
100 |     })
101 |     assert response.status_code == 400, f"Expected 400, got {response.status_code}"
102 |     print("PASSED: Short password returns 400")
103 |
104 |
105 | class TestAuthMe:
106 |     """Test /api/auth/me endpoint with Bearer token"""
107 |
108 |     def test_me_with_valid_token(self):
109 |         """GET /api/auth/me with valid Bearer token returns user info"""
110 |         # First login to get token
111 |         login_resp = requests.post(f"{BASE_URL}/api/auth/login", json={
112 |             "email": ADMIN_EMAIL,
113 |             "password": ADMIN_PASSWORD
114 |         })
115 |         token = login_resp.json()["access_token"]
116 |
117 |         # Call /me with Bearer token
118 |         response = requests.get(f"{BASE_URL}/api/auth/me", headers={
119 |             "Authorization": f"Bearer {token}"
120 |         })
121 |         assert response.status_code == 200, f"Expected 200, got {response.status_code}: {response.text}"
122 |
123 |         data = response.json()
124 |         assert data["email"] == ADMIN_EMAIL
125 |         assert data["role"] == "admin"
126 |         assert data["subscription_status"] == "pro"
127 |         print(f"PASSED: /me with Bearer token returns user info")
128 |
129 |     def test_me_without_auth(self):
130 |         """GET /api/auth/me without token returns 401"""
```

```
131 |         response = requests.get(f"{BASE_URL}/api/auth/me")
132 |         assert response.status_code == 401, f"Expected 401, got {response.status_code}"
133 |         print("PASSED: /me without auth returns 401")
134 |
135 |     def test_me_with_invalid_token(self):
136 |         """GET /api/auth/me with invalid token returns 401"""
137 |         response = requests.get(f"{BASE_URL}/api/auth/me", headers={
138 |             "Authorization": "Bearer invalid_token_here"
139 |         })
140 |         assert response.status_code == 401, f"Expected 401, got {response.status_code}"
141 |         print("PASSED: /me with invalid token returns 401")
142 |
143 |
144 | class TestAuthRefresh:
145 |     """Test token refresh endpoint"""
146 |
147 |     def test_refresh_token(self):
148 |         """POST /api/auth/refresh with valid refresh_token returns new access_token"""
149 |         # Login to get refresh token
150 |         login_resp = requests.post(f"{BASE_URL}/api/auth/login", json={
151 |             "email": ADMIN_EMAIL,
152 |             "password": ADMIN_PASSWORD
153 |         })
154 |         refresh_token = login_resp.json()["refresh_token"]
155 |
156 |         # Refresh
157 |         response = requests.post(f"{BASE_URL}/api/auth/refresh", json={
158 |             "refresh_token": refresh_token
159 |         })
160 |         assert response.status_code == 200, f"Expected 200, got {response.status_code}: {response.text}"
161 |
162 |         data = response.json()
163 |         assert "access_token" in data, "Missing access_token in refresh response"
164 |         assert len(data["access_token"]) > 50
165 |         print("PASSED: Refresh returns new access_token")
166 |
167 |     def test_refresh_without_token(self):
168 |         """POST /api/auth/refresh without token returns 401"""
169 |         response = requests.post(f"{BASE_URL}/api/auth/refresh", json={})
170 |         assert response.status_code == 401, f"Expected 401, got {response.status_code}"
171 |         print("PASSED: Refresh without token returns 401")
172 |
173 |
174 | class TestHypothesisFreeUser:
175 |     """Test hypothesis endpoint for free/unauthenticated users"""
176 |
177 |     def test_hypothesis_without_auth(self):
178 |         """GET /api/hypothesis/AAPL without auth returns is_pro=false with teaser"""
179 |         response = requests.get(f"{BASE_URL}/api/hypothesis/AAPL")
180 |         assert response.status_code == 200, f"Expected 200, got {response.status_code}: {response.text}"
181 |
182 |         data = response.json()
183 |         assert data["is_pro"] == False, f"Expected is_pro=false, got {data.get('is_pro')}"
184 |         assert data["symbol"] == "AAPL"
185 |         assert "teaser" in data, "Missing teaser for free user"
186 |         assert data["teaser"]["verdict"] == "LOCKED"
187 |         assert "data_sources_count" in data["teaser"]
188 |         assert "world_events_count" in data["teaser"]
189 |         assert "congressional_trades_count" in data["teaser"]
190 |         print(f"PASSED: Hypothesis without auth returns is_pro=false, verdict=LOCKED")
191 |
192 |     def test_hypothesis_free_user_with_token(self):
193 |         """GET /api/hypothesis/AAPL with free user token returns is_pro=false"""
194 |         # Register a new free user
195 |         unique_email = f"free_user_{int(time.time())}@test.com"
196 |         reg_resp = requests.post(f"{BASE_URL}/api/auth/register", json={
197 |             "email": unique_email,
```

```
198 |         "password": "TestPass123!",
199 |         "name": "Free User"
200 |     })
201 |     token = reg_resp.json()["access_token"]
202 |
203 |     # Get hypothesis with free user token
204 |     response = requests.get(f"{BASE_URL}/api/hypothesis/TSLA", headers={
205 |         "Authorization": f"Bearer {token}"
206 |     })
207 |     assert response.status_code == 200, f"Expected 200, got {response.status_code}"
208 |
209 |     data = response.json()
210 |     assert data["is_pro"] == False, f"Expected is_pro=false for free user"
211 |     assert data["teaser"]["verdict"] == "LOCKED"
212 |     print("PASSED: Free user with token gets is_pro=false, verdict=LOCKED")
213 |
214 |
215 | class TestHypothesisProUser:
216 |     """Test hypothesis endpoint for Pro users (admin)"""
217 |
218 |     def test_hypothesis_pro_user(self):
219 |         """GET /api/hypothesis/AAPL with admin (Pro) token returns full hypothesis
220 |         NOTE: This test takes 60-90 seconds due to AI generation
221 |         """
222 |         # Login as admin (Pro user)
223 |         login_resp = requests.post(f"{BASE_URL}/api/auth/login", json={
224 |             "email": ADMIN_EMAIL,
225 |             "password": ADMIN_PASSWORD
226 |         })
227 |         token = login_resp.json()["access_token"]
228 |
229 |         print("Fetching hypothesis for Pro user (this takes 60-90 seconds)...")
230 |         # Get hypothesis with Pro user token - increased timeout
231 |         response = requests.get(f"{BASE_URL}/api/hypothesis/AAPL", headers={
232 |             "Authorization": f"Bearer {token}"
233 |         }, timeout=120)
234 |
235 |         assert response.status_code == 200, f"Expected 200, got {response.status_code}: {response.text}"
236 |
237 |         data = response.json()
238 |         assert data["is_pro"] == True, f"Expected is_pro=true for Pro user, got {data.get('is_pro')}"
239 |         assert data["symbol"] == "AAPL"
240 |
241 |         # Verify full hypothesis fields
242 |         assert "verdict" in data, "Missing verdict in Pro hypothesis"
243 |         assert data["verdict"] in ["BUY", "SELL", "HOLD", "NEUTRAL"], f"Invalid verdict: {data.get('verdict')}"
244 |         assert "confidence" in data, "Missing confidence"
245 |         assert "thesis" in data, "Missing thesis"
246 |
247 |         print(f"PASSED: Pro user gets full hypothesis (verdict={data['verdict']}, confidence={data.get('confidence')})")
248 |
249 |
250 | class TestLogout:
251 |     """Test logout endpoint"""
252 |
253 |     def test_logout(self):
254 |         """POST /api/auth/logout returns success"""
255 |         response = requests.post(f"{BASE_URL}/api/auth/logout")
256 |         assert response.status_code == 200, f"Expected 200, got {response.status_code}"
257 |         data = response.json()
258 |         assert "message" in data
259 |         print("PASSED: Logout returns success message")
260 |
261 |
262 | if __name__ == "__main__":
263 |     pytest.main([__file__, "-v", "--tb=short"])
264 |
```


FILE: backend/tests/test_chat_image.py

230 lines | 9001 characters

```
1 | """
2 | Test suite for RISEQUALAI Chat API with Image Upload Support
3 | Tests: POST /api/chat with text-only, image+text, and backward compatibility
4 | """
5 | import pytest
6 | import requests
7 | import os
8 | import base64
9 | import io
10 | from PIL import Image
11 |
12 | BASE_URL = os.environ.get('REACT_APP_BACKEND_URL', '').rstrip('/')
13 |
14 | # Generate a simple test image with real visual features (candlestick-like pattern)
15 | def generate_test_chart_image():
16 |     """Generate a simple chart-like image with visual features for testing"""
17 |     img = Image.new('RGB', (200, 150), color=(30, 30, 35)) # Dark background
18 |
19 |     # Draw some candlestick-like bars
20 |     from PIL import ImageDraw
21 |     draw = ImageDraw.Draw(img)
22 |
23 |     # Draw grid lines
24 |     for i in range(0, 200, 40):
25 |         draw.line([(i, 0), (i, 150)], fill=(50, 50, 55), width=1)
26 |     for i in range(0, 150, 30):
27 |         draw.line([(0, i), (200, i)], fill=(50, 50, 55), width=1)
28 |
29 |     # Draw candlestick-like bars (green and red) - ensure y0 < y1
30 |     bars = [
31 |         (20, 40, 80, 'green'), # x, y_top, y_bottom (y_top < y_bottom)
32 |         (50, 60, 90, 'red'),
33 |         (80, 50, 70, 'green'),
34 |         (110, 70, 100, 'red'),
35 |         (140, 40, 60, 'green'),
36 |         (170, 55, 85, 'red'),
37 |     ]
38 |     for x, y_top, y_bottom, color in bars:
39 |         fill_color = (0, 200, 100) if color == 'green' else (200, 50, 50)
40 |         # Rectangle: [x0, y0, x1, y1] where y0 < y1
41 |         draw.rectangle([x-5, y_top, x+5, y_bottom], fill=fill_color)
42 |         # Wick
43 |         draw.line([(x, y_top-10), (x, y_bottom+10)], fill=fill_color, width=1)
44 |
45 |     # Convert to base64
46 |     buffer = io.BytesIO()
47 |     img.save(buffer, format='PNG')
48 |     buffer.seek(0)
49 |     return base64.b64encode(buffer.read()).decode('utf-8')
50 |
51 |
52 | def generate_jpeg_test_image():
53 |     """Generate a JPEG test image"""
54 |     img = Image.new('RGB', (100, 100), color=(100, 150, 200))
55 |     from PIL import ImageDraw
56 |     draw = ImageDraw.Draw(img)
57 |     draw.ellipse([20, 20, 80, 80], fill=(255, 200, 100))
58 |
59 |     buffer = io.BytesIO()
60 |     img.save(buffer, format='JPEG', quality=85)
61 |     buffer.seek(0)
62 |     return base64.b64encode(buffer.read()).decode('utf-8')
63 |
```

```
64 |
65 | class TestChatAPIBasic:
66 |     """Basic chat API tests - text only"""
67 |
68 |     def test_api_root_accessible(self):
69 |         """Test that API root is accessible"""
70 |         response = requests.get(f"{BASE_URL}/api/")
71 |         assert response.status_code == 200
72 |         data = response.json()
73 |         assert "message" in data
74 |         print(f"? API root accessible: {data['message']}")
75 |
76 |     def test_chat_text_only_message(self):
77 |         """Test POST /api/chat with text-only message"""
78 |         payload = {
79 |             "message": "What is a stock option?",
80 |             "sessionId": "test_session_text_001"
81 |         }
82 |         response = requests.post(f"{BASE_URL}/api/chat", json=payload, timeout=60)
83 |
84 |         assert response.status_code == 200, f"Expected 200, got {response.status_code}: {response.text}"
85 |         data = response.json()
86 |
87 |         # Validate response structure
88 |         assert "response" in data, "Response should contain 'response' field"
89 |         assert "sessionId" in data, "Response should contain 'sessionId' field"
90 |         assert data["sessionId"] == "test_session_text_001"
91 |         assert len(data["response"]) > 0, "AI response should not be empty"
92 |
93 |         print(f"? Text-only chat works. Response length: {len(data['response'])} chars")
94 |         print(f"  AI Response preview: {data['response'][:100]}...")
95 |
96 |     def test_chat_with_null_image_base64(self):
97 |         """Test POST /api/chat with null image_base64 (backward compatibility)"""
98 |         payload = {
99 |             "message": "Tell me about AAPL stock",
100 |             "sessionId": "test_session_null_img_002",
101 |             "image_base64": None
102 |         }
103 |         response = requests.post(f"{BASE_URL}/api/chat", json=payload, timeout=60)
104 |
105 |         assert response.status_code == 200, f"Expected 200, got {response.status_code}: {response.text}"
106 |         data = response.json()
107 |
108 |         assert "response" in data
109 |         assert len(data["response"]) > 0
110 |         print(f"? Chat with null image_base64 works (backward compat)")
111 |
112 |     def test_chat_without_image_base64_field(self):
113 |         """Test POST /api/chat without image_base64 field at all (backward compatibility)"""
114 |         payload = {
115 |             "message": "What is a put option?",
116 |             "sessionId": "test_session_no_img_003"
117 |         }
118 |         response = requests.post(f"{BASE_URL}/api/chat", json=payload, timeout=60)
119 |
120 |         assert response.status_code == 200, f"Expected 200, got {response.status_code}: {response.text}"
121 |         data = response.json()
122 |
123 |         assert "response" in data
124 |         assert len(data["response"]) > 0
125 |         print(f"? Chat without image_base64 field works (backward compat)")
126 |
127 |
128 | class TestChatAPIWithImage:
129 |     """Chat API tests with image upload (vision)"""
130 |
```

```
131 |     def test_chat_with_png_image(self):
132 |         """Test POST /api/chat with PNG image for chart analysis"""
133 |         image_base64 = generate_test_chart_image()
134 |
135 |         payload = {
136 |             "message": "Analyze this stock chart and identify any patterns",
137 |             "sessionId": "test_session_png_img_004",
138 |             "image_base64": image_base64
139 |         }
140 |         response = requests.post(f"{BASE_URL}/api/chat", json=payload, timeout=90)
141 |
142 |         assert response.status_code == 200, f"Expected 200, got {response.status_code}: {response.text}"
143 |         data = response.json()
144 |
145 |         assert "response" in data, "Response should contain 'response' field"
146 |         assert "sessionId" in data
147 |         assert len(data["response"]) > 0, "AI should provide analysis"
148 |
149 |         # The AI should acknowledge it received an image
150 |         response_lower = data["response"].lower()
151 |         # Check if AI mentions anything related to chart/image/analysis
152 |         print(f"? Chat with PNG image works. Response length: {len(data['response'])} chars")
153 |         print(f"  AI Response preview: {data['response'][:200]}...")
154 |
155 |     def test_chat_with_jpeg_image(self):
156 |         """Test POST /api/chat with JPEG image"""
157 |         image_base64 = generate_jpeg_test_image()
158 |
159 |         payload = {
160 |             "message": "What do you see in this image?",
161 |             "sessionId": "test_session_jpeg_img_005",
162 |             "image_base64": image_base64
163 |         }
164 |         response = requests.post(f"{BASE_URL}/api/chat", json=payload, timeout=90)
165 |
166 |         assert response.status_code == 200, f"Expected 200, got {response.status_code}: {response.text}"
167 |         data = response.json()
168 |
169 |         assert "response" in data
170 |         assert len(data["response"]) > 0
171 |         print(f"? Chat with JPEG image works")
172 |
173 |     def test_chat_image_only_no_text(self):
174 |         """Test POST /api/chat with image but minimal text"""
175 |         image_base64 = generate_test_chart_image()
176 |
177 |         payload = {
178 |             "message": "", # Empty message, should still work with image
179 |             "sessionId": "test_session_img_only_006",
180 |             "image_base64": image_base64
181 |         }
182 |         response = requests.post(f"{BASE_URL}/api/chat", json=payload, timeout=90)
183 |
184 |         # This might fail if backend requires message - let's check
185 |         if response.status_code == 200:
186 |             data = response.json()
187 |             assert "response" in data
188 |             print(f"? Chat with image-only (empty message) works")
189 |         elif response.status_code == 422:
190 |             print(f"? Chat requires non-empty message (validation error) - this is acceptable")
191 |         else:
192 |             print(f"? Unexpected status {response.status_code}: {response.text}")
193 |
194 |
195 | class TestChatHistory:
196 |     """Test chat history endpoint"""
197 |
```

```
198 |     def test_get_chat_history(self):
199 |         """Test GET /api/chat/history/{session_id}"""
200 |         session_id = "test_session_history_007"
201 |
202 |         # First send a message to create history
203 |         payload = {
204 |             "message": "Hello, this is a test message for history",
205 |             "sessionId": session_id
206 |         }
207 |         requests.post(f"{BASE_URL}/api/chat", json=payload, timeout=60)
208 |
209 |         # Now get history
210 |         response = requests.get(f"{BASE_URL}/api/chat/history/{session_id}")
211 |
212 |         assert response.status_code == 200
213 |         data = response.json()
214 |         assert "messages" in data
215 |         print(f"? Chat history endpoint works. Messages count: {len(data['messages'])}")
216 |
217 |     def test_get_nonexistent_session_history(self):
218 |         """Test GET /api/chat/history for non-existent session"""
219 |         response = requests.get(f"{BASE_URL}/api/chat/history/nonexistent_session_xyz")
220 |
221 |         assert response.status_code == 200
222 |         data = response.json()
223 |         assert "messages" in data
224 |         assert len(data["messages"]) == 0
225 |         print(f"? Non-existent session returns empty messages array")
226 |
227 |
228 | if __name__ == "__main__":
229 |     pytest.main([__file__, "-v", "--tb=short"])
230 |
```

FILE: backend/tests/test_company_research.py

147 lines | 6049 characters

```
1 | """
2 | Test Company Research API - Perplexity-style research feature
3 | Tests GET /api/research/{symbol} endpoint
4 | """
5 | import pytest
6 | import requests
7 | import os
8 | import time
9 |
10 | BASE_URL = os.environ.get('REACT_APP_BACKEND_URL', '').rstrip('/')
11 |
12 | class TestCompanyResearchAPI:
13 |     """Tests for the Perplexity-style company research endpoint"""
14 |
15 |     def test_api_root_accessible(self):
16 |         """Verify API is running"""
17 |         response = requests.get(f"{BASE_URL}/api/", timeout=30)
18 |         assert response.status_code == 200
19 |         data = response.json()
20 |         assert "RISEDUALAI" in data.get("message", "")
21 |         print("TEST PASSED: API root accessible")
22 |
23 |     def test_research_aapl_returns_data(self):
24 |         """Test research endpoint with AAPL ticker"""
25 |         response = requests.get(f"{BASE_URL}/api/research/AAPL", timeout=90)
26 |         assert response.status_code == 200
27 |         data = response.json()
28 |
29 |         # Verify required fields exist
```

```
30 |         assert "symbol" in data, "Missing 'symbol' field"
31 |         assert "company_name" in data, "Missing 'company_name' field"
32 |         assert "overview" in data, "Missing 'overview' field"
33 |         assert "synthesis" in data, "Missing 'synthesis' field"
34 |         assert "sources" in data, "Missing 'sources' field"
35 |
36 |         # Verify symbol is correct
37 |         assert data["symbol"] == "AAPL"
38 |         # company_name may be just the symbol if Alpha Vantage rate limited
39 |         assert "AAPL" in data["company_name"] or "Apple" in data["company_name"]
40 |
41 |         # Verify synthesis is a non-empty string
42 |         assert isinstance(data["synthesis"], str)
43 |         assert len(data["synthesis"]) > 50, "Synthesis should be substantial"
44 |
45 |         # Verify sources is a list
46 |         assert isinstance(data["sources"], list)
47 |
48 |         print(f"TEST PASSED: AAPL research returned - company: {data['company_name']}, sources: {len(data['sources'])}")
49 |
50 |     def test_research_msft_returns_data(self):
51 |         """Test research endpoint with MSFT ticker"""
52 |         response = requests.get(f"{BASE_URL}/api/research/MSFT", timeout=90)
53 |         assert response.status_code == 200
54 |         data = response.json()
55 |
56 |         # Verify required fields
57 |         assert data["symbol"] == "MSFT"
58 |         # company_name may be just the symbol if Alpha Vantage rate limited
59 |         assert "MSFT" in data["company_name"] or "Microsoft" in data["company_name"]
60 |         assert isinstance(data["synthesis"], str)
61 |         assert len(data["synthesis"]) > 50
62 |
63 |         print(f"TEST PASSED: MSFT research returned - company: {data['company_name']}")
64 |
65 |     def test_research_response_structure(self):
66 |         """Verify the complete response structure"""
67 |         response = requests.get(f"{BASE_URL}/api/research/GOOGL", timeout=90)
68 |         assert response.status_code == 200
69 |         data = response.json()
70 |
71 |         # Check all expected top-level fields
72 |         expected_fields = ["symbol", "company_name", "overview", "quote", "synthesis", "sources", "news_count"]
73 |         for field in expected_fields:
74 |             assert field in data, f"Missing field: {field}"
75 |
76 |         # Verify news_count is an integer
77 |         assert isinstance(data["news_count"], int)
78 |
79 |         print(f"TEST PASSED: Response structure verified for GOOGL - news_count: {data['news_count']}")
80 |
81 |     def test_research_overview_financial_metrics(self):
82 |         """Verify overview contains key financial metrics (may be None due to rate limits)"""
83 |         response = requests.get(f"{BASE_URL}/api/research/AAPL", timeout=60)
84 |         assert response.status_code == 200
85 |         data = response.json()
86 |
87 |         overview = data.get("overview")
88 |
89 |         # Overview may be None due to Alpha Vantage rate limits (5 calls/min)
90 |         if overview is None:
91 |             print("TEST PASSED: Overview is None (likely due to Alpha Vantage rate limit)")
92 |             return
93 |
94 |         # Check for key financial metrics
95 |         financial_fields = [
96 |             "market_cap", "pe_ratio", "eps", "sector", "industry",
```

```
97 |         "week_52_high", "week_52_low", "dividend_yield", "beta"
98 |     ]
99 |
100 |     present_fields = [f for f in financial_fields if f in overview]
101 |     assert len(present_fields) >= 5, f"Expected at least 5 financial fields, got {len(present_fields)}"
102 |
103 |     print(f"TEST PASSED: Overview has {len(present_fields)} financial metrics")
104 |
105 |     def test_research_sources_structure(self):
106 |         """Verify sources array structure"""
107 |         response = requests.get(f"{BASE_URL}/api/research/NVDA", timeout=90)
108 |         assert response.status_code == 200
109 |         data = response.json()
110 |
111 |         sources = data.get("sources", [])
112 |
113 |         # Sources should be a list
114 |         assert isinstance(sources, list)
115 |
116 |         # If sources exist, verify structure
117 |         if len(sources) > 0:
118 |             source = sources[0]
119 |             assert "number" in source, "Source missing 'number' field"
120 |             assert "title" in source, "Source missing 'title' field"
121 |             assert "source" in source, "Source missing 'source' field"
122 |             print(f"TEST PASSED: Sources structure verified - {len(sources)} sources found")
123 |         else:
124 |             print("TEST PASSED: Sources array is empty (may be due to no relevant news)")
125 |
126 |     def test_research_lowercase_symbol(self):
127 |         """Test that lowercase symbols work"""
128 |         response = requests.get(f"{BASE_URL}/api/research/tsla", timeout=90)
129 |         assert response.status_code == 200
130 |         data = response.json()
131 |
132 |         # Symbol should be uppercased in response
133 |         assert data["symbol"] == "TSLA"
134 |         print("TEST PASSED: Lowercase symbol 'tsla' converted to 'TSLA'")
135 |
136 |
137 | @pytest.fixture(scope="module")
138 | def api_client():
139 |     """Shared requests session"""
140 |     session = requests.Session()
141 |     session.headers.update({"Content-Type": "application/json"})
142 |     return session
143 |
144 |
145 | if __name__ == "__main__":
146 |     pytest.main([__file__, "-v", "--tb=short"])
147 |
```

FILE: backend/tests/test_daily_digest.py

443 lines | 19805 characters

```
1 | """
2 | Test suite for Daily Digest Email feature.
3 | Tests: digest status, opt-in/opt-out, preview (admin), trigger (admin), APScheduler setup.
4 | """
5 | import pytest
6 | import requests
7 | import os
8 |
9 | BASE_URL = os.environ.get('REACT_APP_BACKEND_URL', '').rstrip('/')
10 |
11 | # Test credentials from test_credentials.md
```

```
12 | OWNER_EMAIL = "managingdirector@redslateholdings.com"
13 | OWNER_PASSWORD = "RedSlate2026!"
14 | FREE_USER_EMAIL = "freeuser_test@test.com"
15 | FREE_USER_PASSWORD = "Test1234!"
16 |
17 |
18 | class TestDigestEndpointsAuth:
19 |     """Test digest endpoints require authentication."""
20 |
21 |     def test_digest_status_requires_auth(self):
22 |         """GET /api/digest/status should return 401 without auth."""
23 |         response = requests.get(f"{BASE_URL}/api/digest/status")
24 |         assert response.status_code == 401, f"Expected 401, got {response.status_code}"
25 |         print("PASSED: /api/digest/status requires authentication")
26 |
27 |     def test_digest_opt_out_requires_auth(self):
28 |         """POST /api/digest/opt-out should return 401 without auth."""
29 |         response = requests.post(f"{BASE_URL}/api/digest/opt-out")
30 |         assert response.status_code == 401, f"Expected 401, got {response.status_code}"
31 |         print("PASSED: /api/digest/opt-out requires authentication")
32 |
33 |     def test_digest_opt_in_requires_auth(self):
34 |         """POST /api/digest/opt-in should return 401 without auth."""
35 |         response = requests.post(f"{BASE_URL}/api/digest/opt-in")
36 |         assert response.status_code == 401, f"Expected 401, got {response.status_code}"
37 |         print("PASSED: /api/digest/opt-in requires authentication")
38 |
39 |     def test_digest_preview_requires_auth(self):
40 |         """GET /api/digest/preview should return 401 without auth."""
41 |         response = requests.get(f"{BASE_URL}/api/digest/preview")
42 |         assert response.status_code == 401, f"Expected 401, got {response.status_code}"
43 |         print("PASSED: /api/digest/preview requires authentication")
44 |
45 |     def test_digest_trigger_requires_auth(self):
46 |         """POST /api/digest/trigger should return 401 without auth."""
47 |         response = requests.post(f"{BASE_URL}/api/digest/trigger")
48 |         assert response.status_code == 401, f"Expected 401, got {response.status_code}"
49 |         print("PASSED: /api/digest/trigger requires authentication")
50 |
51 |
52 | class TestDigestStatusEndpoint:
53 |     """Test GET /api/digest/status for authenticated users."""
54 |
55 |     @pytest.fixture
56 |     def owner_token(self):
57 |         """Get owner auth token."""
58 |         response = requests.post(f"{BASE_URL}/api/auth/login", json={
59 |             "email": OWNER_EMAIL,
60 |             "password": OWNER_PASSWORD
61 |         })
62 |         if response.status_code != 200:
63 |             pytest.skip(f"Owner login failed: {response.status_code}")
64 |         return response.json().get("access_token")
65 |
66 |     def test_digest_status_returns_subscribed_true_by_default(self, owner_token):
67 |         """GET /api/digest/status should return subscribed:true by default."""
68 |         headers = {"Authorization": f"Bearer {owner_token}"}
69 |         response = requests.get(f"{BASE_URL}/api/digest/status", headers=headers)
70 |
71 |         assert response.status_code == 200, f"Expected 200, got {response.status_code}"
72 |         data = response.json()
73 |         assert "subscribed" in data, "Response should contain 'subscribed' field"
74 |         # Default is subscribed (digest_opt_out is False or not set)
75 |         print(f"PASSED: /api/digest/status returns subscribed={data['subscribed']}")
76 |
77 |
78 | class TestDigestOptOutOptIn:
```

```
79 |     """Test opt-out and opt-in flow."""
80 |
81 |     @pytest.fixture
82 |     def owner_token(self):
83 |         """Get owner auth token."""
84 |         response = requests.post(f"{BASE_URL}/api/auth/login", json={
85 |             "email": OWNER_EMAIL,
86 |             "password": OWNER_PASSWORD
87 |         })
88 |         if response.status_code != 200:
89 |             pytest.skip(f"Owner login failed: {response.status_code}")
90 |         return response.json().get("access_token")
91 |
92 |     def test_opt_out_sets_subscribed_to_false(self, owner_token):
93 |         """POST /api/digest/opt-out should set subscribed to false."""
94 |         headers = {"Authorization": f"Bearer {owner_token}"}
95 |
96 |         # Opt out
97 |         response = requests.post(f"{BASE_URL}/api/digest/opt-out", headers=headers)
98 |         assert response.status_code == 200, f"Expected 200, got {response.status_code}"
99 |         data = response.json()
100 |         assert "message" in data, "Response should contain 'message' field"
101 |         print(f"PASSED: opt-out response: {data['message']}")
102 |
103 |         # Verify status is now false
104 |         status_response = requests.get(f"{BASE_URL}/api/digest/status", headers=headers)
105 |         assert status_response.status_code == 200
106 |         status_data = status_response.json()
107 |         assert status_data["subscribed"] == False, f"Expected subscribed=False after opt-out, got {status_data['subscribed']}"
108 |         print("PASSED: After opt-out, subscribed=False")
109 |
110 |     def test_opt_in_sets_subscribed_to_true(self, owner_token):
111 |         """POST /api/digest/opt-in should set subscribed back to true."""
112 |         headers = {"Authorization": f"Bearer {owner_token}"}
113 |
114 |         # Opt in
115 |         response = requests.post(f"{BASE_URL}/api/digest/opt-in", headers=headers)
116 |         assert response.status_code == 200, f"Expected 200, got {response.status_code}"
117 |         data = response.json()
118 |         assert "message" in data, "Response should contain 'message' field"
119 |         print(f"PASSED: opt-in response: {data['message']}")
120 |
121 |         # Verify status is now true
122 |         status_response = requests.get(f"{BASE_URL}/api/digest/status", headers=headers)
123 |         assert status_response.status_code == 200
124 |         status_data = status_response.json()
125 |         assert status_data["subscribed"] == True, f"Expected subscribed=True after opt-in, got {status_data['subscribed']}"
126 |         print("PASSED: After opt-in, subscribed=True")
127 |
128 |
129 | class TestDigestPreviewAdminOnly:
130 |     """Test GET /api/digest/preview is admin-only."""
131 |
132 |     @pytest.fixture
133 |     def owner_token(self):
134 |         """Get owner auth token."""
135 |         response = requests.post(f"{BASE_URL}/api/auth/login", json={
136 |             "email": OWNER_EMAIL,
137 |             "password": OWNER_PASSWORD
138 |         })
139 |         if response.status_code != 200:
140 |             pytest.skip(f"Owner login failed: {response.status_code}")
141 |         return response.json().get("access_token")
142 |
143 |     @pytest.fixture
144 |     def free_user_token(self):
145 |         """Get free user auth token (create if needed)."""
```



```
146 |         # Try to login first
147 |         response = requests.post(f"{BASE_URL}/api/auth/login", json={
148 |             "email": FREE_USER_EMAIL,
149 |             "password": FREE_USER_PASSWORD
150 |         })
151 |         if response.status_code == 200:
152 |             return response.json().get("access_token")
153 |
154 |         # Create free user if doesn't exist
155 |         register_response = requests.post(f"{BASE_URL}/api/auth/register", json={
156 |             "email": FREE_USER_EMAIL,
157 |             "password": FREE_USER_PASSWORD,
158 |             "name": "Free Test User"
159 |         })
160 |         if register_response.status_code in [200, 201]:
161 |             # Login after registration
162 |             login_response = requests.post(f"{BASE_URL}/api/auth/login", json={
163 |                 "email": FREE_USER_EMAIL,
164 |                 "password": FREE_USER_PASSWORD
165 |             })
166 |             if login_response.status_code == 200:
167 |                 return login_response.json().get("access_token")
168 |
169 |         pytest.skip("Could not create/login free user")
170 |
171 |     def test_preview_returns_html_and_data_summary_for_admin(self, owner_token):
172 |         """GET /api/digest/preview should return HTML and data_summary for admin."""
173 |         headers = {"Authorization": f"Bearer {owner_token}"}
174 |         response = requests.get(f"{BASE_URL}/api/digest/preview", headers=headers)
175 |
176 |         assert response.status_code == 200, f"Expected 200, got {response.status_code}"
177 |         data = response.json()
178 |
179 |         # Check response structure
180 |         assert "html" in data, "Response should contain 'html' field"
181 |         assert "data_summary" in data, "Response should contain 'data_summary' field"
182 |
183 |         # Check HTML is valid
184 |         html = data["html"]
185 |         assert "<!DOCTYPE html>" in html, "HTML should be valid document"
186 |         assert "RISEDUAL AI" in html, "HTML should contain app name"
187 |         assert "Daily Market Digest" in html, "HTML should contain digest title"
188 |
189 |         # Check data_summary structure
190 |         summary = data["data_summary"]
191 |         assert "predictions" in summary, "data_summary should have 'predictions' count"
192 |         assert "dark_pool" in summary, "data_summary should have 'dark_pool' count"
193 |         assert "signals" in summary, "data_summary should have 'signals' count"
194 |
195 |         print(f"PASSED: Preview returns HTML ({len(html)} chars) and data_summary: {summary}")
196 |
197 |     def test_preview_returns_403_for_free_user(self, free_user_token):
198 |         """GET /api/digest/preview should return 403 for non-admin users."""
199 |         headers = {"Authorization": f"Bearer {free_user_token}"}
200 |         response = requests.get(f"{BASE_URL}/api/digest/preview", headers=headers)
201 |
202 |         assert response.status_code == 403, f"Expected 403 for free user, got {response.status_code}"
203 |         print("PASSED: Preview returns 403 for free user (admin only)")
204 |
205 |
206 |     class TestDigestTriggerAdminOnly:
207 |         """Test POST /api/digest/trigger is admin-only and returns skipped with placeholder key."""
208 |
209 |         @pytest.fixture
210 |         def owner_token(self):
211 |             """Get owner auth token."""
212 |             response = requests.post(f"{BASE_URL}/api/auth/login", json={
```

```
213 |         "email": OWNER_EMAIL,
214 |         "password": OWNER_PASSWORD
215 |     })
216 |     if response.status_code != 200:
217 |         pytest.skip(f"Owner login failed: {response.status_code}")
218 |     return response.json().get("access_token")
219 |
220 | @pytest.fixture
221 | def free_user_token(self):
222 |     """Get free user auth token."""
223 |     response = requests.post(f"{BASE_URL}/api/auth/login", json={
224 |         "email": FREE_USER_EMAIL,
225 |         "password": FREE_USER_PASSWORD
226 |     })
227 |     if response.status_code == 200:
228 |         return response.json().get("access_token")
229 |     pytest.skip("Free user login failed")
230 |
231 | def test_trigger_returns_skipped_true_with_placeholder_key(self, owner_token):
232 |     """POST /api/digest/trigger should return skipped:true with placeholder API key."""
233 |     headers = {"Authorization": f"Bearer {owner_token}"}
234 |     response = requests.post(f"{BASE_URL}/api/digest/trigger", headers=headers)
235 |
236 |     assert response.status_code == 200, f"Expected 200, got {response.status_code}"
237 |     data = response.json()
238 |
239 |     # With placeholder key, should return skipped:true
240 |     assert "skipped" in data, "Response should contain 'skipped' field"
241 |     assert data["skipped"] == True, f"Expected skipped=True with placeholder key, got {data['skipped']}"
242 |     assert "reason" in data, "Response should contain 'reason' field"
243 |     assert data["reason"] == "no_api_key", f"Expected reason='no_api_key', got {data['reason']}"
244 |
245 |     print(f"PASSED: Trigger returns skipped=True with reason={data['reason']}")
246 |
247 | def test_trigger_returns_403_for_free_user(self, free_user_token):
248 |     """POST /api/digest/trigger should return 403 for non-admin users."""
249 |     headers = {"Authorization": f"Bearer {free_user_token}"}
250 |     response = requests.post(f"{BASE_URL}/api/digest/trigger", headers=headers)
251 |
252 |     assert response.status_code == 403, f"Expected 403 for free user, got {response.status_code}"
253 |     print("PASSED: Trigger returns 403 for free user (admin only)")
254 |
255 |
256 | class TestDigestServiceModule:
257 |     """Test digest_service.py module functions."""
258 |
259 |     def test_digest_service_imports_without_errors(self):
260 |         """digest_service.py should import without errors."""
261 |         import sys
262 |         sys.path.insert(0, '/app/backend')
263 |         try:
264 |             from services.digest_service import collect_digest_data, build_digest_html, send_daily_digest
265 |             print("PASSED: digest_service.py imports successfully")
266 |         except ImportError as e:
267 |             pytest.fail(f"Failed to import digest_service: {e}")
268 |
269 |     def test_build_digest_html_generates_valid_html_for_pro_user(self):
270 |         """build_digest_html should generate valid HTML for Pro users."""
271 |         import sys
272 |         sys.path.insert(0, '/app/backend')
273 |         from services.digest_service import build_digest_html
274 |
275 |         # Sample data
276 |         data = {
277 |             "predictions": [
278 |                 {"ticker": "AAPL", "verdict": "BULLISH", "confidence": 85, "summary": "Strong buy"},
279 |                 {"ticker": "TSLA", "verdict": "BEARISH", "confidence": 70, "summary": "Sell signal"},
```

```
280 |         ],
281 |         "dark_pool": [
282 |             {"ticker": "NVDA", "volume": 1000000, "sentiment": "bullish"},
283 |         ],
284 |         "signals": [
285 |             {"ticker": "MSFT", "signal": "BUY", "strength": "strong"},
286 |         ],
287 |         "timestamp": "2026-01-06T06:00:00Z"
288 |     }
289 |
290 |     html = build_digest_html(data, is_pro=True, user_name="Pro User")
291 |
292 |     # Validate HTML structure
293 |     assert "<!DOCTYPE html>" in html, "Should be valid HTML document"
294 |     assert "Good Morning, Pro User" in html, "Should contain user greeting"
295 |     assert "AAPL" in html, "Should contain ticker data"
296 |     assert "BULLISH" in html, "Should contain verdict"
297 |     assert "Upgrade to Pro" not in html, "Pro user should NOT see upgrade CTA"
298 |
299 |     print("PASSED: build_digest_html generates valid HTML for Pro user (no upgrade CTA)")
300 |
301 |     def test_build_digest_html_generates_blurred_rows_for_free_user(self):
302 |         """build_digest_html should generate blurred rows and upgrade CTA for free users."""
303 |         import sys
304 |         sys.path.insert(0, '/app/backend')
305 |         from services.digest_service import build_digest_html
306 |
307 |         # Sample data with multiple items
308 |         data = {
309 |             "predictions": [
310 |                 {"ticker": "AAPL", "verdict": "BULLISH", "confidence": 85, "summary": "Strong buy"},
311 |                 {"ticker": "TSLA", "verdict": "BEARISH", "confidence": 70, "summary": "Sell signal"},
312 |                 {"ticker": "NVDA", "verdict": "NEUTRAL", "confidence": 60, "summary": "Hold"},
313 |             ],
314 |             "dark_pool": [
315 |                 {"ticker": "NVDA", "volume": 1000000, "sentiment": "bullish"},
316 |                 {"ticker": "AMD", "volume": 500000, "sentiment": "bearish"},
317 |             ],
318 |             "signals": [
319 |                 {"ticker": "MSFT", "signal": "BUY", "strength": "strong"},
320 |                 {"ticker": "GOOG", "signal": "SELL", "strength": "weak"},
321 |             ],
322 |             "timestamp": "2026-01-06T06:00:00Z"
323 |         }
324 |
325 |         html = build_digest_html(data, is_pro=False, user_name="Free User")
326 |
327 |         # Validate HTML structure
328 |         assert "<!DOCTYPE html>" in html, "Should be valid HTML document"
329 |         assert "Good Morning, Free User" in html, "Should contain user greeting"
330 |         assert "AAPL" in html, "First item should be visible"
331 |         assert "filter:blur(4px)" in html, "Should have blurred rows for free user"
332 |         assert "Upgrade to Pro" in html, "Free user should see upgrade CTA"
333 |         assert "$45/mo" in html, "Should show pricing in CTA"
334 |
335 |         print("PASSED: build_digest_html generates blurred rows and upgrade CTA for free user")
336 |
337 |     def test_build_digest_html_handles_empty_data_gracefully(self):
338 |         """build_digest_html should handle empty collections gracefully."""
339 |         import sys
340 |         sys.path.insert(0, '/app/backend')
341 |         from services.digest_service import build_digest_html
342 |
343 |         # Empty data
344 |         data = {
345 |             "predictions": [],
346 |             "dark_pool": [],
```

```
347 |         "signals": [],
348 |         "timestamp": "2026-01-06T06:00:00Z"
349 |     }
350 |
351 |     html = build_digest_html(data, is_pro=True, user_name="Test User")
352 |
353 |     # Should not crash and should show "No recent data available"
354 |     assert "<!DOCTYPE html>" in html, "Should be valid HTML document"
355 |     assert "No recent data available" in html, "Should show no data message"
356 |
357 |     print("PASSED: build_digest_html handles empty data gracefully")
358 |
359 |
360 | class TestAPSchedulerSetup:
361 |     """Test APScheduler is configured correctly."""
362 |
363 |     def test_apscheduler_log_message_present(self):
364 |         """Backend logs should show APScheduler started message."""
365 |         import subprocess
366 |         result = subprocess.run(
367 |             ["tail", "-n", "100", "/var/log/supervisor/backend.err.log"],
368 |             capture_output=True, text=True
369 |         )
370 |         logs = result.stdout
371 |
372 |         assert "Daily digest scheduler started (6:00 AM UTC)" in logs, \
373 |             "Backend logs should contain APScheduler startup message"
374 |
375 |         print("PASSED: APScheduler is running and configured for 6:00 AM UTC")
376 |
377 |
378 | class TestDigestFreeUserFlow:
379 |     """Test complete flow for free user digest preferences."""
380 |
381 |     @pytest.fixture
382 |     def free_user_token(self):
383 |         """Get or create free user token."""
384 |         # Try login
385 |         response = requests.post(f"{BASE_URL}/api/auth/login", json={
386 |             "email": FREE_USER_EMAIL,
387 |             "password": FREE_USER_PASSWORD
388 |         })
389 |         if response.status_code == 200:
390 |             return response.json().get("access_token")
391 |
392 |         # Register if needed
393 |         requests.post(f"{BASE_URL}/api/auth/register", json={
394 |             "email": FREE_USER_EMAIL,
395 |             "password": FREE_USER_PASSWORD,
396 |             "name": "Free Test User"
397 |         })
398 |
399 |         # Login again
400 |         response = requests.post(f"{BASE_URL}/api/auth/login", json={
401 |             "email": FREE_USER_EMAIL,
402 |             "password": FREE_USER_PASSWORD
403 |         })
404 |         if response.status_code == 200:
405 |             return response.json().get("access_token")
406 |         pytest.skip("Could not get free user token")
407 |
408 |     def test_free_user_can_check_digest_status(self, free_user_token):
409 |         """Free user should be able to check their digest subscription status."""
410 |         headers = {"Authorization": f"Bearer {free_user_token}"}
411 |         response = requests.get(f"{BASE_URL}/api/digest/status", headers=headers)
412 |
413 |         assert response.status_code == 200, f"Expected 200, got {response.status_code}"
```

```
414 |         data = response.json()
415 |         assert "subscribed" in data
416 |         print(f"PASSED: Free user can check digest status: subscribed={data['subscribed']}")
417 |
418 |     def test_free_user_can_opt_out_and_opt_in(self, free_user_token):
419 |         """Free user should be able to opt-out and opt-in to digest."""
420 |         headers = {"Authorization": f"Bearer {free_user_token}"}
421 |
422 |         # Opt out
423 |         opt_out_response = requests.post(f"{BASE_URL}/api/digest/opt-out", headers=headers)
424 |         assert opt_out_response.status_code == 200
425 |
426 |         # Verify opted out
427 |         status_response = requests.get(f"{BASE_URL}/api/digest/status", headers=headers)
428 |         assert status_response.json()["subscribed"] == False
429 |
430 |         # Opt back in
431 |         opt_in_response = requests.post(f"{BASE_URL}/api/digest/opt-in", headers=headers)
432 |         assert opt_in_response.status_code == 200
433 |
434 |         # Verify opted in
435 |         status_response = requests.get(f"{BASE_URL}/api/digest/status", headers=headers)
436 |         assert status_response.json()["subscribed"] == True
437 |
438 |         print("PASSED: Free user can opt-out and opt-in to digest")
439 |
440 |
441 | if __name__ == "__main__":
442 |     pytest.main([__file__, "-v", "--tb=short"])
443 |
```

FILE: backend/tests/test_email_notifications.py

302 lines | 13734 characters

```
1 | """
2 | Test suite for Resend email notification integration in RISEDUAL AI referral system.
3 | Tests: email_service.py functions, graceful degradation with placeholder API key,
4 | and referral flow integration with email notifications.
5 | """
6 | import pytest
7 | import requests
8 | import os
9 | import sys
10 | import asyncio
11 |
12 | # Add backend to path for imports
13 | sys.path.insert(0, '/app/backend')
14 |
15 | BASE_URL = os.environ.get('REACT_APP_BACKEND_URL', '').rstrip('/')
16 |
17 | # Test credentials from test_credentials.md
18 | OWNER_EMAIL = "managingdirector@redslateholdings.com"
19 | OWNER_PASSWORD = "RedSlate2026!"
20 |
21 |
22 | class TestEmailServiceModule:
23 |     """Test email_service.py module loading and configuration"""
24 |
25 |     def test_email_service_imports_without_errors(self):
26 |         """Verify email_service.py can be imported without errors"""
27 |         try:
28 |             from services.email_service import (
29 |                 send_referral_signup_email,
30 |                 send_reward_earned_email,
31 |                 send_welcome_referral_email,
32 |                 _is_configured
```

```
33 |         )
34 |         assert callable(send_referral_signup_email)
35 |         assert callable(send_reward_earned_email)
36 |         assert callable(send_welcome_referral_email)
37 |         assert callable(_is_configured)
38 |         print("PASS: email_service.py imports successfully")
39 |     except ImportError as e:
40 |         pytest.fail(f"Failed to import email_service: {e}")
41 |
42 |     def test_is_configured_returns_false_with_placeholder_key(self):
43 |         """_is_configured() should return False when API key is placeholder"""
44 |         from services.email_service import _is_configured, RESEND_API_KEY
45 |
46 |         result = _is_configured()
47 |         assert result is False, f"Expected False with placeholder key, got {result}"
48 |         assert RESEND_API_KEY.startswith('re_YOUR'), f"Expected placeholder key, got {RESEND_API_KEY}"
49 |         print(f"PASS: _is_configured() returns False (API key: {RESEND_API_KEY[:15]}...)")
50 |
51 |     def test_resend_api_key_in_env(self):
52 |         """Verify RESEND_API_KEY is set in environment"""
53 |         from services.email_service import RESEND_API_KEY
54 |
55 |         assert RESEND_API_KEY is not None, "RESEND_API_KEY should not be None"
56 |         assert len(RESEND_API_KEY) > 0, "RESEND_API_KEY should not be empty"
57 |         assert RESEND_API_KEY == "re_YOUR_API_KEY_HERE", f"Expected placeholder, got {RESEND_API_KEY}"
58 |         print(f"PASS: RESEND_API_KEY is set to placeholder value")
59 |
60 |     def test_sender_email_in_env(self):
61 |         """Verify SENDER_EMAIL is set in environment"""
62 |         from services.email_service import SENDER_EMAIL
63 |
64 |         assert SENDER_EMAIL is not None, "SENDER_EMAIL should not be None"
65 |         assert len(SENDER_EMAIL) > 0, "SENDER_EMAIL should not be empty"
66 |         assert "@" in SENDER_EMAIL, "SENDER_EMAIL should be a valid email format"
67 |         print(f"PASS: SENDER_EMAIL is set to {SENDER_EMAIL}")
68 |
69 |
70 | class TestEmailFunctionsGracefulDegradation:
71 |     """Test that email functions return False gracefully when not configured"""
72 |
73 |     @pytest.mark.asyncio
74 |     async def test_send_referral_signup_email_returns_false_when_not_configured(self):
75 |         """send_referral_signup_email should return False without crashing"""
76 |         from services.email_service import send_referral_signup_email
77 |
78 |         result = await send_referral_signup_email(
79 |             referrer_email="test@example.com",
80 |             referrer_name="Test User",
81 |             referred_email="friend@example.com"
82 |         )
83 |
84 |         assert result is False, f"Expected False when not configured, got {result}"
85 |         print("PASS: send_referral_signup_email returns False gracefully")
86 |
87 |     @pytest.mark.asyncio
88 |     async def test_send_reward_earned_email_returns_false_when_not_configured(self):
89 |         """send_reward_earned_email should return False without crashing"""
90 |         from services.email_service import send_reward_earned_email
91 |
92 |         result = await send_reward_earned_email(
93 |             referrer_email="test@example.com",
94 |             referrer_name="Test User",
95 |             referred_email="friend@example.com"
96 |         )
97 |
98 |         assert result is False, f"Expected False when not configured, got {result}"
99 |         print("PASS: send_reward_earned_email returns False gracefully")
```

```
100 |
101 |     @pytest.mark.asyncio
102 |     async def test_send_welcome_referral_email_returns_false_when_not_configured(self):
103 |         """send_welcome_referral_email should return False without crashing"""
104 |         from services.email_service import send_welcome_referral_email
105 |
106 |         result = await send_welcome_referral_email(
107 |             user_email="newuser@example.com",
108 |             user_name="New User",
109 |             referrer_name="Referrer Name"
110 |         )
111 |
112 |         assert result is False, f"Expected False when not configured, got {result}"
113 |         print("PASS: send_welcome_referral_email returns False gracefully")
114 |
115 |
116 | class TestEmailHTMLTemplates:
117 |     """Test that HTML templates render correctly"""
118 |
119 |     def test_referral_signup_html_renders(self):
120 |         """_referral_signup_html should generate valid HTML string"""
121 |         from services.email_service import _referral_signup_html
122 |
123 |         html = _referral_signup_html("John Doe", "friend@example.com")
124 |
125 |         assert isinstance(html, str), "Should return a string"
126 |         assert len(html) > 100, "HTML should have substantial content"
127 |         assert "<!DOCTYPE html>" in html, "Should be valid HTML"
128 |         assert "John Doe" in html, "Should contain referrer name"
129 |         assert "friend@example.com" in html, "Should contain referred email"
130 |         assert "Your friend just signed up" in html, "Should contain expected heading"
131 |         assert "RISEDUAL AI" in html, "Should contain app name"
132 |         print("PASS: _referral_signup_html renders correctly")
133 |
134 |     def test_reward_earned_html_renders(self):
135 |         """_reward_earned_html should generate valid HTML string"""
136 |         from services.email_service import _reward_earned_html
137 |
138 |         html = _reward_earned_html("Jane Smith", "subscriber@example.com")
139 |
140 |         assert isinstance(html, str), "Should return a string"
141 |         assert len(html) > 100, "HTML should have substantial content"
142 |         assert "<!DOCTYPE html>" in html, "Should be valid HTML"
143 |         assert "Jane Smith" in html, "Should contain referrer name"
144 |         assert "subscriber@example.com" in html, "Should contain referred email"
145 |         assert "You earned a free month" in html, "Should contain expected heading"
146 |         assert "+1 Month Free" in html, "Should contain reward text"
147 |         print("PASS: _reward_earned_html renders correctly")
148 |
149 |     def test_welcome_referral_html_renders(self):
150 |         """_welcome_referral_html should generate valid HTML string"""
151 |         from services.email_service import _welcome_referral_html
152 |
153 |         html = _welcome_referral_html("New User", "Referrer Name")
154 |
155 |         assert isinstance(html, str), "Should return a string"
156 |         assert len(html) > 100, "HTML should have substantial content"
157 |         assert "<!DOCTYPE html>" in html, "Should be valid HTML"
158 |         assert "New User" in html, "Should contain user name"
159 |         assert "Referrer Name" in html, "Should contain referrer name"
160 |         assert "Welcome to RISEDUAL AI" in html, "Should contain expected heading"
161 |         assert "7-Day Pro Trial" in html, "Should contain trial info"
162 |         print("PASS: _welcome_referral_html renders correctly")
163 |
164 |
165 | class TestReferralFlowWithPlaceholderEmail:
166 |     """Test that referral flows work correctly even with placeholder email key"""
```

```
167 |
168 |     @pytest.fixture
169 |     def owner_token(self):
170 |         """Get owner authentication token"""
171 |         response = requests.post(f"{BASE_URL}/api/auth/login", json={
172 |             "email": OWNER_EMAIL,
173 |             "password": OWNER_PASSWORD
174 |         })
175 |         if response.status_code == 200:
176 |             data = response.json()
177 |             return data.get("access_token") or data.get("token")
178 |         pytest.skip(f"Owner login failed: {response.status_code}")
179 |
180 |     def test_referral_validate_endpoint_works(self):
181 |         """Referral code validation should work regardless of email config"""
182 |         # Get owner's referral code first
183 |         response = requests.post(f"{BASE_URL}/api/auth/login", json={
184 |             "email": OWNER_EMAIL,
185 |             "password": OWNER_PASSWORD
186 |         })
187 |         if response.status_code != 200:
188 |             pytest.skip("Login failed")
189 |
190 |         token = response.json().get("access_token") or response.json().get("token")
191 |
192 |         # Get referral info
193 |         headers = {"Authorization": f"Bearer {token}"}
194 |         info_response = requests.get(f"{BASE_URL}/api/referral/info", headers=headers)
195 |
196 |         if info_response.status_code == 200:
197 |             code = info_response.json().get("code")
198 |             if code:
199 |                 # Validate the code
200 |                 validate_response = requests.get(f"{BASE_URL}/api/referral/validate/{code}")
201 |                 assert validate_response.status_code == 200
202 |                 data = validate_response.json()
203 |                 assert data.get("valid") is True
204 |                 print(f"PASS: Referral code {code} validates correctly")
205 |             else:
206 |                 print("PASS: Referral info endpoint works (no code yet)")
207 |         else:
208 |             pytest.skip(f"Referral info failed: {info_response.status_code}")
209 |
210 |     def test_referral_info_endpoint_works(self, owner_token):
211 |         """Referral info endpoint should work regardless of email config"""
212 |         headers = {"Authorization": f"Bearer {owner_token}"}
213 |         response = requests.get(f"{BASE_URL}/api/referral/info", headers=headers)
214 |
215 |         assert response.status_code == 200, f"Expected 200, got {response.status_code}"
216 |         data = response.json()
217 |
218 |         # Verify response structure
219 |         assert "code" in data, "Response should contain 'code'"
220 |         assert "total_referrals" in data, "Response should contain 'total_referrals'"
221 |         assert "completed_referrals" in data, "Response should contain 'completed_referrals'"
222 |         assert "rewards_earned" in data, "Response should contain 'rewards_earned'"
223 |
224 |         print(f"PASS: Referral info works - code: {data['code']}, total: {data['total_referrals']}")
225 |
226 |     def test_referral_leaderboard_endpoint_works(self):
227 |         """Leaderboard endpoint should work regardless of email config"""
228 |         response = requests.get(f"{BASE_URL}/api/referral/leaderboard")
229 |
230 |         assert response.status_code == 200, f"Expected 200, got {response.status_code}"
231 |         data = response.json()
232 |
233 |         assert "leaderboard" in data, "Response should contain 'leaderboard'"
```



```
234 |         assert "total_participants" in data, "Response should contain 'total_participants'"
235 |
236 |         print(f"PASS: Leaderboard works - {len(data['leaderboard'])} entries, {data['total_participants']} participants")
237 |
238 |
239 | class TestReferralPyEmailIntegration:
240 |     """Test that referral.py correctly uses asyncio.create_task for email sends"""
241 |
242 |     def test_referral_py_uses_asyncio_create_task_for_reward_email(self):
243 |         """Verify referral.py uses asyncio.create_task for non-blocking email sends in complete_referral_reward"""
244 |         import inspect
245 |         from routes.referral import complete_referral_reward
246 |
247 |         source = inspect.getsource(complete_referral_reward)
248 |
249 |         assert "asyncio.create_task" in source, "complete_referral_reward should use asyncio.create_task"
250 |         assert "send_reward_earned_email" in source, "complete_referral_reward should call send_reward_earned_email"
251 |         print("PASS: complete_referral_reward uses asyncio.create_task for non-blocking email")
252 |
253 |     def test_email_service_import_in_complete_referral_reward(self):
254 |         """Verify email_service is imported in complete_referral_reward"""
255 |         import inspect
256 |         from routes.referral import complete_referral_reward
257 |
258 |         source = inspect.getsource(complete_referral_reward)
259 |
260 |         assert "from services.email_service import" in source, "Should import from email_service"
261 |         print("PASS: complete_referral_reward imports email_service correctly")
262 |
263 |
264 | class TestResendPackageInstalled:
265 |     """Verify resend package is installed"""
266 |
267 |     def test_resend_package_importable(self):
268 |         """resend package should be importable"""
269 |         try:
270 |             import resend
271 |             assert hasattr(resend, 'Emails'), "resend should have Emails attribute"
272 |             assert hasattr(resend, 'api_key'), "resend should have api_key attribute"
273 |             print(f"PASS: resend package installed and importable")
274 |         except ImportError as e:
275 |             pytest.fail(f"resend package not installed: {e}")
276 |
277 |
278 | class TestBackendEnvConfiguration:
279 |     """Test backend .env has required email configuration"""
280 |
281 |     def test_env_file_has_resend_api_key(self):
282 |         """backend/.env should have RESEND_API_KEY"""
283 |         env_path = "/app/backend/.env"
284 |         with open(env_path, 'r') as f:
285 |             content = f.read()
286 |
287 |         assert "RESEND_API_KEY=" in content, "RESEND_API_KEY should be in .env"
288 |         print("PASS: RESEND_API_KEY found in backend/.env")
289 |
290 |     def test_env_file_has_sender_email(self):
291 |         """backend/.env should have SENDER_EMAIL"""
292 |         env_path = "/app/backend/.env"
293 |         with open(env_path, 'r') as f:
294 |             content = f.read()
295 |
296 |         assert "SENDER_EMAIL=" in content, "SENDER_EMAIL should be in .env"
297 |         print("PASS: SENDER_EMAIL found in backend/.env")
298 |
299 |
300 | if __name__ == "__main__":
```

```
301 |     pytest.main([__file__, "-v", "--tb=short"])
302 |
```

FILE: backend/tests/test_leaderboard_feature.py*176 lines | 8315 characters*

```
1 | """
2 | Test suite for Referral Leaderboard feature
3 | Tests: GET /api/referral/leaderboard endpoint (PUBLIC - no auth required)
4 | """
5 | import pytest
6 | import requests
7 | import os
8 |
9 | BASE_URL = os.environ.get('REACT_APP_BACKEND_URL', '').rstrip('/')
10 |
11 | # Test credentials
12 | OWNER_EMAIL = "managingdirector@redslateholdings.com"
13 | OWNER_PASSWORD = "RedSlate2026!"
14 |
15 |
16 | class TestLeaderboardEndpoint:
17 |     """Tests for the public leaderboard endpoint"""
18 |
19 |     def test_leaderboard_is_public_no_auth_required(self):
20 |         """Leaderboard should be accessible without authentication"""
21 |         response = requests.get(f"{BASE_URL}/api/referral/leaderboard")
22 |         assert response.status_code == 200, f"Expected 200, got {response.status_code}"
23 |         data = response.json()
24 |         assert "leaderboard" in data, "Response should contain 'leaderboard' key"
25 |         assert "total_participants" in data, "Response should contain 'total_participants' key"
26 |         print(f"? Leaderboard is public (no auth required)")
27 |
28 |     def test_leaderboard_returns_array(self):
29 |         """Leaderboard should return an array"""
30 |         response = requests.get(f"{BASE_URL}/api/referral/leaderboard")
31 |         assert response.status_code == 200
32 |         data = response.json()
33 |         assert isinstance(data["leaderboard"], list), "leaderboard should be a list"
34 |         print(f"? Leaderboard returns array with {len(data['leaderboard'])} entries")
35 |
36 |     def test_leaderboard_entry_structure(self):
37 |         """Each leaderboard entry should have rank, name, referrals"""
38 |         response = requests.get(f"{BASE_URL}/api/referral/leaderboard")
39 |         assert response.status_code == 200
40 |         data = response.json()
41 |
42 |         if len(data["leaderboard"]) > 0:
43 |             entry = data["leaderboard"][0]
44 |             assert "rank" in entry, "Entry should have 'rank'"
45 |             assert "name" in entry, "Entry should have 'name'"
46 |             assert "referrals" in entry, "Entry should have 'referrals'"
47 |             assert isinstance(entry["rank"], int), "rank should be integer"
48 |             assert isinstance(entry["name"], str), "name should be string"
49 |             assert isinstance(entry["referrals"], int), "referrals should be integer"
50 |             print(f"? Entry structure correct: rank={entry['rank']}, name={entry['name']}, referrals={entry['referrals']}")
51 |         else:
52 |             print("? Leaderboard is empty (no completed referrals yet)")
53 |
54 |     def test_leaderboard_names_are_masked(self):
55 |         """Names should be partially masked (e.g., 'John' -> 'J***')"""
56 |         response = requests.get(f"{BASE_URL}/api/referral/leaderboard")
57 |         assert response.status_code == 200
58 |         data = response.json()
59 |
60 |         if len(data["leaderboard"]) > 0:
```

```

61 |         for entry in data["leaderboard"]:
62 |             name = entry["name"]
63 |             # Masked names should contain asterisks
64 |             if name != "Anonymous":
65 |                 assert "*" in name, f"Name '{name}' should be masked with asterisks"
66 |                 # First character of each word should be visible
67 |                 parts = name.split()
68 |                 for part in parts:
69 |                     assert len(part) > 1, f"Masked part '{part}' should have at least 2 chars"
70 |                     assert part[0] != "*", f"First char of '{part}' should not be asterisk"
71 |                 print(f"? Name properly masked: {name}")
72 |             else:
73 |                 print("? Leaderboard is empty - no names to mask")
74 |
75 |     def test_leaderboard_sorted_by_referrals_descending(self):
76 |         """Leaderboard should be sorted by referral count (highest first)"""
77 |         response = requests.get(f"{BASE_URL}/api/referral/leaderboard")
78 |         assert response.status_code == 200
79 |         data = response.json()
80 |
81 |         if len(data["leaderboard"]) > 1:
82 |             referral_counts = [entry["referrals"] for entry in data["leaderboard"]]
83 |             assert referral_counts == sorted(referral_counts, reverse=True), \
84 |                 f"Leaderboard not sorted descending: {referral_counts}"
85 |             print(f"? Leaderboard sorted by referrals descending: {referral_counts}")
86 |         else:
87 |             print("? Only 0-1 entries - sorting not applicable")
88 |
89 |     def test_leaderboard_ranks_are_sequential(self):
90 |         """Ranks should be sequential starting from 1"""
91 |         response = requests.get(f"{BASE_URL}/api/referral/leaderboard")
92 |         assert response.status_code == 200
93 |         data = response.json()
94 |
95 |         if len(data["leaderboard"]) > 0:
96 |             ranks = [entry["rank"] for entry in data["leaderboard"]]
97 |             expected_ranks = list(range(1, len(ranks) + 1))
98 |             assert ranks == expected_ranks, f"Ranks should be sequential: got {ranks}, expected {expected_ranks}"
99 |             print(f"? Ranks are sequential: {ranks}")
100 |        else:
101 |            print("? Leaderboard is empty - no ranks to check")
102 |
103 |     def test_leaderboard_limited_to_top_10(self):
104 |         """Leaderboard should be limited to top 10 entries"""
105 |         response = requests.get(f"{BASE_URL}/api/referral/leaderboard")
106 |         assert response.status_code == 200
107 |         data = response.json()
108 |
109 |         assert len(data["leaderboard"]) <= 10, \
110 |             f"Leaderboard should have max 10 entries, got {len(data['leaderboard'])}"
111 |         print(f"? Leaderboard limited to top 10 (currently {len(data['leaderboard'])} entries)")
112 |
113 |     def test_total_participants_count(self):
114 |         """total_participants should be a non-negative integer"""
115 |         response = requests.get(f"{BASE_URL}/api/referral/leaderboard")
116 |         assert response.status_code == 200
117 |         data = response.json()
118 |
119 |         assert isinstance(data["total_participants"], int), "total_participants should be integer"
120 |         assert data["total_participants"] >= 0, "total_participants should be non-negative"
121 |         print(f"? total_participants = {data['total_participants']}")
122 |
123 |
124 | class TestNameMaskingLogic:
125 |     """Tests for the _mask_name helper function behavior"""
126 |
127 |     def test_owner_has_completed_referrals(self):

```

```

128 |         """Verify owner account has completed referrals (should appear on leaderboard)"""
129 |         # Login as owner
130 |         login_response = requests.post(f"{BASE_URL}/api/auth/login", json={
131 |             "email": OWNER_EMAIL,
132 |             "password": OWNER_PASSWORD
133 |         })
134 |         assert login_response.status_code == 200, f"Owner login failed: {login_response.text}"
135 |         token = login_response.json().get("access_token")
136 |
137 |         # Get referral info
138 |         headers = {"Authorization": f"Bearer {token}"}
139 |         info_response = requests.get(f"{BASE_URL}/api/referral/info", headers=headers)
140 |         assert info_response.status_code == 200
141 |         info = info_response.json()
142 |
143 |         completed = info.get("completed_referrals", 0)
144 |         print(f"? Owner has {completed} completed referrals")
145 |
146 |         # If owner has completed referrals, they should be on leaderboard
147 |         if completed > 0:
148 |             leaderboard_response = requests.get(f"{BASE_URL}/api/referral/leaderboard")
149 |             leaderboard = leaderboard_response.json()["leaderboard"]
150 |
151 |             # Find entry with matching referral count
152 |             matching_entries = [e for e in leaderboard if e["referrals"] == completed]
153 |             assert len(matching_entries) > 0, \
154 |                 f"Owner with {completed} referrals should appear on leaderboard"
155 |             print(f"? Owner appears on leaderboard with {completed} referrals")
156 |
157 |
158 | class TestLeaderboardDataIntegrity:
159 |     """Tests for data integrity between referral info and leaderboard"""
160 |
161 |     def test_leaderboard_only_shows_completed_referrals(self):
162 |         """Leaderboard should only count completed referrals, not pending"""
163 |         response = requests.get(f"{BASE_URL}/api/referral/leaderboard")
164 |         assert response.status_code == 200
165 |         data = response.json()
166 |
167 |         # All entries should have referrals > 0
168 |         for entry in data["leaderboard"]:
169 |             assert entry["referrals"] > 0, \
170 |                 f"Entry with 0 referrals should not be on leaderboard: {entry}"
171 |         print(f"? All leaderboard entries have completed referrals > 0")
172 |
173 |
174 | if __name__ == "__main__":
175 |     pytest.main([__file__, "-v", "--tb=short"])
176 |

```

FILE: backend/tests/test_macro_data.py

213 lines | 12100 characters

```

1 | """
2 | Test suite for RISEDUALAI Macro Data Endpoints
3 | Tests: /api/gov-filings, /api/world-events, /api/foreign-markets, /api/market/prediction
4 | """
5 | import pytest
6 | import requests
7 | import os
8 | import time
9 |
10 | BASE_URL = os.environ.get('REACT_APP_BACKEND_URL', '').rstrip('/')
11 |
12 | class TestGovFilings:
13 |     """Test /api/gov-filings endpoint - Congressional trades, Fed announcements, SEC filings"""

```

```
14 |
15 |     def test_gov_filings_returns_200(self):
16 |         """Test that gov-filings endpoint returns 200"""
17 |         response = requests.get(f"{BASE_URL}/api/gov-filings", timeout=60)
18 |         assert response.status_code == 200, f"Expected 200, got {response.status_code}: {response.text}"
19 |         print(f"? GET /api/gov-filings returns 200")
20 |
21 |     def test_gov_filings_has_congressional_trades(self):
22 |         """Test that gov-filings returns congressional_trades array"""
23 |         response = requests.get(f"{BASE_URL}/api/gov-filings", timeout=60)
24 |         data = response.json()
25 |         assert 'congressional_trades' in data, "Missing congressional_trades field"
26 |         assert isinstance(data['congressional_trades'], list), "congressional_trades should be a list"
27 |         assert 'congressional_count' in data, "Missing congressional_count field"
28 |         print(f"? Congressional trades: {data['congressional_count']} items")
29 |
30 |     def test_gov_filings_has_fed_announcements(self):
31 |         """Test that gov-filings returns fed_announcements array"""
32 |         response = requests.get(f"{BASE_URL}/api/gov-filings", timeout=60)
33 |         data = response.json()
34 |         assert 'fed_announcements' in data, "Missing fed_announcements field"
35 |         assert isinstance(data['fed_announcements'], list), "fed_announcements should be a list"
36 |         assert 'fed_count' in data, "Missing fed_count field"
37 |         print(f"? Fed announcements: {data['fed_count']} items")
38 |
39 |     def test_gov_filings_has_insider_trades(self):
40 |         """Test that gov-filings returns insider_trades array"""
41 |         response = requests.get(f"{BASE_URL}/api/gov-filings", timeout=60)
42 |         data = response.json()
43 |         assert 'insider_trades' in data, "Missing insider_trades field"
44 |         assert isinstance(data['insider_trades'], list), "insider_trades should be a list"
45 |         assert 'insider_count' in data, "Missing insider_count field"
46 |         print(f"? Insider trades: {data['insider_count']} items")
47 |
48 |     def test_gov_filings_data_counts(self):
49 |         """Test that gov-filings returns data (>0 for congressional and fed)"""
50 |         response = requests.get(f"{BASE_URL}/api/gov-filings", timeout=60)
51 |         data = response.json()
52 |         # Congressional trades should have data (Capitol Trades scraping)
53 |         assert data['congressional_count'] >= 0, "Congressional count should be >= 0"
54 |         # Fed announcements should have data (Fed RSS feed)
55 |         assert data['fed_count'] >= 0, "Fed count should be >= 0"
56 |         # Insider trades may be 0 if SEC API is rate limited
57 |         assert data['insider_count'] >= 0, "Insider count should be >= 0"
58 |         print(f"? Data counts - Congressional: {data['congressional_count']}, Fed: {data['fed_count']}, Insider: {data['insider_co ...")
59 |
60 |
61 | class TestWorldEvents:
62 |     """Test /api/world-events endpoint - Geopolitical events and sector impact"""
63 |
64 |     def test_world_events_returns_200(self):
65 |         """Test that world-events endpoint returns 200"""
66 |         response = requests.get(f"{BASE_URL}/api/world-events", timeout=60)
67 |         assert response.status_code == 200, f"Expected 200, got {response.status_code}: {response.text}"
68 |         print(f"? GET /api/world-events returns 200")
69 |
70 |     def test_world_events_has_total_events(self):
71 |         """Test that world-events returns total_events count"""
72 |         response = requests.get(f"{BASE_URL}/api/world-events", timeout=60)
73 |         data = response.json()
74 |         assert 'total_events' in data, "Missing total_events field"
75 |         assert isinstance(data['total_events'], int), "total_events should be an integer"
76 |         print(f"? Total events: {data['total_events']}")
77 |
78 |     def test_world_events_has_high_impact_events(self):
79 |         """Test that world-events returns high_impact_events array"""
80 |         response = requests.get(f"{BASE_URL}/api/world-events", timeout=60)
```

```
81 |         data = response.json()
82 |         assert 'high_impact_events' in data, "Missing high_impact_events field"
83 |         assert isinstance(data['high_impact_events'], list), "high_impact_events should be a list"
84 |         print(f"? High impact events: {len(data['high_impact_events'])} items")
85 |
86 |     def test_world_events_has_affected_sectors(self):
87 |         """Test that world-events returns affected_sectors array"""
88 |         response = requests.get(f"{BASE_URL}/api/world-events", timeout=60)
89 |         data = response.json()
90 |         assert 'affected_sectors' in data, "Missing affected_sectors field"
91 |         assert isinstance(data['affected_sectors'], list), "affected_sectors should be a list"
92 |         print(f"? Affected sectors: {len(data['affected_sectors'])} sectors")
93 |
94 |     def test_world_events_data_present(self):
95 |         """Test that world-events returns actual data (total_events > 0)"""
96 |         response = requests.get(f"{BASE_URL}/api/world-events", timeout=60)
97 |         data = response.json()
98 |         # Should have some events from news scraping
99 |         assert data['total_events'] >= 0, "total_events should be >= 0"
100 |         print(f"? World events data present: {data['total_events']} events")
101 |
102 |
103 | class TestForeignMarkets:
104 |     """Test /api/foreign-markets endpoint - International indices, commodities, currencies"""
105 |
106 |     def test_foreign_markets_returns_200(self):
107 |         """Test that foreign-markets endpoint returns 200"""
108 |         response = requests.get(f"{BASE_URL}/api/foreign-markets", timeout=60)
109 |         assert response.status_code == 200, f"Expected 200, got {response.status_code}: {response.text}"
110 |         print(f"? GET /api/foreign-markets returns 200")
111 |
112 |     def test_foreign_markets_has_asia(self):
113 |         """Test that foreign-markets returns asia array"""
114 |         response = requests.get(f"{BASE_URL}/api/foreign-markets", timeout=60)
115 |         data = response.json()
116 |         assert 'asia' in data, "Missing asia field"
117 |         assert isinstance(data['asia'], list), "asia should be a list"
118 |         print(f"? Asia markets: {len(data['asia'])} indices")
119 |
120 |     def test_foreign_markets_has_europe(self):
121 |         """Test that foreign-markets returns europe array"""
122 |         response = requests.get(f"{BASE_URL}/api/foreign-markets", timeout=60)
123 |         data = response.json()
124 |         assert 'europe' in data, "Missing europe field"
125 |         assert isinstance(data['europe'], list), "europe should be a list"
126 |         print(f"? Europe markets: {len(data['europe'])} indices")
127 |
128 |     def test_foreign_markets_has_americas(self):
129 |         """Test that foreign-markets returns americas array"""
130 |         response = requests.get(f"{BASE_URL}/api/foreign-markets", timeout=60)
131 |         data = response.json()
132 |         assert 'americas' in data, "Missing americas field"
133 |         assert isinstance(data['americas'], list), "americas should be a list"
134 |         print(f"? Americas markets: {len(data['americas'])} indices")
135 |
136 |     def test_foreign_markets_has_commodities(self):
137 |         """Test that foreign-markets returns commodities array"""
138 |         response = requests.get(f"{BASE_URL}/api/foreign-markets", timeout=60)
139 |         data = response.json()
140 |         assert 'commodities' in data, "Missing commodities field"
141 |         assert isinstance(data['commodities'], list), "commodities should be a list"
142 |         print(f"? Commodities: {len(data['commodities'])} items")
143 |
144 |     def test_foreign_markets_has_currencies(self):
145 |         """Test that foreign-markets returns currencies array"""
146 |         response = requests.get(f"{BASE_URL}/api/foreign-markets", timeout=60)
147 |         data = response.json()
```

```

148 |         assert 'currencies' in data, "Missing currencies field"
149 |         assert isinstance(data['currencies'], list), "currencies should be a list"
150 |         print(f"? Currencies: {len(data['currencies'])} pairs")
151 |
152 |
153 | class TestMarketPrediction:
154 |     """Test /api/market/prediction endpoint - AI-powered prediction with macro data"""
155 |
156 |     def test_market_prediction_returns_200(self):
157 |         """Test that market/prediction endpoint returns 200 (long timeout due to scraping + AI)"""
158 |         response = requests.get(f"{BASE_URL}/api/market/prediction", timeout=180)
159 |         assert response.status_code == 200, f"Expected 200, got {response.status_code}: {response.text}"
160 |         print(f"? GET /api/market/prediction returns 200")
161 |
162 |     def test_market_prediction_has_direction(self):
163 |         """Test that prediction returns overall_direction"""
164 |         response = requests.get(f"{BASE_URL}/api/market/prediction", timeout=180)
165 |         data = response.json()
166 |         assert 'overall_direction' in data, "Missing overall_direction field"
167 |         assert data['overall_direction'] in ['BULLISH', 'BEARISH', 'NEUTRAL'], f"Invalid direction: {data['overall_direction']}"
168 |         print(f"? Overall direction: {data['overall_direction']}")
169 |
170 |     def test_market_prediction_has_confidence(self):
171 |         """Test that prediction returns confidence_score"""
172 |         response = requests.get(f"{BASE_URL}/api/market/prediction", timeout=180)
173 |         data = response.json()
174 |         assert 'confidence_score' in data, "Missing confidence_score field"
175 |         assert isinstance(data['confidence_score'], (int, float)), "confidence_score should be numeric"
176 |         assert 0 <= data['confidence_score'] <= 100, f"confidence_score should be 0-100, got {data['confidence_score']}"
177 |         print(f"? Confidence score: {data['confidence_score']}%")
178 |
179 |     def test_market_prediction_has_data_sources(self):
180 |         """Test that prediction returns data_sources with macro counts"""
181 |         response = requests.get(f"{BASE_URL}/api/market/prediction", timeout=180)
182 |         data = response.json()
183 |         assert 'data_sources' in data, "Missing data_sources field"
184 |         ds = data['data_sources']
185 |         # Check for new macro data source counts
186 |         assert 'world_events' in ds, "Missing world_events in data_sources"
187 |         assert 'foreign_markets' in ds, "Missing foreign_markets in data_sources"
188 |         assert 'congressional_trades' in ds, "Missing congressional_trades in data_sources"
189 |         assert 'fed_announcements' in ds, "Missing fed_announcements in data_sources"
190 |         print(f"? Data sources - World Events: {ds['world_events']}, Foreign Markets: {ds['foreign_markets']}, Congressional: {ds[ ...
191 |
192 |     def test_market_prediction_has_macro_data(self):
193 |         """Test that prediction returns macro_data object with sub-objects"""
194 |         response = requests.get(f"{BASE_URL}/api/market/prediction", timeout=180)
195 |         data = response.json()
196 |         assert 'macro_data' in data, "Missing macro_data field"
197 |         md = data['macro_data']
198 |         # Check for world_events sub-object
199 |         assert 'world_events' in md, "Missing world_events in macro_data"
200 |         assert 'total' in md['world_events'], "Missing total in macro_data.world_events"
201 |         # Check for foreign_markets sub-object
202 |         assert 'foreign_markets' in md, "Missing foreign_markets in macro_data"
203 |         assert 'total_indices' in md['foreign_markets'], "Missing total_indices in macro_data.foreign_markets"
204 |         # Check for gov_filings sub-object
205 |         assert 'gov_filings' in md, "Missing gov_filings in macro_data"
206 |         assert 'congressional_trades' in md['gov_filings'], "Missing congressional_trades in macro_data.gov_filings"
207 |         assert 'fed_announcements' in md['gov_filings'], "Missing fed_announcements in macro_data.gov_filings"
208 |         print(f"? Macro data present - World Events: {md['world_events']['total']}, Foreign Indices: {md['foreign_markets']['total' ...
209 |
210 |
211 | if __name__ == "__main__":
212 |     pytest.main([__file__, "-v", "--tb=short"])
213 |

```

FILE: backend/tests/test_paywall_features.py

330 lines | 13131 characters

```
1 | """
2 | Test paywall features for RISEDUAL AI trading platform.
3 | Tests: Chat rate limits, Watchlist caps, Portfolio Analyzer, Market Signals, Export
4 | """
5 | import pytest
6 | import requests
7 | import os
8 |
9 | BASE_URL = os.environ.get('REACT_APP_BACKEND_URL', '').rstrip('/')
10 |
11 | # Test credentials
12 | OWNER_EMAIL = "managingdirector@redslateholdings.com"
13 | OWNER_PASSWORD = "RedSlate2026!"
14 | ADMIN_EMAIL = "admin@risedual.ai"
15 | ADMIN_PASSWORD = "RiseDual2026!"
16 |
17 |
18 | class TestAuthLogin:
19 |     """Test authentication endpoints"""
20 |
21 |     def test_owner_login_returns_pro_status(self):
22 |         """Owner account should have pro subscription"""
23 |         response = requests.post(f"{BASE_URL}/api/auth/login", json={
24 |             "email": OWNER_EMAIL,
25 |             "password": OWNER_PASSWORD
26 |         })
27 |         assert response.status_code == 200, f"Login failed: {response.text}"
28 |         data = response.json()
29 |         assert "access_token" in data, "No access_token in response"
30 |         assert data.get("subscription_status") == "pro", f"Expected pro, got {data.get('subscription_status')}"
31 |         assert data.get("role") == "owner", f"Expected owner role, got {data.get('role')}"
32 |
33 |     def test_admin_login_returns_pro_status(self):
34 |         """Admin account should have pro subscription"""
35 |         response = requests.post(f"{BASE_URL}/api/auth/login", json={
36 |             "email": ADMIN_EMAIL,
37 |             "password": ADMIN_PASSWORD
38 |         })
39 |         assert response.status_code == 200, f"Login failed: {response.text}"
40 |         data = response.json()
41 |         assert "access_token" in data
42 |         assert data.get("subscription_status") == "pro"
43 |
44 |
45 | class TestChatRateLimit:
46 |     """Test AI Chat rate limit endpoint"""
47 |
48 |     def test_chat_limit_unauthenticated(self):
49 |         """Unauthenticated users get default free limit"""
50 |         response = requests.get(f"{BASE_URL}/api/chat/limit")
51 |         assert response.status_code == 200
52 |         data = response.json()
53 |         assert data.get("limit") == 5, f"Expected limit 5, got {data.get('limit')}"
54 |         assert data.get("remaining") == 5
55 |         assert data.get("is_pro") == False
56 |
57 |     def test_chat_limit_pro_user(self):
58 |         """Pro users get unlimited (-1)"""
59 |         # Login as owner (pro)
60 |         login_res = requests.post(f"{BASE_URL}/api/auth/login", json={
61 |             "email": OWNER_EMAIL,
62 |             "password": OWNER_PASSWORD
63 |         })
```



```

64 |         token = login_res.json().get("access_token")
65 |
66 |         response = requests.get(
67 |             f"{BASE_URL}/api/chat/limit",
68 |             headers={"Authorization": f"Bearer {token}"})
69 |     )
70 |     assert response.status_code == 200
71 |     data = response.json()
72 |     assert data.get("limit") == -1, f"Pro user should have unlimited (-1), got {data.get('limit')}"
73 |     assert data.get("is_pro") == True
74 |
75 |
76 | class TestWatchlistCap:
77 |     """Test watchlist cap for free users"""
78 |
79 |     @pytest.fixture
80 |     def free_user_token(self):
81 |         """Create/login a free test user"""
82 |         test_email = f"TEST_free_user_{os.urandom(4).hex()}@test.com"
83 |         # Register new user (will be free by default)
84 |         reg_res = requests.post(f"{BASE_URL}/api/auth/register", json={
85 |             "email": test_email,
86 |             "password": "TestPass123!",
87 |             "name": "Test Free User"
88 |         })
89 |         if reg_res.status_code == 200:
90 |             return reg_res.json().get("access_token")
91 |         # If registration fails (user exists), try login
92 |         login_res = requests.post(f"{BASE_URL}/api/auth/login", json={
93 |             "email": test_email,
94 |             "password": "TestPass123!"
95 |         })
96 |         return login_res.json().get("access_token")
97 |
98 |     def test_watchlist_add_exceeds_cap_free_user(self, free_user_token):
99 |         """Free user should get 403 after adding 3 tickers"""
100 |         if not free_user_token:
101 |             pytest.skip("Could not create free user")
102 |
103 |         headers = {"Authorization": f"Bearer {free_user_token}"}
104 |
105 |         # Add 3 tickers (should succeed)
106 |         for i, ticker in enumerate(["AAPL", "MSFT", "GOOGL"]):
107 |             res = requests.post(
108 |                 f"{BASE_URL}/api/workspace/watchlist/add",
109 |                 json={"ticker": ticker},
110 |                 headers=headers
111 |             )
112 |             # First 3 should succeed (200) or already exist
113 |             assert res.status_code in [200, 400], f"Ticker {i+1} failed: {res.text}"
114 |
115 |             # 4th ticker should fail with 403
116 |             res = requests.post(
117 |                 f"{BASE_URL}/api/workspace/watchlist/add",
118 |                 json={"ticker": "NVDA"},
119 |                 headers=headers
120 |             )
121 |             assert res.status_code == 403, f"Expected 403 for 4th ticker, got {res.status_code}: {res.text}"
122 |             assert "limited to 3" in res.json().get("detail", "").lower() or "free" in res.json().get("detail", "").lower()
123 |
124 |     def test_watchlist_add_pro_user_no_cap(self):
125 |         """Pro user should have no watchlist cap"""
126 |         login_res = requests.post(f"{BASE_URL}/api/auth/login", json={
127 |             "email": OWNER_EMAIL,
128 |             "password": OWNER_PASSWORD
129 |         })
130 |         token = login_res.json().get("access_token")

```

```

131 |         headers = {"Authorization": f"Bearer {token}"}
132 |
133 |         # Pro user can add many tickers
134 |         for ticker in ["TEST1", "TEST2", "TEST3", "TEST4", "TEST5"]:
135 |             res = requests.post(
136 |                 f"{BASE_URL}/api/workspace/watchlist/add",
137 |                 json={"ticker": ticker},
138 |                 headers=headers
139 |             )
140 |             assert res.status_code == 200, f"Pro user failed to add {ticker}: {res.text}"
141 |
142 |
143 | class TestMarketSignals:
144 |     """Test Market Signals Pro-only endpoint"""
145 |
146 |     def test_signals_unauthenticated(self):
147 |         """Unauthenticated request should return 401"""
148 |         response = requests.get(f"{BASE_URL}/api/signals")
149 |         assert response.status_code == 401
150 |
151 |     def test_signals_free_user_empty(self):
152 |         """Free user should get empty signals with is_pro=False"""
153 |         # Register a new free user
154 |         test_email = f"TEST_signals_free_{os.urandom(4).hex()}@test.com"
155 |         reg_res = requests.post(f"{BASE_URL}/api/auth/register", json={
156 |             "email": test_email,
157 |             "password": "TestPass123!",
158 |             "name": "Test Signals Free"
159 |         })
160 |         if reg_res.status_code != 200:
161 |             pytest.skip("Could not create free user")
162 |
163 |         token = reg_res.json().get("access_token")
164 |         response = requests.get(
165 |             f"{BASE_URL}/api/signals",
166 |             headers={"Authorization": f"Bearer {token}"}
167 |         )
168 |         assert response.status_code == 200
169 |         data = response.json()
170 |         assert data.get("signals") == [], f"Free user should get empty signals"
171 |         assert data.get("is_pro") == False
172 |
173 |     def test_signals_pro_user(self):
174 |         """Pro user should get signals with is_pro=True"""
175 |         login_res = requests.post(f"{BASE_URL}/api/auth/login", json={
176 |             "email": OWNER_EMAIL,
177 |             "password": OWNER_PASSWORD
178 |         })
179 |         token = login_res.json().get("access_token")
180 |
181 |         response = requests.get(
182 |             f"{BASE_URL}/api/signals",
183 |             headers={"Authorization": f"Bearer {token}"}
184 |         )
185 |         assert response.status_code == 200
186 |         data = response.json()
187 |         assert data.get("is_pro") == True
188 |
189 |
190 | class TestPortfolioAnalyzer:
191 |     """Test Portfolio Analyzer Pro-only endpoint"""
192 |
193 |     def test_portfolio_analyze_unauthenticated(self):
194 |         """Unauthenticated request should return 401"""
195 |         response = requests.post(
196 |             f"{BASE_URL}/api/portfolio/analyze",
197 |             json={"holdings": [{"ticker": "AAPL", "shares": 10, "avg_price": 150}]}

```

```

198 |     )
199 |     assert response.status_code == 401
200 |
201 |     def test_portfolio_analyze_free_user_403(self):
202 |         """Free user should get 403"""
203 |         test_email = f"TEST_portfolio_free_{os.urandom(4).hex()}@test.com"
204 |         reg_res = requests.post(f"{BASE_URL}/api/auth/register", json={
205 |             "email": test_email,
206 |             "password": "TestPass123!",
207 |             "name": "Test Portfolio Free"
208 |         })
209 |         if reg_res.status_code != 200:
210 |             pytest.skip("Could not create free user")
211 |
212 |         token = reg_res.json().get("access_token")
213 |         response = requests.post(
214 |             f"{BASE_URL}/api/portfolio/analyze",
215 |             json={"holdings": [{"ticker": "AAPL", "shares": 10, "avg_price": 150}]},
216 |             headers={"Authorization": f"Bearer {token}"})
217 |
218 |         assert response.status_code == 403, f"Expected 403, got {response.status_code}: {response.text}"
219 |         assert "pro feature" in response.json().get("detail", "").lower()
220 |
221 |     def test_portfolio_analyze_pro_user_success(self):
222 |         """Pro user should get analysis result"""
223 |         login_res = requests.post(f"{BASE_URL}/api/auth/login", json={
224 |             "email": OWNER_EMAIL,
225 |             "password": OWNER_PASSWORD
226 |         })
227 |         token = login_res.json().get("access_token")
228 |
229 |         response = requests.post(
230 |             f"{BASE_URL}/api/portfolio/analyze",
231 |             json={"holdings": [
232 |                 {"ticker": "AAPL", "shares": 10, "avg_price": 150},
233 |                 {"ticker": "MSFT", "shares": 5, "avg_price": 300}
234 |             ]},
235 |             headers={"Authorization": f"Bearer {token}"},
236 |             timeout=60 # AI analysis may take time
237 |         )
238 |         assert response.status_code == 200, f"Pro user analysis failed: {response.text}"
239 |         data = response.json()
240 |         assert "health_score" in data or "summary" in data, f"Missing analysis fields: {data.keys()}"
241 |
242 |
243 | class TestHypothesisPaywall:
244 |     """Test AI Hypothesis paywall"""
245 |
246 |     def test_hypothesis_free_user_locked(self):
247 |         """Free user should get locked/teaser response"""
248 |         test_email = f"TEST_hypothesis_free_{os.urandom(4).hex()}@test.com"
249 |         reg_res = requests.post(f"{BASE_URL}/api/auth/register", json={
250 |             "email": test_email,
251 |             "password": "TestPass123!",
252 |             "name": "Test Hypothesis Free"
253 |         })
254 |         if reg_res.status_code != 200:
255 |             pytest.skip("Could not create free user")
256 |
257 |         token = reg_res.json().get("access_token")
258 |         response = requests.get(
259 |             f"{BASE_URL}/api/hypothesis/AAPL",
260 |             headers={"Authorization": f"Bearer {token}"},
261 |             timeout=60
262 |         )
263 |         assert response.status_code == 200
264 |         data = response.json()

```

```
265 |         assert data.get("is_pro") == False, f"Free user should have is_pro=False"
266 |         assert "teaser" in data, f"Free user should get teaser: {data.keys()}"
267 |         assert data.get("teaser", {}).get("verdict") == "LOCKED"
268 |
269 |     def test_hypothesis_pro_user_full(self):
270 |         """Pro user should get full hypothesis"""
271 |         login_res = requests.post(f"{BASE_URL}/api/auth/login", json={
272 |             "email": OWNER_EMAIL,
273 |             "password": OWNER_PASSWORD
274 |         })
275 |         token = login_res.json().get("access_token")
276 |
277 |         response = requests.get(
278 |             f"{BASE_URL}/api/hypothesis/AAPL",
279 |             headers={"Authorization": f"Bearer {token}"},
280 |             timeout=90 # AI may take time
281 |         )
282 |         assert response.status_code == 200
283 |         data = response.json()
284 |         assert data.get("is_pro") == True, f"Pro user should have is_pro=True"
285 |         assert "verdict" in data, f"Pro user should get verdict: {data.keys()}"
286 |         assert data.get("verdict") in ["BUY", "SELL", "HOLD"], f"Invalid verdict: {data.get('verdict')}"
287 |
288 |
289 | class TestExportHypothesis:
290 |     """Test PDF/text export Pro-only endpoint"""
291 |
292 |     def test_export_unauthenticated(self):
293 |         """Unauthenticated request should return 401"""
294 |         response = requests.get(f"{BASE_URL}/api/export/hypothesis/AAPL")
295 |         assert response.status_code == 401
296 |
297 |     def test_export_free_user_403(self):
298 |         """Free user should get 403"""
299 |         test_email = f"TEST_export_free_{os.urandom(4).hex()}@test.com"
300 |         reg_res = requests.post(f"{BASE_URL}/api/auth/register", json={
301 |             "email": test_email,
302 |             "password": "TestPass123!",
303 |             "name": "Test Export Free"
304 |         })
305 |         if reg_res.status_code != 200:
306 |             pytest.skip("Could not create free user")
307 |
308 |         token = reg_res.json().get("access_token")
309 |         response = requests.get(
310 |             f"{BASE_URL}/api/export/hypothesis/AAPL",
311 |             headers={"Authorization": f"Bearer {token}"},
312 |         )
313 |         assert response.status_code == 403
314 |         assert "pro feature" in response.json().get("detail", "").lower()
315 |
316 |
317 | class TestDarkPoolEndpoint:
318 |     """Test Dark Pool data endpoint (public but blur wall on frontend)"""
319 |
320 |     def test_dark_pool_returns_data(self):
321 |         """Dark pool endpoint should return data (blur is frontend-only)"""
322 |         response = requests.get(f"{BASE_URL}/api/dark-pool")
323 |         assert response.status_code == 200
324 |         data = response.json()
325 |         assert isinstance(data, list), f"Expected list, got {type(data)}"
326 |
327 |
328 | if __name__ == "__main__":
329 |     pytest.main([__file__, "-v", "--tb=short"])
330 |
```

FILE: backend/tests/test_promo_campaign.py

446 lines | 20028 characters

```
1 | """
2 | Test suite for Promo Campaign feature.
3 | Tests: POST /api/promo/create, GET /api/promo/active, GET /api/promo/progress,
4 |       GET /api/promo/all, PUT /api/promo/{id}/toggle, DELETE /api/promo/{id}
5 | """
6 | import pytest
7 | import requests
8 | import os
9 | from datetime import datetime, timedelta
10 |
11 | BASE_URL = os.environ.get('REACT_APP_BACKEND_URL', '').rstrip('/')
12 |
13 | # Test credentials
14 | OWNER_EMAIL = "managingdirector@redslateholdings.com"
15 | OWNER_PASSWORD = "RedSlate2026!"
16 | FREE_USER_EMAIL = "freeuser_test@test.com"
17 | FREE_USER_PASSWORD = "Test1234!"
18 |
19 | # Known active promo ID from context
20 | ACTIVE_PROMO_ID = "69d0d11e8617ff93a0953ad9"
21 |
22 |
23 | class TestPromoActiveEndpoint:
24 |     """Test GET /api/promo/active - public endpoint"""
25 |
26 |     def test_active_promo_is_public(self):
27 |         """Active promo endpoint should not require authentication"""
28 |         response = requests.get(f"{BASE_URL}/api/promo/active")
29 |         assert response.status_code == 200, f"Expected 200, got {response.status_code}: {response.text}"
30 |         print("PASSED: /api/promo/active is public (no auth required)")
31 |
32 |     def test_active_promo_response_structure(self):
33 |         """Active promo should return promo object or null"""
34 |         response = requests.get(f"{BASE_URL}/api/promo/active")
35 |         assert response.status_code == 200
36 |         data = response.json()
37 |         assert "promo" in data, "Response should have 'promo' key"
38 |         print(f"PASSED: Response has 'promo' key, value: {data['promo']} is not None")
39 |
40 |     def test_active_promo_fields(self):
41 |         """If active promo exists, verify all required fields"""
42 |         response = requests.get(f"{BASE_URL}/api/promo/active")
43 |         assert response.status_code == 200
44 |         data = response.json()
45 |
46 |         if data["promo"]:
47 |             promo = data["promo"]
48 |             required_fields = ["id", "title", "message", "referral_target", "reward_months", "start_date", "end_date", "is_active" ...
49 |             for field in required_fields:
50 |                 assert field in promo, f"Missing field: {field}"
51 |                 assert promo["is_active"] == True, "Active promo should have is_active=True"
52 |                 print(f"PASSED: Active promo has all required fields. Title: {promo['title']}")
53 |             else:
54 |                 print("INFO: No active promo currently exists")
55 |
56 |
57 | class TestPromoProgressEndpoint:
58 |     """Test GET /api/promo/progress - requires authentication"""
59 |
60 |     @pytest.fixture
61 |     def owner_token(self):
62 |         """Get owner auth token"""
63 |         response = requests.post(f"{BASE_URL}/api/auth/login", json={
```

```

64 |         "email": OWNER_EMAIL,
65 |         "password": OWNER_PASSWORD
66 |     })
67 |     if response.status_code == 200:
68 |         return response.json().get("access_token")
69 |     pytest.skip(f"Owner login failed: {response.status_code}")
70 |
71 |     def test_progress_requires_auth(self):
72 |         """Progress endpoint should require authentication"""
73 |         response = requests.get(f"{BASE_URL}/api/promo/progress")
74 |         assert response.status_code == 401, f"Expected 401 without auth, got {response.status_code}"
75 |         print("PASSED: /api/promo/progress requires authentication (401 without token)")
76 |
77 |     def test_progress_with_auth(self, owner_token):
78 |         """Progress endpoint should return user's referral progress"""
79 |         headers = {"Authorization": f"Bearer {owner_token}"}
80 |         response = requests.get(f"{BASE_URL}/api/promo/progress", headers=headers)
81 |         assert response.status_code == 200, f"Expected 200, got {response.status_code}: {response.text}"
82 |
83 |         data = response.json()
84 |         assert "progress" in data, "Response should have 'progress' key"
85 |         assert "target" in data, "Response should have 'target' key"
86 |         assert "completed" in data, "Response should have 'completed' key"
87 |         print(f"PASSED: Progress endpoint returns data. Progress: {data['progress']}/{data['target']}, Completed: {data['completed']}")
88 |
89 |     def test_progress_response_structure(self, owner_token):
90 |         """Verify progress response has all required fields"""
91 |         headers = {"Authorization": f"Bearer {owner_token}"}
92 |         response = requests.get(f"{BASE_URL}/api/promo/progress", headers=headers)
93 |         assert response.status_code == 200
94 |
95 |         data = response.json()
96 |         # If there's an active promo, verify promo object
97 |         if data.get("promo"):
98 |             promo = data["promo"]
99 |             assert "id" in promo
100 |             assert "title" in promo
101 |             assert "referral_target" in promo
102 |             print(f"PASSED: Progress includes promo details. Promo: {promo['title']}")
103 |         else:
104 |             print("INFO: No active promo, progress returns null promo")
105 |
106 |
107 | class TestPromoAllEndpoint:
108 |     """Test GET /api/promo/all - admin only"""
109 |
110 |     @pytest.fixture
111 |     def owner_token(self):
112 |         """Get owner auth token"""
113 |         response = requests.post(f"{BASE_URL}/api/auth/login", json={
114 |             "email": OWNER_EMAIL,
115 |             "password": OWNER_PASSWORD
116 |         })
117 |         if response.status_code == 200:
118 |             return response.json().get("access_token")
119 |         pytest.skip(f"Owner login failed: {response.status_code}")
120 |
121 |     @pytest.fixture
122 |     def free_user_token(self):
123 |         """Get free user auth token (non-admin)"""
124 |         # First try to login
125 |         response = requests.post(f"{BASE_URL}/api/auth/login", json={
126 |             "email": FREE_USER_EMAIL,
127 |             "password": FREE_USER_PASSWORD
128 |         })
129 |         if response.status_code == 200:
130 |             return response.json().get("access_token")

```

```
131 |
132 |     # If login fails, try to register
133 |     response = requests.post(f"{BASE_URL}/api/auth/register", json={
134 |         "email": FREE_USER_EMAIL,
135 |         "password": FREE_USER_PASSWORD,
136 |         "name": "Free Test User"
137 |     })
138 |     if response.status_code in [200, 201]:
139 |         return response.json().get("access_token")
140 |
141 |     pytest.skip(f"Free user login/register failed: {response.status_code}")
142 |
143 |     def test_all_promos_requires_auth(self):
144 |         """All promos endpoint should require authentication"""
145 |         response = requests.get(f"{BASE_URL}/api/promo/all")
146 |         assert response.status_code == 401, f"Expected 401 without auth, got {response.status_code}"
147 |         print("PASSED: /api/promo/all requires authentication (401 without token)")
148 |
149 |     def test_all_promos_admin_only(self, free_user_token):
150 |         """Non-admin users should get 403"""
151 |         headers = {"Authorization": f"Bearer {free_user_token}"}
152 |         response = requests.get(f"{BASE_URL}/api/promo/all", headers=headers)
153 |         assert response.status_code == 403, f"Expected 403 for non-admin, got {response.status_code}: {response.text}"
154 |         print("PASSED: /api/promo/all returns 403 for non-admin users")
155 |
156 |     def test_all_promos_owner_access(self, owner_token):
157 |         """Owner should be able to access all promos"""
158 |         headers = {"Authorization": f"Bearer {owner_token}"}
159 |         response = requests.get(f"{BASE_URL}/api/promo/all", headers=headers)
160 |         assert response.status_code == 200, f"Expected 200 for owner, got {response.status_code}: {response.text}"
161 |
162 |         data = response.json()
163 |         assert "promos" in data, "Response should have 'promos' key"
164 |         assert isinstance(data["promos"], list), "Promos should be a list"
165 |         print(f"PASSED: Owner can access all promos. Count: {len(data['promos'])}")
166 |
167 |
168 | class TestPromoCreateEndpoint:
169 |     """Test POST /api/promo/create - admin only"""
170 |
171 |     @pytest.fixture
172 |     def owner_token(self):
173 |         """Get owner auth token"""
174 |         response = requests.post(f"{BASE_URL}/api/auth/login", json={
175 |             "email": OWNER_EMAIL,
176 |             "password": OWNER_PASSWORD
177 |         })
178 |         if response.status_code == 200:
179 |             return response.json().get("access_token")
180 |         pytest.skip(f"Owner login failed: {response.status_code}")
181 |
182 |     @pytest.fixture
183 |     def free_user_token(self):
184 |         """Get free user auth token (non-admin)"""
185 |         response = requests.post(f"{BASE_URL}/api/auth/login", json={
186 |             "email": FREE_USER_EMAIL,
187 |             "password": FREE_USER_PASSWORD
188 |         })
189 |         if response.status_code == 200:
190 |             return response.json().get("access_token")
191 |         pytest.skip(f"Free user login failed: {response.status_code}")
192 |
193 |     def test_create_promo_requires_auth(self):
194 |         """Create promo should require authentication"""
195 |         promo_data = {
196 |             "title": "Test Promo",
197 |             "message": "Test message",
```

```

198 |         "referral_target": 3,
199 |         "reward_months": 2,
200 |         "start_date": datetime.now().isoformat(),
201 |         "end_date": (datetime.now() + timedelta(days=30)).isoformat()
202 |     }
203 |     response = requests.post(f"{BASE_URL}/api/promo/create", json=promo_data)
204 |     assert response.status_code == 401, f"Expected 401 without auth, got {response.status_code}"
205 |     print("PASSED: /api/promo/create requires authentication (401 without token)")
206 |
207 |     def test_create_promo_admin_only(self, free_user_token):
208 |         """Non-admin users should get 403"""
209 |         headers = {"Authorization": f"Bearer {free_user_token}"}
210 |         promo_data = {
211 |             "title": "Test Promo",
212 |             "message": "Test message",
213 |             "referral_target": 3,
214 |             "reward_months": 2,
215 |             "start_date": datetime.now().isoformat(),
216 |             "end_date": (datetime.now() + timedelta(days=30)).isoformat()
217 |         }
218 |         response = requests.post(f"{BASE_URL}/api/promo/create", json=promo_data, headers=headers)
219 |         assert response.status_code == 403, f"Expected 403 for non-admin, got {response.status_code}: {response.text}"
220 |         print("PASSED: /api/promo/create returns 403 for non-admin users")
221 |
222 |     def test_create_promo_owner_success(self, owner_token):
223 |         """Owner should be able to create a promo"""
224 |         headers = {"Authorization": f"Bearer {owner_token}", "Content-Type": "application/json"}
225 |         promo_data = {
226 |             "title": "TEST_Promo_Campaign",
227 |             "message": "Test promo for automated testing",
228 |             "referral_target": 5,
229 |             "reward_months": 1,
230 |             "start_date": datetime.now().isoformat(),
231 |             "end_date": (datetime.now() + timedelta(days=7)).isoformat()
232 |         }
233 |         response = requests.post(f"{BASE_URL}/api/promo/create", json=promo_data, headers=headers)
234 |         assert response.status_code == 200, f"Expected 200, got {response.status_code}: {response.text}"
235 |
236 |         data = response.json()
237 |         assert "id" in data, "Response should have 'id'"
238 |         assert data["title"] == "TEST_Promo_Campaign"
239 |         assert data["referral_target"] == 5
240 |         assert data["reward_months"] == 1
241 |         print(f"PASSED: Owner created promo successfully. ID: {data['id']}")
242 |
243 |         # Store ID for cleanup
244 |         return data["id"]
245 |
246 |
247 | class TestPromoToggleEndpoint:
248 |     """Test PUT /api/promo/{id}/toggle - admin only"""
249 |
250 |     @pytest.fixture
251 |     def owner_token(self):
252 |         """Get owner auth token"""
253 |         response = requests.post(f"{BASE_URL}/api/auth/login", json={
254 |             "email": OWNER_EMAIL,
255 |             "password": OWNER_PASSWORD
256 |         })
257 |         if response.status_code == 200:
258 |             return response.json().get("access_token")
259 |         pytest.skip(f"Owner login failed: {response.status_code}")
260 |
261 |     @pytest.fixture
262 |     def free_user_token(self):
263 |         """Get free user auth token (non-admin)"""
264 |         response = requests.post(f"{BASE_URL}/api/auth/login", json={

```



```
265 |         "email": FREE_USER_EMAIL,
266 |         "password": FREE_USER_PASSWORD
267 |     })
268 |     if response.status_code == 200:
269 |         return response.json().get("access_token")
270 |     pytest.skip(f"Free user login failed: {response.status_code}")
271 |
272 |     def test_toggle_promo_requires_auth(self):
273 |         """Toggle promo should require authentication"""
274 |         response = requests.put(f"{BASE_URL}/api/promo/{ACTIVE_PROMO_ID}/toggle")
275 |         assert response.status_code == 401, f"Expected 401 without auth, got {response.status_code}"
276 |         print("PASSED: /api/promo/{id}/toggle requires authentication (401 without token)")
277 |
278 |     def test_toggle_promo_admin_only(self, free_user_token):
279 |         """Non-admin users should get 403"""
280 |         headers = {"Authorization": f"Bearer {free_user_token}"}
281 |         response = requests.put(f"{BASE_URL}/api/promo/{ACTIVE_PROMO_ID}/toggle", headers=headers)
282 |         assert response.status_code == 403, f"Expected 403 for non-admin, got {response.status_code}: {response.text}"
283 |         print("PASSED: /api/promo/{id}/toggle returns 403 for non-admin users")
284 |
285 |     def test_toggle_promo_owner_success(self, owner_token):
286 |         """Owner should be able to toggle promo status"""
287 |         headers = {"Authorization": f"Bearer {owner_token}"}
288 |
289 |         # Get current status first
290 |         response = requests.get(f"{BASE_URL}/api/promo/all", headers=headers)
291 |         if response.status_code == 200:
292 |             promos = response.json().get("promos", [])
293 |             if promos:
294 |                 promo_id = promos[0]["id"]
295 |                 original_status = promos[0]["is_active"]
296 |
297 |                 # Toggle
298 |                 response = requests.put(f"{BASE_URL}/api/promo/{promo_id}/toggle", headers=headers)
299 |                 assert response.status_code == 200, f"Expected 200, got {response.status_code}: {response.text}"
300 |
301 |                 data = response.json()
302 |                 assert data["is_active"] != original_status, "Status should be toggled"
303 |                 print(f"PASSED: Owner toggled promo. New status: {data['is_active']}")
304 |
305 |                 # Toggle back to restore original state
306 |                 requests.put(f"{BASE_URL}/api/promo/{promo_id}/toggle", headers=headers)
307 |                 print("INFO: Restored original promo status")
308 |             else:
309 |                 pytest.skip("No promos available to toggle")
310 |         else:
311 |             pytest.skip(f"Could not fetch promos: {response.status_code}")
312 |
313 |
314 | class TestPromoDeleteEndpoint:
315 |     """Test DELETE /api/promo/{id} - admin only"""
316 |
317 |     @pytest.fixture
318 |     def owner_token(self):
319 |         """Get owner auth token"""
320 |         response = requests.post(f"{BASE_URL}/api/auth/login", json={
321 |             "email": OWNER_EMAIL,
322 |             "password": OWNER_PASSWORD
323 |         })
324 |         if response.status_code == 200:
325 |             return response.json().get("access_token")
326 |         pytest.skip(f"Owner login failed: {response.status_code}")
327 |
328 |     @pytest.fixture
329 |     def free_user_token(self):
330 |         """Get free user auth token (non-admin)"""
331 |         response = requests.post(f"{BASE_URL}/api/auth/login", json={
```

```
332 |         "email": FREE_USER_EMAIL,
333 |         "password": FREE_USER_PASSWORD
334 |     })
335 |     if response.status_code == 200:
336 |         return response.json().get("access_token")
337 |     pytest.skip(f"Free user login failed: {response.status_code}")
338 |
339 |     def test_delete_promo_requires_auth(self):
340 |         """Delete promo should require authentication"""
341 |         response = requests.delete(f"{BASE_URL}/api/promo/fake_id_123")
342 |         assert response.status_code == 401, f"Expected 401 without auth, got {response.status_code}"
343 |         print("PASSED: /api/promo/{id} DELETE requires authentication (401 without token)")
344 |
345 |     def test_delete_promo_admin_only(self, free_user_token):
346 |         """Non-admin users should get 403"""
347 |         headers = {"Authorization": f"Bearer {free_user_token}"}
348 |         response = requests.delete(f"{BASE_URL}/api/promo/fake_id_123", headers=headers)
349 |         assert response.status_code == 403, f"Expected 403 for non-admin, got {response.status_code}: {response.text}"
350 |         print("PASSED: /api/promo/{id} DELETE returns 403 for non-admin users")
351 |
352 |     def test_delete_promo_not_found(self, owner_token):
353 |         """Delete non-existent promo should return 404"""
354 |         headers = {"Authorization": f"Bearer {owner_token}"}
355 |         response = requests.delete(f"{BASE_URL}/api/promo/00000000000000000000000000000000", headers=headers)
356 |         assert response.status_code == 404, f"Expected 404 for non-existent promo, got {response.status_code}"
357 |         print("PASSED: Delete non-existent promo returns 404")
358 |
359 |
360 | class TestPromoCreateAndDelete:
361 |     """Test full create-delete cycle"""
362 |
363 |     @pytest.fixture
364 |     def owner_token(self):
365 |         """Get owner auth token"""
366 |         response = requests.post(f"{BASE_URL}/api/auth/login", json={
367 |             "email": OWNER_EMAIL,
368 |             "password": OWNER_PASSWORD
369 |         })
370 |         if response.status_code == 200:
371 |             return response.json().get("access_token")
372 |         pytest.skip(f"Owner login failed: {response.status_code}")
373 |
374 |     def test_create_and_delete_promo(self, owner_token):
375 |         """Create a promo and then delete it"""
376 |         headers = {"Authorization": f"Bearer {owner_token}", "Content-Type": "application/json"}
377 |
378 |         # Create
379 |         promo_data = {
380 |             "title": "TEST_Delete_Me_Promo",
381 |             "message": "This promo will be deleted",
382 |             "referral_target": 10,
383 |             "reward_months": 3,
384 |             "start_date": datetime.now().isoformat(),
385 |             "end_date": (datetime.now() + timedelta(days=1)).isoformat()
386 |         }
387 |         create_response = requests.post(f"{BASE_URL}/api/promo/create", json=promo_data, headers=headers)
388 |         assert create_response.status_code == 200, f"Create failed: {create_response.status_code}"
389 |
390 |         promo_id = create_response.json()["id"]
391 |         print(f"Created test promo with ID: {promo_id}")
392 |
393 |         # Verify it exists
394 |         all_response = requests.get(f"{BASE_URL}/api/promo/all", headers=headers)
395 |         assert all_response.status_code == 200
396 |         promos = all_response.json()["promos"]
397 |         assert any(p["id"] == promo_id for p in promos), "Created promo should be in list"
398 |
```

```
399 |         # Delete
400 |         delete_response = requests.delete(f"{BASE_URL}/api/promo/{promo_id}", headers=headers)
401 |         assert delete_response.status_code == 200, f"Delete failed: {delete_response.status_code}"
402 |
403 |         # Verify it's gone
404 |         all_response = requests.get(f"{BASE_URL}/api/promo/all", headers=headers)
405 |         promos = all_response.json()["promos"]
406 |         assert not any(p["id"] == promo_id for p in promos), "Deleted promo should not be in list"
407 |
408 |         print("PASSED: Create and delete promo cycle completed successfully")
409 |
410 |
411 | class TestCleanupTestPromos:
412 |     """Cleanup any TEST_ prefixed promos"""
413 |
414 |     @pytest.fixture
415 |     def owner_token(self):
416 |         """Get owner auth token"""
417 |         response = requests.post(f"{BASE_URL}/api/auth/login", json={
418 |             "email": OWNER_EMAIL,
419 |             "password": OWNER_PASSWORD
420 |         })
421 |         if response.status_code == 200:
422 |             return response.json().get("access_token")
423 |         pytest.skip(f"Owner login failed: {response.status_code}")
424 |
425 |     def test_cleanup_test_promos(self, owner_token):
426 |         """Delete any promos with TEST_ prefix"""
427 |         headers = {"Authorization": f"Bearer {owner_token}"}
428 |
429 |         response = requests.get(f"{BASE_URL}/api/promo/all", headers=headers)
430 |         if response.status_code == 200:
431 |             promos = response.json().get("promos", [])
432 |             test_promos = [p for p in promos if p["title"].startswith("TEST_")]
433 |
434 |             for promo in test_promos:
435 |                 delete_response = requests.delete(f"{BASE_URL}/api/promo/{promo['id']}", headers=headers)
436 |                 if delete_response.status_code == 200:
437 |                     print(f"Cleaned up test promo: {promo['title']}")
438 |
439 |             print(f"PASSED: Cleanup complete. Removed {len(test_promos)} test promos")
440 |         else:
441 |             print("INFO: Could not fetch promos for cleanup")
442 |
443 |
444 | if __name__ == "__main__":
445 |     pytest.main([__file__, "-v", "--tb=short"])
446 |
```

FILE: backend/tests/test_push_notifications.py

562 lines | 26895 characters

```
1 | """
2 | Push Notification Feature Tests for RISEDUAL AI
3 | Tests: VAPID key endpoint, subscribe, unsubscribe, status, test, broadcast
4 | """
5 | import pytest
6 | import requests
7 | import os
8 |
9 | BASE_URL = os.environ.get('REACT_APP_BACKEND_URL', '').rstrip('/')
10 |
11 | # Test credentials from test_credentials.md
12 | OWNER_EMAIL = "managingdirector@redslateholdings.com"
13 | OWNER_PASSWORD = "RedSlate2026!"
14 | FREE_USER_EMAIL = "freeuser_test@test.com"
```

```
15 | FREE_USER_PASSWORD = "Test1234!"
16 |
17 |
18 | @pytest.fixture(scope="module")
19 | def api_client():
20 |     """Shared requests session"""
21 |     session = requests.Session()
22 |     session.headers.update({"Content-Type": "application/json"})
23 |     return session
24 |
25 |
26 | @pytest.fixture(scope="module")
27 | def owner_token(api_client):
28 |     """Get owner authentication token"""
29 |     response = api_client.post(f"{BASE_URL}/api/auth/login", json={
30 |         "email": OWNER_EMAIL,
31 |         "password": OWNER_PASSWORD
32 |     })
33 |     if response.status_code == 200:
34 |         return response.json().get("access_token")
35 |     pytest.skip(f"Owner authentication failed: {response.status_code}")
36 |
37 |
38 | @pytest.fixture(scope="module")
39 | def free_user_token(api_client):
40 |     """Get free user authentication token"""
41 |     response = api_client.post(f"{BASE_URL}/api/auth/login", json={
42 |         "email": FREE_USER_EMAIL,
43 |         "password": FREE_USER_PASSWORD
44 |     })
45 |     if response.status_code == 200:
46 |         return response.json().get("access_token")
47 |     pytest.skip(f"Free user authentication failed: {response.status_code}")
48 |
49 |
50 | @pytest.fixture
51 | def owner_client(api_client, owner_token):
52 |     """Session with owner auth header"""
53 |     api_client.headers.update({"Authorization": f"Bearer {owner_token}"})
54 |     return api_client
55 |
56 |
57 | @pytest.fixture
58 | def free_user_client(api_client, free_user_token):
59 |     """Session with free user auth header"""
60 |     api_client.headers.update({"Authorization": f"Bearer {free_user_token}"})
61 |     return api_client
62 |
63 |
64 | # =====
65 | # Test: VAPID Key Endpoint (Public)
66 | # =====
67 | class TestVapidKeyEndpoint:
68 |     """Test GET /api/push/vapid-key - public endpoint"""
69 |
70 |     def test_vapid_key_returns_public_key(self, api_client):
71 |         """VAPID key endpoint should return the public key without auth"""
72 |         response = api_client.get(f"{BASE_URL}/api/push/vapid-key")
73 |         assert response.status_code == 200, f"Expected 200, got {response.status_code}"
74 |
75 |         data = response.json()
76 |         assert "public_key" in data, "Response should contain 'public_key'"
77 |         assert isinstance(data["public_key"], str), "public_key should be a string"
78 |         assert len(data["public_key"]) > 0, "public_key should not be empty"
79 |         # VAPID public key should be base64url encoded
80 |         assert data["public_key"].startswith("BF"), "VAPID public key should start with 'B'"
81 |         print(f"? VAPID public key returned: {data['public_key'][:20]}...")
```

```
82 |
83 |
84 | # =====
85 | # Test: Push Status Endpoint (Authenticated)
86 | # =====
87 | class TestPushStatusEndpoint:
88 |     """Test GET /api/push/status - requires authentication"""
89 |
90 |     def test_status_requires_auth(self, api_client):
91 |         """Status endpoint should require authentication"""
92 |         # Clear any existing auth header
93 |         api_client.headers.pop("Authorization", None)
94 |         response = api_client.get(f"{BASE_URL}/api/push/status")
95 |         assert response.status_code in [401, 403], f"Expected 401/403, got {response.status_code}"
96 |         print("? Status endpoint requires authentication")
97 |
98 |     def test_status_returns_subscribed_and_is_pro_for_owner(self, api_client, owner_token):
99 |         """Owner should get status with subscribed and is_pro fields"""
100 |         api_client.headers.update({"Authorization": f"Bearer {owner_token}"})
101 |         response = api_client.get(f"{BASE_URL}/api/push/status")
102 |         assert response.status_code == 200, f"Expected 200, got {response.status_code}"
103 |
104 |         data = response.json()
105 |         assert "subscribed" in data, "Response should contain 'subscribed'"
106 |         assert "is_pro" in data, "Response should contain 'is_pro'"
107 |         assert isinstance(data["subscribed"], bool), "subscribed should be boolean"
108 |         assert isinstance(data["is_pro"], bool), "is_pro should be boolean"
109 |         assert data["is_pro"] == True, "Owner should have is_pro=True"
110 |         print(f"? Owner status: subscribed={data['subscribed']}, is_pro={data['is_pro']}")
111 |
112 |     def test_status_returns_is_pro_false_for_free_user(self, api_client, free_user_token):
113 |         """Free user should have is_pro=False"""
114 |         api_client.headers.update({"Authorization": f"Bearer {free_user_token}"})
115 |         response = api_client.get(f"{BASE_URL}/api/push/status")
116 |         assert response.status_code == 200, f"Expected 200, got {response.status_code}"
117 |
118 |         data = response.json()
119 |         assert "subscribed" in data, "Response should contain 'subscribed'"
120 |         assert "is_pro" in data, "Response should contain 'is_pro'"
121 |         assert data["is_pro"] == False, "Free user should have is_pro=False"
122 |         print(f"? Free user status: subscribed={data['subscribed']}, is_pro={data['is_pro']}")
123 |
124 |
125 | # =====
126 | # Test: Subscribe Endpoint (Authenticated)
127 | # =====
128 | class TestSubscribeEndpoint:
129 |     """Test POST /api/push/subscribe - requires authentication"""
130 |
131 |     def test_subscribe_requires_auth(self, api_client):
132 |         """Subscribe endpoint should require authentication"""
133 |         api_client.headers.pop("Authorization", None)
134 |         response = api_client.post(f"{BASE_URL}/api/push/subscribe", json={
135 |             "subscription": {"endpoint": "https://test.example.com", "keys": {"p256dh": "test", "auth": "test"}}
136 |         })
137 |         assert response.status_code in [401, 403], f"Expected 401/403, got {response.status_code}"
138 |         print("? Subscribe endpoint requires authentication")
139 |
140 |     def test_subscribe_stores_subscription_for_owner(self, api_client, owner_token):
141 |         """Owner should be able to subscribe to push notifications"""
142 |         api_client.headers.update({"Authorization": f"Bearer {owner_token}"})
143 |
144 |         # Create a mock subscription object (similar to what browser would send)
145 |         mock_subscription = {
146 |             "endpoint": "https://fcm.googleapis.com/fcm/send/test-owner-endpoint",
147 |             "keys": {
148 |                 "p256dh": "BNcRdreALRFXtK00UHK1EtK2wtaz5Ry4YfYCA_0QTpQtUubVLuIs0VJXg7A8u-Ts1XbjhazAkj7I99e8QcYP7DkM",
```

```

149 |         "auth": "tBHItJI5svbpez7KI4CCXg"
150 |     }
151 | }
152 |
153 | response = api_client.post(f"{BASE_URL}/api/push/subscribe", json={
154 |     "subscription": mock_subscription
155 | })
156 | assert response.status_code == 200, f"Expected 200, got {response.status_code}"
157 |
158 | data = response.json()
159 | assert "message" in data, "Response should contain 'message'"
160 | assert "enabled" in data["message"].lower() or "success" in data["message"].lower() or "push" in data["message"].lower(), ...
161 |     f"Message should indicate success: {data['message']}"
162 | print(f"? Owner subscribed: {data['message']}")
163 |
164 | # Verify subscription status
165 | status_response = api_client.get(f"{BASE_URL}/api/push/status")
166 | assert status_response.status_code == 200
167 | status_data = status_response.json()
168 | assert status_data["subscribed"] == True, "Owner should now be subscribed"
169 | print(f"? Owner subscription verified via status endpoint")
170 |
171 | def test_subscribe_stores_subscription_for_free_user(self, api_client, free_user_token):
172 |     """Free user should be able to subscribe to push notifications"""
173 |     api_client.headers.update({"Authorization": f"Bearer {free_user_token}"})
174 |
175 |     mock_subscription = {
176 |         "endpoint": "https://fcm.googleapis.com/fcm/send/test-free-user-endpoint",
177 |         "keys": {
178 |             "p256dh": "BNcRdreALRFXTk00UHK1EtK2wtaz5Ry4YfYCA_0QTpQtUvVLu0VJXg7A8u-Ts1XbjhazAkj7I99e8QcYP7DkM",
179 |             "auth": "tBHItJI5svbpez7KI4CCXg"
180 |         }
181 |     }
182 |
183 |     response = api_client.post(f"{BASE_URL}/api/push/subscribe", json={
184 |         "subscription": mock_subscription
185 |     })
186 |     assert response.status_code == 200, f"Expected 200, got {response.status_code}"
187 |
188 |     data = response.json()
189 |     assert "message" in data, "Response should contain 'message'"
190 |     print(f"? Free user subscribed: {data['message']}")
191 |
192 |
193 | # =====
194 | # Test: Unsubscribe Endpoint (Authenticated)
195 | # =====
196 | class TestUnsubscribeEndpoint:
197 |     """Test POST /api/push/unsubscribe - requires authentication"""
198 |
199 |     def test_unsubscribe_requires_auth(self, api_client):
200 |         """Unsubscribe endpoint should require authentication"""
201 |         api_client.headers.pop("Authorization", None)
202 |         response = api_client.post(f"{BASE_URL}/api/push/unsubscribe")
203 |         assert response.status_code in [401, 403], f"Expected 401/403, got {response.status_code}"
204 |         print(f"? Unsubscribe endpoint requires authentication")
205 |
206 |     def test_unsubscribe_removes_subscription(self, api_client, free_user_token):
207 |         """Free user should be able to unsubscribe from push notifications"""
208 |         api_client.headers.update({"Authorization": f"Bearer {free_user_token}"})
209 |
210 |         # First subscribe
211 |         mock_subscription = {
212 |             "endpoint": "https://fcm.googleapis.com/fcm/send/test-unsubscribe-endpoint",
213 |             "keys": {"p256dh": "test", "auth": "test"}
214 |         }
215 |         api_client.post(f"{BASE_URL}/api/push/subscribe", json={"subscription": mock_subscription})

```

```
216 |
217 |     # Then unsubscribe
218 |     response = api_client.post(f"{BASE_URL}/api/push/unsubscribe")
219 |     assert response.status_code == 200, f"Expected 200, got {response.status_code}"
220 |
221 |     data = response.json()
222 |     assert "message" in data, "Response should contain 'message'"
223 |     assert "disabled" in data["message"].lower() or "removed" in data["message"].lower() or "push" in data["message"].lower(), ...
224 |     f"Message should indicate unsubscribed: {data['message']}"
225 |     print(f"? Free user unsubscribed: {data['message']}")
226 |
227 |     # Verify subscription status
228 |     status_response = api_client.get(f"{BASE_URL}/api/push/status")
229 |     assert status_response.status_code == 200
230 |     status_data = status_response.json()
231 |     assert status_data["subscribed"] == False, "User should now be unsubscribed"
232 |     print("? Unsubscription verified via status endpoint")
233 |
234 |
235 | # =====
236 | # Test: Test Push Endpoint (Admin Only)
237 | # =====
238 | class TestTestPushEndpoint:
239 |     """Test POST /api/push/test - requires admin/owner role"""
240 |
241 |     def test_test_push_requires_auth(self, api_client):
242 |         """Test push endpoint should require authentication"""
243 |         api_client.headers.pop("Authorization", None)
244 |         response = api_client.post(f"{BASE_URL}/api/push/test")
245 |         assert response.status_code in [401, 403], f"Expected 401/403, got {response.status_code}"
246 |         print("? Test push endpoint requires authentication")
247 |
248 |     def test_test_push_returns_403_for_free_user(self, api_client, free_user_token):
249 |         """Free user should get 403 when trying to send test push"""
250 |         api_client.headers.update({"Authorization": f"Bearer {free_user_token}"})
251 |         response = api_client.post(f"{BASE_URL}/api/push/test")
252 |         assert response.status_code == 403, f"Expected 403, got {response.status_code}"
253 |
254 |         data = response.json()
255 |         assert "detail" in data, "Response should contain 'detail'"
256 |         assert "admin" in data["detail"].lower(), f"Error should mention admin: {data['detail']}"
257 |         print(f"? Free user correctly denied: {data['detail']}")
258 |
259 |     def test_test_push_works_for_owner(self, api_client, owner_token):
260 |         """Owner should be able to send test push (may fail if no subscription)"""
261 |         api_client.headers.update({"Authorization": f"Bearer {owner_token}"})
262 |
263 |         # First ensure owner is subscribed
264 |         mock_subscription = {
265 |             "endpoint": "https://fcm.googleapis.com/fcm/send/test-owner-push-endpoint",
266 |             "keys": {"p256dh": "test", "auth": "test"}
267 |         }
268 |         api_client.post(f"{BASE_URL}/api/push/subscribe", json={"subscription": mock_subscription})
269 |
270 |         response = api_client.post(f"{BASE_URL}/api/push/test")
271 |         # Should be 200 (success or failure in sending) or 400 (no subscription)
272 |         assert response.status_code in [200, 400], f"Expected 200 or 400, got {response.status_code}"
273 |
274 |         data = response.json()
275 |         if response.status_code == 200:
276 |             assert "sent" in data, "Response should contain 'sent'"
277 |             print(f"? Owner test push result: sent={data['sent']}")
278 |         else:
279 |             print(f"? Owner test push returned 400 (expected if no valid subscription): {data}")
280 |
281 |
282 | # =====
```

```
283 | # Test: Broadcast Endpoint (Admin Only)
284 | # =====
285 | class TestBroadcastEndpoint:
286 |     """Test POST /api/push/broadcast - requires admin/owner role"""
287 |
288 |     def test_broadcast_requires_auth(self, api_client):
289 |         """Broadcast endpoint should require authentication"""
290 |         api_client.headers.pop("Authorization", None)
291 |         response = api_client.post(f"{BASE_URL}/api/push/broadcast", json={
292 |             "title": "Test", "message": "Test message"
293 |         })
294 |         assert response.status_code in [401, 403], f"Expected 401/403, got {response.status_code}"
295 |         print(f"? Broadcast endpoint requires authentication")
296 |
297 |     def test_broadcast_returns_403_for_free_user(self, api_client, free_user_token):
298 |         """Free user should get 403 when trying to broadcast"""
299 |         api_client.headers.update({"Authorization": f"Bearer {free_user_token}"})
300 |         response = api_client.post(f"{BASE_URL}/api/push/broadcast", json={
301 |             "title": "Test", "message": "Test message"
302 |         })
303 |         assert response.status_code == 403, f"Expected 403, got {response.status_code}"
304 |
305 |         data = response.json()
306 |         assert "detail" in data, "Response should contain 'detail'"
307 |         assert "admin" in data["detail"].lower(), f"Error should mention admin: {data['detail']}"
308 |         print(f"? Free user correctly denied broadcast: {data['detail']}")
309 |
310 |     def test_broadcast_works_for_owner(self, api_client, owner_token):
311 |         """Owner should be able to broadcast push notifications"""
312 |         api_client.headers.update({"Authorization": f"Bearer {owner_token}"})
313 |         response = api_client.post(f"{BASE_URL}/api/push/broadcast", json={
314 |             "title": "RISEDUAL AI Test Broadcast",
315 |             "message": "This is a test broadcast message"
316 |         })
317 |         assert response.status_code == 200, f"Expected 200, got {response.status_code}"
318 |
319 |         data = response.json()
320 |         # Should return sent count (may be 0 if no valid subscriptions)
321 |         assert "sent" in data or "skipped" in data, f"Response should contain 'sent' or 'skipped': {data}"
322 |         print(f"? Owner broadcast result: {data}")
323 |
324 |
325 | # =====
326 | # Test: Push Service Module
327 | # =====
328 | class TestPushServiceModule:
329 |     """Test push_service.py module loads and functions correctly"""
330 |
331 |     def test_push_service_imports_without_errors(self):
332 |         """push_service.py should import without errors"""
333 |         try:
334 |             import sys
335 |             sys.path.insert(0, '/app/backend')
336 |             from services.push_service import (
337 |                 _is_configured, send_push, broadcast_notification,
338 |                 notify_prediction_flip, notify_dark_pool_spike,
339 |                 notify_watchlist_alert, notify_market_signal,
340 |                 FREE_DAILY_LIMIT, NOTIF_PREDICTION_FLIP, NOTIF_DARK_POOL_SPIKE,
341 |                 NOTIF_WATCHLIST_ALERT, NOTIF_MARKET_SIGNAL
342 |             )
343 |             print(f"? push_service.py imports successfully")
344 |         except ImportError as e:
345 |             pytest.fail(f"Failed to import push_service: {e}")
346 |
347 |     def test_is_configured_returns_true_via_api(self, api_client):
348 |         """VAPID keys should be configured (verified via API)"""
349 |         # Test via API since direct import doesn't load .env
```



```

350 |         response = api_client.get(f"{BASE_URL}/api/push/vapid-key")
351 |         assert response.status_code == 200
352 |         data = response.json()
353 |         assert data.get("public_key"), "VAPID public key should be set"
354 |         assert len(data["public_key"]) > 50, "VAPID key should be valid length"
355 |         print("? VAPID keys configured (verified via API)")
356 |
357 |     def test_free_daily_limit_is_one(self):
358 |         """FREE_DAILY_LIMIT should be 1"""
359 |         import sys
360 |         sys.path.insert(0, '/app/backend')
361 |         from services.push_service import FREE_DAILY_LIMIT
362 |
363 |         assert FREE_DAILY_LIMIT == 1, f"FREE_DAILY_LIMIT should be 1, got {FREE_DAILY_LIMIT}"
364 |         print(f"? FREE_DAILY_LIMIT = {FREE_DAILY_LIMIT}")
365 |
366 |     def test_notification_types_defined(self):
367 |         """Notification type constants should be defined"""
368 |         import sys
369 |         sys.path.insert(0, '/app/backend')
370 |         from services.push_service import (
371 |             NOTIF_PREDICTION_FLIP, NOTIF_DARK_POOL_SPIKE,
372 |             NOTIF_WATCHLIST_ALERT, NOTIF_MARKET_SIGNAL
373 |         )
374 |
375 |         assert NOTIF_PREDICTION_FLIP == "prediction_flip"
376 |         assert NOTIF_DARK_POOL_SPIKE == "dark_pool_spike"
377 |         assert NOTIF_WATCHLIST_ALERT == "watchlist_alert"
378 |         assert NOTIF_MARKET_SIGNAL == "market_signal"
379 |         print("? All notification type constants defined correctly")
380 |
381 |
382 | # =====
383 | # Test: Service Worker File
384 | # =====
385 | class TestServiceWorkerFile:
386 |     """Test service-worker.js has required push handlers"""
387 |
388 |     def test_service_worker_has_push_handler(self):
389 |         """service-worker.js should have push event listener"""
390 |         sw_path = "/app/frontend/public/service-worker.js"
391 |         with open(sw_path, 'r') as f:
392 |             content = f.read()
393 |
394 |         assert "addEventListener('push'" in content or 'addEventListener("push"' in content, \
395 |             "service-worker.js should have push event listener"
396 |         print("? service-worker.js has push event listener")
397 |
398 |     def test_service_worker_has_notificationclick_handler(self):
399 |         """service-worker.js should have notificationclick event listener"""
400 |         sw_path = "/app/frontend/public/service-worker.js"
401 |         with open(sw_path, 'r') as f:
402 |             content = f.read()
403 |
404 |         assert "addEventListener('notificationclick'" in content or 'addEventListener("notificationclick"' in content, \
405 |             "service-worker.js should have notificationclick event listener"
406 |         print("? service-worker.js has notificationclick event listener")
407 |
408 |     def test_service_worker_shows_notification(self):
409 |         """service-worker.js push handler should call showNotification"""
410 |         sw_path = "/app/frontend/public/service-worker.js"
411 |         with open(sw_path, 'r') as f:
412 |             content = f.read()
413 |
414 |         assert "showNotification" in content, \
415 |             "service-worker.js should call showNotification in push handler"
416 |         print("? service-worker.js calls showNotification")

```

```
417 |
418 |
419 | # =====
420 | # Test: Frontend Hook File
421 | # =====
422 | class TestFrontendHookFile:
423 |     """Test usePushNotifications.js hook exists and has required functions"""
424 |
425 |     def test_hook_file_exists(self):
426 |         """usePushNotifications.js should exist"""
427 |         hook_path = "/app/frontend/src/hooks/usePushNotifications.js"
428 |         import os
429 |         assert os.path.exists(hook_path), f"Hook file should exist at {hook_path}"
430 |         print("? usePushNotifications.js exists")
431 |
432 |     def test_hook_exports_usePushNotifications(self):
433 |         """Hook should export usePushNotifications function"""
434 |         hook_path = "/app/frontend/src/hooks/usePushNotifications.js"
435 |         with open(hook_path, 'r') as f:
436 |             content = f.read()
437 |
438 |             assert "export function usePushNotifications" in content or "export { usePushNotifications" in content, \
439 |                 "Hook should export usePushNotifications"
440 |             print("? usePushNotifications function exported")
441 |
442 |     def test_hook_has_subscribe_function(self):
443 |         """Hook should have subscribe function"""
444 |         hook_path = "/app/frontend/src/hooks/usePushNotifications.js"
445 |         with open(hook_path, 'r') as f:
446 |             content = f.read()
447 |
448 |             assert "subscribe" in content, "Hook should have subscribe function"
449 |             assert "/api/push/subscribe" in content, "Hook should call /api/push/subscribe endpoint"
450 |             print("? Hook has subscribe function with correct endpoint")
451 |
452 |     def test_hook_has_unsubscribe_function(self):
453 |         """Hook should have unsubscribe function"""
454 |         hook_path = "/app/frontend/src/hooks/usePushNotifications.js"
455 |         with open(hook_path, 'r') as f:
456 |             content = f.read()
457 |
458 |             assert "unsubscribe" in content, "Hook should have unsubscribe function"
459 |             assert "/api/push/unsubscribe" in content, "Hook should call /api/push/unsubscribe endpoint"
460 |             print("? Hook has unsubscribe function with correct endpoint")
461 |
462 |     def test_hook_uses_vapid_key_from_env(self):
463 |         """Hook should use VAPID key from environment"""
464 |         hook_path = "/app/frontend/src/hooks/usePushNotifications.js"
465 |         with open(hook_path, 'r') as f:
466 |             content = f.read()
467 |
468 |             assert "REACT_APP_VAPID_PUBLIC_KEY" in content, \
469 |                 "Hook should use REACT_APP_VAPID_PUBLIC_KEY from environment"
470 |             print("? Hook uses REACT_APP_VAPID_PUBLIC_KEY from environment")
471 |
472 |
473 | # =====
474 | # Test: Frontend Environment Variable
475 | # =====
476 | class TestFrontendEnvVariable:
477 |     """Test REACT_APP_VAPID_PUBLIC_KEY is set in frontend .env"""
478 |
479 |     def test_vapid_key_in_frontend_env(self):
480 |         """REACT_APP_VAPID_PUBLIC_KEY should be set in frontend .env"""
481 |         env_path = "/app/frontend/.env"
482 |         with open(env_path, 'r') as f:
483 |             content = f.read()
```

```
484 |
485 |     assert "REACT_APP_VAPID_PUBLIC_KEY=" in content, \
486 |           "REACT_APP_VAPID_PUBLIC_KEY should be set in frontend .env"
487 |
488 |     # Extract the value
489 |     for line in content.split('\n'):
490 |         if line.startswith('REACT_APP_VAPID_PUBLIC_KEY='):
491 |             value = line.split('=', 1)[1].strip()
492 |             assert len(value) > 0, "VAPID key should not be empty"
493 |             assert value.startswith('B'), "VAPID public key should start with 'B'"
494 |             print(f"? REACT_APP_VAPID_PUBLIC_KEY set: {value[:20]}...")
495 |             return
496 |
497 |     pytest.fail("Could not find REACT_APP_VAPID_PUBLIC_KEY value")
498 |
499 |
500 | # =====
501 | # Test: UserWorkspace PushToggle Component
502 | # =====
503 | class TestUserWorkspacePushToggle:
504 |     """Test PushToggle component exists in UserWorkspace.jsx"""
505 |
506 |     def test_push_toggle_component_exists(self):
507 |         """PushToggle component should be defined in UserWorkspace.jsx"""
508 |         component_path = "/app/frontend/src/components/UserWorkspace.jsx"
509 |         with open(component_path, 'r') as f:
510 |             content = f.read()
511 |
512 |         assert "const PushToggle" in content or "function PushToggle" in content, \
513 |               "PushToggle component should be defined"
514 |         print("? PushToggle component defined in UserWorkspace.jsx")
515 |
516 |     def test_push_toggle_uses_hook(self):
517 |         """PushToggle should use usePushNotifications hook"""
518 |         component_path = "/app/frontend/src/components/UserWorkspace.jsx"
519 |         with open(component_path, 'r') as f:
520 |             content = f.read()
521 |
522 |         assert "usePushNotifications" in content, \
523 |               "PushToggle should use usePushNotifications hook"
524 |         print("? PushToggle uses usePushNotifications hook")
525 |
526 |     def test_push_toggle_has_data_testid(self):
527 |         """PushToggle should have data-testid attribute"""
528 |         component_path = "/app/frontend/src/components/UserWorkspace.jsx"
529 |         with open(component_path, 'r') as f:
530 |             content = f.read()
531 |
532 |         assert 'data-testid="push-toggle"' in content, \
533 |               "PushToggle should have data-testid='push-toggle'"
534 |         assert 'data-testid="push-toggle-btn"' in content, \
535 |               "PushToggle button should have data-testid='push-toggle-btn'"
536 |         print("? PushToggle has data-testid attributes")
537 |
538 |     def test_push_toggle_in_referrals_tab(self):
539 |         """PushToggle should be rendered in ReferralsTab"""
540 |         component_path = "/app/frontend/src/components/UserWorkspace.jsx"
541 |         with open(component_path, 'r') as f:
542 |             content = f.read()
543 |
544 |         # Check that PushToggle is used in ReferralsTab
545 |         assert "<PushToggle" in content, "PushToggle should be rendered"
546 |         print("? PushToggle is rendered in component")
547 |
548 |     def test_push_toggle_shows_states(self):
549 |         """PushToggle should show Enable/Enabled/Blocked states"""
550 |         component_path = "/app/frontend/src/components/UserWorkspace.jsx"
```

```
551 |         with open(component_path, 'r') as f:
552 |             content = f.read()
553 |
554 |         assert "Enable" in content, "PushToggle should show 'Enable' state"
555 |         assert "Enabled" in content, "PushToggle should show 'Enabled' state"
556 |         assert "Blocked" in content, "PushToggle should show 'Blocked' state"
557 |         print("? PushToggle shows Enable/Enabled/Blocked states")
558 |
559 |
560 | if __name__ == "__main__":
561 |     pytest.main([__file__, "-v", "--tb=short"])
562 |
```

FILE: backend/tests/test_referral_feature.py*415 lines | 17865 characters*

```
1 | """
2 | Test Referral Feature - RISEDUAL AI
3 | Tests: referral code generation, validation, signup with ref code, self-referral prevention,
4 |         referral completion on Pro upgrade, 12-month rolling cap enforcement
5 | """
6 | import pytest
7 | import requests
8 | import os
9 | import uuid
10 |
11 | BASE_URL = os.environ.get('REACT_APP_BACKEND_URL', '').rstrip('/')
12 |
13 | # Test credentials
14 | OWNER_EMAIL = "managingdirector@redslateholdings.com"
15 | OWNER_PASSWORD = "RedSlate2026!"
16 | ADMIN_EMAIL = "admin@risedual.ai"
17 | ADMIN_PASSWORD = "RiseDual2026!"
18 |
19 |
20 | class TestReferralInfo:
21 |     """Test GET /api/referral/info endpoint"""
22 |
23 |     def test_referral_info_unauthenticated(self):
24 |         """Unauthenticated request should return 401"""
25 |         res = requests.get(f"{BASE_URL}/api/referral/info")
26 |         assert res.status_code == 401, f"Expected 401, got {res.status_code}"
27 |         print("PASSED: Unauthenticated referral info returns 401")
28 |
29 |     def test_referral_info_authenticated_owner(self):
30 |         """Owner should get referral code and stats"""
31 |         # Login as owner
32 |         login_res = requests.post(f"{BASE_URL}/api/auth/login", json={
33 |             "email": OWNER_EMAIL,
34 |             "password": OWNER_PASSWORD
35 |         })
36 |         assert login_res.status_code == 200, f"Login failed: {login_res.text}"
37 |         token = login_res.json().get("access_token")
38 |
39 |         # Get referral info
40 |         res = requests.get(
41 |             f"{BASE_URL}/api/referral/info",
42 |             headers={"Authorization": f"Bearer {token}"}
43 |         )
44 |         assert res.status_code == 200, f"Expected 200, got {res.status_code}: {res.text}"
45 |
46 |         data = res.json()
47 |         # Validate response structure
48 |         assert "code" in data, "Response missing 'code'"
49 |         assert "total_referrals" in data, "Response missing 'total_referrals'"
50 |         assert "completed_referrals" in data, "Response missing 'completed_referrals'"

```

```
51 |         assert "rewards_earned" in data, "Response missing 'rewards_earned'"
52 |         assert "rewards_remaining" in data, "Response missing 'rewards_remaining'"
53 |         assert "reward_cap" in data, "Response missing 'reward_cap'"
54 |         assert "referrals" in data, "Response missing 'referrals'"
55 |
56 |         # Validate data types
57 |         assert isinstance(data["code"], str), "code should be string"
58 |         assert len(data["code"]) == 8, f"code should be 8 chars, got {len(data['code'])}"
59 |         assert isinstance(data["total_referrals"], int), "total_referrals should be int"
60 |         assert isinstance(data["rewards_remaining"], int), "rewards_remaining should be int"
61 |         assert data["reward_cap"] == 12, f"reward_cap should be 12, got {data['reward_cap']}"
62 |         assert isinstance(data["referrals"], list), "referrals should be list"
63 |
64 |         print(f"PASSED: Owner referral info - code={data['code']}, total={data['total_referrals']}, earned={data['rewards_earned']} ...")
65 |         return data["code"]
66 |
67 |
68 | class TestReferralValidation:
69 |     """Test GET /api/referral/validate/{code} endpoint"""
70 |
71 |     def test_validate_valid_code(self):
72 |         """Valid referral code should return valid:true"""
73 |         # First get owner's code
74 |         login_res = requests.post(f"{BASE_URL}/api/auth/login", json={
75 |             "email": OWNER_EMAIL,
76 |             "password": OWNER_PASSWORD
77 |         })
78 |         token = login_res.json().get("access_token")
79 |
80 |         info_res = requests.get(
81 |             f"{BASE_URL}/api/referral/info",
82 |             headers={"Authorization": f"Bearer {token}"}
83 |         )
84 |         owner_code = info_res.json()["code"]
85 |
86 |         # Validate the code (no auth required)
87 |         res = requests.get(f"{BASE_URL}/api/referral/validate/{owner_code}")
88 |         assert res.status_code == 200, f"Expected 200, got {res.status_code}"
89 |
90 |         data = res.json()
91 |         assert data["valid"] == True, f"Expected valid=True, got {data}"
92 |         assert data["code"] == owner_code, f"Expected code={owner_code}, got {data['code']}"
93 |         print(f"PASSED: Valid code {owner_code} returns valid=True")
94 |
95 |     def test_validate_invalid_code(self):
96 |         """Invalid referral code should return valid:false"""
97 |         res = requests.get(f"{BASE_URL}/api/referral/validate/INVALID123")
98 |         assert res.status_code == 200, f"Expected 200, got {res.status_code}"
99 |
100 |         data = res.json()
101 |         assert data["valid"] == False, f"Expected valid=False, got {data}"
102 |         print("PASSED: Invalid code returns valid=False")
103 |
104 |     def test_validate_lowercase_code(self):
105 |         """Lowercase code should be normalized to uppercase"""
106 |         # Get owner's code
107 |         login_res = requests.post(f"{BASE_URL}/api/auth/login", json={
108 |             "email": OWNER_EMAIL,
109 |             "password": OWNER_PASSWORD
110 |         })
111 |         token = login_res.json().get("access_token")
112 |
113 |         info_res = requests.get(
114 |             f"{BASE_URL}/api/referral/info",
115 |             headers={"Authorization": f"Bearer {token}"}
116 |         )
117 |         owner_code = info_res.json()["code"]
```

```
118 |
119 |     # Validate with lowercase
120 |     res = requests.get(f"{BASE_URL}/api/referral/validate/{owner_code.lower()}")
121 |     assert res.status_code == 200
122 |
123 |     data = res.json()
124 |     assert data["valid"] == True, f"Lowercase code should be valid, got {data}"
125 |     print(f"PASSED: Lowercase code {owner_code.lower()} normalized and validated")
126 |
127 |
128 | class TestReferralSignup:
129 |     """Test registration with referral code"""
130 |
131 |     def test_register_with_valid_ref_code(self):
132 |         """Registration with valid ref code should set subscription_status to 'trial'"""
133 |         # Get owner's referral code
134 |         login_res = requests.post(f"{BASE_URL}/api/auth/login", json={
135 |             "email": OWNER_EMAIL,
136 |             "password": OWNER_PASSWORD
137 |         })
138 |         token = login_res.json().get("access_token")
139 |
140 |         info_res = requests.get(
141 |             f"{BASE_URL}/api/referral/info",
142 |             headers={"Authorization": f"Bearer {token}"})
143 |         )
144 |         owner_code = info_res.json()["code"]
145 |
146 |         # Register new user with ref code
147 |         unique_email = f"test_referred_{uuid.uuid4().hex[:8]}@test.com"
148 |         reg_res = requests.post(f"{BASE_URL}/api/auth/register", json={
149 |             "email": unique_email,
150 |             "password": "Test1234!",
151 |             "name": "Test Referred User",
152 |             "ref_code": owner_code
153 |         })
154 |         assert reg_res.status_code == 200, f"Registration failed: {reg_res.text}"
155 |
156 |         data = reg_res.json()
157 |         assert data["subscription_status"] == "trial", f"Expected subscription_status='trial', got {data.get('subscription_status')} ..."
158 |         assert "trial_ends_at" in data or data.get("subscription_status") == "trial", "Trial should be granted"
159 |
160 |         print(f"PASSED: User {unique_email} registered with ref code, subscription_status=trial")
161 |         return unique_email, data.get("access_token")
162 |
163 |     def test_register_without_ref_code(self):
164 |         """Registration without ref code should set subscription_status to 'free'"""
165 |         unique_email = f"test_noreferral_{uuid.uuid4().hex[:8]}@test.com"
166 |         reg_res = requests.post(f"{BASE_URL}/api/auth/register", json={
167 |             "email": unique_email,
168 |             "password": "Test1234!",
169 |             "name": "Test No Referral User"
170 |         })
171 |         assert reg_res.status_code == 200, f"Registration failed: {reg_res.text}"
172 |
173 |         data = reg_res.json()
174 |         assert data["subscription_status"] == "free", f"Expected subscription_status='free', got {data.get('subscription_status')} ..."
175 |
176 |         print(f"PASSED: User {unique_email} registered without ref code, subscription_status=free")
177 |
178 |
179 | class TestSelfReferralPrevention:
180 |     """Test that users cannot use their own referral code"""
181 |
182 |     def test_self_referral_prevented(self):
183 |         """User should not be able to refer themselves"""
184 |         # Create a new user first
```

```
185 |         unique_email = f"test_selfreferral_{uuid.uuid4().hex[:8]}@test.com"
186 |         reg_res = requests.post(f"{BASE_URL}/api/auth/register", json={
187 |             "email": unique_email,
188 |             "password": "Test1234!",
189 |             "name": "Test Self Referral"
190 |         })
191 |         assert reg_res.status_code == 200
192 |         token = reg_res.json().get("access_token")
193 |
194 |         # Get their referral code
195 |         info_res = requests.get(
196 |             f"{BASE_URL}/api/referral/info",
197 |             headers={"Authorization": f"Bearer {token}"})
198 |
199 |         assert info_res.status_code == 200
200 |         user_code = info_res.json()["code"]
201 |
202 |         # Try to register another account with their own code
203 |         # (This tests the backend logic - in practice, user would already be registered)
204 |         # The self-referral check happens in process_referral_signup
205 |
206 |         # Verify the user's referral stats don't show self-referral
207 |         info_res2 = requests.get(
208 |             f"{BASE_URL}/api/referral/info",
209 |             headers={"Authorization": f"Bearer {token}"})
210 |
211 |         data = info_res2.json()
212 |
213 |         # Check that no self-referral was recorded
214 |         for ref in data.get("referrals", []):
215 |             assert ref.get("referred_email") != unique_email, "Self-referral should not be recorded"
216 |
217 |         print(f"PASSED: Self-referral prevention verified for {unique_email}")
218 |
219 |
220 | class TestReferralCompletion:
221 |     """Test referral completion when referred user upgrades to Pro"""
222 |
223 |     def test_referral_completes_on_pro_upgrade(self):
224 |         """When referred user upgrades to Pro, referral status should become 'completed'"""
225 |         # Get owner's referral code
226 |         login_res = requests.post(f"{BASE_URL}/api/auth/login", json={
227 |             "email": OWNER_EMAIL,
228 |             "password": OWNER_PASSWORD
229 |         })
230 |         owner_token = login_res.json().get("access_token")
231 |
232 |         info_res = requests.get(
233 |             f"{BASE_URL}/api/referral/info",
234 |             headers={"Authorization": f"Bearer {owner_token}"})
235 |
236 |         owner_code = info_res.json()["code"]
237 |         initial_completed = info_res.json()["completed_referrals"]
238 |
239 |         # Register new user with ref code
240 |         unique_email = f"test_upgrade_{uuid.uuid4().hex[:8]}@test.com"
241 |         reg_res = requests.post(f"{BASE_URL}/api/auth/register", json={
242 |             "email": unique_email,
243 |             "password": "Test1234!",
244 |             "name": "Test Upgrade User",
245 |             "ref_code": owner_code
246 |         })
247 |         assert reg_res.status_code == 200
248 |         referred_token = reg_res.json().get("access_token")
249 |
250 |         # Verify referral is pending
251 |         info_res2 = requests.get(
```

```
252 |         f"{BASE_URL}/api/referral/info",
253 |         headers={"Authorization": f"Bearer {owner_token}"}}
254 |     )
255 |     referrals = info_res2.json()["referrals"]
256 |     pending_ref = next((r for r in referrals if r.get("referred_email") == unique_email), None)
257 |     assert pending_ref is not None, f"Referral for {unique_email} not found"
258 |     assert pending_ref["status"] == "pending", f"Expected status='pending', got {pending_ref['status']}"
259 |
260 |     # Upgrade referred user to Pro
261 |     upgrade_res = requests.post(
262 |         f"{BASE_URL}/api/user/subscription",
263 |         headers={"Authorization": f"Bearer {referred_token}"},
264 |         json={"status": "pro", "plan": "monthly"}
265 |     )
266 |     assert upgrade_res.status_code == 200, f"Upgrade failed: {upgrade_res.text}"
267 |
268 |     # Verify referral is now completed
269 |     info_res3 = requests.get(
270 |         f"{BASE_URL}/api/referral/info",
271 |         headers={"Authorization": f"Bearer {owner_token}"}}
272 |     )
273 |     referrals_after = info_res3.json()["referrals"]
274 |     completed_ref = next((r for r in referrals_after if r.get("referred_email") == unique_email), None)
275 |     assert completed_ref is not None, f"Referral for {unique_email} not found after upgrade"
276 |     assert completed_ref["status"] == "completed", f"Expected status='completed', got {completed_ref['status']}"
277 |     assert completed_ref["reward_granted"] == True, f"Expected reward_granted=True, got {completed_ref['reward_granted']}"
278 |
279 |     print(f"PASSED: Referral for {unique_email} completed on Pro upgrade, reward_granted=True")
280 |
281 |
282 | class TestDuplicateReferralPrevention:
283 |     """Test that same user cannot be referred twice"""
284 |
285 |     def test_duplicate_referral_prevented(self):
286 |         """Same user should not be able to be referred twice"""
287 |         # Get owner's referral code
288 |         login_res = requests.post(f"{BASE_URL}/api/auth/login", json={
289 |             "email": OWNER_EMAIL,
290 |             "password": OWNER_PASSWORD
291 |         })
292 |         owner_token = login_res.json().get("access_token")
293 |
294 |         info_res = requests.get(
295 |             f"{BASE_URL}/api/referral/info",
296 |             headers={"Authorization": f"Bearer {owner_token}"}}
297 |         )
298 |         owner_code = info_res.json()["code"]
299 |
300 |         # Get admin's referral code
301 |         admin_login = requests.post(f"{BASE_URL}/api/auth/login", json={
302 |             "email": ADMIN_EMAIL,
303 |             "password": ADMIN_PASSWORD
304 |         })
305 |         admin_token = admin_login.json().get("access_token")
306 |
307 |         admin_info = requests.get(
308 |             f"{BASE_URL}/api/referral/info",
309 |             headers={"Authorization": f"Bearer {admin_token}"}}
310 |         )
311 |         admin_code = admin_info.json()["code"]
312 |
313 |         # Register user with owner's code
314 |         unique_email = f"test_duplicate_{uuid.uuid4().hex[:8]}@test.com"
315 |         reg_res = requests.post(f"{BASE_URL}/api/auth/register", json={
316 |             "email": unique_email,
317 |             "password": "Test1234!",
318 |             "name": "Test Duplicate User",
```



```

319 |         "ref_code": owner_code
320 |     })
321 |     assert reg_res.status_code == 200
322 |
323 |     # Check owner's referrals
324 |     info_res2 = requests.get(
325 |         f"{BASE_URL}/api/referral/info",
326 |         headers={"Authorization": f"Bearer {owner_token}"})
327 |
328 |     owner_referrals = [r for r in info_res2.json()["referrals"] if r.get("referred_email") == unique_email]
329 |
330 |     # Check admin's referrals (should not have this user)
331 |     admin_info2 = requests.get(
332 |         f"{BASE_URL}/api/referral/info",
333 |         headers={"Authorization": f"Bearer {admin_token}"})
334 |
335 |     admin_referrals = [r for r in admin_info2.json()["referrals"] if r.get("referred_email") == unique_email]
336 |
337 |     assert len(owner_referrals) == 1, f"Owner should have exactly 1 referral for {unique_email}"
338 |     assert len(admin_referrals) == 0, f"Admin should have 0 referrals for {unique_email}"
339 |
340 |     print(f"PASSED: Duplicate referral prevention verified - user {unique_email} only referred once")
341 |
342 |
343 | class TestRollingCapEnforcement:
344 |     """Test 12-month rolling cap enforcement"""
345 |
346 |     def test_reward_cap_info_displayed(self):
347 |         """Referral info should show reward cap of 12"""
348 |         login_res = requests.post(f"{BASE_URL}/api/auth/login", json={
349 |             "email": OWNER_EMAIL,
350 |             "password": OWNER_PASSWORD
351 |         })
352 |         token = login_res.json().get("access_token")
353 |
354 |         info_res = requests.get(
355 |             f"{BASE_URL}/api/referral/info",
356 |             headers={"Authorization": f"Bearer {token}"})
357 |
358 |         data = info_res.json()
359 |
360 |         assert data["reward_cap"] == 12, f"Expected reward_cap=12, got {data['reward_cap']}"
361 |         assert data["rewards_remaining"] <= 12, f"rewards_remaining should be <= 12"
362 |         assert data["rewards_remaining"] >= 0, f"rewards_remaining should be >= 0"
363 |         assert data["rewards_earned"] + data["rewards_remaining"] <= 12, "earned + remaining should be <= cap"
364 |
365 |         print(f"PASSED: Reward cap displayed correctly - cap=12, earned={data['rewards_earned']}, remaining={data['rewards_remaini ...
366 |
367 |
368 | class TestIsPro:
369 |     """Test that isPro includes 'trial' subscription status"""
370 |
371 |     def test_trial_user_has_pro_access(self):
372 |         """User with subscription_status='trial' should have Pro access"""
373 |         # Get owner's referral code
374 |         login_res = requests.post(f"{BASE_URL}/api/auth/login", json={
375 |             "email": OWNER_EMAIL,
376 |             "password": OWNER_PASSWORD
377 |         })
378 |         owner_token = login_res.json().get("access_token")
379 |
380 |         info_res = requests.get(
381 |             f"{BASE_URL}/api/referral/info",
382 |             headers={"Authorization": f"Bearer {owner_token}"})
383 |
384 |         owner_code = info_res.json()["code"]
385 |

```

```
386 |         # Register new user with ref code (gets trial)
387 |         unique_email = f"test_trial_{uuid.uuid4().hex[:8]}@test.com"
388 |         reg_res = requests.post(f"{BASE_URL}/api/auth/register", json={
389 |             "email": unique_email,
390 |             "password": "Test1234!",
391 |             "name": "Test Trial User",
392 |             "ref_code": owner_code
393 |         })
394 |         assert reg_res.status_code == 200
395 |         data = reg_res.json()
396 |         trial_token = data.get("access_token")
397 |
398 |         assert data["subscription_status"] == "trial", f"Expected trial, got {data['subscription_status']}"
399 |
400 |         # Verify trial user can access Pro features (e.g., signals)
401 |         signals_res = requests.get(
402 |             f"{BASE_URL}/api/signals",
403 |             headers={"Authorization": f"Bearer {trial_token}"})
404 |
405 |         assert signals_res.status_code == 200, f"Trial user should access Pro features, got {signals_res.status_code}"
406 |
407 |         signals_data = signals_res.json()
408 |         assert signals_data.get("is_pro") == True, f"Trial user should have is_pro=True, got {signals_data}"
409 |
410 |         print(f"PASSED: Trial user {unique_email} has Pro access (is_pro=True)")
411 |
412 |
413 | if __name__ == "__main__":
414 |     pytest.main([__file__, "-v", "--tb=short"])
415 |
```

FILE: [backend/tests/test_social_share_feature.py](#)

148 lines | 6584 characters

```
1 | """
2 | Test Social Share Feature - Backend API Tests
3 | Tests for referral endpoints that support the social share buttons feature
4 | """
5 | import pytest
6 | import requests
7 | import os
8 |
9 | BASE_URL = os.environ.get('REACT_APP_BACKEND_URL', '').rstrip('/')
10 |
11 | class TestReferralLeaderboardAPI:
12 |     """Tests for /api/referral/leaderboard endpoint (public, no auth)"""
13 |
14 |     def test_leaderboard_is_public(self):
15 |         """Leaderboard should be accessible without authentication"""
16 |         response = requests.get(f"{BASE_URL}/api/referral/leaderboard")
17 |         assert response.status_code == 200, f"Expected 200, got {response.status_code}"
18 |         print("PASS: Leaderboard endpoint is public (no auth required)")
19 |
20 |     def test_leaderboard_response_structure(self):
21 |         """Leaderboard should return correct structure"""
22 |         response = requests.get(f"{BASE_URL}/api/referral/leaderboard")
23 |         assert response.status_code == 200
24 |         data = response.json()
25 |
26 |         assert "leaderboard" in data, "Response should contain 'leaderboard' key"
27 |         assert "total_participants" in data, "Response should contain 'total_participants' key"
28 |         assert isinstance(data["leaderboard"], list), "Leaderboard should be a list"
29 |         assert isinstance(data["total_participants"], int), "total_participants should be an integer"
30 |         print(f"PASS: Leaderboard structure correct - {len(data['leaderboard'])} entries, {data['total_participants']} participant ...")
31 |
32 |     def test_leaderboard_entry_fields(self):
```

```

33 |         """Each leaderboard entry should have rank, name, referrals"""
34 |         response = requests.get(f"{BASE_URL}/api/referral/leaderboard")
35 |         assert response.status_code == 200
36 |         data = response.json()
37 |
38 |         if len(data["leaderboard"]) > 0:
39 |             entry = data["leaderboard"][0]
40 |             assert "rank" in entry, "Entry should have 'rank'"
41 |             assert "name" in entry, "Entry should have 'name'"
42 |             assert "referrals" in entry, "Entry should have 'referrals'"
43 |             print(f"PASS: Leaderboard entry has correct fields: {entry}")
44 |         else:
45 |             print("SKIP: No leaderboard entries to verify")
46 |
47 |
48 | class TestReferralInfoAPI:
49 |     """Tests for /api/referral/info endpoint (requires auth)"""
50 |
51 |     @pytest.fixture
52 |     def auth_token(self):
53 |         """Get authentication token for owner account"""
54 |         response = requests.post(f"{BASE_URL}/api/auth/login", json={
55 |             "email": "managingdirector@redslateholdings.com",
56 |             "password": "RedSlate2026!"
57 |         })
58 |         if response.status_code == 200:
59 |             return response.json().get("access_token")
60 |         pytest.skip("Authentication failed - skipping authenticated tests")
61 |
62 |     def test_referral_info_requires_auth(self):
63 |         """Referral info should require authentication"""
64 |         response = requests.get(f"{BASE_URL}/api/referral/info")
65 |         assert response.status_code == 401, f"Expected 401 without auth, got {response.status_code}"
66 |         print("PASS: Referral info requires authentication")
67 |
68 |     def test_referral_info_returns_code(self, auth_token):
69 |         """Referral info should return user's referral code"""
70 |         response = requests.get(
71 |             f"{BASE_URL}/api/referral/info",
72 |             headers={"Authorization": f"Bearer {auth_token}"})
73 |
74 |         assert response.status_code == 200, f"Expected 200, got {response.status_code}"
75 |         data = response.json()
76 |
77 |         assert "code" in data, "Response should contain 'code'"
78 |         assert isinstance(data["code"], str), "Code should be a string"
79 |         assert len(data["code"]) > 0, "Code should not be empty"
80 |         print(f"PASS: Referral info returns code: {data['code']}")
81 |
82 |     def test_referral_info_structure(self, auth_token):
83 |         """Referral info should return complete structure for social sharing"""
84 |         response = requests.get(
85 |             f"{BASE_URL}/api/referral/info",
86 |             headers={"Authorization": f"Bearer {auth_token}"})
87 |
88 |         assert response.status_code == 200
89 |         data = response.json()
90 |
91 |         # Required fields for social share feature
92 |         required_fields = ["code", "total_referrals", "completed_referrals", "rewards_earned", "rewards_remaining", "reward_cap", ...]
93 |         for field in required_fields:
94 |             assert field in data, f"Response should contain '{field}'"
95 |
96 |         print(f"PASS: Referral info has all required fields for social sharing")
97 |         print(f" - Code: {data['code']}")
98 |         print(f" - Total referrals: {data['total_referrals']}")
99 |         print(f" - Completed: {data['completed_referrals']}")

```

```
100 |
101 |
102 | class TestSocialShareURLConstruction:
103 |     """Tests to verify share URL construction logic (client-side, but we verify the data)"""
104 |
105 |     @pytest.fixture
106 |     def auth_token(self):
107 |         """Get authentication token"""
108 |         response = requests.post(f"{BASE_URL}/api/auth/login", json={
109 |             "email": "managingdirector@redslateholdings.com",
110 |             "password": "RedSlate2026!"
111 |         })
112 |         if response.status_code == 200:
113 |             return response.json().get("access_token")
114 |         pytest.skip("Authentication failed")
115 |
116 |     def test_referral_code_format(self, auth_token):
117 |         """Referral code should be alphanumeric and suitable for URL"""
118 |         response = requests.get(
119 |             f"{BASE_URL}/api/referral/info",
120 |             headers={"Authorization": f"Bearer {auth_token}"},
121 |         )
122 |         assert response.status_code == 200
123 |         code = response.json()["code"]
124 |
125 |         # Code should be alphanumeric (safe for URLs)
126 |         assert code.isalnum(), f"Code should be alphanumeric, got: {code}"
127 |         print(f"PASS: Referral code '{code}' is URL-safe (alphanumeric)")
128 |
129 |     def test_referral_link_can_be_constructed(self, auth_token):
130 |         """Verify referral link can be constructed from code"""
131 |         response = requests.get(
132 |             f"{BASE_URL}/api/referral/info",
133 |             headers={"Authorization": f"Bearer {auth_token}"},
134 |         )
135 |         assert response.status_code == 200
136 |         code = response.json()["code"]
137 |
138 |         # Construct referral link (as frontend does)
139 |         base_url = "https://risedual-trading.preview.emergentagent.com"
140 |         referral_link = f"{base_url}?ref={code}"
141 |
142 |         assert "?ref=" in referral_link, "Referral link should contain ?ref= parameter"
143 |         print(f"PASS: Referral link constructed: {referral_link}")
144 |
145 |
146 | if __name__ == "__main__":
147 |     pytest.main([__file__, "-v", "--tb=short"])
148 |
```

FILE: backend/tests/test_stripe_payment.py

264 lines | 10712 characters

```
1 | """
2 | Stripe Payment Integration Tests for RISEDUALAI
3 | Tests: checkout session creation, payment status polling, webhook endpoint, MongoDB persistence
4 | """
5 | import pytest
6 | import requests
7 | import os
8 | from datetime import datetime
9 |
10 | # Get BASE_URL from environment
11 | BASE_URL = os.environ.get('REACT_APP_BACKEND_URL', '').rstrip('/')
12 | if not BASE_URL:
13 |     raise ValueError("REACT_APP_BACKEND_URL environment variable not set")
```

```
14 |
15 | API_URL = f"{BASE_URL}/api"
16 |
17 |
18 | class TestStripePaymentIntegration:
19 |     """Stripe payment integration tests"""
20 |
21 |     # Store session_id for cross-test usage
22 |     created_session_id = None
23 |
24 |     def test_api_root_accessible(self):
25 |         """Test that API root is accessible"""
26 |         response = requests.get(f"{API_URL}/")
27 |         assert response.status_code == 200
28 |         data = response.json()
29 |         assert "message" in data
30 |         assert "RISEDUALAI" in data["message"]
31 |         print("? API root accessible")
32 |
33 |     def test_create_checkout_session_success(self):
34 |         """Test POST /api/subscription/create-checkout-session creates Stripe checkout session"""
35 |         payload = {
36 |             "origin_url": BASE_URL
37 |         }
38 |         response = requests.post(
39 |             f"{API_URL}/subscription/create-checkout-session",
40 |             json=payload,
41 |             headers={"Content-Type": "application/json"}
42 |         )
43 |
44 |         # Status code assertion
45 |         assert response.status_code == 200, f"Expected 200, got {response.status_code}: {response.text}"
46 |
47 |         # Data assertions
48 |         data = response.json()
49 |         assert "url" in data, "Response should contain 'url'"
50 |         assert "session_id" in data, "Response should contain 'session_id'"
51 |
52 |         # Verify URL is a valid Stripe checkout URL
53 |         assert data["url"].startswith("https://checkout.stripe.com/"), \
54 |             f"URL should be Stripe checkout URL, got: {data['url'][:50]}..."
55 |
56 |         # Verify session_id format (Stripe session IDs start with 'cs_')
57 |         assert data["session_id"].startswith("cs_"), \
58 |             f"Session ID should start with 'cs_', got: {data['session_id'][:20]}..."
59 |
60 |         # Store for subsequent tests
61 |         TestStripePaymentIntegration.created_session_id = data["session_id"]
62 |
63 |         print(f"? Checkout session created: {data['session_id'][:30]}...")
64 |         print(f"? Stripe URL: {data['url'][:60]}...")
65 |
66 |     def test_create_checkout_session_missing_origin_url(self):
67 |         """Test checkout session creation fails without origin_url"""
68 |         response = requests.post(
69 |             f"{API_URL}/subscription/create-checkout-session",
70 |             json={},
71 |             headers={"Content-Type": "application/json"}
72 |         )
73 |
74 |         # Should return 422 (validation error) for missing required field
75 |         assert response.status_code == 422, f"Expected 422 for missing origin_url, got {response.status_code}"
76 |         print("? Validation error returned for missing origin_url")
77 |
78 |     def test_get_payment_status_for_created_session(self):
79 |         """Test GET /api/subscription/status/{session_id} returns payment status"""
80 |         session_id = TestStripePaymentIntegration.created_session_id
```

```
81 |         if not session_id:
82 |             pytest.skip("No session_id from previous test")
83 |
84 |         response = requests.get(f"{API_URL}/subscription/status/{session_id}")
85 |
86 |         # Status code assertion
87 |         assert response.status_code == 200, f"Expected 200, got {response.status_code}: {response.text}"
88 |
89 |         # Data assertions
90 |         data = response.json()
91 |         assert "status" in data, "Response should contain 'status'"
92 |         assert "payment_status" in data, "Response should contain 'payment_status'"
93 |         assert "amount_total" in data, "Response should contain 'amount_total'"
94 |         assert "currency" in data, "Response should contain 'currency'"
95 |
96 |         # Verify expected values for unpaid session
97 |         # Status should be 'open' or 'expired' for unpaid sessions
98 |         assert data["status"] in ["open", "expired", "complete"], \
99 |             f"Unexpected status: {data['status']}"
100 |
101 |         # Payment status should be 'unpaid' for new sessions
102 |         assert data["payment_status"] in ["unpaid", "paid", "no_payment_required"], \
103 |             f"Unexpected payment_status: {data['payment_status']}"
104 |
105 |         # Amount should be $50 (5000 cents)
106 |         if data["amount_total"]:
107 |             assert data["amount_total"] == 5000, \
108 |                 f"Expected amount_total 5000 (cents), got {data['amount_total']}"
109 |
110 |         # Currency should be USD
111 |         if data["currency"]:
112 |             assert data["currency"].lower() == "usd", \
113 |                 f"Expected currency 'usd', got {data['currency']}"
114 |
115 |         print(f"? Payment status retrieved: status={data['status']}, payment_status={data['payment_status']}")
116 |
117 |     def test_get_payment_status_invalid_session(self):
118 |         """Test payment status for invalid session ID returns error"""
119 |         invalid_session_id = "cs_invalid_session_12345"
120 |
121 |         response = requests.get(f"{API_URL}/subscription/status/{invalid_session_id}")
122 |
123 |         # Should return 500 (Stripe API error) for invalid session
124 |         assert response.status_code == 500, \
125 |             f"Expected 500 for invalid session, got {response.status_code}"
126 |         print(f"? Error returned for invalid session ID")
127 |
128 |     def test_webhook_endpoint_exists(self):
129 |         """Test POST /api/webhook/stripe endpoint exists and responds"""
130 |         # Send empty body - should fail validation but endpoint should exist
131 |         response = requests.post(
132 |             f"{API_URL}/webhook/stripe",
133 |             data=b"",
134 |             headers={"Content-Type": "application/json"}
135 |         )
136 |
137 |         # Endpoint should exist (not 404)
138 |         assert response.status_code != 404, "Webhook endpoint should exist"
139 |
140 |         # Without valid signature, should return 400 or 500
141 |         assert response.status_code in [400, 500], \
142 |             f"Expected 400/500 for invalid webhook, got {response.status_code}"
143 |
144 |         print(f"? Webhook endpoint exists (returned {response.status_code} for invalid request)")
145 |
146 |     def test_webhook_endpoint_requires_signature(self):
147 |         """Test webhook endpoint requires Stripe-Signature header"""
```

```
148 |         # Send with fake body but no signature
149 |         fake_event = {
150 |             "type": "checkout.session.completed",
151 |             "data": {"object": {"id": "cs_test_123"}}
152 |         }
153 |
154 |         response = requests.post(
155 |             f"{API_URL}/webhook/stripe",
156 |             json=fake_event,
157 |             headers={"Content-Type": "application/json"}
158 |         )
159 |
160 |         # Should fail without valid signature
161 |         assert response.status_code in [400, 500], \
162 |             f"Expected 400/500 without signature, got {response.status_code}"
163 |
164 |         print("? Webhook correctly rejects requests without valid signature")
165 |
166 |
167 | class TestMongoDBPersistence:
168 |     """Test MongoDB persistence of payment transactions"""
169 |
170 |     def test_checkout_creates_db_entry(self):
171 |         """Test that creating checkout session creates entry in payment_transactions"""
172 |         # Create a new checkout session
173 |         payload = {"origin_url": BASE_URL}
174 |         response = requests.post(
175 |             f"{API_URL}/subscription/create-checkout-session",
176 |             json=payload,
177 |             headers={"Content-Type": "application/json"}
178 |         )
179 |
180 |         assert response.status_code == 200
181 |         data = response.json()
182 |         session_id = data["session_id"]
183 |
184 |         # Now check status - this should also verify DB entry exists
185 |         status_response = requests.get(f"{API_URL}/subscription/status/{session_id}")
186 |         assert status_response.status_code == 200
187 |
188 |         status_data = status_response.json()
189 |         # If we can get status, the session exists in Stripe
190 |         # The DB entry is created during checkout creation
191 |         assert "status" in status_data
192 |
193 |         print(f"? Checkout session {session_id[:20]}... created and status retrievable")
194 |
195 |
196 | class TestCheckoutSessionDetails:
197 |     """Test checkout session configuration details"""
198 |
199 |     def test_checkout_session_has_correct_amount(self):
200 |         """Test checkout session is configured for $50"""
201 |         payload = {"origin_url": BASE_URL}
202 |         response = requests.post(
203 |             f"{API_URL}/subscription/create-checkout-session",
204 |             json=payload,
205 |             headers={"Content-Type": "application/json"}
206 |         )
207 |
208 |         assert response.status_code == 200
209 |         data = response.json()
210 |         session_id = data["session_id"]
211 |
212 |         # Get status to verify amount
213 |         status_response = requests.get(f"{API_URL}/subscription/status/{session_id}")
214 |         assert status_response.status_code == 200
```

```
215 |
216 |     status_data = status_response.json()
217 |
218 |     # Amount should be 5000 cents ($50)
219 |     if status_data.get("amount_total"):
220 |         assert status_data["amount_total"] == 5000, \
221 |             f"Expected $50 (5000 cents), got {status_data['amount_total']} cents"
222 |         print(f"? Checkout amount is correct: ${status_data['amount_total']/100}")
223 |     else:
224 |         print("? amount_total not returned (may be null for unpaid sessions)")
225 |
226 |     def test_checkout_session_currency_is_usd(self):
227 |         """Test checkout session uses USD currency"""
228 |         payload = {"origin_url": BASE_URL}
229 |         response = requests.post(
230 |             f"{API_URL}/subscription/create-checkout-session",
231 |             json=payload,
232 |             headers={"Content-Type": "application/json"})
233 |
234 |
235 |         assert response.status_code == 200
236 |         data = response.json()
237 |         session_id = data["session_id"]
238 |
239 |         # Get status to verify currency
240 |         status_response = requests.get(f"{API_URL}/subscription/status/{session_id}")
241 |         assert status_response.status_code == 200
242 |
243 |         status_data = status_response.json()
244 |
245 |         if status_data.get("currency"):
246 |             assert status_data["currency"].lower() == "usd", \
247 |                 f"Expected USD, got {status_data['currency']}"
248 |             print(f"? Currency is correct: {status_data['currency'].upper()}")
249 |         else:
250 |             print("? currency not returned (may be null for unpaid sessions)")
251 |
252 |
253 | # Fixtures
254 | @pytest.fixture(scope="module")
255 | def api_client():
256 |     """Shared requests session"""
257 |     session = requests.Session()
258 |     session.headers.update({"Content-Type": "application/json"})
259 |     return session
260 |
261 |
262 | if __name__ == "__main__":
263 |     pytest.main([__file__, "-v", "--tb=short"])
264 |
```

FILE: backend/tests/test_subscription_plans.py

151 lines | 7658 characters

```
1 | """
2 | Test suite for RISEDUALAI Subscription Plans - Monthly and Annual pricing
3 | Tests the dual subscription pricing feature with Stripe checkout integration
4 | """
5 | import pytest
6 | import requests
7 | import os
8 |
9 | BASE_URL = os.environ.get('REACT_APP_BACKEND_URL', '').rstrip('/')
10 |
11 | class TestSubscriptionCheckout:
12 |     """Tests for subscription checkout endpoint with monthly/annual plans"""
```



```
13 |
14 |     def test_monthly_checkout_returns_200(self):
15 |         """POST /api/subscription/create-checkout-session with plan=monthly returns 200"""
16 |         response = requests.post(
17 |             f"{BASE_URL}/api/subscription/create-checkout-session",
18 |             json={"origin_url": "https://risedual.com", "plan": "monthly"},
19 |             headers={"Content-Type": "application/json"}
20 |         )
21 |         assert response.status_code == 200, f"Expected 200, got {response.status_code}"
22 |         print("Monthly checkout returns 200 - PASSED")
23 |
24 |     def test_monthly_checkout_returns_valid_url(self):
25 |         """POST /api/subscription/create-checkout-session with plan=monthly returns valid Stripe URL"""
26 |         response = requests.post(
27 |             f"{BASE_URL}/api/subscription/create-checkout-session",
28 |             json={"origin_url": "https://risedual.com", "plan": "monthly"},
29 |             headers={"Content-Type": "application/json"}
30 |         )
31 |         data = response.json()
32 |         assert "url" in data, "Response should contain 'url' field"
33 |         assert "checkout.stripe.com" in data["url"], "URL should be a Stripe checkout URL"
34 |         assert "session_id" in data, "Response should contain 'session_id' field"
35 |         print(f"Monthly checkout URL: {data['url'][:80]}...")
36 |         print("Monthly checkout returns valid Stripe URL - PASSED")
37 |
38 |     def test_annual_checkout_returns_200(self):
39 |         """POST /api/subscription/create-checkout-session with plan=annual returns 200"""
40 |         response = requests.post(
41 |             f"{BASE_URL}/api/subscription/create-checkout-session",
42 |             json={"origin_url": "https://risedual.com", "plan": "annual"},
43 |             headers={"Content-Type": "application/json"}
44 |         )
45 |         assert response.status_code == 200, f"Expected 200, got {response.status_code}"
46 |         print("Annual checkout returns 200 - PASSED")
47 |
48 |     def test_annual_checkout_returns_valid_url(self):
49 |         """POST /api/subscription/create-checkout-session with plan=annual returns valid Stripe URL"""
50 |         response = requests.post(
51 |             f"{BASE_URL}/api/subscription/create-checkout-session",
52 |             json={"origin_url": "https://risedual.com", "plan": "annual"},
53 |             headers={"Content-Type": "application/json"}
54 |         )
55 |         data = response.json()
56 |         assert "url" in data, "Response should contain 'url' field"
57 |         assert "checkout.stripe.com" in data["url"], "URL should be a Stripe checkout URL"
58 |         assert "session_id" in data, "Response should contain 'session_id' field"
59 |         print(f"Annual checkout URL: {data['url'][:80]}...")
60 |         print("Annual checkout returns valid Stripe URL - PASSED")
61 |
62 |     def test_default_plan_is_monthly(self):
63 |         """POST /api/subscription/create-checkout-session without plan defaults to monthly"""
64 |         response = requests.post(
65 |             f"{BASE_URL}/api/subscription/create-checkout-session",
66 |             json={"origin_url": "https://risedual.com"},
67 |             headers={"Content-Type": "application/json"}
68 |         )
69 |         assert response.status_code == 200, f"Expected 200, got {response.status_code}"
70 |         data = response.json()
71 |         assert "url" in data, "Response should contain 'url' field"
72 |         print("Default plan checkout works - PASSED")
73 |
74 |     def test_invalid_plan_defaults_to_monthly(self):
75 |         """POST /api/subscription/create-checkout-session with invalid plan defaults to monthly"""
76 |         response = requests.post(
77 |             f"{BASE_URL}/api/subscription/create-checkout-session",
78 |             json={"origin_url": "https://risedual.com", "plan": "invalid_plan"},
79 |             headers={"Content-Type": "application/json"}
```

```
80 |     )
81 |     assert response.status_code == 200, f"Expected 200, got {response.status_code}"
82 |     data = response.json()
83 |     assert "url" in data, "Response should contain 'url' field"
84 |     print("Invalid plan defaults to monthly - PASSED")
85 |
86 |
87 | class TestForeignMarketsEndpoint:
88 |     """Tests for foreign markets endpoint used by auto-refresh feature"""
89 |
90 |     def test_foreign_markets_returns_200(self):
91 |         """GET /api/foreign-markets returns 200"""
92 |         response = requests.get(f"{BASE_URL}/api/foreign-markets", timeout=120)
93 |         assert response.status_code == 200, f"Expected 200, got {response.status_code}"
94 |         print("Foreign markets returns 200 - PASSED")
95 |
96 |     def test_foreign_markets_has_asia(self):
97 |         """GET /api/foreign-markets returns Asia indices"""
98 |         response = requests.get(f"{BASE_URL}/api/foreign-markets", timeout=120)
99 |         data = response.json()
100 |         assert "asia" in data, "Response should contain 'asia' field"
101 |         assert len(data["asia"]) > 0, "Asia should have at least one index"
102 |         print(f"Asia indices: {len(data['asia'])} - PASSED")
103 |
104 |     def test_foreign_markets_has_europe(self):
105 |         """GET /api/foreign-markets returns Europe indices"""
106 |         response = requests.get(f"{BASE_URL}/api/foreign-markets", timeout=120)
107 |         data = response.json()
108 |         assert "europe" in data, "Response should contain 'europe' field"
109 |         assert len(data["europe"]) > 0, "Europe should have at least one index"
110 |         print(f"Europe indices: {len(data['europe'])} - PASSED")
111 |
112 |     def test_foreign_markets_has_americas(self):
113 |         """GET /api/foreign-markets returns Americas indices"""
114 |         response = requests.get(f"{BASE_URL}/api/foreign-markets", timeout=120)
115 |         data = response.json()
116 |         assert "americas" in data, "Response should contain 'americas' field"
117 |         assert len(data["americas"]) > 0, "Americas should have at least one index"
118 |         print(f"Americas indices: {len(data['americas'])} - PASSED")
119 |
120 |     def test_foreign_markets_has_commodities(self):
121 |         """GET /api/foreign-markets returns commodities"""
122 |         response = requests.get(f"{BASE_URL}/api/foreign-markets", timeout=120)
123 |         data = response.json()
124 |         assert "commodities" in data, "Response should contain 'commodities' field"
125 |         print(f"Commodities: {len(data.get('commodities', []))} - PASSED")
126 |
127 |     def test_foreign_markets_has_currencies(self):
128 |         """GET /api/foreign-markets returns currencies"""
129 |         response = requests.get(f"{BASE_URL}/api/foreign-markets", timeout=120)
130 |         data = response.json()
131 |         assert "currencies" in data, "Response should contain 'currencies' field"
132 |         print(f"Currencies: {len(data.get('currencies', []))} - PASSED")
133 |
134 |     def test_market_data_has_required_fields(self):
135 |         """Each market entry has required fields: symbol, name, price, change_percent"""
136 |         response = requests.get(f"{BASE_URL}/api/foreign-markets", timeout=120)
137 |         data = response.json()
138 |
139 |         # Check first Asia index
140 |         if data.get("asia"):
141 |             market = data["asia"][0]
142 |             assert "symbol" in market, "Market should have 'symbol'"
143 |             assert "name" in market, "Market should have 'name'"
144 |             assert "price" in market, "Market should have 'price'"
145 |             assert "change_percent" in market, "Market should have 'change_percent'"
146 |             print(f"Market data fields verified: {market['name']} - PASSED")
```

```
147 |  
148 |  
149 | if __name__ == "__main__":  
150 |     pytest.main([__file__, "-v", "--tb=short"])  
151 |
```

FILE: backend/tests/test_trading_journal.py*417 lines | 19990 characters*

```
1 | """Trading Journal Feature Tests - CRUD, Analytics, Hypothesis Attachment, Free Limit"""  
2 | import pytest  
3 | import requests  
4 | import os  
5 |  
6 | BASE_URL = os.environ.get('REACT_APP_BACKEND_URL', '').rstrip('/')  
7 |  
8 | # Test credentials  
9 | OWNER_EMAIL = "managingdirector@redslateholdings.com"  
10 | OWNER_PASSWORD = "RedSlate2026!"  
11 | FREE_USER_EMAIL = "freeuser_test@test.com"  
12 | FREE_USER_PASSWORD = "Test1234!"  
13 |  
14 |  
15 | @pytest.fixture(scope="module")  
16 | def owner_token():  
17 |     """Get owner (Pro) auth token"""  
18 |     response = requests.post(f"{BASE_URL}/api/auth/login", json={  
19 |         "email": OWNER_EMAIL,  
20 |         "password": OWNER_PASSWORD  
21 |     })  
22 |     if response.status_code == 200:  
23 |         return response.json().get("access_token")  
24 |     pytest.skip(f"Owner login failed: {response.status_code} - {response.text}")  
25 |  
26 |  
27 | @pytest.fixture(scope="module")  
28 | def free_user_token():  
29 |     """Get free user auth token"""  
30 |     response = requests.post(f"{BASE_URL}/api/auth/login", json={  
31 |         "email": FREE_USER_EMAIL,  
32 |         "password": FREE_USER_PASSWORD  
33 |     })  
34 |     if response.status_code == 200:  
35 |         return response.json().get("access_token")  
36 |     pytest.skip(f"Free user login failed: {response.status_code} - {response.text}")  
37 |  
38 |  
39 | @pytest.fixture  
40 | def owner_headers(owner_token):  
41 |     return {"Authorization": f"Bearer {owner_token}", "Content-Type": "application/json"}  
42 |  
43 |  
44 | @pytest.fixture  
45 | def free_headers(free_user_token):  
46 |     return {"Authorization": f"Bearer {free_user_token}", "Content-Type": "application/json"}  
47 |  
48 |  
49 | class TestJournalAuthentication:  
50 |     """Test that journal endpoints require authentication"""  
51 |  
52 |     def test_get_trades_requires_auth(self):  
53 |         """GET /api/journal/trades requires authentication"""  
54 |         response = requests.get(f"{BASE_URL}/api/journal/trades")  
55 |         assert response.status_code == 401, f"Expected 401, got {response.status_code}"  
56 |         print("PASSED: GET /api/journal/trades requires auth")  
57 |
```

```
58 |     def test_create_trade_requires_auth(self):
59 |         """POST /api/journal/trade requires authentication"""
60 |         response = requests.post(f"{BASE_URL}/api/journal/trade", json={
61 |             "ticker": "TEST", "side": "buy", "entry_price": 100, "quantity": 10, "entry_date": "2026-01-15"
62 |         })
63 |         assert response.status_code == 401, f"Expected 401, got {response.status_code}"
64 |         print("PASSED: POST /api/journal/trade requires auth")
65 |
66 |     def test_analytics_requires_auth(self):
67 |         """GET /api/journal/analytics requires authentication"""
68 |         response = requests.get(f"{BASE_URL}/api/journal/analytics")
69 |         assert response.status_code == 401, f"Expected 401, got {response.status_code}"
70 |         print("PASSED: GET /api/journal/analytics requires auth")
71 |
72 |
73 | class TestJournalGetTrades:
74 |     """Test GET /api/journal/trades endpoint"""
75 |
76 |     def test_owner_can_get_trades(self, owner_headers):
77 |         """Owner can retrieve their trades"""
78 |         response = requests.get(f"{BASE_URL}/api/journal/trades", headers=owner_headers)
79 |         assert response.status_code == 200, f"Expected 200, got {response.status_code}: {response.text}"
80 |         data = response.json()
81 |         assert "trades" in data, "Response should have 'trades' key"
82 |         assert "total" in data, "Response should have 'total' key"
83 |         assert "limit" in data, "Response should have 'limit' key"
84 |         assert isinstance(data["trades"], list), "trades should be a list"
85 |         # Owner is Pro, so limit should be -1 (unlimited)
86 |         assert data["limit"] == -1, f"Pro user should have limit=-1, got {data['limit']}"
87 |         print(f"PASSED: Owner has {data['total']} trades, limit={data['limit']}")
88 |
89 |     def test_free_user_can_get_trades(self, free_headers):
90 |         """Free user can retrieve their trades with limit info"""
91 |         response = requests.get(f"{BASE_URL}/api/journal/trades", headers=free_headers)
92 |         assert response.status_code == 200, f"Expected 200, got {response.status_code}: {response.text}"
93 |         data = response.json()
94 |         assert "trades" in data
95 |         assert "total" in data
96 |         assert "limit" in data
97 |         # Free user should have limit=5
98 |         assert data["limit"] == 5, f"Free user should have limit=5, got {data['limit']}"
99 |         print(f"PASSED: Free user has {data['total']} trades, limit={data['limit']}")
100 |
101 |
102 | class TestJournalCreateTrade:
103 |     """Test POST /api/journal/trade endpoint"""
104 |
105 |     def test_owner_can_create_trade(self, owner_headers):
106 |         """Owner (Pro) can create a new trade"""
107 |         trade_data = {
108 |             "ticker": "TEST_NVDA",
109 |             "side": "buy",
110 |             "entry_price": 850.50,
111 |             "quantity": 5,
112 |             "entry_date": "2026-01-15",
113 |             "notes": "Test trade for pytest"
114 |         }
115 |         response = requests.post(f"{BASE_URL}/api/journal/trade", headers=owner_headers, json=trade_data)
116 |         assert response.status_code == 200, f"Expected 200, got {response.status_code}: {response.text}"
117 |         data = response.json()
118 |         assert "id" in data, "Response should have 'id'"
119 |         assert data["ticker"] == "TEST_NVDA", f"Ticker should be TEST_NVDA, got {data['ticker']}"
120 |         assert data["side"] == "buy", f"Side should be buy, got {data['side']}"
121 |         assert data["entry_price"] == 850.50, f"Entry price should be 850.50, got {data['entry_price']}"
122 |         assert data["quantity"] == 5, f"Quantity should be 5, got {data['quantity']}"
123 |         assert data["status"] == "open", f"Status should be open, got {data['status']}"
124 |         assert data["pnl"] == 0, f"Open trade should have pnl=0, got {data['pnl']}"
```

```

125 |         print(f"PASSED: Created trade {data['id']} for TEST_NVDA")
126 |         return data["id"]
127 |
128 |     def test_create_trade_with_exit_price_calculates_pnl(self, owner_headers):
129 |         """Creating a closed trade (with exit price) calculates P&L correctly"""
130 |         trade_data = {
131 |             "ticker": "TEST_G00G",
132 |             "side": "buy",
133 |             "entry_price": 100.00,
134 |             "exit_price": 120.00,
135 |             "quantity": 10,
136 |             "entry_date": "2026-01-10",
137 |             "exit_date": "2026-01-15",
138 |             "notes": "Closed trade test"
139 |         }
140 |         response = requests.post(f"{BASE_URL}/api/journal/trade", headers=owner_headers, json=trade_data)
141 |         assert response.status_code == 200, f"Expected 200, got {response.status_code}: {response.text}"
142 |         data = response.json()
143 |         assert data["status"] == "closed", f"Trade with exit price should be closed, got {data['status']}"
144 |         # P&L for buy: (exit - entry) * qty = (120 - 100) * 10 = 200
145 |         assert data["pnl"] == 200.00, f"P&L should be 200.00, got {data['pnl']}"
146 |         # P&L percent: ((120 - 100) / 100) * 100 = 20%
147 |         assert data["pnl_percent"] == 20.0, f"P&L percent should be 20.0, got {data['pnl_percent']}"
148 |         print(f"PASSED: Closed trade P&L calculated correctly: ${data['pnl']} ({data['pnl_percent']}%)")
149 |         return data["id"]
150 |
151 |     def test_sell_side_pnl_calculation(self, owner_headers):
152 |         """Sell (short) trade P&L calculated correctly: (entry - exit) * qty"""
153 |         trade_data = {
154 |             "ticker": "TEST_SHORT",
155 |             "side": "sell",
156 |             "entry_price": 150.00,
157 |             "exit_price": 130.00,
158 |             "quantity": 10,
159 |             "entry_date": "2026-01-10",
160 |             "exit_date": "2026-01-15",
161 |             "notes": "Short trade test"
162 |         }
163 |         response = requests.post(f"{BASE_URL}/api/journal/trade", headers=owner_headers, json=trade_data)
164 |         assert response.status_code == 200, f"Expected 200, got {response.status_code}: {response.text}"
165 |         data = response.json()
166 |         # P&L for sell: (entry - exit) * qty = (150 - 130) * 10 = 200
167 |         assert data["pnl"] == 200.00, f"Short P&L should be 200.00, got {data['pnl']}"
168 |         print(f"PASSED: Short trade P&L calculated correctly: ${data['pnl']}")
169 |         return data["id"]
170 |
171 |     def test_free_user_at_limit_gets_403(self, free_headers):
172 |         """Free user at 5 trade limit gets 403 when trying to create more"""
173 |         # First check current count
174 |         response = requests.get(f"{BASE_URL}/api/journal/trades", headers=free_headers)
175 |         data = response.json()
176 |         current_count = data["total"]
177 |         print(f"Free user has {current_count} trades")
178 |
179 |         if current_count >= 5:
180 |             # Try to create a trade - should get 403
181 |             trade_data = {
182 |                 "ticker": "TEST_BLOCKED",
183 |                 "side": "buy",
184 |                 "entry_price": 100,
185 |                 "quantity": 1,
186 |                 "entry_date": "2026-01-15"
187 |             }
188 |             response = requests.post(f"{BASE_URL}/api/journal/trade", headers=free_headers, json=trade_data)
189 |             assert response.status_code == 403, f"Expected 403 at limit, got {response.status_code}: {response.text}"
190 |             data = response.json()
191 |             assert "Free accounts limited to 5 trades" in data.get("detail", ""), f"Expected limit message, got {data}"

```

```

192 |         print("PASSED: Free user at limit gets 403")
193 |     else:
194 |         pytest.skip(f"Free user only has {current_count} trades, not at limit")
195 |
196 |
197 | class TestJournalUpdateTrade:
198 |     """Test PUT /api/journal/trade/{id} endpoint"""
199 |
200 |     def test_close_open_trade(self, owner_headers):
201 |         """Can close an open trade by adding exit price"""
202 |         # First create an open trade
203 |         trade_data = {
204 |             "ticker": "TEST_CLOSE",
205 |             "side": "buy",
206 |             "entry_price": 200.00,
207 |             "quantity": 5,
208 |             "entry_date": "2026-01-10"
209 |         }
210 |         create_response = requests.post(f"{BASE_URL}/api/journal/trade", headers=owner_headers, json=trade_data)
211 |         assert create_response.status_code == 200
212 |         trade_id = create_response.json()["id"]
213 |
214 |         # Now close it
215 |         update_data = {
216 |             "exit_price": 220.00,
217 |             "exit_date": "2026-01-15"
218 |         }
219 |         response = requests.put(f"{BASE_URL}/api/journal/trade/{trade_id}", headers=owner_headers, json=update_data)
220 |         assert response.status_code == 200, f"Expected 200, got {response.status_code}: {response.text}"
221 |         data = response.json()
222 |         assert data["status"] == "closed", f"Trade should be closed, got {data['status']}"
223 |         assert data["exit_price"] == 220.00, f"Exit price should be 220.00, got {data['exit_price']}"
224 |         # P&L: (220 - 200) * 5 = 100
225 |         assert data["pnl"] == 100.00, f"P&L should be 100.00, got {data['pnl']}"
226 |         print(f"PASSED: Closed trade {trade_id}, P&L=${data['pnl']}")
227 |
228 |     def test_update_notes(self, owner_headers):
229 |         """Can update trade notes"""
230 |         # Create a trade
231 |         trade_data = {
232 |             "ticker": "TEST_NOTES",
233 |             "side": "buy",
234 |             "entry_price": 100.00,
235 |             "quantity": 1,
236 |             "entry_date": "2026-01-15"
237 |         }
238 |         create_response = requests.post(f"{BASE_URL}/api/journal/trade", headers=owner_headers, json=trade_data)
239 |         trade_id = create_response.json()["id"]
240 |
241 |         # Update notes
242 |         update_data = {"notes": "Updated notes for testing"}
243 |         response = requests.put(f"{BASE_URL}/api/journal/trade/{trade_id}", headers=owner_headers, json=update_data)
244 |         assert response.status_code == 200
245 |         data = response.json()
246 |         assert data["notes"] == "Updated notes for testing", f"Notes should be updated, got {data['notes']}"
247 |         print("PASSED: Updated trade notes")
248 |
249 |     def test_update_nonexistent_trade_returns_404(self, owner_headers):
250 |         """Updating non-existent trade returns 404"""
251 |         response = requests.put(f"{BASE_URL}/api/journal/trade/000000000000000000000000", headers=owner_headers, json={"notes": "t ...
252 |         assert response.status_code == 404, f"Expected 404, got {response.status_code}"
253 |         print("PASSED: Non-existent trade returns 404")
254 |
255 |
256 | class TestJournalDeleteTrade:
257 |     """Test DELETE /api/journal/trade/{id} endpoint"""
258 |

```

```

259 |     def test_delete_trade(self, owner_headers):
260 |         """Can delete a trade"""
261 |         # Create a trade to delete
262 |         trade_data = {
263 |             "ticker": "TEST_DELETE",
264 |             "side": "buy",
265 |             "entry_price": 100.00,
266 |             "quantity": 1,
267 |             "entry_date": "2026-01-15"
268 |         }
269 |         create_response = requests.post(f"{BASE_URL}/api/journal/trade", headers=owner_headers, json=trade_data)
270 |         trade_id = create_response.json()["id"]
271 |
272 |         # Delete it
273 |         response = requests.delete(f"{BASE_URL}/api/journal/trade/{trade_id}", headers=owner_headers)
274 |         assert response.status_code == 200, f"Expected 200, got {response.status_code}: {response.text}"
275 |         data = response.json()
276 |         assert data.get("message") == "Trade deleted", f"Expected delete message, got {data}"
277 |
278 |         # Verify it's gone
279 |         get_response = requests.get(f"{BASE_URL}/api/journal/trades", headers=owner_headers)
280 |         trades = get_response.json()["trades"]
281 |         trade_ids = [t["id"] for t in trades]
282 |         assert trade_id not in trade_ids, "Deleted trade should not appear in list"
283 |         print(f"PASSED: Deleted trade {trade_id}")
284 |
285 |     def test_delete_nonexistent_trade_returns_404(self, owner_headers):
286 |         """Deleting non-existent trade returns 404"""
287 |         response = requests.delete(f"{BASE_URL}/api/journal/trade/000000000000000000000000", headers=owner_headers)
288 |         assert response.status_code == 404, f"Expected 404, got {response.status_code}"
289 |         print("PASSED: Delete non-existent trade returns 404")
290 |
291 |
292 | class TestJournalAttachHypothesis:
293 |     """Test POST /api/journal/trade/{id}/attach-hypothesis endpoint"""
294 |
295 |     def test_attach_hypothesis_requires_auth(self):
296 |         """Attach hypothesis requires authentication"""
297 |         response = requests.post(f"{BASE_URL}/api/journal/trade/000000000000000000000000/attach-hypothesis")
298 |         assert response.status_code == 401, f"Expected 401, got {response.status_code}"
299 |         print("PASSED: Attach hypothesis requires auth")
300 |
301 |     def test_attach_hypothesis_to_nonexistent_trade(self, owner_headers):
302 |         """Attaching hypothesis to non-existent trade returns 404"""
303 |         response = requests.post(f"{BASE_URL}/api/journal/trade/000000000000000000000000/attach-hypothesis", headers=owner_headers ...)
304 |         assert response.status_code == 404, f"Expected 404, got {response.status_code}"
305 |         print("PASSED: Attach to non-existent trade returns 404")
306 |
307 |     def test_attach_hypothesis_no_hypothesis_found(self, owner_headers):
308 |         """Attaching hypothesis when no hypothesis exists for ticker returns 404"""
309 |         # Create a trade with a ticker that likely has no hypothesis
310 |         trade_data = {
311 |             "ticker": "XYZNONEXISTENT",
312 |             "side": "buy",
313 |             "entry_price": 100.00,
314 |             "quantity": 1,
315 |             "entry_date": "2026-01-15"
316 |         }
317 |         create_response = requests.post(f"{BASE_URL}/api/journal/trade", headers=owner_headers, json=trade_data)
318 |         trade_id = create_response.json()["id"]
319 |
320 |         # Try to attach hypothesis
321 |         response = requests.post(f"{BASE_URL}/api/journal/trade/{trade_id}/attach-hypothesis", headers=owner_headers)
322 |         # Should return 404 because no hypothesis exists for this ticker
323 |         assert response.status_code == 404, f"Expected 404 (no hypothesis), got {response.status_code}: {response.text}"
324 |         print("PASSED: No hypothesis found returns 404")
325 |

```

```

326 |
327 | class TestJournalAnalytics:
328 |     """Test GET /api/journal/analytics endpoint"""
329 |
330 |     def test_analytics_returns_correct_structure(self, owner_headers):
331 |         """Analytics endpoint returns correct data structure"""
332 |         response = requests.get(f"{BASE_URL}/api/journal/analytics", headers=owner_headers)
333 |         assert response.status_code == 200, f"Expected 200, got {response.status_code}: {response.text}"
334 |         data = response.json()
335 |
336 |         # Check required fields
337 |         required_fields = [
338 |             "total_trades", "open_trades", "closed_trades", "total_pnl",
339 |             "win_rate", "avg_gain", "avg_loss", "best_trade", "worst_trade",
340 |             "by_ticker", "pnl_timeline"
341 |         ]
342 |         for field in required_fields:
343 |             assert field in data, f"Analytics should have '{field}' field"
344 |
345 |         print(f"PASSED: Analytics structure correct")
346 |         print(f" - Total trades: {data['total_trades']}")
347 |         print(f" - Open: {data['open_trades']}, Closed: {data['closed_trades']}")
348 |         print(f" - Total P&L: ${data['total_pnl']}")
349 |         print(f" - Win rate: {data['win_rate']}%")
350 |
351 |     def test_analytics_pnl_timeline_structure(self, owner_headers):
352 |         """Analytics pnl_timeline has correct structure"""
353 |         response = requests.get(f"{BASE_URL}/api/journal/analytics", headers=owner_headers)
354 |         data = response.json()
355 |
356 |         if data["pnl_timeline"]:
357 |             timeline_entry = data["pnl_timeline"][0]
358 |             assert "date" in timeline_entry, "Timeline entry should have 'date'"
359 |             assert "pnl" in timeline_entry, "Timeline entry should have 'pnl'"
360 |             assert "cumulative" in timeline_entry, "Timeline entry should have 'cumulative'"
361 |             assert "ticker" in timeline_entry, "Timeline entry should have 'ticker'"
362 |             print(f"PASSED: P&L timeline structure correct ({len(data['pnl_timeline'])} entries)")
363 |         else:
364 |             print("PASSED: P&L timeline empty (no closed trades)")
365 |
366 |     def test_analytics_by_ticker_structure(self, owner_headers):
367 |         """Analytics by_ticker has correct structure"""
368 |         response = requests.get(f"{BASE_URL}/api/journal/analytics", headers=owner_headers)
369 |         data = response.json()
370 |
371 |         if data["by_ticker"]:
372 |             ticker, ticker_data = list(data["by_ticker"].items())[0]
373 |             assert "pnl" in ticker_data, "Ticker data should have 'pnl'"
374 |             assert "trades" in ticker_data, "Ticker data should have 'trades'"
375 |             assert "wins" in ticker_data, "Ticker data should have 'wins'"
376 |             print(f"PASSED: By-ticker structure correct ({len(data['by_ticker'])} tickers)")
377 |         else:
378 |             print("PASSED: By-ticker empty (no closed trades)")
379 |
380 |     def test_analytics_best_worst_trade(self, owner_headers):
381 |         """Analytics best/worst trade have correct structure"""
382 |         response = requests.get(f"{BASE_URL}/api/journal/analytics", headers=owner_headers)
383 |         data = response.json()
384 |
385 |         if data["best_trade"]:
386 |             assert "ticker" in data["best_trade"], "Best trade should have 'ticker'"
387 |             assert "pnl" in data["best_trade"], "Best trade should have 'pnl'"
388 |             assert "pnl_percent" in data["best_trade"], "Best trade should have 'pnl_percent'"
389 |             print(f"PASSED: Best trade: {data['best_trade']['ticker']} +${data['best_trade']['pnl']}")
390 |
391 |         if data["worst_trade"]:
392 |             assert "ticker" in data["worst_trade"], "Worst trade should have 'ticker'"

```



```
393 |         assert "pnl" in data["worst_trade"], "Worst trade should have 'pnl'"
394 |         print(f"PASSED: Worst trade: {data['worst_trade']['ticker']} ${data['worst_trade']['pnl']}")
395 |
396 |
397 | class TestJournalCleanup:
398 |     """Cleanup test trades created during testing"""
399 |
400 |     def test_cleanup_test_trades(self, owner_headers):
401 |         """Delete all TEST_ prefixed trades"""
402 |         response = requests.get(f"{BASE_URL}/api/journal/trades?limit=100", headers=owner_headers)
403 |         trades = response.json()["trades"]
404 |
405 |         deleted_count = 0
406 |         for trade in trades:
407 |             if trade["ticker"].startswith("TEST_"):
408 |                 del_response = requests.delete(f"{BASE_URL}/api/journal/trade/{trade['id']}", headers=owner_headers)
409 |                 if del_response.status_code == 200:
410 |                     deleted_count += 1
411 |
412 |         print(f"PASSED: Cleaned up {deleted_count} test trades")
413 |
414 |
415 | if __name__ == "__main__":
416 |     pytest.main([__file__, "-v", "--tb=short"])
417 |
```